參考用書：Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

演算法

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

一、分析：成長率、機率與攤銷成本

01 演算法在計算中的角色 3
02 起步 12
03 函數的成長 26
05 機率分析與隨機演算法 39
17 攤銷分析 59

二、工具箱：排序與資料結構

06 堆積排序 69
08 線性時間排序 81
10 基本資料結構 91
11 雜湊表 101
12 二元搜尋樹 113
13 紅黑樹 124
14 擴充資料結構 146
18 B 樹 156
19 費氏堆積 166
20 van Emde Boas 樹 176

三、遞迴與分治：從遞迴式到演算法

04 分治法 187
07 快速排序 207
09 中位數與次序統計量 215
30 多項式與快速傅立葉轉換 225

四、設計方法：動態規劃與貪心

15 動態規劃 236
16 貪婪演算法 253

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

演算法

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

五、圖演算法：走訪、連通、最短路徑與流

22 基礎圖演算法 265
21 不相交集合 280
23 最小生成樹 295
24 單源最短路徑 309
25 全點對最短路徑 324
26 最大流 337

六、計算幾何

33 計算幾何 361

七、延伸方法：平行、矩陣、最佳化與字串

27 多執行緒演算法 376
28 矩陣運算 400
29 線性規劃 413
31 數論演算法 432
32 字串匹配 455

八、計算理論：難解性與近似

34 NP 完備性 469
35 近似演算法 494

查用附錄：求和、集合、機率與矩陣

A 求和 512
B 集合、關係、函數、圖與樹 522
C 計數與機率 536
D 矩陣 553

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

01 演算法在計算中的角色

Part I Foundations

本章定位

先把合法輸入、可接受輸出與終止條件寫成契約，再依相同成本模型比較演算法、資料表示與硬體；本章用重複鍵排序、路徑/巡迴及交會點數例，區分一般正確性、隨機保證與有限測試。

前置知識

基本函數、集合、程式執行與離散輸入表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 把問題、輸入集合、輸出關係與演算法規格分開。
- 用正確性、終止性與資源成長描述演算法品質。
- 區分抽象效率與特定硬體上的常數成本。
- 1.1 Algorithms
- 1.2 Algorithms as a technology

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1 Algorithms

computational problem先定合法輸入集合D，以及每個x∈D可接受的輸出集合R(x)。例如同一組鍵可有多個合法排序結果，故規格通常是關係，不必是唯一公式。instance是其中一個具體x；algorithm是把輸入轉為輸出的明確可執行程序。

確定性演算法A的total correctness要求對**每個**x∈D均終止，且結果A(x)∈R(x)。partial correctness只要求「若終止，結果合法」；還須另證終止，才得到total correctness。合法輸入之外是拒絕、未定義或另一種結果，也必須先約定。

有限程式文字可以描述無限迴圈；「只有有限行偽碼」不證明每次執行都有限。測試若只核若干x，也不涵蓋規格中的全稱量詞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序同時保有序與每次出現

輸入A=[(3,a),(1,b),(3,c)]，每筆為(key,identity)；keys具total order且比較成本另有明定。合法輸出必同時滿足①keys非下降；②與輸入是同一批records的permutation，不遺失、不複製。[(1,b),(3,a),(3,c)]符合兩者；[(1,b),(3,a)]有序卻遺失c，不是排序答案。

若再要求stability，相等key在輸出中保留輸入先後，所以a須先於c。非穩定的[(1,b),(3,c),(3,a)]仍可符合前兩項，但不符穩定排序規格。空陣列的結果是空陣列，兩項條件都成立。

若key可能含沒有一般total-order語義的值，必須先定比較政策；不能默默把無法比較的值當成已滿足排序條件。record identity讓重複鍵的保存與先後可以直接檢查。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個可完整證明的演算法

規格：A長n≥0，找值v；輸出最小i使A[i]=v，若不存在回NIL。索引從1開始，邏輯and採由左到右short-circuit。

```
FIRST-MATCH(A, v)
    n = A.length
    i = 1
    while i <= n and A[i] != v
        i = i + 1
    if i <= n
        return i
    return NIL
```

while開始前，A[1..i−1]沒有v。初始空前綴成立；每步確認A[i]≠v才使i增加，因此維持。離開時若i≤n，該處等v且前面皆不等，正是最小索引；否則i=n+1，全陣列皆無v。variant n−i+1每次迭代減1、迴圈內非負，故至多n次迭代，證明終止。

成功、失敗與空輸入的完整 trace

A=[4,9,4]，v=4：i=1檢查1≤3、A[1]=4，while直接停止，回1。第二個4位於3，不影響「最小索引」。

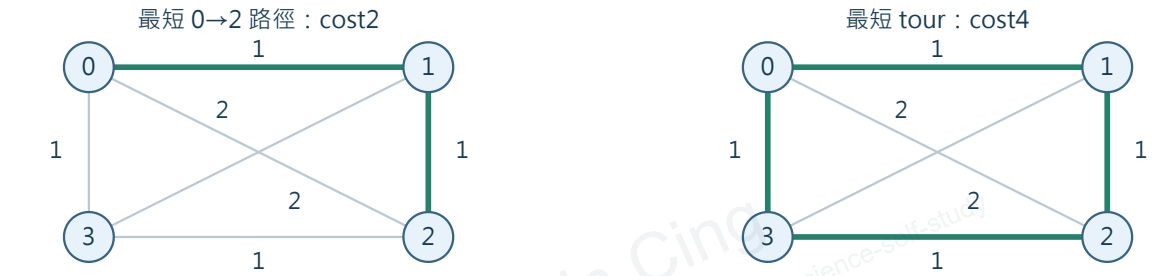
同一A找v=7：

檢查前i	i≤n	鍵比較	後續狀態
1	真	4≠7	i=2
2	真	9≠7	i=3
3	真	4≠7	i=4
4	假	不讀A[4]	回NIL

n=0時第一次i≤n已假，也不讀A[1]。有界索引與short-circuit條件是偽碼可實作的一部分，不能省略後再用測試碰運氣。

問題相似，合法答案卻不同

相似輸入，輸出契約不同



兩圖是同一個無向完整圖；對角線交點不是頂點。路徑不用訪全部點，tour須訪全點再返回。

無向完整圖的外圈四邊權1、兩條對角線權2。從0到2的最短路徑cost2，可走0→2或0→1→2，不要求拜訪3。TSP optimization要求訪問所有點恰一次再回起點，最小tour是0→1→2→3→0，cost4。

固定起點0、把反向視為同一tour，四點完整無向圖只有三種：01230成本4、01320成本6、02130成本6。這個有限枚舉證本**四點instance**的最優值；不是一般TSP的多項式算法。

表示法決定可行性與搜尋空間

LCS要求順序不變的subsequence，可刪位置而不必連續。X=ABA、Y=BAA的AA與BA都是長2共同subsequence；長3只能各取整串，兩串不同，所以最長長度2。共同substring則另要求位置連續，是另一問題。

X的8種位置選擇對應ε、A、B、A、AB、AA、BA、ABA；重複字元使不同位置子集得到相同字串，故2^n數的是位置選法，不保證2^n種不同字串。

依賴圖a→c、b→c、c→d要求先有a/b再做c、最後d；a,b,c,d與b,a,c,d合法，a,c,b,d不合法。若再加d→b，就要求b<c<d<b而矛盾，沒有topological order。選擇graph而非單一list表示依賴，才能清楚檢查這種無解情況。

困難性與近似的量詞

decision problem只問yes/no；optimization problem找最佳值/解。TSP decision問「是否存在tour成本≤B」，optimization找最小成本tour。NP-complete通常用在decision版，optimization版談NP-hard，不能省略問題版本。

NP-complete的歸約性質表示，若其中一個decision problem有多項式算法，所有NP problems都有。單憑NP-complete標籤不構成「已證不存在多項式算法」；還要有相應複雜度分離前提。

approximation要有明確可行解與目標界，如minimization中cost(output)≤ρ・OPT；不能由「答案看起來好」推出此保證。heuristic、exact exponential algorithm、特殊輸入類別與approximation解不同需求，其時間/正確性前提應分開。

隨機答案要說明對誰取機率

固定合法輸入x，隨機bits為r。Monte Carlo保證可以是Pr_r[A(x;r)正確]≥2/3且時間有界；機率取自內部亂數，不是「隨機挑一些輸入測試有2/3通過」。每個x的保證不能由未定義的平均輸入成功率代替。

對相同binary decision獨立執行3次、多數表決，錯誤機率至多$C(3,2)(1/3)^2(2/3)+(1/3)^3=7/27$，正確至少20/27。獨立性不可省；若三次共用同一隨機選擇，可能一起錯而毫無改善。一般search輸出若有多個不同正確解，也不能直接做字串多數表決。

零錯誤隨機算法與終止/期望時間是另一組規格。有限期望時間可推出幾乎必然終止，但不表示每條無限亂數序列都在統一有限步內結束；這與確定性total correctness的量詞不同。

資料結構與平行性各改變哪個成本

找任意元素的array可能要掃n項；已排序array可用binary search，但插入可能搬Θ(n)項；hash table的期望時間依hash/負載模型而定。沒有一種資料結構同時無條件把所有操作變成常數。

平行工作例：4個彼此獨立unit tasks全部完成後，再接4個unit tasks的串行chain。work W=8、span S=5。1、2、4個processors的完成時間分別8、6、5；每個P下T_P≥max(W/P,S)，speedup W/T_P≤W/S=1.6。

W/S是加速比上界，不是「processor超過1.6就再無收益」。本例P=2已超過1.6，增加到4仍由6降5。排程可達的時間還取決於DAG與整數processor數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

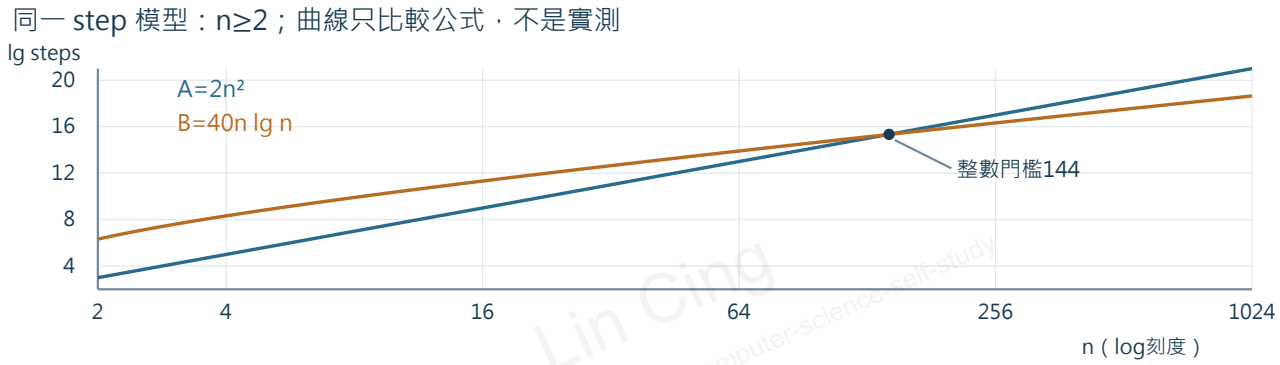
1.2 Algorithms as a technology

算法與硬體共同決定時間。若算法所需操作數為I(n)、機器對這些操作的有效處理率為R，簡化模型T(n)=I(n)/R。比較時要讓n、操作定義、R與單位一致；memory、I/O與同步可使單一R模型失效。

令I_A(n)=2n²、I_B(n)=40n lg n，先限定n≥2、同一種抽象step與相同R。A常數較小但二次成長，B是n log n。這兩個是明確模型，不是聲稱任何insertion/merge sort實作的固定精確指令數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

交會點必須核對兩側



B<A等價n−20lg n>0。g(n)=n−20lg n，其導數1−20/(n ln2)，n>20/ln2後嚴格遞增。g(143)<0、g(144)>0，因此n≥2的整數範圍內，B從n=144起持續勝出；小區間也由先降後升的形狀排除漏解。

交會點必須核對兩側 （續）

n	A steps	B steps	較少step
16	512	2560	A
128	32768	35840	A
256	131072	81920	B
1024	2097152	409600	B

圖使用log座標呈現同一模型。曲線相交附近的連續n不等於整數題目的門檻，strict inequality也須核對是否恰相等。

更快硬體仍要除回同一時間單位

改令A機器R_A=10^9steps/s，B機器R_B=10^7steps/s，A原始吞吐快100倍；算法仍用上一頁兩式。n=10^6時A需2×10^{12}steps，時間2000s；B需約7.9726274×10^8steps，時間79.7263s。B雖處理率較慢，總時間仍約快25.086倍。

n=1000時A為.002s，B約.0398631s，反而A較快。此模型的時間交會改為$n-2000\lg n>0$，不能繼續套相同R時的144門檻。

大O只給上界集合，不能直接比較兩個帶未知常數的式子哪個實作較快。固定常數下$n/\log n$最終趨無窮，才支持「足夠大時較低成長階可抵銷常數劣勢」的趨勢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸入大小與可用記憶體也是技術條件

整數N以binary表示的長度$L=\lfloor\lg N\rfloor+1$。迴圈執行N次對數值是$\Theta(N)$，對輸入bits卻可達$\Theta(2^L)$，不能稱其為一般bit-size的線性時間。array長n若每key是w-bit word，輸入占$\Theta(nw)$ bits，單次比較是否$O(1)$依word/cost模型。

若固定記憶體只容納M筆，n>M時原本的in-memory模型必改，外部I/O與buffer策略可能主導。平行/快取/網路不是可任意忽略的小常數，分析應先選問題規模與成本邊界。

算法會出現在application、compiler、runtime、OS、network與hardware設計各層。所需的是能把新問題定成契約、選合適表示法並證明正確/成本，而不是只記一個已知程式碼名稱。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 01。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

02 起步

Part I Foundations

本章定位

以穩定插入與明確邊界合併建立迴圈／歸納證明，逐步核對洞、record保存、inversions與RAM成本，再對任意長度推導merge sort的時間及峰值空間。

前置知識

陣列、迴圈不變量、基本遞迴與對數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用迴圈不變量證明插入排序正確。
- 從 RAM 模型的逐行成本建立最壞情況界限。
- 用分治遞迴描述合併排序並求解 T(n)=2T(n/2)+Theta(n)。
- 2.1 Insertion sort
- 2.2 Analyzing algorithms
- 2.3 Designing algorithms

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.1 Insertion sort

本章排序對象是records，每筆具有可比較的key與不變identity；keys形成total order，單次比較、搬移一筆record均為常數成本。輸出須keys非下降，且每筆原record恰出現一次；stable另要求等key保留原先後。索引採1起算，空區間不讀取元素。

incremental方法維護已排序prefix，再將下一筆插入。in-place表示除輸入外只需常數個word；不表示陣列在每個中途時刻都已排序，也不表示暫時沒有複製值。

以下and由左向右short-circuit。key是暫存record的值；若record是物件參照，只搬移參照、不修改物件內容。n≤1直接返回，避免把空陣列套進「第一筆已排序」的初始化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

穩定的原地插入

```
INSERTION-SORT(A)
    n = A.length
    if n <= 1: return
    j = 2
    while j <= n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
        j = j + 1
```

嚴格大於才右移：等key不跨過新record。i=0時不求值A[i]，所以不越界。內層每次i減1，至多j−1次；外層j每次加1，至多n−1次，兩層均終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

外層不變量：有序與保存要一起證

令S是呼叫開始時的輸入副本。每次外層判斷前：A[1..j−1]是S[1..j−1]的**穩定排序**，而A[j..n]=S[j..n]。此處identity區分重複key，避免只證「值看起來相同」。

初始化j=2，單筆prefix本就有序且穩定，suffix未變。維護：內層只搬移prefix中嚴格大於key的尾段，再把key放在其左邊；舊prefix內相對先後未變，等key全留在新record前。故prefix擴成S[1..j]的穩定排序，suffix仍未變。

終止時j=n+1，不變量給A[1..n]是全部輸入的穩定排序。n≤1由提前返回獨立成立。不變量證明結果，前頁的遞減／遞增界證明一定到達結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

內層真正保存的是「陣列扣掉洞，再加key」

固定一次j，令B為該次開始的A[1..j]，B[1..j−1]已排序。把h=i+1視為邏輯洞，洞內記憶體的舊值不計入資料表示。

每次內層判斷前：A[1..i]=B[1..i]；A[i+2..j]=B[i+1..j−1]，這段每個key都嚴格大於暫存key=B[j]。非洞位置連同暫存key，恰保存B的全部records。初始化i=j−1，右段為空。

若A[i]>key，將A[i]複製到洞，i減1，新洞左移一格；上述兩段等式仍成立。若i=0或A[i]≤key，左段均≤key、右段均>key，填洞便有序且保存全部records。直接聲稱「每次shift後整段A仍是permutation」是錯的：複製造成的重複須由洞的表示消除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組records的完整外層結果

輸入[(7,a),(2,b),(5,c),(2,d),(1,e)]，以7a等簡記；identity不是排序次鍵。

j / 暫存key	由右至左的key比較	右移次數h_j	填洞後A
2 / 2b	7>2真；到邊界	1	2b,7a,5c,2d,1e
3 / 5c	7>5真；2>5假	1	2b,5c,7a,2d,1e
4 / 2d	7>2真；5>2真；2>2假	2	2b,2d,5c,7a,1e
5 / 1e	7>1、5>1、2d>1、2b>1皆真；到邊界	4	1e,2b,2d,5c,7a

2b始終先於2d。共8次右移、10次key比較；while整個guard共12次求值，因i=0的兩次不執行key比較。這三種計數不可混用。

一次插入內的每個狀態

固定j=4，B=[2b,5c,7a,2d]，key=2d；□只表示邏輯洞，並非額外寫入陣列。

guard前i	非洞表示	動作
3	[2b,5c,7a,□] + key2d	7>2，A[4]←7a
2	[2b,5c,□,7a] + key2d	5>2，A[3]←5c
1	[2b,□,5c,7a] + key2d	2>2為假，停止右移
1	[2b,2d,5c,7a]	A[2]←key，填洞

一次插入內的每個狀態 （續）

同一次j=4：先保存key，再右移、填洞

i=3：7a → 右側洞	2b	5c	7a	□	key=2d（暫存）
i=2：5c → 右側洞	2b	5c	□	7a	key=2d（暫存）
i=1：2b≤2d，停止	2b	□	5c	7a	key=2d（暫存）
填入2d；2b仍在前	2b	2d	5c	7a	穩定且保存全體

□是被排除的邏輯位置；實體記憶體可仍含舊值，不能把重複值當成兩筆資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

偽碼的參數、別名與控制語義

本章函式傳參是值傳遞；陣列值是儲存體的reference，故A[i]賦值修改呼叫者看得到的陣列。B=A複製reference，兩者alias同一儲存體；B=copy(A)才有獨立陣列。將局部A重新綁到新陣列，不會把呼叫端的變數也改綁。

例如A=[3,1]，B=A後B[1]=8，A也成[8,1]；若改成B=copy(A)，B[1]=8不影響A。record identity與記憶體位址不是同一個概念；排序可重排references而不重建物件。

縮排界定block；return結束本次呼叫。NIL表示無物件，不是合法陣列位置。本章while的最後一次失敗guard也計成本；自然語言「排序某集合」若隱含非定長子程序，不能算一個常數步驟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.2 Analyzing algorithms

單處理器word-RAM依序執行load/store、比較、分支與有限word算術。陣列位置可在常數時間存取；word至少能表達索引，典型$w=\Theta(\lg(n+1))$ bits，不能把任意長整數塞進一個word再宣稱任意運算$O(1)$。

這個模型沒有直接計cache miss、page fault、I/O及平行工作；實測時間還取決於實作與硬體。若key是長字串，比較可能要逐字讀取；本章常數比較前提就須另改。若中間算術溢出word，必須指定多word表示及其成本。

輸入大小依問題選：排序n筆records；L-bit整數以L衡量；圖通常需(V,E)兩個參數。數值N與編碼長度$\lg N$不可互換。以下成本是primitive operations，不是秒數。

把每行執行次數寫出來

對$n \geq 2$，令h_j為第j次插入的右移數，$t_j=h_j+1$為內層guard次數，$I=\Sigma h_j$。本章while版本的外層$j=2$另計一次，外層guard共n次。

操作	次數
n設定、提前返回判斷、j初始化	各1
外層guard	n
key暫存、i初始化、最後填洞、j加1	各$n-1$
內層guard	$I+n-1$
右移$A[i+1] \leftarrow A[i]$、i減1	各I

每操作成本$c_s>0$、與n無關時，$T(A)=c_0+c_1 n+c_2(n-1)+c_3(I+n-1)+c_4 I$，這裡$c_2$合併四項、$c_4$合併兩項。故$T(A)=\Theta(n+I)$；包含$n=0/1$可寫$\Theta(1+n+I)$。

為什麼右移數恰等於inversions

inversion是原輸入的索引對(u,v)，u<v且key(S[u])>key(S[v])，總數I(S)。相等key不形成inversion。

第j次插入前，prefix保存前j−1筆。它們中key大於S[j]者恰是排序prefix的一個尾段；演算法逐筆右移此尾段，各一次，其他record不移。因此h_j等於右端點v=j的inversions數。每個inversion有唯一右端點，Σh_j=I(S)，不是只對某個數例成立。

前例右端點j=2,3,4,5分別有1,1,2,4對，I=8。已排序輸入I=0，時間Θ(n)；嚴格反序且keys相異時I=n(n−1)/2，時間Θ(n^2)。只有少量inversions的輸入可接近線性，這是adaptive性質。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較次數、寫入次數與最佳／最差

對每j，key比較數為h_j+1（若h_j<j−1，最後比較為假），或h_j（若一路到i=0，由short-circuit結束）。設b為插入到最左端的次數，則C_key=I+n−1−b；整體guard=I+n−1；array writes=I+n−1，暫存key的寫入不算array write。

已排序時h_j=0、b=0，C_key=n−1；相異反序時h_j=j−1、b=n−1，C_key=n(n−1)/2。輸入長度相同也可有不同成本。

令$T_{\text{best}}(n)=\min_{|x|=n}T(x)$，$T_{\text{worst}}(n)=\max_{|x|=n}T(x)$。最佳／最差指定的是哪個函數；O/Ω/Θ指定對函數的漸近界。不能把O當「最差」或Ω當「最佳」的同義詞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平均成本必須先給分布

假設n個**相異**key的n!個排列等機率。每對原位置u<v，令X_uv=1當S[u]>S[v]，否則0；交換這兩位置是一一對應，所以Pr(X_uv=1)=1/2。

$I = \sum_{u<v} X_{uv}$，期望線性性給E[I]=ΣE[X_uv]=n(n−1)/4。這裡不需要各X獨立，事實上共用位置的指標通常相依。由T(A)=Θ(1+n+I)，且界中常數對每個輸入相同，E[T]=Θ(n^2)（n→∞）。

這不是「任何真實資料平均都平方」：若分布集中於已排序輸入，期望是Θ(n)。隨機輸入的期望與固定輸入、演算法自己抽亂數的期望，分母不同，必須分開陳述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從確切公式到成長階

對固定RAM costs，最差時間可整理成an^2+bn+c且a>0。取足夠大n，使$|b|n+|c| \le (a/2)n^2$，則$(a/2)n^2 \le T_{worst}(n) \le (3a/2)n^2$；這才支持Θ($n^2$)。

低階項和常數會影響有限n的快慢，Θ只說最終同階。$T_1=1000n$與$T_2=n^2$在1<n<1000時T_2較小，在n>1000時T_1較小；不等式前須使用同一成本單位。

空輸入與單筆輸入仍要正確處理；漸近忽略有限多n，不代表程式可以忽略它們。最佳常數時間的提前返回檢查，也不能自動把其他所有輸入的成本改善。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

2.3 Designing algorithms

divide-and-conquer把問題分為嚴格更小的同類問題，遞迴求解，再合併答案。只說「分兩半」不夠：須定邊界、base case、合併前提及結果。merge sort取A[p..r]，長m=r−p+1；p>r表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

合法非空呼叫要求1≤p≤r≤n；整陣列呼叫(1,n)，n=0時(1,0)直接返回。左右長分別ceil(m/2)、floor(m/2)，m≥2時都小於m。q寫成差值式避免p+r溢出的常見實作風險，但仍須確保合法索引範圍可表達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不依賴額外key值的MERGE

前提1≤p≤q<r≤n，兩半已穩定排序。L、R是**獨立副本**，不能是會被覆寫A影響的alias。len(L)=q−p+1，len(R)=r−q。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key
            A[k] = L[i]; i = i+1
        else
            A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

相等先取L，因原輸入左半records全先於右半，穩定性得以延續。只檢查長度不需key域之外的∞。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MERGE的保存、順序與終止證明

主迴圈前，k−p=(i−1)+(j−1)；A[p..k−1]恰是已消耗的L[1..i−1]、R[1..j−1]之穩定合併，已排序且其中每個key≤所有未消耗key。其餘未消耗records仍在獨立L/R中。

初始化空輸出成立。若兩邊非空，各自首筆就是各邊最小未消耗值；選較小者（等值選左）恰為全體下一筆。消耗對應一筆、k加1，保持長度等式、保存、順序與穩定性。未消耗總數每次減1，故終止。

退出主迴圈時至少一邊耗盡；另一邊已排序且不小於輸出尾，所以逐筆接上保留同一不變量。最後i=len(L)+1、j=len(R)+1、k=r+1，全部m筆恰輸出一次。若不複製L/R，A[k]可能覆寫尚未讀的輸入；證明所需的剩餘資料就消失。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併

L=[1a,4b,4c]、R=[2d,4e,6f]，原輸入左段先於右段。表列主迴圈每次比較；i,j為比較前的1-based位置。

i,j	比較	取出後的輸出prefix
1,1	1a≤2d	1a
2,1	4b>2d	1a,2d
2,2	4b≤4e	1a,2d,4b
3,2	4c≤4e	1a,2d,4b,4c
4,2	L耗盡，不再比較key	接上4e,6f

共4次key比較、6次輸出寫入，另有6筆copy進L/R。若最後兩筆交錯耗盡，最多m−1=5次key比較；「MERGE是Θ(m)」不表示每次恰做m次key比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有重複key的完整合併 （續）

消耗一筆，輸出一筆；兩邊副本不會被A回寫破壞

	L			R			
開始	1a	4b	4c	2d	4e	6f	輸出：空
取1a、2d後	—	4b	4c	—	4e	6f	輸出：1a,2d
等值先取左	—	—	—	—	4e	6f	輸出：1a,2d,4b,4c

L耗盡 → 依序接4e、6f。相同key的4b、4c、4e保持原本從左至右的先後。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sentinel是有前提的另一種表示

若使用sentinel，可在L/R尾各加一筆key=∞，固定m次從較小首筆取出，不用每次檢查耗盡。但∞必須嚴格大於每個合法key，且不是合法record。將最大合法整數當∞，遇到同值輸入就可能錯取sentinel，甚至下一步越界。

例如合法key允許+∞，L=[+∞]、R=[+∞,+∞]，加上相同∞哨兵後「等值選左」可能連取左真record、左sentinel，第三次便讀超過L。這不是正確的域外標記。

本章長度檢查版對任意total-order key域都可用，代價只是明確處理耗盡。若改為tagged sentinel，使它的排序順位嚴格高於所有real record，也須保證sentinel絕不輸出成資料。

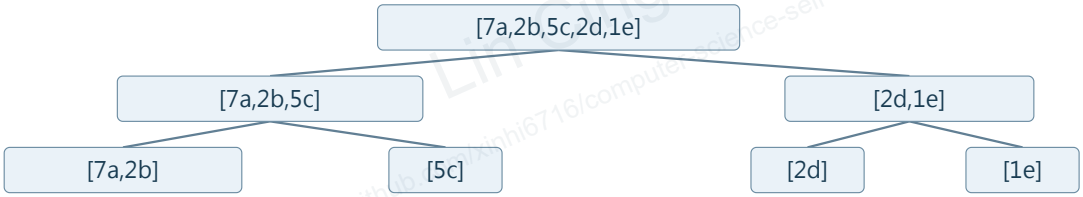
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序

對區間長m作strong induction。m≤1已排序且不變。m≥2時兩個子問題都嚴格較小，歸納假設使兩半各自穩定排序且保持各半全部records；MERGE前提成立。前頁證明使整段穩定排序，區間外未改，歸納完成。

終止由子問題長度遞減至0/1保證；正確性不依賴n是2的冪。輸入[7a,2b,5c,2d,1e]分為長3、2，再分2/1與1/1；各節點合併結果如下圖。左側2b與右側2d相等，根合併仍先2b。

五筆record：向下分割，向上回傳穩定排序結果



自底向上： [2b,7a] → [2b,5c,7a]；右半 → [1e,2d]

根合併： [1e,2b,2d,5c,7a]；相等key2：左2b先於右2d

左長3、右長2；單筆直接返回。圖省略[7a,2b]再分為兩個單筆的最後一層。

遞迴式的成本不能漏掉合併

對m≥2，分割計q為Θ(1)，兩個遞迴呼叫成本相加，MERGE複製m筆並輸出m筆，為Θ(m)，故

T(m)=T(ceil(m/2))+T(floor(m/2))+Θ(m)，T(0),T(1)=Θ(1)。

先對m=2^h採精確代理式U(1)=d、U(m)=2U(m/2)+cm（c,d>0）。深度ℓ=0..h−1有2^ℓ個internal nodes，每個成本cm/2^ℓ，該層總cm；h層合cmh。最底m個葉各d，故U(m)=cm lg m+dm。

真成本每internal node夾於c_low・size與c_high・size、葉成本也有固定上下界，將兩者各自代入便得到Θ(m lg m)，而非假設同一c恰代表所有指令和base case。

任意長度的界與一個六筆成本表

平衡拆分到深度ℓ後，各未終止區間長度至多ceil(m/2^ℓ)，所以height≤ceil(lg m)。同一深度的區間互不重疊，總長至多m，給internal成本O(m lg m)。

在深度ℓ<floor(lg m)，每段至少floor(m/2^ℓ)≥2，尚無葉；各段總長恰m，給floor(lg m)層各Ω(m)。m≥2時floor(lg m)≥(lg m)/2，故下界Ω(m lg m)。這是任意m的雙邊證明。

m=6的深度	問題長度	internal長度總和
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1（另兩葉已在上層）	0

本例copy進buffers與回寫A各16筆，不是把每一層都誤算成6筆；base case共6個。

峰值空間不是所有配置次數的總和

本版MERGE在兩次遞迴返回後才建立L/R，完成合併即可釋放；單次長m的buffers合m筆。子呼叫的buffers不與父MERGE buffers同時存活，因此峰值輔助空間Θ(n)，另有Θ(lg n)呼叫stack，合仍Θ(n)（n≥2）。

整趟累計buffer配置量可以是Θ(n lg n)，但它不是同時需要的記憶體。若實作保留每個已完成呼叫的buffer不釋放，就換了生命期模型；不能套用上述峰值結論。也可預先配置n筆scratch反覆重用，並保留同樣的非alias前提。

insertion sort只需key、i、j等常數額外word，較省空間；merge sort在本實作保證Θ(n lg n)時間但非in-place。兩者都穩定，前提分別是strict >右移與等值先取左。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 02。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

03 函數的成長

Part I Foundations

本章定位

以完整量詞、常數與反例建立五種漸近關係，區分輸入成本與函數界，再推導floor/mod、固定底數成長比較、factorial、log-star與Fibonacci的可用條件。

前置知識

實數、對數、多項式與基本不等式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用正式量詞定義 O、Omega 與 Theta。
- 證明多項式最高次項的漸近界限。
- 辨識對數底、指數與階乘分析中的常見等式。
- 3.1 Asymptotic notation
- 3.2 Standard notations and common functions

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.1 Asymptotic notation

先選定函數及極限方向。本節n取自然數並趨向∞；f最終非負、比較基準g最終嚴格正。若g可為0，須回到不等式定義，不能直接除以g。有限多個小n的異常可由門檻n_0排除，程式的合法小輸入仍須另保正確。

O(g)、Ω(g)、Θ(g)是**函數集合**；寫f=O(g)慣例表示f∈O(g)，不是數值相等。f可以是時間、空間、比較次數或純數學函數。定理只看函數，不自行指定它是最佳、最差還是平均成本。

例如insertion的最差函數Θ(n^2)，最佳函數Θ(n)。對所有輸入的共同上界O(n^2)不給所有輸入共同下界Ω(n^2)。函數與輸入量詞須同時交代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種界的量詞與常數

符號	精確定義：下式對所有整數n≥n_0成立
f∈O(g)	存在c>0,n_0，使0≤f(n)≤c g(n)
f∈Ω(g)	存在c>0,n_0，使0≤c g(n)≤f(n)
f∈Θ(g)	存在c_1,c_2>0,n_0，使0≤c_1g(n)≤f(n)≤c_2g(n)

常數可依f/g及固定演算法而定，不能依n變。例如f=n^2、g=n，令c=n不能證O(n)。門檻不必最小；任何更大n_0都仍可用。緊界意味上下兩側都成立，不表示f/g必有極限，也不表示兩演算法同秒數。

Θ(g)⊆O(g)且Θ(g)⊆Ω(g)。n∈O(n^2)是正確但寬鬆的界；把O當成Θ會丟失這個區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個有具體常數的雙界證明

取$f(n)=3n^2-8n+5$。$n\geq 4$時，$8/n\leq 2$、$5/n^2\geq 0$，所以$f/n^2=3-8/n+5/n^2\geq 1$；另一方面$n\geq 1$時$-8n+5\leq 0$，$f\leq 3n^2$。可選$c_1=1, c_2=3, n_0=4$，證$f\in\Theta(n^2)$。

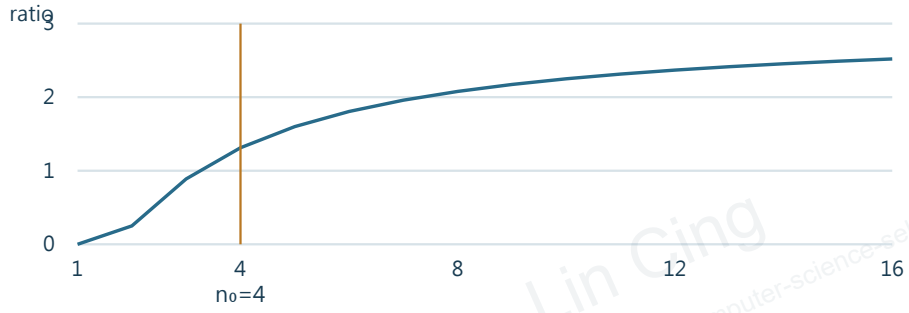
逐值檢查：$n=4$時$f=21$，$n^2=16$，$3n^2=48$；$n=8$時$f=133$，界$64\leq 133\leq 192$。有限表不是全稱證明；上段不等式才涵蓋所有$n\geq 4$。

反證$f\notin O(n)$：若$f\leq cn$且$n\geq 4$有$f\geq n^2$，則$n\leq c$。取整數$n>\max(n_0,4,c)$矛盾。反例不是只找一個超過某個猜測c的點，而是對**任意候選常數**都能找到更大的n。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個有具體常數的雙界證明 (續)

雙界要控制整個尾段，而非只看幾個點



比值 $f(n)/n^2$

$f=3n^2-8n+5$

$n\geq 4$：$1\leq$比值≤ 3

$c_1=1$；$c_2=3$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Θ等於O與Ω的交集

若f∈Θ(g)，直接取定義中的c_2證O，取c_1證Ω。反向若$f \le c_U g$對$n \ge N_U$、$f \ge c_L g$對$n \ge N_L$，取$n_0 = max(N_U, N_L)$，同時得到$c_L g \le f \le c_U g$，故f∈Θ(g)。

對最終非負f,g，max(f,g)≤f+g≤2max(f,g)，故f+g=Θ(max(f,g))。這個結論依非負性；若允許相消，f=n、g=−n會使和為0。

若$p(n) = a_d n^d + \sum_{i<d} a_i n^i$、$a_d > 0$且d≥1，令$M = \sum_{i<d} |a_i|$。$n \ge max(1, 2M/a_d)$時，低階絕對和≤$Mn^{d-1} \le a_d n^d/2$，得$(a_d/2)n^d \le p \le (3a_d/2)n^d$。d=0時為正定常數Θ(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o與little-ω：把「存在c」改成「任意c」

f∈o(g)：對**每個**c>0，存在$n_0(c)$，使0≤f(n)<c g(n)對所有n≥n_0成立。f∈ω(g)：對每個c>0，存在$n_0(c)$，使0≤c g(n)<f(n)。門檻可以依c，但不能在固定c後再依n。

例如7n∈o(n^2)：給c>0，取整數n_0>7/c即可。$7n^2 \notin o(n^2)$：取c=7便要求$7n^2 < 7n^2$。$n^2 \in \omega(7n)$：給c>0，取n>7c。對正g，o等價f/g→0；對最終正f,g，ω等價f/g→∞。

o(g)⊆O(g)，但不是所有「非緊O界」都是o。f=n在偶數、$f=n^2$在奇數時，f∈O(n^2)卻不在Θ(n^2)，也不在o(n^2)；其比值反覆回到1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極限法何時能下結論

對最終正g，若L=lim f/g存在：L=0給o(g)；0<L<∞給Θ(g)；L=∞且f最終正給ω(g)。有限正L的證明：取ε=L/2，充分大時L/2<f/g<3L/2，即雙界。

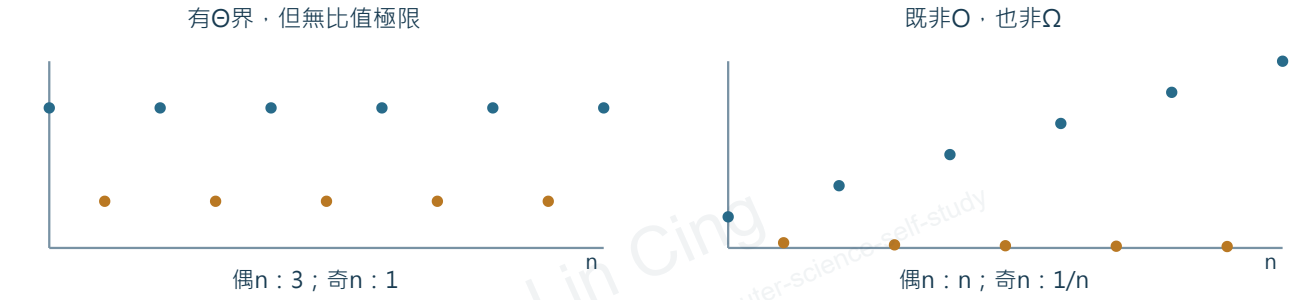
極限不存在不能直接判「不可比較」。$f = n(2 + (-1)^n)$，g=n，比值為1或3，沒有極限卻有n≤f≤3n，仍為Θ(n)。

真正不可比較例：$f=n^2$（偶數n）、f=1（奇數n），g=n。偶數比值n無上界，故非O(g)；奇數比值1/n趨0，故非Ω(g)。取對應奇偶子序列便可逐一否定任意常數界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

極限法何時能下結論 （續）

畫的是f/g：左圖夾在1與3，右圖兩個子序列分離



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

關係的代數性質與證明

對最終正函數，O、Ω、Θ具反身與傳遞性，Θ具對稱性；f∈O(g)當且僅當g∈Ω(f)，f∈o(g)當且僅當g∈ω(f)。例如f≤ag、g≤bh，取共同門檻便有f≤ab h，證O傳遞；Ω同理反向，Θ兩側各做一次。

o傳遞也須對任意常數：給c>0，取充分大n使f<g（使用常數1），且g<c h，則f<c h。ω傳遞以倒置的o關係得到。o與ω不反身，且o(g)∩ω(g)=∅：同取c=1的兩個共同門檻，會要求f<g且g<f。

這些關係不是實數全序：前頁的奇偶例不符合O、Ω任一方向，因此不能假定任意兩函數一定有小於、同階、大於三者之一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

式子中的匿名函數與不可逆等號

寫$4n^2+6n+2=4n^2+\Theta(n)$，表示選了某個h∈Θ(n)，這裡h=6n+2。寫$4n^2+\Theta(n)=\Theta(n^2)$，表示**每個**左側h都可在右側找到相符的函數；不是左右集合完全相等，不能倒過來用。

兩個O項各可選不同匿名函數；O(n)−O(n)無法保證Θ(n)，例如n−n=0。若要討論有號誤差，應明寫|e(n)|≤cn，因本章非負版本O集合不自動包含負函數。

$\sum_{i=1}^{n} h(i)$，若h(i)=O(i)，隱含同一c及門檻對所有充分大i有效；有限prefix和為常數，故和$O(n^2)$。若讓每項各自依n選不同常數，像$h_n(i) = n \cdot i$，就不符合這個單一h假設，總和可達$\Theta(n^3)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對演算法輸入的界必須一致

令T(x)≥0，$T_{\max}(n) = \max_{|x|=n} T(x)$，$T_{\min}(n) = min_{|x|=n} T(x)$，假設各集合非空且極值存在。所有輸入共同Θ(g)的精確意思是存在同一c_L, c_U, n_0，使n≥n₀的**每個**x都滿足$c_L g(n) \le T(x) \le c_U g(n)$。

它等價$T_{\max} \in O(g)$且$T_{\min} \in \Omega(g)$：共同界取max/min得到一方向；反向以$T_{\min} \le T(x) \le T_{\max}$及共同門檻得到。只有$T_{\max} \in \Theta(g)$不夠，因它的下界只是某些最慢輸入。

若同時有n,m兩參數，須另約定量詞範圍。本章採n≥n₀且m≥m₀同時成立的矩形尾域；若要求「n或m足夠大」的更強尾域，需明寫or，不能無聲交換兩種定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 Standard notations and common functions

monotonically increasing（單調不減）：m≤n⇒f(m)≤f(n)；strictly increasing則m<n⇒f(m)<f(n)。decreasing對應反向不等式，不要把nondecreasing誤當必嚴格上升。

若f,g單調不減，f+g亦然；若g的值落在f的定義域，f∘g亦然。若再皆非負，f(m)g(m)≤f(n)g(m)≤f(n)g(n)，積也單調不減；沒有非負條件時不一定。

floor(x)是≤x的最大整數，ceiling(x)是≥x的最小整數，所以x−1<floor x≤x≤ceil x<x+1。例x=−2.3：floor=−3、ceil=−2，不能把「截掉小數」當一般floor。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數分割、巢狀floor／ceiling與mod

整數n有ceil(n/2)+floor(n/2)=n；偶數2k給k+k，奇數2k+1給(k+1)+k。正整數a,b與x≥0有floor(floor(x/a)/b)=floor(x/(ab))；因對整數k，kb≤floor(x/a)等價kab≤x，兩式可接受的最大k相同。ceiling對最小k及反向界同理。

正整數a,b有ceil(a/b)≤(a+b−1)/b、floor(a/b)≥(a−b+1)/b，可由a=qb+r、0≤r<b分情況r=0／r>0證得。

對整數u與正模數m，u mod m=u−m floor(u/m)∈{0,⋯,m−1}。例−17 mod5=−17−5(−4)=3，17 mod5=2。程式語言若remainder符號跟dividend，不可直接當此數學mod。u≡v(mod m)當且僅當m整除u−v。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

冪次、指數與常數的範圍

對a>0及實數u,v，$a^{u+v}=a^u a^v$，$(a^u)^v=a^{uv}$，a⁰=1、$a^{-u}=1/a^u$。本章a為**固定常數**時，a>1使a^n增加，0<a<1使其減少。n^b對n>0，b>0增加、b<0減少。

polynomially bounded是存在固定k使$f=O(n^k)$，不只指f本身是多項式。$n^{\lg n}$不符合：對任意固定k，$f/n^k=n^{\lg n-k}\to\infty$。$2^{\lg n}=n$是多項式，但$2^n$是指數；看到符號2不代表函數一定指數成長。

0⁰若出現組合公式須另外定義，本章在空乘積慣例下採1，不主張它是每個極限的共同值。實數底數、指數的恆等式也不能無條件搬到負底數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼固定底數指數超過任何多項式

設a=1+δ>1、b為固定實數。若b≤0，$n^b/a^n \le 1/a^n \to 0$。若b>0，選整數k>b。二項式展開給$a^n \ge binom(n,k)\delta^k$。

n≥2k時，$n(n-1)\dots(n-k+1) \ge (n/2)^k$，所以$a^n \ge \delta^k n^k/(2^k k!)$。因此$0 \le n^b/a^n \le (2^k k!/\delta^k)n^{b-k} \to 0$，得$n^b = o(a^n)$。k依b但不依n。

對固定ε>0及任意固定b，令t=lg n，則$(\lg n)^b/n^\varepsilon = t^b/(2^\varepsilon)^t \to 0$。因此任何固定polylog都小於每個正次冪。底數若隨n接近1，就不符合固定a>1前提；例如$(1+1/n)^n$趨e。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對數的底數、次方與複合不可混

lg n=log₂n，$\ln n = log_e n$；$\lg^k n = (\lg n)^k$，lg lg n=lg(lg n)，$\lg^{(k)} n$表示反覆取lg共k次。log只作用於緊鄰項，lg n+q不是lg(n+q)。所有實數log的引數須>0，底數>0且≠1。

由$b^{log_b x} = x$，取log_c得$log_b x = log_c x/log_c b$。固定底b,c>1只差正定常數，所以$log_b n = \Theta(log_c n)$。若b=n，$log_n n = 1$，與lg n不同階。

$log_b(xy) = log_b x + log_b y$、$log_b(x^r) = r log_b x$；$a^{log_b c} = c^{log_b a}$，因兩邊取ln都為(ln a ln c)/(ln b)。例$3^{lg 8} = 27 = 8^{lg 3}$，不是$3^{(\lg)^8}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

e與ln的可用界及誤差

$e^x = \sum_{k\ge 0} x^k/k!$；由凸性（導數$e^x$遞增，x=0切線斜率1）得$e^x \ge 1+x$，全實數成立。Taylor積分／Lagrange餘項給$e^x = 1 + x + (e^\xi/2)x^2$，ξ介於0與x，所以|x|≤1時$e^{-1}x^2/2 \le e^x - 1 - x \le ex^2/2$，證x→0時餘項Θ(x²)。

對x>−1，x/(1+x)≤ln(1+x)≤x。上界可用h=x−ln(1+x)，h(0)=0且h'=x/(1+x)，在0兩側都使h≥0；下界用q=ln(1+x)−x/(1+x)，q'=x/(1+x)²同理。若|x|<1，$\ln(1+x) = \sum_{k\ge 1}(-1)^{k+1}x^k/k$。

固定x，令u=x/n，當n大到1+u>0，兩個ln界夾出n ln(1+x/n)→x，再取exp，得$(1+x/n)^n \to e^x$。這些是有定義域的界，不是把所有近似式當成恆等式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Factorial：先用乘積得到一般界

整數n≥0，0!=1，n!=n(n−1)!。每個乘數≤n，故$n! \le n^n$。更強地，前floor(n/2)個乘數≤n/2，剩餘≤n，所以$n!/n^n \le 2^{-\operatorname{floor}(n/2)} \to 0$，證$n! = o(n^n)$。

至少floor(n/2)個尾端乘數≥n/2，故$n! \ge (n/2)^{\operatorname{floor}(n/2)}$。取lg後，lg(n!)介於floor(n/2)lg(n/2)與n lg n，得Θ(n lg n)。同一下界扣去n lg a，對任意固定a>0皆趨∞，所以$n!/a^n \to \infty$；尤其$n! = \omega(2^n)$。

這些界不需先假設Stirling。數例6!=720、2⁶=64、6⁶=46656；一個有限數例不能代替上述極限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling的精度與指數中的常數

Stirling界（此處陳述標準近似定理）：n≥1時

$n! = \mathrm{sqrt}(2\pi n)(n/e)^n e^{\alpha_n}$，$1/(12n+1) < \alpha_n < 1/(12n)$。

因此相對誤差因子$e^{\alpha_n} = 1 + \Theta(1/n)$，且lg(n!)=n lg n−(lg e)n+(1/2)lg(2πn)+Θ(1/n)。n=6的主項$\sqrt{12\pi}(6/e)^6$約710.078，乘上介於$e^{1/73}$與$e^{1/72}$的因子夾住720。

「log底數只差常數」不能推成指數底數也可忽略。$2^{n+3} = 8 \cdot 2^n$，故$\Theta(2^n)$；$2^{3n}/2^n = 4^n \to \infty$，不在$O(2^n)$。若f=Θ(n)，只能說$2^f = 2^{\Theta(n)}$，不能說必為$\Theta(2^n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

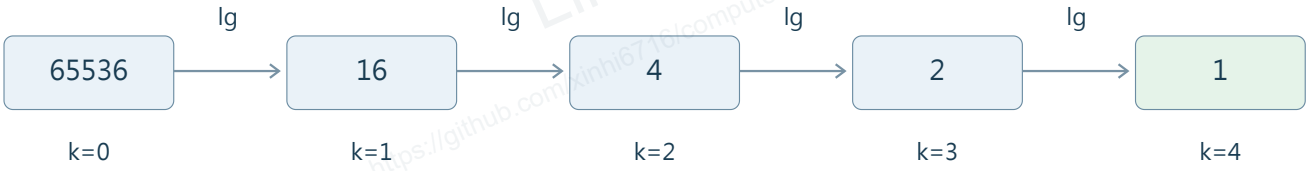
Functional iteration與log-star的停止點

$f^{(0)}(n) = n$，$f^{(k+1)}(n) = f(f^{(k)}(n))$。若f(n)=3n，則$f^{(k)}(n) = 3^k n$，不是$(3n)^k$。

對n>0，$\lg^* n = \min\{k \ge 0 : \lg^{(k)} n \le 1\}$；只在前一值>1時繼續取lg，因此不會讀log非正數。n≤1時lg*=0。例如65536→16→4→2→1，故lg*65536=4；$\lg^{(2)} 65536 = 4$，而lg²65536=256。

令tower門檻a_0=1、$a_{k+1} = 2^{a_k}$，則$n \in (a_{k-1}, a_k]$時lg*n=k（k≥1）。1,2,4,16,65536依序是k=0..4的上界。log-star雖慢仍無界：每個有限k都有更大n超過a_k。

反覆取lg：每個箭頭都算一次，不再對1取lg



只在目前值>1時繼續；第4次得到1，立即停止，所以lg*65536=4。

連取兩次lg得4；(lg65536)²=256，iteration與power是兩種不同運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci閉式不是猜到就算證完

F_0=0、F_1=1、$F_n = F_{n-1} + F_{n-2}$。設φ=(1+sqrt5)/2、ψ=(1−sqrt5)/2，兩者都滿足z^2=z+1。令$G_n = (\varphi^n - \psi^n)/\sqrt{5}$；$G_0$=0、$G_1$=1，且對n≥2，$\varphi^n = \varphi^{n-1} + \varphi^{n-2}$、ψ也同理，故$G_n = G_{n-1} + G_{n-2}$。由歸納$F_n = G_n$。

|ψ|<1給$|F_n - \varphi^n/\sqrt{5}| = |\psi|^n/\sqrt{5} < 1/2$（含n=0），故$F_n = \mathrm{floor}(\varphi^n/\sqrt{5} + 1/2)$。n=6時$\varphi^6$/sqrt5≈8.024922，$\psi^6$/sqrt5≈0.024922，相減=8。

由誤差<1/2且φ^n無界，充分大n有$\varphi^n/(2\sqrt{5}) \le F_n \le 2\varphi^n/\sqrt{5}$，故$F_n = \Theta(\varphi^n)$。閉式實數運算不代表能用有限精度浮點O(1)精確輸出任意大F_n；其輸出已有Θ(n) bits。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 03。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

05 機率分析與隨機演算法

Part I Foundations

本章定位

機率分析先固定樣本空間與依賴條件；本章從雇用指示與均勻洗牌的一般證明，推到生日碰撞、桶等待、重疊streak與線上遴選，再以隨機遞迴及Quicksort區分期望、事件機率、最壞與尾界。

前置知識

離散機率、期望值、隨機排列與基本界限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 區分輸入分布假設與演算法內部隨機性。
- 用指示隨機變數與期望線性性避免錯誤的獨立性假設。
- 分析隨機排列與隨機化演算法的期望成本。
- 從隨機分割與比較指示變數完整推導期望值及資料結構成本。
- 5.1 The hiring problem
- 5.2 Indicator random variables
- 5.3 Randomized algorithms
- 5.4 Probabilistic analysis and further uses of indicator random variables

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.1 The hiring problem

n位候選人的能力有相異且可比較的排名，較大較好。每次面試後若比目前雇員更好，立即替換；先用比任何人都差的虛擬候選人初始化。這個問題計的是替換成本，不是只有比較次數。

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]; hire best; hires += 1
    return (best, hires)
```

第i輪後best是已面試i人的最大者，歸納立即成立。每人面試費ci、每次雇用費ch時，總費用為 n・ci+X・ch，其中X是雇用次數。n=0回dummy、X=0；n≥1時1≤X≤n。

最壞輸入、輸入分布與內部亂數

排名[1,2,⋯,n]每次刷新紀錄，X=n；[n,n−1,⋯,1]只有第一次，X=1。確定性HIRE在固定輸入上的成本也是固定的。

平均情形分析必須明列輸入分布。本章先假設相異排名的n!種順序等機率；這是比「每個位置有相同邊際分布」更強的聯合假設。若先由演算法自己均勻洗牌，則任意固定排名輸入都被轉成此分布，期望改為對內部亂數取，不依賴未知的輸入分布。

這不表示每次都快或從此沒有壞執行：洗牌仍可能產生遞增順序。隨機化保證的是每個固定輸入的期望費用，不是所有亂數結果的最壞費用。

5.2 Indicator random variables

事件E的指示變數 I_E 在事件成立時為1，否則0。因此 $E[I_E] = 1P(E) + 0P(E^c) = P(E)$。若 $X = \sum_{i=1}^{m} X_i$，有限樣本空間中交換兩個有限和得

$$E[X] = \sum_{\omega} P(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} P(\omega) X_i(\omega) = \sum_{i} E[X_i].$$

這是期望線性性，不需Xi獨立。獨立性可能在「算出每個事件機率」或把交集機率相乘時需要。若X1=X2是同一次公平硬幣的正面指示，E[X1+X2]=1仍成立，但P(X1=X2=1)=1/2不是1/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雇用次數的調和和證明

Xi表示第i位是否被雇用。固定前i位的候選人集合後，其i!種相對順序等機率，最大者落在第i位的機率1/i；這正是Xi=1的條件。因此

$$E[X] = \sum_{i=1}^{n} P(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

不需先求X的整個分布，也不需假設各次刷新事件互相獨立。由遞減函數1/x的積分比較，$\ln(n+1) \le H_n \le 1 + \ln n$，所以E[X]=Θ(ln n)，期望總費用nci+chHn；面試費沒有變成對數。

排名[2,5,1,4,6,3]依序刷新於位置1、2、5，X=3；同一n=6的均勻平均為H6=49/20=2.45，單一trace不必恰等於平均。只有一次雇用若且唯若最佳者第一個，機率1/n；每人都雇用只有唯一遞增排列，機率1/n!。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

指示變數也能數成對事件

均勻排列n個相異鍵，每對位置i<j的逆序指示Iij有機率1/2，因交換這兩個位置給大於、小於的雙射。逆序數V=Σi<j Iij，故 $E[V] = \binom{n}{2}/2 = n(n-1)/4$。各對共享位置造成相依，並不妨礙期望線性性。

均勻把n頂帽子一對一還給n人，每人拿回自己的機率1/n，所以固定點數的期望為1（n≥1）。這個期望不是「一定有一人拿對」；n=2時可能0或2，各半。n=0則固定點數0，不能再用1/n。

這兩例的共同方法是先證明每個物件的指示和恰為目標計數，再計單項機率；不要把不相交事件的機率加法，和可相依變數的期望加法混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.3 Randomized algorithms

假設RANDOM(a,b)回傳包含兩端的整數，每個機率1/(b−a+1)，不同呼叫相互獨立。實作的有限狀態偽亂數產生器只是此機率模型的近似來源；分析中仍須明列均勻及獨立假設。

若只有公平獨立bits，可用拒絕抽樣產生m=b−a+1種等機率結果。m=1直接回a；否則取k=⌈lg m⌉個bits形成z∈[0,2^k−1]，z<m便回a+z，否則重抽整組。每次接受率m/2^k>1/2，條件於接受後各合法z等機率；重試輪數期望<2，期望bit成本O(lg m)。

不能直接用z mod m消除超出範圍：當m不整除2^k，各餘數的前像數不同。例如3個bits mod3的0、1各有3個前像，2只有2個，不均勻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

原地均勻洗牌：完整程序與聯合不變量

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap A[i],A[j]
```

把元素視為相異身分，即使其鍵值相同也分別標記。第i輪前，任何指定的i−1個不同元素序列，成為A[1..i−1]的機率是(n−i+1)!/n!。初始空序列唯一，機率1；固定此前綴後，每個剩餘身分恰在suffix一個位置，本輪以1/(n−i+1)機率選到指定第i個元素。

故指定長i前綴的機率為(n−i)!/n!，不變量保持。終止時每個完整排列機率0!/n!=1/n!，是完整聯合均勻性證明。n=0迴圈不執行、唯一空排列機率1。單位成本亂數模型下Θ(n)時間、O(1)額外空間；用bits實作亂數時另計其成本。

同一個洗牌逐輪狀態

A=[A,B,C,D]，依序抽j=3、4、3、4。

i	可抽索引	本輪j	swap後全陣列	已固定前綴
1	1..4	3	[C,B,A,D]	C
2	2..4	4	[C,D,A,B]	C,D
3	3..4	3	[C,D,A,B]	C,D,A
4	4	4	[C,D,A,B]	C,D,A,B

這條亂數路徑的機率為1/4・1/3・1/2・1=1/24。任意目標排列也對應唯一一組合法j序列：依次找到目標第i個身分在剩餘suffix的位置，交換並繼續，所以共有4!=24條等機率路徑。

同一個洗牌逐輪狀態 （續）

從 [A,B,C,D] 開始：每輪只從尚未固定的 suffix 選一個

	位置1	位置2	位置3	位置4	本輪機率
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

綠格已固定、不再改動；這條路徑機率 1/4·1/3·1/2·1=1/24。

均勻邊際不足：兩種錯誤洗牌

只隨機選一個循環平移，確實讓每個元素以1/n機率到每個位置；但n≥3時只有n種輸出，不是n!種。聯合排列不均勻。

另一種程序讓i=1..n、每次j均勻抽整個1..n再swap。n=3時有3^3=27條等機率路徑；若六種排列均勻，每種必對應27/6條路徑，不是整數，故不可能。錯在後面步驟會再次移動先前位置，原地洗牌的不變量已不成立。

若只洗前m輪再取前m個，得到每個長m有序樣本機率$(n-m)!/n!$；忽略順序，每個m子集有m!種排序，機率$1/\binom{n}{m}$。這需要suffix仍可隨機索引；若原始n元素必先建成陣列，建表成本不能漏算。

隨機優先值排序：條件與碰撞

給每個元素獨立均勻優先值Pi∈{1,⋯,M}，再依Pi排序。條件於所有Pi不同，由標籤交換對稱性，n!種相對順序等機率；排序可用Θ(n lg n)比較時間與O(n)儲存優先值。

每對碰撞機率1/M，union bound給 $P(\exists i<j: P_i=P_j)\le \binom{n}{2}/M$。取M=$n^3$、n≥2時≤(n−1)/(2$n^2$)<1/(2n)。這使碰撞很少，沒有使它不可能。

若平手保留原順序，無條件分布仍有偏差。n=2時，第一元素先出的機率P(P1≤P2)=(M+1)/(2M)>1/2。可遇任何碰撞就整組重抽，接受事件的標籤對稱性保證均勻；此M下期望重抽輪數有常數界。也可對每個平手群再作均勻洗牌。原地SHUFFLE則一開始就避免這個條件缺口。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機化把期望保證移到每個固定輸入

RANDOMIZED-HIRE先SHUFFLE，再執行HIRE。對任何固定的相異排名輸入，洗牌輸出是均勻排名排列，故期望雇用數Hn，期望費用nci+chHn；另有Θ(n)洗牌操作，不能把時間與雇用費混成同一單位。

若候選人只能依外部到達順序面試、不能重新安排，則無法執行這個洗牌步驟。此時Hn仍需輸入順序的分布假設，不能假裝內部亂數已控制外部到達。

本章的隨機洗牌、隨機pivot在每條亂數路徑都維持答案正確，隨機的是成本；這類通常稱Las Vegas。容許小機率錯答的Monte Carlo需另給錯誤機率及其量詞，期望時間本身不會提供正確率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

5.4 Probabilistic analysis and further uses of indicator random variables

本節四種模型各有不同隨機變數：生日問題數碰撞對或碰撞是否存在；balls and bins數負載及等待時間；streak數視窗及最長長度；線上遴選數選到最佳者的成功機率。期望、機率與最大值不可互換。

會用三個可自足推出的工具：事件聯集的指示至多等於各事件指示之和，所以P(∪Ei)≤ΣP(Ei)；非負X滿足X≥tI(X≥t)，所以Markov界P(X≥t)≤E[X]/t；非負整數L恰為Σj≥1 I(L≥j)，所以 $E[L]=\sum_{j\ge 1}P(L\ge j)$。本章有限和情況可直接交換；幾何等待的無限和由非負部分和單調極限得到。

使用獨立性時會明指哪組試驗互相獨立，不把相鄰且重疊的視窗誤乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

生日問題：碰撞事件的精確機率

k個人各自獨立均勻抽n種日期，n≥1。k≤n時，前j人皆不同的條件下，第j+1人有n−j個可選不碰撞日，所以

$$P(\text{皆不同})=\prod_{j=0}^{k-1}(1-j/n),\qquad P(\text{有碰撞})=1-\prod_{j=0}^{k-1}(1-j/n).$$

k=0、1時無碰撞；k>n由鴿籠原理必碰撞，直接為1，不延伸乘出負因子。對2≤k≤n，1−x≤e^(−x)給P(皆不同)≤e^(−k(k−1)/(2n))，故k(k−1)≥2n ln2是碰撞機率至少1/2的一個充分條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

生日問題：對數的期望不是事件機率

對每對i<j，Iij表示日期相同。獨立且均勻使P(Iij=1)=Σr (1/n)²=1/n，故碰撞對數X的期望為 $E[X] = \binom{k}{2}/n$ 。不需要各個Iij互相獨立。

事件有碰撞是X≥1，Markov僅給P(X≥1)≤E[X]；E[X]≥1不能一般推出P(X≥1)≥1/2。例X以1/M機率等於M²，否則0，則期望M很大，正值機率卻很小。

獨立均勻日期模型同時有精確補集公式與此期望。取k=o(√n)，由union bound機率趨0；取k≥C√n、C足夠大，指數補集界給固定正的碰撞機率。這才把事件門檻證成√n量級。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

十六種日期的完整數例

n=16、k=5時，皆不同機率為15・14・13・12/16⁴=4095/8192，碰撞機率4097/8192≈0.500122；碰撞對數的期望卻是10/16=5/8。兩個值回答不同問題。

指數補集上界在k=5只給e^(−10/16)>1/2，尚不能證明超過半數；k=6時才給e^(−15/16)<1/2。充分上界不必恰好找出精確門檻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

十六種日期的完整數例 (續)

同一日期樣本與整個機率模型：樣本計數和期望要分開

本次唯一碰撞對：(2,4)

模型：16日，5人。P(有碰撞)=4097/8192≈0.500122。

E[碰撞對數]=10/16=5/8；這個期望不是上面的事件機率。

若只有均勻邊際，日期可全等於同一隨機Z，碰撞機率是1，單對也不是1/n。若只有兩兩獨立，單對期望仍成立，但不能直接乘出全體無碰撞機率。

兩兩獨立與完全獨立的差異

令U,V獨立均勻於{0,1,2}，三人的日期為B1=U、B2=V、B3=2U−V mod3。任取兩個日期都能唯一還原U,V，所以各對均勻於9種有序組合，兩兩獨立。

但U=V時三人同日，共3種；U≠V時三個日期皆不同，共6種。因此有碰撞的機率1/3，與三個完全獨立日期的1−3・2・$1/3^3$=7/9不同。兩個模型的碰撞對期望都為3・(1/3)=1：前者以1/3機率出現3對，後者則有不同分布。

此反例顯示：計E[X]用兩兩獨立已足夠；精確無碰撞連乘是更強的聯合資訊。不能由相同的期望反推相同分布。

Balls and bins：負載、空桶與單球桶

t顆球各自獨立均勻投入b≥1個桶。固定桶負載N是t個落入該桶的指示和，E[N]=t/b；且

$$P(N=r)=\binom{t}{r}(1/b)^r(1-1/b)^{t-r}.$$

令Z0為空桶數，Z1為恰有一球的桶數，則E[Z0]=b(1−1/b)^t、E[Z1]=t(1−1/b)^(t−1)，後者t≥1。t=0直接Z0=b、Z1=0；b=1的退化情況依直接事件判斷，或採0^0=1的有限乘積慣例。

t=b=4時E[N]=1，E[Z0]=4(3/4)^4=81/64，E[Z1]=4(3/4)^3=27/16。各桶負載不獨立，因總和固定t；平均一球也不表示每桶各一球。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

幾何等待與收集所有桶的期望

固定桶每次命中率1/b，等待T至少t次才命中的尾式為P(T>t)=(1−1/b)^t，因此E[T]=Σt≥0 P(T>t)=b。這需要各次投球獨立。

收集所有桶時，已見i個不同桶後，下一顆命中新桶的機率(b−i)/b，等待時間Ti的期望b/(b−i)。不論各階段如何相關，線性性給

$$E[C]=\sum_{i=0}^{b-1}E[T_i]=b\sum_{j=1}^{b}1/j=bH_b=\Theta(b\ln b)\quad(b\geq 2).$$

b=1時C=1。b=4各階段期望1、4/3、2、4，總25/3。這是等待時間的期望，不是到第25/3球才一次全部出現。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

幾何等待與收集所有桶的期望 (續)

四桶的收集過程：同一桶重複命中會留在原狀態

狀態 = 已見不同桶數；每次命中新桶才前進。

一次實際投球：2,2,4,1,4,3；新增桶在第1,3,4,6球。
各階段期望相加：1+4/3+2+4=25/3；不是固定停止時刻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

覆蓋機率與第一次碰撞是不同時間尺度

投完整數t球，任一指定桶仍空的機率(1−1/b)^t≤e^(−t/b)。union bound給 $P(C > t) \le be^{-t/b}$。因此t≥b(ln b+c)時未集滿機率≤e^(−c)，不需空桶事件獨立。

若改問首次有桶收到第二球的時間D，則P(D>k)正是前k球皆不同，回到生日問題。上界P(D>k)≤e^(−k(k−1)/(2b))。k≥2時k(k−1)≥k²/2，因此尾和E[D]≤2+Σk≥2 e^(−k²/(4b))≤2+∫0^∞ e^(−x²/(4b))dx=2+√(πb)。對k≤√b/2，union bound給碰撞率≤k(k−1)/(2b)≤1/8，因此前Θ(√b)個尾機率至少7/8，給Ω(√b)。

所以第一次碰撞期望Θ(√b)，集滿期望Θ(b ln b)。這兩個停止條件不同，不能把同一個balls/bins名稱當成同一個答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Streak：視窗次數與最長長度

獨立公平擲硬幣n次，L為最長連續H長度；無H時L=0。對1≤k≤n，令Aik表示從i開始的k格全H，合法起點1..n−k+1。P(Aik)=2^(−k)，所以k視窗數Xk的期望為(n−k+1)/2^k。

相鄰視窗可重疊，並不獨立。例如A1,3與A2,3同時成立需前4格H，機率1/16，不是$(1/8)^2$。E[Xk]仍可相加；但P(L≥k)=P(∪i Aik)只能由聯集或其他聯合分析求。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Streak：視窗次數與最長長度 （續）

同一段四連 H 會包含兩個長3視窗，不能當成獨立事件

長3全H視窗：起點1、5、6，共3個。

最長run：5..8，長度4。

重疊視窗可同時成立；第5與第6視窗共享兩次擲幣。

這個實際序列在i=1、5、6各有長3全H視窗，共3個；第二段四連H會貢獻兩個視窗，不是兩段互不相交的run。最長L=4，不是視窗數3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最長 streak 的一般上界與尾界

union bound給P(L≥k)≤min(1,n・2^(−k))。令k0=⌈lg n⌉，利用整數尾和，n≥1時

$$E[L] = \sum_{k=1}^{n} P(L \ge k) \le k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \le \lceil \lg n \rceil + 1.$$

對任何r≥1，門檻k=⌈r lg n⌉給P(L≥k)≤n^(1−r)。例如n=1000、k=20時精細union bound是981/2²⁰<1/1000；k=30時≤1000/2³⁰<10^(−6)。這是單次執行的尾機率保證，比只給期望更有資訊。

n=0直接L=0，不在lg n式中代0；若k>n則事件不可能、機率0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

下界要利用不重疊的獨立區塊

對充分大n，取s=⌊(lg n)/2⌋≥1，m=⌊n/s⌋，切出m個互不重疊的長s區塊，剩餘不足s格不使用。每塊全H機率2^(−s)≥1/√n，且不同塊用到不同的獨立硬幣，事件互相獨立。

$$P(L < s) \le (1 - 2^{-s})^m \le e^{-m2^{-s}} \le e^{-m/\sqrt{n}} \longrightarrow 0,$$

因m=Θ(n/lg n)，m/√n→∞。因此E[L]≥sP(L≥s)=Ω(lg n)。與前頁上界合併得Θ(lg n)。只有「重疊視窗期望很多」不足以證這個下界；本證明真正取得至少一塊成功的高機率。

例n=16，取s=2、m=8，至少一個獨立二格區塊全H的機率1−(3/4)^8=58975/65536，因此E[L]≥2・58975/65536。這只是可核對的有限下界，不是L的精確期望。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線上遴選的完整模型與程序

n≥2位候選人能力相異，順序均勻隨機；每次只能比較已見者，拒絕後不能召回，只能雇用一次。選定整數1≤k<n，先拒絕前k位並記其最高分M；之後第一位超過M者就雇用；若始終沒有，雇用最後一位。

```
ONLINE-SELECT(n,k):
    M = -infinity
    for i = 1 to k: M = max(M, score(i))
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

這個M不必在第二個迴圈更新：在第一次超過M前，所有已見分數皆≤M；第一次超過時立即返回。已知n、均勻順序、相異分數與無召回是分析前提，不是在任意到達分布下都保證1/e。

選到最佳者的成功機率：以條件機率證明

設最佳者在位置i。i≤k一定失敗；i>k時，它能被選到若且唯若前i−1人的最高者在前k位：若最高者在後段較早位置，就早已觸發返回；反之在i前都不會超過M。

P(最佳者在i)=1/n。條件於此事，前i−1人的相對順序仍均勻，最大者位置均勻，所以成功條件機率k/(i−1)。不同i互斥，得

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

這個條件式已完整證明，不需未經證明便斷言兩事件獨立。n=1直接選唯一者、成功率1；k=0另定義為直接選第一人、成功率1/n，不把0乘未定義的調和項。

有限 n 的最佳門檻與 1/e 極限

令qk=Σj=k..n−1 1/j。精確差分為 $P_{k+1}-P_k=(q_{k+1}-1)/n$。qk嚴格下降，因此最佳門檻在qk≥1、q(k+1)≤1的位置；若恰等1可能有相鄰並列最大，不是每個n都恰好n/e。

對k≥1，積分夾界給 $\ln(n/k)\le q_k\le \ln(n/k)+1/k$，所以

$$\frac{k}{n}\ln\frac{n}{k}\le P_k\le \frac{k}{n}\ln\frac{n}{k}+\frac{1}{n}.$$

令x=k/n，f(x)=x ln(1/x)，f'=ln(1/x)−1，f''=−1/x<0，最大在x=1/e。取k在n/e附近的整數，f(k/n)=1/e−O(1/n)，成功率至少1/e−O(1/n)，且最優成功率≤1/e+1/n，故極限1/e。此界無多餘的1/k因子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六人的門檻與逐人選取都算出來

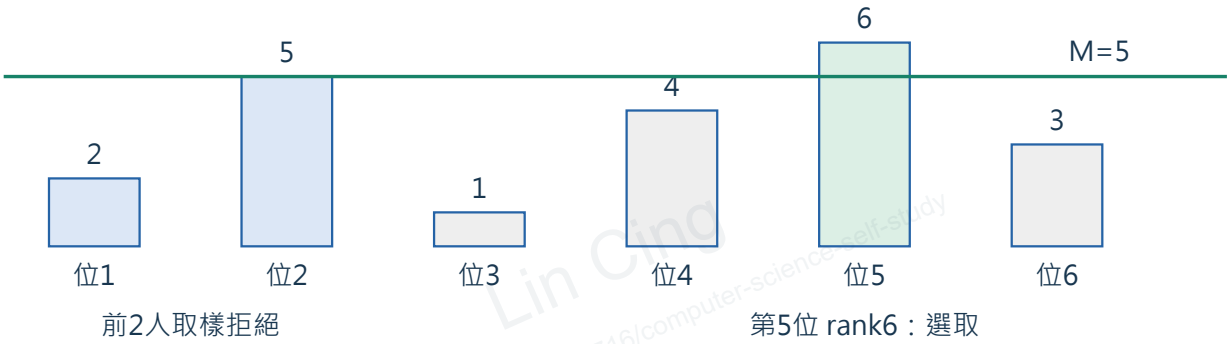
n=6，各門檻精確成功率：k=1為137/360，k=2為77/180，k=3為47/120，k=4為3/10，k=5為1/6。q2=77/60>1、q3=47/60<1，故k=2最優。

排名[2,5,1,4,6,3]，前兩人拒絕並得到M=5；位置3的1、位置4的4都未過門檻；位置5的6過門檻，立即選取，位置6不必面試。這次選到全域最佳，但換一個排列可能失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六人的門檻與逐人選取都算出來 （續）

排名 [2,5,1,4,6,3]：取樣最高5，第5人首次超越門檻



失敗例[5,6,1,2,3,4]的最佳6在取樣段已拒絕，後段無人超過M=6，最後依規則選4；此例說明正確執行策略不等於每次都成功。

延伸：隨機遞迴分割的條件式

大小n≥2的工作先輸出n個星號，均勻選p∈{1,⋯,n−1}，遞迴處理p與n−p；大小1輸出一個驚嘆號後停止。令Ln、Sn分別為驚嘆號與星號的期望，L1=1、S1=0。

固定p後，兩段成本相加；再對所有p取條件期望，得到

$$L_n = \frac{2}{n-1}\sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1}\sum_{j=1}^{n-1} S_j.$$

每次分割保存大小總和，終止葉的大小都是1，因此每一棵執行樹恰有n葉，Ln=n不只是均值。高度與星號數仍可能隨p變；把「極端pivot可成鏈」誤當葉數也變動，混淆了不同樹量。

隨機分割的完整封閉式

Ln的強歸納：假設j<n皆Lj=j，代入得2/(n−1)・n(n−1)/2=n。對Sn，寫成(n−1)Sn=n(n−1)+2Σj<n Sj，再將n+1式減去此式，得到

$$S_{n+1} = 2 + \frac{n+1}{n} S_n.$$

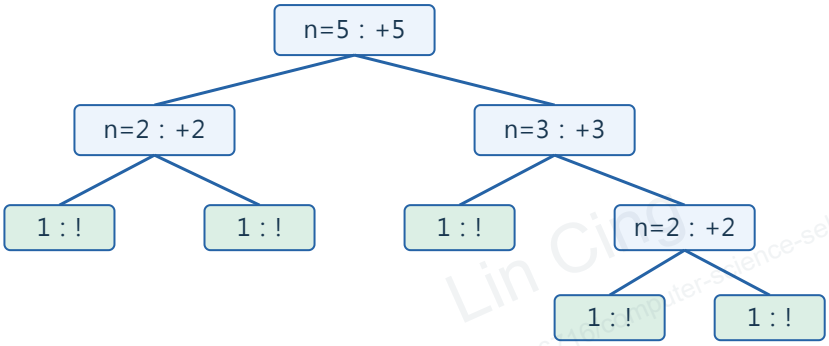
令Qn=Sn/n，則Q(n+1)=Qn+2/(n+1)。由Q1=0，累加得Qn=2Hn−2，故Sn=2n(Hn−1)=Θ(n ln n)。這是期望，不是每條路徑的固定成本。

n	每次執行葉數	E[星號數]
2	2	2
3	3	5
4	4	26/3
5	5	77/6
6	6	87/5

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機分割的完整封閉式 (續)

五個單位葉完整展開：守恆的是葉數，不是星號成本



本次葉數=5

本次星號=12

E[星號]=77/6

+數字：本輪星號

! : 一個驚嘆號

圖示 p=2、之後 3=1+2；葉數始終5，但星號總數會隨分割而變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：隨機 Quicksort 的期望比較數

鍵相異時，對每個大小≥2的子問題，獨立均勻選一個元素作pivot，將其餘元素各與pivot比較一次，分入小於與大於兩組，遞迴排序兩組後串接。基例大小0、1直接回傳。兩組的鍵序與遞迴歸納保證排序正確；此處分析鍵比較數，不含複製、配置及亂數成本。

把鍵按值記z1<⋯<zn，Iij表示zi,zj是否直接比較。每對最多比較一次：只有其中一個作pivot時可能比較，之後pivot被移出。可等價地先均勻產生所有鍵的優先順序，每個子問題選其中最早者為pivot；剩餘鍵的相對順序仍均勻，故等價於各子問題均勻抽pivot。區間zi..zj中最早者因此均勻於該區間；區間外的pivot只會把此整段完整留在同一child，不會先分開這對。

若第一個是兩端便比較；若是中間便永遠分開。因此P(Iij=1)=2/(j−i+1)，總比較數X=Σi<j Iij。

Quicksort 的和式、數值與邊界

按距離d=j−i分組，距離d共有n−d對，得到

$$E[X] = 2\sum_{d=1}^{n-1} \frac{n-d}{d+1} = 2\sum_{r=2}^{n} \left(\frac{n+1}{r} - 1 \right) = 2(n+1)H_n - 4n.$$

n=5時2(4/2+3/3+2/4+1/5)=37/5；n=2,3,4,6則依序1、8/3、29/6、103/10。若分割每個元素僅作常數工作，總時間Θ(n+X)，期望Θ(n lg n)。極端pivot序列仍需n(n−1)/2次比較；高機率時間界還需要另外證明，不能由期望自動推出。

相異鍵或明確的重複鍵處理是必要條件。二路Lomuto把所有相等鍵送到同一側時，隨機pivot也不能避免全等輸入的平方退化；第7、9章的三路分割另處理此問題。本節不能把相異鍵的成對機率直接移植過去。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 05。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§5.1-5.4 and 7.1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17 攤銷分析

Part IV Advanced Design and Analysis Techniques

本章定位

資料結構的一次操作可能很昂貴，整段序列卻仍有線性總成本。本章以stack、counter與動態表推導三種攤銷方法，核心是初始信用、每個前綴的帳目，以及擴縮交界是否仍能支付搬移。

前置知識

序列操作與總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 區分攤銷成本與平均成本。
- 用 aggregate、accounting、potential 三法分析同一結構。
- 證明動態擴張表在加倍策略下 O(1) 攤銷。
- 17.1 Aggregate analysis
- 17.2 The accounting method
- 17.3 The potential method
- 17.4 Dynamic tables

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.1 Aggregate analysis

攤銷分析對任意合法操作序列給總成本上界。若從固定初態出發，任意長度 m 的序列都滿足 Σc_i≤Am+B，就能說每次操作有 O(1) 攤銷成本。這不表示每個 c_i 都是常數，也沒有對隨機輸入取平均。

以空 stack 的 PUSH、POP、MULTIPOP(k) 為例。PUSH 成本1，成功 POP 成本1，MULTIPOP 移除 q=min(k,size) 項，成本 q；呼叫與空測試的固定開銷另加 O(1)。

每個元素只被推入一次、彈出至多一次，所以整段序列的彈出工作不超過 PUSH 次數。m 次操作至多 m 次 PUSH，元素工作總成本≤2m；即使某次 MULTIPOP 花 Θ(m)，任意整段序列仍是 O(m)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二進位計數器的總翻轉次數

固定 b≥1 位元 counter 初值全0，INCREMENT 從最低位開始，把連續尾端1翻成0，再把第一個0翻成1；全1則溢位回全0。成本以翻轉次數計。

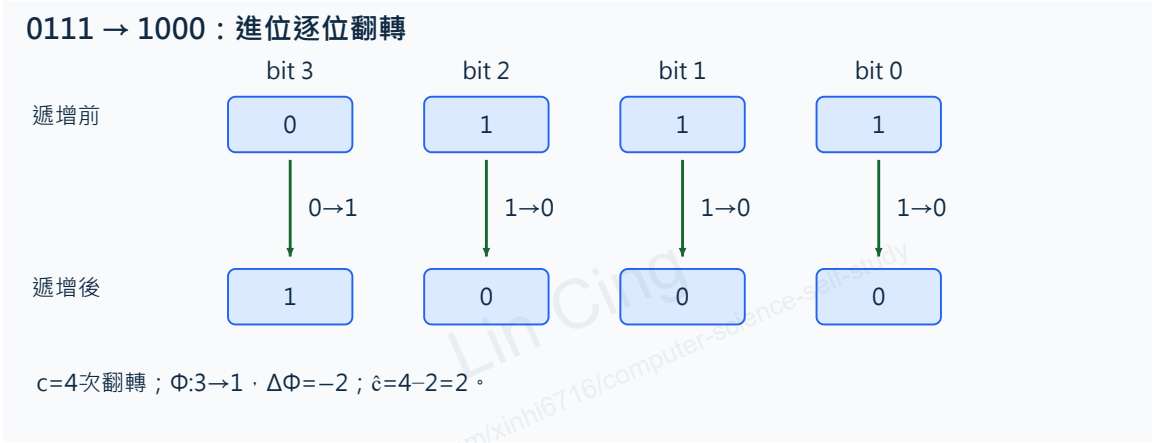
第 j 位每 2^j 次遞增翻轉一次。m 次遞增總翻轉數為

`Σ_(j=0)^(b−1) floor(m/2^j) < 2m`。

不需要假設各位元獨立。位元0的 m 次翻轉也給 Ω(m) 下界，故總成本 Θ(m)。若每次都掃完 b 位才停止，即使高位沒有翻轉也算成本，這個實作就不是本模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二進位計數器的總翻轉次數 （續）



c=4次翻轉；Φ:3→1，ΔΦ=−2；ĉ=4−2=2。

倍增表的幾何和上界

先只考慮插入，起始 (元素數n,容量M)=(0,0)。第一次配置1格；其後滿表時容量倍增。成本定義為1次寫入加搬移舊元素數，配置/釋放的固定開銷計常數。

若插入 m≥2 個元素，最後一次搬移量 L 是小於 m 的最大2冪。所有搬移量為1,2,4,⋯,L，總和 2L−1<2m，故

$T(m)=m+2L-1<3m$。

不能把搬移量一般寫成≤m。m=5時搬移1+2+4=7>5，總成本12；m=8時搬移7，總成本15。只有在某些特定終點，搬移量才剛好小於插入次數。

若新儲存區每格都須初始化，初始化也要計成本；其幾何和仍為 O(m)，但本節精確常數3需相應調整。

17.2 The accounting method

會計法先向每操作收一個攤銷費用，收入減實際支出形成信用。要證明所有操作前綴的信用都不負，才能用總收費上界總實際成本。

空 stack 可對 PUSH 收2、POP/MULTIPOP 收0。推入時1支付本次工作，另1附在新元素上；這項元素日後彈出時使用自己的1。因不可能彈出不存在的元素，任何前綴不會透支。若每呼叫另有固定成本，再加固定收費即可。

counter 每次遞增收2。每次0→1先付1翻轉費、另存1在該位元；每次1→0用該位元原有的1支付。溢位全翻回0時全部由舊信用支付。初值全0沒有信用；非零初值則要另給初始1位元足夠信用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增表的收費與真實序列

每插入收3。由前述任意前綴 T(m)<3m，信用 $B_m=3m-T(m)\geq 0$，故收費足夠。下表從空表開始，不把搬移發生位置平移到下一次操作。

次數	操作後(n,M)	搬移量	c_i	累計成本	信用B_i
1	(1,1)	0	1	1	2
2	(2,2)	1	2	3	3
3	(3,4)	2	3	6	3
4	(4,4)	0	1	7	5
5	(5,8)	4	5	12	3
6	(6,8)	0	1	13	5
7	(7,8)	0	1	14	7
8	(8,8)	0	1	15	9

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增表的收費與真實序列 （續）

第3次才是容量2→4；第4次不搬移。這個整體信用帳也可由下一節的勢能不等式逐步證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.3 The potential method

狀態 D_i 的勢能為 Φ(D_i)，定義

$\hat{c}_i=c_i+\Phi(D_i)-\Phi(D_{i-1})$。

對 m 次操作相加，中間勢能完全抵消：

$\sum c_i=\sum \hat{c}_i+\Phi(D_0)-\Phi(D_m)$。

若所有可達終態滿足 $\Phi(D_m)\geq\Phi(D_0)$，就得到 $\sum c_i\leq\sum \hat{c}_i$。常用充分條件是 $\Phi(D_0)=0$、$\Phi(D_i)\geq 0$。若初始勢能為正，則一般是 $\sum c_i\leq\sum \hat{c}_i+\Phi(D_0)$，不能無故丟掉初始項。

勢能整體平移常數不改變 $\hat{c}_i$；真正重要的是末態相對初態的下界，不是函數是否在所有狀態絕對非負。單次 $\hat{c}_i$ 可以負，因先前蓄積的勢能會補貼本次成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stack 與 Counter 的勢能證明

空 stack 取 Φ=size。PUSH 真實成本1、ΔΦ=1，ĉ=2。移除q項的 POP/MULTIPOP 真實成本q、ΔΦ=−q，ĉ=0。Φ始終非負、初值0，立刻得任意 m 次操作的元素成本≤2m。

counter 取 Φ=1位元數。若此次有t個尾端1翻為0，再有一個0翻為1，c=t+1、ΔΦ=1−t，所以 ĉ=2。若全b位為1而溢位，c=b、ΔΦ=−b，所以 ĉ=0。初值全0，故總翻轉≤2m。

若counter起值為任意b位數，Φ(D_0)≤b，得到總翻轉≤2m+b。第一次從全1開始遞增可花b次翻轉，這與該界一致；不能仍宣稱從任意初態開始每個前綴只需2m。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

17.4 Dynamic tables

現在同時支援插入與刪除末端。容量保持2冪，規則固定如下。

```
insert:
    if M==0: allocate M=1
    elif n==M: allocate 2M; copy n items; M=2M
    write new item; n++
delete-last:
    require n>0
    n--
    if n==0: free storage; M=0
    elif M>=4 and n<=M/4:
        allocate M/2; copy n items; M=M/2
```

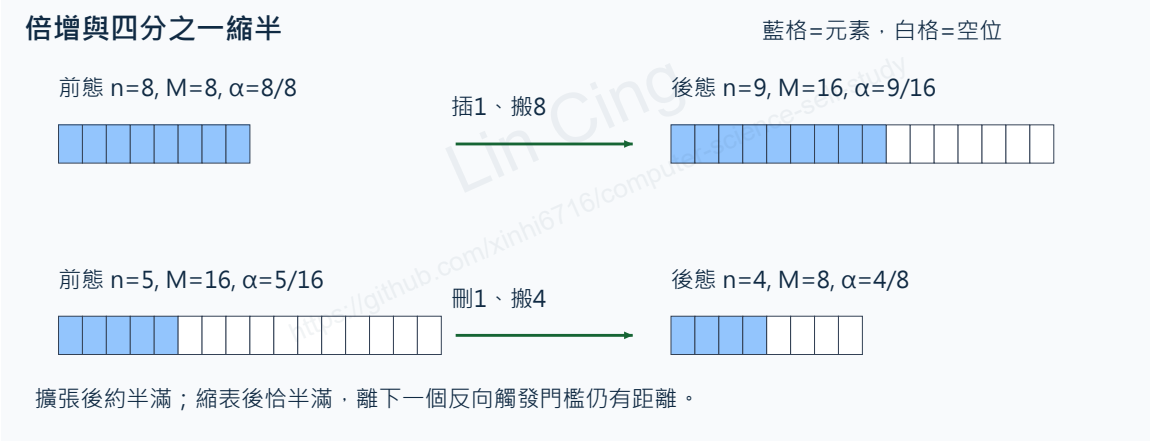
本文用「刪除後n≤M/4就縮半」，與原書「低於1/4」的觸發邊界稍異，因此以下精確數值按本文規則自行證明，不能混用原書邊界的算式。成功刪除的基本成本1，另加縮表搬移n項。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何縮表要留緩衝區

若在刪到半滿時就縮半，取滿表(n,M)=(k,k)，k是2冪且足夠大。插入1項，搬k項變(k+1,2k)；刪除1項後n=k，立即縮回M=k，又搬k項。重複此兩步，每輪Θ(k)，m次操作可達Θ(mk)。

改成1/4縮半後，擴表後約半滿，須再刪Θ(M)次才縮；縮表後半滿，須再插Θ(M)次才擴。這提供蓄積信用的空間，下一頁用分段勢能證明所有混合操作序列，而不只用這個直覺。



分段勢能的定義與幾何

空表取Φ(0,0)=0。非空表定義負載率α=n/M，

```
Φ(n,M) = 2n−M      若 n>=M/2
         M/2−n     若 n<M/2
```

兩段在半滿都為0，且各自非負。滿表Φ=M，足以支付下一次搬M項；四分之一滿表Φ=M/4，與縮表搬移量同階。

只用2n−M並不適合允許刪除的版本。例如合法可達的n=3、M=8時為−2，不能套初態0及末態非負的證明。分段勢能在相同狀態是1，且觸發縮表的具體前後態仍需逐案計算。

插入的逐案攤銷界

首次空表插入：(0,0)→(1,1)，c=1、Φ:0→1，ĉ=2。

無擴表且原本至少半滿，ΔΦ=2，因此ĉ=3。原本低於半滿時，新狀態最多恰達半滿；兩態使用低負載線段連續值，ΔΦ=−1，故ĉ=0。

滿表擴張時，操作前(n,M)=(M,M)，操作後(M+1,2M)。真實成本M+1，勢能由M變2，所以

ĉ=(M+1)+(2−M)=3。

每種插入情況的ĉ≤3。這個推導同時包含「在半滿邊界切換分支」，不假設操作前後一定套同一條式子。

刪除的逐案攤銷界

無縮表且刪後仍至少半滿，ΔΦ=−2，ĉ=−1。若由恰半滿降一項，勢能0→1，ĉ=2；若原本低於半滿而不縮，ΔΦ=1，ĉ=2。

若本次縮表，操作前必為 n_old=M/4+1，刪後n_new=M/4，容量變M/2。操作前Φ=M/4−1，操作後恰半滿Φ=0，真實成本1+M/4，所以

ĉ=1+M/4−(M/4−1)=2。

刪到空時直接釋放，c=1，Φ由非負值降為0，故ĉ≤1。所有操作的ĉ≤3，初勢能0、末勢能≥0，因此任意m次合法操作的實際成本≤3m。這是確定的最壞序列界，沒有機率假設。

完整混合追蹤與模型邊界

從空表插入至n=5，最後(n,M,Φ)=(5,8,2)。接著刪除：

操作	前態(n,M,Φ)	後態(n,M,Φ)	搬移	c	ĉ
delete	(5,8,2)	(4,8,0)	0	1	−1
delete	(4,8,0)	(3,8,1)	0	1	2
delete	(3,8,1)	(2,4,0)	2	3	2
delete	(2,4,0)	(1,2,0)	1	2	2
delete	(1,2,0)	(0,0,0)	0	1	1

五次插入成本12、五次刪除成本8，總20；攤銷費用總和也為20，因兩端Φ皆0。某些操作ĉ為負不表示實際花負時間。

這裡只刪末端，若刪陣列中間並維持連續順序，另有搬移後綴的Θ(n)成本，不能沿用O(1)攤銷。成長倍率、收縮倍率或門檻更改時，須重新驗證勢能斜率與每個邊界。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 17。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

06 堆積排序

Part II Sorting and Order Statistics

本章定位

Binary heap以陣列索引表示近似完全二元樹，支援取極值與優先權更新。本章把父子索引、heap-size及每輪交換對齊，再證明由下往上建堆的線性成本，並區分排序與priority queue的操作需求。

前置知識

完全二元樹、陣列索引與排序不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以 max-heap 性質操作最大優先權佇列。
- 證明 max-heapify 成本與 O(n) 線性建堆。
- 用堆積完成選擇排序式流程並分析 O(n log n)。
- 6.1 Heaps
- 6.2 Maintaining the heap property
- 6.3 Building a heap
- 6.4 The heapsort algorithm
- 6.5 Priority queues

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps

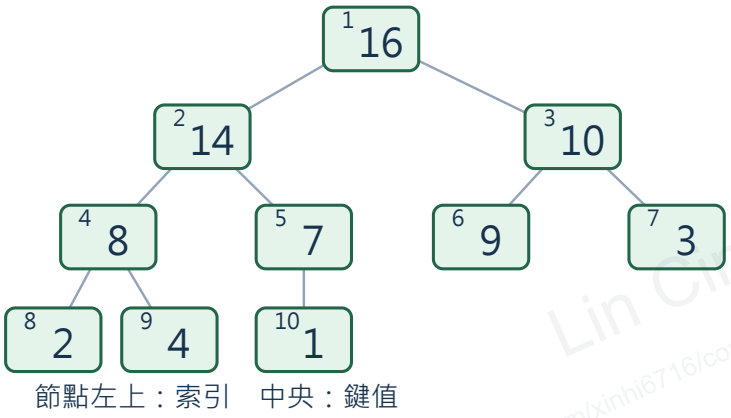
Binary heap的形狀是近似完全二元樹。除最後一層外皆填滿，最後一層由左向右填。陣列採1-index，根1，parent(i)=⌊i/2⌋，left(i)=2i，right(i)=2i+1；只有索引≤heap-size的節點存在。

max-heap的條件是每個非根i滿足A[parent(i)]≥A[i]。沿祖先鏈反覆比較即知根是最大鍵，但同層、左右兩棵子樹沒有排序關係，因此不可直接binary search任意鍵。

陣列length與heap-size不同。heapsort把固定好的尾端排除在heap-size之外，不能讓heapify再碰尾端。若有n≥1個heap節點，深度h的最左索引為2^h，所以樹高⌊log2 n⌋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps (續)



索引固定，鍵值移動

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

每條邊都滿足：父鍵 ≥ 子鍵
同一列的節點不保證有序

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 Maintaining the heap property

MAX-HEAPIFY(A,i,s)的前提是左右child子樹已各自為max-heap，只有i處可能小於child；s是heap-size。若i不在1..s，應由介面拒絕或只對空heap略過呼叫。

```
while true:
    largest=i
    if 2i<=s and A[2i]>A[largest]: largest=2i
    if 2i+1<=s and A[2i+1]>A[largest]: largest=2i+1
    if largest==i: return
    swap(A[i],A[largest])
    i=largest
```

先在兩child中取較大再與父比較，也與此偽碼等價。只因一個child比另一個child大就交換、完全不比較父，則不正確。

Heapify 的正確性與高度成本

若三者最大就是父，兩條父子邊皆合法，子樹原本合法，所以整棵完成。否則最大child的根鍵也不小於其所有後代，把它換到父後，父到兩child的邊皆合法；唯一可能違規的是舊父鍵下移後的新位置。

以剩餘高度歸納，下移的child子樹會被正確修復，另一子樹不變。每次i變為2i或2i+1，嚴格下降一層，故終止。高度h的子樹最壞O(h+1)，根最壞O(log(n+1))；若父原已大於child，實際可O(1)。

例A=[16,4,10,8,7,9,3]、s=7，i=2比較8與7，交換4↔8，得[16,8,10,4,7,9,3]。到i=4時child索引8、9都大於7，立刻停止；值3在索引7，是節點3的child，不是節點4的child。

6.3 Building a heap

```
s=length(A)
for i=floor(s/2) down to1:
    MAX-HEAPIFY(A,i,s)
```

迴圈開始時，每個索引大於i的節點都是合法heap的根。初始化時，[n/2]+1..n都是葉，條件成立。處理i時其child索引較大，兩棵子樹可套heapify前提；完成後i也成為合法heap根，維持下一輪。i降至0時根1合法，整個陣列完成。

空或單元素陣列不執行迴圈，已是heap。i=[n/2]不一定是葉：n=10時i=5仍有child10，必須比較A5與A10。

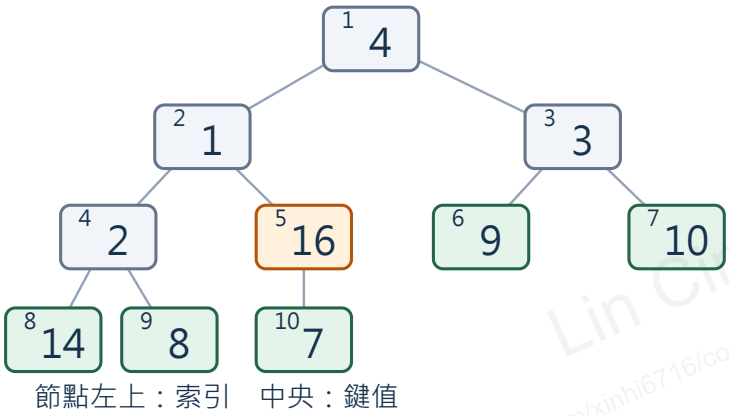
五輪完整陣列與交換

輸入A=[4,1,3,2,16,9,10,14,8,7]，s=10。

i	比較與交換順序	本輪完成的完整陣列
5	A5=16≥A10=7，無交換	[4,1,3,2,16,9,10,14,8,7]
4	child鍵14、8選14；索引4↔8	[4,1,3,14,16,9,10,2,8,7]
3	child鍵9、10選10；索引3↔7	[4,1,10,14,16,9,3,2,8,7]
2	14、16選16；2↔5，再5↔10	[4,16,10,14,7,9,3,2,8,1]
1	先選16再14再8；1↔2、2↔4、4↔9	[16,14,10,8,7,9,3,2,4,1]

以下五張圖逐輪對上同一份資料。橘框是本輪處理的根，綠框標示已保證是heap的子樹根；灰框仍待處理。索引不隨交換改變，改變的是該位置的鍵值。

建堆i=5：與唯一Child比較



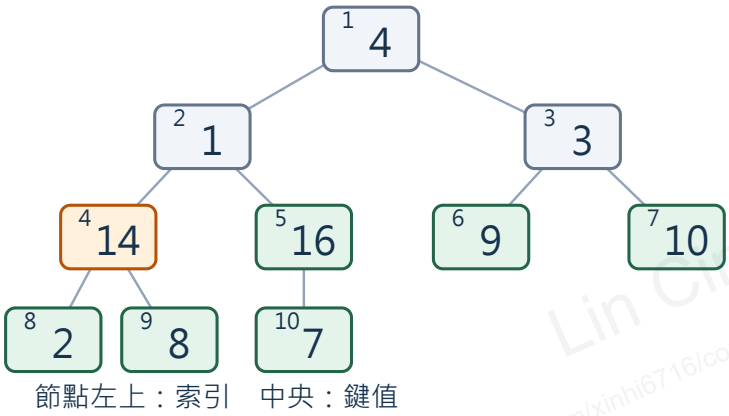
索引固定，鍵值移動

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置5的16已大於位置10的7，沒有交換。位置6..10是葉；位置5的整棵子樹現在也已合法。

建堆i=4：把14提升



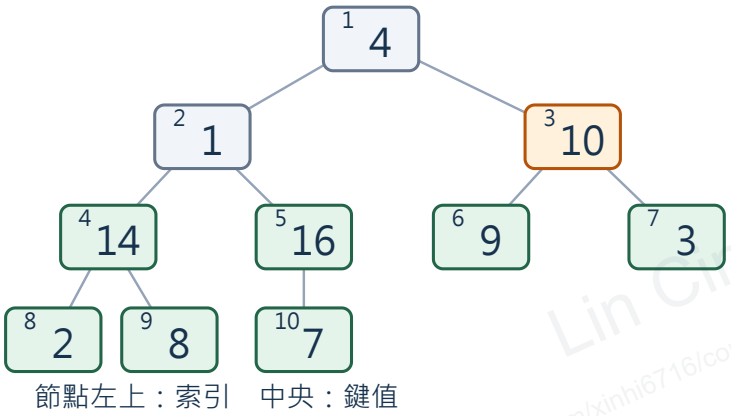
索引固定，鍵值移動

1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置4值2，children分別14、8。先選最大14，交換索引4、8，舊2到葉8停止。沒有把索引9的8也換走。

建堆i=3：把10提升



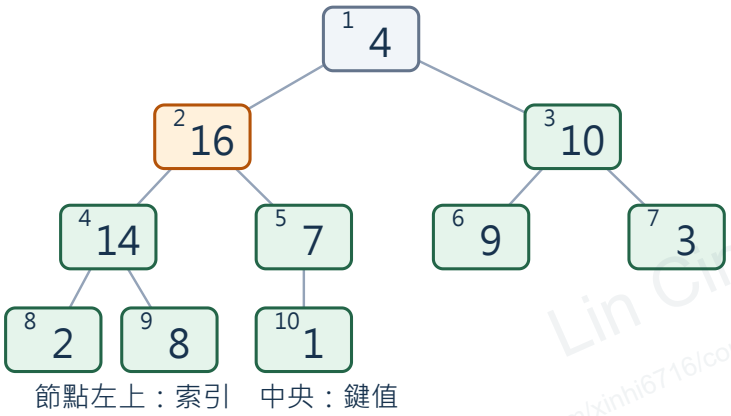
索引固定，鍵值移動

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置3值3，children是索引6的9與索引7的10。交換3、7，舊3到葉7停止；索引6的9不變。

建堆i=2：沿5再到10



索引固定，鍵值移動

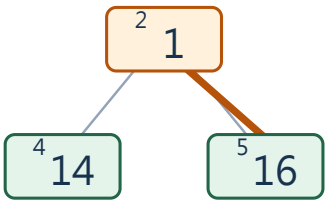
1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置2值1，先與索引5的16交換；舊1到了位置5，還有child10的7，再交換5、10。索引4的14並沒有因本輪被下移。

i=2：舊鍵 1 如何下沉？

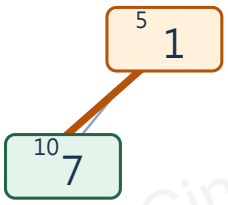
比較 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

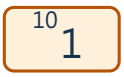
比較 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

到葉，停止



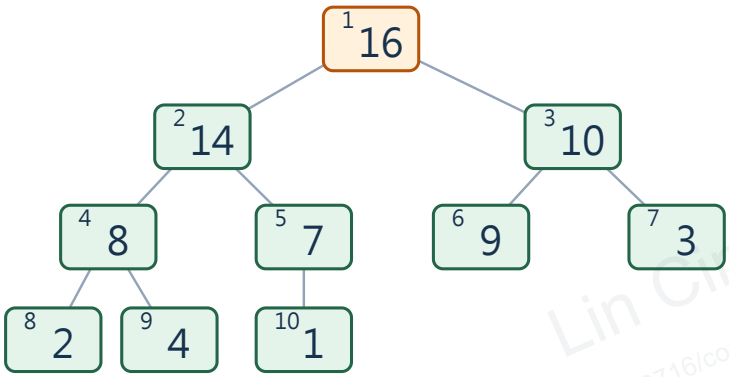
20, 21 > heap-size 10

不再讀取 child

舊鍵 1 留在索引 10

橘框追蹤同一個舊鍵 1；粗邊指出這次選中的較大 child。每個節點左上是索引，中央是鍵值；未畫出的子樹保持不變。

建堆i=1：沿2、4、9下降



節點左上：索引　中央：鍵值

索引固定，鍵值移動

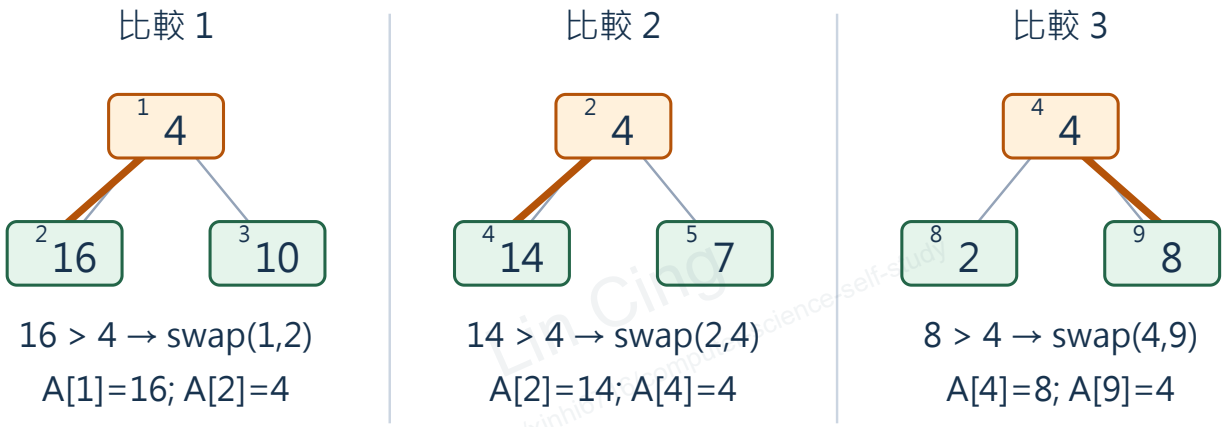
1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

原根4依次與16、14、8交換，停在索引9。最後父子逐一為16≥14,10；14≥8,7；10≥9,3；8≥2,4；7≥1，十個鍵無增減。

i=1：舊根 4 的三次比較



每框都是交換前的局部狀態，底行列出交換後的兩個鍵。4 到索引 9 後是葉；索引 18、19 超過 heap-size=10，停止。不是沿最終陣列的數值順序走訪。

每次交換後，都能核對完整陣列

固定索引為 1..10；下表接續 i=3 完成後的陣列。與前兩張局部圖逐步對照。

本輪根	交換索引	交換後 A[1..10]
2	2 ↔ 5	[4,16,10,14,1,9,3,2,8,7]
2	5 ↔ 10	[4,16,10,14,7,9,3,2,8,1]
1	1 ↔ 2	[16,4,10,14,7,9,3,2,8,1]
1	2 ↔ 4	[16,14,10,4,7,9,3,2,8,1]
1	4 ↔ 9	[16,14,10,8,7,9,3,2,4,1]

自我檢查：遮住結果，先選較大的 child，再判斷是否比父大。每次只能交換兩格；其他八格與元素多重集合不變。

為何Build-Heap是線性

令height(i)為位置i子樹高度。它至少h高，當且僅當最左的h代後裔索引i×2^h≤n，所以至少h高的節點數恰為⌊n/2^h⌋。

把每節點高度分成一層一層加總，

$$\sum_i \text{height}(i) = \sum_{h \geq 1} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

每次heapify成本O(height(i)+1)，且呼叫數⌊n/2⌋，所以總成本O(n)。對n≥2，迴圈本身要處理⌊n/2⌋個節點，給Ω(n)，故build的時間Θ(n)。例如n=15的高度和為7+3+1=11。

逐項INSERT的上浮高度之和則可Θ(n log n)，那不是同一批由下而上的操作，不能套此高度總和。

6.4 The heapsort algorithm

```
BUILD-MAX-HEAP(A)
for end=length(A) down to2:
    swap(A[1],A[end])
    MAX-HEAPIFY(A,1,end-1)
```

每輪前，A[1..end]為max-heap；A[end+1..n]已遞增排序，且其每個鍵都不小於heap內任一鍵。交換把目前最大值放到A[end]，縮小s後只修剩餘heap，因此不變量維持。end=1時全陣列遞增，且所有操作是交換，元素多重集合不變。

時間O(n log(n+1))，由不同鍵比較排序的資訊下界，最壞時間Θ(n log n)。全相等輸入在本章遇相等即停的heapify實作可能只需Θ(n)，不能把最壞界說成每一份輸入都相同。迭代heapify只用O(1)額外空間；遞迴版另有O(log n)呼叫堆疊。

資訊下界的理由是n個不同鍵有n!種輸入次序，正確比較決策樹至少需要n!個葉；高度d的二元樹最多2^d葉，所以d≥log2(n!)。乘積後半至少⌊n/2⌋項各≥n/2，故log2(n!)≥⌊n/2⌋log2(n/2)=Ω(n log n)。因此至少一種輸入要這麼多次比較。

十元素Heapsort的全部輪次

每輪先swap根與heap末端，再heapify剩餘前綴。

end後的新s	完成修復的heap	已排序尾端
9	[14,8,10,4,7,9,3,2,1]	[16]
8	[10,8,9,4,7,1,3,2]	[14,16]
7	[9,8,3,4,7,1,2]	[10,14,16]
6	[8,7,3,4,2,1]	[9,10,14,16]
5	[7,4,3,1,2]	[8,9,10,14,16]
4	[4,2,3,1]	[7,8,9,10,14,16]
3	[3,2,1]	[4,7,8,9,10,14,16]
2	[2,1]	[3,4,7,8,9,10,14,16]
1	[1]	[2,3,4,7,8,9,10,14,16]

十元素Heapsort的全部輪次 （續）

第二輪原根1只沿索引1→3→6下沉，與10、9交換；沒有再與8或2交換。追蹤的是交換路徑，不可把最終陣列中的所有值串成路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5 Priority queues

max-priority queue的MAXIMUM直接讀A1，O(1)；EXTRACT-MAX先保存A1、以最後鍵補根、s減1，再heapify，最壞O(log(n+1))。空時須回傳EMPTY，不可讀A1。

INCREASE-KEY(A,i,k)要求k≥原鍵，改鍵後反覆與父比較：若父較小就交換並向上；父已不小於k或到根即停。只有祖先邊可能違規，向上後每次修好一條，故正確且最壞O(log n)。

INSERT在尾端加新位置，先設特殊−∞再increase-key；也可直接寫新鍵後執行同一上浮，不需型別真的能表示−∞。動態陣列擴容的成本須另作攤銷分析；固定容量版本先檢查FULL。

例A=[20,15,18,7,12,9,17,2]，位置8升為16。先8↔4得[20,15,18,16,12,9,17,7]，再4↔2得[20,16,18,15,12,9,17,7]；到父20停止。

降鍵、指定刪除與穩定性

max-heap降鍵只可能破壞往child的邊，因此向下heapify；升鍵只向上。刪指定位置i可用末鍵補i、s減1，若新鍵大於父就上浮，否則向下heapify。若i就是末端，直接縮短即可。兩分支皆最壞O(log n)，但搜尋任意鍵找到i仍可能Θ(n)。

若外界用handle指定元素，每次swap都要同步更新handle到陣列位置的對照；陣列索引本身不會永久指向同一筆資料。

heapsort一般不穩定。例如(2,a),(2,b),(1,c)，相等鍵heapify不交換時，最終可成(1,c),(2,b),(2,a)，同鍵a、b順序反轉。需要穩定性時可另保存原序號作比較鍵，但這會改變比較規則與儲存模型。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 06。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

08 線性時間排序

Part II Sorting and Order Statistics

本章定位

線性排序以鍵值表示或輸入分布換取速度，必須先分清比較模型的下界。本章把counting的區塊位置、LSD的穩定性與bucket的平方桶大小逐一推導，並明列鍵範圍、位元數、分布及空間成本。

前置知識

比較決策樹、整數鍵值範圍與穩定排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 證明比較排序的 Omega(n log n) 下界。
- 分析 counting sort 的適用條件與穩定版本。
- 理解 radix sort 與 bucket sort 的成本假設。
- 8.1 Lower bounds for sorting
- 8.2 Counting sort
- 8.3 Radix sort
- 8.4 Bucket sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

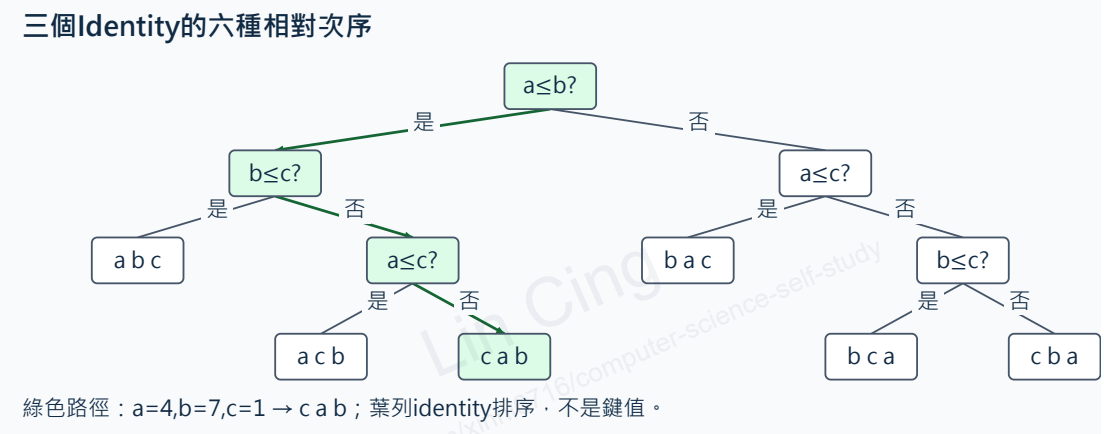
8.1 Lower bounds for sorting

比較排序只能透過兩鍵的比較結果取得次序資訊，不能讀鍵值當索引。本節先限制n個相異鍵，於是每次≤比較只有兩個可能分支。固定n與演算法後，內節點標比較的兩個原始identity，根到葉記錄一次執行；葉必須決定完整的identity排序。

n!種相對次序必須導向可區分的葉：若兩個不同次序到同一葉，演算法在相同結果序列後只能輸出同一個identity排列，至少一個輸入會錯。即使演算法有多個葉代表同一排序，也至少需n!個可達葉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1 Lower bounds for sorting （續）



圖的每片葉標的是identity順序，例如「c a b」表示c<a<b，不是演算法可任意輸出的六組數值。a=4、b=7、c=1走左、右、右，得到c a b。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最壞下界的量詞與模型

高度h的二元樹至多2^h葉，故h≥⌈log₂(n!)⌉。不用Stirling也能求同階界：n!≤n^n，而最後⌈n/2⌉個因子各至少n/2，所以

$(n/2)^{(n/2)} \le n! \le n^n$，因此$\log_2(n!)=\Theta(n \log n)$。

結論是：對每個正確的比較排序、每個n，至少存在一個長度n輸入需要Ω(n log n)次比較。它不是「每個輸入都需n log n」，例如insertion sort遇已排序資料只需n−1次鍵比較。

相異鍵只是取出所有輸入中的一個困難子集，所以允許重複鍵不會降低一般排序的最壞下界。Count/radix利用整數表示和陣列索引，已離開比較模型，不與定理矛盾；同時也多了承受鍵範圍與位元數的成本。

平均比較數也需要證明

若n!種相對次序均勻，選每種次序的一片可達葉，深度為$d_1,\cdots,d_N$，N=n!。不同葉的根路徑是prefix-free二元字串，故Kraft不等式$\Sigma 2^{-d_i}\le 1$；可由把深度d葉視為[0,1)中長度2^{-d}的互不相交二元區間證明。

算術平均≥幾何平均給$2^{-(\Sigma d_i)/N} \le (\Sigma 2^{-d_i})/N \le 1/N$，取log得平均深度≥$\log_2 N=\Omega(n \log n)$。不是只用最高葉深推出平均深度。

對每次都正確的隨機比較排序，固定內部亂數得到一棵正確決策樹；對均勻輸入平均後，每棵樹仍有此下界，再對亂數取期望。故至少一個輸入的期望比較數也達Ω(n log n)。這裡沒有處理允許錯誤輸出的Monte Carlo排序。

8.2 Counting sort

鍵是0..k的整數，記錄另含satellite data（隨鍵搬移的其他欄位）。本節輸入/輸出位置採1-index，計數表鍵採0-index。

```
C[0..k]=0
for j=1..n: C[key(A[j])]++
for v=1..k: C[v]+=C[v-1]
for j=n downto 1:
    v=key(A[j])
    B[C[v]]=A[j]
    C[v]--
return B
```

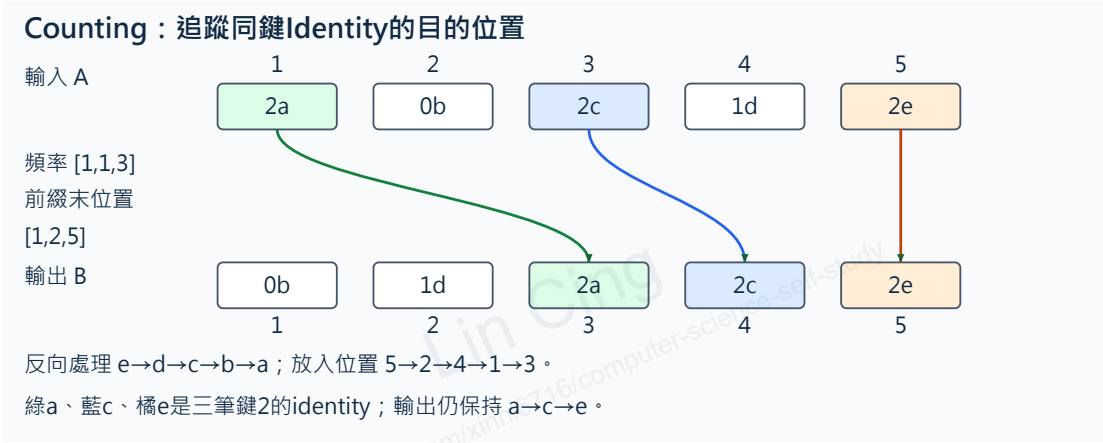
計數後C[v]=該鍵頻率；前綴和後C[v]=鍵≤v的記錄數，也就是v區塊最後一格。回填過程會修改C；若還需要範圍查詢，須先保存未消耗的前綴表P。

從頻率到實際輸出位置

原創例A=[2a,0b,2c,1d,2e]，字母是identity。頻率C=[1,1,3]，累積末位置[1,2,5]。三個區塊為鍵0的位置1、鍵1的位置2、鍵2的位置3..5。

掃描j	記錄	放入B位置	放完後C
5	2e	5	[1,2,4]
4	1d	2	[1,1,4]
3	2c	4	[1,1,3]
2	0b	1	[0,1,3]
1	2a	3	[0,1,2]

從頻率到實際輸出位置 （續）



輸出[0b,1d,2a,2c,2e]。不能只看裸鍵排序就宣稱穩定，必須追蹤a,c,e的相對次序。

回填不變量、穩定性與成本

固定鍵v，令L=原始P[v−1]（v=0時L=0）、R=P[v]。處理某個suffix後，該suffix內鍵v的記錄已佔區塊右端；C[v]正指向尚未填滿區塊的最右位置。反向遇到下一個v，就填C[v]並減一，不重覆、不越出[L+1,R]。處理完全部記錄，各鍵恰填滿其頻率大小的區塊，區塊依鍵遞增排列，故正確。

若同鍵記錄x在輸入先於y，反向掃描先放y於較右格，再放x於較左格，所以輸出x仍先於y。若沿用「末位置後減一」卻改正向掃描，鍵排序仍正確，但同鍵identity反轉。正向也可穩定：改用每個鍵的「起始位置後加一」，兩種規格不能混搭。

四段迴圈時間Θ(n+k+1)，額外空間Θ(n+k+1)，包含輸出B；不是原地排序。鍵可負時先用v−min平移，但範圍k=max−min仍可能巨大；讀寫整個鍵及索引是本章word-RAM的一步假設。

保存P後，0≤a≤b≤k的閉區間計數為P[b]−P[a−1]，約定P[−1]=0。例[1,2]含P[2]−P[0]=5−1=4筆；查詢O(1)，預處理Θ(n+k+1)。

8.3 Radix sort

對固定d位、每位0..r−1的非負整數，LSD（least-significant digit，最低有效位優先）由低位到高位，每輪對目前位做穩定排序。

歸納不變量：完成第t輪後，整個序列已按最低t位的數值排序。t=0沒有條件；第t輪先依第t位分組，不同組的第t位決定大小。同組內靠穩定性保留前t−1位的排序，故合併成最低t位的完整次序。t=d後就是完整鍵序。

「下一輪保留低位排序」只適用同一高位組，不能誤讀成輸出依然全域按低位遞增。位排序不穩定時，這個歸納步失效；例如[12,11]先個位成[11,12]，十位同為1若反轉回[12,11]就錯。

三輪算例與相同位數的次序

三輪算例與相同位數的次序 （續）

輪次	完整序列
輸入	[412,205,421,115,212,405]
個位	[421,412,212,205,115,405]
十位	[205,405,412,212,115,421]
百位	[115,205,212,405,412,421]

十位輪的205、405同為0，必保留前輪的205在405之前；412、212、115同為1，依前輪順序保留。百位輪中205在212之前是已排序低兩位05<12的結果。

把整個陣列反過來由百位→十位→個位排序，即使每輪穩定，也可能讓低位成為最後的主序。正確的MSD方法要在相同高位群內遞迴，不能把兩種演算法混為一談。

位元數、radix與成本的共同分母

每輪穩定counting sort的時間Θ(n+r)，d輪共Θ(d(n+r))，額外空間Θ(n+r)。d為常數且r=O(n)才直接得到Θ(n)；鍵若長到無法常數時間取位，還要計位元存取成本。

b位元鍵每輪取s位（1≤s≤b），則r=2^s、d=⌈b/s⌉，成本Θ(⌈b/s⌉(n+2^s))。最高不足s位可補0，仍可用同一表。n≥2時選s=min(b,⌊log₂n⌋)，有2^s≤n，得到O(n⌈b/min(b,⌊log₂n⌋)⌉)；若b=O(log n)就是O(n)。

以「每輪n+r單位」這個簡化模型比較32位元鍵：s=8需4(n+256)，s=16需2(n+65536)。後者較小當且僅當n>65024，n=65024時相同。這些係數是題設模型，不能當成實測時間；cache、配置、記憶體頻寬會改變實機選擇。

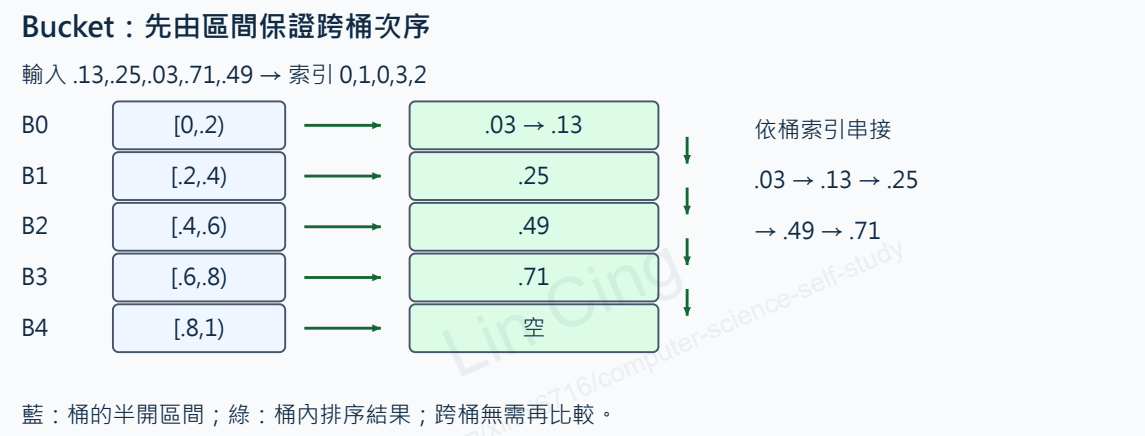
8.4 Bucket sort

輸入n個實數x∈[0,1)，n≥1，建立n個桶B[0..n−1]，放入B[⌊nx⌋]；每桶用insertion sort排序，再依桶索引串接。n=0直接回空序列，不計⌊nx⌋。x=1不在定義域，不能拿它索引B[n]。

正確性不依賴分布：若x≤y，則⌊nx⌋≤⌊ny⌋。不同桶的所有元素已按區間有序，同桶由內部排序完成，所以串接即全排序。每桶按輸入順序append，再用穩定insertion sort，才也保留同鍵identity次序；頭插會先改變順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4 Bucket sort （續）



例n=5，輸入[.13,.25,.03,.71,.49]，桶索引依序[0,1,0,3,2]，不是[0,0,0,1,2]。B0排序為[.03,.13]，B1=[.25]、B2=[.49]、B3=[.71]、B4空；輸出[.03,.13,.25,.49,.71]。

線性期望要計算平方桶大小

設N_i為第i桶大小。建立、分配、串接共$\Theta(n)$，桶內insertion sort總成本$O(\Sigma N_i^2)$，所以$T=O(n+\Sigma N_i^2)$。僅知$E[N_i]=1$不足以把$E[N_i^2]$替換成1。

假設各輸入獨立且均勻$[0,1)$。X_{ij}表示第j筆落桶i，$N_i=\Sigma_j X_{ij}$。$X_{ij}^2=X_{ij}$，$E[X_{ij}]=1/n$；$j\neq k$時，輸入獨立給$E[X_{ij}X_{ik}]=1/n^2$。因此

$E[N_i^2]=\Sigma_j 1/n+\Sigma_{(j\neq k)} 1/n^2=1+n(n-1)/n^2=2-1/n$。

總和$E[\Sigma N_i^2]=2n-1$，故$E[T]=O(n)$，而讀取n筆需$\Omega(n)$，合為$\Theta(n)$。這裡用的是不同記錄間的獨立性；不同桶的N_i彼此反而相關，因$\Sigma N_i=n$，但期望線性性不要求它們獨立。

分布、相關性與最壞情況

若各記錄獨立但桶機率為p_i，$\Sigma p_i=1$，同樣展開得

$E[\Sigma N_i^2]=n+n(n-1)\Sigma p_i^2$。

均勻時$\Sigma p_i^2=1/n$。只要此和為$O(1/n)$，仍有線性期望；若單桶機率固定為1，平方成本就是n^2。分布越集中，撞桶的成對機率Σp_i^2越高。

邊際均勻仍不夠：先均勻抽$J\in\{0,\dots,n-1\}$，令所有記錄$x_j=(J+1/2)/n$。每筆落每桶的邊際機率都$1/n$，$E[N_i]=1$，但每次所有n筆同桶，所以$\Sigma N_i^2=n^2$。此反例針對平方成本界，並不宣稱相等鍵的穩定insertion sort本身要平方時間。

真實最壞時間的達界族可取$n\geq 2$、$x_j=(n-j+1)/(2n^2)$，$j=1..n$：全部在第一桶且嚴格遞減，insertion sort搬移$n(n-1)/2$次，故$\Theta(n^2)$。把桶內排序換成$O(m\log(m+1))$最壞演算法，可令總最壞$O(n\log n)$，並保留原獨立均勻模型的線性期望，因$m\log(m+1)=O(m^2)$。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 08。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10 基本資料結構

Part III Data Structures

本章定位

基本資料結構的困難常在表示法與操作前提：物理陣列順序不等於queue順序，節點handle不等於鍵，free slot也不等於清零。本章以具體前後狀態證明queue、串列、配置器與樹表示的連結不變量和成本。

前置知識

陣列、指標、抽象資料型態與漸近分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用陣列或鏈結表示堆疊與佇列。
- 維持 linked list 的操作不變量與邊界條件。
- 以索引鏈結表示沒有原生指標的物件。
- 10.1 Stacks and queues
- 10.2 Linked lists
- 10.3 Implementing pointers and objects
- 10.4 Representing rooted trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.1 Stacks and queues

ADT（abstract data type，抽象資料型態）先規定操作的語義，再選表示法。Stack的pop移除最後加入且尚未移除的元素，即LIFO；queue的dequeue移除最早加入且尚未移除的元素，即FIFO。兩者的delete都由規則決定對象，不接收任意鍵或任意位置。

固定容量m≥1的陣列stack，本章採0-index、top=有效元素數，資料佔S[0..top−1]，0≤top≤m。

```
push(x): if top==m: error overflow
         S[top]=x; top++
pop():   if top==0: error underflow
         top--; return S[top]
```

Push前先檢查，才不會寫越界；pop前先檢查，才不會減到−1。歸納看每次只延伸或縮短有效前綴，最後有效值正是最近push，故滿足LIFO。Pop後陣列可能還留舊bits，但不屬於邏輯stack；是否清除引用是記憶體管理的額外問題。

環狀Queue的表示不變量

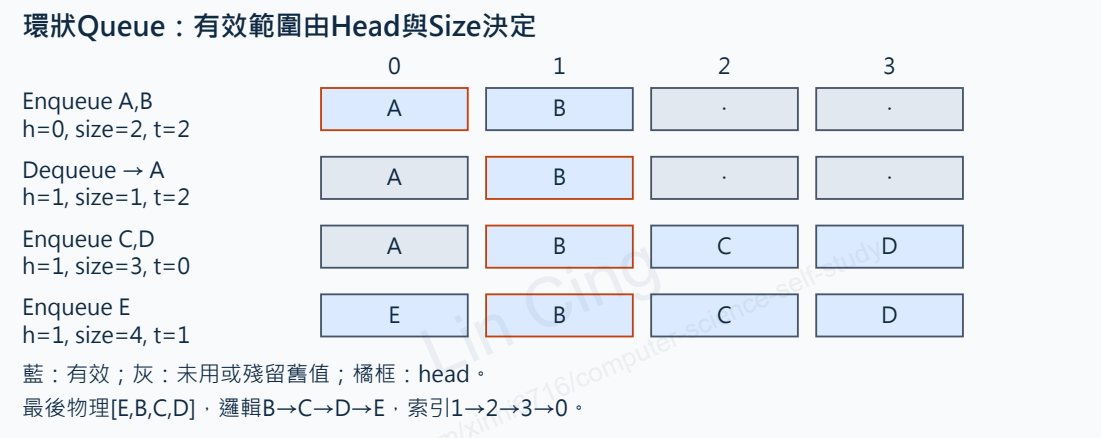
用Q[0..m−1]、head、size，tail定義為下一個寫入位置：tail=(head+size) mod m。第j個邏輯元素位於Q[(head+j) mod m]，0≤j<size。

```
enqueue(x): if size==m: error overflow
            Q[(head+size) mod m]=x; size++
dequeue():  if size==0: error underflow
            x=Q[head]; head=(head+1) mod m; size--; return x
```

Enqueue在原邏輯序列末尾新增，dequeue回傳原第0項並把head前進一格；模m處理跨尾端，故FIFO和索引不變量都維持。size=0與size=m均可有head=tail，必須靠size分辨。

原書也教「保留一格」版本：只存head/tail時，空為head=tail，滿為(tail+1) mod m=head，實際容量m−1。本章用size版本可容m筆，不能把兩種滿判斷混搭。

回繞不會改變FIFO順序



容量4：enqueue A,B；dequeue回A；enqueue C,D；再enqueue E。最終物理[E,B,C,D]，head=1、size=4、tail=1，邏輯B→C→D→E，索引1→2→3→0。此時再enqueue必上溢，下一次dequeue回B。

回繞不會改變FIFO順序 （續）

固定陣列的push/pop/enqueue/dequeue各最壞O(1)，前提是每筆資料可用O(1)搬移；建立m槽陣列本身需Θ(m)空間及相應配置。若滿時自動擴表，單次搬移可能Θ(m)，要改用第17章攤銷分析，不能沿用固定容量的單次最壞界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.2 Linked lists

陣列由索引決定鄰接，linked list由next連結決定邏輯順序；位置相鄰不要求記憶體地址相鄰。Singly linked只有next，doubly linked另有prev。無序串列search最壞必看n筆，Θ(n)；已持有合法節點handle的局部splice則可O(1)。

沒有sentinel時，首節點prev=NIL、尾節點next=NIL，刪除要另改head/tail。Circular doubly linked的sentinel N是實際的dummy節點：N.next指首、N.prev指尾，兩端接回N；空串列N.next=N.prev=N。

表示不變量：沿next由N回到N恰訪問每個資料節點一次，且每條邊u.next=v都有v.prev=u。N不代表任何資料鍵，search遇N表示失敗，delete不可刪N。

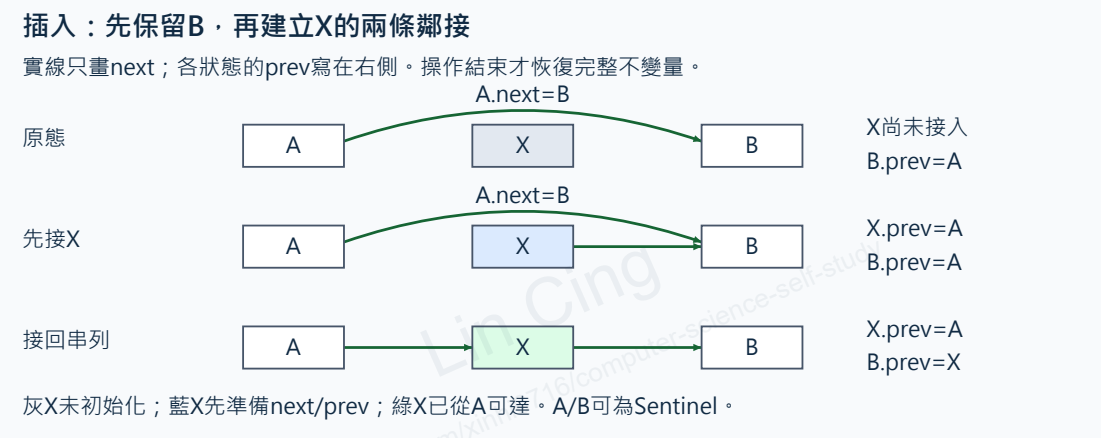
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先保存舊鄰居，再接入新節點

要在已相鄰的A、B間插入未在任何串列的新X，先保存B=A.next，依序寫X.prev=A、X.next=B，最後寫A.next=X、B.prev=X。讀舊鄰居必須在覆寫其來源前完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先保存舊鄰居，再接入新節點 （續）



中間步驟不必每一步都滿足完整雙向不變量；這是單執行緒、整個操作結束後恢復不變量的規格。若有並行讀者或中途例外，需另設同步/交易規則，不能把這四次寫入當成原子操作。

終態A.next=X、X.prev=A、X.next=B、B.prev=X，其餘邊不變，所以全域循環與雙向一致性維持。A或B可為sentinel，同一段程式涵蓋首插、尾插及空串列插入（A=B=N）。

刪除的前提、邊界與成本

對有效資料節點x，先保存a=x.prev、b=x.next，寫a.next=b、b.prev=a。原本a→x→b改成a→b，反向連結同步，其他邊不變。只有一個資料節點時a=b=N，兩次寫入恢復N自環。

例N↔A↔B↔N刪A後是N↔B↔N，仍有B，不能稱為空串列。可把x.next/x.prev清成NIL作除錯，但清除不會使外部舊handle自動失效，也不能取代正確的生命週期規則。

O(1)刪除的前提是呼叫者已給「目前屬於此串列、尚未刪除」的有效handle；若只有鍵，還要先search，最壞Θ(n)。若每次都從頭掃描驗證membership，也會花Θ(n)，不能省略這項成本後仍宣稱整次O(1)。

Singly linked已知前驅可O(1)刪其後繼；只有要刪的x通常不夠，特別是尾節點無法靠複製後繼解決。複製後繼還會改變identity，不是一般handle語義下的任意節點刪除。

10.3 Implementing pointers and objects

沒有原生pointer也能用slot索引當handle。容量m的平行陣列key[0..m−1]、next、prev以同一索引i表示一個物件，NIL採−1，避免與合法slot0混淆。

例active list為slot0→slot2，key[0]=10、key[2]=30；next[0]=2、prev[2]=0。Logical successor是next[0]=2，沒有理由是物理相鄰slot1。

若改用單一陣列，每個固定三欄物件的base=3i，key=A[base]、next=A[base+1]、prev=A[base+2]。必須明定儲存的pointer是slot索引i還是base：本例欄內仍存slot索引，讀下個物件時重新乘3。原書也可直接存base位址，則不可再乘3。

多陣列與單陣列只是layout不同，不改變串列語義。若物件可有不同長度，簡單3i映射就不適用，需要額外的區塊配置/長度資訊，不能直接套以下固定slot free list。

Free list是未配置Slot的Stack

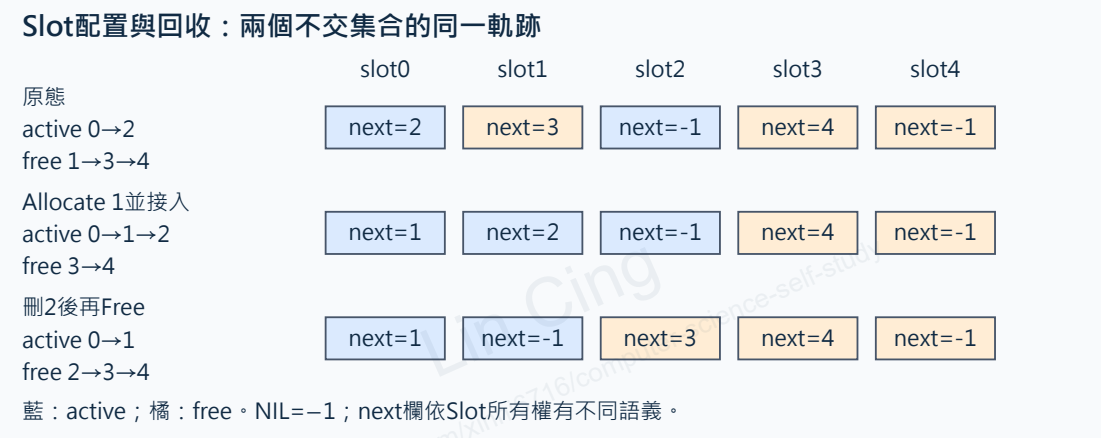
使用free指向第一個未配置slot，空時NIL；free slot的next欄重新作為free list連結。Allocate取首slot，free把已解除連結的slot插回首端：

```
allocate(): if free==NIL: error out-of-space
            x=free; free=next[x]; return x
free-object(x): next[x]=free; free=x
```

所有slot分成已配置與free兩個不交集合，聯集為全部m個slot；free list無環、不重複。Allocate把一個slot由free轉為已配置，呼叫者再初始化key/prev/next並接入資料結構。Free前必須先把x從所有持有它的結構移除，且x尚未free，否則可能破壞active邊或形成free list自環。

Free list只讀next，不需要把free slot的prev/key清零；重新配置後，必須在讀取前初始化要用的欄位。此配置器本身O(1)，不包含search、解構大型satellite data或主動清除所有外部引用。

配置、接入、解除、回收的同一軌跡



初態active0→2、free1→3→4。Allocate回1，free改3；初始化slot1並接於0與2之間，active0→1→2。接著先從active解除2，再free-object(2)，得active0→1、free2→3→4。

配置、接入、解除、回收的同一軌跡 (續)

若free(2)先改next[2]，卻還讓active節點指向2，下一次走訪就會跑入free slots。若同一slot再配置給另一物件，舊handle=2也不會自動分辨「舊物件2」與「新物件2」；使用者不可在free後繼續使用舊handle。若系統必須偵測這種stale handle，可另用(slot,generation)並驗證版本，但那是增加的表示規格。

把物件搬到緊密位置可能改善locality，卻會改變索引handle。只有能更新全部參照、或透過穩定間接表定位時才可搬移；「原物件內容相同」不代表所有指向舊位置的連結仍正確。

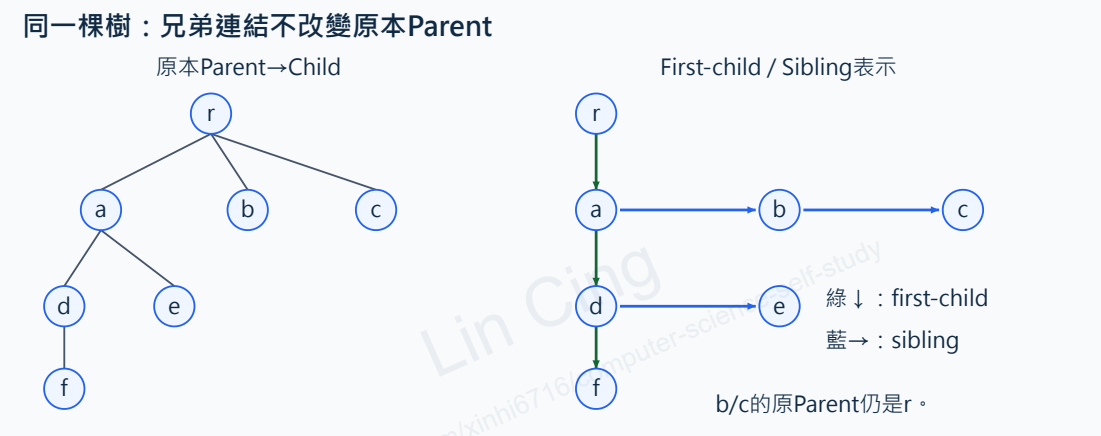
10.4 Representing rooted trees

Binary tree節點可存parent、left、right，根parent=NIL、缺child用NIL；空樹root=NIL。一般有根樹仍是每個非根節點恰一個parent，差別在child數可不固定，不是可以多個parent。

First-child/right-sibling（最左孩子/右兄弟）每節點存first、sibling與可選parent。first[x]是x的第一個child；sibling[x]是與x同父的下一個child。最後一個兄弟sibling=NIL，無child者first=NIL，root沒有兄弟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

10.4 Representing rooted trees （續）



同一例r有a,b,c，a有d,e，d有f。first[r]=a；sib[a]=b、sib[b]=c；first[a]=d、sib[d]=e；first[d]=f。sib邊不是原樹的親子邊，例如b的原parent仍是r，不是a。

從表示法重建走訪與操作成本

```
visit-tree(x):
    if x==NIL: return
    output key[x]
    c=first[x]
    while c!=NIL:
        visit-tree(c)
        c=sibling[c]
```

每個node輸出一次；每個原樹parent的while恰列出其child一次，全樹child總數n−1，所以時間Θ(n)，不是把n個node各乘一個最壞n。Stack深度取決於原樹高度h，O(h+1)，鏈狀可Θ(n)，不是無條件O(log n)。此例preorder為r,a,d,f,e,b,c。

列x的全部children需Θ(deg(x)+1)，parent欄存在時回parent O(1)；只存parent則找全部children可能需掃全部node。表示法要配合操作：heap用完整樹陣列索引省pointer，disjoint-set只往根走可只存parent。

若把first/sibling機械地當binary left/right遞迴，起點若有兄弟也會走進它們；那是「以x開頭的兄弟森林」，不一定是原樹以x為根的子樹。上面先列children的版本會停在正確的原子樹邊界。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 10。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11 雜湊表

Part III Data Structures

本章定位

Hash table以鍵值映射節省稀疏universe的空間，但速度依賴碰撞模型與負載。本章分開證明chaining、universal family、probe排列與兩層perfect hashing，並用Tombstone、去重和失敗query追蹤正確性，讓期望界與最壞保證各有明確條件。

前置知識

機率、模算術與平均情況分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 從 direct-address 推到雜湊平均成本。
- 評估除法、乘法與 universal hashing 的假設。
- 分析 open addressing 的探測與完美雜湊。
- 11.1 Direct-address tables
- 11.2 Hash tables
- 11.3 Hash functions
- 11.4 Open addressing
- 11.5 Perfect hashing

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.1 Direct-address tables

Dictionary維護key→record的映射，本章先規定每個key至多一筆，insert同鍵時更新既有record。Universe U={0,⋯,M−1}夠小時，直接用T[key]，空槽為EMPTY，不與合法record混用。

Search讀T[k]、insert寫T[k]、delete清該槽，各最壞O(1)，空間Θ(M)。初始化每槽為EMPTY通常Θ(M)，不是因每次query O(1)就能把建表成本也寫成O(1)。位元集合可每key存一bit，但不包含satellite data。

例U=0..9，存在鍵2、5、8，只有T[2]、T[5]、T[8]指向record。這是用鍵的數值作索引，沒有碰撞。若鍵是長字串，讀完整key與轉換本身就需依長度付費；以下O(1)的hash與相等比較，採可在固定個word操作的鍵模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.2 Hash tables

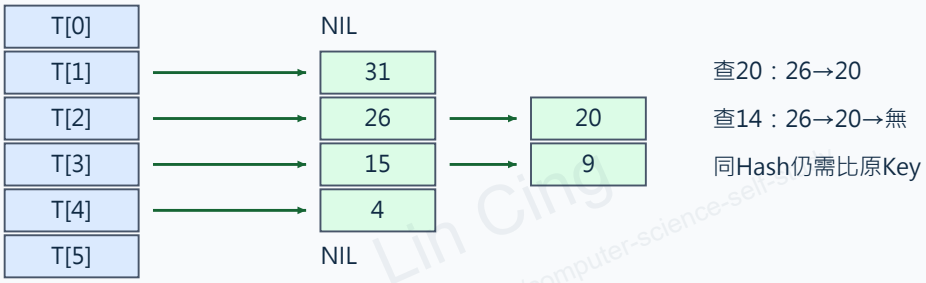
當|U|遠大於實際n筆，改用m槽和h:U→{0,⋯,m−1}。由鴿籠原理，|U|>m時任意固定h必存在不同鍵碰撞。Chaining把同槽記錄接成list，search仍必比原key，hash相同不是key相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.2 Hash tables （續）

Chaining：槽與鏈結記錄是兩層存取

依序插4,9,15,20,26,31；m=6，h(k)=k mod6，頭插。



藍：槽；綠：Record。失敗查14仍掃槽2，不把Hash值當存在證明。

例m=6、h(k)=k mod6，依序插4,9,15,20,26,31且頭插。槽2為26→20，槽3為15→9，槽1為31、槽4為4。找20查兩個record，失敗查14也掃槽2的兩筆。這張特定表的成本不是對隨機h取期望的平均公式。

頭插O(1)以已知新key不存在為前提；本章dictionary更新語義要先search去重，故不能把整個insert都當無條件最壞O(1)。已持有有效handle且list為雙鏈時unlink O(1)，按key刪除仍要付search成本。

Chaining的兩種期望搜尋

令α=n/m，可大於1。Simple uniform hashing假設不同固定key的槽獨立均勻，hash計算O(1)。失敗query k不在表中，對每個已存x，I_x=[h(x)=h(k)]的期望1/m，掃描長度E[ΣI_x]=n/m=α，加固定索引成本得Θ(1+α)。

成功搜尋另假設每筆已存記錄等可能被問到。固定插入次序$x_1,\cdots,x_n$、頭插且沒有刪除，找x_i會檢查自己與之後插入的同槽記錄，所以

`E[成功檢查筆數] = (1/n)Σ_i(1+(n−i)/m) = 1+(n−1)/(2m)`。

這個較精確式需要平均query與頭插次序，不能把「每個槽等可能被成功查詢」當成理由：大chain含更多key，較可能被選到。若只需O(1+α)上界，可用包含該key的整條chain長度，不必採等機率成功query。

保住α=O(1)才有常數期望；任何固定表仍可能有一條n長chain，單次最壞Θ(n)。若擴表重選h，搬移和重新計hash的成本也要計，動態表攤銷另見第17章。

11.3 Hash functions

Division method為h(k)=k mod m。若m=2^p，只保留最低p位，例0與8、6與14在m=8時碰撞。選質數有時能避開特定規律，但不是對任意key集合的均勻性定理；選好的m也不能消除鴿籠限制。

長字串可用Horner逐字計算：$H_0=0$，$H_{i+1}=(B \cdot H_i+c_i) \bmod m$，得到完整base-B整數的mod m而不建立巨大整數，時間Θ(字元數)。若B≡1 mod m，任何字元重排有相同和，仍會碰撞，公式本身不保證分布好。

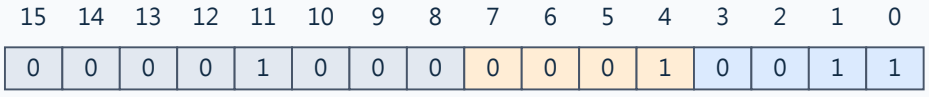
Multiplication method採0<A<1，$h(k)=\lfloor m \cdot \mathrm{frac}(kA) \rfloor$。若A=s/2^w、m=2^p，先算k・s的低w位r_0，再取r_0的最高p位，不能誤取整個2w位乘積的最高p位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.3 Hash functions （續）

Multiplication Hash：取低Word的高位

w=8, s=159, k=13；k·s=2067=8·256+19；m=16取p=4位。



高Word 00001000 = 8　　低Word 00010011 = 19

橘色是低Word的高4位：0001 → h=1。

等價計算：(2067 mod256) >>4 = 19 >>4 = 1。

灰：捨棄的高Word；橘：取出的位；藍：低Word其餘位。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例w=8、s=159、k=13，乘積2067=8・256+19，低word=00010011。m=16取低word高四位0001，h=1；算⌊16・19/256⌋也得1。

Universal不是「看起來夠亂」

函數族H稱universal，如果對每對固定相異鍵x,y，都有Pr_(h均勻取H)[h(x)=h(y)]≤1/m。鍵集合與query必須固定且獨立於這次h的選擇；不是假設使用者key天生均勻。

對固定query k，碰撞對指示變數線性求和，若k不在表，E[chain長度]≤n/m；若在表則≤1+(n−1)/m。這只用pairwise碰撞界，完全不要求各碰撞事件互相獨立，也不聲稱每個槽的輸出都均勻。

因此初始化m槽之後，固定操作序列、最多O(m)筆同時存在時，chaining每次search及需去重的insert期望O(1)，總t次操作期望O(t)；初始化的Θ(m)仍另外計。若對手看到h或由回應自適應選新鍵，原本「固定鍵對」保證不能直接沿用。Universality也不是密碼學不可預測性，無須為此假定h是永久秘密。

質數Affine族的完整碰撞證明

選質數P大於所有合法key，a均勻取1..P−1、b取0..P−1，定義h_(a,b)(k)=((ak+b) mod P) mod m。

固定x≠y，先看r=(ax+b) mod P、s=(ay+b) mod P。因a、x−y皆在域F_P非零，r−s=a(x−y)≠0。反過來，每對r≠s唯一決定a=(r−s)(x−y)^(−1) mod P、b=r−ax mod P。因此均勻(a,b)對應均勻的P(P−1)個有序相異(r,s)。

固定r後，另P−1個可能s中，與r同mod m者至多⌈P/m⌉−1≤(P−1)/m，碰撞機率至多1/m，故H universal。若m≥P，mod m根本不再合併r,s，碰撞機率0，也成立。

需要P為質數是為了每個非零x−y可逆；任意合數不能照搬證明。運算成本假設P、乘積模運算與key都在指定word-RAM可O(1)處理的範圍；任意精度大整數須另外計費。

11.4 Open addressing

所有record直接放m槽內，故n≤m。每key有固定probe序列h(k,0),⋯,h(k,m−1)，同key搜尋/插入必重現同序列。為保證有空槽就可插，序列須覆蓋每槽，常要求是全表排列。

槽有三態：EMPTY（從本次建表起未曾使用）、DELETED（tombstone，曾刪除）、OCCUPIED(record)。Search遇key相等回傳；遇EMPTY可判不存在；遇DELETED繼續，最多m次探測。全表沒有EMPTY時仍會終止，不得用無限while等空槽。

EMPTY可提早停的理由是：若key曾被插到更後面，插入時不可能跳過當時EMPTY；後續刪除只變DELETED而不變EMPTY，所以沒有無故新生的停止缺口。

Tombstone與不重複Key的Insert

```
insert(k,value):
    first_deleted=NIL
    for i=0..m-1:
        j=h(k,i)
        if T[j] is OCCUPIED and T[j].key==k:
            update value; return j
        if T[j] is DELETED and first_deleted==NIL:
            first_deleted=j
        if T[j] is EMPTY:
            place at first_deleted if set, otherwise j; return
    if first_deleted!=NIL: place there; return
    error full
```

不能看到第一個DELETED就立即插入，否則同key可能還在後面，造成重複record。只有呼叫者已保證key全新時，才可省略後續去重掃描。Delete先search，找到後設DELETED，未找到則狀態不變。

Tombstone與不重複Key的Insert （續）

Tombstone：保住Probe路徑，也保住Key唯一性

m=7，home=k mod7，線性探測；查15沿1→2→3跨過DEL。

	0	1	2	3	4	5	6
插1,8,15	·	1	8	15	·	·	·
刪1後	·	DEL	8	15	·	·	·
插新22後	·	22	8	15	·	·	·

先插8只更新槽2；插22記住DEL槽1，探到EMPTY槽4才回填1。

灰：EMPTY可停；橘：DELETED不可停；藍：OCCUPIED需比較Key。

同一條Probe路徑如何保持可搜尋

m=7、home(k)=k mod7。依序插1、8、15，分別在槽1、2、3。刪1把槽1設DELETED；找15仍探槽1→2→3，回槽3，不能在槽1停止。

此時insert(8,new-value)會記住槽1可重用，但繼續到槽2找到8後更新，不在槽1再造一筆8。插新22時依序探1(DELETED)、2(8)、3(15)、4(EMPTY)，確認不存在才把22寫回第一個DELETED槽1。

活躍負載α=n/m下降不等於probe變短：大量tombstone會讓很低n的表仍沒有EMPTY，失敗search花m次。下面的簡潔期望界先假定無刪除/tombstone；若需套到動態工作負載，必須另外限制tombstone比例或重建並計其成本。

三種Probe公式與覆蓋條件

Linear probing：$(h_0(k)+i) \bmod m$一定覆蓋全表，但相鄰cluster會吸附後來home落在該區的key，形成primary clustering。相同home必有同probe序列。

Quadratic probing：$(h_0(k)+c_1 i+c_2 i^2) \bmod m$不會對任意參數自動覆蓋全表。例m=7、偏移$i^2$只得到0,1,4,2,2,4,1，尚有空槽也可能找不到。相同home仍同序列，稱secondary clustering。

一個可證全覆蓋的版本為$m=2^w$、偏移$f(i)=i(i+1)/2$（先整數除2再mod m）。若$f(i)=f(j) \bmod 2^w$，則2^{w+1}整除$(i-j)(i+j+1)$。兩因子一奇一偶，偶因子必被2^{w+1}整除；在$0 \le i,j<2^w$範圍，只有$i-j=0$可行，故i=j，序列為排列。例如m=8偏移0,1,3,6,2,7,5,4。

Double hashing：$(h_1(k)+i \cdot h_2(k)) \bmod m$，週期為$m/\gcd(h_2(k),m)$。寫$m=gm'$、$h_2=gd'$、$\gcd(m',d')=1$，回起點等價$m'$整除i，所以最小正週期$m'$。必須互質才覆蓋全表；m質數配非零step，或m為2冪配奇step皆可。

理想Uniform probing的失敗搜尋界

Uniform hashing在此指每key的probe permutation均勻取自m!種，且新query相對於現有表仍為均勻未偏的排列。這比chaining的一個槽均勻更強；linear/quadratic/double hashing都不能直接當成滿足此理想模型。

無tombstone、$n<m$，令X是失敗search到第一EMPTY的探測數。前$i-1$個槽都滿的機率為

$\Pr(X \ge i)=\prod_{j=0}^{i-2}(n-j)/(m-j) \le (n/m)^{i-1}=\alpha^{i-1}$，$i \le n+1$；更大的尾機率為0。

因X非負整數，$X=\sum_{i \ge 1} I[X \ge i]$，取期望得$E[X]=\sum \Pr(X \ge i) \le \sum_{i \ge 1} \alpha^{i-1}=1/(1-\alpha)$。

這是without-replacement的條件乘積，不是假設連續probe獨立。新key插入遇首EMPTY也用相同探測成本。例m=10,n=8的界為5，但不是每個query恰花5次，也不是實際linear probing的保證。

成功搜尋界必須平均插入時負載

先固定n筆相異key的插入序列，不刪除；成功query均勻選一筆。第i+1筆插入時只有i筆在表，期望probe≤m/(m−i)。後來不移動該key，也不刪除之前probe遇到的佔用槽，所以搜尋重現當時路徑。

E[成功probe] ≤ (1/n)Σ_(i=0)^(n−1) m/(m−i) = (1/α)Σ_(k=m−n+1)^m1/k

≤(1/α)∫_(m−n)^m dx/x = (1/α)ln(1/(1−α))。

積分界成立因1/x遞減，區間[k−1,k]積分≥1/k。α=0沒有成功query，不能把1/α直接代0；當α→0，右式極限為1。α→1時兩種期望界都發散，所以工程上需保留容量，但具體閾值要連同實作與成本模型決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

11.5 Perfect hashing

Static dictionary的key集合建好後固定。FKS兩層配置讓完成建表後，每個query的最壞memory access為常數，同時使用O(n)空間；隨機重試發生在建表，不能把建表也稱最壞O(n)。

空集合直接回不存在，不做mod0。以下n≥1，第一層m=n槽，從universal族抽h，bucket j含n_j筆。令Y為無序碰撞key對數，有恆等式Σn_j²=n+2Y。每對碰撞機率≤1/n，故

E[Σn_j²] ≤ n+2·binom(n,2)/n = 2n−1。

Markov給Pr(Σn_j²≥4n)<1/2，因此重試直到Σn_j²<4n，期望少於2輪，每輪O(n)分桶。這裡不是僅說「期望空間O(n)」就宣稱某次建表已具確定線性大小；接受條件必須真的檢查。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

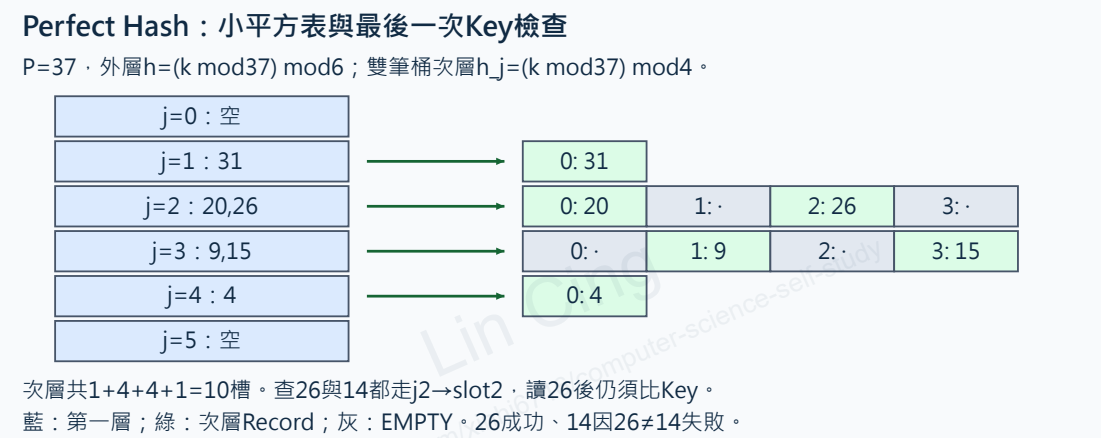
次層平方表為何很快找到無碰撞函數

每個有s=n_j≥2筆的桶配置s^2槽，另從universal族獨立抽h_j。碰撞對數Z滿足E[Z]≤binom(s,2)/s^2=(s−1)/(2s)<1/2，所以Pr(Z≥1)≤E[Z]<1/2。真檢查無碰撞才接受，期望少於2輪。

每次試驗若配置/清空s^2槽，成本O(s^2)；接受的第一層保證Σs^2<4n，故全部次層的期望總建表O(n)，連同第一層仍為O(n)。兩種隨機性分別對第一層重試與條件於已接受分桶的次層重試計，不能只看某個桶的平均就省掉總成本。

空桶不配置次層，單筆桶可直接存record，不需做mod0。Lookup先計j=h(k)，若非空再計位置h_j(k)，讀槽後仍比完整key。對不在集合的query，可能落到已占用槽，不能因兩層無碰撞就直接回答存在。

兩層表的實際配置與失敗Query



原創K={4,9,15,20,26,31}，P=37，第一層h(k)=(k mod37) mod6，桶大小[0,1,2,2,1,0]，平方槽數0+1+4+4+1+0=10<24。兩個雙筆桶用h_j(k)=(k mod37) mod4，分別[20,EMPTY,26,EMPTY]、[EMPTY,9,EMPTY,15]，無碰撞。

兩層表的實際配置與失敗Query （續）

查26：第一層2→次層2→讀26，相等所以成功。查14也走2→2，卻讀到26≠14，必回不存在。一次固定查詢只做常數個word操作和memory access，這是建表成功後的最壞保證；增加新key或更改hash參數可能破壞無碰撞性，須另重建/驗證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 11。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12 二元搜尋樹

Part III Data Structures

本章定位

BST把全域鍵序轉成根到葉的一條查詢路徑，但普通BST不自動平衡。本章用同一棵十鍵樹對齊搜尋、後繼與雙子刪除的所有parent更新，再直接證明隨機插入的高度矩界，區分完整走訪、平均depth與height。

前置知識

遞迴、鏈結節點與隨機排列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立並查詢 binary search tree。
- 實作插入與三種刪除情境。
- 分析隨機建樹的期望高度。
- 12.1 What is a binary search tree?
- 12.2 Querying a binary search tree
- 12.3 Insertion and deletion
- 12.4 Randomly built binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.1 What is a binary search tree?

每節點含key、left、right、parent。本章主要演算法採相異key：x左子樹所有鍵<x.key，右子樹所有鍵>x.key。根parent=NIL，缺child為NIL，空樹root=NIL；必須是無環且每個非根恰一個parent的連通樹。

條件是整棵子樹的全域限制，不是只比直接child。例如根5、左子3、3的右子6，雖3<5且6>3，6仍違反「根的左子樹全部<5」。正確驗證可遞迴傳下允許區間(lo,hi)，左側改hi=x.key、右側改lo=x.key，Θ(n)檢查所有節點，不需每節點重掃子樹。

若要容重複key，可每distinct key存一個count，或以(key,identity)作全序。後者的相鄰identity可能同key，successor就不是「嚴格較大的key」。以下數例皆相異，隨機高度分析也明列此條件。

中序有序性與完整走訪成本

```
inorder(x):
    if x==NIL: return
    inorder(x.left)
    output x.key
    inorder(x.right)
```

歸納：左右子樹的中序各已排序，左全部<x.key<右全部，串成左、根、右即排序。反向看，若一棵合法binary tree的所有key相異且中序嚴格遞增，左右在中序中分別位於根前後，故也滿足全域BST性質。

n個實節點各呼叫左右child一次，所以總呼叫含NIL正好2n+1。每次固定工作O(1)，給Θ(n+1)，非空樹慣寫Θ(n)，不是O(h)；每次輸出一筆也提供Ω(n)下界。遞迴stack最深h+1，鏈狀可Θ(n)。以下統一邊高：單節點h=0、空樹h=−1；非空樹路徑操作寫O(h+1)，空樹查詢與首插另為O(1)。

Heap只要求父子間大小，沒有左整棵<根<右整棵的關係，不能用同一個inorder就得到排序。

12.2 Querying a binary search tree

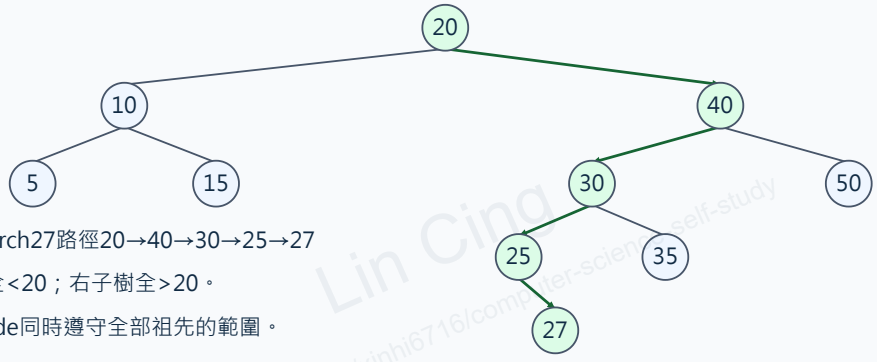
Search從root比較：相等回node；小於走left、大於走right，NIL回不存在。迴圈不變量是「若答案存在，必在當前x子樹」；全域BST界保證丟棄另一側不會漏答案。每步下移一層，時間O(h+1)、迭代額外空間O(1)。

Minimum在非空子樹一路走left，直到無left：若有left，左側有更小鍵；若無left，root不大於其右側全部，故可回傳。Maximum左右互換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.2 Querying a binary search tree （續）

BST查詢：路徑來自全域子樹範圍



綠：Search27路徑20→40→30→25→27

左子樹全<20；右子樹全>20。

每個Node同時遵守全部祖先的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

此例search27走20→40→30→25→27；search26到27後向left遇NIL，回不存在。Minimum=5，maximum=50。一次query沿路徑，完整inorder則為5,10,15,20,25,27,30,35,40,50，兩種成本不能混淆。

Successor的兩分支與停止理由

若x.right非空，successor=min(x.right)，因右子樹全部>x，而其最小比任何更右候選更接近。若right空，沿parent上走，持續略過「目前節點是parent.right」的邊，第一次由left上到的parent就是答案；到NIL則沒有後繼。

```
successor(x):
    if x.right!=NIL: return minimum(x.right)
    y=x.parent
    while y!=NIL and x==y.right:
        x=y; y=y.parent
    return y
```

上溯略過的parent都<x的原key，不能當successor。首次由left進入的y包含原x於其左子樹，y較大；其左子樹中原x之後沒有尚未排除的候選（原x的right空，沿途都是右側最後分支），故y最小。Predecessor把left/right對換。

例successor20走右子40的left鏈30→25，得到25。Successor27上到25時來自right，略過25；再上30時來自left，得到30。Successor50一路略過40、20，到NIL。無須重新從root按key搜尋，也沒有重新排序。

連續Successor不能逐次粗乘Height

從minimum開始列全部n鍵，可反覆successor而總時間Θ(n)。沿完整中序過程，每條child邊至多下降一次、上升一次，加n次輸出就是O(n)；即使某一次successor走Θ(h)，昂貴路徑也不能在每個node都重新付一次。

任意起點連做k次successor（找到NIL便停止），分別數上行U與下行D。每次successor至多一條「由left child上到parent」的邊，該parent就是本次輸出的答案，共至多k條。其餘上行都從right child到parent；該parent在進其right前已輸出，而且一旦上行越過它，之後不再回到其子樹，所以每個parent最多付一次。

這些已輸出的parent，若是在本次k個答案之前就輸出，必位於起始node的祖先鏈，至多h+1個；其他至多k個。因此U≤2k+h+O(1)。每次下行depth加1、上行減1，整段的D−U等於終點depth減起點depth，至多h+O(1)，故D≤U+h+O(1)。總路徑工作D+U加k次固定開銷為O(k+h+1)，不需假定每次上行或下行都常數。

這個較細界不影響單次O(h+1)，但解釋了為何完整走訪不是O(nh)。若要自行重建每次successor都從root按key搜尋，那是另一種演算法，不能沿用上述共用邊的帳。

12.3 Insertion and deletion

Insert沿search比較路徑尋找最後NIL，保留其parent p；新節點z初始化left=right=NIL、parent=p，再依比較接p.left或p.right。空樹p=NIL則root=z。

沿路徑累積的(lo,hi)限制正是z.key可放的區間，走到NIL表示沒有同key，接葉後不影響其他鍵；新葉滿足所有祖先界，故BST性質保留。若相等則依dictionary政策更新既有record，不再接新node。

例先前樹中插26，路徑20→40→30→25→27，最後27.left=NIL，令27.left=z26、z26.parent=27。新鍵同時滿足25<26<27，不能只因26>25就任意接在25右側而覆蓋舊子樹27。

最壞O(h+1)，但以1,2,⋯,n插入空樹會形成右鏈，總比較0+1+⋯+(n−1)=Θ(n^2)。中序再O(n)不會把這個建樹排序變成O(n log n)。

Transplant只替換入口連結

```
transplant(T,u,v):
    p=u.parent
    if p==NIL: T.root=v
    elif u==p.left: p.left=v
    else: p.right=v
    if v!=NIL: v.parent=p
```

Transplant讓v接管u在原parent的位置，成本O(1)，但不會自動設定v.left/v.right，也不會自動刪除u的satellite data。呼叫者必須證明v子樹的key都符合原入口的全域區間；此工具不能拿任意子樹亂換。

Delete z若無left，transplant(z,z.right)，right可NIL，涵蓋leaf與只有右child；若無right，transplant(z,z.left)。原剩餘子樹全部本已在z的合法祖先區間內，所以提升後仍合法，parent同步，根刪除也由同一個工具處理。

雙子Delete的完整指標規格

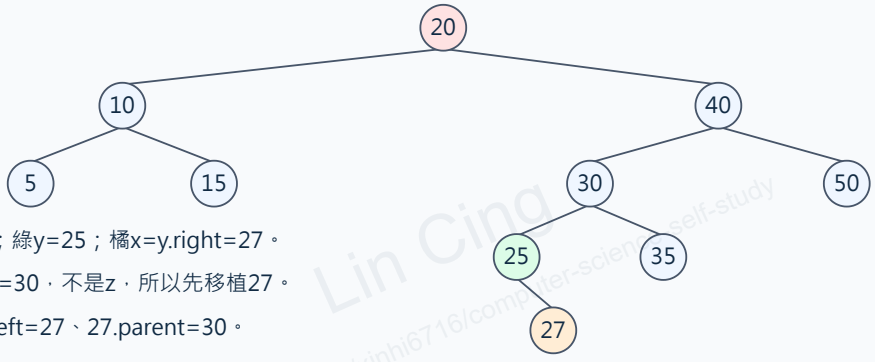
```
delete(T,z):
    if z.left==NIL: transplant(T,z,z.right)
    elif z.right==NIL: transplant(T,z,z.left)
    else:
        y=minimum(z.right)
        if y.parent!=z:
            transplant(T,y,y.right)
            y.right=z.right
            y.right.parent=y
        transplant(T,z,y)
        y.left=z.left
        y.left.parent=y
```

y是右子樹最小，所以無left；它可能有right，不能直接遺棄。若y不是z.right，先讓y.right接y的舊位，再讓y承接原右子樹。若y就是z.right，必須跳過這段，否則y.right=z.right會把y接到自己形成環；原y.right應原封保留。

最後y接z的原父位置（或成root），承接原left。新left所有鍵<z<y；新right是移除y後的原右子樹，全部>y。y.key本屬z的舊子樹，也符合z的全部祖先界，故全域BST性質、可達節點集合與parent方向皆正確。

後繼有Right child時不能丟失它

刪除前：後繼25還有Right child27



紅z=20；綠y=25；橘x=y.right=27。

y.parent=30，不是z，所以先移植27。

先設30.left=27、27.parent=30。

後繼有Right child時不能丟失它 （續）

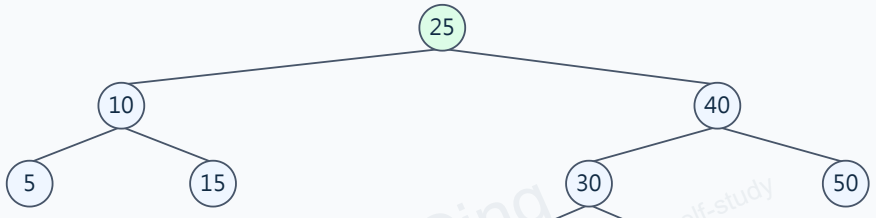
此例z=20，y=25不是直接right40，而是30.left；y.right=27。依序執行：

階段	真正修改的連結
取代y舊位	30.left=27，27.parent=30
接原right	25.right=40，40.parent=25
接原root	root=25，25.parent=NIL
接原left	25.left=10，10.parent=25

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼有Right child時不能丟失它 （續）

刪除後：新Root與後繼舊位都接好



綠25接管Root，parent=NIL。

橘27接到30.left，parent=30。

只移除20，其他9個Identity都保留。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成後只移除node20，其餘9個identity都仍可達，中序為5,10,15,25,27,30,35,40,50。這是移動node25，沒有把「複製key再刪node」的另一種handle語義混入。

刪除後的Handle與成本

傳入z必須是此樹目前有效node；delete完成後，外部不得繼續把z當作樹成員。其他node identity保持，特別是原y25只是移位置，其handle仍指同一個record。若改採複製y的key/value到z再刪y，外部handle的觀察結果會不同，需另外定義。

Delete除minimum路徑O(h+1)外只作固定次數transplant/指標更新，故O(h+1)；若從key起始先search，仍同階，但不是O(1)。搜尋、插入、刪除的好成本取決於height，普通BST沒有自動平衡保證。

若固定資料直接由已排序陣列遞迴取中點建平衡樹，可Θ(n)建好；任意未排序資料若透過比較先決定完整BST再中序輸出，就能解比較排序，故其一般最壞建樹成本至少Ω(n log n)。兩個輸入契約不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.4 Randomly built binary search trees

模型：n個固定相異key，由空樹開始，n!種插入排列等機率，只插入不刪除。它不同於「所有合法BST形狀等機率」。最先插入的key成root，故root秩R均勻1..n。

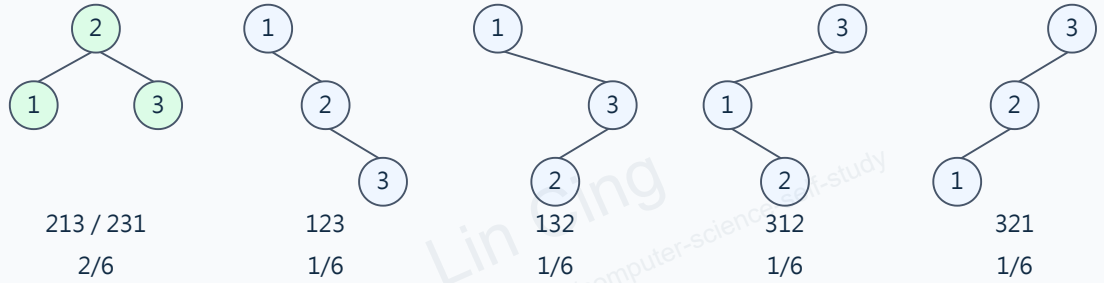
條件於R=r，左子樹含r−1個較小key，右子樹含n−r個較大key。任意指定左右內部相對排列，都有binom(n−1,r−1)種交錯方式，因此條件於root後，兩側的相對插入排列各均勻且彼此獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

12.4 Randomly built binary search trees （續）

均勻插入排列如何對Shape加權

六種插入排列均勻，五種Shape不是均勻。單Node邊高0。



綠形狀Height1，其他Height2：E[H3]=(2×1+4×2)/6=5/3。

n=3時root2的平衡形狀由[2,1,3]與[2,3,1]兩種排列產生，機率2/6；其餘四個鏈形狀各1/6。Height為1或2，$E[H_3]=(2\times1+4\times2)/6=5/3$，不是五種形狀均勻所得9/5。

先控制Exponential height

固定某一node的期望depth為O(log n)，不能直接推出全樹最大depth的期望也O(log n)。我們改直接控制height的指數。

令H_n為邊高，$H_0=-1$，Z_n=2^(H_n)，所以$Z_0=1/2$、$Z_1=1$。這和原書為方便另定$Z_0=0$的慣例不同；本章保持Z=2^H在空樹也成立，後面使用相應較寬常數。

給定root的左側大小i，height滿足H_n=1+max(H_i,H_(n−1−i))，因此`Z_n=2 max(Z_i,Z_(n−1−i))≤2(Z_i+Z_(n−1−i))`。

設a_n=E[Z_n]。條件於root後兩子樹仍按其規模均勻建成，平均各個i得到

`a_n ≤ (2/n)Σ_(i=0)^(n−1)(a_i+a_(n−1−i)) = (4/n)Σ_(i=0)^(n−1)a_i`。

此上界只在max≤sum後用期望線性性，不需要誤寫E[max(X,Y)]=max(E[X],E[Y])。

多項式Moment界與完整高度推導

取b_n=(1/2)binom(n+3,3)，$b_0=1/2=a_0$。由hockey-stick恆等式
Σ_(i=0)^(n−1)binom(i+3,3)=binom(n+3,4)，得

(4/n)Σ_(i=0)^(n−1)b_i = (2/n)binom(n+3,4) = (1/2)binom(n+3,3)=b_n。

恆等式可用Pascal式binom(i+4,4)−binom(i+3,4)=binom(i+3,3)望遠鏡求和。歸納a_i≤b_i即推出a_n≤b_n=O(n^3)，含n=1基例$a_1=1\le b_1=2$。

$\log_2$是凹函數，Jensen給$E[H_n]=E[\log_2 Z_n]\le\log_2 E[Z_n]\le\log_2 b_n=O(\log n)$。另一方面，n節點高度h最多容2^(h+1)−1筆，故每棵樹$h\ge\log_2(n+1)-1$，n增大時也給Ω(log n)。合為E[H_n]=Θ(log n)。

尾機率、適用範圍與平衡保證

對c>3，由Z_n≥0與Markov：

$Pr(H_n\ge c\log_2 n)=Pr(Z_n\ge n^c)\le E[Z_n]/n^c\le b_n/n^c=O(n^{(3-c)})$。

c=4給O(1/n)尾界；要失敗機率O(n^(−k))可取c≥k+3。這是隨機排列模型下的高機率界，不是普通BST對每個輸入都有固定height≤4log n。

按1..n輸入仍確定形成n−1高鏈；刪除後再插入也未必保留這個隨機建樹分布，不能把只插入的定理當成任意動態序列保證。若需要每次最壞O(log n)，應使用有明確不變量的平衡樹，例如下一章red-black tree，而不是把資料「大概隨機」當成證明。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 12。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13 紅黑樹

Part III Data Structures

本章定位

紅黑樹用黑高與局部色限制保證對數高度；本章由NIL與精確父子接線，逐案證明插入三類、刪除四類及其鏡射，以相同鍵值前後圖核對額外黑、根分支與record身分。

前置知識

二元搜尋樹、旋轉與不變量證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以紅黑性質界定近似平衡 BST。
- 推導旋轉前後的中序不變性。
- 追蹤插入與刪除修正並驗證黑高。
- 13.1 Properties of red-black trees
- 13.2 Rotations
- 13.3 Insertion
- 13.4 Deletion

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先認識圖中的角色與路徑計數

深色節點是黑 B，淡紅節點是紅 R；鍵值後的 B／R 同時標出顏色。橘框標記目前修正位置，不代表第三種節點色。旋轉圖只談連結，不指定節點顏色。

情境	圖中的角色
旋轉	x：舊局部根；y：右 child；p：x 的舊父；α、β、γ：保留中序區間的子樹
插入	z：目前紅節點；p：父；g：祖父；u：叔
刪除	x：缺黑位置；p：父；w：兄弟；near：靠 x 的姪；far：遠離 x 的姪

x +1 只在證明中多算一黑。圖底的完整路徑黑數**含起點與末端 NIL**；$bh(x)$ 則不含 x。未畫的空 child 仍是黑 NIL；near／far 若為 NIL，會明畫。

13.1 Properties of red-black trees

BST 之外多一個 color，保證的是高度

每個內部節點存key、data、left、right、parent、color。以下示範互異key；重複插入回傳既有record。若要保留重複key，可改以(key,唯一ID)作嚴格總序，以下BST論證不變。

紅黑性質為：(1)每節點紅或黑；(2)根黑；(3)每個外部NIL葉黑；(4)紅節點的兩child皆黑；(5)任一點到其後代NIL的所有路徑黑節點數相同。最後一項是局部每個子樹都要成立，不只是整樹某兩條路相等。

用單一黑sentinel NIL表示全部空child及根的parent；空樹root=NIL。NIL沒有有效key/data，NIL.left=NIL.right=NIL便於安全讀color。正常節點非NIL的child必指回正確parent。所有NIL參照共用物件，不能同時宣稱每個空位置都有自己獨立的NIL.parent。

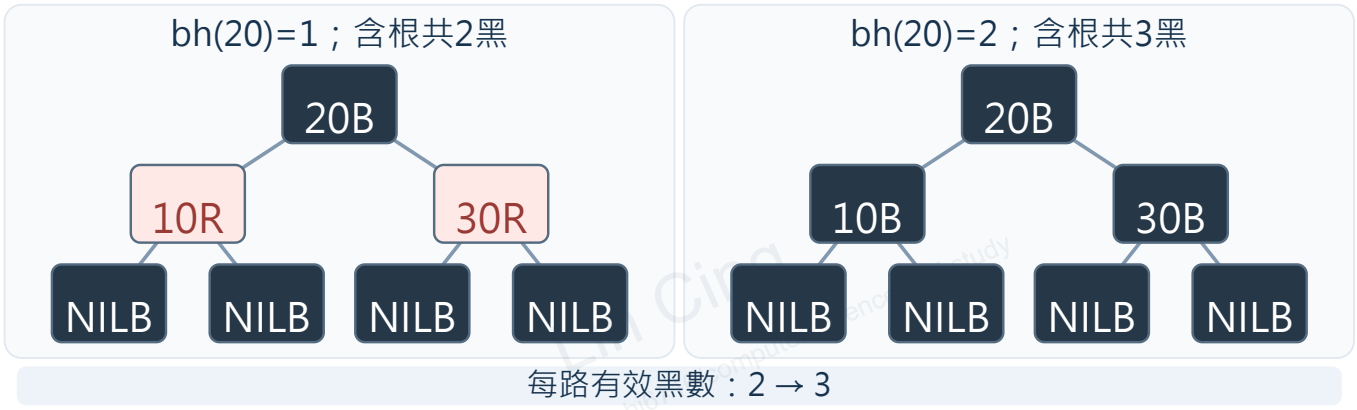
黑高定義：不含自己，但含末端 NIL

$bh(x)$為從x到任一後代NIL的黑點數，**不含x、含末端NIL**；$bh(NIL) = 0$。若child紅，$bh(child) = bh(x)$；若child黑，$bh(child) = bh(x) - 1$。這是依child是否被計入父的路徑決定，不可顛倒。

根20B、左右child10R及30R、其餘全NIL，則bh(20)=1、bh(10)=bh(30)=1，NIL為0。將兩child都改黑，則bh(20)=2，而bh(10)=bh(30)=1。每條路含根的黑數由2變3，但bh仍各排除其起點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

黑高定義：不含自己，但含末端 NIL （續）



本章h計root到NIL的最長邊數，空樹h=0；只有一個黑根時h=1。若採只算內部節點邊數的高度，非空時少1，不應在同一式中交換定義。

最小節點數與高度界的完整歸納

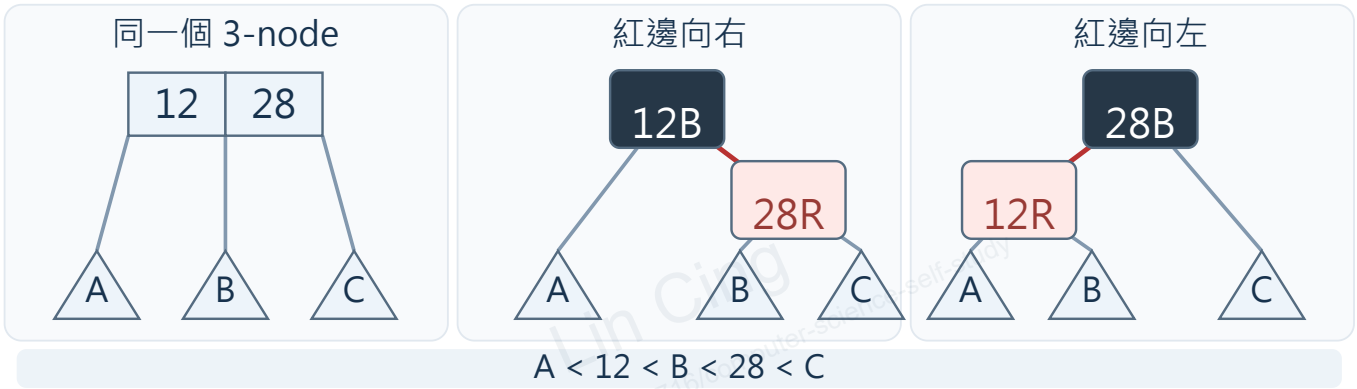
引理：以x為根的子樹至少$2^{bh(x)}-1$個內部節點。對高度歸納：x=NIL時0=2⁰−1；內部x的兩child高度較小，且黑高至少bh(x)−1，因此各至少$2^{bh(x)-1}-1$個內部節點（bh(x)≥1）。加上x得到

$$n_x \geq 2(2^{bh(x)-1}-1)+1 = 2^{bh(x)}-1.$$

取一條最長root-NIL路，含B個黑點與R個紅點。根與末端NIL黑，無相鄰紅；每個紅點可配給前面不同的黑點，故R≤B−1。路有h+1點，h+1=B+R≤2B−1，得h≤2(B−1)=2bh(root)。與節點引理合併，$n \geq 2^{h/2}-1$，故$h \leq 2\log_2(n+1)$。空樹亦滿足0≤0。

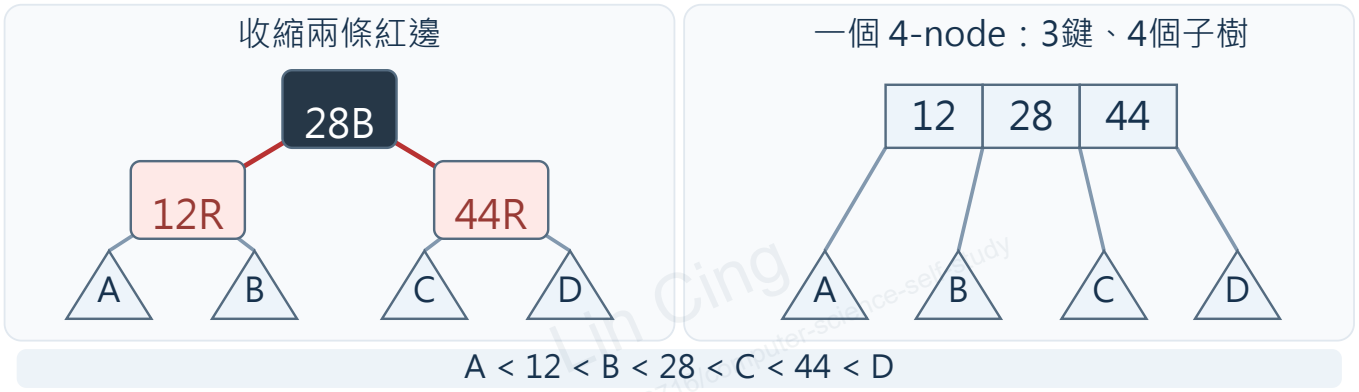
因此search/min/max/successor/predecessor走一條路需O(log(n+1))；插入、刪除還須證修正成本。輸出所有n點的中序、前序、後序走訪仍Θ(n)。紅黑樹也不要求左右高度差≤1，不能當作AVL的定義。

同一個 3-node，為何有兩種紅黑表示？



把紅 child（子節點）與黑 parent（父節點）合併，即為一個多鍵節點；單獨黑點則為 2-node。A、B、C 是黑根子樹（可為 NIL），黑高相同；不等式適用於各子樹內所有鍵。兩種表示中序相同，**一般紅黑樹不要求紅邊左傾**。

收縮紅邊後，黑高相等對應多元樹的等深葉



不允許連續紅邊，所以一個黑點至多與兩個紅 child 合併，得到 2／3／4-node；各子樹區間不變。每條 root–NIL 路扣除末端 NIL 一黑後，黑點數等於收縮後的多鍵節點數；各路黑數相等，外部葉便等深。紅邊採用 child 的色；根另外約定為黑。

13.2 Rotations

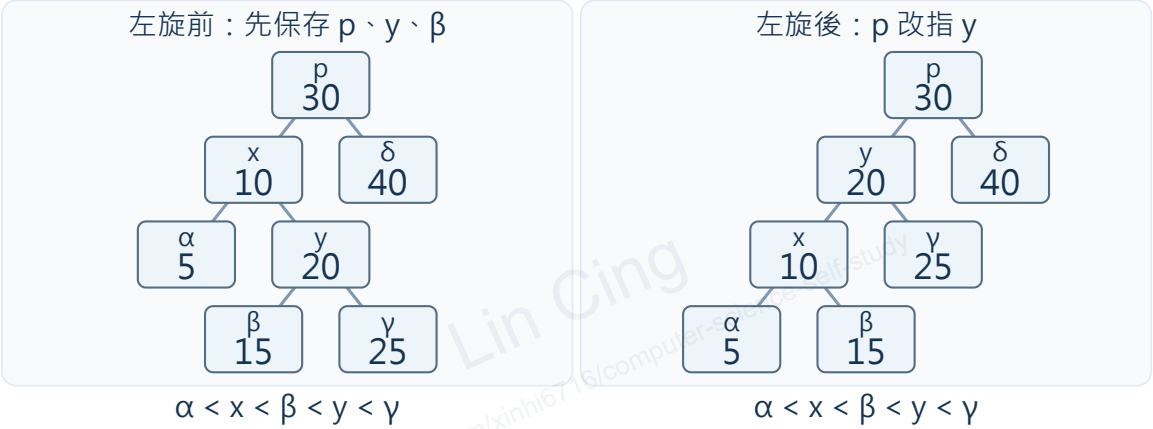
左旋先保存舊父，才接新局部根

前提x≠NIL、y=x.right≠NIL；令p=x.parent、β=y.left。α=x.left、γ=y.right的內部連結不動。

```
LEFT-ROTATE(T,x):
    p=x.parent; y=x.right; beta=y.left
    x.right=beta
    若 beta!=NIL: beta.parent=x
    y.parent=p
    若 p==NIL: T.root=y
    否則若 x==p.left: p.left=y
    否則: p.right=y
    y.left=x
    x.parent=y
```

必須更新**x的舊父p指向y**；y的舊父就是x，不可把這兩者混淆。β=NIL時不寫其parent，避免覆蓋刪除修正正在使用的共享sentinel上下文。旋轉本身不變更任何color、key或data。

13.2 Rotations （續）



指標表與右旋完整鏡射

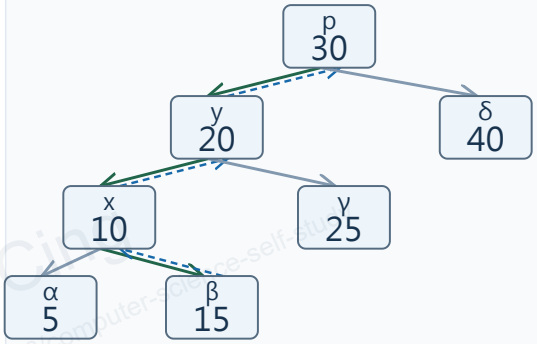
例p30、x10、y20、β15，x是p.left。順序寫10.right=15、15.parent=10、20.parent=30、30.left=20、20.left=10、10.parent=20。若p=NIL，則取代30.left的寫入是root=20，並令20.parent=NIL。

```
RIGHT-ROTATE(T,y):
    p=y.parent; x=y.left; beta=x.right
    y.left=beta
    若 beta!=NIL: beta.parent=y
    x.parent=p
    若 p==NIL: T.root=x
    否則若 y==p.right: p.right=x
    否則: p.left=x
    x.right=y
    y.parent=x
```

指標表與右旋完整鏡射 （續）

1. 10.right = 15
2. 15.parent = 10
3. 20.parent = 30
4. 30.left = 20
5. 20.left = 10
6. 10.parent = 20

實線 ↓ : child
虛線 ↑ : parent



若 p=NIL：步驟4改 root=20；步驟3為20.parent=NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何旋轉保序，為何不能單憑旋轉宣稱紅黑合法

左旋前後中序皆α,x,β,y,γ：α的鍵<x，β在x與y之間，γ>y；β改為x.right仍合法。根外只把p原指x的同一側改成y，其餘範圍不動。非NIL的β.parent、x.parent、y.parent分別改x、y、p，所以內外結構一致。右旋完全對偶。

只寫常數個欄位，時間O(1)。搜尋修復位置仍需O(log n)，不因單次旋轉常數就變O(1)插刪。旋轉保BST與record身分，**不自動保黑高或紅紅規則**；下一節的重上色必須配合局部黑數證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.3 Insertion

紅葉插入只可能破壞根黑或父子紅紅

以普通BST搜尋空位置並建立z，z.left=z.right=NIL、z.color=R，再接parent。紅葉取代原黑NIL時，原路末端仍有同一個黑NIL，故不改所有相關黑數；若先插黑色，只有經此位置的路多一黑，會破壞性質5。

若空樹插第一點，它紅且為root，只需最後塗黑；否則僅可能z與紅父p相鄰。修正迴圈不變量為：z紅；若z.parent是root，該parent必黑；最多只有根紅或z/parent紅紅其中一種違規；其餘色性質與BST序均成立。進入迴圈時p紅，所以p不可能是根，祖父g存在，且g黑（否則另有一組紅紅）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入修正完整流程

以下p是g.left；另一半把left/right及LEFT/RIGHT-ROTATE全部交換。

```
當 z.parent.color==R:
    若 z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        若 u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        否則:
            若 z==z.parent.right:               // I2，接著做I3
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    否則: 執行完全左右鏡射的同三分支
T.root.color=B
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

插入修正完整流程 （續）

I2與I3不是互斥if；I2轉換後必落入I3。同時要用更新後的z.parent與祖父，不能再拿旋轉前的p當修復對象。

p是g.right時	右側的明確動作
I1：叔g.left紅	p、叔黑；g紅；z=g
I2：叔黑且z=p.left	z=p；RIGHT-ROTATE(z)，接I3
I3：叔黑且外側	新z.parent黑、祖父紅；LEFT-ROTATE(祖父)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1：叔紅重上色的黑數不變

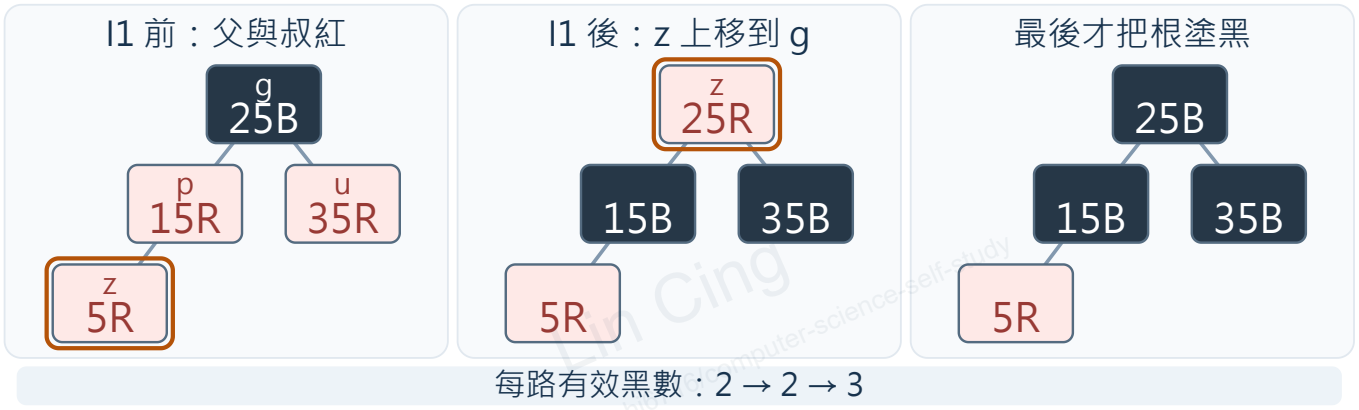
令父p與叔u以下每條路各提供b個黑點，包含末端NIL。原g黑、p/u紅，經g的每條完整路黑數為1+b；改g紅、p/u黑後仍1+b。雖然不含g的bh(g)由b變b+1，**祖先穿過g的完整黑數沒有增加**。

新z設g，紅紅只可能上移至g及其父；原z-p已因p黑消除。若g變成根，暫時只根紅，最後根黑才使全樹每條路同增一黑，不破相等性。

原創例依序插25、15、35、5：新5R的父15R、叔35R、祖25B。I1把15/35黑、25紅，z移25；parent為NIL黑，結束後root25黑。非根g時不得套用這最後同增一黑的根分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I1：叔紅重上色的黑數不變 （續）



I2：把內側折線轉成外側，不跳過下一步

叔黑且p=g.left、z=p.right時，先令z=p，再左旋舊p。原p與新節點皆紅，旋轉未改任何路上的黑點貢獻；BST中序保持。旋轉後舊p變成新局部根的左紅child，形成外側鏈，仍須I3才能消除紅紅。

插35、15、25：插25後root35B、左15R、15.right=25R。左旋15後root仍35B，35.left=25R，25.left=15R，算法z現在指15。紅紅由15/25保留，不能此時返回。

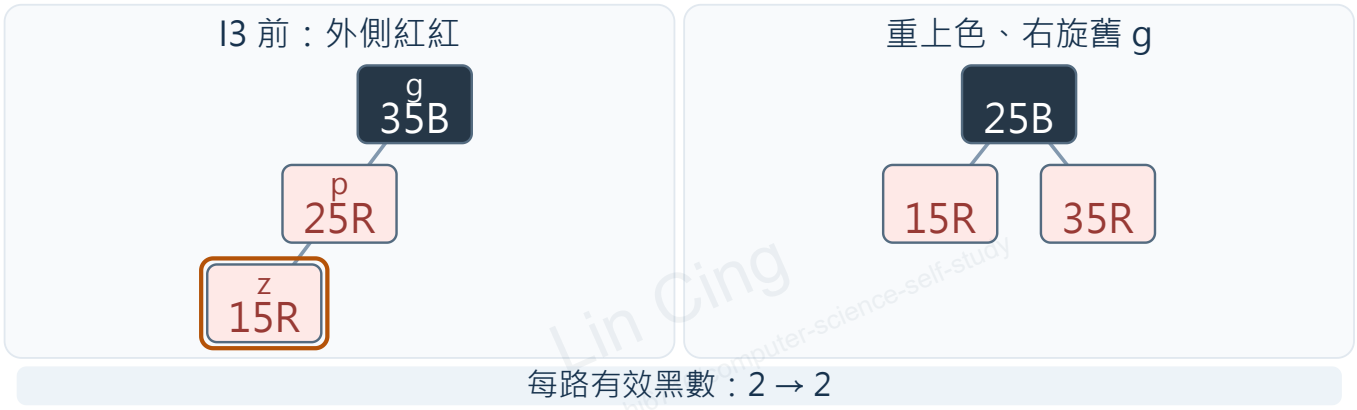
I3：重上色加旋轉，恢復全部局部路徑

外側鏈gB-left pR-left zR，叔黑。將p黑、g紅，右旋g，使p成局部根。設α、β為z兩child，γ為p原right，δ為g原right；四棵邊界子樹均黑根（可為NIL），各完整向下路提供b黑點。原經g的完整路為1+b；改後經新p亦1+b。z仍紅，但父p黑；g紅的兩childγ、δ皆黑，故沒有紅紅，局部根p黑也不會與外父衝突。

續前例：25塗黑、35塗紅，再右旋35，得到25B、左15R、右35R。中序15,25,35保留，bh(root)=1，每條root-NIL路含2黑。鏡射例插15、35、25，先右旋35，再左旋15，同得這棵樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I3：重上色加旋轉，恢復全部局部路徑 （續）



I1每次z上移兩層且不旋轉；I2+I3做至多兩次旋轉後必結束。故重上色O(log(n+1))、旋轉最多2次，連BST插入總O(log(n+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.4 Deletion

先分清 z、y、x 與 y 的原色

z是要刪除的record；若z少於兩child，y=z；若有兩child，y是z.right中的最小點，會整個移到z位置。x是填補**y原位置**的唯一可能child（若y為後繼，取y.right，因後繼沒有left；若y=z，取z唯一非NIL的child，兩child皆NIL時取NIL）。要保存yOriginalColor，再把移來的y.color設z.color；是否缺黑取決於yOriginalColor，不是移動後y的色，也不是只看z.color。

```
RB-TRANSPLANT(u,v):
    若 u.parent==NIL: root=v
    否則若 u==u.parent.left: u.parent.left=v
    否則: u.parent.right=v
    v.parent=u.parent              // 即使 v==NIL 也寫
```

這次NIL.parent保存當前x空位置的parent，供找sibling使用；它不是全部空child的通用父。根parent=NIL。旋轉遇到β=NIL則不寫β.parent，以免覆寫這個x上下文。穩定操作結束後可重設NIL.parent=NIL；color必始終黑。

RB-DELETE 的完整物件移接

```
RB-DELETE(z):                  // z 是已找到的有效record
    y=z; yOriginalColor=y.color
    若 z.left==NIL:
        x=z.right; RB-TRANSPLANT(z,z.right)
    否則若 z.right==NIL:
        x=z.left; RB-TRANSPLANT(z,z.left)
    否則:
        y=MINIMUM(z.right); yOriginalColor=y.color; x=y.right
        若 y.parent==z:
            x.parent=y          // 包含x為NIL的分支
        否則:
            RB-TRANSPLANT(y,y.right)
            y.right=z.right; y.right.parent=y
        RB-TRANSPLANT(z,y)
        y.left=z.left; y.left.parent=y
        y.color=z.color
    若 yOriginalColor==B: RB-DELETE-FIXUP(x)
```

本章以整個y物件移動，y自己的key/data與handle保持，不把z的data錯套給y。API可先SEARCH(key)，找不到直接NOT_FOUND而不修改；上面程序的前提是z存在，不對NIL作普通刪除。

後繼移接的具體位置與 NIL parent

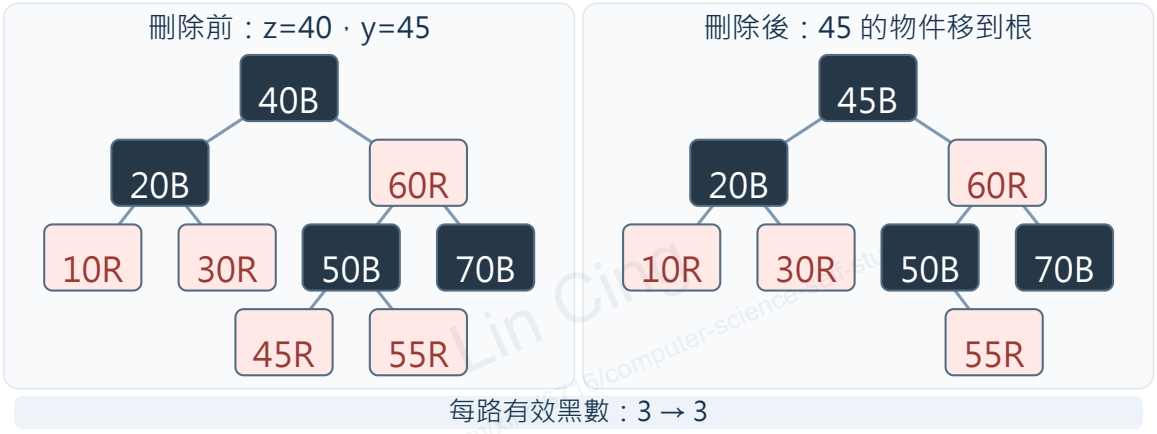
依序插40、20、60、10、30、50、70、45、55，得到root40B，左20B(10R,30R)，右60R，其左50B(45R,55R)、右70B。刪z40時y45R，x=NIL，y.parent50≠z。

先移除50.left的45，令50.left=NIL且NIL.parent=50；y.right接60並改60.parent=45；用y替root40，再接y.left=20、20.parent=45，令y.color=B。Y原色R，所以不修黑；root45B，原45的data仍跟隨45，只有40的record被移除。

若y就是z.right，則不可先把x.parent留在將刪的z，必指定x.parent=y。這兩個分支對非NIL與NIL的x都要成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼移接的具體位置與 NIL parent （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

缺一黑以 x 位置追蹤，不新增第三種 color

若y原紅，移除不減黑數，y在新位置取z原色，其他局部紅黑關係亦保留；無須fix。若y原黑，把它的一份黑暫時計到x位置，令所有經x的路多算一黑，其他路的黑數才與刪前一致。

這是證明用的「extra black」，實際x.color仍只有R/B。X紅時直接塗黑吸收extra；x是root時可把extra從全樹路徑一起去掉；其餘迴圈x為黑且非根。兄弟w不能NIL，否則它只有一個普通NIL黑，而x分支至少有NIL加extra兩黑，與有效黑數相等矛盾。

以下x在p.left，w=p.right；near=w.left、far=w.right。右側缺黑時全部左右交換：near=w.right、far=w.left，旋轉方向也反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DELETE-FIXUP 的完整左側流程

```
當 x!=root 且 x.color==B:
    若 x==x.parent.left:
        w=x.parent.right
        若 w.color==R:                        // D1
            w.color=B; x.parent.color=R
            LEFT-ROTATE(T,x.parent)
            w=x.parent.right
        若 w.left.color==B 且 w.right.color==B: // D2
            w.color=R
            x=x.parent
        否則:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

DELETE-FIXUP 的完整左側流程 （續）

```
            若 w.right.color==B:               // D3，接著做D4
                w.left.color=B; w.color=R
                RIGHT-ROTATE(T,w)
                w=x.parent.right
            w.color=x.parent.color             // D4
            x.parent.color=B; w.right.color=B
            LEFT-ROTATE(T,x.parent)
            x=root
    否則：執行完全左右鏡射的同四分支
x.color=B
```

D1之後重新取w，D3之後也重新取w；分類不是四個互斥elif。單一迭代可能D1→D2，或D1→D3→D4；後兩者與直接D4都會終止。

DELETE-FIXUP 的完整左側流程 （續）

x是p.right時	w=p.left，near=w.right，far=w.left
D1：w紅	w黑、p紅；RIGHT-ROTATE(p)；重取w
D2：w兩child黑	w紅；x=p，上移或最後塗黑吸收
D3：far黑、near紅	w.right黑、w紅；LEFT-ROTATE(w)；重取w
D4：far紅	w取p原色；p與w.left黑；RIGHT-ROTATE(p)；x=root

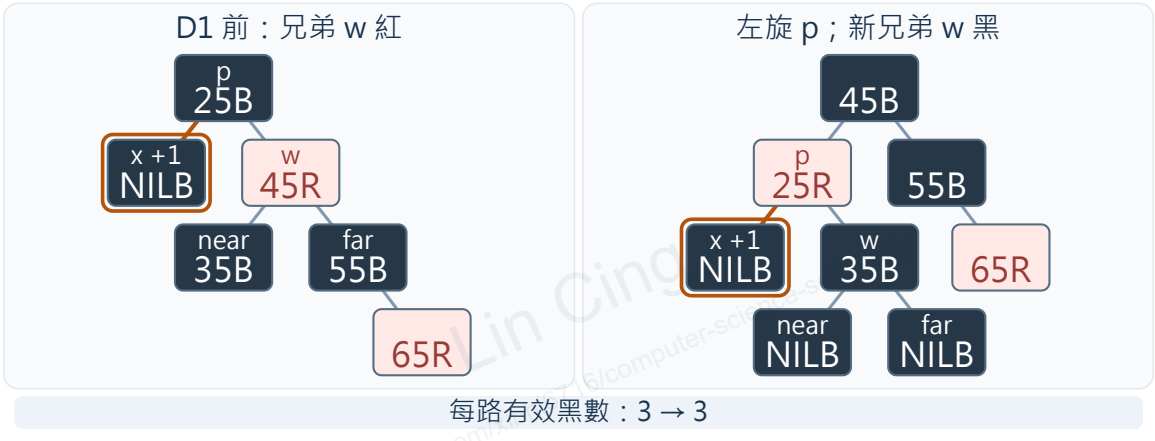
D1：紅兄弟轉成黑兄弟，extra 仍在原位置

共同計數記x分支**實際**完整黑數b，含x（若NIL便含NIL）；加extra後b+1。D1的w紅迫使p及w兩child黑。w每個child的完整黑數為b+1，所以旋轉前從p含自己的有效黑數皆b+2。

令w黑、p紅、左旋p。新局部根w黑，其左p紅下的x有效b+1與w原近child b+1相等，右側w原遠child亦b+1，所以新局部總仍b+2。Extra沒有被消除，x的新兄弟為原w的黑近child；因此只是轉成D2／D3／D4。

數例依序插15、25、35、45、55、65，再刪15。移除後x=NIL是25B.left，w45R；D1將45黑、25紅，左旋25。新root45B，左25R(right35B)、右55B(right65R)，x仍為25.left。根或祖先的完整黑數不因這個轉換改變。

D1：紅兄弟轉成黑兄弟，extra 仍在原位置 （續）



D2：兩個黑 child，缺黑向上或被紅父吸收

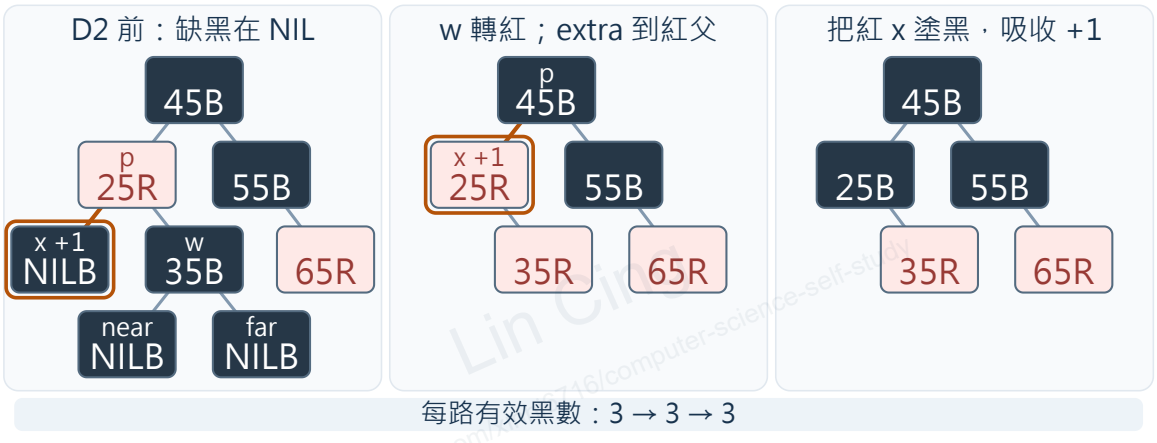
此時w黑且near/far都黑。令x實際黑數b、有效b+1，w完整黑數b+1、每個child完整黑數b。W改紅後其完整黑數變b，兩邊**實際**黑數恢復相等，但整個p子樹相對祖先少一黑，故將extra改附p，即x=p。

若p原紅，最後p塗黑恰補一黑，結束；若p原黑且非root，再迭代上移；若p是root，從所有路一起去extra，結束。D1剛把p變紅，所以D1後若接D2必會在這一輪後吸收，不會再上升到另一次D1。

續前例：w35B兩child NIL，改35R，x由NIL改25R，退出迴圈後25黑。最後45B的左25B(right35R)，右55B(right65R)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D2：兩個黑 child，缺黑向上或被紅父吸收 （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D2 的根分支：整棵樹可同降一黑

若原樹為25B、兩黑葉15B/35B，刪15則D2令35紅、x移root25，從全部根葉路一起去掉extra，最後25B(right35R)。Root的bh從2降1，所有路仍相等；這與紅父塗黑吸收是不同的結束分支。

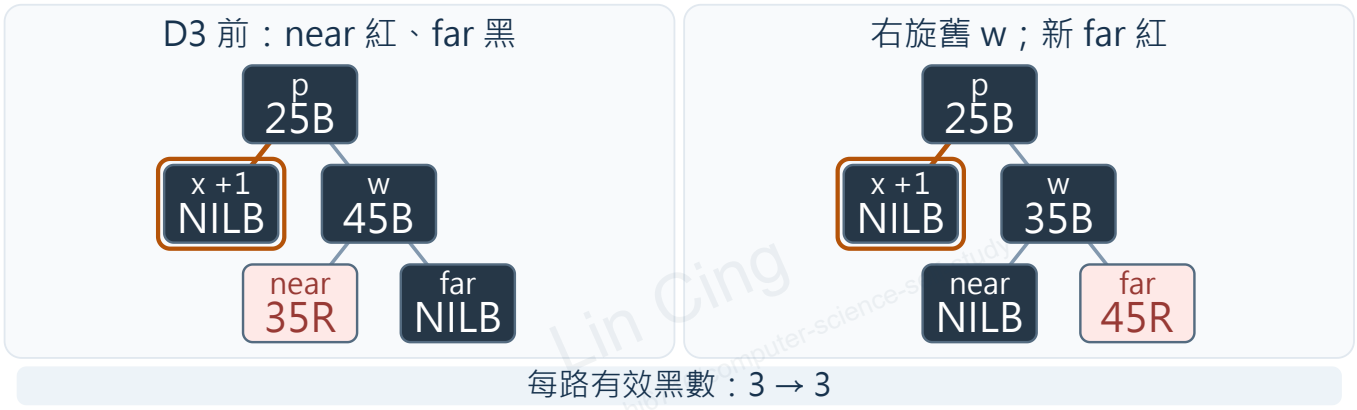
D3：近紅遠黑，先把可用紅色轉到遠端

W黑、near=N紅、far黑。令x實際黑數b；w兩child的完整黑數皆b。N紅的兩child各完整黑數b，far亦b。將N黑、w紅並右旋w：新兄弟N黑，其left舊N.left為b，right舊w紅的兩child舊N.right及far亦b，所以新兄弟完整黑數仍b+1；p以上不變。

新兄弟的far就是舊w，顏色紅，故立即進D4，extra仍在x。此步沒有「上移缺黑」，也未終止。

數例插15、25、45、35後root25B、左15B、右45B(left35R)。刪15後x=NIL，w45B、near35R、far=NIL黑。35黑、45紅，右旋45，得到root25B、右35B(right45R)，接著必D4。

D3：近紅遠黑，先把可用紅色轉到遠端 （續）



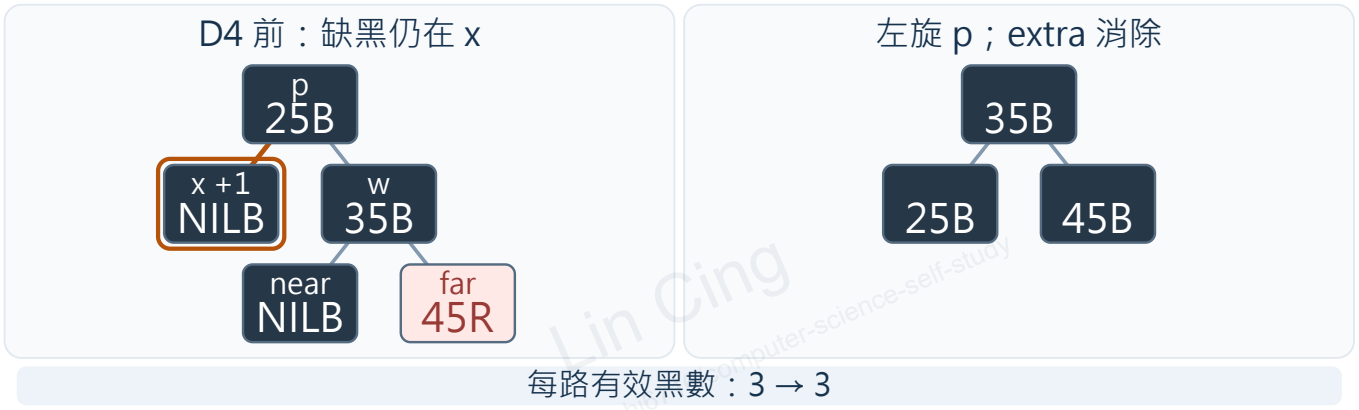
D4：遠紅，旋轉後兩側直接補齊

令p原色的黑貢獻c∈{0,1}，x實際黑數b，w黑的near與far完整黑數皆b；far原紅。令w.color=p原色、p黑、far黑，左旋p。新根w左子p黑，p下的x（已不再加extra）與near皆b，所以左子樹完整黑數b+1；右far由紅轉黑，也為b+1。新局部總c+b+1，等於原p下含extra的有效總c+(b+1)。

若新w紅，其兩child p及far皆黑；原p變黑不會新造紅紅，w對外保留p的原色，所以外父關係亦保留。Extra已被結構與色變消除，令x=root結束，再保證root黑。

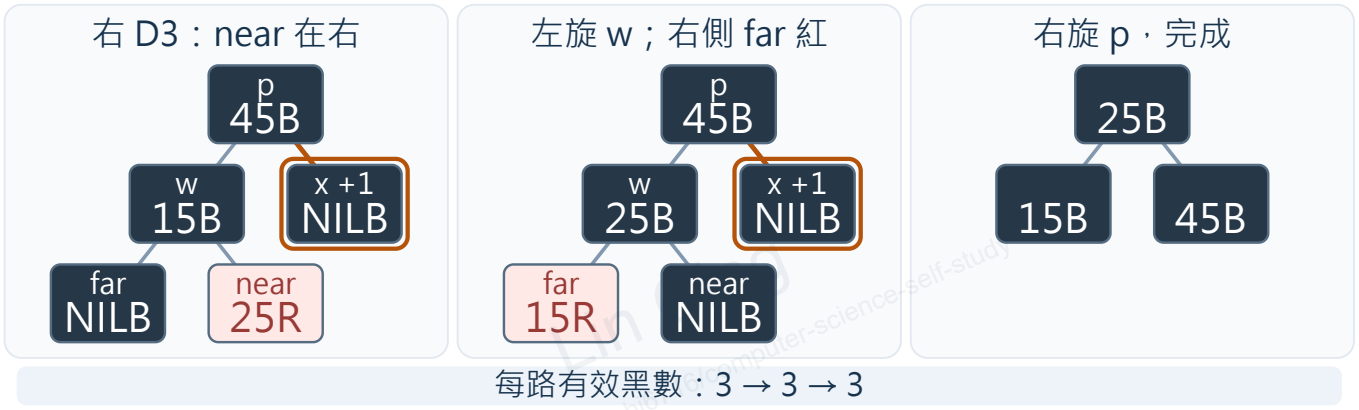
續D3數例：w35繼承25原黑，25黑、far45黑，左旋25，得到35B、兩child25B及45B。三條key為25,35,45，每條root-NIL含3黑，不再有extra。

D4：遠紅，旋轉後兩側直接補齊 （續）



鏡射例插15、35、45、25後刪35：後繼45B直接接新根，x=NIL是45.right，所以NIL.parent必設45。兄弟15B的near25R、far=NIL；右側D3左旋15，右側D4右旋45，最後25B、兩child15B及45B。

D4：遠紅，旋轉後兩側直接補齊 （續）



同一個 D3→D4 例子，逐路把黑數加出來

以下每條路**含起點與 NIL**。D3 不改 x=NIL 的位置；D4 才消除 extra。

階段與路徑	實際黑數	extra	有效總數
D3 前：25B → NIL(x)	2	1	3
D3 前：25B → 45B → 35R → NIL	3	0	3
D3 後：25B → 35B → 45R → NIL	3	0	3
D4 後：35B → 25B → NIL	3	0	3
D4 後：35B → 45B → NIL	3	0	3

黑點各加 1、紅點加 0，只有穿過 x 的路再加 extra。未列出的另一側 NIL 路依同一規則也為 3；可對照前後圖逐一驗算。D4 後無須 extra，兩側已各補齊一黑。

四類守恆與總成本如何接成一般證明

BST移接先保住序與全部未刪record。若移除紅則直接合法；移除黑時，以x位置的extra讓祖先到葉黑數暫時平衡。每輪D1與D3保持同一缺黑位置及有效黑數；D2把唯一extra上移；D4完全消除。紅紅只可能在紅x吸收時，最後塗黑修復；w變紅的D2其兩child本黑，旋轉重上色也已逐案檢查。因此終止時五性質全部恢復。

D2每次沿祖先上移一層，最多O(log(n+1))次且不旋轉。D1之後p紅，若D2便吸收、否則經D3/D4結束；故D1最多一次，D3最多一次，D4最多一次，旋轉總最多3次。加搜尋z、搜尋後繼及移接，刪除總O(log(n+1))。NIL/root/紅child直接吸收為合法基例，刪最後一點得到空黑root=NIL。

這是任意合法紅黑樹的一般證明；有限程式另逐次檢查全域鍵序、雙向parent、紅child黑、所有NIL路黑數、root色及未刪record物件身分，並包含左右各插入3類、刪除4類。重上色可沿高度發生，不能把「最多3次旋轉」寫成整個刪除O(1)。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 13。

Lin Cing，個人資料結構手寫筆記，紅黑樹與2-3-4 tree對照。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14 擴充資料結構

Part III Data Structures

本章定位

擴充資料結構要讓新查詢變快，同時把修改摘要的成本算清楚。本章用size建立動態rank/select，再證明何時能局部更新，並以精確max和閉區間端點推導interval tree只走一條路仍不漏答案。

前置知識

BST、size 欄位、區間與不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以 subtree size 支援動態秩。
- 用固定步驟選擇可靠附加資訊。
- 設計 interval tree 回答重疊查詢。
- 14.1 Dynamic order statistics
- 14.2 How to augment a data structure
- 14.3 Interval trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.1 Dynamic order statistics

第9章在靜態無序陣列找一次第k小需線性時間；若資料持續插入/刪除而反覆查rank/select，可在平衡BST保存子樹摘要。Order-statistic tree在red-black tree每node x加size，定義包含x本身的實節點數，NIL.size=0：

`size(x)=size(x.left)+size(x.right)+1`。

Rank採中序identity位置的1-index；相同key若各存一node，需用(key,identity)定全序，或明確以當前中序順序定義節點rank，不能只問重複值「唯一的rank」。本章數例鍵相異。

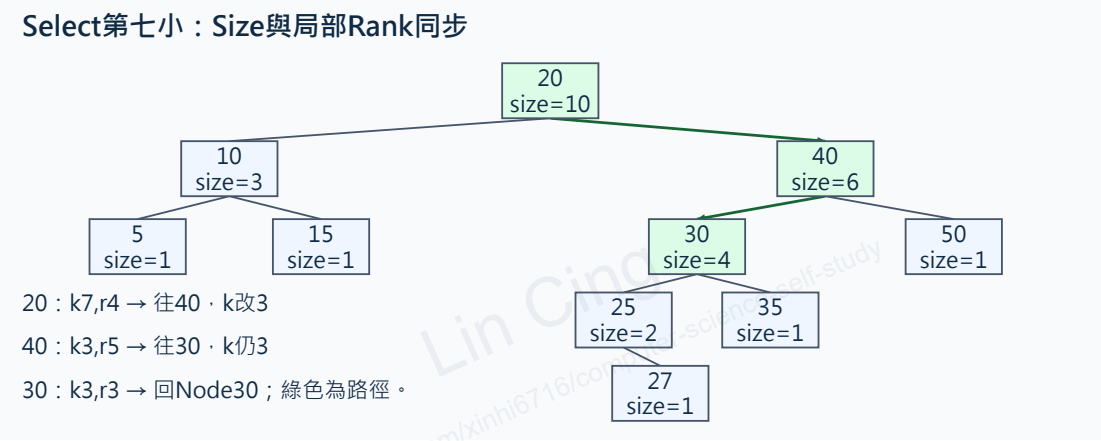
查詢的成本先是O(height+1)；只有底層維持第13章red-black性質，才可換成最壞O(log(n+1))。在普通BST上加size不會自動產生平衡。

Select的局部秩不變量

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

不變量：答案是目前x子樹的局部第k個identity。左子樹有r−1個節點、x佔第r位；往左不扣，往右扣掉完整左子樹及x共r筆。合法k條件保證所選側非空且新k合法，故不會遞迴進NIL；每次下降一層，終止時回正確node。

Select的局部秩不變量 （續）



第7小：root20的left.size=3、局部r=4，往40改k=3；40的left.size=4、r=5，往30仍k=3；30的left.size=2、r=3，回30。不能把全樹rank7直接拿到每層比較。

Rank每次只增加一塊尚未計入的前綴

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

不變量：r是原x在目前u子樹內的中序rank。若u是left child，parent及右側都在x之後，不新增前綴；若u是right child，parent.left全體和parent本身都在x之前，且不在先前u子樹內，恰加left.size+1，沒有重覆。到root即全樹rank。

同例rank27：初r=1；27是25.right，加0+1得2；25是30.left、30是40.left，各不加；40是20.right，加3+1得6。中序5,10,15,20,25,27,30,35,40,50的第6格確為27。

更新Size要跟著最終結構走

插入新葉size=1，只有祖先數量加1。可沿搜尋路徑計數，或接好後由parent往root重算；每個祖先O(1)，共O(height+1)。同key更新record不應誤把size加1。

刪除不能只對「被要求刪的z」一路減1後就結束：雙子情況後繼y從原位搬到z位置，y自身的子樹集合也改變。安全方法是完成第12章的transplant後，從受影響的最深祖先依序往上重算。

例刪20、搬25，先由25原父30往上：size30從4變3，size40從6變5，新root25須重算成1+3+5=9，不能保留25原size2。直接後繼分支可從搬上來的y開始；單子/leaf分支從z的原parent開始。

一般受影響node位於常數條祖先路徑的聯集，長O(height+1)。Red-black修復的旋轉還需同步更新size，下一節說明何時能只改兩個node。

14.2 How to augment a data structure

設計擴充時要同時回答四件事：底層支援哪些基本操作？需要保存哪個摘要？每種修改如何維持摘要？新query如何用它剪枝或定位？只有寫一條f公式，不算完成更新與query的證明。

常用充分條件：每node的f可在O(1)時間由自身不隨recolor改變的key/data、左右child的f計算，NIL的f有明確常數基例。局部資料或child連結變動只可能影響該node與祖先，按後代先、祖先後重算。

Red-black插入的初次接葉、刪除的常數次transplant，各造成常數條O(log n)祖先路徑；每次rotation先修兩個轉軸node，再視需要往祖先傳播，最多O(log n)額外工作。第13章插入最多2次rotation、刪除最多3次，所以總仍O(log n)。

此一般論證只保證「每次rotation含傳播最多O(log n)」，不是所有f都能rotation後只重算兩格。若f依顏色，還須把recolor對依賴路徑的影響納入更新，不能直接用「顏色不改結構」忽略它。

Size、Max與一般摘要的差別

Left-rotate(x)，y=x.right升上來，β=y.left移給x。先完成pointer更新，再pull(x)、pull(y)，因新y依賴新x；反過來會讀到舊x摘要。

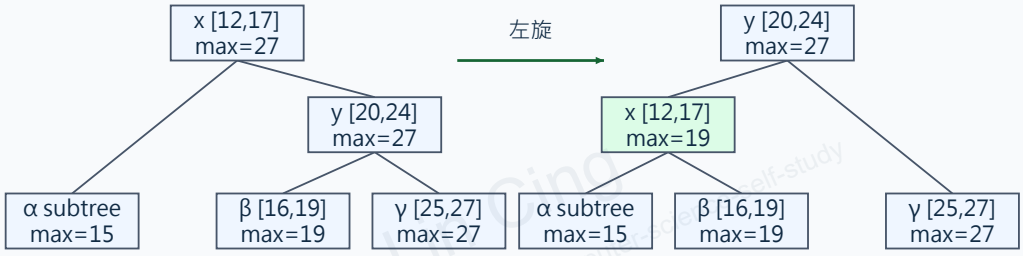
Size與「子樹所有interval的最大high」只依該子樹包含的內容，rotation後新y包含的內容恰為舊x整棵子樹，所以新y摘要等於舊x摘要，其他祖先的摘要不改。這兩種情況只需O(1)局部重算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size、Max與一般摘要的差別 (續)

局部Rotation：下移Node先更新摘要

原x=[12,17]、y=[20,24]；α.max15、β=[16,19]、γ=[25,27]。



先Pull新x：max(17,15,19)=19；再Pull新y：max(24,19,27)=27。

綠框：下移x的摘要改變；其他藍框摘要在旋轉前後相同。

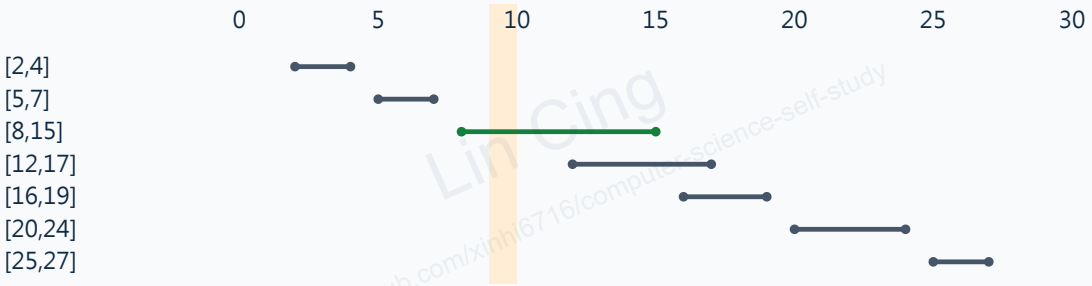
一般height摘要雖也由兩個child算出，rotation卻可能改變整棵局部子樹height，祖先仍須傳播。相反，若在每node直接存全樹rank，插新minimum會讓所有舊node的rank加1，需要Θ(n)次明寫；存depth也可能因一次rotation使大子樹全體depth改變。子樹size能在修改局部維護、查詢時沿路組合，是選它的原因。

14.3 Interval trees

本節interval是閉區間[l,h]，l≤h，允許退化為單點[l,l]。兩區間i、q重疊當且僅當i.low≤q.high且q.low≤i.high；等價於排除i完全在q左方（i.high<q.low）與右方（q.high<i.low）。

閉Interval與同一個Overlap Query

橘帶是閉Query [9,10]；實線兩端實心點表示閉端點。



綠：[8,15]重疊Query；灰：其餘不重疊。例：[4,5]與[5,7]在5重疊。

14.3 Interval trees （續）

以low作BST主鍵；同low可加identity作tie-break，仍保留「left.low≤x.low≤right.low」。每node存max(x)=max(x.high,left.max,right.max)，NIL.max=−∞。這是**子樹**的最高high，不是該node的high，也不是最大的low。

本例root[12,17]的high17、max27；left[5,7]的max15來自其right[8,15]。只看root.high或當前interval，會漏掉伸得更遠的後代。

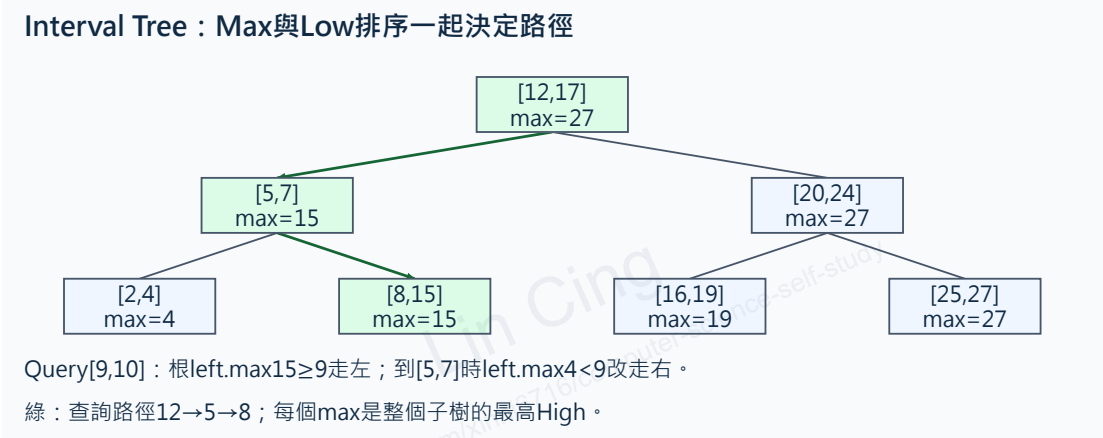
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條路徑的Overlap search

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條路徑的Overlap search （續）



Query[9,10]：root[12,17]不重疊，但left.max15≥9，往[5,7]；它也不重疊，left[2,4].max4<9，改往right[8,15]，重疊成功。

Query[18,18]：root不重疊且left.max15<18，往[20,24]；它不重疊、left.max19≥18，往[16,19]成功。
Query[28,29]依序[12,17]→[20,24]→[25,27]→NIL，回不存在。

往右的剪枝理由

維持不變量：若整棵樹存在重疊interval，當前x子樹也有一個。初始化root當然成立，當前x若重疊就回傳。若不重疊，要證明選方向不丟掉所有解。

當left=NIL，左側沒有候選；或left.max<q.low，任意左interval j都有j.high≤left.max<q.low，完全在query左方，所以整個left可排除。既然x也不重疊，若有答案只能在right，往右保持不變量。

這個證明使用所有left後代的high上界；「x.high<q.low」本身不足以推論left也全部太左。若只將例子的root改成[12,13]，query為[14,14]，root.high13<14，但left內的[8,15]仍重疊，不能因此剪掉left。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

往左時為何不必另回來找Right

若left.max≥q.low，令j是left中達到該max的interval。若left本有任何重疊，走左沒問題。若left完全沒有重疊，則j.high≥q.low卻不重疊，只能是j.low>q.high，不能是j.high<q.low。

對任意right後代k，BST主鍵序給`k.low≥x.low≥j.low>q.high`，所以right也全在query右方。加上x不重疊，此時x子樹本就沒有答案。故無論left有沒有答案，選left都保持「原本若有解，所選側必有解」。

這是選一條路徑仍正確的核心，不是宣稱只要left.max≥q.low，left一定有重疊。到NIL時所選子樹沒有解，由不變量反推全樹沒有解。每次下降一層，時間O(height+1)，red-black底層給O(log(n+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

修改Interval後Max也必須向上修

插入/刪除後，沿受影響祖先pull max；rotation後先新x再新y。不能只在插入時把祖先max變大，刪掉最高high或縮短interval後，max也可能要下降；必須從自身high與兩child重新取max。

同例刪[25,27]，right[20,24]的max由27降24，root的max也由27降24。這說明摘要必須能下降；單是這次刪除，留下舊值的兩個node並非被讀取的left child，尚不是漏搜的具體反例。

真正的stale-left反例：先把原[8,15]改為[8,27]（low不變），其父[5,7]的max成27。再刪掉[8,27]，正確left subtree只剩[5,7]與[2,4]，max應降為7；假設錯留27。查[22,22]時，root[12,17]不重疊，卻因left.max27≥22錯往[5,7]，接著因[2,4].max4<22走到空right，回NIL；真正重疊的right[20,24]被漏掉。若max被低估，則可能直接剪掉真答案。要維持**精確**max，不是任意上界。

注意：[5,7]與[4,5]在閉端點5重疊；若應用採半開[5,7)、[4,5)，它們不重疊。半開非空interval須l<h，overlap改嚴格不等式，左側可行判斷也改left.max>q.low。不能只改圖上的括號而保留舊比較式。

找一個、找最左與列全部是不同Query

本節演算法只承諾回任一個重疊record，不保證low最小，也不列全部。若要列全部，可遞迴：left.max≥q.low才走left；檢查x；若x.low≤q.high且right.max≥q.low才走right。每個被訪nodeO(1)，最壞O(n)，不能直接把單一路徑O(log n)套到所有輸出。

另一個清楚可證的列舉法是重複overlap-search、暫時刪掉找到的record，記在列表；共找到k筆加最後一次失敗query，時間O((k+1)log(n+1))，再插回也同階，但會暫時修改樹，需符合呼叫者契約。輸出k筆本身至少Ω(k)，而要精確達O(log n+k)需另外分析或選更合適的表示，不能只寫「依輸出量」當證明。

Order-statistic與interval摘要可同時保存size、max，pull各O(1)，兩者的更新仍同階；但rank是依BST鍵(low,identity)排序，並不是high排序或重疊筆數，這些問題需要不同摘要/演算法。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 14。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

18 B 樹

Part V Advanced Data Structures

本章定位

B-tree在一個頁內放多個鍵，降低搜尋路徑的磁碟頁數。插入與刪除的難點是搬鍵時同步搬child指標，保住全域鍵序、每節點容量及所有葉同深度。本章分別計算頁I/O與頁內CPU工作。

前置知識

多叉搜尋樹、磁碟 I/O 與高度平衡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以最小度 t 描述 B-tree 的節點容量。
- 分析搜尋、插入分裂與刪除合併的 I/O。
- 檢查根節點與鍵數的邊界。
- 18.1 Definition of B-trees
- 18.2 Basic operations on B-trees
- 18.3 Deleting a key from a B-tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

18.1 Definition of B-trees

B-tree 把多個排序鍵與多個子指標放在同一個節點，通常讓一節點對應一個磁碟頁。本章假設鍵互異，每鍵的資料或記錄指標隨鍵一起搬移；重複鍵插入回報已存在。這是內節點也存資料的 B-tree，與只在葉存資料、父節點複製 separator 的 B+ tree 不同。

最小度 t 是整數且 t≥2。每個非根節點有 t−1..2t−1 個鍵；非空根有1..2t−1個鍵。具有k個鍵的內節點恰有k+1個child，且所有葉深度相同，記為高度h，根深度0。

節點鍵 a0<a1<⋯<a_(k−1) 把child的全域範圍分開：child0的所有鍵<a0，childi的鍵在(a_(i−1),a_i)，childk的所有鍵>a_(k−1)。只檢查每節點內排序不夠，還必須檢查整棵子樹範圍。

空樹以零鍵的葉根表示，高度0；高度上界定理只用在n≥1的樹。

高度上界與下界

最少鍵的非空高度h樹，根只有1鍵且2個child，其餘節點恰有t−1鍵。深度i≥1至少有2t^(i−1)個節點，因此

$n \geq 1+\Sigma_{i=1}^{h} 2t^{i-1}(t-1)=2t^h-1$，

故 $h \leq \log_t((n+1)/2)$。h=0時空和為0，同樣成立。

相反，每節點至多2t−1鍵、最多2t個child，所以

$n \leq (2t-1)\Sigma_{i=0}^{h}(2t)^i=(2t)^{(h+1)}-1$，

故 $h \geq \log_{(2t)}(n+1)-1$。兩界分別控制最稀疏與最密集的樹，不能混用來猜「通常幾層」。

十億個鍵的頁數界

取t=100、n=10^9。高度2最多200^3−1=7,999,999鍵，不夠；高度3最多200^4−1=1,599,999,999鍵，容得下，所以最小可能高度3。

高度4最少2×100^4−1=199,999,999鍵，仍可；高度5最少19,999,999,999鍵，太多，所以最大可能高度4。因此任意符合條件的樹h∈{3,4}，搜尋走4或5個節點。

若根已常駐記憶體且其他頁起初不在快取，這是3或4次頁讀取；根也不在記憶體則是4或5次。單節點內用線性掃描要O(t)比較，故CPU O(th)；搜尋可改binary search為O(h log t)比較，但插入/刪除的陣列搬移仍可能O(t)每層。

每頁可容納的t由頁大小、key/record/child指標大小與metadata決定。I/O次數不是硬碟延遲測量，也不能直接用鍵比較次數代替。

18.2 Basic operations on B-trees

節點陣列採0-based。搜尋k時找第一個大於等於k的鍵位置i；若不存在令i=鍵數。

```
search(x,k):
    i = first index with x.keys[i]>=k, or len(x.keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x is leaf: return NOT_FOUND
    read page x.children[i]
    return search(x.children[i],k)
```

若沒有相等鍵，節點分隔性保證至多一個child可能含k；若它也不存在，其餘child全被鍵序排除。每步降低剩餘高度，最後在葉成功或失敗。用於插入前的重複鍵檢查最多另花一條搜尋路徑，不改變漸近I/O界。

分裂滿子節點的全部移接

前提為父p未滿、y=p.children[i]有2t−1鍵。先保存中位鍵median=y.keys[t−1]，再分割：

```
z.keys = y.keys[t .. 2t-2]
y.keys = y.keys[0 .. t-2]
if y is internal:
    z.children = 原y.children[t .. 2t-1]
    y.children = 原y.children[0 .. t-1]
p.keys.insert(i,median)
p.children.insert(i+1,z)
write y,z,p
```

左右各t−1鍵；若為內節點，各有t個child。median從子節點移出成為父的真實鍵，不在兩邊重複保留。各child原來依median分成左右範圍，因此排序成立，葉深度不變。父未滿保證加1鍵仍≤2t−1。

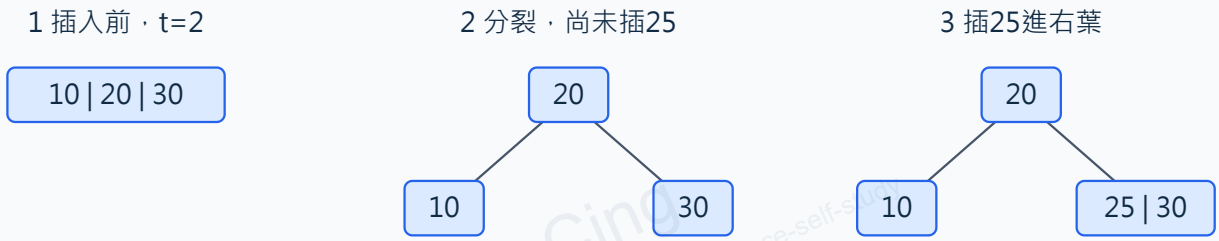
插入前先分裂，保持下行節點未滿

```
insert(k):
    if search(root,k) succeeds: return EXISTS
    if root is full:
        root = new empty internal node with old root as child0
        split(root,0)
    x=root
    while x is internal:
        i = child index selected by k
        if x.children[i] is full:
            split(x,i)
            if k>x.keys[i]: i++
        x=x.children[i]
    insert k into x.keys in sorted position
```

迴圈不變量是x未滿。若下一child滿，當場分裂後選一個t−1鍵的child，因此下次仍未滿；葉插入最多加1，不溢位。每層至多一次split，唯獨根分裂會增加高度，且所有葉同時增加一層。

插入前先分裂，保持下行節點未滿 (續)

滿根分裂，再插入25的三態



中位鍵20移到父，原葉不再存20；25>20，所以到右葉插入。

根分裂的數值核驗

t=2時滿葉根[10,20,30]，插入25：

階段	根	children	全部鍵的中序
插入前	[10,20,30]葉	無	10,20,30
先分裂	[20]	[10]、[30]	10,20,30
插25	[20]	[10]、[25,30]	10,20,25,30

20只出現在根，並沒有留在葉。插入25時因25>20選child1，該葉原有1鍵，插入後2鍵仍≤3。兩葉深度都是1。

split的頁寫入為舊child、新child、父，共常數次；父與舊child的讀取是否已在快取必須明定。O(h)表示隨樹高線性，不是所有插入固定恰有相同頁讀寫數。

18.3 Deleting a key from a B-tree

刪除採較強下行條件。除根外，每次遞迴進入的節點先有至少t鍵，因此沿路失去1鍵後仍有t−1鍵。根可以在最後由0鍵內節點縮成唯一child。

找到鍵k時分三種：

1. x是葉，直接刪k。
2. x是內節點，k左child至少t鍵，則以左子樹最大鍵pred取代k，再在左子樹刪pred。若左child只有t−1而右child至少t鍵，對稱改用successor。
3. 兩child都只有t−1鍵，把左child、父鍵k、右child合成2t−1鍵節點，再遞迴在合成節點刪k。

不能直接從內節點陣列移除k而保留全部child，因鍵數與child數及區間分隔都會失配。用pred/successor替換時，記錄資料也隨鍵移動。

還沒找到鍵時，先修復將下降的Child

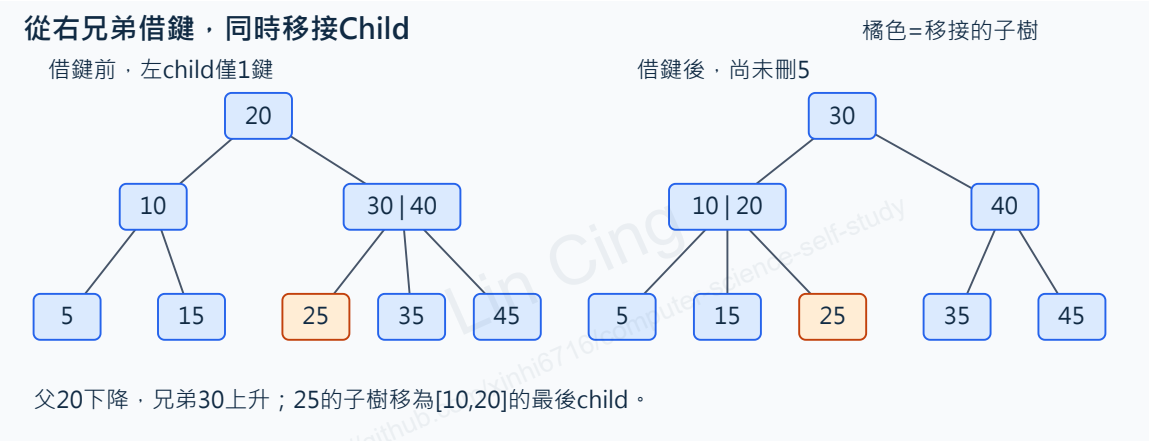
若x尚無k，依搜尋規則決定child y。若y已有至少t鍵，直接下降；否則y只有t−1鍵：

- 若相鄰左兄弟至少t鍵，父分隔鍵下降成y的第一鍵，左兄弟最大鍵上升補父；內節點還要把左兄弟最右child移成y的第一child。
- 否則若右兄弟至少t鍵，父分隔鍵下降成y的最後鍵，右兄弟最小鍵上升補父；內節點還要把右兄弟最左child移成y的最後child。
- 否則與任一相鄰兄弟及兩者間父鍵合併。合併後有2t−1鍵，再對該節點下降；若合併左兄弟，搜尋child索引須減1。

借鍵使兩個child都保持下限，父鍵數不變；合併使父少1鍵、少1個child，child鍵序與等深度仍成立。不存在的k也可能沿路觸發合法重整，但最後回報NOT_FOUND，鍵集合不能改變。

從右兄弟借鍵時，Child指標也要搬

以下t=2，原樹搜尋刪除5時，根的左child只有1鍵，不可直接下降；右child有2鍵，可借。



操作後根30，左child[10,20]有三個葉[5]、[15]、[25]；右child[40]有[35]、[45]。25這棵子樹不能留在右側，因25<新父30且20<25，必須是[10,20]的最後child。

從右兄弟借鍵時，Child指標也要搬 （續）

這張圖是下降前的修復，尚未刪5。接著在左child內，待降葉[5]與右兄弟[15]都只有1鍵，所以把[5]、父鍵10、[15]合成[5,10,15]，再刪5得[10,15]。左內節點剩[20]，children為[10,15]、[25]；根30及右子樹不變。所有葉仍深度2。

內節點替換與根縮高的數例

若根[20]、左葉[5,10]、右葉[30]，刪20採前驅10。新根[10]、左葉[5]、右葉[30]，中序為5,10,30。不是在父複製10後忘了從左葉刪掉它。

若根[20]、左葉[5]、右葉[30]，兩邊皆只有t−1鍵，不能任意拿走一邊的鍵。先合成[5,20,30]，舊根暫時為空且僅一child；遞迴在合成葉刪20得[5,30]，最後用這個唯一child取代空內根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

內節點替換與根縮高的數例 （續）

只有根空了才能縮高；一般內節點不可以把唯一child直接接到祖父，否則會破壞所有葉同深度。最後一個鍵刪掉時保留零鍵葉根，代表空樹。

刪除偽碼與正確性歸納

```
delete(x,k):
    i = first key >=k, or key count
    if key i equals k:
        if leaf: erase key i; return FOUND
        if child i has >=t keys: replace k by pred; delete(child i,pred)
        elif child i+1 has >=t keys: replace k by succ; delete(child i+1,succ)
        else: y=merge(child i,key i,child i+1); delete(y,k)
        return FOUND
    if leaf: return NOT_FOUND
    y=child i
    if y has t-1 keys: borrow from a rich neighbor, else merge as above
    return delete(the repaired target child,k)
after top call: if root has0keys and1child: root=its child
```

以剩餘高度歸納。葉刪除在下行條件下不低於t−1；內部替換保持鍵序與總鍵集合只少目標k；借鍵/合併先保證子問題合法且高度下降。非根x進入時至少t鍵，合併使x少1仍合法；根例外由最後縮高處理。這同時證明終止、鍵集合正確與結構不變量。

操作成本與實作檢查

每次下降只處理常數個相鄰節點，頁讀寫O(1)每層，所以搜尋、插入、刪除皆O(h+1)=O(1+log_t(n+1))頁存取。節點內搬移鍵/child最多O(t)，CPU O(t(h+1))。取得pred或succ若分開搜尋再刪，至多增加同一子樹的常數次路徑；也可合併成一次下降。

這些界假設一節點可放一頁且根快取模型固定，未含併發鎖、交易日誌、頁壓縮與碎片管理。它們是額外實作契約，不能從抽象B-tree直接推得。

每次操作後可檢查鍵全域範圍、非根鍵數下限、child數=鍵數+1、所有葉同深度、無重複child指標、總中序與參考集合相同。有限程式檢查能抓實作錯誤，前面的歸納才給任意合法操作的正確性。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 18。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

19 費氏堆積

Part V Advanced Data Structures

本章定位

Fibonacci heap用延後整理讓meld與decrease-key有低攤銷成本。必須同時理解root/child環、失子mark、級聯cut與degree碰撞，再以同一勢能支付操作並用Fibonacci下界限制整理陣列大小。

前置知識

可合併堆積、攤銷分析與切割標記。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以根串列與 child list 表示較鬆散的 heap。
- 攤銷 merge、insert、decrease-key。
- 用 degree bound 證明 consolidate 成本。
- 19.1 Structure of Fibonacci heaps
- 19.2 Mergeable-heap operations
- 19.3 Decreasing a key and deleting a node
- 19.4 Bounding the maximum degree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

19.1 Structure of Fibonacci heaps

Fibonacci heap 是多棵 min-heap 有序樹的集合。每個非根x滿足parent.key≤x.key；root list保存所有根，H.min指向最小鍵根，H.n是總節點數。decrease-key/erase以有效handle指定節點，不包含先搜尋任意鍵的成本。

每節點保存key、degree、parent、child、left、right、mark。degree是當前child數，child指向一個child；同層以環狀雙向串列相連。本文把所有根mark清為false；非根mark=true表示自上次連到現父後已失去一個child。

普通鍵可重複，但handle identity不同。刪除後handle失效；meld消耗兩個輸入heap的所有權，輸入不能再各自修改同一批節點。這些條件讓指標與成本主張有明確對象。

環狀串列的常數次指標操作

移除x時先保存next=x.right。若x是唯一節點，就把該串列head設NIL；否則設x.left.right=x.right、x.right.left=x.left，必要時head=next。最後令x.left=x.right=x，並更新所在容器的head/degree。

兩個非空環A、B可直接串接。取a∈A、b∈B，先保存an=a.right、bp=b.left，再設

```
a.right=b; b.left=a
bp.right=an; an.left=bp
```

這只改兩條相鄰邊，沒有走訪全部節點；空環直接回傳另一環。把x加入root list前，先從原child ring移除、degree減1，再設x.parent=NIL、mark=false。若先覆寫x.right才嘗試走原環，可能遺失後繼。

19.2 Mergeable-heap operations

MAKE-HEAP建立空root list、min=NIL、n=0。INSERT建立degree0、parent/child=NIL、mark=false的單節點環，接到root list，比較更新min，n加1。MINIMUM只回傳min，空heap回傳NIL。

MELD把兩個root rings接起，min取兩個min較小者，n相加；不合併樹、不掃描節點。各環已有最小根，min比較即足夠。這些操作都是O(1)真實成本。

用足夠大的固定常數C取勢能$\Psi=C(t+2m)$，t為根數、m為標記節點數。對多個heap取勢能總和，初始空heap為0且始終非負。INSERT使t增加1、Ψ增加C，故O(1)攤銷；MELD保留根/標記總數，ΔΨ=0，故仍O(1)。

Extract-Min 的完整次序

```
extract-min(H):
    z=H.min
    if z==NIL: return NIL
    while z.child ring is nonempty:
        detach one child x from z
        x.parent=NIL; x.mark=false
        append x to root list
    remove z from root list; H.n--
    if H.n==0: H.min=NIL
    else: consolidate roots by degree; recompute H.min
    invalidate z's heap handle; return z
```

移除最小根後，其child都成為根，因樹內heap order仍成立；其餘根本來不小於z。新min要在整理後的根中重新找，不能把舊z指標留著，也不能把整棵z子樹一併刪除。

Consolidate 的碰撞迴圈

```
A[0..D] = NIL
while old root list is nonempty:
    x=detach one root; d=x.degree
    while A[d]!=NIL:
        y=A[d]; A[d]=NIL
        if y.key<x.key: swap(x,y)
        make y a child of x; y.parent=x; y.mark=false
        x.degree++; d=x.degree
    A[d]=x
rebuild root list from nonempty A slots
H.min = smallest-key root
```

每次link合併相同degree的根，把較大鍵根接到較小鍵下，保持heap order；d加1後必須再次查A。結束後每種degree至多一根。D須至少⌊log_φ n⌋，可按需擴充；錯用⌊log_2 n⌋未必放得下合法最大degree。

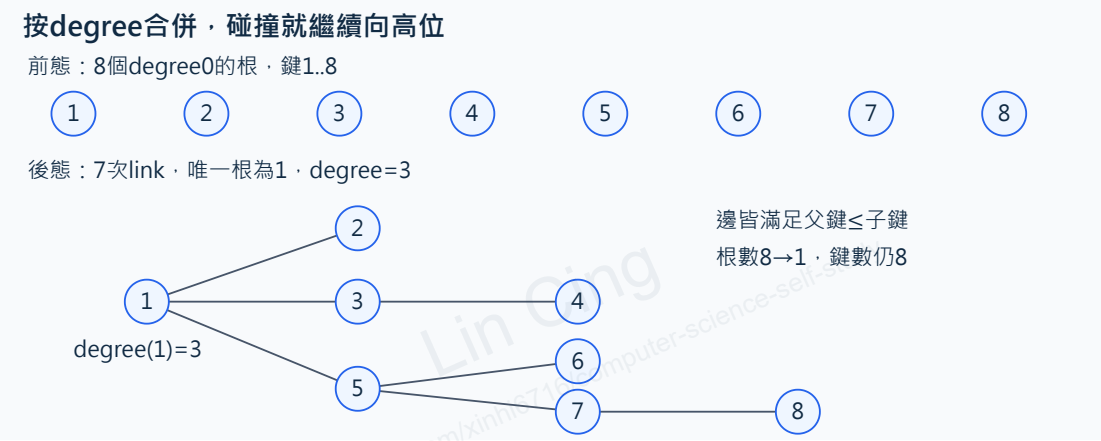
直接逐一消耗舊root ring時，degree陣列為O(log n)輔助空間。若用snapshot避免走訪時改鏈，則snapshot可能有Θ(n)根，不能還宣稱全部輔助空間只有O(log n)。

八個根的數值合併軌跡

依序插入0..8後extract0，留下8棵單節點根1..8，按此順序整理：

讀入根	這一輪link	完成後A中的根，以degree:key表示
1	無	0:1
2	2接到1	1:1
3	無	0:3，1:1
4	4接3；3接1	2:1
5	無	0:5，2:1
6	6接5	1:5，2:1
7	無	0:7，1:5，2:1
8	8接7；7接5；5接1	3:1

八個根的數值合併軌跡 （續）



7次link把根數8降到1，鍵集合仍是1..8。樹不必是完全樹；正確性依據是每條parent-child邊的heap order及實際degree。

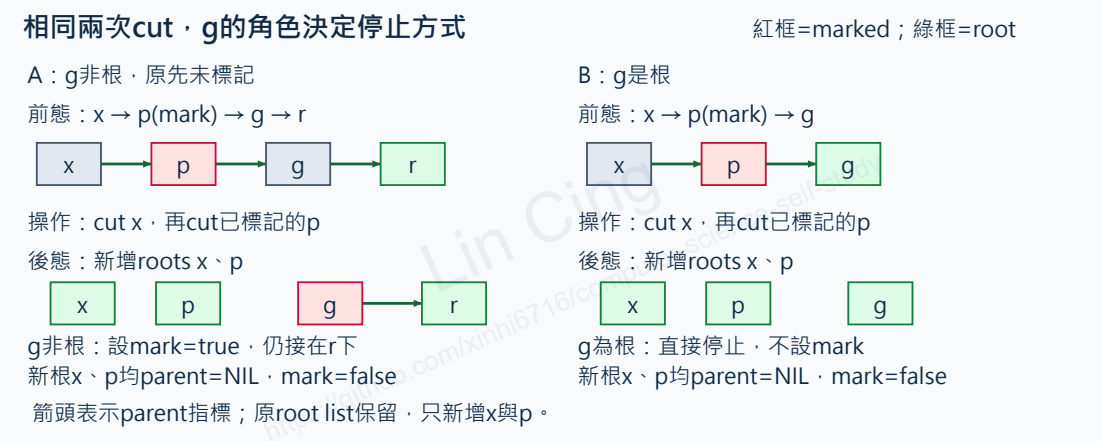
19.3 Decreasing a key and deleting a node

```
decrease-key(H,x,new):
    require new<=x.key and x is a valid handle
    x.key=new; p=x.parent
    if p!=NIL and x.key<p.key:
        cut x from p to root list
        cascading-cut(p)
    if x.key<H.min.key: H.min=x
cascading-cut(y):
    p=y.parent
    if p==NIL: return
    if y.mark==false: y.mark=true; return
    cut y from p to root list
    cascading-cut(p)
```

cut清除被切節點的mark，減少舊父degree並保持雙向環。降低x只可能破壞x與父的邊，與child的heap order反而更寬鬆；切斷違規邊後所有樹仍有序。mark規則另用來保護degree下界，不是為了比較最小鍵。

級聯切割的兩個停止分支

若p先前已標記，切走其第二個child x後，p也被切到root list。接著檢查p的舊父g：g非根且尚未標記，就設g.mark=true停止；g是根就直接停止，不因本次失子標記。



圖中root list是「新增x、p」，原本的根仍保留。`x.parent=NIL`、`p.parent=NIL`且mark=false；g若非根仍接在原父下，只改mark。若g也已標記，還要繼續cut g，不可套未標記分支。

合法操作歷史中的真實節點

從空heap插入0..64、extract0，得到64節點的已整理heap；節點id原本等於初始鍵，降鍵後id不改。

分支	第一次操作	第二次操作	第二次完成後
g是根	id6降−10，cut6，標記p=id5	id7降−11，cut7再cut5	roots={1,6,7,5}，g=id1不標記
g非根	id14降−10，cut14，標記p=id13	id15降−11，cut15再cut13	roots={1,14,15,13}，g=id9標記，仍在id1下

兩例第一次操作後t=2、m=1，t+2m=4。第二次根分支變t=4、m=0，勢能尺度仍4；非根分支t=4、m=1，變6。這些父子由同一組合法link、cut歷史產生，不是假定任意形狀一定能出現。

Decrease-Key 的完整勢能計費

若沒有cut，實際成本與勢能變化都是O(1)。若共cut c≥1個節點，根數增加c。除最先因鍵值違規被切的x外，其餘c−1個級聯cut節點在被切前都已標記，並在cut後清除；最後至多新標記一個未標記非根。

因此$\Delta m \le -(c-1)+1 = 2-c$，

$\Delta(t+2m) \le c+2(2-c) = 4-c$。

若實際成本≤ac+b，固定C≥a，則

$\hat{c} \le ac+b+C(4-c) \le b+4C = O(1)$。

這明確放大同一勢能Ψ=C(t+2m)，使釋放的信用支付實際指標操作常數。單次decrease-key可沿很長祖先鏈；常數界是任意操作序列的攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Delete 的成本與特殊最小值

CLRS的抽象DELETE(x)先把x降為嚴格小於所有普通鍵的特殊−∞，再extract-min，所以O(1)+O(log n)=O(log n)攤銷。不能把delete與decrease-key一起寫成O(1)。

−∞是抽象鍵域的額外元素，不表示C++某整數型別的最小值一定可用。若heap原已允許同樣的最小值，平手可能讓extract-min刪掉別的handle。

也可直接刪指定handle：若x非根，先cut x並對舊父做cascading-cut；接著從root list移除這個x、把其children提升成根、consolidate並重算min。此流程毋須改key，仍是O(1)攤銷cut階段加O(log n)攤銷根移除，不是省掉consolidate的O(1)刪除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

19.4 Bounding the maximum degree

固定節點x，當前k個child依它們連到x的時間先後排列為y1..yk。y1的degree≥0。對i≥2，yi當時連入x時，較早且現在仍存在的i−1個child都已在，所以x當時degree≥i−1。

link只合併相同degree的根，故yi當時degree也≥i−1。yi只要仍掛在x下，自那時起就至多失去一個child；失去第二個時早已被cut。因此現在

$degree(y_i) \geq i-2$。

這使用「當前仍存child的加入順序」，不是假設x從未失去child，也不是說現在所有child的degree都相同。

子樹大小至少是 Fibonacci 數

定F0=0、F1=1、$F_{j+2}=F_{j+1}+F_j$。證明任意合法節點x滿足$size(x) \geq F_{degree(x)+2}$，以子樹節點數N做強歸納。

N=1時degree0、F2=1成立。N>1時，每個child子樹都小於N，歸納假設可用於其實際degree，不必假設child degree小於父degree。由上一頁對yi的界與F單調性，

$size(x) \geq 1+1+\sum_{i=2}^{k} F_i = F_{k+2}$，$k \geq 1$。

最後等式由$\sum_{i=0}^{k} F_i = F_{k+2}-1$配上F0=0、F1=1得到；k=1時空和也給2=F3。

再令$\phi=(1+\sqrt{5})/2$，F2=1=ϕ^0、F3=2≥ϕ；由前兩階相加，$F_{k+2} \geq \phi^{k-1}+\phi^{k-2}=\phi^k$。故$n \geq size(x) \geq \phi^k$，得到$k \leq \log_\phi n$。這是degree界，沒有把樹高宣稱為O(log n)。

Extract-Min 的勢能總帳

設移除前有t根，最小根至多D個child，移除並提升後至多t−1+D根。每次link根數減1，所以整個整理的實際成本O(t+D)，不會因while碰撞巢狀就變O(tD)。

整理後degree各不相同，至多D+1根，標記數不增加；因此$\Delta(t+2m)\le D+1-t$。在與decrease-key共用的足夠大C下，

$\hat{c}\le a(t+D)+b+C(D+1-t)=O(D+1)$。

由degree證明D=O(log(n+1))，extract-min與delete都是O(log(n+1))攤銷。minimum、insert、meld、decrease-key為O(1)攤銷，其中前三者還是O(1)實際成本。這些結論從同一非負勢能望遠鏡求和得到；有限操作測試只檢查圖、數值及實作狀態。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 19。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20 van Emde Boas 樹

Part V Advanced Data Structures

本章定位

van Emde Boas樹利用整數鍵的高低位元，快速找前驅與後繼。雙對數時間的關鍵是min/max快取讓每層只走一條昂貴遞迴；最小值獨立儲存與線性universe配置成本則決定更新規則和使用條件。

前置知識

整數字 universe、位元分割與遞迴摘要。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較 bitmap、summary 與 clustered 結構。
- 解釋 vEB 遞迴結構的 universe 與 summary。
- 推導 insert、successor 與 min/max 時間。
- 20.1 Preliminary approaches
- 20.2 A recursive structure
- 20.3 The van Emde Boas tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.1 Preliminary approaches

整數集合的universe為U={0,⋯,u−1}，u=2^b且b≥1。successor(x)指嚴格大於x的最小集合元素，predecessor則嚴格小於；x不必已在集合中，沒有答案時回NIL。

本章採word-RAM，字長w≥b，合法鍵、位移、遮罩及陣列索引各花O(1)。這不是逐位運算的bit-complexity模型；若一個鍵跨多個機器字，不能沿用同一常數。

bitmap用u位，member可直接讀位元。但successor一般要往後掃；即使每字可快速找第一個1，最壞仍可能掃Θ(u/w)字。只有整個universe放一字且模型提供相應bit-scan時，才可把它當O(1)。紅黑樹則以O(n)節點、O(log(n+1))時間處理任意比較鍵，無需先配置整個u。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分群可以省掃描，但不自動得雙對數

u=256可分16群，每群16鍵。summary中第i位為1代表群i非空。找successor時，先判斷本群是否有更大鍵，再找後續非空群，最後取該群最小鍵。

若兩層都採逐位掃描，最壞工作約群內16加summary16，仍依群大小增加。若群寬m，粗略最壞界為O(m+u/m)，在m≈$\sqrt{u}$時為O($\sqrt{u}$)，不是O(log log u)。

改成多層遞迴後，還必須數清楚每層到底有幾個昂貴子呼叫。下一節的proto結構示範「每次universe開根號」本身不足以保證目標時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.2 A recursive structure

把b個位元拆成上半⌈b/2⌉位與下半⌊b/2⌋位。設

```
upper=2^ceil(b/2); lower=2^floor(b/2)
high(x)=floor(x/lower); low(x)=x mod lower
index(h,l)=h*lower+l
```

有upper個cluster，每個universe大小lower；summary自身的universe大小upper，保存非空cluster的索引。不是在所有地方都把兩種大小一律寫成同一個√u。

u=32時b=5、upper=8、lower=4。x=27拆成high=6、low=3，再組回6×4+3=27。若把兩者都取√32再四捨五入，會失去完整且不重疊的位元分割。

Proto-vEB 的兩個遞迴為何仍慢

先看b為2冪、可恰好等分的情形。沒有快取min/max的proto-vEB，求最小值要遞迴找summary的最小非空群，再遞迴找該群最小鍵：

$T_{min}(u)=2T_{min}(\sqrt{u})+\Theta(1)$。

令S(b)=T(2^b)，得S(b)=2S(b/2)+Θ(1)=Θ(b)，即Θ(log u)。插入若每次都同時遞迴cluster與summary，也有此界。

proto-successor可能先遞迴本群、失敗後遞迴summary，再求新群minimum，所以最壞

$T_{succ}(u)=2T_{succ}(\sqrt{u})+\Theta(\log u)$。

換元得S(b)=2S(b/2)+Θ(b)=Θ(b log b)，即Θ(log u log log u)。改進必須減少每層的昂貴分支，不能只引用遞迴深度。

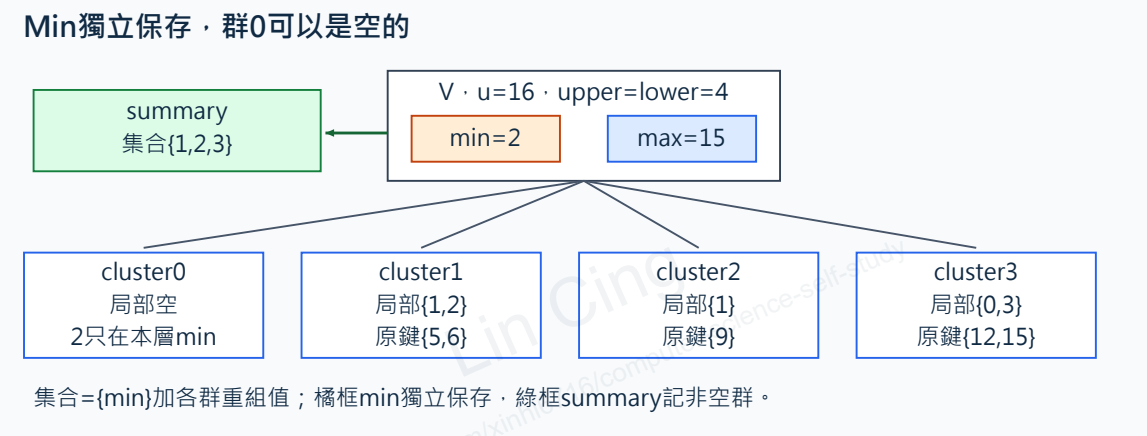
20.3 The van Emde Boas tree

每個vEB節點額外保存min、max，空集合兩者皆NIL。min不放入任何cluster；除min外的所有鍵依high/low唯一放入cluster。summary只保存這些cluster中非空者的索引。

因此單元素時min=max且所有cluster空；至少兩元素時max也在某個cluster中。u=2是基底，只須min/max，沒有再配置cluster。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

20.3 The van Emde Boas tree （續）



數例u=16、S={2,5,6,9,12,15}：min=2、max=15；cluster0空，cluster1存局部{1,2}，cluster2存{1}，cluster3存{0,3}；summary={1,2,3}。不能因high(2)=0就強迫summary包含0，因2只在本層min。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

空、單元素與Member

```
member(V,x):
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return member(V.cluster[high(x)],low(x))
minimum(V): return V.min
maximum(V): return V.max
empty-insert(V,x): V.min=V.max=x
```

member每層只進一個cluster，且universe嚴格變小。min/max與空/單元素判斷皆O(1)；empty-insert不用遞迴填整棵樹，因新min本來就不屬於cluster。這個常數基底也會讓更新只剩一條昂貴遞迴。

公開介面先確認x是0..u−1整數。INSERT已存在鍵、DELETE不存在鍵可先以member偵測，回報false而不更動集合；加一次同階查詢不改變漸近時間。下列內部更新偽碼採「鍵尚未存在／鍵確實存在」前提。

Successor 的互斥分支

```
successor(V,x):
    if V.min==NIL or x>=V.max: return NIL
    if x<V.min: return V.min
    if V.u==2: return 1
    h=high(x); l=low(x); C=V.cluster[h]
    if C.max!=NIL and l<C.max:
        return index(h,successor(C,l))
    j=successor(V.summary,h)
    if j==NIL: return NIL
    return index(j,V.cluster[j].min)
```

先讀C.max就能知道同群是否存在更大值。如果存在，只遞迴C；否則直接遞迴summary，不先花時間搜尋已知不會成功的C。最後讀下一群min是O(1)，所以每層至多一個昂貴子呼叫。

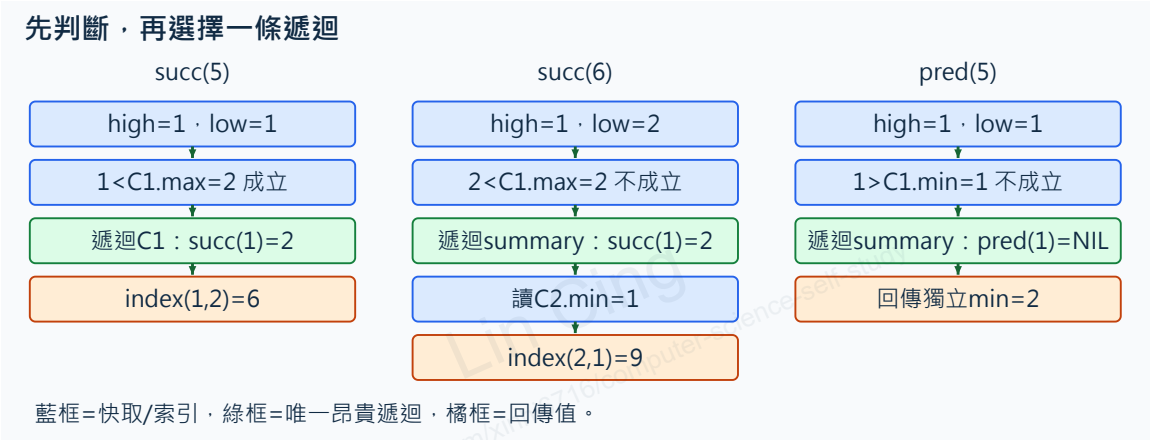
Predecessor 必須補回全域Min

```
predecessor(V,x):
    if V.min==NIL or x<=V.min: return NIL
    if x>V.max: return V.max
    if V.u==2: return 0
    h=high(x); l=low(x); C=V.cluster[h]
    if C.min!=NIL and l>C.min:
        return index(h,predecessor(C,l))
    j=predecessor(V.summary,h)
    if j==NIL: return V.min
    return index(j,V.cluster[j].max)
```

到j=NIL分支時，前面已確認x>V.min，所以全域min仍是合法答案。不能把successor的NIL分支完全鏡射，因min刻意沒有放在cluster裡。

例S={2,5,6,9,12,15}，pred(5)在cluster1內沒有更小局部鍵，summary也沒有比1更小的群，仍須回2。

同群與跨群的實際查詢



succ(5)：high=1、low=1，cluster1.max=2，1<2成立，只遞迴cluster1，得到局部2，組回1×4+2=6。

succ(6)：high=1、low=2，不小於cluster1.max=2，所以不遞迴cluster1。summary的succ(1)=2，讀cluster2.min=1，組回2×4+1=9。

同群與跨群的實際查詢 (續)

查詢值不必屬於S。succ(3)也走cluster0空的跨群分支，得到cluster1.min=1，組回5；succ(15)=NIL，pred(2)=NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insert 的Min交換與空群分支

```
insert-new(V,x):
    if V.min==NIL: empty-insert(V,x); return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        h=high(x); l=low(x); C=V.cluster[h]
        if C.min==NIL:
            insert-new(V.summary,h)
            empty-insert(C,l)
        else: insert-new(C,l)
    if x>V.max: V.max=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

若新x比min小，交換後新x直接留在min，原min成為接下來要插入cluster的值。空群分支雖更新summary與cluster兩處，但empty-insert是O(1)；非空群只遞迴cluster，不動summary，所以仍至多一條昂貴遞迴。

例向上述S插1：min變1，原min2送入cluster0的局部2，summary新增0。最後min=1、max=15，cluster0={2}，而不是把新min1也放進cluster0。

Delete 的基底與最小值提升

```
delete-present(V,x):
    if V.min==V.max: V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=(1 if x==0 else 0); return
    if x==V.min:
        h=V.summary.min
        x=index(h,V.cluster[h].min)
        V.min=x
    h=high(x); l=low(x); C=V.cluster[h]
    delete-present(C,l)
```

此程序接著執行下一頁的summary/max收尾區塊。多元素時刪min，必須先找cluster中最小的元素提升成新min，然後從原cluster刪掉這個提升值，保持min不重複儲存。此處變數x改為提升值，已不再是呼叫者原本的舊min。

Delete 的Summary與Max完整收尾

接續前頁的C刪除：

```
if C.min==NIL:
    delete-present(V.summary,h)
    if x==V.max:
        j=V.summary.max
        if j==NIL: V.max=V.min
        else: V.max=index(j,V.cluster[j].max)
elif x==V.max:
    V.max=index(h,C.max)
```

若C刪後為空，表示刪前只有單一元素，剛才的delete-present(C,l)只做O(1)基底工作，才會接著遞迴summary。若cluster刪除真的沿多層走，C不會變空，就不會再遞迴summary。兩個昂貴分支仍互斥。

若不更新summary，日後可能走進空群；若不更新max，successor可能因錯誤的上界判斷而選錯分支，這兩欄不是可有可無的快取。

由同一資料重算更新結果

從S={2,5,6,9,12,15}出發：

操作後	min	max	cluster0	cluster1	cluster2	cluster3	summary
原集合	2	15	空	{1,2}	{1}	{0,3}	{1,2,3}
insert1	1	15	{2}	{1,2}	{1}	{0,3}	{0,1,2,3}
delete1	2	15	空	{1,2}	{1}	{0,3}	{1,2,3}
delete15	2	12	空	{1,2}	{1}	{0}	{1,2,3}

delete1先把cluster0的2提升到min，再刪局部2，群0空才刪summary中的0。delete15後群3仍有局部0，故summary保留3，新max=3×4+0=12。每列都可由min與所有cluster組回完整集合，沒有遺失或重複提升值。

正確性與雙對數時間

用b歸納，基底u=2由min/max精確代表空、單元素或{0,1}。假設較小universe的操作正確，每層用互斥high範圍分群，index是high/low的反函數，min再獨立保存。

member只到唯一可能的群；successor/predecessor以群max/min判斷是否能在本群找到，否則用summary找最近群，並另處理獨立min。insert交換與delete提升都只移轉一個代表，summary恰與群非空條件同步，所以表示不變量與集合操作皆維持。

每操作最多一個非O(1)子呼叫，其bit數≤⌈b/2⌉。因此

`T(2^b)≤T(2^ceil(b/2))+O(1)`。

對b≥2，⌈b/2⌉≤2b/3，至b=1只需O(log b)層，時間O(1+log log u)。這個結論依賴前面逐項證明的互斥分支，不能單靠樹的層數。

預先配置的空間遞迴

原版預先建立所有summary與cluster，空集合也佔Θ(u)字，初始化Θ(u)時間。令P(u)包含節點欄位與cluster指標陣列，u>2時

$P(2^b)=2^{\lceil b/2\rceil}\,P(2^{\lfloor b/2\rfloor})+P(2^{\lceil b/2\rceil})+\Theta(2^{\lceil b/2\rceil})$。

設$F(b)=P(2^b)/2^b$、$\varepsilon_b=2^{(-\lfloor b/2\rfloor)}$，則對固定c，

$F(b)\le F(\lfloor b/2\rfloor)+\varepsilon_b\,F(\lceil b/2\rceil)+c\,\varepsilon_b$。

取$G=F+c$，可得$G(b)\le(1+\varepsilon_b)\max(G(\lfloor b/2\rfloor),G(\lceil b/2\rceil))$。接著沿較大那項反覆下降到b=1，可把整個倍率界成有限乘積，而不把summary成本忽略掉。

空間線性界及配置模式的限制

沿上述halving路徑從基底反向數r層，bit數至少$2^{(r-1)}+1$，因此ε依序至多1/2、1/2、1/4、1/16、…，總和有與b無關的常數界。由$1+x\le e^x$，

$\prod(1+\varepsilon)\le\exp(\sum\varepsilon)=O(1)$，

所以$F(b)=O(1)$，$P(u)=O(u)$。只走cluster分支到u=2，就已把原universe分成u/2個基底節點，給Ω(u)，合得Θ(u)。初始化寫入這些欄位同階Θ(u)。

若u很大而n很小，預先配置可能完全不划算。改成按需建立群與hash dictionary時，建群成本、dictionary時間與空間都需要另證；不能只寫「lazy」就同時承諾O(n)空間及所有操作最壞O(log log u)。本章已證的界明確限於預先配置、直接陣列索引的word-RAM版本。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 20。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

04 分治法

Part I Foundations

本章定位

分治的正確性靠完整分類與遞迴契約；本章以最大子陣列及Strassen七乘積建立算法，再用同一常數的歸納、逐層求和與完整主定理證明分析時間，明列取整、對數邊界與不等比例條件。

前置知識

遞迴、陣列區間、矩陣乘法的基本索引與歸納法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用分治計算最大連續子陣列。
- 解釋 Strassen 矩陣乘法的分塊與次立方成本。
- 使用代入法、遞迴樹與主定理分析遞迴式。
- 在子問題大小不相等時，辨識 Akra–Bazzi 的外部適用條件並完成積分。
- 4.1 The maximum-subarray problem
- 4.2 Strassen’s algorithm for matrix multiplication
- 4.3 The substitution method for solving recurrences
- 4.4 The recursion-tree method for solving recurrences
- 4.5 The master method for solving recurrences
- 4.6 Proof of the master theorem

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分治的規格與成本要分開建立

分治（divide and conquer）先把輸入拆成更小的同類問題，遞迴解決，再合併答案。正確性由「子問題答案足以涵蓋原問題」與歸納建立；成本則把每次呼叫的分割、合併、配置、複製逐項計入。

本章以1-based陣列及包含端點的區間為主。時間採精確實數／整數的單位成本算術模型；若整數位元長度增長，乘加不能仍無條件算O(1)。矩陣乘法先用精確算術證明，不把代數恆等式當成浮點誤差保證。

遞迴等式 $T(n) = \sum_i T(n_i) + f(n)$ 描述完整成本；若只知 $T(n) \le \sum_i T(n_i) + f(n)$，通常只能推出上界。還需基例、每個 $n_i < n$ 的終止條件及分割的實際成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.1 The maximum-subarray problem

給n≥1個數A[1..n]，求一個非空、連續區間[i,j]，使 $\sum_{k=i}^{j} A[k]$ 最大，回傳(i,j,sum)。相同最大值可有多個答案；除非另訂規則，只需回傳其中一個。

價格P0,⋯,Pn若必須先買後賣，令Ai=Pi−P(i−1)，則 $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$。故區間i..j對應第i−1天買、第j天賣。若允許不交易，則可另選空區間、收益0；全負輸入時兩種規格答案不同。

逐一枚舉起點，再延長終點並累加，可用Θ(n^2)時間、O(1)額外空間求解；若每個區間又從頭加總，才會變成Θ(n^3)。前綴和也可把一次區間求和降到O(1)，但仍有Θ(n^2)個候選區間。

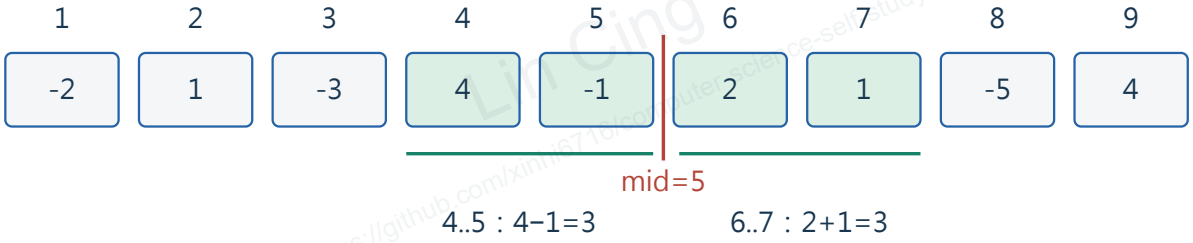
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨中線必須由 suffix 與 prefix 組成

對low<high，令mid=⌊(low+high)/2⌋。任一非空區間恰屬三類：完全在low..mid、完全在mid+1..high、或i≤mid<j。第三類必包含mid與mid+1，可唯一拆成左半的suffix i..mid與右半的prefix mid+1..j。

兩側端點可獨立選擇，故最佳跨線和等於「最佳非空左suffix和」加「最佳非空右prefix和」。不需要枚舉每個(i,j)配對；分別掃描兩側即可。只取左右兩半各自的最大子陣列會出錯，因它們未必接到中線。

索引與非空跨線區間：A[4..7] 是本輪答案



跨線和 3+3=6；左suffix與右prefix必須緊接中線。

跨線掃描的完整程序

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

左迴圈每次更新後，total恰為當前i..mid的和，left是已掃過所有suffix的最大和；初始化沒有候選，−∞不會錯選空區間。每輪新增唯一一個候選並取最大，因此不變量成立。終止時所有左suffix均已納入；右側同理。掃描總次數(mid−low+1)+(high−mid)=high−low+1。

最大子陣列的遞迴與一般證明

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = floor((low+high)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

對區間長度強歸納：單元素只有自身；較長時兩個遞迴區間均非空且較短，故由歸納假設求得前兩類最大值，CROSS已證求得第三類最大值。三類互斥且完整，三者取最大即全域最大，長度嚴格縮小也保證終止。

$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n)$。順序遞迴使用O(lg n)stack，每層只存常數個索引與和；若額外複製子陣列，需另計配置與活躍空間，不能沿用此空間界。

九元素數例：每次跨線比較都可核對

A=[−2,1,−3,4,−1,2,1,−5,4]，low=1、high=9、mid=5。

左索引i	A[i..5]累計	目前最大與起點
5	−1	−1，i=5
4	3	3，i=4
3	0	3，i=4
2	1	3，i=4
1	−1	3，i=4

右側j=6,7,8,9的prefix和依序2、3、−2、2，所以最佳是6..7、和3。跨線區間4..7和6；左半最佳4..4和4，右半最佳9..9和4，故本輪選(4,7,6)。全負例[−8,−2,−5]會選(2,2,−2)，不會回0。

線性方法的狀態與不變量

令end_i為必須以i結尾的最佳非空區間和。這種區間要麼只有Ai，要麼由i−1結尾的最佳區間接上Ai，因此 $end_i = \max(A_i, end_{i-1} + A_i)$。全域best_i=max(best_(i−1),end_i)，初值end_1=best_1=A1。

每輪若選Ai便重設起點i；否則保留上一個起點。best改善時保存該起點及終點i。歸納可證end與best分別是定義中的兩個最優值，故Θ(n)時間、O(1)額外空間即可回傳區間。

九元素例end為[−2,1,−2,4,3,5,6,1,5]，best為[−2,1,1,4,4,5,6,6,6]，最佳4..7。分治不是此問題的最快方法，但它展示「類別覆蓋＋合併規格」如何構成正確性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.2 Strassen’s algorithm for matrix multiplication

對n×n矩陣，$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$。三層迴圈先令Cij=0，再依k加入乘積。內圈不變量是已完成前k項總和；n^2個輸出各n次乘加，故Θ(n^3)。這只是這個程序的成本，並未證明所有矩陣乘法都需Ω(n^3)。

把A、B分成2×2區塊，令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，各字母是n/2階矩陣。區塊乘法給C11=ae+bg、C12=af+bh、C21=ce+dg、C22=cf+dh。順序必須保留；區塊矩陣一般不交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八次遞迴乘法的完整成本

程序輸入為同階方陣，先補零至2的冪次階數，再取回原尺寸的結果。每個遞迴呼叫由輸入view的rows取得自己的階數，不另傳原矩陣的n。

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

索引view只傳起點、stride與階數，分割可$O(1)$；若真的複製區塊，是$\Theta(n^2)$，仍和四個區塊加法同階。遞迴式為$8T(n/2)+\Theta(n^2)$，基例$\Theta(1)$，解$\Theta(n^3)$。

常數個矩陣加法可合成$\Theta(n^2)$，但不能把$8T(n/2)$改成$T(n/2)$：這個8每層都增殖，決定葉數n^3。對固定一次呼叫吸收常數，與改變整棵遞迴樹的分支數，是不同操作。

Strassen 的七個乘積全部列出

以下七個式子以遞迴STRASSEN計算；其輸入的加減皆是逐元素區塊運算。

$$
\begin{aligned}
M_1 &= (a+d)(e+h), & M_2 &= (c+d)e,\\
M_3 &= a(f-h), & M_4 &= d(g-e),\\
M_5 &= (a+b)h, & M_6 &= (c-a)(e+f),\\
M_7 &= (b-d)(g+h).
\end{aligned}
$$

回收結果為 $C_{11}=M_1+M_4-M_5+M_7$，$C_{12}=M_3+M_5$，$C_{21}=M_2+M_4$，$C_{22}=M_1-M_2+M_3+M_6$。n=1直接做一個純量乘法；n>1依此分割、計七乘積並回收四塊，構成完整遞迴算法。這套M編號是本章的定義，不與其他教材的P編號混用。

四個輸出區塊為何正確

只需分配律與加法消去，沒有交換矩陣乘法的次序：

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

基例純量乘法正確；若七個較小乘積由歸納假設正確，上述恆等式回收到AB的全部四塊，完成一般正確性證明。需要可加、可減及分配律；只有加法、沒有減法的一般semiring不能直接使用這七式。

七乘積的數值與四塊回收

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]，所以a=2,b=−1,c=3,d=4,e=5,f=2,g=−2,h=1。

乘積	代入	值
M1	(2+4)(5+1)	36
M2	(3+4)5	35
M3	2(2−1)	2
M4	4(−2−5)	−28
M5	(2−1)1	1
M6	(3−2)(5+2)	7
M7	(−1−4)(−2+1)	5

四塊依序36−28−1+5=12、2+1=3、35−28=7、36−35+2+7=10。直接乘法也得[[10+2,4−1],[15−8,6+4]]=[[12,3],[7,10]]。

七乘積的數值與四塊回收 (續)

同一輸入的七乘積：四個回收位置不能互換

A

2	-1
3	4

B

5	2
-2	1

M_1=36；M_2=35

M_3=2；M_4=−28

M_5=1；M_6=7

M_7=5

C

12	3
7	10

C_{11}=36−28−1+5=12；C_{12}=2+1=3

C_{21}=35−28=7；C_{22}=36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的時間、補零與記憶體

分割view為O(1)，準備七乘積輸入共10次區塊加減，回收四塊共8次區塊加減。因此 $T(n) = 7T(n/2) + \Theta(n^2)$，$T(1) = \Theta(1)$，解 $\Theta(n^{\lg 7})$，$\lg 7 \approx 2.80735$。加減的常數較大，小矩陣不一定較快；可在固定閾值改用普通乘法，不改漸近指數。

任意正n可補零到m=2^⌈lg n⌉，n≤m<2n。補零矩陣左上乘積仍是AB，取回n×n即可；m^(lg7) <7n^(lg7)，所以同階。n=0直接回空矩陣，不遞迴。

深度優先、每個呼叫只保留常數個當層區塊，依次完成七個子呼叫並適時釋放暫存時，峰值額外空間滿足 $S(n) \le S(n/2) + O(n^2) = O(n^2)$，另有O(lg n)stack。若同時保留全部子問題工作區，空間遞迴不同，不能聲稱同一界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.3 The substitution method for solving recurrences

代入法先猜候選界，再以強歸納找出同一組固定常數，使所有較小輸入的假設能推出當前輸入。不能在每層重新選一個較大的常數。

例$T(n)=2T(\lfloor n/2\rfloor)+n$，$T(1)=1$，$n\geq 2$。欲證$T(n)\leq cn\lg n$：對$n\geq 4$，$m=\lfloor n/2\rfloor\geq 2$，代入得$2cm\lg m+n\leq cn\lg(n/2)+n=cn\lg n-(c-1)n\leq cn\lg n$，取$c\geq 1$。

但$n=1$右側為0，不能拿來當歸納基。改以$n=2$、3為基：$T2=4$、$T3=5$，$c\geq 2$可覆蓋；$n\geq 4$的child均≥ 2，證明閉合。也可改猜$cn(\lg n+1)$，一併涵蓋$n=1$。漸近結論不容許漏掉實際會被呼叫、又未被基例處理的小輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一常數的下界與強化假設

對二冪n，$T(n)=2T(n/2)+n$、$T1=1$，代入候選$T(n)\geq dn\lg n$得$T(n)\geq dn\lg n+(1-d)n$，取$0<d\leq 1$便成立；配基例得到$\Omega(n\lg n)$。非二冪可用後面的取整樹證明延伸，與上界合成$\Theta(n\lg n)$。

另一例$T(n)=T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+1$，$T1=1$。直接假設$T(n)\leq cn$會得到$cn+1$，不能閉合；這不表示線性猜測錯。強化為$T(n)\leq cn-d$，代入得$cn-2d+1\leq cn-d$，只需$d\geq 1$、$c\geq d+1$覆蓋$T1$。實際$c=2,d=1$還給精確解$2n-1$。

若把「$cn+n$仍是$O(n)$」當成第一個遞迴的線性證明，就是偷偷把c改成$c+1$，並未推出原假設；這種錯誤會把$n\lg n$誤證成線性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

換變數與真正縮小的條件

對T(n)=2T(√n)+lg n，先在n=2^(2^k)上定義，T2=Θ(1)。令m=lg n、S(m)=T(2^m)，則S(m)=2S(m/2)+m，得到S(m)=Θ(m lg m)，還原為T(n)=Θ(lg n・lg lg n)。不能把S的輸入m直接當成原n。

固定平移也要檢查終止。例如T(n)=2T(n/2+17)+n在n>34才有child<n；設n=34+x、U(x)=T(34+x)，得到U(x)=2U(x/2)+x+34。以足夠大的常數區間為基，樹求和為Θ(x lg x)=Θ(n lg n)。整數版本再保留floor誤差；不能只給T1而讓n=34反覆呼叫自己。

上述換元例的域已明列；把二冪上的證明推到任意n，仍需另外的取整、單調比較或適當基例，不是自動成立。

4.4 The recursion-tree method for solving recurrences

樹節點記的是本次呼叫的非遞迴成本f(size)，child才記遞迴呼叫；若在每個節點放完整T(size)後又加子樹，就會重複計費。要另外算深度、每層節點數及葉成本。

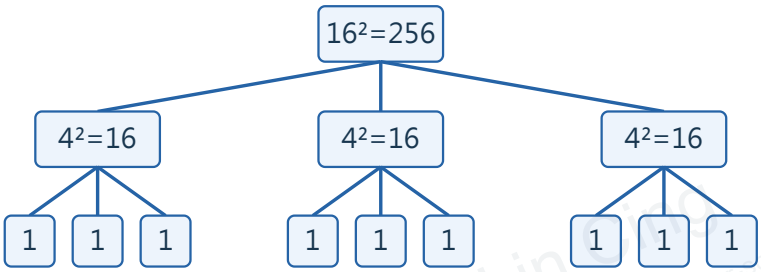
取原創遞迴T(n)=3T(n/4)+n²、T1=1，n=4^L。深度j有3^j個節點，每個成本(n/4^j)²，故層成本n²(3/16)^j。內層j=0..L−1，葉有3^L=n^(log₄3)，所以

$$T(n) = n^2 \sum_{j=0}^{L-1} (3/16)^j + n^{\log_4 3} = \Theta(n^2).$$

上界來自收斂幾何和≤16/13及葉成本≤n²，下界來自根的n²。n=16的實際層成本為256、48、9，總313；葉層9已是基例成本，不能再加9次f(1)。

4.4 The recursion-tree method for solving recurrences （續）

三個 n/4 子問題：每層成本乘 3/16，而不是 3/4



深度0：1×256=256

深度1：3×16=48

深度2：9×1=9

T(16)=256+48+9=313；最下排已是基例，不重複計費。

不等分割：先界深度，再界每層質量

令整數n≥2，a=⌊n/3⌋、b=n−a；對足夠大n用T(n)=T(a)+T(b)+cn，常數小輸入直接處理。兩個child大小和等於n，所以尚未終止的某一層，其輸入總量至多n，層成本至多cn。

最長路每步至多2n/3+1，深度O(lg n)，葉共O(n)，所以總成本O(n lg n)。下界不能只說「每層都是n」直到最深葉，因短路徑會提早終止。前⌊log₃(n/K)⌋層尚無任何路徑進入固定基例（選K足夠大吸收取整），這些完整層每層成本cn，故Ω(n lg n)。

若非遞迴項只有O(n)而非Θ(n)，上述上界仍成立，但不能由此推出相同下界；例如合併成本實為0時，葉成本可能只給Θ(n)。

4.5 The master method for solving recurrences

設a≥1、b>1固定，T有正且有界的常數大小基例，f≥0且最終為正，遞迴T(n)=aT(n/b)+f(n)。令p=log_b a。先用精確冪理解，再依下一節的條件處理整數取整。

情形	必須核對的條件	結論
1	存在固定ε>0，f(n)=O(n^(p−ε))	Θ(n^p)
2	f(n)=Θ(n^p)	Θ(n^p lg n)
3	固定ε>0，f(n)=Ω(n^(p+ε))；且af(n/b)≤cf(n)，固定c<1、充分大n成立	Θ(f(n))

f比n^p小或大，並不足以取代多項式分離；ε必須與n無關。情形3的正則性限制向下各層總成本，使它們可由根成本支付。a若是實數，可視為遞迴權重；描述實際a個呼叫時a必為整數。

同一張表的完整判定數例

遞迴的(a,b,f)	p與必要核對	結果
(9,3,n)	p=2，差n^1，情形1	Θ(n^2)
(1,3/2,1)	p=0，情形2	Θ(lg n)
(3,4,n lg n)	p≈0.7925；選ε=0.1；3f(n/4)/f(n)≤3/4，情形3	Θ(n lg n)
(8,2,n^2)	p=3，ε=1	Θ(n^3)
(7,2,n^2)	p=lg7，任取0<ε≤lg7−2	Θ(n^(lg7))

T=2T(n/2)+n lg n不屬經典三情形：lg n比任何固定n^ε小，f/n不是多項式分離；但每層求和仍可解。工具不適用不代表遞迴沒有解。

對數邊界必須真的求和

在精確冪n=b^L、固定常數大小基例下，若 $f(n)=\Theta(n^p(\lg n)^k)$，任何固定實數k都可由層和分析：由葉端令r=L−j，層成本為Θ(n^p r^k)，所以

$$T(n)=\begin{cases}\Theta(n^p(\lg n)^{k+1}) & k>-1,\\ \Theta(n^p\lg\lg n) & k=-1,\\ \Theta(n^p) & k<-1.\end{cases}$$

k>−1由積分比較 $\sum_{r=1}^{L} r^k=\Theta(L^{k+1})$；k=−1是調和和Θ(lg L)；k<−1收斂到正常數，加上葉成本仍Θ(n^p)。原書擴充練習的k≥0是其中一部分。此處在n大於固定基例時使用log，避開lg1=0的負次方。

例如2T(n/2)+n/lg n為Θ(n lg lg n)，不是Θ(n)；9T(n/3)+n²lg²n為Θ(n²lg³n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正則性缺失的明確反例

只看n=2^L、T1=1，令f(n)=n²當L偶數、f(n)=n³當L奇數。每個n都有f(n)≥n²，對a=b=2、p=1已滿足多項式分離，但不能得T=Θ(f)。

在偶數L且n≥4時，子問題n/2的指數奇數，因此 $T(n)\ge 2f(n/2)=n^3/4$。當前f(n)=n²，故T(n)/f(n)≥n/4無界。相同計算給2f(n/2)/f(n)=n/4，正是正則性失敗。

這不是反駁主定理，而是展示缺條件後結論會失效。也不能以「floor只差1」推任意震盪f值接近；取整的實際自變數仍要放進所用條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

4.6 Proof of the master theorem

先令n=b^L，基例T1=t>0。反覆展開恰L次，或按樹深度歸納，可得精確式

$$T(n) = a^L T(1) + \sum_{j=0}^{L-1} a^j f(n/b^j) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

葉成本tn^p提供所有情形的基本正下界。情形1若f(x)≤Cx^(p−ε)，層成本≤Cn^(p−ε)(b^ε)^j，幾何比b^ε>1，因此總和O(n^(p−ε)(b^ε)^L)=O(n^p)。固定小x不滿足上界的末幾層也只有O(n^p)，可由有限個f值吸收；加葉成本得Θ(n^p)。

幾何和是往葉端增加，不是遞減。例如a=b=2、ε=1/2，比為√2>1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

主定理等高與根主導的兩個證明

情形2：a=b^p使每層 $a^j \Theta((n/b^j)^p) = \Theta(n^p)$。有L−O(1)=Θ(lg n)個大於固定基例的層，雙向界相加得Θ(n^p lg n)，葉成本被其涵蓋。

情形3：反覆使用 $af(x/b) \le cf(x)$，在大小仍充分大的每層得到 $a^j f(n/b^j) \le c^j f(n)$。因此這些內層總和≤f(n)/(1−c)。剩下固定大小層及葉的總成本O(n^p)；由f(n)=Ω(n^(p+ε))，它們也是O(f(n))。根本身f(n)給下界，所以Θ(f(n))。

固定小輸入的總成本不是整棵樹的O(1)：可能有Θ(n^p)個小節點。本證明將它們明確併入葉階，而不是漏算。三情形均同時交付上、下界。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

取整的誤差不會逐層線性累積

每條路徑令q0=n，q(j+1)是⌊qj/b⌋或⌈qj/b⌉。兩者皆滿足 $|q_{j+1} - q_j/b| \le 1$。若C≥b/(b−1)，由歸納

$$|q_j - n/b^j| \le C,$$

因下一層誤差至多C/b+1≤C。選固定基例範圍q≤n0，n0>C+2且夠大使child<q；則所有路徑進基例的深度均為log_b n+O(1)。在距葉端至少固定層數的位置，n/b^j≥2C，所以實際qj與n/b^j在固定倍數內。

實際a個child時，各路徑深度只差常數，故葉數Θ(a^(log_b n))=Θ(n^p)，所有固定大小底層成本也是O(n^p)。a=1時是一條路、常數個底層節點；公式仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從精確冪推到所有整數的成本條件

情形1：在離基底固定層數以上，qj與n/b^j同階，即使p−ε為負也有qj^(p−ε)=Θ((n/b^j)^(p−ε))；套f(qj)的多項式上界，得到與精確冪相同的增加幾何和。底層O(n^p)另計。

情形2：同樣有f(qj)=Θ((n/b^j)^p)，每個完整的大規模層成本Θ(n^p)，且有Θ(lg n)層；不完整的固定個底層不改此界。對數擴充對遠離固定基例的層也以r=距底層深度比較，log qj與r同階，得到前述三種對數和。

情形3必把正則性施於實際child：若可能有floor或ceiling，充分條件是 $a\max\{f(\lfloor x/b\rfloor), f(\lceil x/b\rceil)\} \le cf(x)$，c<1。如此逐層總成本仍以c倍收縮；加O(n^p)底層並用多項式分離即可。只證qj相差常數，對任意f不能省略這個條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正則性也能推出多項式分離

在精確冪上，若af(x/b)≤cf(x)、0<c<1，且固定底層f(x0)>0，反向沿遞迴k層得到f(b^k x0)≥(a/c)^k f(x0)。因 $\log_b(a/c) = p + \log_b(1/c)$，所以f(n)=Ω(n^(p+ε))，ε=log_b(1/c)>0。

這解釋為何在正且良好基底下，某些版本不另列多項式分離；本章保留標準三情形的完整假設以便直接核對。這項推導在已明列的精確冪域成立；對一般實數或整數域還要控制最後落入哪個固定區間及該區間的正下界。

本章至此已給主定理各情形的一般證明與整數延伸；下一部分的不等比例工具則明列為引用定理，不能混稱同一個證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Akra–Bazzi：不等子問題的引用定理

主定理處理同一大小 `n/b` 的 a 個子問題；Akra–Bazzi 允許多種固定比例。這是 CLRS 第三版第 4 章以外的延伸工具。本節採 MIT 6.042J 的連續形式，另以正且有上下界的基例作為充分假設。

令 `T(x)` 在 `0<=x<=x_0` 滿足 `0<d_1<=T(x)<=d_2`，其中 d_1、d_2 是固定常數；對 `x>x_0` 定義

`T(x)=sum_{i=1}^k a_i T(b_i x+h_i(x))+g(x)`。

須同時滿足：

- k 有限且固定，所有 `a_i>0`、`0<b_i<1` 為固定常數。
- `g>=0` 可微，且存在固定 C、D，使充分大 x 的 `|g'(x)|<=D x^C`；本節各例的 g 在基底區間也連續。
- 每個 `|h_i(x)|=O(x/log^2 x)`。
- `x_0` 足夠大，使每個 child argument 都在 `[0,rho*x]`，其中固定 `rho<1`。遞迴因而有定義，且會進入基例。

令唯一實數 p 滿足 `sum_i a_i b_i^p=1`，則

`T(x)=Theta(x^p (1+integral_1^x g(u)/u^{p+1} du))`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Akra–Bazzi：不等子問題的引用定理 （續）

正基例提供葉層的正成本，不能改成容許整個問題成本為零：`T≡0,g≡0` 會使右側的正 Theta 下界失效。本節引用這個定理；以下推導特徵根、積分與適用性，不把它的完整證明與主定理證明混為一談。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取整與小擾動的界線

對固定比例 b_i，floor 和 ceiling 相對 `b_i x` 的誤差絕對值至多 1，因此符合上述擾動界。加一個固定常數也同樣符合；仍須把基底範圍選得足夠大，確保 child 真正縮小。

若差異為 `epsilon*x`，其中 epsilon>0 固定，則它不是 `O(x/log^2 x)`；這是比例改變，必須重新求特徵根。這個檢查直接關係到指數，不能以「都是二個遞迴呼叫」代替。

不同來源可能使用除法參數 `x/c_i`、`c_i>1`。它與本節乘法比例的對應是 `b_i=1/c_i`；特徵方程因而分別寫成 `sum_i a_i c_i^{-p}=1` 和 `sum_i a_i b_i^p=1`，不可在同一個例子中互換符號定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不等子問題的唯一指數與積分

依本節的乘法比例定義 `F(p)=sum_i a_i b_i^p`。每個 `b_i^p` 對 p 嚴格遞減，所以 `F` 也嚴格遞減。它在 `p->-∞` 趨近無限大，在 `p->∞` 趨近 0；由介值定理，對任何有限正係數 `a_i` 都恰有一個實根，不能只在本例先假定根存在。`F(0)=sum_i a_i`：大於 1 時根為正，等於 1 時根為 0，小於 1 時根為負。p 可以是分數；公式給的是 Theta 階，不是精確閉式解。

取

`T(x)=1` 於整個 `0<=x<=1` 基例區間；`x>1` 時 `T(x)=T(x/3)+T(2x/3)+x`。

此時 `a_1=a_2=1`、`b_1=1/3`、`b_2=2/3`、`g(n)=n`。特徵方程為

`(1/3)^p+(2/3)^p=1`。

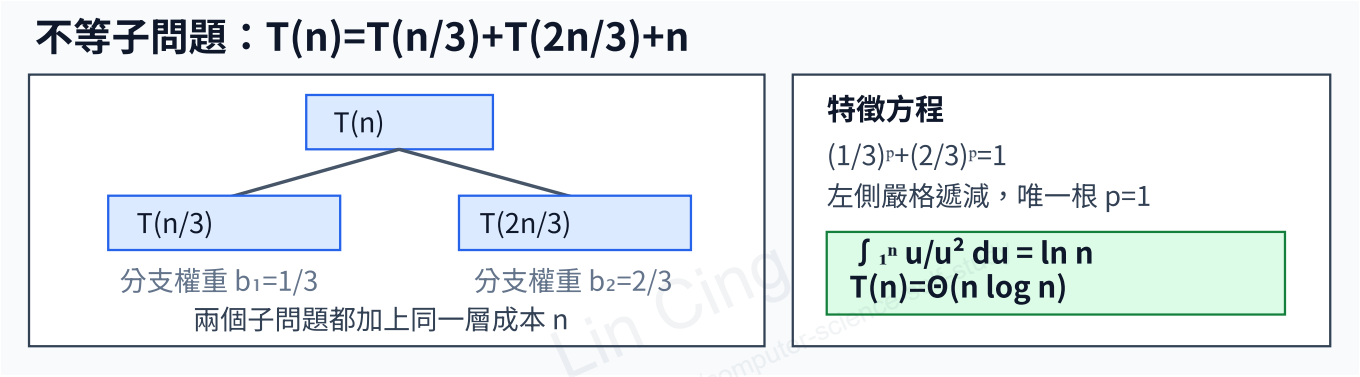
代 `p=1` 得 `1/3+2/3=1`；因左側嚴格遞減，沒有第二個根。代入積分：

`T(n)=Theta(n(1+integral_1^n u/u^2 du))`

`=Theta(n(1+ln n))=Theta(n log n)`。

這個結果不依賴某個固定 n 的枚舉。下界與上界都由一般定理同時給出。

不等子問題的唯一指數與積分 （續）



擾動過大時不能照套

考慮

$T(n)=T(n/2+n/4)+T(n/2)+n$。

若保留表面參數 $b_1=b_2=1/2$，會得到 $(1/2)^p+(1/2)^p=1$，即 $p=1$，預測 $\Theta(n \log n)$。但第一個 child 並未落在 $n/2+O(n/\log^{1+\epsilon}n)$；擾動是 $n/4$，違反小擾動條件。

這個遞迴必須先改寫成

$T(n)=T(3n/4)+T(n/2)+n$。

現在 $b_1=3/4$、$b_2=1/2$，特徵方程是

$(3/4)^p+(1/2)^p=1$。

數值解 $p\approx1.507126591639$，代回積分得 $\Theta(n^p)$，不是 $\Theta(n \log n)$。因此「看起來有兩個 1/2 子問題」不構成套用條件；必須先確認每個實際 child 大小，並核對擾動界。

主定理與 Akra–Bazzi 的判定順序

遇到遞迴時，先按模型分類，再選工具。

1. 所有子問題都是同一大小 n/b 的 a 份：先檢查主定理的三種情形與正則性。
2. 子問題有固定多種比例，例如 $n/3$ 與 $2n/3$：可用 Akra–Bazzi，先解 p 再算積分。
3. 索引只在 floor、ceiling 或 $O(n/\log^2 n)$ 的小範圍內擾動：可保留原比例；若擾動已達固定比例，必須改寫成真實的基底比例。
4. g 有負值、不符合已述平滑性條件，或基例沒有良好定義時，定理前提不足；改用展開或直接歸納，不要以公式硬套。

對 $T(n)=T(n/3)+T(2n/3)+n$，主定理的三情形不能直接表示不同 child 大小；Akra–Bazzi 正是在處理這個缺口。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 04。

M. Akra and L. Bazzi, "On the Solution of Linear Recurrence Equations," *Computational Optimization and Applications* 10(2), 1998, doi:10.1023/A:1018353700639.

K. Mehlhorn, *A Master Theorem for Recurrences*, Max Planck Institute for Informatics, 2008, Theorems 1-3.

MIT 6.042J, *Mathematics for Computer Science*, Fall 2010, § 10.4.3, Theorem 10.4.1. 官方講義. 本章採正且有界基例的充分條件。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

07 快速排序

Part II Sorting and Order Statistics

本章定位

Quicksort的分割只建立兩側與pivot的大小關係，還要遞迴才完成排序。本章從Lomuto不變量追到完整數值狀態，再把最壞時間、內部亂數的期望比較數與stack深度分開分析，特別處理重複鍵造成的退化。

前置知識

陣列分割、遞迴與期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 實作 partition 並維持正確的分割不變量。
- 區分最壞、最好與期望快速排序。
- 分析隨機選 pivot 如何解除對抗性輸入。
- 7.1 Description of quicksort
- 7.2 Performance of quicksort
- 7.3 A randomized version of quicksort
- 7.4 Analysis of quicksort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Description of quicksort

Quicksort先選pivot，將其他鍵按與pivot的大小分開，再遞迴排序兩側。partition完成時只保證區域間的大小關係，兩側內部尚未排序。本節採1-index、Lomuto、最後位置r為pivot。

```
quicksort(A,p,r):
    if p>=r: return
    q=partition(A,p,r)
    quicksort(A,p,q-1)
    quicksort(A,q+1,r)
partition(A,p,r):
    pivot=A[r]; i=p-1
    for j=p..r-1:
        if A[j]<=pivot:
            i++; swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

空區間與單元素是基底；兩個遞迴子區間都排除q，所以嚴格縮小。若還把q放回同一側，遇pivot在端點就可能無限遞迴。

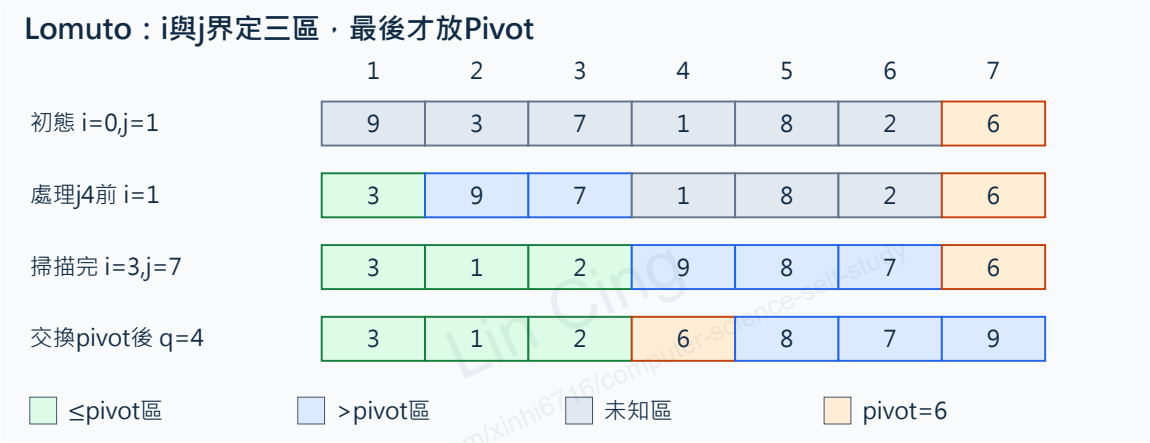
Lomuto的迴圈不變量

每次處理j前，A[p..i]≤pivot，A[i+1..j−1]>pivot，A[j..r−1]尚未處理，A[r]=pivot。空區間採通常的空集合慣例。

初始化i=p−1、j=p，前兩區都空。若A[j]≤pivot，i加1後與A[j]交換，新的小鍵移到≤區尾端；被換到j的舊值原屬>區，下一輪j加1後仍在正確區域。若A[j]>pivot，只增j，>區擴大。

結束j=r時沒有未知區。把pivot與A[i+1]交換，得到q=i+1，左區≤pivot、右區>pivot，pivot在q。交換保留元素多重集合，每一側長度小於原區間。以區間長度歸納，兩側排序後接上pivot，全段即排序正確。

Lomuto的迴圈不變量 （續）



完整數值分割追蹤

A=[9,3,7,1,8,2,6]，p=1、r=7、pivot=6，初i=0。

j	判斷	i更新	交換後陣列
1	9>6	0	[9,3,7,1,8,2,6]
2	3≤6	1	[3,9,7,1,8,2,6]，swap1,2
3	7>6	1	[3,9,7,1,8,2,6]
4	1≤6	2	[3,1,7,9,8,2,6]，swap2,4
5	8>6	2	[3,1,7,9,8,2,6]
6	2≤6	3	[3,1,2,9,8,7,6]，swap3,6
結束	pivot放i+1	q=4	[3,1,2,6,8,7,9]，swap4,7

左[3,1,2]都≤6，右[8,7,9]都>6，內部並未排序。partition比較正好r−p次，時間Θ(r−p+1)、額外空間O(1)。

7.2 Performance of quicksort

若每次pivot都是最小或最大，子問題長度0與n−1，partition做n−1次比較，總比較

$(n-1)+(n-2)+\dots+1=n(n-1)/2=\Theta(n^2)$。

例如已排序的n個不同鍵、固定取最後pivot，每次都是最大，這是一整族最壞輸入，不只是某個小例子的時間觀察。

上界也要涵蓋所有分割，而不是只算這個壞例子。令C(n)為最壞比較數，子問題長度a+b=n−1。歸納假設給

$C(a)+C(b)+(n-1) \le a(a-1)/2+b(b-1)/2+(n-1) = n(n-1)/2-ab \le n(n-1)/2$。

基例n=0、1皆為0，故所有輸入至多n(n−1)/2次比較，端點分割恰達界。每次partition固定開銷O(1)，每個被選pivot不再出現，總呼叫O(n)，故總時間O(n+C)=O(n^2)，配上上述族得最壞Θ(n^2)。

若每次非空子問題的較大側最多ρn、固定ρ<1，每層所有partition處理量至多n，最長分支O(log_(1/ρ)n)層，所以總O(n log n)。1:9分割仍是此情況，不需要恰好平分。

7.2 Performance of quicksort （續）

完全均分的n=2^k−1個鍵有k層，第d層剩餘處理量為n−(2^d−1)；前一半層數每層仍為Θ(n)，給Ω(n log n)，配上上界得此均分情況Θ(n log n)。這個下界來自該次遞迴工作量，沒有把比較排序的「至少某個輸入很慢」誤用成每個輸入都慢。

重複鍵如何破壞二路分割

對全相等A=[5,5,5,5,5]，Lomuto每次判斷≤都成立，q=r，左側長n−1、右側0。即使每輪先隨機選pivot，換來的仍是相同5，總比較仍10；一般n為n(n−1)/2。

因此下一節的隨機O(n log n)分析先假設鍵互異。若要支援大量重複鍵的效率，可使用小於/等於/大於三路partition，整段等值區不再遞迴；第9章會完整列出區間邊界與rank換算。

常見的「隨機pivot解決一切最壞輸入」缺少鍵相異或適當重複鍵處理的前提。穩定性也是不同性質，quicksort的長距離swap一般會改變相等鍵identity順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴空間與只遞迴較小側

原偽碼的兩次遞迴最壞可有n層堆疊，即使partition原地進行，整個quicksort也不是無條件O(1)額外空間。

```
while p<r:
    q=partition(A,p,r)
    if q-p < r-q:
        quicksort-small-stack(A,p,q-1)
        p=q+1
    else:
        quicksort-small-stack(A,q+1,r)
        r=q-1
```

每次真正的遞迴只選較小側，長度≤(n−1)/2；較大側由while繼續，不保留新一層frame。每層遞迴規模至少減半，故最壞堆疊O(log(n+1))。所有partition仍會執行，時間最壞仍Θ(n^2)，不是藉減少stack就保證時間O(n log n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.3 A randomized version of quicksort

對目前A[p..r]，從p..r均勻隨機選k，先swap(A[k],A[r])再用相同Lomuto partition。後續每次在當前子區間重新均勻選擇。鍵互異時，pivot的秩對1..n均勻，任何固定輸入排列都有同一個期望分析。

這是對演算法內部亂數取期望，不要求使用者資料的排列均勻。某次執行仍可能一直抽到端點而花 $\Theta(n^2)$；下面證明期望值，尚未把它當成每次執行的保證或未證的尾機率界。

理想模型採條件於目前狀態仍均勻的抽樣。固定且公開的偽亂數序列是可重現的實作選擇，其對抗性保證必須另外說明，不能把形式上的random呼叫當作數學均勻性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.4 Analysis of quicksort

令z1<z2<⋯<zn為排序後的不同鍵。對i<j，X_ij為zi與zj在任何一次partition中直接比較的指示變數。每對至多比較一次，因直接比較時至少一個是pivot，該pivot之後不再遞迴。

看秩區間Zi,j={zi,⋯,zj}。在其中任何鍵首次成為pivot前，兩端仍同屬某個子問題；區間外pivot只會整個保留這一段，不能把兩端分開。

若最先被選中的是zi或zj，該次partition直接比較兩端；若先選內部鍵，兩端被分至不同子問題，永不再比較。區間中每鍵等可能先成pivot，所以

`Pr(X_ij=1)=2/(j−i+1)`。

這個事件是「最先選中區間端點」，不是「區間任意元素先成pivot就會比較」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望比較數的精確求和

總比較C=Σ_(i<j)X_ij。無論各X_ij是否獨立，期望線性性都給

E[C]=Σ_(i<j)2/(j−i+1)

=2Σ_(k=2)^n (n+1−k)/k

=2(n+1)H_n−4n，其中H_n=Σ_(k=1)^n1/k。

由ln(n+1)≤H_n≤1+ln n得E[C]=Θ(n log n)。加上partition迴圈與遞迴固定開銷同階，期望時間O(n log n)。不能以「所有比較事件獨立」計算變異數或高機率界，因它們並不獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四個秩的完整比較機率表

元素對	區間大小	比較機率
(1,2)、(2,3)、(3,4)	2	各1
(1,3)、(2,4)	3	各2/3
(1,4)	4	1/2

總期望=3+4/3+1/2=29/6，與2×5×H4−16=29/6一致。兩端1、4若先遇pivot2或3就分離，先遇1或4才比較；四種首選各1/4，所以機率1/2。

相鄰秩必比較不代表所有pair必比較。精確公式也不是每次執行的比較數，例如最壞4鍵需6次，29/6是對所有亂數選擇的平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 07。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

09 中位數與次序統計量

Part II Sorting and Order Statistics

本章定位

選擇問題只需找特定秩，不必先把全部資料排序。本章從極值的精確比較帳出發，統一三路分割與局部秩，完整證明隨機選擇的期望線性時間，再以五元素組的實際排除證據推導最壞線性保證。

前置知識

選擇問題、partition 與機率分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以 ceil(3n/2)-2 次比較同時找最小與最大並證明最壞下界。
- 分析隨機 select 的期望線性時間。
- 用分組中位數建立最壞情況線性選擇。
- 9.1 Minimum and maximum
- 9.2 Selection in expected linear time
- 9.3 Selection in worst-case linear time

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

9.1 Minimum and maximum

第k小指把輸入多重集合排序後第k個值，k採1-index、1≤k≤n。重複鍵佔不同位置但可回傳相同值；偶數n的lower median採k=[n/2]，不必是中間兩值的算術平均。本章鍵必須可全序比較，空輸入或非法k應在入口拒絕。

找minimum的線性掃描維持「目前min是已讀前綴的最小值」，每讀一筆只與min比，共n−1次。下界：除真正minimum外，每個identity都必須至少在一次比較中被證明大於別人；一次比較至多使一個identity首次失去minimum資格，故至少n−1次。Maximum對稱。

同時找兩者不必分開花2n−2次。每對先比出小者和大者，小者只挑戰min，大者只挑戰max。偶數n先用一次比較初始化前兩筆；奇數n先把第一筆同時設為min/max。

成對處理的完整比較帳

例[14,3,9,1,8,5,12,6]。首對14與3用1次，min3/max14。之後每對用「組內1次、min1次、max1次」。

新的一對	組內結果	min比較後	max比較後	累積次數
14,3	小3、大14	3	14	1
9,1	小1、大9	1	14	4
8,5	小5、大8	1	14	7
12,6	小6、大12	1	14	10

偶數n：1+3(n−2)/2=3n/2−2。奇數n：3(n−1)/2。統一為[3n/2]−2，n=1時0次。每對處理後min/max涵蓋更長前綴，初始化、保持與終止證明和單掃描相同，但比較資訊被兩個目標共用。

同時找極值的最壞下界

以下對相異鍵使用可維持一致全序的adversary（對手）回答。每筆初有minimum與maximum兩個候選資格；結束只剩真正min的min資格與真正max的max資格，要消去2n−2個資格。

兩個從未比較的鍵相遇時，可一次消去兩個資格：較大者失去min、較小者失去max。這種比較至多⌊n/2⌋次。其餘比較，對手可讓至多一個新資格消失：

比較前狀態	回答規則
兩者都只可能是max	比出一個較大，消去另一個max資格
兩者都只可能是min	比出一個較小，消去另一個min資格
一個只可能max、一個只可能min	回答前者較大，兩者都不失去資格
有一個從未比較	選結果只消去它的一個資格；另一個至多不新增損失

已失去兩種資格的鍵不可能再造成雙重消去。規則相容於目前比較的偏序：仍可能max的鍵從未輸過，仍可能min的鍵從未贏過，讓前者大於後者不會造成有向循環；未比較鍵沒有既有約束。

若共做c次，最多消去c+⌊n/2⌋個資格，故$c \geq 2n-2-\lfloor n/2 \rfloor = \lceil 3n/2 \rceil - 2$，成對法達到最壞比較數下界。

Tournament還保留了第二小的候選

找minimum的比較可組成平衡淘汰賽，並保存每個winner直接打敗的鍵。第二小只可能在minimum直接打敗的清單中：任何未直接輸給minimum的鍵，都輸給了另一個比它小的非minimum鍵，不能是第二小。

8鍵例的第一輪winner為3、1、5、6，第二輪1、5，第三輪1。冠軍1直接打敗9、3、5，故第二小為這三者的minimum=3。淘汰賽7次、清單再2次，共9次。

一般$n \geq 2$用平衡賽程（不足可安排bye，不與虛構值比較），總比n−1次；冠軍實際對手$d \leq \lceil \log_2 n \rceil$，清單再d−1次，所以最壞至多$n+\lceil \log_2 n \rceil - 2$。這裡鍵相異，或用(key,identity)打破平手來定義第二個identity。

9.2 Selection in expected linear time

Select只遞迴含答案的一側，而quicksort遞迴兩側。要處理重複鍵，本章統一使用三路partition；陣列索引p..r採0-index，但傳入k永遠是「目前子陣列內第k小」的1-index局部秩。

```
partition3(A,p,r,x):
    lt=p; i=p; gt=r
    while i<=gt:
        if A[i]<x: swap(A[lt],A[i]); lt++; i++
        elif A[i]>x: swap(A[i],A[gt]); gt--
        else: i++
    return lt,gt
```

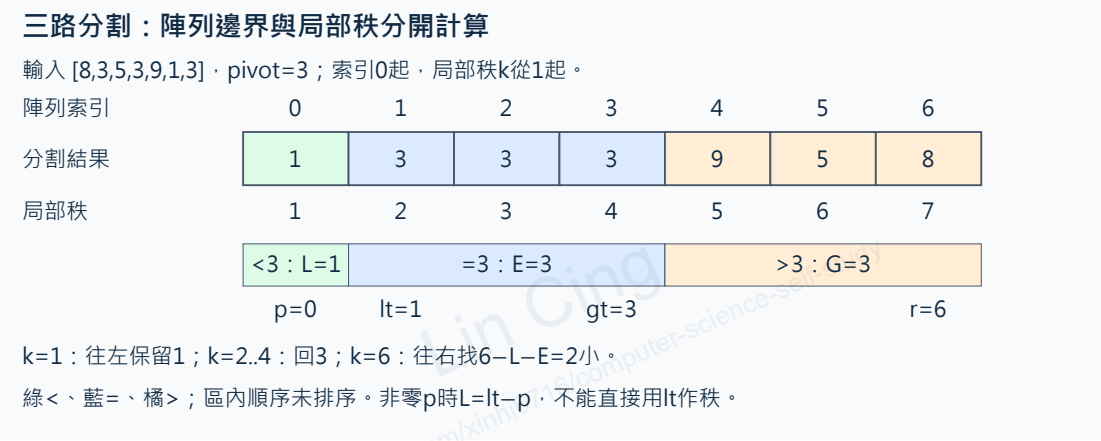
迴圈前A[p..lt−1]<x、A[lt..i−1]=x、A[i..gt]未知、A[gt+1..r]>x。小鍵交換後擴小區與等值區；大鍵從gt換來的是未知鍵，所以i不能前進；等鍵只擴等區。未知區長gt−i+1每輪減1，必終止，且交換保留多重集合。結束得到完整三區，區內未必排序。

局部秩只在往右時扣除兩個區

```
select-random(A,p,r,k):
    assert 1<=k<=r-p+1
    while p<r:
        x=A[uniform-integer(p,r)]
        lt,gt=partition3(A,p,r,x)
        L=lt-p; E=gt-lt+1
        if k<=L: r=lt-1
        elif k<=L+E: return x
        else: k-=L+E; p=gt+1
    return A[p]
```

選pivot是目前某個實際鍵，所以E≥1。左分支有k≤L，右分支有1≤k−L−E≤r−gt，遞迴不可能落空區；各次規模至少少E筆。回傳等值區時，排序後的局部秩L+1..L+E全是x。

局部秩只在往右時扣除兩個區 （續）



此圖A=[8,3,5,3,9,1,3]、x=3得到L=1、E=3、G=3。k=4落等值區直接回3；k=1進左側仍找第1小；k=6進右側找第6−1−3=2小。不要將「原陣列絕對位置」與「目前子區間的局部秩」混用。另一例[3,3,1,3,2,3]可得[1,2,3,3,3,3]，等值閉區間(lt,gt)=(2,5)。

一個需要繼續往右的數例

A=[7,2,9,4,5]找第3小。若先取x=2，分成小區空、等值[2]、大區含[7,9,4,5]，L=0、E=1；往右必須改找第3−1=2小，不能仍找第3小。

若下一次選x=5，右側大區再分成小區[4]、等值[5]、大區[7,9]，局部L=1、E=1，k=2落等值區，回5。每輪只保留含答案的區塊，無須把被丟棄區內部排好。

全等[5,5,5,5,5]只掃一次、等值區涵蓋全部，$\Theta(n)$結束。若改回二路Lomuto的≤版本、找最小秩，每次q=r，正好做4+3+2+1=10次pivot鍵比較；隨機挑哪個5都不能改變這個$\Theta(n^2)$退化。

期望線性的完整代入證明

每次條件於目前陣列，pivot均勻取自各identity，未來選取亦保持條件均勻。先用相異鍵看pivot秩R：R均勻1..n，下一個子問題長度至多m_R=max(R−1,n−R)。

令F(A,k)是該狀態剩餘的期望時間。歸納假設所有較短狀態F(B,j)≤a|B|，本層掃描與固定開銷≤cn。條件於pivot後套歸納，不必假設未知T(n)單調或拆開相依隨機變數：

`F(A,k) ≤ cn+(a/n)Σ_(r=1)^n max(r−1,n−r)`。

n=2h時，此和/n=(3h²−h)/(2h)=3n/4−1/2；n=2h+1時為(3h²+2h)/(2h+1)≤3n/4。因此F≤(c+3a/4)n≤an，只需a≥4c並涵蓋n=1基例，O(n)歸納成立。初次必掃n筆給Ω(n)，故期望Θ(n)。

不能直接寫E[T(n)]≤E[T(3n/4)]+cn當成已證事實；單次pivot可能極差，這裡是對所有可能秩的後續成本平均。

重複鍵與最壞情形的界線

三路版本的同一證明可包含重複鍵：只為分析替相等鍵附上固定identity次序，建立唯一秩R，實際partition仍只比較鍵。若答案在<x側，其長度≤R−1；若在>x側，其長度≤n−R；若在=x側立刻終止。因此每種pivot identity仍有同一個m_R上界，平均歸納照用，不需要真的排序identity。

最壞執行仍可Θ(n²)：相異鍵、找minimum、每次剛好抽到最大鍵，規模依序n,n−1,⋯。這事件雖小但可能，不能以期望線性取代最壞保證。迭代式不保留遞迴frame；原地partition加固定變數只需O(1)額外空間。

研究中位數或Top-k時，排序是可行的O(n log n)方法，但不是必要成本。Select回傳一個次序統計量不會同時交付完整排序，故不受排序Ω(n log n)下界限制。

9.3 Selection in worst-case linear time

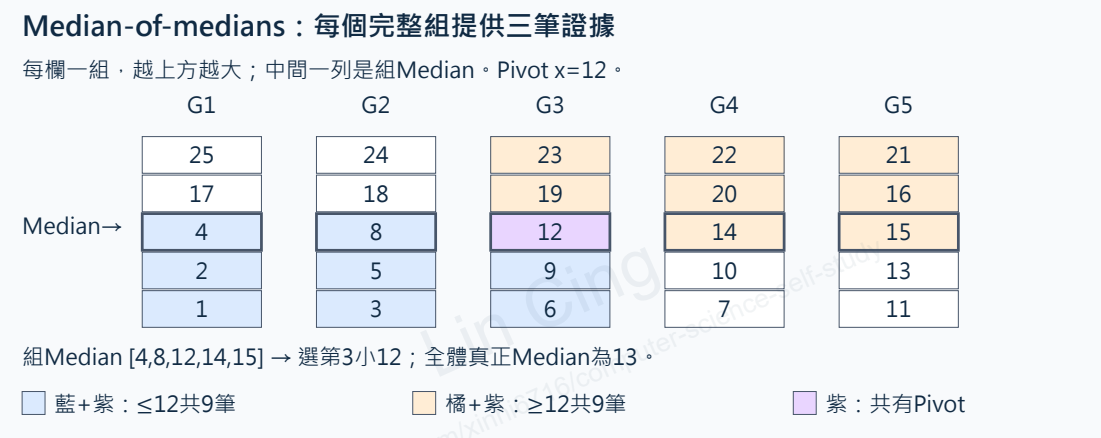
Deterministic select以median-of-medians挑pivot：n≤5時直接排序回第k小；否則按目前位置每5筆一組，最後可不足5筆。各組排序取lower median，再遞迴select這g=⌈n/5⌉個median的第⌈g/2⌉小作x。

以x做同樣的三路partition，若k落等值區回x，否則只遞迴嚴格小側或大側、依前頁規則換算局部秩。這次用來挑pivot的遞迴與用來找答案的遞迴，都必須納入成本。

各組至多5筆，排序每組O(1)，全部O(n)。x必是原輸入中的值，E≥1，故演算法正確與終止仍由相同分割不變量和長度歸納保證；效率另靠以下排除數量證明。

Lin Ching
https://github.com/xinhi6716/computer-science-self-study

五元素組如何提供排除證據



原創25筆的五組排序後如圖，組median為[4,8,12,14,15]，median-of-medians是12，不是全體真正median=13。它只需讓兩側足夠小，不必一次找到目標。

圖中m≤12的前三組，每組下面三個值都≤12；m≥12的後三組，每組上面三個值都≥12。這些是可逐格核對的9筆證據，兩組證據在pivot處重疊沒有問題，因分析的是兩個互斥遞迴分支。

五元素組如何提供排除證據 （續）

若值有重複，保證是≤x/≥x，不一定嚴格；三路版本遞迴嚴格區時，整個等值區一併排除，所以此非嚴格證據仍能控制子問題大小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取整、殘組與線性時間歸納

至少[g/2]個組median≥x，也至少[g/2]個≤x（g偶數採lower median仍滿足）。保守扣掉可能不足5筆的殘組、以及含所選pivot identity的那組，剩至少[g/2]−2個完整組，每組提供3筆對應的非嚴格大小證據。

因此每一側能被排除至少`3([g/2]−2) ≥ 3n/10−6`筆；嚴格遞迴區至多7n/10+6筆。當右式排除下界為負時只是無資訊的保守界，不代表演算法真的擴大問題。

令非遞迴工作≤cn。歸納假設較短輸入時間≤an，對n≥140有

`T(n) ≤ a⌈n/5⌉+a(7n/10+6)+cn ≤ (9a/10+c)n+7a`。

若a≥20c，則a/10−c≥a/20，而n≥140給(a/10−c)n≥7a，故T(n)≤an。有限的1≤n<140狀態可藉增大a涵蓋；140是證明門檻，不要求實作真的把140筆全部當base排序。兩個遞迴的規模都小於n，歸納有效。初次處理n筆又給Ω(n)，合為最壞Θ(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個第16小的完整選擇路徑

本例用保留掃描順序的L/E/G列表呈現，方便讀者核對；原地三路交換可有不同區內排列，但區長與rank換算相同。

當前n,k	分組median→所選x	L,E,G大小	下一步
25,16	[4,8,12,14,15]→12	11,1,13	大區找第4小
13,4	[19,20,16]→19	6,1,6	小區找第4小
6,4	[15,16]→lower median15	2,1,3	大區找第1小
3,1	base排序[16,17,18]	—	回16

第一次大區依序[17,25,18,24,19,23,14,20,22,13,15,16,21]，分5、5、3筆得到median19、20、16。接著小區[17,18,14,13,15,16]，前5筆median15、殘組median16；x=15之後大區為[17,18,16]。最後得到16，與完整排序只作驗算的第16格一致。

組大小改變時，證明也要改

若用固定奇數組大小s=2t+1，理想化大n時每半組median提供t+1筆排除證據，兩個遞迴的規模比例和約 `1/s + (1−(t+1)/(2s)) = 1+(1−t)/(2s)`。

s=5時t=2，和9/10<1，才有線性代入的餘裕；s=7時和6/7<1也可線性，但常數與殘組需重算。s=3時和1，只由這個粗界得到T(n)≤T(n/3)+T(2n/3+O(1))+O(n)，可證O(n log n)，卻不能單憑上界遞迴式斷言實際演算法必然Ω(n log n)。若主張非線性，需要能讓壞分割跨遞迴層持續發生的額外達界證明。

基礎實作用列表保存分組、median與L/E/G，峰值額外空間O(n)，因沿任一尚未返回的遞迴路徑規模幾何縮小；要宣稱原地O(1)空間，還需另設計median搬移與遞迴stack管理，本章不把這項額外保證混入時間定理。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 09。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30 多項式與快速傅立葉轉換

Part VII Selected Topics

本章定位

多項式乘法藉相異點求值與插值轉成逐點乘法；本章從單位根正交性證明DFT與inverse，再逐層核對偶奇分解、bit-reversal、蝶形索引與補零，連結精確算術和浮點限制。

前置知識

複數根、點值表示與遞迴分治。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較係數與點值表示的乘法成本。
- 推導 DFT 與逆 DFT。
- 實作 radix-2 FFT 並理解單位根條件。
- 30.1 Representing polynomials
- 30.2 The DFT and FFT
- 30.3 Efficient FFT implementations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.1 Representing polynomials

係數表示$A(x)=\Sigma_{j=0}^{n-1}a_jx^j$的degree bound是n，也就是degree<n；最高係數可以為0，實際degree可能更小。零多項式在程式可用單一係數[0]表示，不把degree bound與向量中第一個非零位置混淆。以下係數依**低次到高次**排列。

同一degree bound的加法逐係數相加，$\Theta(n)$。在一點z求值可用Horner：先$v=a_{n-1}$，再對j=n−2,⋯,0做$v=a_j+zv$，每次保持$v=\Sigma_{r=j}^{n-1}a_rz^{r-j}$，最後得到A(z)，共n−1次乘與n−1次加。任選n個點分別做Horner，則$\Theta(n^2)$，不是$\Theta(n)$。

兩個長度r、s係數陣列相乘，$c_k=\Sigma_{i+j=k}a_ib_j$，輸出長r+s−1；直接卷積有rs次係數乘法，$\Theta(rs)$。FFT將改善的是整個表示轉換加乘法流程，不只是中間的pointwise product。

相異點值的存在性與唯一性

給n個相異$x_0,\cdots,x_{n-1}$及值$y_0,\cdots,y_{n-1}$。Lagrange公式$A(x)=\Sigma_k y_k\ell_k(x)$，其中$\ell_k(x)=\Pi_{j\neq k}(x-x_j)/(x_k-x_j)$，每個分母非零、degree<n，且$\ell_k(x_i)$只有i=k時為1，其餘為0，故插值多項式存在。

若兩個degree<n多項式同時吻合，差H有n個不同根。非零degree d多項式至多d根：遇根α可依因式定理寫$H=(x-\alpha)Q$，其餘不同根亦為Q的根，對degree歸納即可。因此H必為零，唯一性成立。少於n點時，可在一個解上加任意常數倍Π已知點$(x-x_j)$，仍吻合，故一般不唯一。

矩陣寫法是$V[i,j]=x_i^j$、$Va=y$，V稱Vandermonde matrix。其$\det=\Pi_{j<k}(x_k-x_j)$：兩個x相同會有相同row，故每個差都是det多項式的因子；總degree正好$n(n-1)/2$，對照單項$x_1x_2^2\cdots x_{n-1}^{n-1}$的係數1便確定常數。因此相異點讓V可逆，也可用一般線性系統求插值，但直接LUP是$\Theta(n^3)$。

公式也可在$O(n^2)$算術操作內轉成係數：先逐因子建$Q(x)=\Pi_j(x-x_j)$，每步更新總$O(n^2)$；對每個k用synthetic division在線性時間求$Q/(x-x_k)$，分母各用O(n)算，再逐係數加總。若每次從頭重建每個ℓ_k的多項式，則不能直接宣稱相同的二次成本。

點值相乘需要足夠長度

在**相同且相異的N個點**存A、B的值，$C(x_k)=A(x_k)B(x_k)$只需N次乘法。要唯一重建乘積，須$N \geq r+s-1$，並將兩個係數陣列都補零到N；radix-2選不小於該值的2冪。

例如$A=1+2x$、$B=1-x$，$r=s=2$，乘積$1+x-2x^2$需3係數，所以選$N=4$。不能只因兩個輸入都長2就直接做2點轉換。

若在N次單位根只留N點，$x^N=1$，所以插值得到的是模x^N-1的循環卷積：位置k收到$c_k+c_{k+N}+c_{k+2N}+\cdots$，高次項會繞回。例$(1+x)^2=1+2x+x^2$，$N=2$時$x^2$與常數無法區分，反轉只得[2,2]；[1,2,1]才是普通乘積。補零不是可有可無的格式處理。

30.2 The DFT and FFT

本章固定**正指數forward**：$\omega_n=e^{2\pi i/n}$，$y_k=\sum_{j=0}^{n-1}a_j\omega_n^{jk}=A(\omega_n^k)$。其他領域常把forward指數設負；只要正逆成對就成立，但不能混用兩種慣例。

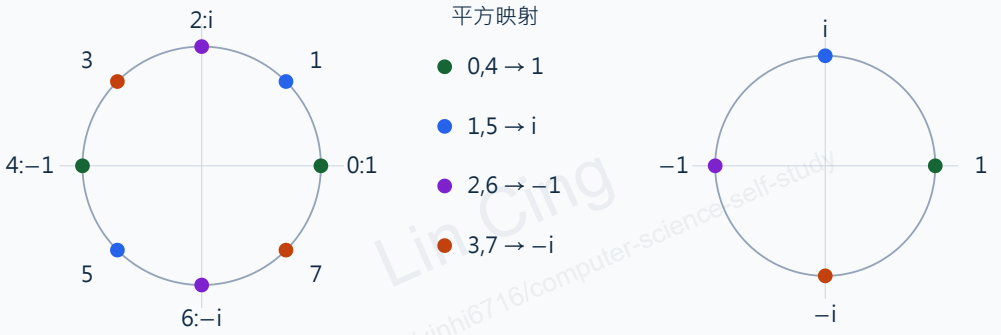
ω_n是primitive root，亦即$\omega_n^k=1$當且僅當n整除k。其n個冪相異，構成合法插值點。對正整數d有$\omega_{dn}^{dk}=e^{2\pi ik/n}=\omega_n^k$（cancellation）。n偶數時$\omega_n^{n/2}=-1$，且$(\omega_n^k)^2=\omega_{n/2}^k$；k與k+n/2的平方相同，全部n個點平方後只剩n/2個不同點，各出現兩次（halving）。

幾何級數給正交性：$\sum_{j=0}^{n-1}\omega_n^{jt}$在$n \mid t$時每項1、和n；否則$\omega_n^t \neq 1$，和$=(1-(\omega_n^t)^n)/(1-\omega_n^t)=0$。分母是否非零正是primitive條件要處理的地方，不能把$\omega=1$當成任意n的替代root。

30.2 The DFT and FFT （續）

八個單位根平方後，恰剩四個不同點

左標k表示ω_8^k；同色相對兩點平方後，落到右側同色的一點。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正交性推導 Inverse DFT

定義$F[k,j]=\omega_n^{jk}$。令$G[j,k]=(1/n)\omega_n^{-jk}$，則$(GF)[j,\ell]=(1/n)\Sigma_k\omega_n^{k(l-j)}$。j=ℓ時為1，其餘因$0\le j,\ell<n$而n不整除ℓ−j，為0，所以GF=I，G就是$F^{-1}$。

因此$a_j=(1/n)\Sigma_k y_k\omega_n^{-jk}$：使用負指數twiddle，**最後除n一次**。也可每層分攤正規化，但不能每層除n，更不能完全漏除。

若錯把forward再做一次後除n，利用同一和式得到F^2a/n的第j項是a[(-j) mod n]，是循環索引反轉；對任意複數輸入不等於「取共軛」。n=4、a=[2,5+i,7,−1]會變[2,−1,7,5+i]，可直接分辨兩種錯法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

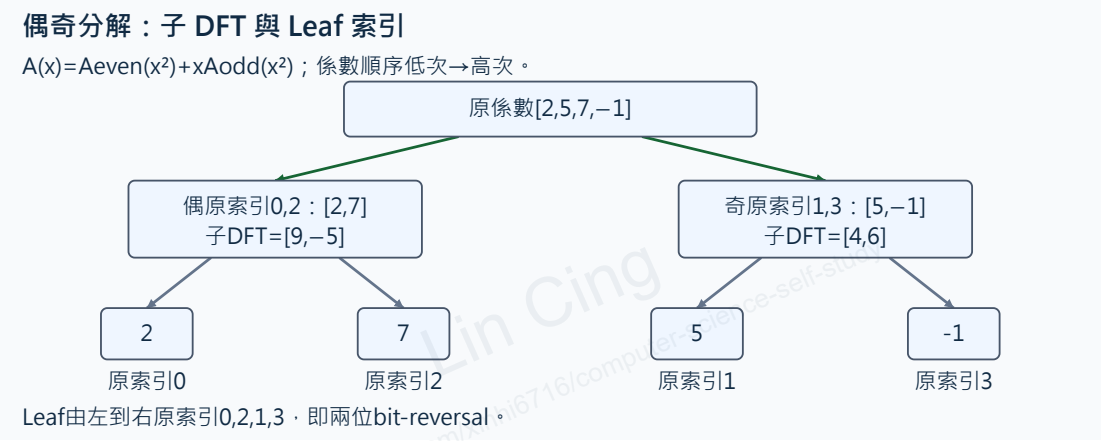
偶奇拆解的兩個子問題

設n是正2冪，$A_e(z)=a_0+a_2z+\cdots$、$A_o(z)=a_1+a_3z+\cdots$，則$A(x)=A_e(x^2)+xA_o(x^2)$。由halving lemma，在全部n個root求A只需兩個n/2點DFT E、O。

對$0\le k<n/2$，$y_k=E_k+\omega_n^kO_k$，$y_{k+n/2}=E_k-\omega_n^kO_k$，因$\omega_n^{k+n/2}=-\omega_n^k$。一次算$t=\omega_n^kO_k$，使用加、減產生兩個輸出，就是butterfly。遞迴基例n=1直接回$[a_0]$；歸納子轉換正確，再用等式合併全部n個值，證明整個FFT正確。

取a=[2,5,7,−1]。偶係數[2,7]的2點DFT為[9,−5]；奇係數[5,−1]為[4,6]。k=0用twiddle1，得$y_0=13$、$y_2=5$；k=1用i，得$y_1=-5+6i$、$y_3=-5-6i$。四個中間值彼此不同，方便分辨接線是否真正正確。

偶奇拆解的兩個子問題 （續）



遞迴算法與完整成本

```
FFT(a,sign):
  n=len(a)；要求n為正2冪
  if n==1: return [a[0]]
  E=FFT([a[0],a[2],...], sign)
  O=FFT([a[1],a[3],...], sign)
  w=1; wm=exp(sign*2πi/n)
  for k=0..n/2-1:
    t=w*O[k]
    y[k]=E[k]+t; y[k+n/2]=E[k]-t
    w=w*wm
  return y
```

forward取sign=+1；inverse取−1，最外層回來後除n。每層Θ(n)拆分/合併，T(n)=2T(n/2)+Θ(n)，$\log_2 n$層，Θ(n log n)複數算術。n/2 butterflies每層，共(n/2)$\log_2 n$個。

複製偶奇陣列且及時釋放子結果的順序實作，可用O(n)峰值額外空間與O(log n)stack；若保存所有層的陣列歷史則O(n log n)。不能只由遞迴深度就宣稱全部記憶體O(log n)，因每層還有輸出與子陣列。

卷積的 Forward、點乘、Inverse 全部算值

A=[1,2,0,0]、B=[1,−1,0,0]，N=4、ω_4=i。

k	DFT(A)	DFT(B)	逐點乘積
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

對[0,3+i,−2,3−i]做負號DFT，未除4的結果[4,4,−8,0]；除4得[1,1,−2,0]，截取前3項即$1+x-2x^2$。兩次forward加一次inverse與Θ(N)點乘，總Θ(N log N)，輸出degree bound不足N的條件使循環與普通卷積一致。

卷積的 Forward、點乘、Inverse 全部算值 (續)

一次卷積的全部頻點與反插值

A=1+2x，B=1−x，均補到N=4；$\omega_4=i$，forward取正指數。

k	DFT(A)	DFT(B)	逐點乘積
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

負號DFT先得[4,4,−8,0]，除4→[1,1,−2,0]。

綠格 = 乘積點值；取前3個係數，得到$1+x-2x^2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.3 Efficient FFT implementations

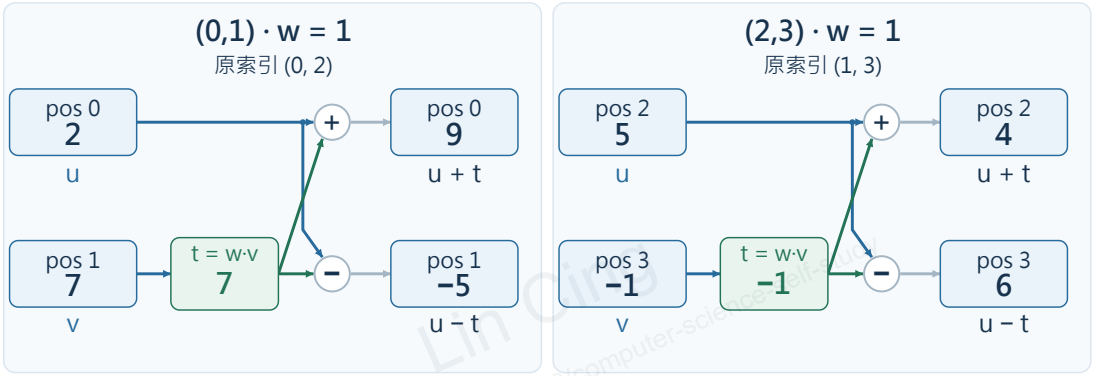
遞迴每層先取偶索引再取奇索引，也就是先看原索引最低位，再看下一位。若$n=2^b$，從左到右的leaf順序就是把原索引b個bits反轉後的順序。rev是自身的逆，因此可寫A[rev(j)]=a[j]，也可寫A[j]=a[rev(j)]。

n=4：原索引00,01,10,11反轉為00,10,01,11，輸入[2,5,7,−1]重排為[2,7,5,−1]。重排後stage1配對的是**實體位置(0,1)、(2,3)**，分別含原索引(0,2)、(1,3)；不能把「原索引」與「目前陣列位置」寫成同一個欄位。

stage1兩個twiddle皆1，實體陣列變[9,−5,4,6]。stage2的m=4，配對實體位置(0,2)與(1,3)，依序使用1與i，輸出按位置0,1,2,3是[13,−5+6i,5,−5−6i]。stage2若又配相鄰位置就是另一張圖、另一個運算。

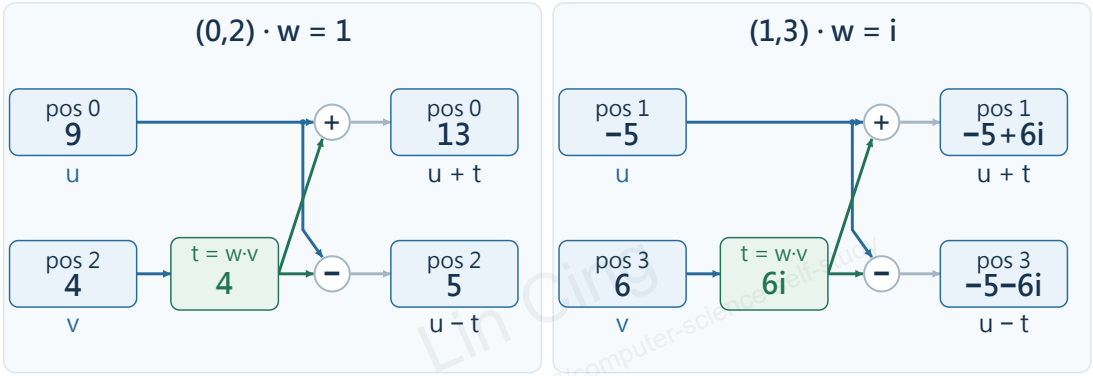
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.3 Efficient FFT implementations （續）



Stage1：實體陣列 [2,7,5,−1] → [9,−5,4,6]

Stage2 必須跨半塊合併



Stage2：依實體位置0,1,2,3輸出 [13,−5+6i,5,−5−6i]

j	u=A[j]	t=ω₄ʲA[j+2]	新A[j]	新A[j+2]
0	9	1 • 4=4	13	5
1	−5	i • 6=6i	−5+6i	−5−6i

Stage2 必須跨半塊合併 （續）

另一組a=[1,2,0,−1]依同一規則：bit-reversal→[1,0,2,−1]，stage1→[1,1,1,3]，stage2→[2,1+3i,0,1−3i]。這個例子有多個相同1，單看最終數字可能掩蓋接線錯誤；上面的不同中間值讓每條運算的來源更清楚。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Iterative FFT 的實體索引契約

```
將a按bit-reversal放入A
for s=1..log₂n:
  m=2^s; wm=exp(sign*2πi/m)
  for base=0,m,2m,...,n−m:
    w=1
    for j=0..m/2−1:
      u=A[base+j]; t=w*A[base+j+m/2]
      A[base+j]=u+t
      A[base+j+m/2]=u−t
      w=w*wm
inverse時最後每項除n
```

每輪s開始，長度m區塊的左右半塊各已是相應偶／奇子問題的DFT；butterfly做與遞迴相同的合併，輪末每塊為完整m點DFT。s=1時半塊為單點，基例成立；最後m=n完成整體。先保存u再覆寫上格，否則下格的u−t會誤讀已更新值。

每stage恰n/2個butterflies，共Θ(n log n)。逐索引反轉b位元的簡單重排需O(n log n)位元步驟；可用額外n長陣列，或利用rev的2-cycle就地交換。twiddle只依m與j，不依base，同stage各block可共用其表，若存所有stage表總Θ(n)項；這不改變主要算術級序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Parallel FFT Circuit 的 Work 與 Depth

固定twiddle並把bit-reversal視為接線的bounded-fan-in電路，每stage n/2個互不衝突butterflies可同時做；每stage的輸出只有下一stage才讀。n=8有3stage、共12個butterflies，電路work $\Theta(n \log n)$、depth $\Theta(\log n)$，因每條輸入到輸出的路徑穿過log n次合併。

這個depth計算假設一次複數乘、加各為常數層，且沒有把twiddle串行生成、處理器通訊或實體wire長度藏在其中。軟體多執行緒實作若共用陣列，須在每stage結束同步；同stage各butterfly寫不同兩格，下一stage讀前一stage結果，才能使用相同DAG分析。若僅有P處理器，work下界W/P仍在，不會因電路畫成log n層就自動只需log n時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

精確算術與浮點 FFT 的界線

$\Theta(n \log n)$是複數算術操作模型；浮點root與每次乘加都有誤差。整數係數卷積最後四捨五入要有「每格絕對誤差<1/2」的保證，不能只因少數round-trip吻合就保證任意長度／大係數都精確。

一種精確替代是在質數域F_p用NTT：要求$N \mid (p-1)$，選order恰為N的ω，則N在域中可逆，幾何和與inverse proof仍成立。**複合模數上僅宣稱primitive並不自動保證所需分母都是unit**，這裡的敘述限定質數域。

同一$A=[1,2,0,0]$、$B=[1,-1,0,0]$，$p=17$、$N=4$、$\omega=13$，$\omega^2=16\neq 1$、$\omega^4=1$，且$4^{-1}=13$。$NTT(A)=[3,10,16,9]$，$NTT(B)=[0,5,2,14]$，點乘mod17得$[0,16,15,7]$，inverse得$[1,1,15,0]$。把15解碼為-2，還原原乘積。

解碼也需要幅度界：若輸入整數係數$abs \le B$，長度r,s，每格卷積$abs \le \min(r,s)B^2$；取$p>2\min(r,s)B^2$才可從單模數唯一還原有號係數。本例$r=s=2$、$B=2$，p17>16足夠。單有模數正確結果，不代表任意大整數的原值沒有繞回。

NTT的FFT級序以mod p算術為單位；若p隨資料幅度增長，位元成本還需計入log p位元整數的乘加，不能把任意長整數乘法當作一次固定時間操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 30。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15 動態規劃

Part IV Advanced Design and Analysis Techniques

本章定位

動態規劃的重點，是把「剩下要解什麼」定義清楚，再把同一個子問題的答案存下來。本章從木料切割開始，接著處理矩陣鏈、最長共同子序列與最佳二元搜尋樹。每個問題都要回答三件事：狀態記什麼、為什麼轉移不漏解，以及算完最佳值後如何找回具體答案。

前置知識

遞迴、最佳化與子問題重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 rod cutting 與 LCS 的最佳子結構。
- 用維度與計算次序設計 DP 表。
- 推導 matrix-chain 與 optimal BST 的區間 DP。
- 由 choice 表完成重建，並判斷 DP 條件在陌生變形中是否仍成立。
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Rod cutting

有一根長度為n的木料，長度i的整段售價是p_i。每段長度必須是正整數，可以完全不切，但整根木料都必須賣出；本節先不計切割費用。我們要找的是總售價最高的切法。

先固定第一段的長度i：這一段收入p_i，剩下的n−i仍是同一種切割問題。令r[n]表示長度n能得到的最高收益，空木料的收益r[0]=0，因此

$$r[n] = \max_{1 \le i \le n} \{p_i + r[n-i]\}.$$

為什麼剩下的部分也要用最佳切法？ 假設一個最佳方案先切下長度i，但後面的切法不是長度n−i的最佳方案。把後半段換成更高收益的切法，第一段與總長度都不變，總收入卻提高了，與原方案最佳矛盾。

反過來，每個候選p_i+r[n−i]都對應一種合法切法，所以候選最大值不可能超過真正的最佳收益。前一段證明最佳方案也一定出現在候選裡；兩個方向合起來，才得到這個等式。

若每次都重算相同長度，呼叫數滿足$T(0) = 1$、$T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n$。這裡T數的是函式呼叫，不是收益。把每個長度只算一次後，轉移總數降為$\sum_{j=1}^{n} j = \Theta(n^2)$。

由短到長填表，再保存第一段的選擇

偽碼用p[1]表示長度1的售價，依長度由短到長計算。choice[length]另外保存最佳方案的第一段長度，讓我們最後能重建切法。非空木料的初始收益要設為−∞：即使所有售價為負，題目仍要求整根賣出，不能用0偷偷代表丟棄。

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

進入某個length時，更短的長度都已算好。內圈把每一種第一段長度都比較完，才確定r[length]與choice[length]。重建時，每取出一段，剩餘長度就至少減1；到0便完成，最多輸出n段。

Rod Cutting 的完整表與切割重建

取 `p=[1,5,8,9]`，`choice[n]` 記錄取得最大值的第一段長度：

n	候選 p_i+r[n-i]	r[n]	choice[n]
0	空	0	0
1	1	1	1
2	5, 1+1=2	5	2
3	8, 1+5=6, 5+1=6	8	3
4	9, 1+8=9, 5+5=10, 8+1=9	10	2

回溯由 n=4 開始：`choice[4]=2`，收益加上 `p_2=5`，剩長 2；`choice[2]=2`，再加 5，剩 0。故最佳切法 2+2，總收益 10。表已算完才回溯，不能邊猜切點邊改動已定的最優值。

若每刀收固定費用c，只在切完第一段後仍有剩料時扣c，仍能用「剩餘長度」當狀態。若改成最多只能切成k段，未來能否再切就取決於已用段數，狀態必須多記這項資訊。判斷是否增加維度，要看它會不會改變後續的合法選擇。

15.2 Matrix-chain multiplication

矩陣的先後順序固定；我們只能選括號，不能重排矩陣。設A_i的維度是$p_{i-1} \times p_i$。若最後一次乘法把區間切在k，左側結果為$p_{i-1} \times p_k$，右側為$p_k \times p_j$，所以這個切點的總成本是

$$c_k = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j.$$

前兩項是左右子鏈已付出的成本，最後一項才是本次相乘新增的純量乘法次數。枚舉$k = i, \dots, j-1$後取最小：$m[i,j] = \min_k c_k$。單一矩陣不需要乘法，所以$m[i,i] = 0$。

固定最後切點k後，左右結果的維度已經確定。如果某一側還能用更便宜的括號化，就能替換那一側，而不改變最後一次乘法的維度與成本；整體也會變得更便宜。因此最佳方案的左右子鏈必須各自最佳。另一方面，每一種完整括號化都有最後切點，枚舉所有k便不會漏解。

依區間長度l=j−i+1遞增填表，計算某一格時，左右子鏈都比它短，答案已經存在。split[i,j]保存達到最低成本的切點。共有Θ(n²)個區間，每格最多比較O(n)個切點，因此時間上界為O(n³)。

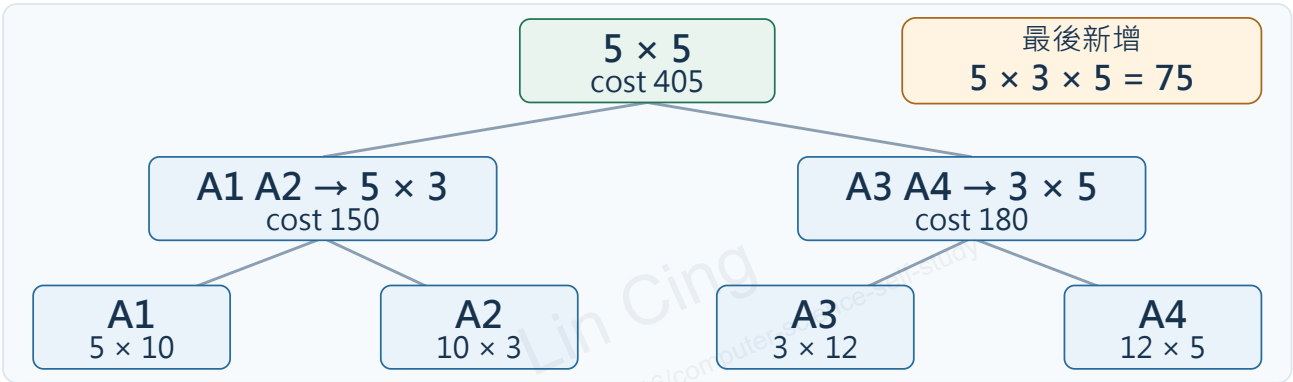
這個版本實際比較的切點總數是$\sum_{l=2}^{n}(n-l+1)(l-1) = n(n^2-1)/6$。每個候選只做常數次讀表與算術，所以時間是$\Theta(n^3)$，不只有一個上界。

Matrix-Chain 的填表與括號輸出

```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

每項轉移皆讀取嚴格較短的區間，所以 length 的順序合法。重建樹有 n 葉、n−1 個內節點，輸出括號需 Θ(n) 次節點操作；不含實際矩陣元素的乘法。

Matrix-Chain 的填表與括號輸出 （續）



150 + 180 + 75 = 405；累計成本不等於本次乘法成本

Matrix-Chain 的完整表與括號重建

取 `p=[5,10,3,12,5]`，A1.A4 的 m 上三角如下：

j-i+1	值
1	m11=m22=m33=m44=0
2	m12=150、m23=360、m34=180
3	m13=330、m24=330
4	m14=405

例如 `m24=min(0+180+10*3*5, 360+0+10*12*5)=330`，split[2,4]=2；`m14=min(0+330+5*10*5, 150+180+5*3*5, 330+0+5*12*5)=405`，split[1,4]=2。

回溯 `split[1,4]=2`，得到 `(A1A2)` 與 `(A3A4)`；兩段各以 `split[1,2]=1`、`split[3,4]=3` 展開，最終 `((A1A2)(A3A4))`。

15.3 Elements of dynamic programming

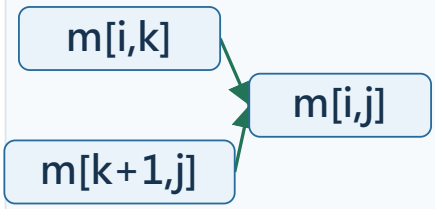
狀態必須保留後續決策需要的資訊。替一個大問題命名還不夠：如果兩種情況被放進同一格，卻有不同的合法後續選擇，這一格就無法代表它們。本章採用可以依序填表的DP；證明時要交代以下四件事。

1. 最優解去掉第一個決策後，剩下的是對應子問題的最優解。
2. 每個原問題候選決策都能由已定義的子狀態組合，且沒有漏掉合法解。
3. 基底值正確，且依賴圖無環。
4. 拓撲次序保證填值時所有先決狀態已經算好。

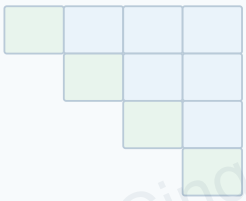
最佳子結構說明「為什麼可以用子問題的最佳解組合」，重疊子問題則說明「為什麼存表能省下重算」。兩者的工作不同。若狀態依賴有環，不能直接套本章的一次填表流程。要重建答案，可以另存選擇表，也可以回頭檢查完整值表；只剩最後一個最佳值，通常不足以找回整個方案。

15.3 Elements of dynamic programming （續）

1 子問題 → 父問題



2 依長度填表



length = 1 → 2 → 3 → 4

3 choice → 具體解

choice[4] = 2

4 → 2 → 0

choice[2] = 2

pieces = [2,2]

總時間 = 狀態處理成本 + 所有轉移成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態圖決定快取的成本

rod 的依賴圖有 n+1 個狀態，長度 j 向 0..j−1 各有一個依賴邊，共 n(n+1)/2 條。若每狀態與每條邊只處理常數次，時間是 Θ(V+E)=Θ(n^2)。

自頂向下 memo 在第一次計算狀態時展開全部候選，之後立即返回已存值。visited 必須獨立於數值，例如以 None 區分未算；不能把合法的負收益當作「未計算」。由小到大填表沒有遞迴堆疊，memo 最多有 n 層呼叫堆疊，兩者另存表均 Θ(n)。

最長簡單路徑說明了狀態遺漏資訊的後果。兩段各自最長的簡單路徑u→v與v→w，可能都經過頂點z；拼起來就重複z，不再簡單。只記起終點無法排除這種情況。加入已訪問集合S可以修正，但狀態數會變成指數；若輸入圖本來就是DAG，則可直接依拓撲順序求最長路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.4 Longest common subsequence

令$c[i,j]$表示兩個前綴$X[1..i]$、$Y[1..j]$的最長共同子序列長度。子序列保留字元順序，但可以跳過字元，不要求連續。c表先回答長度；實際字串稍後再回溯。任一前綴為空時，長度為0。

先看兩個末字元相同的情況，記共同字元為c。如果一個LCS不以c結尾，就不會使用任一輸入的末位置，於是還能在後面接c，與「最長」矛盾。若它以c結尾，可以把最後一次配對放到這兩個末位置；拿掉c後，剩下的字元都落在兩個較短前綴裡。

若兩個末字元不同，任何共同子序列都不可能同時使用這兩個末位置：兩者一旦使用，都必須是共同序列的最後一個字元，卻又互不相同。因此至少有一個末位置未被使用，只需比較「去掉X末字元」與「去掉Y末字元」兩個子問題。

`c[i,j]=max(c[i-1,j], c[i,j-1])`。

任一共同子序列都落在其中一個分支，而每個分支的解也都是原問題的合法子序列，所以取兩者最大值正確。每格只需常數次比較；兩個字串皆非空時，完整填表時間為Θ(mn)。空字串可直接回傳長度0。

LCS 的兩向界與完整實作

末字元相同時，刪去最後一次配對，得到的長度至多是$c[i-1,j-1]$；反過來，任何較短前綴的LCS都能接上這個共同末字元。因此$c[i,j]=c[i-1,j-1]+1$。這裡需要的是一條正確的最佳配對路徑，不必聲稱所有最佳配對都經過同一格。

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

完整c表已經足以判斷回溯方向，不一定要另存方向表。若只求長度，可沿較短字串保留兩列，空間為O(min(m,n)+1)。不過，上面這段回溯會讀取先前各列，不能直接把完整表丟掉後照用。

LCS 的完整表與回溯

X=ABCBDAB、Y=BDCABA，本例 c[7,6]=4。加上空前綴後的 c 表為：

```
      ε B D C A B A
ε     0 0 0 0 0 0 0
A     0 0 0 0 1 1 1
B     0 1 1 1 1 2 2
C     0 1 1 2 2 2 2
B     0 1 1 2 2 3 3
D     0 1 2 2 2 3 3
A     0 1 2 2 3 3 4
B     0 1 2 2 3 4 4
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCS 的完整表與回溯 （續）

回溯時，表格中的「上」代表 i 減一，「左」代表 j 減一：

當前格	動作	下一格	收集的反序字元
(7,6)	上，兩候選同為 4	(6,6)	空
(6,6)	A=A，左上	(5,5)	A
(5,5)	D≠B，上方值 3 較大	(4,5)	A
(4,5)	B=B，左上	(3,4)	AB
(3,4)	C≠A，左方值 2 較大	(3,3)	AB
(3,3)	C=C，左上	(2,2)	ABC
(2,2)	B≠D，左方值 1 較大	(2,1)	ABC
(2,1)	B=B，左上	(1,0)	ABCB

反向輸出 BCBA。每步至少減一個索引，故回溯 O(m+n)，即使原表有 mn 格也不必掃完。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

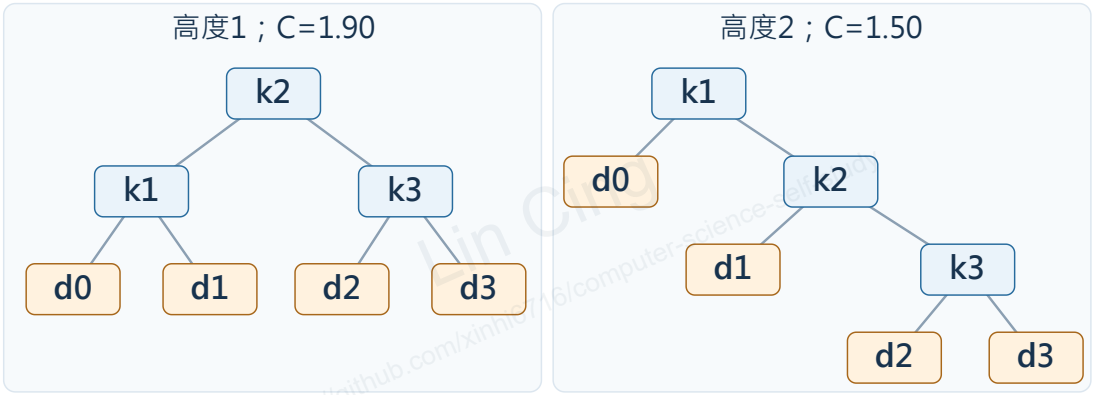
LCS 的完整表與回溯 (續)

i ↓ / j →	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	**4**

綠箭頭：回溯；橘邊：取字元；A,B,C,B 反轉 → BCBA

例A：樹高較小，不代表加權搜尋比較較少

查詢機率為：p=(0.50,0.10,0.05)，q=(0.15,0.10,0.05,0.05)。根深度0；高度只算實鍵間的邊；C只計實鍵比較。



C：只計實鍵比較；dummy 不再多算一次

例A：每一種成功與失敗查詢都要加權

成功搜尋比較數為鍵深度加1；失敗搜尋比較數為dummy深度，**不多算一次dummy檢查**。

查詢	機率	平衡樹：比較數 × 機率	右斜樹：比較數 × 機率
k1	0.50	2 × 0.50 = 1.00	1 × 0.50 = 0.50
k2	0.10	1 × 0.10 = 0.10	2 × 0.10 = 0.20
k3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d0	0.15	2 × 0.15 = 0.30	1 × 0.15 = 0.15
d1	0.10	2 × 0.10 = 0.20	2 × 0.10 = 0.20
d2	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
總計	1.00	**C = 1.90**	**C = 1.50**

成功成本：1.20／0.85；失敗成本：0.70／0.65。查詢常落在k1或d0，右斜樹讓這些高機率結果較早結束，足以補償其他結果多做的比較。

15.5 Optimal binary search trees

給定互異且已排序的鍵k1<k2<⋯<kn，我們要選一棵BST，讓指定查詢分布下的平均搜尋成本最低。鍵的順序固定，能改的是誰當根、左右子樹怎麼安排。成功查詢與失敗查詢都必須計入。

查到鍵k_i的機率為p_i。找不到的查詢按排序間隙分成$d_0,\dots,d_n$，機率為$q_0,\dots,q_n$：例如d_0在最小鍵之前，d_i在相鄰鍵k_i,k_{i+1}之間。這些虛葉（dummy leaves）代表搜尋停止的空位置，不是一筆要比較的真實資料。所有p與q的總和為1。先定義把虛葉步驟也計入的擴充成本E：

$$E(T)=\sum_{r=1}^{n} p_r(\operatorname{depth}_T(k_r)+1)+\sum_{r=0}^{n} q_r(\operatorname{depth}_T(d_r)+1).$$

令$e[i,j]$為鍵$k_i..k_j$這段區間的最低擴充成本；$w[i,j]$是該區間全部成功鍵與失敗間隙的機率總和。即使沒有任何鍵，仍有一個失敗間隙，所以空段的初始化是

$$e[i,i-1]=w[i,i-1]=q_{i-1}.$$

如果選k_r當根，兩側子樹的每種查詢結果都要先經過這個根，深度因此各增加1。左右子問題的既有成本可以保留，再加上整段機率總和w[i,j]：

$$e[i,j]=\min_{i\le r\le j}\{e[i,r-1]+e[r+1,j]+w[i,j]\}.$$

15.5 Optimal binary search trees （續）

固定根後，兩側區間的鍵與間隙都已確定。若其中一側不是該區間的最佳樹，換成成本更低的同區間子樹，就能降低整體成本，與最佳性矛盾。每一棵合法BST都有某個根，因此比較所有根及其最佳子樹便涵蓋全部樹形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一棵樹，兩種成本只差失敗機率總和

C(T)只數實際的鍵比較；E(T)還把失敗搜尋抵達虛葉的那一步算一次。這是兩種計費模型，不代表實作真的會與一筆虛構資料比較。接下來的區間表e採用E型成本。

$$C(T) = \sum_{r=1}^{n} p_r(\operatorname{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r \operatorname{depth}_T(d_r).$$

$$E(T) = C(T) + \sum_{r=0}^{n} q_r.$$

例A的差為0.35：平衡樹E=2.25，右斜樹E=1.85。這是與樹形無關的常數，**最優樹不變，報出的成本卻不同**。

沒有鍵的區間，實鍵比較成本C[i,i−1]=0，擴充成本則為e[i,i−1]=q[i−1]。這不表示C忽略失敗查詢：空段掛到父節點下時，仍要比較父的鍵，這一步由父區間的w計入。使用哪種模型，初始化與後續加總就要一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨科對照：先確認搜尋模型，再比較答案

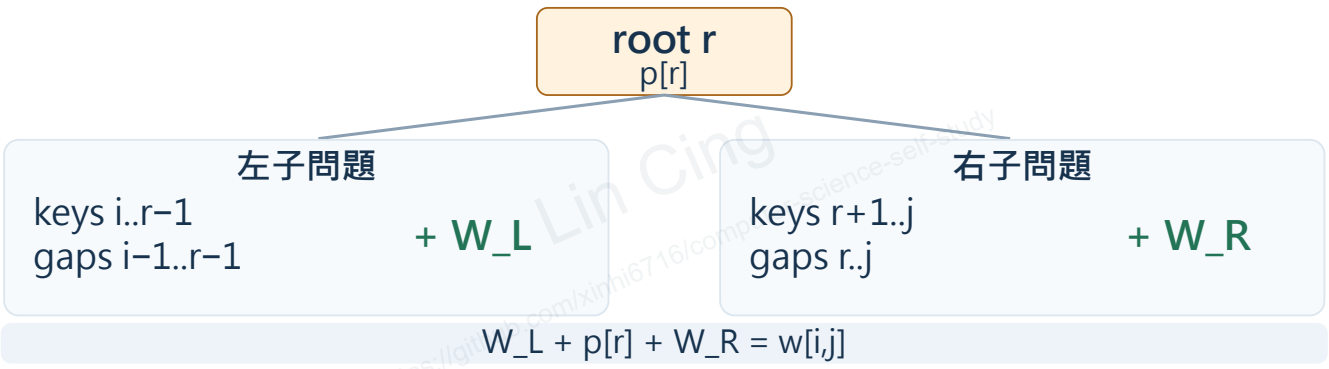
資料結構第10章先用「只有成功查詢」建立遞迴；本章依CLRS §15.5，將成功鍵與失敗間隙一起納入。兩者解的問題可能不同，不能只比較表格最後一個數字。

要確認的約定	只有成功查詢	本章的完整查詢模型
輸入權重	p；此模型下總和為1	p與q；兩者合計為1
空區間的成本	0	擴充成本E為q；實鍵成本C為0
每格新增成本	區間成功機率總和	區間成功與失敗機率總和w

q不全為0時，把q刪掉再將p正規化，會改變問題，最佳樹也可能改變。 相反地，在相同p、q下由E換成C，只扣掉常數Σq，最佳樹不變。讀其他教材時，也要核對區間端點是否包含、空樹初值與root欄位的含義。

為何加整段w，而不是只加根的p？

$$w[i,j]=\sum_{t=i}^{j}p_t+\sum_{t=i-1}^{j}q_t=W_L+p_r+W_R.$$



W_L, W_R是左右子樹機率總和；各查詢下降一層，增加W_L+W_R，根命中增加p_r。**機率沿用全域值，不將子區間重新正規化。**

Optimal BST 的填表及成本慣例

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

偽碼用長度控制計算順序。每個根r都讀取已完成的左、右區間，root[i,j]存的是根的索引r，不是成本。例B的最低擴充成本為2.10；若只數實鍵比較，扣除全部失敗機率0.40後是1.70，最佳樹形不變。

重建時，先檢查區間是否為空：j<i就放入失敗間隙d[i−1]；否則取r=root[i,j]，建立鍵k_r，再遞迴處理[i,r−1]與[r+1,j]。把空區間也交代清楚，才能重建完整的成功鍵與失敗間隙。

例B：Optimal BST 的完整表與樹重建

取三鍵 p=(0.20,0.30,0.10)，dummy q=(0.10,0.05,0.15,0.10)。先算：

區間	w	e	最優根
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

例如 `e[1,2]` 以 k2 為根時，成本 `e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45`。root[1,3]=2；左子問題 root[1,1]=1，右 root[3,3]=3，故樹根 k2，左右 child 分別 k1、k3，dummy 依 BST 區間掛放。

例B：Optimal BST 的完整表與樹重建 (續)

以長度填表，e、w、root 各 Theta(n^2) 格；每格枚舉最多 n 個根，共 Theta(n^3)。本章偽碼採這個已證明的版本。

候選根總數為 $\Sigma_{l=1}^{n}(n-l+1)l=n(n+1)(n+2)/6$。w 每格以 w[i,j−1]+p_j+q_j 常數時間更新，若在每個候選根內重掃整個機率區間，會多乘一個 n，不能沿用本分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例B：逐一比較根，讀取正確的子問題

e[i,j]：空段到長區間

i / j	0	1	2	3
1	0.10	0.50	1.45	**2.10**
2		0.05	0.70	1.35
3			0.15	0.60
4				0.10

候選根；w[1,3]=1

r=1
0.10 + 1.35 + 1 = 2.45

r=2
0.50 + 0.60 + 1 = 2.10

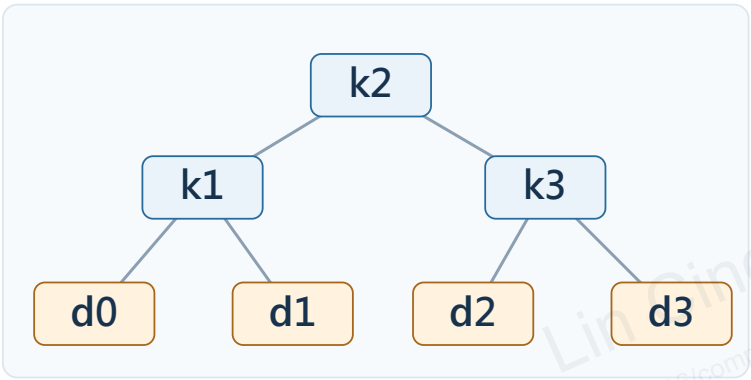
r=3
1.45 + 0.10 + 1 = 2.55

綠格：r=2 讀取的子問題；橘格：本次結果

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

綠格是選r=2時讀取的e[1,1]、e[3,3]。它們來自較短區間；橘格e[1,3]只在三個根都比較後確定為2.10。空段的q不可漏掉。

例B：從root表回到實際搜尋樹



例B：含 dummy 檢查

k2: 0.30 × 1 = 0.30

k1,k3: 0.30 × 2 = 0.60

d0..d3: 0.40 × 3 = 1.20

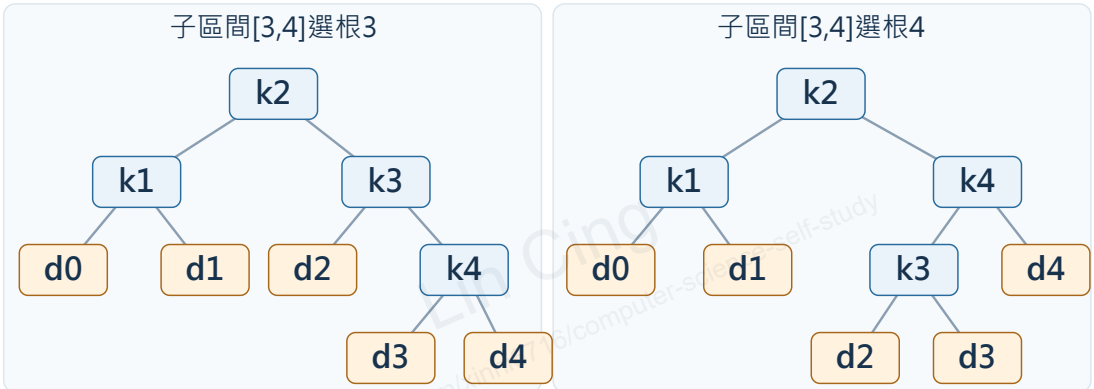
E = 2.10

C = E − 0.40 = 1.70

d0 | k1 | d1 | k2 | d2 | k3 | d3

例C：整數權重與兩棵同樣最優的樹

P=(3,3,1,1)、Q=(2,3,1,1,1)是權重，總和16，失敗權重和8。右子區間[3,4]選3或4同樣最優。



兩棵樹：S=32；除以總權重16後，C=2，E=2.50

序列對齊：配對、錯配與空位各有分數

全域序列比對允許 match、mismatch 與 gap，最後一欄可能是：

- `(x_i,y_j)` 配對，得分 match 或 mismatch；
- `x_i` 配 gap；
- `y_j` 配 gap。

若 `F[i,j]` 是前綴最高分，三種情形給出

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j), F[i-1,j]+gap, F[i,j-1]+gap)`。

這仍是前綴DP，但每次要比較三種最後一步。對X=ACGTA、Y=AGCAC，設定相同字元+3、不同字元−2、每個空位−3，最佳分數為1，可重建ACGTA-與A-GCAC。空位表示其中一側缺少字元，並不是原序列中的字元；每個空位都要付分數，不能只比較同一位置的兩個字元。

無限供應背包：先說清楚重量是恰好還是至多

每種物品可以取任意多次，物品i的重量為s_i、價值為v_i。令f[c]表示「重量恰好為c」時能得到的最高價值。這個「恰好」要先說清楚；無法湊出的重量不是價值0，而是不可行。

`f[0]=0`，`f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i)`。

若容量只是上限K，答案是所有0≤c≤K的f[c]最大值；若要求恰好裝滿，才只看f[K]。把「未使用的一單位容量」也當成轉移，則是在定義另一個「重量至多c」的狀態，不能與現在的表義混用。

取物品A=(重量3,價值5)、B=(4,7)、C=(7,12)，容量上限K=13。以恰好重量定義的表為：

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22]`。

最高價值22出現在重量13，可選A+A+A+B。只看價值重量比會先取B，三件B價值21，反而少1。這個例子否定一般的比值貪婪規則；DP則把每種可行的最後一件物品都比較過。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 15。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§15.1-15.5.

Lin Cing，個人資料結構手寫筆記：OBST加權搜尋成本、狀態區間與遞迴式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法

Part IV Advanced Design and Analysis Techniques

本章定位

貪婪演算法每步只保留一個選擇，節省枚舉其他候選的成本。本章用活動交換、Huffman葉交換與matroid增廣性證明這一步何時安全，再處理加權目標與容量限制如何使原規則失效。

前置知識

排序、交換論證、樹結構與選擇公理性質。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用交換法證明 activity selection。
- 辨識 greedy-choice 與最佳子結構。
- 建 Huffman code 並以 matroid 理解貪婪正確性。
- 在安全選擇失效時給出反例，並把一般證明中的交換步驟寫完整。
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1 An activity-selection problem

已知所有活動的開始、結束時間，要求選最多個兩兩不重疊活動。假設 s_i<f_i，活動已按 finish time 遞增排序，區間採半開形式 `[s_i,f_i)`，故一個活動開始時刻等於另一個結束時刻也相容。

定理。 對任一非空子問題，令 `a_m` 是 finish 最早的活動，則存在最大相容集合包含 `a_m`。

證明。 令 A^* 為子問題的最大解，a_j 為 A^* 中最早結束的活動。因 a_m 是全域最早結束，`f_m<=f_j`。把 a_j 換成 a_m：

- A^* 中原本在 a_j 之後的活動都滿足 `start>=f_j`，所以也與 a_m 相容；
- 子問題沒有更早的 activity 可受影響；
- 集合大小不減，故替換後仍是最大解。

因此 greedy-choice 成立。選 a_m 後只剩 `start>=f_m` 的子問題；對剩餘集合重複同一交換，歸納得到最優。

活動選擇的線性掃描與邊界

```
activity-select(按結束時間排序的活動):
    result=[]; last_finish=-∞
    for (s,f) in 活動:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

每次處理活動 j 前，result 是前 j−1 個中依此規則取出的相容集合，last_finish 是最後所選活動的終點；交換定理使每次選擇可延伸為某個最優解。掃描每活動一次，已排序時 Θ(n)；若尚未排序，另付 O(n log n)。

本章假設 s<f 且區間半開，端點相等可以相接；零長活動需另訂同點使用規則。空輸入直接輸出空集合。最早開始或最短時長不保證最優，例如活動[0,10)、[1,2)、[2,3)使最早開始只選一個，卻有兩個相容短活動。

最短時長先取的反例為 [0,3)、[3,6)、[2,4)。先選長度2的中間活動會擋住兩個長度3的活動，只有一個；最優則選前兩個，得到兩個。

活動選擇的完整掃描

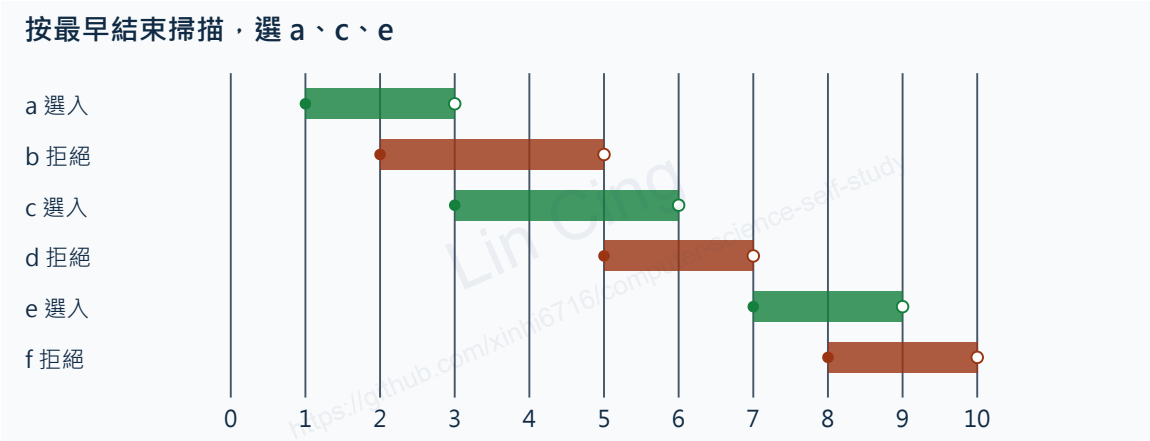
取六個活動：

活動	start	finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

依 finish 排序掃描：選 a；b 的 $2<3$ 拒絕；c 的 $3\geq 3$ 選入；d 的 $5<6$ 拒絕；e 的 $7\geq 6$ 選入；f 的 $8<9$ 拒絕。結果 $\{a,c,e\}$。

活動選擇的完整掃描 （續）

最優解也不只一個。$\{b,d,e\}$ 與 $\{a,c,e\}$ 都有 3 個；交換定理只保證存在一個含最早結束者的最優解，沒有說所有最優解都必須包含它。



16.2 Elements of the greedy strategy

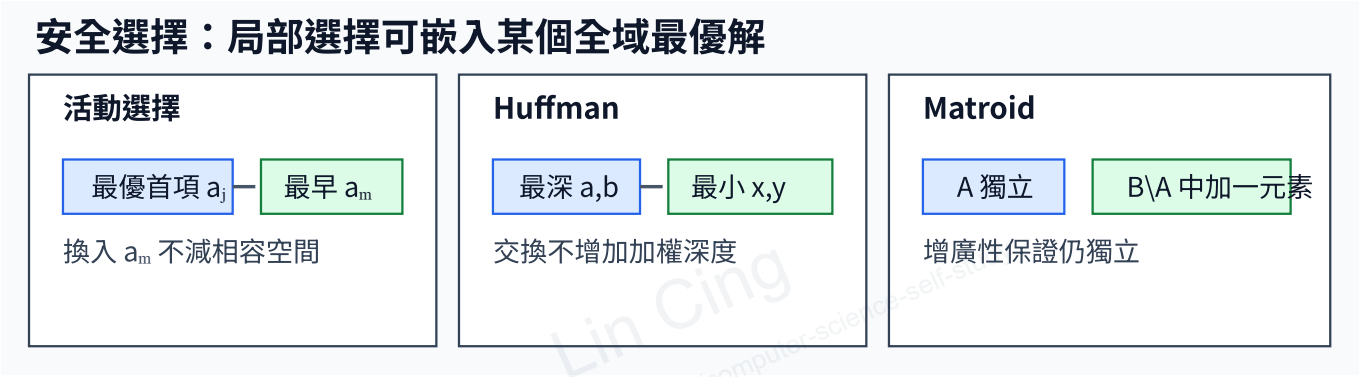
本章的交換型 greedy proof 要回答下列問題：

1. **Greedy choice 安全。** 取 greedy 的 g，對任意最優解 A^* 找第一個不同決策 x，構造 $A' = A^* - x + g$；逐項驗證 A' 可行且價值不低於 A^*。
2. **剩餘問題相同。** 移除 g 後，A^* 的其餘部分必須是明確定義的子問題的可行解，且子問題沒有遺失原來的候選。
3. **歸納終止。** 基底（空集合或單一活動）正確，每次決策使問題嚴格縮小。

這三項成立才可由「每步都安全」推到全域最優。局部立即利益、平均表現或若干成功例都不足夠。

例如任意面額的找零不能只取最大硬幣。面額 {1,3,4} 找6，greedy 得4+1+1三枚，最佳3+3兩枚；此反例否定一般規則。熟悉面額的成功經驗不能代替針對面額族的證明。

16.2 Elements of the greedy strategy （續）



為何加權活動不能用同一交換

把目標改成最大總 value，就不再是最大基數。取

活動	區間	value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

最早 finish greedy 選 short，再選 tail，總 value 2；最優為 long+tail，總 value 101。交換 A^* 中原本最早結束的 long 為 short 後，雖然區間仍相容，但 value 從 100 降到 1，不能保持最優值。

加權版本有最佳子結構。以 1-based 索引按 finish 排序，令 p(j) 為 j 之前 finish 不晚於 s_j 的最大索引，不存在時 p(j)=0；D[j] 表示前 j 個活動的最佳總價值。則 `D[0]=0`，

`D[j]=max(D[j-1], value_j+D[p(j)])`。

為何加權活動不能用同一交換 （續）

排除 j 就只看前 j−1 個；包含 j 則其餘活動必來自前 p(j) 個，而且可獨立最佳化。此二分涵蓋所有解。數例 p=(0,0,2)、D=(0,1,100,101)，由 D3 選 tail，再由 D2 選 long。finish-time greedy 只保證未加權最大基數，不能沿用到這個目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0–1 與 Fractional Knapsack 的不同證明

假設物品重量與價值皆正、容量 C>0。fractional 版本允許取比例 x_i∈[0,1]，其餘目標是最大化 Σv_i x_i。依密度 ρ_i=v_i/w_i 下降排序，先裝滿密度高的物品，至多一件只裝部分。

若全部物品總重不超過 C，全部取走即最優。否則任何最優解必裝滿 C；若仍有空間且有正價值物品未取完，增加一點比例就能改善目標，矛盾。

若某可行最優解在高密度 i 尚未取滿時，卻使用較低密度 j 的正重量，令 δ=min(w_i(1−x_i),w_j x_j)>0，把 δ 重量由 j 移給 i。容量不變，價值增加 δ(ρ_i−ρ_j)≥0。反覆交換直到第一個 greedy 選擇相同，再對剩餘容量歸納，得到某最優解符合整個 greedy 排序。相同密度可任意打破平手。

但在 0–1 版本 x_i 只能是 0 或 1，δ 重量交換通常不合法。例 C=10，物品 (w,v)=(6,12),(5,9),(5,9)。密度 greedy 先取 (6,12)，剩 4 放不下其他，價值 12；0–1 最優取兩件 (5,9)，價值 18。fractional 則先取 (6,12)，再取一件 (5,9) 的 4/5，價值 19.2。

0–1 DP 用 D[i,c] 表示前 i 件、容量至多 c 的最佳值，D[0,c]=D[i,0]=0，D[i,c]=max(D[i−1,c],v_i+D[i−1,c−w_i])，第二項僅在 w_i≤c 時存在。整數容量的時間 Θ(nC) 是偽多項式，因輸入 C 本身只需 Θ(log C) 位元。

16.3 Huffman codes

二元 prefix tree 的葉深度就是碼長。給定頻率 `f_c`，總位元數為

`B(T)=sum_c f_c d_T(c)`。

Huffman 每次從 priority queue 取兩個最小權重樹 x,y，建立權重為 `f_x+f_y` 的父節點。若有 n 個符號，進行 n−1 輪；每輪兩次 extract-min、一次 insert，共 3(n−1) 次 heap 操作，加線性建堆，時間 O(n log n)。

交換左、右 child 只改變 codeword 的 0/1，不改變長度與總成本。頻率相同時可形成多棵不同的最優樹。

Huffman 選擇引理：把最小頻率放到最深兄弟

假設至少兩個字元且頻率皆正。任何最佳 prefix tree 可取成每個內節點恰有兩個 child，因刪掉只有一個 child 的內節點會縮短其下所有碼字而降低成本。取最深葉，其兄弟也必為同深葉，否則兄弟的後代會更深。

令 x、y 為頻率最低的兩字元。先把 x 換到這對最深葉的一個位置；若其中一個位置原本是 y，就選另一位置。這使 y 在第二次交換前仍留在別處，或已經在所需的兄弟位置。

交換 a 與 x 的成本差只涉及這兩個葉：

`B(T')-B(T)=(f_a-f_x)(d_x-d_a)<=0`。

因 `f_a>=f_x` 且 `d_x<=d_a`，差值非正。此時 x 已放妥，剩餘位置的頻率都不小於 y，故再把 y 換到 x 的最深兄弟位置，同樣不增加成本，且不會移走 x。若 x 或 y 原本已在目標位置，該次交換為零。頻率相同時任何相應安排成本相等。

故總存在最佳 prefix code 含這個 greedy choice。這裡需使用「最低頻率」與「最深葉」的兩個交換不等式，不能只說看起來合理。

Huffman 收縮與最優子結構

把 x、y 從字母表刪除，加入虛擬字元 z，令 `f_z=f_x+f_y`。若 T' 是收縮後的最佳樹，把 z 展開成 x、y，得到原樹 T。

除 x、y 以外的葉深度不變；x、y 的深度都是 `d_T'(z)+1`，故

`B(T)=B(T')+f_x+f_y`。

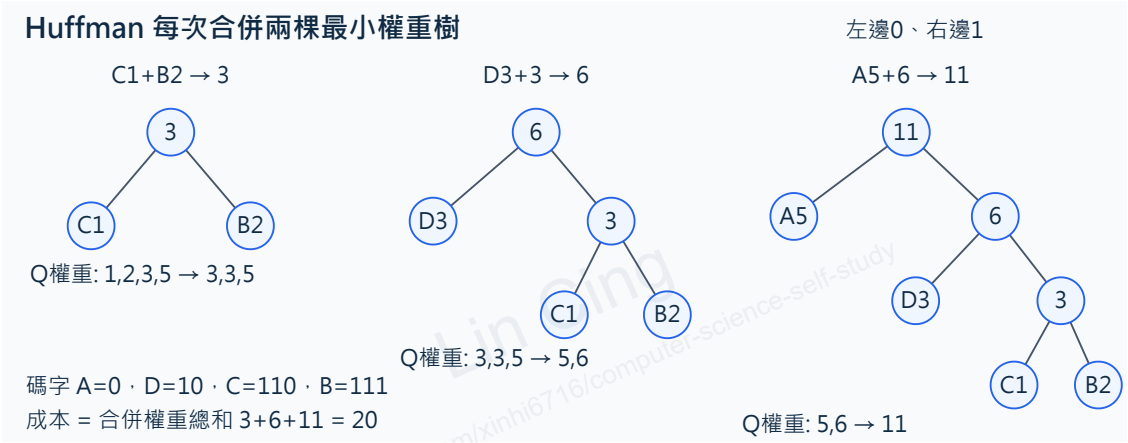
若原問題有更優樹 T*，依上一頁可令 x、y 為最深兄弟；將兄弟收縮成 z，得到

`B(T**)=B(T*)-f_x-f_y<B(T')`，

與 T' 最優矛盾。因此展開收縮樹即原問題最優；再以符號數歸納，Huffman 正確。

頻率 A5、B2、C1、D3 的合併權重為 3、6、11；可取 A=0、D=10、C=110、B=111，成本 `5+6+3+6=20=3+6+11`。

Huffman 收縮與最優子結構 （續）



Huffman 的演算法與單字元邊界

```
Q = 所有正頻率單葉樹組成的 min-heap
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q 的唯一樹 · 若 Q 空則回傳空
```

解碼從根依0、1走向左、右 child；到葉就輸出字元並重回根。prefix 性質保證沒有某個完整碼字同時是更長碼字的前綴，因而字界明確。最後位元若停在內節點，表示輸入截斷，不能默認成功解碼。

空字母表沒有碼樹。只有一個字元時，樹論可給空碼，實際串流通常約定碼0並另傳字元數。本章成本與合併權重總和的等式採至少兩個字元的完整二元樹，不能把單字元實作約定偷換成合併成本。

16.4 Matroids and greedy methods

matroid $M=(S,I)$ 有三條：

- 空集屬於 I；
- 遺傳性：若 A 屬於 I，任何 A 的子集也屬於 I；
- 增廣性：若 A,B 屬於 I 且 $|A|<|B|$，存在 $x \in B\setminus A$ 使 $A\cup\{x\}$ 屬於 I。

假設 S 有限，w(x)>0。greedy 依 weight 降冪掃描，加入後仍獨立就取。令 x 是第一個單點獨立的元素。若不存在，遺傳性使所有非空集合都不獨立，空集就是最優解。

取任一最優集 B。若 $x\notin B$，從 $A=\{x\}$ 開始，當 $|A|<|B|$ 就以增廣性選 $b\in B\setminus A$ 加入 A。每次只從 B 加入，最後 $|A|=|B|$，故恰有一個 $y\in B$ 被省去，$A=B-\{y\}+\{x\}$。由 B 獨立可知 {y} 獨立，x 是權重最高的可行單點，故 $w(x)\geq w(y)$，而 $w(A)=w(B)-w(y)+w(x)\geq w(B)$。因此 A 也最優且含 x。

這裡交換性不是直接宣告「換任一 y 都可行」。必須先以逐步增廣構造出可行的 A，再由等大小推得唯一被省去的 y。

收縮後仍為 Matroid 的完整證明

固定可行的 x，令 $S'=S\setminus\{x\}$，$I'=\{A\subseteq S' : A\cup\{x\}\in I\}$。S' 可以包含在新系統中永不獨立的元素，這些元素不影響結論。

- 空集在 I'，因 $\{x\}\in I$。
- 若 $A\in I'$、$C\subseteq A$，則 $C\cup\{x\}\subseteq A\cup\{x\}$，由原系統遺傳性可得 $C\in I'$。
- 若 $A,B\in I'$ 且 $|A|<|B|$，原系統中的 $A\cup\{x\}$、$B\cup\{x\}$ 都獨立且前者較小。增廣元素 y 必在 $(B\cup\{x\})\setminus(A\cup\{x\})=B\setminus A$，因此 $A\cup\{y\}\in I'$。

對任何含 x 的可行解，其餘部分屬 I'，且權重恰少 w(x)；反之亦然。所以原問題在選定 x 後精確化為收縮系統的最大權獨立集。對 |S| 歸納，後續掃描的可行性正是 I' 的 oracle，greedy 全程最優。

若 x 在某階段不可加入，之後也不會可加入。否則較大集合 $A\cup\{x\}$ 獨立會由遺傳性推出當時較小的集合也可加 x，矛盾。排序 $O(n \log n)$，每次 oracle 成本 $O(f(n))$ 時，總成本 $O(n \log n+n f(n))$。允許負權且目標為任意獨立集時，須略過負權元素；不能無條件取成最大基底。

Graphic Matroid 與非 matroid 反例

對固定無向圖 G=(V,E)，令 S=E，I 為所有不含 cycle 的邊集合，孤立頂點仍列入 V。遺傳性成立。令 A 的分量為 V1..Vk；若 B 每條邊都留在某個 Vi 內，因 B 在 Vi 內仍是森林，邊數至多 |Vi|−1。相加得 |B|≤Σ(|Vi|−1)=|V|−k=|A|，與 |B|>|A| 矛盾。所以存在 B 中一條跨 A 分量的邊，加入 A 不形成 cycle，增廣性成立。

連通圖的基底都是 n−1 邊的生成樹。若要由最大正權定理推得最小成本 MST，可令新權重 w'(e)=K−cost(e)>0；所有基底有相同邊數，故最大化 Σw' 等價於最小化原成本。這使 Kruskal 升冪規則與此定理精確相接，並沒有把最大任意森林與最小任意森林混為一談。

但一個 hereditary system 不一定有增廣性。取 S={a,b,c}、權重 10,9,9，I={∅,{a},{b},{c},{b,c}}。A={a}、B={b,c} 有 |A|<|B|，卻無法把 b 或 c 加入 A；這不是 matroid。Greedy 選 a 得 10，最優 {b,c} 得 18。

16.5 A task-scheduling problem as a matroid

工作 j 處理時間 1、deadline d_j、延遲罰金 w_j。集合 A 可全部準時，當且僅當依 deadline 排好後 `d_(k)>=k`；等價於對每個 t，

`N_t(A)=|{j in A:d_j<=t}|<=t`。

必要性是前 t 個時間槽最多容納 t 個工作。充分性是把 A 按 deadline 遞增排；第 k 個工作的 deadline 至少 k，故不遲到。

所有工作在時間 0 就緒、不可搶占、單機，deadline 是正整數，完成於 d_j 視為準時。n 個工作只需前 n 格，故先將大於 n 的 deadline 截為 n，不改變可行性。

這族獨立集有遺傳性。為證增廣，取 |A|<|B|，令 k 是滿足 N_k(A)=k 的最大整數，0 一定符合。由 |A|<|B|≤n，k<n。A 中有 k 個工作 deadline≤k，而 B 這類工作至多 k 個，所以 B 中 deadline>k 的工作數至少 |B|−k>|A|−k。因此存在 x∈B\A 且 d_x>k。

把 x 加入 A。對 t≤k，新增工作不被計入，所以 N_t 不變；對 t>k，最大性保證 N_t(A)<t，整數性使 N_t(A)≤t−1，新增至多 1 後仍≤t。因此 A∪{x} 可行，增廣性成立。

單位時間排程的 greedy 與完整追蹤

工作：

工作	deadline	penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

按 penalty 由大到小加入：a 可行；b 使 deadlines 排序 1,3，可行；c 使 1,3,3，可行；d 使 1,2,3,3，第 4 個 deadline 3<4，拒絕；e 使 `N_1=2>1`，拒絕。

準時集為 {a,b,c}，依 deadline 排成 b,a,c，分別於 1、2、3 完成。其餘 d,e 放在 4、5，確實遲到。延遲罰金 6+5=11；總 penalty 35，準時工作所保住的 penalty 為 24，達最大。

單位時間排程的 greedy 與完整追蹤 （續）

使用長度 n 的 deadline 次數陣列，計算各前綴 N_t，可在 O(n) 內檢查一次獨立性，n 次檢查共 $O(n^2)$，加最初 penalty 排序 O(n log n)。若每次另作比較排序，oracle 是 O(n log n)，總時間會變成 $O(n^2 \log n)$，不能把排序叫作線性。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 16。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§16.1-16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22 基礎圖演算法

Part VI Graph Algorithms

本章定位

圖的搜尋順序同時決定距離、遞迴祖先與完成時間；本章用同一六點有向圖，從FIFO層序與白路徑定理推導BFS、DFS、拓撲序及SCC的正確性與線性成本。

前置知識

圖表示、顏色、時間戳與線性走訪。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 選擇 adjacency matrix 或 list。
- 推導 BFS 最短路徑性質。
- 用 DFS 完成 topological sort 與 SCC。
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

本章的圖是有限有向圖或無向圖；每個 vertex（頂點）有穩定識別碼，edge（邊）的方向必須明定。路徑可以長度零，因此每個頂點可達自身。令 n=|V|、m=|E|；無向邊只在 E 計一次。除特別註明，本章的最短距離是**邊數**，不是任意權重的總和。

Adjacency list（鄰接串列）有 n 個表頭。有向圖的所有表長之和為 m；無向簡單圖每條邊在兩端各存一次，總長為 2m。空間 Θ(n+m)，列出 u 的鄰點需 Θ(1+outdegree(u))，掃描普通未排序串列查詢 (u,v) 最壞 Θ(1+outdegree(u))。雜湊或排序的鄰接集合可以改變查詢成本，不能把它們的保證套到普通 list。

Adjacency matrix（鄰接矩陣）A 的 row 是起點、column 是終點，A[u,v]=1 表示 u→v；空間 $\Theta(n^2)$、查邊 Θ(1)，但枚舉 u 的鄰點一般要掃 n 個位置。無向圖滿足 $A=A^{\mathsf{T}}$。有權矩陣需另有「不存在」標記：權重 0 可以是合法邊，不能直接與不存在混為一談。

同一個六頂點有向圖

後面的 BFS、DFS 與 SCC 都使用下列**同一組有向邊**，鄰接與外層頂點順序均按表排列。

頂點	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

n=6、m=8，表長總和 8，入度總和也為 8；不是 16。轉置圖 G^{T} 把每個 u→v 放入 $\mathrm{Adj}^{\mathsf{T}}[v]$ 的 u 項：先建立 n 個空表，再掃全部 m 項，時間與新增空間皆 Θ(n+m)。原圖不需破壞。

同一個六頂點有向圖 （續）

一張圖的 List 與 Matrix

Adj[u]：row u → column v；這是有向圖，m=8。

頂點	鄰接串列
a	b,c
b	c
c	a
d	b,e
e	d
f	e

A 的 column

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

綠格：存在的有向邊；8個綠格與8個list項逐一對應。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search

BFS（廣度優先搜尋）以 FIFO queue 維持距離的層序。白色是未發現，灰色是已發現而尚未掃完，黑色是鄰接表已掃完。π[v] 是發現 v 的前驅；∞ 與 NIL 分別表示尚無距離與無前驅，不能用 0 取代。

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

入列之前就改色，同一頂點才不會因兩個父候選重複入列。在每輪 while 開始時，queue 恰為全部灰色頂點；掃描 u 的期間，u 已出列但仍灰色，因此不要把這個不變量誤稱為每一條指令後都成立。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

FIFO 為何保證先處理較近的點

把 queue 依隊首到隊尾寫為 $q_1,\cdots,q_r$。其距離非遞減，且 $d[q_r]\le d[q_1]+1$；空 queue 時條件視為成立。

初始只有 s。若本輪取出 u、$d[u]=k$，剩餘項依歸納只有 k 或 k+1，且非遞減；新發現的每個 v 都取得 k+1 並接在尾端，因此順序保持。本輪結束若 k 層已清空，隊首變成 k+1，下一輪才可能產生 k+2。由此每次 dequeue 的距離也非遞減。

每個有限 d[v] 都有一條實際 π 路徑：s 的長度 0；發現 v 時把 u 的路徑接 $u\to v$，長度恰 $d[u]+1$。因此真最短距離 $\delta(s,v)\le d[v]$，而 π 鏈的 d 每次下降 1，不可能形成 cycle。

但「找到一條路」本身只給上界，還不足以證明最短；下一步需要排除先找到較長路徑的可能。

BFS 最短距離的完整論證

假設有可達 v 最後 $d[v]\neq\delta(s,v)$，選 δ 最小的這種 v。$v\neq s$；取最短路徑中 v 的前一點 u，則 $\delta(s,u)=\delta(s,v)-1$，且依選法 $d[u]=\delta(s,u)$。u 有限距離，必定入列並最終出列。

掃描 $u\to v$ 時，若 v 仍白，就設 $d[v]=d[u]+1=\delta(s,v)$，矛盾。若 v 已發現，設它最初由 w 掃描發現。w 不晚於 u 出列；由 queue 層序，$d[w]\le d[u]$，所以 $d[v]=d[w]+1\le d[u]+1=\delta(s,v)$。結合上一頁 $\delta\le d$，仍得到相等。兩種情況都矛盾，故所有可達點的 d 均正確；不可達點若曾被發現就有實際路徑，亦矛盾，故其 d 保持 ∞。

每個可達非根點有一個 π，反向回溯恰 $d[v]$ 條邊後到 s；它們構成 BFS tree。鄰接順序可以改變 π 與最短路徑的選擇，但不能改變 d。

初始化 $\Theta(n)$，每個頂點最多入列與出列一次，每個可達點的鄰接表掃一次，總時間 $O(n+m)$，queue 與 d、π、color 的額外空間 $O(n)$。使用 matrix 時，每個已出列頂點掃 n 格，最壞 $\Theta(n^2)$。

從 d 出發的完整 Queue Trace

表中的 queue 是本輪**掃描結束後**的內容；括號內是 d 值，新增點同時取得前驅。

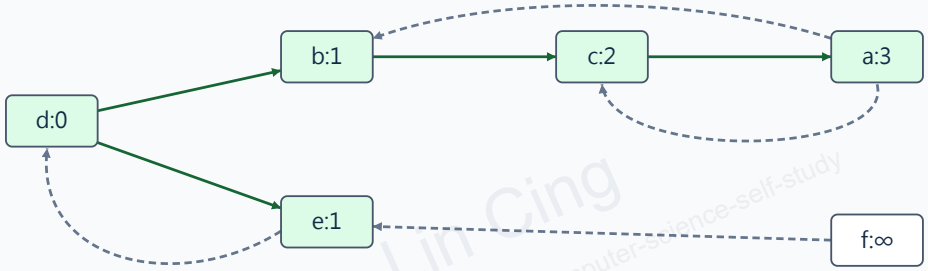
取出的 u	掃描結果	輪末 queue
尚未開始	初始化 d[d]=0	d(0)
d	b 白→d[b]=1,π[b]=d；e 白→d[e]=1,π[e]=d	b(1),e(1)
b	c 白→d[c]=2,π[c]=b	e(1),c(2)
e	d 已黑，不再入列	c(2)
c	a 白→d[a]=3,π[a]=c	a(3)
a	b、c 都已發現，不變	空

最後 (d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,∞)，到 a 的最短路徑是 d→b→c→a。f→e 的存在不表示 d 可逆向到 f。

從 d 出發的完整 Queue Trace （續）

BFS：同一六點圖從 d 出發

格內 = 頂點:距離；綠實線 = π邊；灰虛線 = 其他原圖邊。



隊首順序 d(0) → b(1) → e(1) → c(2) → a(3)；f 保持∞。

有向邊距離、等權前提與最短路徑計數

對任一來源可達的 u→v，δ(s,v)≤δ(s,u)+1；有向圖沒有反向不等式。本例 a→b 從距離 3 指到 1，差為 2。無向邊兩方向都存在時，才有 |d[u]−d[v]|≤1。

等正權 w 時，最短權重是 w・d[v]。一般權重不可用 BFS：s→t 權 9，s→x 權 1，x→t 權 1，BFS 的 1 邊路徑權 9 大於 2 邊路徑權 2。權重全為零時可達路徑都權 0；有負權或負 cycle 更需另行定義問題。

若要算單位權圖最短路徑數，設 ways[s]=1，其餘 0。發現白 v 時照常設 d 與入列；之後對**每條**滿足 d[v]=d[u]+1 的 u→v 加上 ways[u]。同一條簡單圖邊只算一次。所有距離 k−1 的點都在距離 k 的點出列前完成，故出列 u 時 ways[u] 已收到所有最短前驅貢獻。

例如 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5：d=(0,1,1,2,2,3)，ways=(1,1,1,2,2,4)。這是 O(n+m) 次加法；若計入大整數成本，路徑數可能有 Θ(n) 位元，不能把每次加法無條件視為固定時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search

DFS（深度優先搜尋）先完成目前遞迴分支，再處理下一鄰點。外層必須掃全部頂點，才能包含不從第一個根可達的部分。以下 time 是**發現與完成事件**的計數，不是處理邊的計數。

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

每個點只被 Visit 一次，每個鄰接項掃一次：Θ(n+m) 時間，遞迴深度最壞 n、額外空間 O(n)。實際寫迭代版本時，stack frame 要保存「下一個待掃鄰點」或完成事件；只把鄰點一次全 push 的 visited 走訪不自動給相同 DFS forest 或完成時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離

考慮不同頂點 u、v，設 tin[u]<tin[v]。若 v 在 u 完成前發現，當時 u 的 Visit 還在 stack 中；v 是 u 的後代，且 v 的 Visit 必先返回，故 tin[u]<tin[v]<tout[v]<tout[u]。若 v 在 u 完成後才發現，則 tin[u]<tout[u]<tin[v]<tout[v]，兩區間分離。

這排除了交錯 tin[u]<tin[v]<tout[u]<tout[v]。反過來，嚴格包含的時間區間意味發現 v 時 u 仍在 stack，故 v 是 u 的真後代。這就是 parenthesis theorem（括號定理）；它依賴遞迴必須後進先出地返回。

六點圖按 a,b,c,d,e,f 做 DFS，得到下表。括號串為 (a(b(c c)b)a)(d(e e)d)(f f)，所有數字 1 到 12 各使用一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

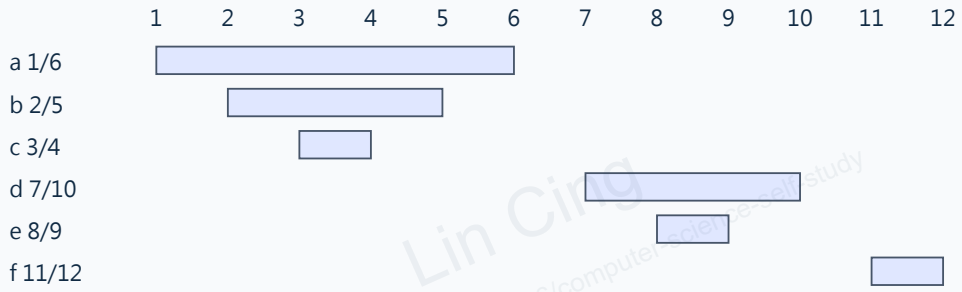
點	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

DFS 的發現 / 完成事件

每條橫帶從 tin 到 tout；子帶巢狀，另兩棵根的區間分離。



a 包住 b,c；d 包住 e；f 是第三根。不同點區間不可能交錯。

White-path Theorem 的時間條件

在 Visit(u) **尚未把 u 改灰之前**，v 是本次 DFS 中 u 的後代（允許 v=u），當且僅當當時有 u 到 v 的路徑，其所有點都為白。這個時間切點讓 u 本身也符合白色條件。

「後代⇒白路徑」：forest 上 u 到 v 的 tree path 是原圖的路徑；u 的各真後代都在 tin[u] 之後才發現，故當時整條 path 為白。

「白路徑⇒後代」：沿白路徑找第一個假設沒有成為 u 後代的點 y，其前點 x 已是 u 的後代（可能就是 u）。Visit(x) 掃 x→y 時，若 y 仍白就遞迴，y 必為後代；若已發現，因 y 在 tin[u] 切點仍白，它是在 u 的尚未結束遞迴期間發現，也必屬 u 的子樹。兩種情況都矛盾。

必須保留「當時全白」：圖 a→b,a→c,c→b，先掃 b。到 c 被發現時 b 已黑；雖然 c 可達 b，b 不會變成 c 的後代。一般可達性不能直接取代 white-path 條件。

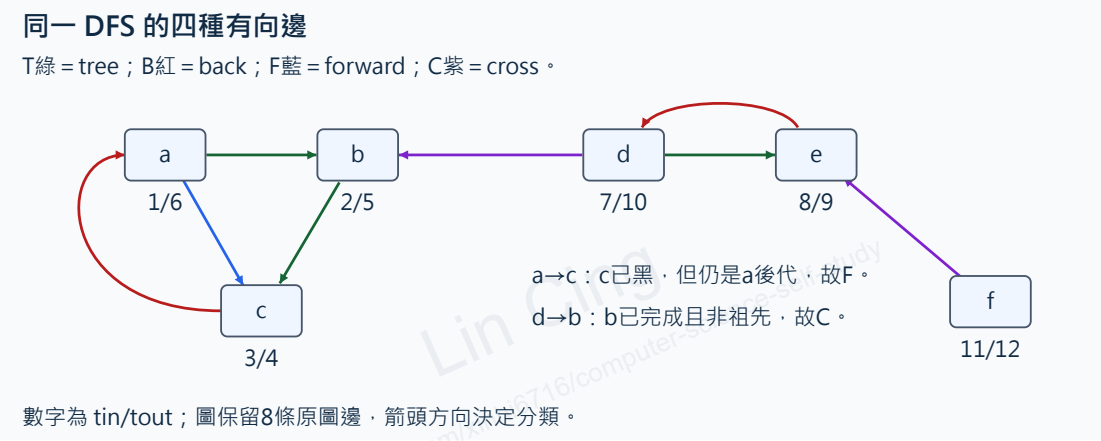
有向 DFS 的四種邊

掃描 u→v 時，u 是當下最深的灰色點，全部灰點恰為一條 stack 祖先鏈。v 白則是 tree edge，v 灰則是 back edge（回邊，含 self-loop）；v 黑時，用 tin 比較：tin[u]<tin[v] 是 forward edge，反之是 cross edge。

邊型	forest 關係	完成後可核對的時間式
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	非 tree 的真後代邊	同上；只靠時間不能區分 tree/forward
back	v 為 u 的祖先，或 v=u	祖先時 tin[v]<tin[u]<tout[u]<tout[v]
cross	彼此無祖先關係	tout[v]<tin[u]；掃描方向不能倒過來讀

本例依實際掃描順序：a→b(T)、b→c(T)、c→a(B)、a→c(F)、d→b(C)、d→e(T)、e→d(B)、f→e(C)。a→c 是在 c 完成後才由 a 掃到，故看到黑色不代表一定是 cross。

有向 DFS 的四種邊 （續）



無向圖的邊型與 Cycle 判斷

無向邊會在兩個 adjacency list 出現，但作為一條邊分類時，取**第一次掃到該邊**的方向。設 tin[u] <tin[v]；因 u 的 list 中有 v，v 必在 u 完成前發現。第一次掃到時，若是 u→v，v 尚白，否則早已由 v→u 掃到，所以是 tree；若先掃 v→u，u 仍灰，是 back。故無向 DFS 只有 tree/back，沒有真正的 forward/cross。

對無向簡單圖檢查 cycle，遇到灰色父節點的反向鄰接項不能當成額外 cycle。若允許平行邊，則須用**邊識別碼**略過同一條 parent edge，不能略過所有指向 parent 的邊；第二條平行邊可形成長度 2 的 cycle（依允許 multigraph cycle 的慣例）。

有向圖則有 cycle ⇔ DFS 出現 back edge。回邊 u→v 搭配 forest 中 v 到 u 的有向 tree path 構成 cycle。反向證明：取 cycle 中最早發現的 v，當時其餘 cycle 點皆白；循 cycle 到 v 的前點 u 是白路徑，由 white-path theorem，u 是 v 的後代，所以最後一條 u→v 為 back edge。self-loop 直接落在同一結論。

22.4 Topological sort

Topological order（拓撲序）是所有頂點的排列，使每條 u→v 都滿足 pos[u]<pos[v]。有向 cycle 要求沿圈嚴格遞增又回到原位，不可能成立；DAG（有向無環圖）才存在此排列。

在 DAG 上做 DFS，把完成順序反轉即可，不必用比較排序。對任一掃描的 u→v：v 不可能灰，否則有 back edge/cycle；v 白時成為後代，tout[v]<tout[u]；v 黑時早已完成，仍有 tout[v]<tout[u]。故按完成時間遞減，所有邊都由前指後。時間 Θ(n+m)，額外空間 O(n)。

實作可在掃邊遇灰時回報 cycle，最後才交付拓撲序。不能對一般有向圖不檢查就把 reverse-finish list 標為合法排序；完成時間總能列出，但邊限制不一定成立。

例 a→c,b→c,c→d：按 a,b,c,d 做 DFS，tin/tout 分別 a=1/6,c=2/5,d=3/4,b=7/8，反轉完成序得 b,a,c,d。a,b,c,d 也合法；**a,c,b,d 不合法，因 b→c 被逆置**。

Kahn 的入度更新與失敗判定

先算 indegree[v]，把所有入度零點放 queue。每次取 u 輸出，對每條 u→v 執行 indegree[v]−=1；當且僅當變成零時入列。計數始終等於尚未輸出頂點誘導子圖中的入度，因此輸出 u 時沒有仍未輸出的前驅，已輸出的 prefix 永遠符合邊的順序。

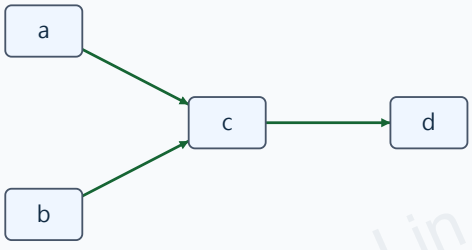
若剩餘非空卻無入度零點，任取殘餘點，反覆沿一條殘餘入邊走；有限頂點必重複，得到 cycle。反之 DAG 的非空誘導子圖必有入度零點，所以演算法會輸出全部 n 點。只輸出少於 n 個就回報失敗；**殘餘點可能只是 cycle 的下游，不代表每個殘餘點都在 cycle 上**。

目前輸出 u	c 的剩餘入度	d 的剩餘入度	輸末 queue
初始	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	空

Kahn 的入度更新與失敗判定 （續）

Kahn：入度與 Queue 的同步狀態

同一 DAG：a→c、b→c、c→d；移除起點後才減各出邊終點入度。



操作	in(c)	in(d)	Queue
初始	2	1	a,b
移除a	1	1	b
移除b	0	1	c
移除c	0	0	d
移除d	0	0	空

輸出 a,b,c,d；a,c,b,d 把 b→c 逆置，不合法。

22.5 Strongly connected components

定義 u~v ⇔ u 可達 v 且 v 可達 u。零長路徑給自反性，定義給對稱性，串接路徑給遞移性，因此它是等價關係；其等價類是 SCC（強連通分量），也就是按包含關係極大的互相可達集合，**不是最大大小的那一類**。

縮合圖 condensation 的點是各 SCC，不同分量間只要有一條原圖邊就加邊。它必為 DAG：若不同類沿圈可達，圈上各類彼此可達，應合為同一等價類，矛盾。把所有邊反向會同時反轉兩方向的可達路徑，所以 G 和 G^T 有完全相同的 SCC。

六點圖的 SCC 為 A={a,b,c}、D={d,e}、F={f}，縮合邊 F→D→A。注意 G^T 有 A→D→F；從 a 隨意在 G^T 起跑可以把三個 SCC 都走到。**轉置本身不會阻止跨分量，必須配合正確根順序與已訪問標記。**

為何不同 SCC 的完成時間有方向

以下 tin、tout 都是**第一趟 G 的 DFS**。令 tin(C)=min tin[v]、F(C)=max tout[v]，v∈C。若不同 SCC 間有 C→D，則 F(C)>F(D)。證明分兩種首次發現先後：

1. 若 tin(C)<tin(D)，C 第一個被發現的 x 當時能沿 C 內部白路徑到邊尾，再跨到 D 並走 D 內部白路徑。由 white-path theorem，C 其餘點及 D 全成 x 的後代，所以 F(C)=tout[x]>F(D)。C 內兩點間可取完全位於 C 的路徑：若路徑經外點，該外點也與 C 互相可達，會屬 C。
2. 若 tin(D)<tin(C)，D 第一點 y 可以在全白的 D 中走到全部 D，所以 F(D)=tout[y]。D 不可能到 C，否則搭配 C→D 使兩類合併；因此 y 的整棵遞迴走不到 C，C 要等 y 完成後才可能發現，故 F(C)>F(D)。

所以 G^T 的跨分量邊 C→D 一定滿足 F(C)<F(D)。這是**分量最大完成時間**的比較；不能要求 C 中每個點都比 D 中每個點晚完成。例如 a↔b、a→c，先掃 a→b，再掃 a→c，得到 tout[b]=3、tout[c]=5、tout[a]=6：F({a,b})=6>F({c})=5，但 b 比 c 早完成。真正跨類的那條 a→c 仍有 tout[a]>tout[c]；不同 SCC 的邊不可能是 back edge。

兩趟 DFS 的完整演算法與不跨界證明

```
SCC(G):
  first=DFS(G,所有頂點的固定順序)
  roots=reverse(first.finished)    // 第一趟的完成序，先保存
  GT=transpose(G)
  第二趟重新設所有頂點為WHITE，π=NIL
  按roots依序，遇WHITE u才在GT做Visit(u)
  每次新Visit根的整棵tree輸出為一個分量
```

歸納已輸出的每棵 tree 恰為一個完整 SCC。下一個白根 u 所在 C 有全部未處理分量中最大的 F(C)：如果 C 的最大完成點已處理，依歸納 C 整類都已處理，與 u 白矛盾。

u 可沿 C 內部路徑走到全部 C；其餘 C 仍白，故第二趟會收齊。若 G^T 有 C→D 的跨界邊，F(D)>F(C)，所以 D 必已處理為黑色，Visit 不遞迴；更不可能經此跳往另一白類。因此此 tree 既不少收也不多收，完成歸納。

兩次 DFS 各 Θ(n+m)，轉置 Θ(n+m)。反轉 append 得到的完成序為 Θ(n)，不必用 O(n log n) 排序；總時間 Θ(n+m)，新增轉置與各陣列空間 O(n+m)。

同一六點圖的兩趟結果

第一趟完成序 c,b,a,e,d,f；因此第二趟候選根序 f,d,e,a,b,c。轉置的表為 a:[c]、b:[a,d]、c:[a,b]、d:[e]、e:[d,f]、f:[]（各表按字母排列）。

候選根	G^T 的實際新發現序	遇到的跨類邊	本次輸出
f	f	無	{f}
d	d,e	e→f，f 已黑	{d,e}
e	跳過；已黑	—	不新增
a	a,c,b	b→d，d 已黑	{a,b,c}
b,c	跳過；已黑	—	不新增

F(A)=6、F(D)=10、F(F)=12，與原縮合邊 F→D→A 一致遞減。若改用第一趟 discovery 遞減，c 可能在 a 前，但這個例子未必失敗；規格正確性不能靠某次剛好成功。最小反例 x→y：第一趟 tin[x]=1,tin[y]=2，若第二趟先從 y 在 G^T 走，會沿 y→x 合併兩個本應分離的 SCC。

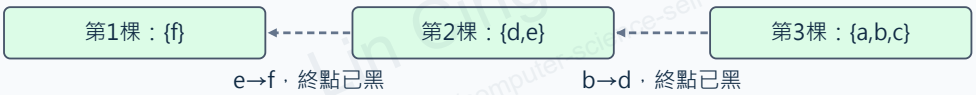
同一六點圖的兩趟結果 （續）

SCC：完成值排序如何阻止第二趟多收

第一趟 G 的分量最大完成值：F({f})=12、F({d,e})=10、F({a,b,c})=6。

原圖縮合 DAG

GT 的根順序與已黑邊界



綠框依序收齊一類；灰虛箭頭可跨類，但不能遞迴到已黑頂點。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 22。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21 不相交集合

Part V Advanced Data Structures

本章定位

不相交集合把只合不分的partition轉成代表查詢；本章以同一組合併對照weighted list與rank forest，再由完整level/iter位能計費推導反Ackermann總界，區分單次成本、序列攤銷與實作前提。

前置知識

Representative、森林、rank 與路徑壓縮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較 linked-list 與 forest 表示。
- 用 union by rank 與 path compression。
- 理解 inverse Ackermann 攤銷界。
- 21.1 Disjoint-set operations
- 21.2 Linked-list representation of disjoint sets
- 21.3 Disjoint-set forests
- 21.4 Analysis of union by rank with path compression

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

21.1 Disjoint-set operations

維護的是 partition，代表元素只是 handle

Universe中的每個已建立元素恰屬一個非空集合，集合兩兩不交。MAKE-SET(x)要求x尚未建立，產生{x}；FIND-SET(x)要求x存在，回傳其集合的一個代表；UNION(x,y)將兩個集合合成一個。代表不自動等於最小鍵，內部parent亦不是原圖的邊。

本章令n為MAKE-SET次數，m為包含MAKE-SET、FIND、UNION的全部API呼叫數，m≥n；為固定分析參數，先完成n個單點再操作。成功合併最多n−1次，但重複UNION API可任意多。實作先取兩代表，若相同直接返回，不得把同一根重新LINK或增加rank。教材採此可重複呼叫的契約，LINK則只接受**不同的兩個根**。

FIND可能壓縮parent但不改partition；UNION只合不分。刪除元素、撤銷合併、動態刪圖邊或輸出全部成員，均不包含於三操作的既有保證，須另設結構。不同集合只要求代表不同，同一集合的所有FIND必回同一代表。

連通分量：為何合併後既不漏也不誤合

對無向圖，先MAKE-SET每個頂點；每來一條(u,v)就UNION(u,v)。不變量：處理的邊所形成子圖，其連通分量恰等於DSU partition。初始只有單點；新邊兩端同分量時兩者都不變，不同分量時恰用該邊接成一個分量，所以歸納成立。結束後FIND(u)=FIND(v)⇔存在原圖無向u-v路。

六點a..f依序加入ab、cd、bc、ab、ef、de：分量依序{ab},{c},{d},{e},{f}；{ab},{cd},{e},{f}；{abcd},{e},{f}；重複ab不變；{abcd},{ef}；{abcdef}。成功合併5次，API UNION有6次。

連通分量：為何合併後既不漏也不誤合 （續）

邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

			集合數
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

靜態圖DFS/BFS已可Θ(n+|E|)找分量；DSU的用途還包括邊逐一加入時即時查連通，或Kruskal辨別新邊是否成cycle。有向圖若忽略方向合併只得到弱連通，不是SCC。

21.2 Linked-list representation of disjoint sets

Set record 與每個成員的反向指標

每集合record存head、tail、size；每成員存next及set指標。代表是record.head的成員。空collection可O(1)建立；MAKE-SET產生單一節點，next=NIL、head=tail=x、size=1、x.set=新record。FIND只回x.set.head，O(1)。

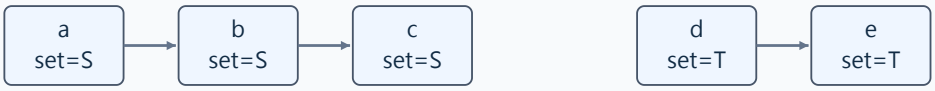
UNION取得S=x.set、T=y.set；若相同返回head。若S.size<T.size交換S,T，同大保留第一個S。將S.tail.next=T.head、S.tail=T.tail、S.size+=T.size，再逐一走T原串列，把每個成員的set改成S，最後廢棄舊record T。串列next順序未在走訪中改變，因此不會遺失後繼；不能只接tail卻不更新set。

一次UNION成本Θ(1+min(|S|,|T|))，不是O(1)。例如長串列a→b→c、短串列d→e，合併後a→b→c→d→e，僅d,e的set改指長record；tail由c改e、size由3改5，d/e的FIND才會回a。

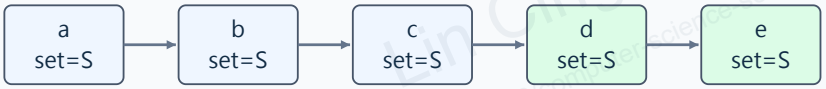
21.2 Linked-list representation of disjoint sets （續）

Weighted list：接 next 之外還要改 set

合併前 S: head=a, tail=c, size=3；T: head=d, tail=e, size=2。



短串列接到長尾；只改 d,e 的 set → S，並令 c.next=d。



合併後 S: head=a, tail=e, size=5；綠格 = 本次改set的成員；箭頭 = next。

不加權的平方反例與加權的 doubling 證明

若總把第二個集合接到第一個尾端，先n個單點，再做UNION(x_2,x_1)、UNION(x_3,x_2)、…，每次新的單點吸收之前整個長串列。改寫成員數依序1,2,…,n−1，合計n(n−1)/2；m=2n−1卻需Θ(n^2)，所以「有tail可O(1)串接」不等於整個UNION O(1)。

Weighted union每次只移短集合。固定成員x每次set改寫，所在集合大小至少倍增；改k次後至少2^k個成員，最多n，故k≤floor($\log_2 n$)。加總n個成員最多n floor($\log_2 n$)次改寫；每次API常數開銷另計m，總O(m+n log n)，小n用log(n+1)。單次兩個各n/2集合合併仍Θ(n)，不能把此總界解讀成每次最壞O(log n)。

上界亦有對應成長序列：$n=2^k$時，每輪成對合併同大小集合，k輪每輪恰改寫n/2個set，共(n/2)$\log_2 n$。MAKE/FIND的O(1)與整段總成本可以同時成立。

八元素的每次改寫帳

同大小保留第一集合。下列次序亦供森林版本使用；字母只是handle，不作大小比較。

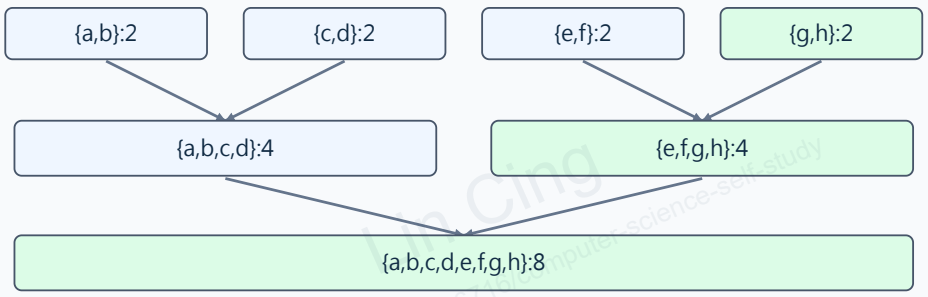
UNION	原大小	改set的成員	新代表／大小	累計改寫
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

h的set依h→g→e→a，改寫時新集合大小2→4→8，恰三次；a完全不必改。所有成員的次數加總12，並非每個成員都三次。若總以單點接到大集合的加權版本，前述平方反例會變成只有n−1次成員改寫。

八元素的每次改寫帳 （續）

成對同大合併：三輪的逐人改寫帳

每輪改寫4個set；綠框是 h 所在集合，大小2→4→8。



三輪總改寫 4+4+4=12；h改3次，a改0次；集合不分裂。

21.3 Disjoint-set forests

Root 自指與 LINK 的前提

每元素存parent、rank，根parent指向自己。MAKE-SET令parent[x]=x、rank[x]=0。UNION不是把任意x直接接到y，而是先找兩根；否則可能切斷原集合或製造cycle。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                    // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

接根後整個舊集合仍沿parent到新根；只有一個root失去自指，沒有反向可回的路，所以仍為森林。不同rank不增加新根rank；相同rank只使保留下來的新根加一。這是union by rank，與linked list按size比較是兩種不同表示法，不能把rank當精確成員數。

兩趟 FIND：覆寫前保存舊父

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

第一趟找根而不改任何指標；第二趟走原路，把沿途parent全部改r。若先覆寫parent[x]再用它繼續走，就會直接跳r而漏壓縮中間節點。遞迴寫法parent[x]=FIND(parent[x])等價於上行找根、返回時改連結；遞迴stack最多路徑高度，兩趟迭代輔助O(1)。

壓縮只把同樹節點改接其祖先根，根與partition都不變；不修改rank。一次走s個節點成本Θ(s)。森林本身O(n)空間，若還要列全部成員，可另維護成員串列；單靠parent不會自動提供O(集合大小)列舉。

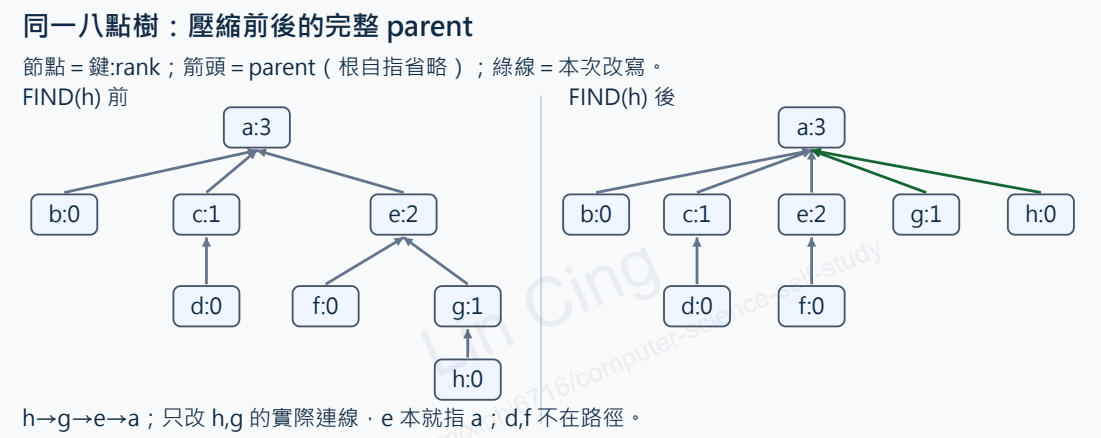
同一八元素的 rank 與壓縮前後

沿前述七次UNION，同rank保留第一根，根rank依a1,c1,e1,g1,a2,e2,a3。最終parent為b→a、c→a、d→c、e→a、f→e、g→e、h→g；a自指。

點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h)前parent	a	a	a	c	a	e	e	g
FIND(h)後parent	a	a	a	c	a	e	a	a

第一趟h→g→e→a三跳，第二趟h.parent=a、g.parent=a、e.parent=a（最後一個原本已如此）。d→c與f→e不在此find path，不改；rank全部保持。再FIND(h)只一跳，但FIND(d)仍兩跳，不能把一次find說成全樹扁平化。

同一八元素的 rank 與壓縮前後 （續）



Rank 的高度上界與歷史成員下界

對操作數歸納，root的目前高度≤其rank：單點為0；不同rank時小樹高度+1≤大rank，相同rank時最多原rank+1；壓縮只會縮短路徑，不增加高度。根rank=r的集合至少2^r個成員：只在兩個rank r−1根合併時升級，兩集合各至少2^{r-1}；其他合併只增加成員。非根的rank固定，沿用它最後為根時所在集合的歷史下界，因此任一rank≤floor($\log_2 n$)。

Rank不是目前高度的下界。例中a.rank=3，但壓縮全部元素後根只有高度1；rank=2的根亦能在四點星狀結構中高度1。它也不是目前subtree大小，因壓縮會移走後代，卻不減rank。

不用壓縮、只by-rank，單次FIND O(log n)，全部m次O(m log n)；成對合併出高度$\log_2 n$，再對深葉重複FIND便達對應量級。沒有by-rank時可先LINK成線性長鏈，使第一次FIND Θ(n)；不能把這種反例套到同時使用兩heuristics的定理。

21.4 Analysis of union by rank with path compression

Ackermann 的迭代不是普通次方

本節固定最終n與α(n)，在同一操作序列中不隨當前集合數改變α。記$f^{(0)}(j)=j$、$f^{(i+1)}(j)=f(f^{(i)}(j))$。對k≥0、j≥1，$A_0(j)=j+1$，$A_k(j)=A_{k-1}^{(j+1)}(j)$。上標是**函數合成次數**。

由$A_0^{(i)}(j)=j+i$得$A_1(j)=2j+1$；再對i歸納$A_1^{(i)}(j)=2^i(j+1)-1$，令i=j+1得$A_2(j)=2^{j+1}(j+1)-1$。所以$A_3(1)=A_2(A_2(1))=A_2(7)=2047$。

單調性亦由歸納建立：A_0嚴格遞增且$A_0(j)\geq j+1$；若A_{k-1}具有兩性質，從較大j開始並多迭代一次，結果必更大，故A_k對j嚴格遞增。又j+1≥2，至少兩次迭代給$A_k(j)>A_{k-1}(j)$；並有$A_k(j)\geq j+1$。以下level界與好節點不等式使用的正是這些單調性。

所用 inverse-Ackermann 版本

本章採用 Cormen 第三版 21.4 的單參數版本。對 `k>=0`、`j>=1` 定義：

- `A_0(j)=j+1`
- `A_k(j)=A_{k-1}^{(j+1)}(j)`，右上角表示把 `A_{k-1}` 迭代 `j+1` 次。
- `alpha(n)=min{k>=0 | A_k(1)>=n}`。

這與常看到的雙參數版本不同。`A_0(1)=2`、`A_1(1)=3`、`A_2(1)=7`、`A_3(1)=2047`、`A_4(1)>10^80`，所以 `alpha(2)=0`、`alpha(8)=3`、`alpha(2048)=4`。

定理的等價寫法會保留小 n 的基例：

`m` 次操作、`n` 次 MAKE-SET 的總成本為 `O(m(1+alpha(n)))`。

當 `n>=3` 時 `alpha(n)>=1`，才可化簡成教科書常見的 `O(m alpha(n))`。若直接對 n=0,1,2 寫 `O(m alpha(n))`，會錯誤地把上界寫成 0。

所用 inverse-Ackermann 版本 （續）

Ackermann 門檻與固定倍率的位能總帳

固定最終 n；$\alpha=\min\{k\ge 0: A_k(1)\ge n\}$；$\Psi=C\Phi$，$C>c$ 為固定正常數。

單參數門檻 $A_k(1)$	同一 Ψ 支付三種操作
$A_0(1)=2$	MAKE：新rank0，$\Delta\Psi=0$，成本 O(1)。
$A_1(1)=3$	LINK：$\Delta\Psi\le C\alpha$，實際成本 O(1)。
$A_2(1)=7$	FIND：s個點，$g\ge\max(0,s-(\alpha+2))$。
$A_3(1)=2047$	$\Delta\Psi\le -Cg$；$cs-Cg\le\max(c,C)(\alpha+2)$。
$A_4(1)>10^{80}$	Ψ初始0、終值≥0 ⇒ Σ實際 ≤ Σ攤銷。

$\alpha(2)=0$、$\alpha(8)=3$、$\alpha(2048)=4$；含小n基例的總界為 $O(m(1+\alpha(n)))$。

rank 與路徑的不變量

MAKE-SET 建立 rank 0 的單點。LINK 把 rank 小的根接到 rank 大的根；rank 相等時選定一個新根並把 rank 加一。FIND-SET 的 path compression 只改 parent，不改 rank。

rank 單調。 本節的圖論記法約定根 x 的 `x.parent=x`；實作若以 NIL 表示根，只需把 parent 判空改寫成同一語意。對操作次數作歸納，可證：

- 根 x 的 parent 指向自己，且其 rank 只在兩個同 rank 根合併時增加一；`MAKE-SET` 建立 rank 0 的根；
- 若 x 不是根，則 `x.rank < x.parent.rank`；
- x 一旦不再是根，其 rank 固定；父節點的 rank 只會增加。

因此任一條 find path 從 x 往根走時，rank 嚴格增加。

rank 的範圍。 對根 x 歸納可證以 x 為根的集合至少 `2^{x.rank}` 個元素。當兩個 rank r 根合併時，總大小至少 `2^r+2^r=2^{r+1}`；不同 rank 合併時較大根的大小不減。非根則沿用它失去根身分時的歷史rank下界。故 `rank(x)<=floor(log_2 n)<=n-1`。

對非根 x，`level(x)` 只看目前父 rank，因此父 rank 單調使 level 不會下降。`iter(x)` 則可能因 level 上升而重設並下降；不能寫成 level 與 iter 都不下降。

level、iter 與位能

令 `alpha=alpha(n)`。對非根且 `r=x.rank>=1` 的節點，定義：

- `level(x)=max{k | x.parent.rank>=A_k(r)}`
- `iter(x)=max{i | x.parent.rank>=A_level(x)^(i)(r)}`

由 `A_0(r)=r+1` 與父 rank 不小於 r+1，得 `level(x)>=0`。對 `n>=3`，`A_alpha(1)>=n>x.parent.rank`，而 `A_alpha(r)>=A_alpha(1)`，故 `level(x)<alpha`。

`iter(x)>=1` 來自 `x.parent.rank>=A_level(x)(r)`。若 `iter(x)>=r+1`，依迭代定義

`x.parent.rank>=A_level(x)^(r+1)(r)=A_{level(x)+1}(r)`，

與 `level(x)` 的最大性矛盾；所以 `1<=iter(x)<=r`。

定義各點位能並令 `Phi=sum phi(x)`：

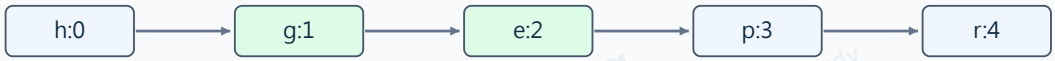
```
phi(x) = alpha * r                          若 x 是根或 r=0
phi(x) = (alpha-level(x))*r - iter(x)       若 x 非根且 r>=1
```

level、iter 與位能 （續）

第一分支是 `alpha` 乘 `r`，不是 `alpha-r`；因此 MAKE-SET 的 rank 0 點位能為 `alpha*0=0`。第二分支在 `0<=level<alpha`、`1<=iter<=r` 下滿足 `0<=phi(x)<alpha*r`。

FIND 的數值：level 與 iter 各自的前進

n=16, α=3；FIND(h) 後 h,g,e,p 都指根 r，rank 全部不變。



	h:0	g:1	e:2	p:3	r:4
前 (level,iter)	—	(0,1)	(0,1)	(0,1)	root
後 (level,iter)	—	(1,1)	(0,2)	(0,1)	root
	φ 0→0	φ 2→1	φ 5→4	φ 8→8	φ 12→12

綠節點 g,e 是好節點：本path位能27→25；rank0、root、最後同level點為例外。

level 上升時 iter 可下降

固定 rank r。若 level 不變而 iter 改變，父 rank 增加只會使同 level 可迭代次數增加，故此情形 iter 不下降。

若 level 由 k 升到 k'>k，iter 可以由 i 降到 i'，甚至回到 1。位能差仍是

```
phi_after-phi_before
= -(k'-k)r + (i-i').
```

因為 k'-k>=1，且由 `1<=i'`、`i<=r` 得 `i-i'<=r-1`，所以差值至多 `-r+(r-1)=-1`。也就是說 (`level` 越大越有利，同 `level` 下 `iter` 越大越有利) 的 lexicographic 次序嚴格增進，`phi` 至少下降 1。

因此前節的位能同時處理「iter 在同 level 上升」與「level 上升造成 iter 重設」；不需要也不可能主張 iter 永不下降。

FIND-SET 的好節點不等式

固定一次 path compression。設 x 在 find path 上，`r=rank(x)>=1`，且路徑後面有非根節點 y 滿足 `level(x)=level(y)=k`。當此 y 存在時稱 x 為好節點。

壓縮前，依三條定義與 rank 的單調性：

```
rank(y)>=rank(parent(x))
rank(parent(y))>=A_k(rank(y))
>=A_k(rank(parent(x)))
>=A_k^(iter(x)+1)(rank(x)).
```

路徑壓縮後，x 與 y 有同一個根，所以

`new parent(x).rank = new parent(y).rank >= old parent(y).rank >= A_k^(iter(x)+1)(rank(x))`。

若這個新界尚未觸發 level 上升，則 iter(x) 至少增加 1。若 iter 增加超過 r，依據 `iter<=rank` 的反證即 level 必須上升。故每一好節點在壓縮後都使 `(level,iter)` lexicographic 前進，前節已證明 phi 至少下降 1。

FIND-SET 的常數級 charging

find path 有 s 個節點。例外至多 `alpha+2` 個：rank 0 的起點（若存在）、根，以及每個 level `0,...,alpha-1` 在路徑上的最後一個節點。所以好節點至少有

`g=max(0,s-(alpha+2))`。

每個好節點令 Phi 至少下降 1。其他 path 節點不使位能增加。若掃描常數使實際成本至多 `c s`，現在明確定義放大後的總位能 `Psi=C Phi`，其中 `C>c` 是本次分析固定的常數。乘正常數不改變前述 ranking 不等式；好節點使 Psi 至少下降 C，所以 `Delta Psi<=-Cg`。因此

```
amortized FIND = actual+Delta Psi
<= c*s-C*g
<= max(c,C)(alpha+2).
```

右側是 `O(alpha+2)=O(1+alpha)`，不是粗糙的 `O(s)-Omega(s-alpha)=O(alpha)` 直接相消。

FIND-SET 的常數級 charging （續）

MAKE-SET 的實際成本 O(1)，新點 rank 0 的 Phi 為 0，所以 Psi 為 0。LINK 的實際成本 O(1)；附上的舊根失去root身分，位能不增；新根原有child因父rank不降，位能不增；只有新根可能在同 rank 合併時升一級，且 Phi 的增量至多 alpha，所以 Psi 的增量至多 $c\ \alpha$。LINK 的攤銷成本是 $O(1)+c\ \alpha=O(1+\alpha)$。因此 MAKE-SET 為 O(1)，LINK 為 $O(1+\alpha)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

總成本與適用範圍

把 UNION 轉成兩次對根的 FIND-SET 再加一次 LINK，操作數最多乘 3，不改變 O 級。把每次操作的實際成本記為 c_i、攤銷成本記為

$a_i=c_i+\Delta\ \Psi_i$。

對整段序列作 telescoping：

$$\sum_i a_i = \sum_i c_i + \Psi_{final}-\Psi_{initial}.$$

所以

$\sum_i c_i = \sum_i a_i-\Psi_{final}+\Psi_{initial}$。

初始資料結構為空，故 $\Psi_{initial}=0$；所有 $\phi(x)$ 非負，故 $\Psi_{final}\geq 0$。於是 $\Psi_{initial}-\Psi_{final}\leq 0$，整段實際成本滿足

$\sum_i c_i\leq\sum_i a_i$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

總成本與適用範圍 （續）

這一步把每次操作的攤銷上界轉成整段操作序列的實際上界。對轉換後序列套用前述 amortized FIND/LINK 界：

`O(m(1+alpha(n)))`。

再轉回原序列仍是同一個界。當 `n>=3` 時，`alpha(n)>=1`，才寫成 `O(m alpha(n))`；`n=0,1,2` 另由 `alpha=0` 的基例處理，不能把總工作量誤寫成 0。

這是整段操作序列的最壞情形上界，不是每次 FIND-SET 都最壞 O(alpha(n))。在 union by rank 下單次 FIND-SET 的路徑高度為 O(log n)，不需要也不能宣稱在該設定中仍存在 Omega(n) 長鏈；若移除 union-by-rank，任意連結才可能形成線性長鏈，但那就不是本章定理的設定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 21。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §21.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23 最小生成樹

Part VI Graph Algorithms

本章定位

最小生成樹以割上的安全邊逐步降低連接成本；本章從交換證明推導Kruskal與Prim，再用同一五點圖追蹤分量、key及parent，建立可獨立檢查的最優與唯一性證書。

前置知識

連通無向圖、cut 與權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cut 性質推導 MST 安全性。
- 比較 Kruskal 與 Prim 的資料結構與成本。
- 辨識負權、非連通與唯一性條件。
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

目標是樹的總權，並非每一條路的長度

給有限無向圖G=(V,E)，n=|V|、m=|E|；每條邊有可比較的有限實數權w(e)。先假設n≥1且G連通。Spanning tree（生成樹）T使用全部頂點、連通且無cycle，恰有n−1條邊；MST使$w(T) = \sum_{e \in T} w(e)$最小。有限個候選樹保證最小值存在，權可以為負、為零或重複。

負權不會破壞MST，但不能刪去「無cycle」條件：三角形三邊皆−1，樹成本−2；若容許一般連通子圖，取三邊成本−3。MST也不是最短路徑樹：在ab:4、ac:5、bc:2中，MST={ab,bc}成本6，a到c沿樹長6；最短路徑卻是ac長5。

n=1時回傳空邊集合、成本0；n=0約定空forest。不連通圖沒有生成樹，minimum spanning forest（MSF）則在每個原連通分量取一棵MST。自環不屬於任何樹；平行邊保留各自edge ID，不能僅以端點辨識刪除對象。

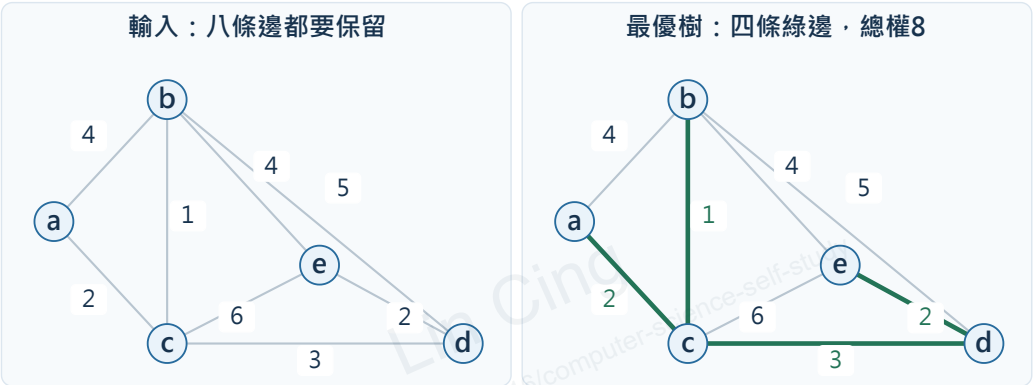
同一五點圖與兩種不同成長順序

本章固定邊表如下，所有後續狀態皆使用同一輸入。Kruskal同權依edge ID先後處理。

ID	0	1	2	3	4	5	6	7
邊	ab	ac	bc	bd	cd	ce	de	be
權	4	2	1	5	3	6	2	4

Kruskal按權選bc、ac、de、cd；Prim從a選ac、cb、cd、de。兩者最後都是{bc,ac,de,cd}，成本1+2+2+3=8，但中途的forest與grown set不同。圖中的灰邊也屬於輸入，不能只看最後綠樹就猜出選取過程。

同一五點圖與兩種不同成長順序 （續）



bc:1 + ac:2 + de:2 + cd:3 = 8

Safe 的意思是仍能延伸成某棵最優樹

維護不變量：目前A⊆E包含在至少一棵MST內。e∉A稱為safe for A，若A∪{e}也包含在某棵MST內。這是最優性的延伸條件，比「加入不成cycle」強；後者只保證還是一片forest。

Cut（割）(S,V−S)要求∅≠S≠V；邊的兩端分居兩側才叫crossing。割respect A表示沒有A中的邊跨割。Light edge是該割跨邊中的最小權者，可以並列。

通用算法由A=∅開始，反覆找safe edge加入，直到|A|=n−1。每次加一條且不變量保證A無cycle；n點n−1邊的forest必連通，所以回傳MST。問題在於如何有效地證明下一條邊safe，不能把「每次選便宜邊」當作證明。

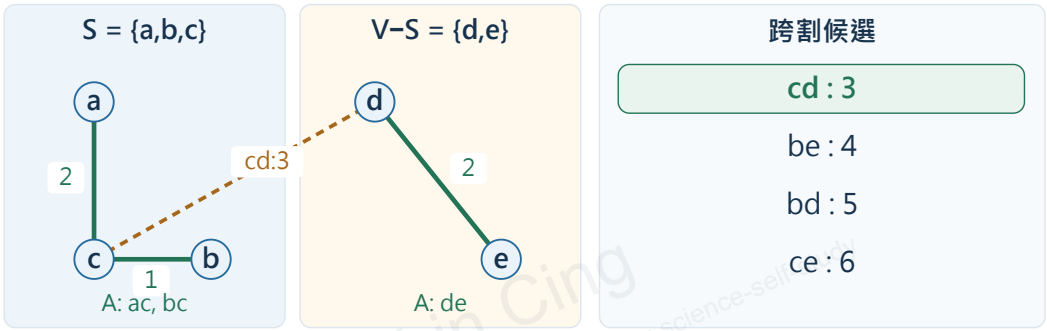
安全邊定理：交換時為何不會丟掉已選的邊

設A可延伸成MST，割(S,V−S)respect A，e是此割的一條light edge。取含A的MST T。若e∈T便已成立；否則T+e恰形成一個cycle。e跨割，cycle要回到起側必另經跨邊f。因割respect A，f∉A。

刪f後cycle被打斷且仍連通，所以T′=T−{f}+{e}仍是生成樹。由w(e)≤w(f)，w(T′)≤w(T)；T已最優，故T′也是MST，且A∪{e}⊆T′。這同時交代結構合法、權不增加、A不受損三件事。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

安全邊定理：交換時為何不會丟掉已選的邊 （續）



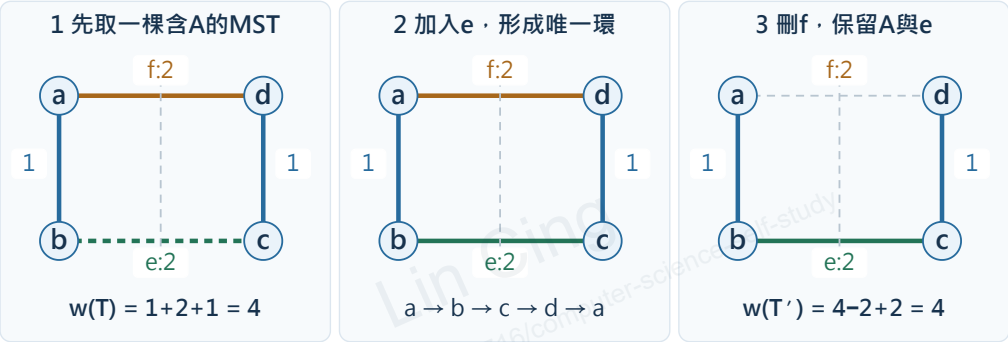
A內三條邊都沒有跨割；cd是最輕跨邊。

respect A：保留已選邊，再加入安全邊 cd

此例S={a,b,c}，A={ac,bc,de}；cd:3、be:4、bd:5、ce:6跨割，故cd safe。de位於另一側內部，並沒有跨割。

交換的三個時刻：一條邊進來，另一條出去

四點圖：ab:1、cd:1、ad:2、bc:2、ac:4、bd:5。A={ab,cd}，割為{a,b}｜{c,d}。



藍邊 A={ab,cd} 始終保留；f不在A中。綠虛線是待加入的e。

同權交換：最優值不變，但找到一棵包含e的MST

T用三條邊達到權重下界1+1+2=4。加入e=bc後，沿a-b-c-d-a的環刪去f=ad，成本4不變且保留A。**只保證存在含A與e的MST；原T就沒有e。**

並列、必選與cycle性質的量詞

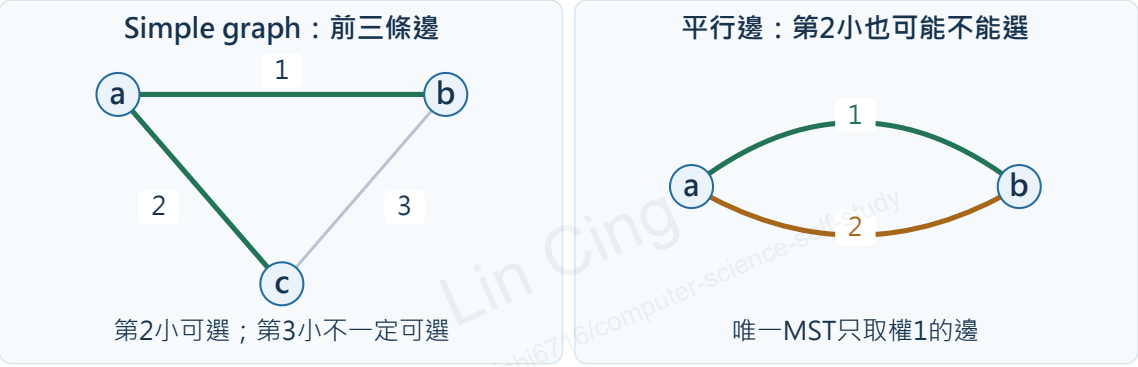
若e是某割的**唯一**light edge，任何不含e的MST都可用上述交換嚴格降低成本，矛盾；所以每棵MST都含e。若只是並列light，只能保證至少有一棵MST含e。三角形各邊權1時有三棵MST，每條邊都是某割的light，卻不能把三條一併選進樹。

Cycle性質：若e是某cycle的唯一最重邊，則e不屬於任何MST。假若MST T含e，刪e分成割；原cycle去e的路必有另一跨邊f，且w(f)<w(e)，交換便嚴格改善。若e只是並列最重，仍可證「存在一棵MST不含e」：從含e的MST做不增成本交換；不能說e一律不可選。

Forest A中任一連通分量C都給出respect A的割(C,V−C)。因此C對外最輕邊safe；這是Kruskal與Prim共同的依據，不是要求每輪全圖最小邊。

全圖第2小邊：結論取決於圖的模型

連通simple graph至少有兩條邊，按非下降權重排序，同權任意排。第2條e必屬某棵MST。



第2小邊的結論要求連通、無自環且無平行邊

若MST T未含e，T+e的環長至少3。除e外至少兩條邊，而全圖嚴格輕於e的邊至多一條，故環上有f不比e輕。刪f後仍最優，得到含e的MST。**平行邊使「環長至少3」失效；第3小邊也沒有相同保證。**

23.2 The algorithms of Kruskal and Prim

Kruskal：讓分量依全域權序合併

```
KRUSKAL(V, E):
    A = 空集合
    對每個 v∈V: MAKE-SET(v)
    將 E 按 (weight, edge-ID) 非下降排序
    對排序後每條 e=(u,v):
        ru = FIND(u); rv = FIND(v)
        若 ru != rv:
            A.add(e); UNION(ru,rv)
    回傳 A
```

DSU代表目前(V,A)的分量，不是原圖的最終連通分量。相同root表示已有一條A路徑，加入e會形成cycle（自環亦在此被拒）。不同root時兩棵樹用一邊連起，仍無cycle。連通輸入接受n−1邊即可提前返回；不必再掃剩餘cycle邊。若要支援MSF，亦可掃到輸入結束。

Kruskal 的一般正確性：被略過的便宜邊去了哪裡

假設接受前A可延伸成MST。對下一條接受的e=(u,v)，令S為u所在的當前DSU分量。S割respect A；若有比e更早的跨邊f，它早已處理過。當時若接受，兩端會合併且此後只合不分，不可能目前仍跨S；當時若拒絕，兩端當時已同分量，現在也仍同分量，也不可能跨S。因此沒有較早跨邊。

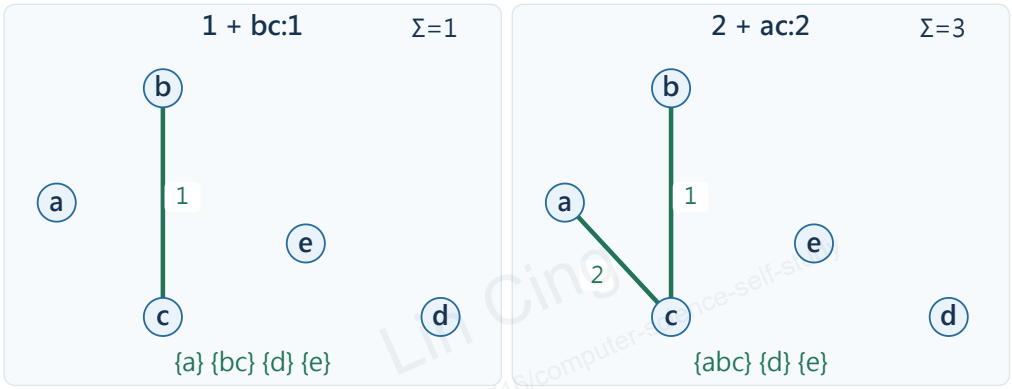
所有尚未掃的邊權不小於e，所以e是這一割的light edge。安全邊定理維持最優延伸不變量。結束時若原圖某分量仍被分成至少兩個A分量，原圖連通路上必有跨A分量邊；掃它時不會被拒，矛盾。故每個原分量都得到生成樹且最優，即MSF。

這個證明不假設權重皆不同，也不假設DSU root剛好是最小頂點；root是實作細節。

Kruskal 真正的每一步

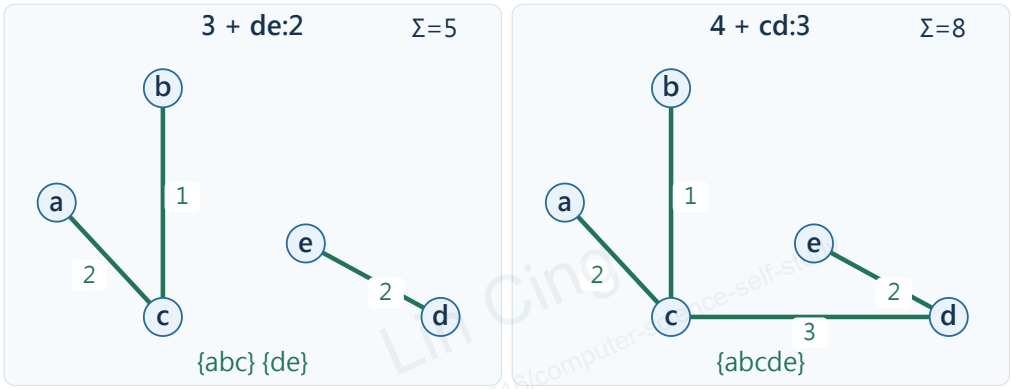
掃描	兩端原分量	決定	處理後分量	累計成本
bc:1	{b},{c}	接受	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	接受	{abc},{d},{e}	3
de:2	{d},{e}	接受	{abc},{de}	5
cd:3	{abc},{de}	接受	{abcde}	8
ab:4	同一分量	拒絕	不變	8
be:4	同一分量	拒絕	不變	8
bd:5	同一分量	拒絕	不變	8
ce:6	同一分量	拒絕	不變	8

Kruskal 真正的每一步 (續)



僅畫已選邊；框下的集合列出所有分量

先長出兩棵樹，再用一條邊合併



第3輪de另成一棵樹；Kruskal先長出forest

第4步已取n−1=4邊，可直接停止；後四列只用來核對完整掃描版本。第3步的de雖不接到{abc}，仍是合法的forest成長，這與Prim不同。

Kruskal 的成本與圖表示前提

比較排序O(m log(m+1))；DSU以union by rank及path compression處理n次MAKE-SET、2m次FIND、至多n−1次UNION，總O((n+m)(1+α(n)))。依第21章固定的版本，α(2)=0；只有n≥3時才能將1+α(n)簡寫成O(α(n))。加上讀入，完整成本為O(m log(m+1)+(n+m)(1+α(n)))；空圖直接返回空森林。排序邊表O(m)空間，DSU O(n)，輸出O(n)。

對連通**simple graph**，n−1≤m<n^2，log m=O(log n)，因此常寫O(m log n)。若允許任意多平行邊，不能再用m<n^2，必保留log m或先用額外演算法去除冗餘邊。負權與同權不影響比較排序。

整數權落在0..W時可用counting sort取代比較排序，成本O(m+W)，但DSU成本仍要加上；W很大時不會更好。這是改變鍵表示的假設，並非推翻比較排序下界。

Prim：key 是跨割的一條邊，不是路徑距離

設已取頂點S。每個v∉S維護$key[v] = \min_{u \in S, (u,v) \in E} w(u,v)$，沒有此邊則∞；parent[v]記實現該值的S端點及edge ID。所有未取頂點各有一個priority-queue項目。第一根r的key特設0，只為先取出，不能把0當輸出樹邊權。

每輪extract最小key。除第一根外，最小項u的parent邊恰是割(S,V−S)的light edge，所以可加入。取u後只需把新出現的候選w(u,v)與key[v]比較，不是key[u]+w(u,v)。

在ab:4、ac:5、bc:2例，取a後key[b]=4、key[c]=5；取b後key[c]=2，但a到c沿樹長4+2=6。Dijkstra則比較d[b]+2=6，保留d[c]=5。Prim允許負權；Dijkstra的正確性另需非負邊。

Prim 的完整 queue 契約

```
PRIM-CONNECTED(V, Adj, r):
    對每個 v: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=空集合
    當 Q 非空:
        u=EXTRACT-MIN(Q); inQ[u]=false
        若 key[u]==∞: 回報輸入不連通
        若 parent[u]!=NIL: A.add(parent[u]所記的邊)
        對 Adj[u]每條 e=(u,v):
            若 inQ[v] 且 w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    回傳 A
```

DECREASE-KEY視為同步更新heap與key儲存。inQ用陣列O(1)查，不可每次掃heap找會員。要輸出MSF時，extract∞代表新原分量：將該u設新根、key[u]=0、不加parent邊，再正常掃鄰居；不能把∞當樹成本。

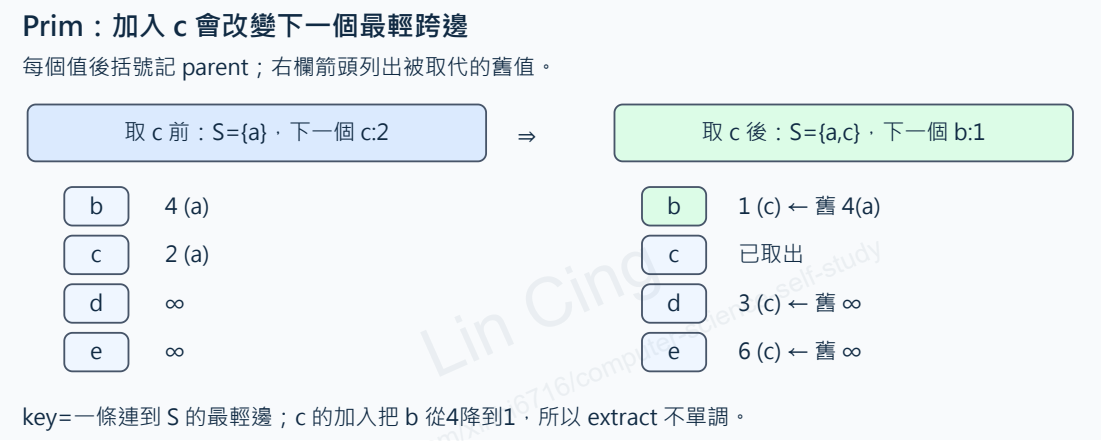
Prim 的不變量、終止與完整數值

每輪開始，S=V−Q，A在S上為一棵樹（第一輪前為空），且包含在某MST中；對Q中各點，key與parent恰表示至S最輕單邊。初始化以根的特例成立。extract後用割定理加入安全邊；掃u的鄰接串列恰補足所有由新點產生的跨邊候選，故不變量恢復。每次Q少一點，n輪終止，輸出n−1邊MST。

本輪extract	掃完後仍在Q的 key(parent)	已接受邊	成本
a:0	b4(a), c2(a), d∞, e∞	無	0
c:2	b1(c), d3(c), e6(c)	ac	2
b:1	d3(c), e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	空	ac,cb,cd,de	8

key extract序列0,2,1,3,2不是遞增。新點能提供更便宜跨邊；最優性靠cut而非key單調。

從新候選到下一條安全邊



圖左S={a}時ac:2最輕；取c後，b的候選由ab:4降成cb:1，d與e才第一次有有限key。下一步必取b:1，不能因「剛取過2」而排除權1。b被取後be:4取代ce:6；再取d才由de:2取代be:4。

每次parent更新要和同輪key一起保存；最後parent不是「第一次碰到的鄰居」。只把頂點順序列出，無法重建這些競爭候選。

Prim 的實作成本不能混用

queue／圖表示	工作拆解	總成本
indexed binary heap＋adjacency list	n次extract、至多2m次掃邊與m次decrease	$O((n+m)\log n)$
Fibonacci heap＋adjacency list	extract共$O(n \log n)$攤銷、decrease共$O(m)$攤銷	$O(m+n \log n)$
線性找最小key＋adjacency list	n輪各掃最多n個key，掃邊共2m	$O(n^2+m)$
adjacency matrix＋線性key	n輪各掃n個key與n個鄰位	$\Theta(n^2)$

對simple graph，$m=O(n^2)$，第三列亦$O(n^2)$。n=1的heap邊界直接返回空樹。heap需indexed handle讓decrease定位O(1)；改成每次push新項、取出時丟舊項的lazy版本，項目數可到O(m)，不可直接套只有n個項目的證明。

$m=\Theta(n)$時binary與Fibonacci均為$O(n \log n)$；$m=\Theta(n^2)$時兩者分別$O(n^2 \log n)$、$O(n^2)$，線性掃key也$O(n^2)$。實際例程用dense Prim驗算選邊，不假裝測量過Fibonacci速度。

不重跑算法也能檢核一棵候選樹

設T是生成樹。以下證書只量化**非自環的非樹邊**e=(u,v)、u≠v，令M_T(u,v)為T中唯一u-v路的最大邊權。自環永不能入樹，直接忽略；不對空的u-u路定最大邊。則

$$T \text{ 是MST} \iff \forall e=(u,v)\in E\setminus T,\ u\neq v:\ w(e)\geq M_T(u,v).$$

必要性：若某w(e)<M，加入e並刪路上最大邊f，仍是樹且嚴格便宜，矛盾。充分性：對任一f∈T，刪f形成割；其他跨邊e都不在T，且T上的e端點路經f，所以w(e)≥M_T≥w(f)。故f為此割light edge。依任意順序加入T的邊時，該割沒有其他T邊跨越，必respect已選prefix；反覆用安全邊定理，整棵T最優。

同一圖非樹邊ab、bd、ce、be的(w,M)依序為(4,2)、(5,3)、(6,3)、(4,3)，全部嚴格大於。因此此例MST唯一。

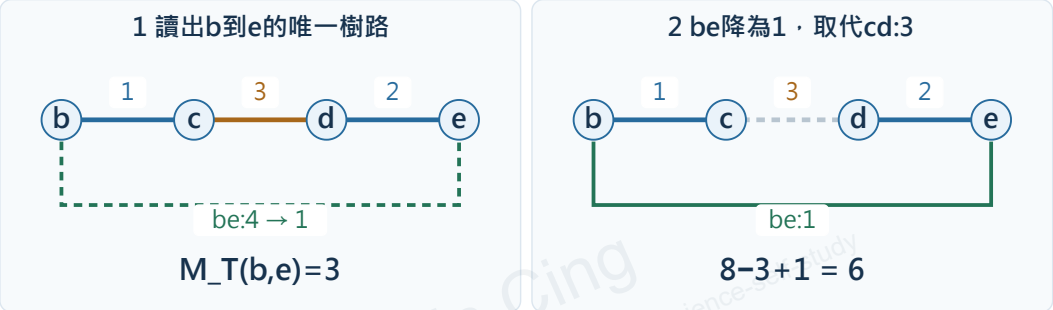
唯一性與邊權變動：證書的兩個用途

T唯一當且僅當每條非自環的非樹邊都有w(e)>M_T。若等號成立，交換一條最大路徑邊得到另一棵同成本樹。反之若全嚴格，前述每個T−f割上，f是唯一light edge，故每棵MST都含全部T邊。這不要求**所有輸入邊權互異**；本章兩條權2仍得到唯一MST。

若只將一條非自環的非樹邊e的權降成w′，其他不變，令M為原T路徑最大值。w′≥M時原T仍通過全部證書；w′<M時刪一條最大邊f並加e。其最優值為w(T)+min(0,w′−M)：不含e的樹權重未變，至少w(T)；任一含e的樹R，刪e分割後，T中u-v路有跨邊g；R−e+g是原權的樹，故w(T)≤w(R)−w′+w(g)≤w(R)−w′+M。這給與新構造相同的下界。

本圖把be由4降到1：T路b-c-d-e權1,3,2，刪cd:3，成本8−3+1=6。若只降到3，舊樹仍最優且出現並列。逐條DFS找路的證書成本O(nm)；這是單邊權變動的更新保證。自環無論多負都不能入樹，改其權不影響MST，亦不執行刪除樹路最大邊。

唯一性與邊權變動：證書的兩個用途 (續)



ac:2保持不變；圖只放大發生交換的b-c-d-e路徑。

新邊比路上最大權更小，才需要替換：w′ < M_T

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 23。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24 單源最短路徑

Part VI Graph Algorithms

本章定位

最短路徑以relax的上界及前驅重建串聯算法；本章逐步證明Bellman–Ford、DAG與Dijkstra各自的適用條件，再用負環證書、差分限制及根樹驗證區分有限距離、不可達與無下界。

前置知識

Edge relaxation、負權與可達性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 維持 shortest-path 與 triangle-inequality 不變量。
- 分別分析 Bellman-Ford、DAG 與 Dijkstra。
- 把 difference constraints 化為負權圖。
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

距離有三種語義，負邊與負環不同

有限有向圖G=(V,E)，n≥1、s∈V，每條邊為有限實數權。路徑成本是邊權和；容許重複頂點的walk可以包含cycle。定義δ(s,v)為所有s-v walk成本的下確界：不可達為∞；可經過負cycle再到v則為−∞；其餘有有限最小值。源點的零長walk成本0，所以δ(s,s)≤0。

若到v的walk沒有可用負cycle，刪去正cycle會改善，刪去零cycle不變，因此存在至多n−1邊的簡單最短路。負邊本身不使問題無解；負cycle也只影響其**由s可達且能往後到達**的點，其他分支仍可能有有限距離。

單目的地可反轉全部邊後以目的地為source；單點對可取單源輸出的指定項。無向負邊若表示成兩條反向arc，便形成可往返的負closed walk，不能沿用「只是單一負邊」的直覺。

Relax：只改進暫定上界

```
INITIALIZE(s):
    對每個 v: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u,v)):
    若 d[u]!=∞ 且 d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u,e.ID)
        回傳 changed=true
    否則回傳 false
```

每個有限d[v]是實際s-v walk成本（對relax次數歸納），故d[v]≥δ(s,v)。只做嚴格改善，等值不改parent。平行邊必記ID；初始化Θ(n)，每次relax在精確算術單位成本模型下O(1)。有限整數實作須防溢位，∞要獨立標記；任意把負值截到某下限，會毀掉負環偵測。

一旦d達到有限δ，之後既不能增加，也不能低於δ，所以永遠固定。此處使用「有限」避免把−∞當成某次普通有限relax的輸出。

Bellman–Ford 的操作、早停與成本

```
BELLMAN-FORD(G,s):
    INITIALIZE(s)
    重複最多 n−1 輪:
        changed=false
        對固定順序每條 e∈E:
            若 RELAX(e)成功: changed=true
        若 changed==false: 回傳 d,parent,「無可達負環」
    對每條 (u,v)∈E:
        若 d[u]!=∞ 且 d[v]>d[u]+w(u,v):
            回傳「存在可達負環」
    回傳 d,parent,「無可達負環」
```

最後一輪是測試，不把某個仍在下降的有限d當最短距離交付。若一整輪沒有更新，所有可達邊已滿足d[v]≤d[u]+w；之後同樣輸入再掃也無變，故可早停。n=1仍要測自環，負自環會在最後檢查被發現。

固定全輪版本成本Θ(n+nm)，在m≥1、n≥2時常寫Θ(nm)；早停最好Θ(n+m)，最壞不變。額外d、parent為O(n)，輸入邊表O(n+m)。負環的完整分類與證書可另加下述DP版本。

完整追蹤：逆向邊序需要多輪傳播

頂點依(s,a,b,c,t,z)。邊按下列**固定順序**掃描：ct:3、at:8、ac:1、bt:12、bc:5、ba:−2、sa:4、sb:1；箭頭方向均由第一字母至第二字母。z孤立。

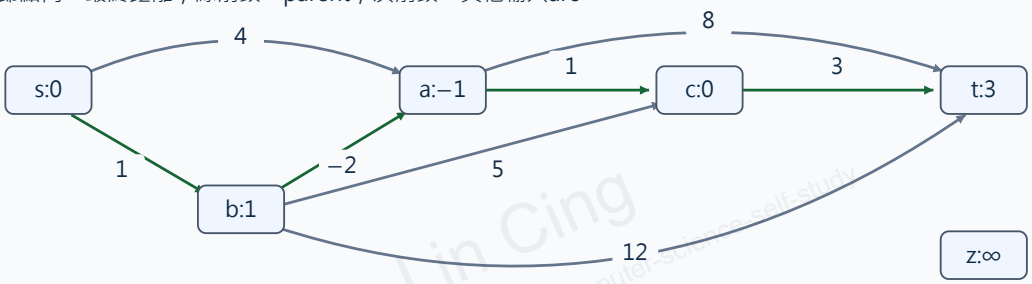
輪末	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

第2輪先以舊a=4產生t=12、c=5，稍後ba才把a改成−1。第3輪先ct得8，再at改7，ac才使c=0；第4輪ct才得3。第5輪無改動早停。最終parent為s→b→a→c→t，成本1−2+1+3=3。

完整追蹤：逆向邊序需要多輪傳播 （續）

Bellman–Ford：同一輸入與最終前驅

節點內 = 最終距離；綠箭頭 = parent；灰箭頭 = 其他輸入arc。



逆向掃描輪末：a 4→−1；c ∞→5→0；t ∞→12→7→3。

樹路 s→b→a→c→t：1−2+1+3=3；z 從 s 不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何 n−1 輪足夠，為何負環一定被看見

若無可達負環，任一可達v有簡單最短路$s=v_0,\cdots,v_k=v$，$k\le n-1$。第i輪必掃(v_{i-1},v_i)，且第i−1輪末$d[v_{i-1}]$已達δ；convergence給$d[v_i]=\delta$。即使輪內又處理更多邊，也不會破壞真值，所以到n−1輪全部正確。這只說原地版本能至少涵蓋i邊路徑，不說其值恰等於「最多i邊」的DP。

若有可達負cycle，各cycle頂點在n−1輪後都有限（任一可達點有≤n−1邊的可達簡單路，不要求該路最短）。若最後所有cycle邊都不能改善，沿圈相加$d[v]\le d[u]+w$，d項消去得到$0\le w(\text{cycle})<0$，矛盾。因此至少一條可達邊仍可改善。反之無負環時所有d已等δ，三角不等式排除任何改善。這是偵測的兩方向，不只以「值繼續降」作描述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

要交付負環證書：用明確的邊數 DP 重建

令$D_i[v]$為最多i邊的最小walk成本，$D_0[s]=0$、其餘∞。

$$D_i[v] = \min\left(D_{i-1}[v],\ \min_{(u,v)\in E}(D_{i-1}[u]+w(u,v))\right).$$

每一層只讀上一層；若保留舊值就記「skip」，若選一條邊就記其ID及上一層u。每個≤i邊walk不是≤i−1邊，就是刪去末邊後的一個≤i−1邊walk；反過來每個候選均合法，故兩向證明recurrence。

計算至第n層。若$D_n[v]<D_{n-1}[v]$，由記錄倒退n層得到**恰n邊**walk：少一邊便屬前一層，不會嚴格改善。分解該walk為一條簡單路及重複點截出的cycles。若所有cycles非負，刪掉它們得到≤n−1邊且成本不大的路，與嚴格改善矛盾。故至少一個截出的cycle權負，輸出其edge ID順序與總權就是可核對證書。

此完整記錄版時間O(n+nm)，$O(n^2+m)$空間；只存兩層可分類但不能保留上述回溯表。不要替此版本聲稱原地BF的O(n)輔助空間。

哪些點應標成 −∞

取seed集合$B=\{v:D_n[v]<D_{n-1}[v]\}$，沿**原圖方向**從B做BFS／DFS，所有到達點標−∞；其餘可達點取D_{n-1}，不可達保∞。

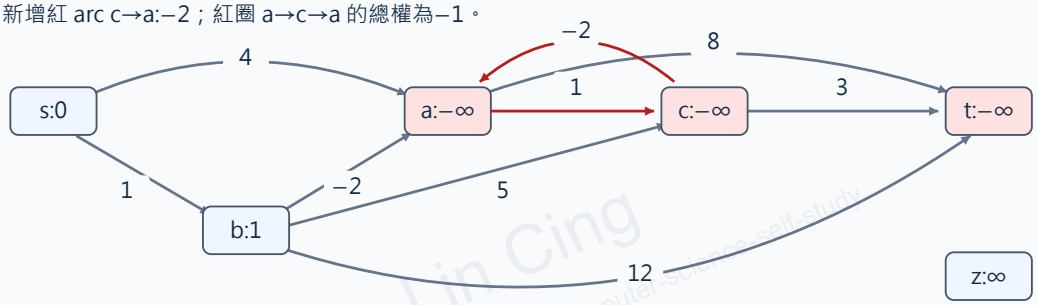
每個seed的n邊walk含負cycle，故seed及其後代都受影響。另一方面，任一可達負cycle的所有點在D_{n-1}已有限。若圈上沒有seed，則每邊$D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$，相加矛盾；故圈上至少一seed，所有受其影響的後代都會被遍歷。這證明標記範圍恰好，並非只標cycle本身。

沿前例加c→a:−2，a→c→a權1−2=−1，則(s,a,b,c,t,z)距離為(0,−∞,1,−∞,−∞,∞)。s、b到不了由cycle往回的出口，仍有限；z仍不可達。附帶一個獨立不可達負圈也不能讓s的BF誤報。

哪些點應標成 −∞ （續）

可達負環只影響其向前後代

新增紅 arc c→a:−2；紅圈 a→c→a 的總權為−1。



圈的後代 a,c,t 標−∞；b=1、s=0，因圈沒有回 s 或 b 的出口。

同一次負環偵測，不可把仍有限的分支或不可達 z 一起抹成−∞。

24.2 Single-source shortest paths in directed acyclic graphs

拓撲序讓每個候選都準時到達

```
DAG-SHORTEST(G,s):
    order=TOPOLOGICAL-SORT(G)
    若發現cycle: 拒絕此算法的輸入
    INITIALIZE(s)
    對 order 中每個 u:
        對 Adj[u] 每條 e: RELAX(e)
    回傳 d,parent
```

拓撲序保證任何s-v路上各邊依路徑先後被處理；path-relaxation使路徑終點取得δ。亦可對拓撲位置歸納：v被處理前，所有入邊起點均已固定且掃過，所以沒有未來才到的更好候選。不可達點保持∞。負邊可以有，因無cycle便無負cycle。拓撲排序及掃邊共Θ(n+m)，輔助O(n)。

前例本來就是DAG，取合法次序s,b,a,c,t,z：處理s後a4,b1；處理b後a−1,c6,t13；處理a後c0,t7；處理c後t3。一輪與BF四輪同結果，差在**頂點拓撲順序**而非改了成本定義。

關鍵路徑：最長路只在 DAG 中直接翻號

在DAG把每個權改−w，最短成本的負值便是原最長成本；或以L[s]=0、其他−∞並用max relax。一般有cycle圖不能這樣直接套DAG算法；正cycle會使最長walk無界。

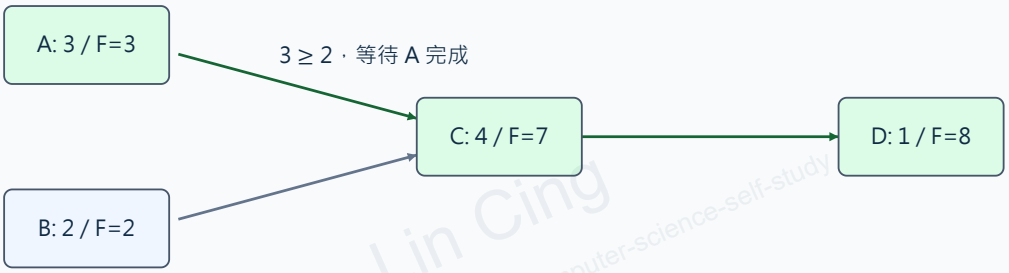
若工作是頂點，duration[v]≥0，依賴u→v表示u完成後v才能開始。最早完成F[v]=duration[v]+max({F[u]:u→v}∪{0})，依拓撲順序算；回溯實現最大值的前驅給critical path。

例如工作A:3、B:2、C:4、D:1，依賴A→C、B→C、C→D。F[A]=3、F[B]=2、F[C]=4+max(3,2)=7、F[D]=1+7=8；critical path A,C,D成本8。無限處理器且只受這些先後條件時，讓每工作一滿足前驅就開始，可達此下界8；有限處理器另有資源限制，不能只以此數當完整排程。

關鍵路徑：最長路只在 DAG 中直接翻號 （續）

工作 DAG：完成時間由較慢前驅決定

格內：工作 / duration / 最早完成 F；綠箭頭在 critical path。



F[C]=4+max(3,2)=7；F[D]=1+7=8。

無限處理器最早完成8；總工量10，一顆處理器仍至少需要10。

24.3 Dijkstra’s algorithm

取最小暫定距離後不再重新打開

此節明列所有邊w≥0。以indexed min-heap保存每個未取頂點各一項，key=d。初始化d[s]=0、其他∞、parent=NIL、inQ=true。

```
當 Q 非空:
    u=EXTRACT-MIN(Q); inQ[u]=false
    若 d[u]==∞: 停止（餘下皆不可達）
    對 Adj[u] 的 e=(u,v):
        若 inQ[v] 且 d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u,e.ID)
            DECREASE-KEY(Q,v,d[v])
```

已取S各點的d已是δ；與Prim相比，更新是**總路長d[u]+w**。已取點不更新也不重入Q。若要提早結束單點對查詢，須等目標真的extract，不能在它第一次被發現就停。

非負權究竟用在哪一行

對下一個有限extract u，若它為第一個錯誤，取一條真最短路s→⋯→x→y→⋯→u，y是路上第一個未取點，x已取。s第一個被取且d[s]=0；其餘情況x確已達δ並掃xy，故convergence保證d[y]=δ(s,y)。

從y到u的後綴權非負，得到δ(s,y)≤δ(s,u)。選最小d給d[u]≤d[y]；上界性給δ(s,u)≤d[u]。合併

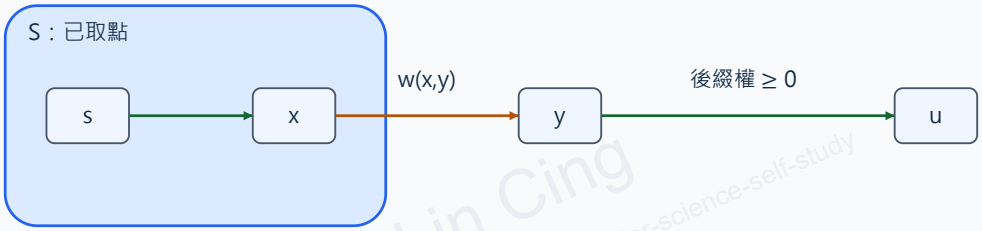
$$\delta(s,u) \le d[u] \le d[y] = \delta(s,y) \le \delta(s,u),$$

全部相等，矛盾，所以extract不會錯。零權邊與相同距離均可，無須嚴格遞增。∞被取時，若仍有可達未取點，該路第一未取點已有有限候選，與取到∞矛盾。

非負權究竟用在哪一行 （續）

Dijkstra：用第一個未取點夾住 u

只追蹤一條真最短路；橘箭頭是它第一次跨出 S 的 arc。



x 已固定且掃過 xy ⇒ d[y]=δ(s,y)。

最小提取 d[u]≤d[y]，非負後綴 δ(s,y)≤δ(s,u)，上下界夾成相等。

數值追蹤與負邊的真正反例

將BF前例只有ba改為+2。逐輪掃完：

extract	未取的有限 d(parent)	已固定距離
s0	a4(s),b1(s)	s0
b1	a3(b),c6(b),t13(b)	s0,b1
a3	c4(a),t11(a)	再加a3
c4	t7(c)	再加c4
t7	無；z∞	再加t7

最短樹s→b→a→c→t，路長1+2+1+3=7。候選t曾13→11→7，第一次發現t不能當最短值。

負邊反例用s→a:2、s→b:5、b→a:−4、a→t:2。取s後先固定a2並令t4，再固定t4，最後b5才提供a1與真正t3。標準不重開版本維持錯值；即使容許更新已取a而不重掃其出邊，t仍錯。此圖為DAG、沒有負環，失敗恰在δ(s,b)=5>δ(s,a)=1，使證明的非負後綴比較失效。

Queue、輸入表示與精確算術成本

indexed binary heap：建heap O(n)、n次extract共O(n log n)、每邊最多一次decrease共O(m log n)，總O((n+m)log n)，n=1特判。Fibonacci heap將decrease降至O(1)攤銷，總O(m+n log n)。每個heap項須能以handle定位，inQ用O(1)陣列查。

線性掃全部未取d配adjacency list為$O(n^2+m)$，simple graph可簡化$O(n^2)$。若以lazy heap推入所有新候選，項目可達m，需另證O((n+m)log(m+1))及O(n+m)空間，不能套只有n項的indexed版本成本。例程使用線性掃key及adjacency list做數值驗算。

上述是實數RAM模型；B位元整數、無負環的簡單路成本需O(B+log n)位元，精確加法／比較成本應另外計入。浮點誤差與任意epsilon可能改變等值和負環判斷；數學正確性並非自動等同浮點穩定性。

24.4 Difference constraints and shortest paths

限制的方向來自移項，不憑語感猜

限制$x_j-x_i\le b$對應arc $i\to j:b$，因為它等同$x_j\le x_i+b$。每列係數矩陣只有一個+1與一個−1，變數可為任意實數。若寫$x_j-x_i\ge b$，先轉成$x_i-x_j\le -b$。等式拆成兩個反向不等式。

加super-source q→每個變數點，權0；q無入邊，確保所有原負cycle都可達。無負cycle時設$x_i=\delta(q,i)$，由每邊三角不等式得$x_j-x_i\le b$，所有限制成立。若存在負cycle，把對應限制相加，左邊每個變數正負各一次全消，得到$0\le \Sigma b<0$，矛盾。這建立可行⇔無負cycle。

原n變數m限制，加q後n+1點、m+n邊，BF為$O(n^2+nm)$。改為原圖所有d初設0，相當已完成q的零邊，跑至多n−1輪原邊及一輪檢查，成本O(n+nm)，包括m=0時仍需初始化，不寫成0。

完整可行與不可行數例

限制為$x_2-x_1\le -2$、$x_3-x_2\le -1$、$x_1-x_3\le 4$，邊為1→2:−2、2→3:−1、3→1:4。全零開始依此序：第一次掃得到(0,−2,−3)，因−3+4=1≥0不改x_1；第二輪不變。

逐條驗證：−2−0=−2≤−2，−3−(−2)=−1≤−1，0−(−3)=3≤4。全體加3得(3,1,0)仍可行，因差值不變。一般可行域沿全一向量平移，沒有固定原點。

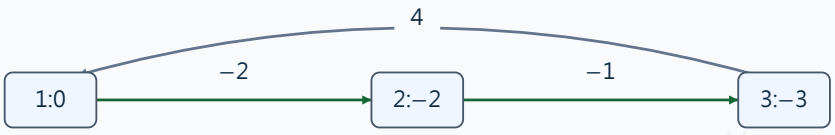
若把最後限制改$x_1-x_3\le 2$，三式相加為$0\le -2-1+2=-1$，無解。負cycle為1→2→3→1；不能只報某個候選不符合，就推所有候選都不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整可行與不可行數例 (續)

同一組限制：可行向量與負環矛盾

每條 arc i→j:b 對應 xj−xi≤b；圖內值為可行 x。



逐條差值：−2−0=−2；−3−(−2)=−1；0−(−3)=3≤4。

若最後權4改成2：三式相加得 0≤−2−1+2=−1，矛盾。

原圈總權1可行；改後總權−1無解。整體平移不改任何差值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些延伸仍保留這個證明

等式$x_j-x_i=b$拆成$i\to j:b$與$j\to i:-b$，兩向合起恰強制相等。欲表示$x_i\le c$及$-x_i\le c$，增加一個reference點r，用$r\to i:c$及$i\to r:c$；求解後整體平移使$x_r=0$，才能把它當絕對界，不能獨立令全部d任意平移卻忘記此基準。

整數右端時BF從整數0用加法/min計算，所得可行解自動整數。若要求**所有**變數整數且右端為實數，先將每個b換成floor(b)：因x_j-x_i為整數，兩個限制等價；再檢查負cycle。若只有部分變數要求整數，這個逐列floor方法不再自動適用。

所有$d\le 0$的規範下，最短距離還是逐座標最大的可行向量：任意可行x沿$q\to\cdots\to v$路反覆用限制，$x_v\le$路徑成本，故$x_v\le\delta(q,v)$。這是加入$x_i\le 0$後的結論，原平移不受限時Σx_i可任意增加。

24.5 Proofs of shortest-paths properties

最優子結構、三角不等式與收斂的完整鏈

假設本段沒有從s可達的負cycle。若一條有限最短路的子段不是最短，用更便宜子段替換便產生更便宜s-v walk，矛盾。對任何arc $u\to v$，接一條s-u最短路再接該arc是候選，故$\delta(s,v)\le\delta(s,u)+w$；u不可達時右邊$\infty$不增加限制。

上界性由初始化及每次有限d為walk成本證明；不可達點若曾變有限，就有實際walk，矛盾，故永保∞。Relax當下結束必有$d[v]\le d[u]+w$；**只保證此當下**，u日後再降，該邊可能要重掃。

若u已真最短且$u\to v$接在某條最短路上，該次relax後$d[v]\le\delta(s,u)+w=\delta(s,v)$，又上界性給反向，故v固定於真值（convergence）。依最短路的arc順序套此論證，第一點s已真值，歸納每次把下一點固定；插入其他relax亦只降上界且不破真值，故path-relaxation成立。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減

假設某次嚴格relax u→v第一次把parent子圖變成cycle。此前已有parent路v→⋯→u。每條舊parent arc x→y，在最後設定parent[y]時有d[y]=d[x]+w；其後y未更新，x只可能下降，所以本次更新前 d[y]≥d[x]+w。沿舊路相加得d[u]≥d[v]+w(v→⋯→u)。

新更新又要求d[v]>d[u]+w(u,v)，合併得cycle總權<0。路上各點皆有限，故由s可達，與假設矛盾。負自環亦直接由嚴格改善推出權負。所以parent子圖始終無cycle。

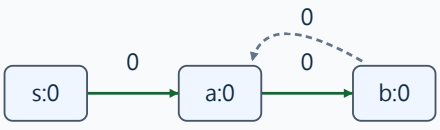
任何有限v≠s必由relax得到非NIL parent；s因d[s]=δ(s,s)=0固定，parent[s]=NIL。沿parent回溯，有限點的parent也有限；有限圖且無cycle，必停在唯一沒有parent的s。每個點只有一個parent，所以得到以s為根、包含所有已發現有限點的樹。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減 （續）

零權邊：距離相等仍須檢查 parent 根結構

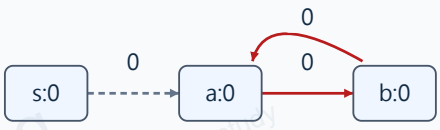
原圖 s→a:0、a→b:0、b→a:0；兩側全部距離都是0。

合法 parent：由 s 有根路



parent[a]=s，parent[b]=a

錯誤 parent：a,b 互為父



parent[a]=b，parent[b]=a

綠 = 合法樹 parent；紅 = 錯誤cycle；灰虛線 = 不在所選parent中的原arc。

Tight等式只驗成本，仍要從 s 遍歷 parent children，確認根連通且無環。

正確距離加合法 parent 才能交付最短路樹

當全部有限d已等δ，對當前parent arc u→v，最後更新時間給d[v]≥d[u]+w；三角不等式給δ(s,v)≤δ(s,u)+w，代入d後兩向合成等號。沿s-v樹路相加，內部d項望遠鏡消去，路權=d[v]−d[s]=δ(s,v)。不可達點仍∞且parent=NIL，故三項最短路樹條件全部成立。

不能改說沿parent距離嚴格下降：零權arc可以相等，負權arc沿正向還會下降；無cycle依賴剛才的嚴格更新時間證明。若不是由這種relax歷史產生，而是事後對每點任選tight predecessor，零權圈可被選成parent cycle。例如s→a:0、a→b:0、b→a:0，d均0；設parent[a]=b、parent[b]=a每邊都tight，卻沒有根路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性時間驗證輸出，連最優性也有證書

給候選d與parent edge IDs，先在原圖從s算可達集合R；要求R內d有限、d[s]=0，R外d=∞且parent=NIL。驗每個v∈R−{s}的parent確為輸入arc u→v，且d[v]=d[u]+w；用parent children做一次DFS，確認從s恰好遍歷R而無重複，parent[s]=NIL。再掃所有u∈R的輸入arc，驗d[v]≤d[u]+w。

樹路的tight性給一條成本d[v]的真路，故δ(s,v)≤d[v]。另一方面沿任意s-v walk把所有邊不等式相加得d[v]≤該walk成本，故d[v]≤δ(s,v)。兩向相等；可達負cycle也會違反相加不等式，所以證書拒絕它。驗證時間Θ(n+m)、空間O(n)，不需重跑BF或Dijkstra。

這同時解釋為何只驗邊不等式不夠（數值可能太小、沒有實現路徑），只驗tight parent不夠（可能零圈或另有更便宜輸入arc）；兩種證據各負責一個方向。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 24。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25 全點對最短路徑

Part VI Graph Algorithms

本章定位

全點對最短路徑有兩種不同DP狀態：邊數與中介點；本章推導min-plus、Floyd與Johnson，連結可重建路徑、零權圈、負環影響範圍及重加權的正負號與成本。

前置知識

矩陣最小值加法與前驅表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 把 min-plus 矩陣乘法接到最短路徑。
- 用 Floyd-Warshall 的中介點 DP 推導轉移。
- 用 Johnson 重加權處理負邊與稀疏圖。
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’ s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP（全點對最短路徑）的輸入是 n 個頂點的有向加權圖，邊權可為負。輸出每一對 (i,j) 的距離；沒有路徑為 $+\infty$。若存在 i 可達、且能走到 j 的負 cycle，任意重走該圈使成本無下界，這一對的距離定為 $-\infty$，並沒有可交付的有限最短路徑。

本節先假設無負 cycle。任何重複頂點的 walk 都包含非負 cycle，刪圈不增加成本；可反覆刪到 simple path。因此 $i \neq j$ 的最短路徑可取至多 $n-1$ 邊，$i=j$ 則可取零長路徑、成本0。

權矩陣 W[i,j] 是直接邊最小權，無邊為∞；W[i,i]=0 表示可不走邊。若要偵測負 self-loop，初始化必取 min(0,w(i,i))，不能把負 self-loop 覆蓋成0。下面的無負環數例對角線均0。

以最多邊數為狀態

令 $L^{(m)}[i,j]$ 為 i 到 j **最多 m 條邊的 walk** 最小權重；不是恰好 m 條邊。$L^{(0)}=I$，其中 I[i,i]=0、非對角為∞。按最後一條邊拆解，有

$L^{(m+1)}[i,j]=\min_k(L^{(m)}[i,k]+W[k,j])$。

任何非空候選可拆成最多 m 邊前綴加最後一邊；反過來每個有限項都可實際串接。較短的舊路徑也保留，因 k=j 時 W[j,j]=0。因此兩方向涵蓋同一組可行 walk，這是轉移等式的理由。即使有負 cycle，固定 m 的定義仍有限且有意義；只是不能在 $m=n-1$ 宣稱已得無限制的最短距離。

定義 min-plus product： $(A\otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$。若 A、B 分別對應最多 a、b 邊，串接最多 a+b 邊；反向沿任一最多 a+b 邊 walk，在第 min(a,實際長度) 邊切開即可，故 $L^{(a)}\otimes L^{(b)}=L^{(a+b)}$。

Min-plus 的代數與兩種算法

結合律來自同一個有限最小值：((A⊗B)⊗C)[i,j]=$\min_{p,\varphi}$(A[i,p]+B[p,q]+C[q,j])=(A⊗(B⊗C))[i,j]。I 是單位矩陣；∞是 min 的中性元，∞+有限值=∞。本階段不把−∞放入乘法，以免遇到∞+(−∞)沒有定義。

```
Extend(A,B):
  C=全∞的新 n×n 矩陣
  for i,j,k: C[i,j]=min(C[i,j],A[i,k]+B[k,j])
  return C
Slow(W): L=I; 重複n−1次 L=Extend(L,W); return L
Square(W): L=W; m=1
  while m<n−1: L=Extend(L,L); m=2m
  return L
```

Slow 做 Θ(n) 次 Θ(n^3) product，n≥2 時 Θ(n^4)；Square 做 ceil($\log_2$(n−1)) 次 squaring，O(n^3 log n)。保留兩個輸入與一個新輸出即可 O(n^2) 工作空間；不能一邊覆寫輸入而仍宣稱每次 product 精確對應同一舊邊數。n=0回空矩陣，n=1直接處理對角線。

普通乘法把 min 換成加法、把加法換成乘法，但 Strassen 的減法公式不能直接套入 min-plus；上述類比並未證明一般環上的快速乘法界也成立。

四點六邊的共同數例

以下全部矩陣、Floyd stages 與 Johnson 都使用：1→2:4、1→3:7、2→3:−2、2→4:6、3→4:3、4→2:1。頂點1無入邊，其餘三點 cycle 2→3→4→2 成本2，沒有負 cycle。

W 的 rows 為 [0,4,7,∞]、[∞,0,−2,6]、[∞,∞,0,3]、[∞,1,∞,0]。

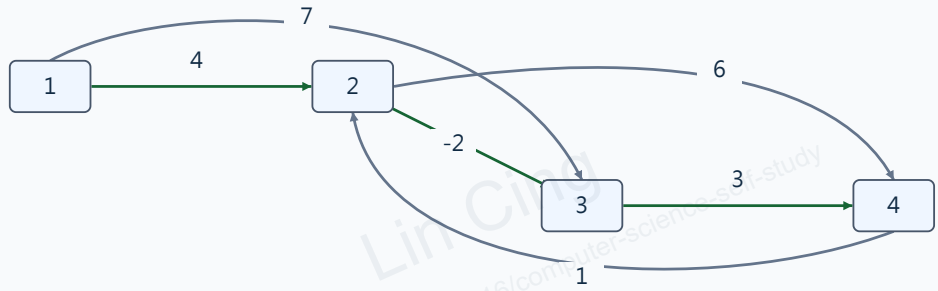
$L^{(2)}$ 的 rows 為 [0,4,2,10]、[∞,0,−2,1]、[∞,4,0,3]、[∞,1,−1,0]。

例如 $L^{(2)}$[1,4]=min(0+∞,4+6,7+3,∞+0)=10，兩條2邊路徑並列；$L^{(2)}$[2,4]=−2+3=1。第三次 Extend 才允許1→2→3→4，$L^{(3)}$[1,4]=5。其餘格不變；n=4，所以 $L^{(3)}$即答案，squaring 的 $L^{(4)}$也相同。

四點六邊的共同數例 （續）

六條有向邊與邊數受限的路徑

綠線 = 最佳3邊路徑；灰線 = 其餘原圖邊。邊旁數字是原權重。



1→2→4=10（2邊）；1→3→4=10（2邊）；1→2→3→4=5（3邊）。

25.2 The Floyd-Warshall algorithm

Floyd–Warshall 的 k 是**允許的中介頂點編號上界**，不是允許的邊數。$D^{(k)}[i,j]$ 是所有中介點在 $\{1,\cdots,k\}$ 的 i→j walk 最小成本；端點 i,j 不受這個集合限制。無負 cycle 時可取 simple path，因此 k 最多出現一次。

最優路徑若不經 k，屬於舊狀態 $D^{(k-1)}$；若經 k，在 k 切開，兩段內部不含 k，因此候選為 $D^{(k-1)}[i,k]+D^{(k-1)}[k,j]$。最優子結構可用替換論證：若任一子段有更便宜的合法替代，串接後成本更小，刪除非負 cycle 也不變差，與最優矛盾。反向串接兩個合法舊子路徑只增加可用中介點 k，故不會取得比狀態定義還小的不合法成本。

所以 $D^{(0)}=W$，且 $D^{(k)}[i,j]=\min(D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j])$。到 k=n 全部中介點皆允許，得到 APSP。逐 k 建新矩陣需 $\Theta(n^3)$ 時間；保留全部 stages 需 $\Theta(n^3)$ 空間，僅留舊／新兩份則 $\Theta(n^2)$。

原地更新為何成立；Loop Order 為何不能任換

無負 cycle 時，每個 stage 的 D[k,k]=0。固定 k 更新 row k 時，候選 D[k,k]+D[k,j]=D[k,j]，不會改值；column k 同理。因此本輪所有 D[i,k]、D[k,j] 都與上一輪相同，各 D[i,j] 一次更新恰實作原轉移，可以只用一個矩陣。

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i,k]!=∞ and D[k,j]!=∞:
        D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

$\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。有限整數實作須避免INF sentinel加負數變成「可達」，以及加法溢位；上面的可達判斷先做，精確算術的數學定理才適用。

反例：單位權鏈1→2→4→3，若依 i,j,k 順序只跑一遍，處理 i=1,j=3 時 D[1,4] 與 D[2,3] 都尚為∞，所以 D[1,3] 留∞；稍後 j=4 才算出 D[1,4]=2，卻不會回頭更新 j=3。正確 k,i,j 得 D[1,3]=3。

每個 k 的完整矩陣

rows/columns 都依1,2,3,4。$D^{(0)}=W$；k=1 因1無入邊，不新增路徑，$D^{(1)}=W$。

stage	row1	row2	row3	row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

k=2 更新三格：(1,3)取4−2=2、(1,4)取4+6=10、(4,3)取1−2=−1。k=3 更新(1,4)取2+3=5、(2,4)取−2+3=1。k=4 更新(3,2)取3+1=4。其餘格的舊值不大於候選，所以保留。

對照 $L^{(2)}$：它已允許2→3→4與3→4→2，但 $D^{(2)}$尚不允許中介3或4，所以兩格分別仍是6與∞。兩者都叫「第二步」，卻不是同一狀態。

每個 k 的完整矩陣 （續）

Floyd 的六次更新：兩個輸入到一個結果

每列只記嚴格改善；右兩個輸入都來自上一個 stage。其餘格不變。

k	(i,j)	舊值	D[i,k]	D[k,j]	新值
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

綠格 = min後的新值；k=1沒有更新，k=2三格、k=3兩格、k=4一格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑輸出：前驅不是中介點

Π[i,j] 表示某條 i→j 最短路徑上，**緊接 j 之前**的點；不是任意中介 k。初始化直接邊 i→j 的 Π[i,j]=i；i=j或不可達為 NIL。採兩個 stage buffer 時，嚴格改善經 k 的值，就取 Π新[i,j]=Π舊[k,j]，因為最後一邊來自 k→j 的後半段；否則沿用舊值。k=j時候選不會嚴格改善，所以不會用空路徑的 NIL覆蓋合法前驅。

等成本時保留舊路徑，不可隨意把 < 改成 ≤。最小例：直接邊1→2權1，在k=2時候選1+0=1；若等值也更新Π[1,2]←Π[2,2]=NIL，就把原本可重建的路徑弄斷。對零權 cycle，更不能只從最後的「距離緊邊」任意選父而不證明無環。

本例Π[1,4]的嚴格改善：k=2時取Π[2,4]=2，暫得1→2→4；k=3時取Π[3,4]=3，且Π[1,3]=2、Π[1,2]=1，最後反向鏈4←3←2←1，成本4−2+3=5。下面給一個額外的 tie rule，使完整 predecessor tree 的無環性可直接由整數下降證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零權 Cycle 下可證無環的前驅建構

把每格改為 pair (cost,hops)，先比 cost，成本相同再選 hops 小者；兩段相加時兩欄分別相加。初始對角(0,0)、直接邊(w,1)、無路(∞,∞)，同樣做 Floyd轉移。無負 cycle 時，正權圈讓cost變差，零權圈讓hops變差，所以 lexicographic（字典序）最優者必有一條simple path，前述分解與原地證明均成立，仍為$\Theta(n^3)$次固定欄運算。

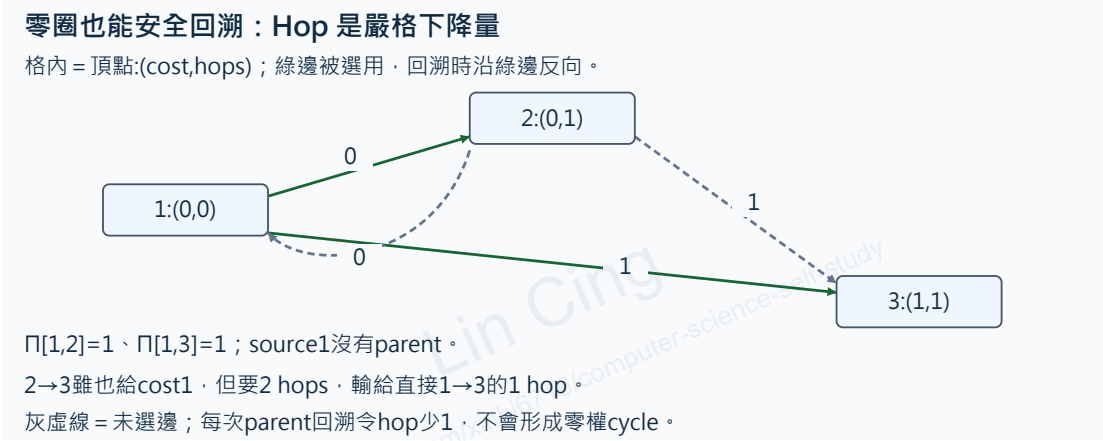
固定source s，對每個有限且v≠s的v，從原圖入邊選一條u→v，使

cost[s,v]=cost[s,u]+w(u,v)，hops[s,v]=hops[s,u]+1。

這樣的u必存在：取一條字典序最優s→v路徑的最後一邊，若其前綴不是字典序最優，可替換而改善整條路徑，矛盾。設定Π[s,v]=u；沿parent每步hops嚴格減1，非負整數不可能無限下降，也不可能有cycle。除了s外有限點都有parent，因此必到s，得到一棵確實重建最短路徑的tree。

對每個s掃全部m邊即可選父，總$O(nm)$，簡單圖$m \le n^2$時不超過Floyd的$\Theta(n^3)$；保存cost/hops/Π為$\Theta(n^2)$。例如1↔2皆權0、1→3與2→3皆權1，source1時hops=(0,1,1)，不能把1的parent設為2，亦不會在1、2之間循環。

零權 Cycle 下可證無環的前驅建構 （續）



負 Cycle 偵測與受影響點對

若跑完Floyd有D[k,k]<0，該值由實際邊串接而成，代表負的closed walk；分解成simple cycles，至少一圈權重負。反向若有負simple cycle，取其最大編號的點k，其餘點都可作k→k中介，Floyd必能找出成本負的closed walk，故某對角線會負。這給偵測的兩方向。

有負環時，固定stage仍能產生一些有限walk值；但它們不能當作受影響pair的最短距離，上一頁依D[k,k]=0的原地等價論證也不適用。標準k,i,j原地更新仍可用於負環偵測：所有有限值都有真walk見證，且不大於相應buffered候選，故不漏掉負對角；只是不同更新細節可得到不同暫存負數。

另算reachability R。只有滿足「存在k，D[k,k]<0、R[i,k]且R[k,j]」的(i,j)標為−∞。充分性是i先到負圈、重複任意多次再到j。必要性是：若i→j相關區域沒有負cycle，刪除其所有非負圈後，只剩有限多simple paths，最小成本有下界，不可能為−∞。不可達仍為∞；其餘pair的Floyd值維持有效。

標記後才交付結果，不把−∞再投入一般∞加法。 Johnson可直接拒絕整張有負環的圖；若要交付部分有限結果，則需這種pair層級語義。

不是整張圖都受負環影響

圖有1→2權2、2→3權−3、3→2權1、3→4權4，點5孤立。負圈2→3→2成本−2。

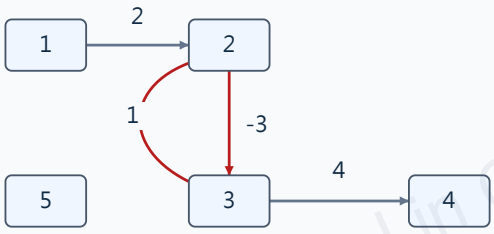
起點＼終點	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

1可以進入負圈再離開到4，所以δ(1,4)=−∞；但沒有回1的路，δ(1,1)=0。4、5不能進負圈，自己的零長距離不變。這比「發現負對角就把所有格標錯誤」更精確。

不是整張圖都受負環影響 （續）

負環影響的是9個點對，並非整張矩陣

紅邊負圈2→3→2成本−2；紅格只標能進入且能離開負圈的點對。



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

1能進圈但不能回1，故δ(1,1)=0；4與5不能進圈。

Transitive Closure：相同分解，不同代數

$R^{(0)}[i,j]=(i=j)$或存在$i\to j$；保留零長路徑，因此對角線true。把 min/+ 換成 OR/AND，得到

$R^{(k)}[i,j]=R^{(k-1)}[i,j]$ OR ($R^{(k-1)}[i,k]$ AND $R^{(k-1)}[k,j]$)。

有路徑時可刪除重複頂點，不管圈的權重；依是否使用中介k切開，兩方向涵蓋全部可達路徑。這是布林版本的正確性，不依賴無負cycle。原地更新也安全，因R[k,k]=true，而R[i,k] OR (R[i,k] AND true)仍是R[i,k]，row k同理。

六邊共同例的終值rows為 [1,1,1,1]、[0,1,1,1]、[0,1,1,1]、[0,1,1,1]。時間$\Theta(n^3)$布林操作，空間$\Theta(n^2)$布林值；bit packing可減儲存與利用word並行，但須明說word大小及採用的實作，不能無條件把bit操作當任意長字串一次完成。

25.3 Johnson’ s algorithm for sparse graphs

Johnson要把每條負邊改成非負，同時保持同一對端點所有路徑的相對成本。給任意有限potential $h:V\to R$，定義 $\hat{w}(u,v)=w(u,v)+h(u)-h(v)$。對路徑$v_0,\cdots,v_r$相加，內部h消去：

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$。

對固定起終點，增加量完全相同，所以成本的嚴格大小與等值都保留，最短路徑**集合**相同；不同tie-breaking仍可能挑出不同predecessor tree。對cycle起終同點，增加量0，故負cycle也完全保留；重加權不可能把原負cycle「修成可解」。

反例：對每條邊一律加常數C，長r的路徑會多rC，不是固定端點常數，可能改變最短路徑。1→4直達權6、1→2權2、2→4權2，原本後者權4較短；每邊加3後直達9而兩段10，答案反轉。

Potential 的存在、計算與負環拒絕

新增super-source s，向每個原點加權0邊，且沒有邊回s。若原圖無負cycle，新圖也沒有；每個v可達，Bellman–Ford得到有限$h(v)=\delta(s,v)$。對任一$u\to v$，接上邊可得$h(v)\le h(u)+w(u,v)$，重排就是$\hat{w}(u,v)\ge 0$，這才允許Dijkstra。

可不用實際存s的n條邊：先設每個h[v]=0（等價先放鬆所有s→v），再同步掃原圖邊n−1輪，最後檢查還有無h[v]>h[u]+w(u,v)。它相當於從任一原點零成本起跑、最多n−1條原邊；無負cycle的最短路徑可取simple，因此足夠。有任意負cycle時，各邊potential不等式不可能同時成立，沿cycle相加會推出$0\le$負數，所以最後必拒絕。

共同例初始h=(0,0,0,0)；第一輪只有2→3把h[3]降為−2，得到(0,0,−2,0)；第二、三輪不變。六條邊依序重加權為4、9、0、6、1、1，全部非負。這裡的h不是從原頂點1起跑的距離；h[2]=0而$\delta(1,2)=4$。

Potential 的存在、計算與負環拒絕 （續）

Johnson 的 Potential 與六條重加權

	h[1]=0		h[2]=0		h[3]=-2		h[4]=0
原邊		w	h[u]	h[v]		ŵ=w+h[u]−h[v]	
1→2		4	0	0		4+(0)−(0)=4	
1→3		7	0	-2		7+(0)−(-2)=9	
2→3		-2	0	-2		-2+(0)−(-2)=0	
2→4		6	0	0		6+(0)−(0)=6	
3→4		3	-2	0		3+(-2)−(0)=1	
4→2		1	0	0		1+(0)−(0)=1	

綠格全非負；h來自新增零邊source，不是從頂點1出發的原距離。

每個來源跑 Dijkstra，再還原距離

```
Johnson(G):
  h=前頁全零初始化的Bellman-Ford potential
  若仍可relax則回報負cycle並停止
  對每條原邊設 ŵ(u,v)=w(u,v)+h[u]-h[v]
  for source u in V:
    以非負ŵ在原圖頂點上做Dijkstra，得distHat[v]
    for v in V:
      D[u,v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

從1出發，初始distHat=(0,∞,∞,∞)。取出1後為(0,4,9,∞)；取出2後，3由9降4、4由∞降10；取出3後4由10降5；最後取4不改善2。故依序取出1:0、2:4、3:4、4:5（等距採編號小者先），終值(0,4,4,5)，還原為(0,4,2,5)。

其餘來源的distHat rows依序為 [∞,0,0,1]、[∞,2,0,1]、[∞,1,1,0]。各格減h[source]再加h[target]，得到前述Floyd最終矩陣。例如3→2原距離2−(−2)+0=4，4→3原距離1−0+(−2)=−1。不可達仍∞，不要對sentinel做有限數的還原。

每個來源跑 Dijkstra，再還原距離 （續）

非負圖的 Dijkstra 與原距離還原

重加權皆非負；格內是distHat。相同距離以編號小者先取。

動作	到1	到2	到3	到4
初始化	0	∞	∞	∞
取出1	0	4	9	∞
取出2	0	4	4	10
取出3	0	4	4	5
取出4	0	4	4	5

綠格 = 本輪嚴格改善；原距離 = distHat − h[1] + h[v]。

終值 (0,4,4,5) 還原為 (0,4,2,5)，與Floyd row1一致。

稀疏性、Priority Queue 與輸出成本

全零potential實作初始化O(n)、n−1輪各掃m邊、最後再掃m邊：O(nm+n)。若直接在擴增圖做一般Bellman–Ford，則為O((n+1)(m+n))，不要忘記新加的n條邊。

Fibonacci heap的Dijkstra為O(m+n log(n+1))攤銷時間；做n次並填n^2輸出，Johnson總O(nm+n^2 log(n+1))。Binary heap則是O(n(m+n)log(n+1))，不能引用前一個去掉m的log因子的界。額外圖與單趟狀態O(n+m)，若保存完整輸出另需Θ(n^2)；逐row串流可省輸出常駐空間，但沒有交付少於n^2格的同一輸出。

n=1000,m=5000時：Floyd有10^9個(i,j,k)更新；potential至多約5×10^6次原邊掃描，binary-heap重複Dijkstra的量級項約$1000\cdot6000\cdot\log_2 1000\approx5.98\times10^7$，Fibonacci-heap界的量級項則$5\times10^6+10^6\cdot\log_2 1000\approx1.50\times10^7$。這些是不同基本操作的成本模型，**不是實測執行時間或可直接相除的加速比**。稠密m=Θ(n^2)時Floyd簡單且Θ(n^3)，Johnson的稀疏優勢不再成立。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 25。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26 最大流

Part VI Graph Algorithms

本章定位

從可行流與割的上下界出發，說明增廣路為何需要撤回舊決策，再連到配對、節點容量與push–relabel。數例同時核對流量守恆、殘餘容量及最優性證書。

前置知識

有向圖、BFS與求和記號；本章會建立流量守恆、殘餘網路及割的定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 flow network 與 flow value。
- 用 residual graph 修正 Ford-Fulkerson 的增廣。
- 推導 max-flow min-cut 並連接 bipartite matching。
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

容量網路有有限頂點V、不同的source s與sink t、容量c(u,v)≥0。無self-loop，非邊容量0；平行同向邊可合併容量。記n=|V|≥2、m為正容量有向邊數；允許孤點與不在s-t路上的點。

本章統一採反對稱淨流f(u,v)=−f(v,u)，並要求f(u,v)≤c(u,v)，所以完整範圍是−c(v,u)≤f(u,v)≤c(u,v)。內點u≠s,t滿足Σ_v f(u,v)=0；流值|f|=Σ_v f(s,v)，是source淨流出，不是邊流絕對值的和。

不能再對全部有序對要求f≥0，否則反對稱會迫使f全為0。容量可為實數；時間先按精確算術操作數計，整數／有理數的位元成本另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非負 Arc Flow 與淨流不能混算

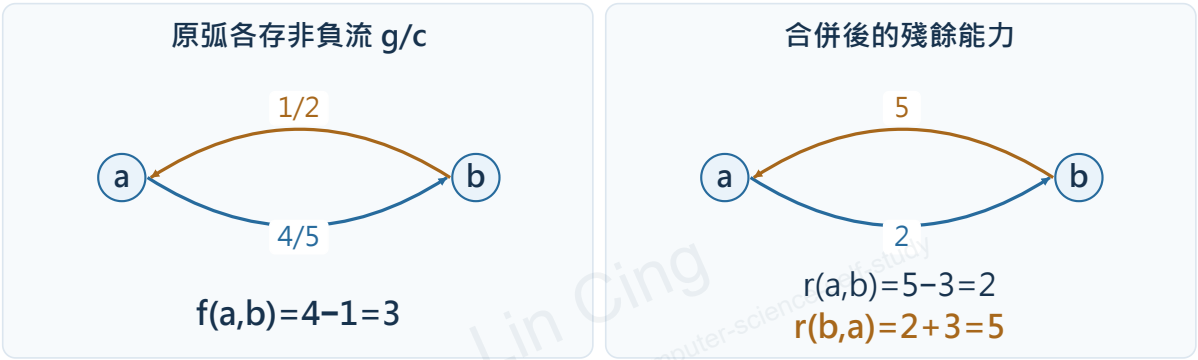
另一表示對原邊存0≤g(u,v)≤c(u,v)，不要求g反對稱。令f(u,v)=g(u,v)−g(v,u)，守恆與淨值不變；雙向都有流時，可取消min(g(u,v),g(v,u))而不改任何點的淨收支。

反過來，給淨流f，令g(u,v)=max(f(u,v),0)。容量上界使g≤c，且g(u,v)−g(v,u)=f(u,v)，故可恢復非負arc flow，兩模型的最大淨值相同。

例c(a,b)=5、c(b,a)=2，若g(a,b)=4、g(b,a)=1，淨f(a,b)=3、f(b,a)=−3。取消雙向各1後，原arc流成3與0。淨流的正向殘餘5−3=2，反向殘餘2−(−3)=5，包含撤回3與新增反向2的可能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反向容量與撤回舊流，各自貢獻多少？



反向5 = 原本可反向送2 + 可撤回淨流3

左圖只看一對反平行弧：c(a,b)=5、c(b,a)=2，原弧流為4與1，淨流為3。右圖b→a可增加5：先撤回淨流3，還能使用原本反向容量2。**殘餘弧不等於原圖有同方向的邊；也不能把反向殘餘一律當作只有撤流。**

Cut 的上界來自內部相消

Cut(S,T)分割全部V，s∈S、t∈T；c(S,T)=Σ_(u∈S,v∈T)c(u,v)，只算S→T方向。把S內所有點的淨流出相加，source貢獻|f|，其他內點守恆為0；S內每對f(u,v)+f(v,u)=0相消，所以

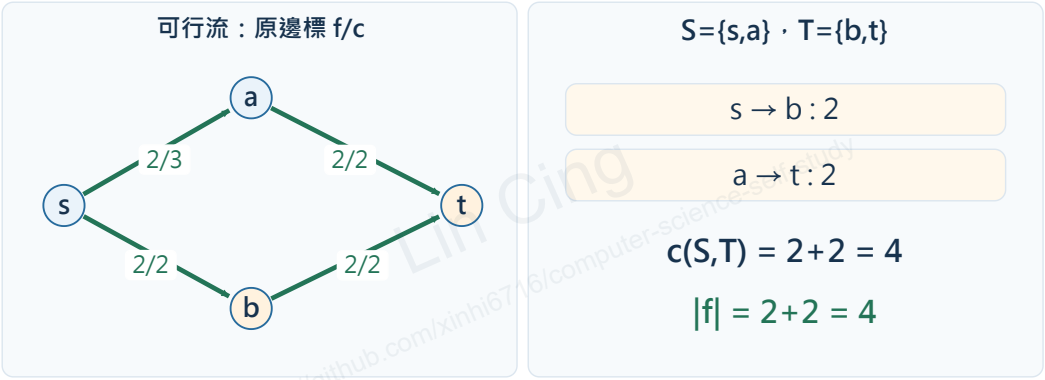
$$|f| = \sum_{u \in S, v \in T} f(u,v) \le \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

跨割必算淨值，不是只加正流。任何可行流給最大值下界，任何cut給上界；兩者相等便是可核驗的最佳性證書。

同一網路交付流與割兩份證書

容量s→a=3、a→t=2、s→b=2、b→t=2。各沿s→a→t、s→b→t送2，內點守恆且不超容量，流值4。

取S={s,a}、T={b,t}，跨割容量為s→b的2加a→t的2，總4。因此最大流同時≥4且≤4，答案就是4。



流給下界、割給上界；兩者相等，便證明最優

只給source總出容量或sink總入容量不能推定中間可達性；此例給了全部邊、可行流與等值cut，才足以證最優。

多 Source／Sink 與有限超邊

若供給端A、需求端B各無個別上限，且A∩B=∅，加supersource s'→各a∈A、各b∈B→supersink t'。超邊容量可用原邊容量總和C，不必實作∞；任何有效運輸量都不超過全部原容量和。若端點有各自上限，就用其上限，不能默認無限。

把原供需淨量接到超邊即得新守恆流；移去超邊則恢復原供需量，總值不變。這是兩向對應，不只是畫一個新source。

若使用非負arc表示且想消除antiparallel原邊，可將u→v容量c改成u→x→v、兩段各容量c。新點x守恆迫使兩段同流，與原邊一一對應。容量不能各改成c/2。

26.2 The Ford-Fulkerson method

殘餘容量$c_f(u,v)=c(u,v)-f(u,v)\geq 0$，residual graph只保留$c_f>0$的有向邊；鄰居候選必含原邊的反向，即使反向原容量0。

對residual簡單s-t路p，取$\delta=\min_{(uv\in p)}c_f(u,v)>0$。沿每條uv同時加$f(u,v)+=\delta$、$f(v,u)-=\delta$。正向不超容量，反向下降也不超其上界；反向下界則等價於正向上界。路內每點的入／出改變抵消，source淨流出增δ，故新f仍可行且值嚴格增加δ。

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u,v) for (u,v) in p)
    for (u,v) in p:
        f(u,v)+=delta; f(v,u)-=delta
return f
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣

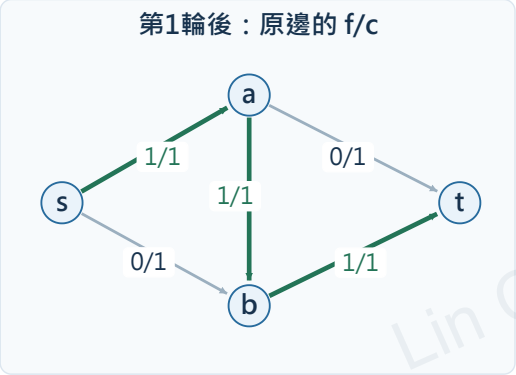
原邊s→a、s→b、a→b、a→t、b→t容量皆1：

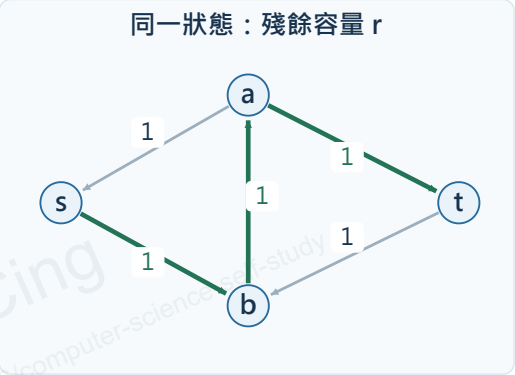
階段	路徑、δ	(f_sa,f_sb,f_ab,f_at,f_bt)	值
初始	—	0, 0, 0, 0, 0	0
第1次	s→a→b→t，1	1, 0, 1, 0, 1	1
第2次	s→b→a→t，1	1, 1, 0, 1, 1	2

第二路的b→a取消舊a→b流，使它由1回0。最後是s→a→t與s→b→t各1。若只保留原方向，第一輪後會錯把「沒有原方向路」當成「沒有residual路」，誤答1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣 （續）

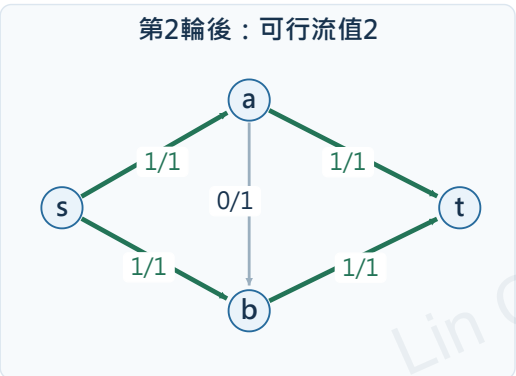




右圖綠路 s→b→a→t 的瓶頸為1；b→a 是撤回能力

若直接儲存殘餘邊，等價更新是r(u,v)−=δ、r(v,u)+=δ；只減正向容量會丟失撤回能力。

撤流後，逐條核對原邊與最優值



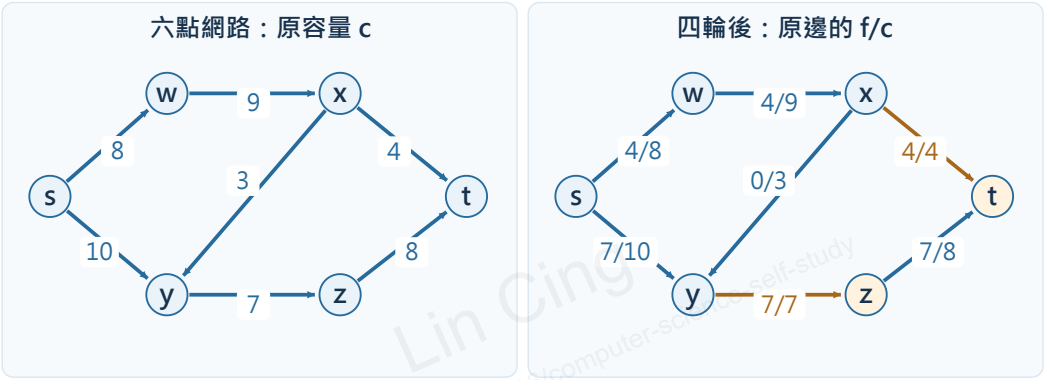
逐條原邊核對

s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b由1退到0；source兩條出邊已飽和，cut {s} 容量2

第二次增廣只更新s→b、b→a、a→t。b→a使淨f(a,b)減1，不是把a→b的原容量刪掉。最後a、b各收1送1；流值2等於割{s}容量2，故已最優。剩下的殘餘a→b仍有容量1，但source到不了a，不能用它再增廣。

六點網路：同一個最優值，可以怎樣走到？



S={s,w,x,y}：跨割容量 x→t:4 加 y→z:7，等於流值11

原圖只有x→y:3，沒有y→x。右圖是四輪增廣後的原邊流量f/c，並非殘餘容量。下面固定流量向量順序為(sw,wx,xy,xt,sy,yz,zt)，逐輪核對中間的撤流。

前兩輪：先走x→y，再沿y→x撤回

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
初始	—	—	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t；(8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t；(10,3,4)	3	(3,3,0,3,3,3,3)

第1輪才創造出殘餘y→x:3。第2輪沿它送3，使原f(x,y)由3變0；不是在原圖額外送一條y→x流。此時x的收支是wx:3流入、xt:3流出；y的收支是sy:3流入、yz:3流出。source淨流出3+3=6，兩輪都增加3。

後兩輪：用流量守恆與割完成檢核

向量仍依(sw,wx,xy,xt,sy,yz,zt)排列。

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
3	s-y-z-t；(7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t；(5,6,1)	1	(4,4,0,4,7,7,7)

流值6→10→11。最終w收4送4，x收4送4，y收7送7，z收7送7。殘餘從s可達S={s,w,x,y}，到不了z或t；原圖跨割只有x→t:4與y→z:7，合計11。**可行流11給下界，割11給上界，所以最大流恰為11。** 只看source原出容量8+10=18，無法得到這個答案。

Max-flow Min-cut 的三向等價

對可行f，以下等價：(i)f最大；(ii)residual無s-t路；(iii)存在cut使|f|=c(S,T)。

(i)⇒(ii)：若有路，增廣得到更大可行流，矛盾。

(ii)⇒(iii)：令S為residual從s可達點、T=V\S。t不可達，所以為合法cut。對u∈S,v∈T，c_f(u,v)=0，否則v也可達；故f(u,v)=c(u,v)，由cut淨流等式得|f|=c(S,T)。

(iii)⇒(i)：任一可行g都有|g|≤c(S,T)=|f|，所以f最大。

定理不要求整數容量，也不要求特定選路。整數性用於終止與成本，不是這個等價定理的前提。

整數性與偽多項式成本

整數容量從f=0開始，residual與δ皆整數，更新後仍整數。每次值至少增1，且有有限cut上界，所以必終止，並得到逐邊整數的最大淨流。設最優值F*，增廣至多F*次。

以adjacency list、一次O(n+m)配置及visited時間戳記，找路只初始化本輪觸及的點；每個新見非source點對應一條掃過的邊，每輪O(m+1)。總O(n+m+(m+1)F*)。F*可能對binary輸入長度指數大，因此是pseudopolynomial。

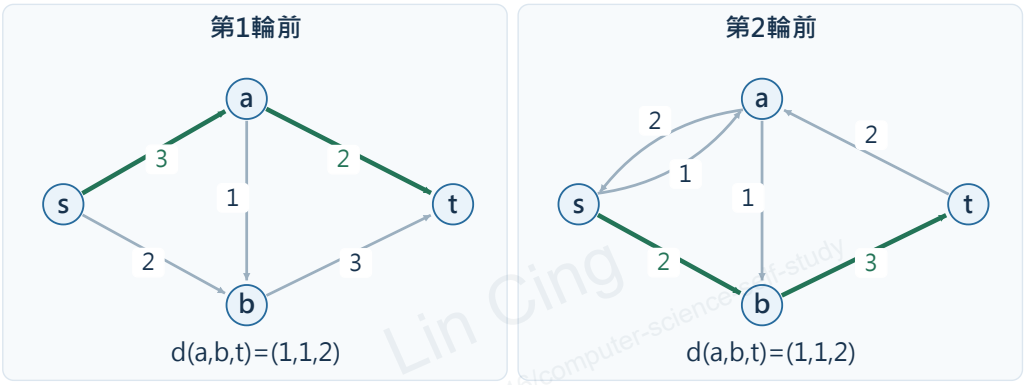
有理容量同乘分母乘積D可整數化；bitlen(D)不超各分母位元數總和，卻不代表D的數值小，所以仍可能有巨大增廣次數。無理容量下「每次至少1」不成立，不能由此證明宣稱任意選路終止。以下BFS版本改用容量值無關的輪數界。

Edmonds–Karp：殘餘邊數最短，不是容量最小

每輪以BFS找最少邊數s-t路，parent回溯增廣。d_f(v)為從s到v的最短邊數，不可達為∞。容量s→a=3、s→b=2、a→b=1、a→t=2、b→t=3，鄰居a先於b：

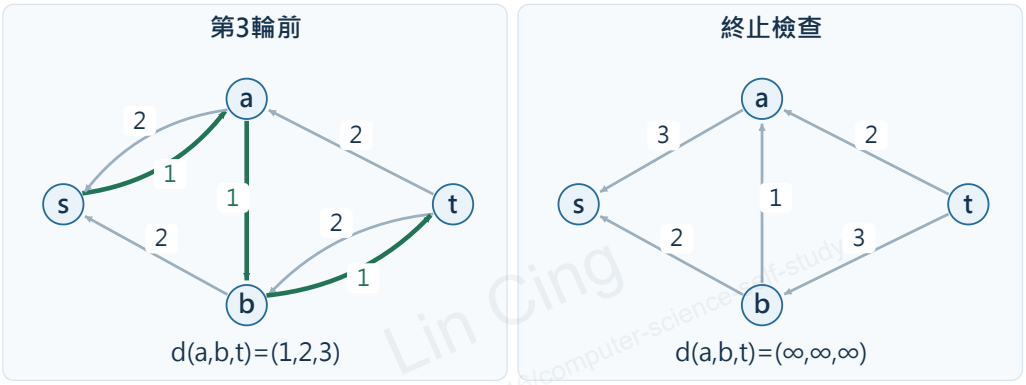
增廣前	d(a),d(b),d(t)	路徑與δ	增廣後f_sa,f_sb,f_ab,f_at,f_bt
第1輪	1, 1, 2	s→a→t，2	2, 0, 0, 2, 0
第2輪	1, 1, 2	s→b→t，2	2, 2, 0, 2, 2
第3輪	1, 2, 3	s→a→b→t，1	3, 2, 1, 2, 3
終止檢查	∞, ∞, ∞	無路	值5，cut {s}容量5

Edmonds–Karp：殘餘邊數最短，不是容量最小 (續)



僅畫r>0的弧；綠色是本輪最少邊數路，δ依次為2、2

第3輪與終止：從殘餘圖讀出割



第3輪δ=1；最終s無出弧，流值5等於cut {s} 容量5

第3輪走s→a→b→t，瓶頸min(1,1,1)=1，流值4→5。更新後只剩a→s:3、b→s:2、b→a:1、t→a:2、t→b:3。從s只能到自己，所以S={s}；原圖跨割容量3+2=5，與流值相等。BFS的∞表示不可達，不是容量為∞。

BFS 距離不下降的反證

增廣前後記d、d'。假設有d'(v)<d(v)，選這類點中d'(v)最小者；令u為新最短路的前驅，d'(u)=d'(v)−1。u不是更小的下降反例，所以d(u)≤d'(u)。

若uv本來就在residual，則d(v)≤d(u)+1≤d'(v)，矛盾。因此uv是剛出現的反向邊，vu在剛才BFS增廣路上。舊最短路的前綴也最短，故d(u)=d(v)+1。

合併得d(v)=d(u)−1≤d'(u)−1=d'(v)−2，與d'(v)<d(v)矛盾。原不可達、新可達的∞情形也適用：上述有限前驅關係會迫使舊v可達。所以每點距離單調不減。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

臨界邊再次飽和之前，距離至少加二

每次增廣至少一條critical邊uv達到瓶頸δ，殘餘變0；此時d(v)=d(u)+1。它要重現，之後必沿vu反向增廣；該輪d'(u)=d'(v)+1。由距離不下降，d'(u)≥d(v)+1=d(u)+2。

因此同一方向每次重新成為critical，尾點距離至少增加2；仍在s-t簡單路上時距離≤n−2，所以每方向O(n)次。候選方向至多2m，總O(nm)次增廣。

每輪BFS／更新O(m+1)，m≥1時總$O(n+m+nm^2)$，通常省初始化寫$O(nm^2)$；m=0直接零流O(n)。有理容量共分母整數化後，所有值的bit長受原輸入長度多項式控制，輪數又與數值無關，因此也是多項式bit演算法。$O(nm^2)$先指算術操作界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逃生問題：限制邊，和限制格點是兩回事

只限制邊會算出2；限制每個格點的通過人數才得到1

座標採(row,column)。可走格點為{(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}，其他格點封閉。兩人從u=(3,2)、v=(2,3)出發，出口為邊界x=(3,5)、y=(5,3)。只限制每條邊容量1，兩條不同路可各送1，卻都經過z=(3,3)。**若題目禁止共用格點，這個流值2並不代表兩人都能逃出。**

把每個格點拆成「進入」與「離開」

對k×k網格，令q為可走格點數、p為不同起點的人數、b為可用邊界出口數。禁止共用任何格點時，建構如下：

原限制	網路表示
每格至多通過一人	每個v拆成v_in→v_out，容量1
可移到相鄰可走格	u_out→v_in與v_out→u_in，各容量1
每個起點有一人	source→起點_in，容量1
從可走邊界離開	邊界_out→sink，容量1

障礙格不建立頂點或邊。起點若已在邊界，也要經自己的內部容量邊再離開。網路有N=2q+2個頂點，邊數M≤q+4k(k−1)+p+b；完整無障礙網格的b=4k−4只適用k≥2，k=1時b=1。

為何最大流等於p，才能讓所有人逃出？

若有p條格點互不重複的逃生路，沿對應網路路徑各送1，所有容量與守恆都滿足。反過來，整數容量保證存在整數最大流；將流分解成source–sink路徑與循環，移除循環不改流值。值為p時，p條source出邊全部飽和；每條內部容量1使任何兩條路都不能使用同一格點。投回原網格便得到每個人的逃生路。這證明兩方向都成立。

本例所有出口路都經z。拆點後的z_in→z_out容量1構成一個割，故最多一人；路u→z→(3,4)→x又給可行流1，所以最大值恰為1。只做邊容量的模型有兩條邊不共用的路，最大值為2；**兩個答案不同，是因為限制不同。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.3 Maximum bipartite matching

二分圖L∪R的matching是不共端點的邊集合。建立s→每個左點容量1、原邊l→r容量1、每個右點→t容量1。

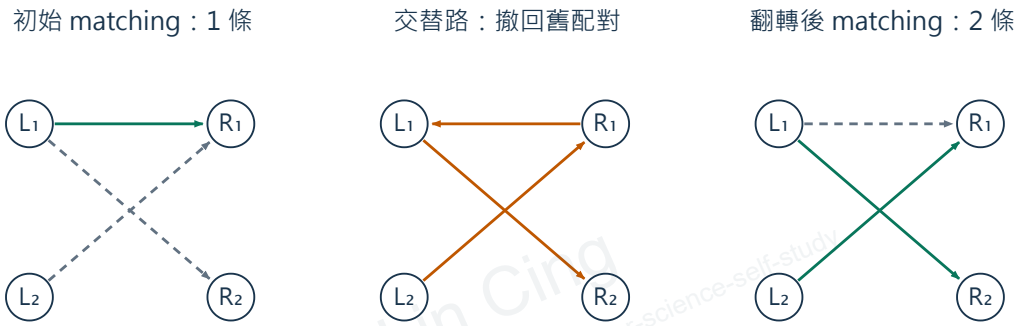
給matching M，沿每條s→l→r→t送1，端點互斥使容量不超，值|M|。反向給整數流：每個左點只有一條容量1入邊，守恆與整數性使其至多一條原匹配出邊為1；右點同理。因此f(l,r)=1的邊形成matching，大小等於流值。

整數增廣保證存在整數最大流，所以最大matching=最大flow。不能說每個最優流皆逐邊整數：$K_{2,2}$的四條左→右邊各送1/2、source與sink邊各1，也有最優值2，卻不能把四條正流邊全選作matching。

最大matching≤min(|L|,|R|)，每輪增1。新網路m'=|E|+|L|+|R|，直接Ford–Fulkerson成本O(|V|+m'+m'・min(|L|,|R|))。有孤點時不能把m'無條件寫成Θ(|E|)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條交替路實際翻轉哪些邊



$L_2 \to R_1 \to L_1 \to R_2$：未配、已配反向、未配

綠實線 = matching；灰虛線 = 未選原邊；橘箭頭 = residual 交替路

L_1可配R_1/R_2，L_2只可配R_1。先配L_1R_1後仍有residual路$s\to L_2\to R_1\to L_1\to R_2\to t$。反向$R_1\to L_1$撤回舊匹配，另取$L_2R_1$、$L_1R_2$，大小由1增2。

Alternating path的未配／已配邊交替；翻轉後每個內點仍接一條matching邊，兩個原未配端點各新增一條。

Kőnig 等式的可重建證明

取最大matching的residual，Z為從s可達點，$C=(L\setminus Z)\cup(R\cap Z)$。對原邊lr，若$l\in Z, r\notin Z$：未配邊有正向residual使r可達；已配時$s\to l$已飽和，能到l的最後邊只能從其匹配右點r反向而來，仍使r可達。兩種都矛盾，所以C覆蓋全部原邊。

每條matching邊的兩端同時可達或不可達：右端可沿反向到左；已配左端可達必先經它的右端。因此C對每條matching恰取一端。未配左點都可由s到達，不入C；未配右點不可達，否則能增廣，也不入C。所以$|C|=|M|$。

任一cover至少為matching每邊取一個不同端點，故二分圖最小cover大小=最大matching大小。一般圖不成立，例如三角形兩值為2與1。

Hall 條件的充分性

要匹配L全部點，必要條件是對每個A⊆L都有|N(A)|≥|A|，因matching給A不同鄰居。

若最大matching未覆蓋L，從未配左點做交替可達搜尋，得到左集合A、右集合B。前頁正反向論證給B=N(A)。所有B點已配，否則可增廣；它們的匹配左端都在A，而A還有至少一個起始未配左點。因此|A|>|B|=|N(A)|，違反Hall條件。

所以必要條件也充分。L空時空matching已覆蓋L，無須尋找未配起點。此結論講的是覆蓋全部左側；若左右同大，才同時成為兩側的perfect matching。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.4 Push-relabel algorithms

Preflow仍滿足反對稱與容量界，但允許暫存excess e(u)=Σ_v f(v,u)≥0，u≠s。Active只指u≠s,t且e(u)>0；sink雖累積excess，仍不discharge。

有效height滿足h(s)=n、h(t)=0，每條residual uv有h(u)≤h(v)+1。初始化其他height與f/e全0，設h(s)=n並飽和source出邊：f(s,v)=c(s,v)、f(v,s)=−c(s,v)，e(v)+=c(s,v)、e(s)−=c(s,v)。

初態所有非source excess非負；source無正殘餘出邊，其他邊從height0出發也滿足標號。Height不是每輪BFS的真距離，而是約束殘餘邊的有效標號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Push／Relabel 的完整條件

```
push(u,v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u],c(u,v)-f(u,v))
    f(u,v)+=d; f(v,u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u,v)>0)
```

Admissible指正殘餘、恰下降1層。Push後殘餘0稱saturating；否則nonsaturating，必使e(u)=0；兩者相等按saturating計。

Active必有正殘餘出邊，否則每個f(u,v)=c(u,v)≥0，導致e(u)=−Σf(u,v)≤0，矛盾。故relabel的min非空。無admissible時，整數有效標號使h(u)≤所有殘餘鄰居height，新height至少增1。

基本操作保持 Preflow 與有效 Height

Push不使e(u)變負，其他非source excess只增；容量／反對稱同增廣證明。Height不變，唯一可能新增的反向vu滿足h(v)=h(u)−1，所以仍有效，但不會admissible。

Relabel不改f/e。新h(u)=1+min h(v)使u出邊有效；對入邊wu，舊有效性有h(w)≤old_h(u)+1≤new_h(u)，所以入邊仍有效，且不admissible。Relabel後至少一條取到最小鄰居height的出邊成admissible。

Admissible network是DAG：沿每邊height減1，繞長k>0的cycle會推出h=h−k，矛盾。Push不新增admissible邊，relabel只可能新增自己的出邊，這兩點支撐後面的list拓撲不變量。

Active 點總有路回 Source，故 Height 有界

令R為active點u在residual可達集合，假設s∉R。R內非source excess皆非負、e(u)>0，所以 Σ_(v∈R)e(v)>0。但R沒有正殘餘出邊，故對v∈R,w∉R，f(v,w)=c(v,w)≥0；內部反對稱抵消給

Σ_(v∈R)e(v)=−Σ_(v∈R,w∉R)f(v,w)≤0，矛盾。

所以u有至多n−1邊的residual簡單路回s。沿路相加有效標號得h(u)≤h(s)+n−1=2n−1。每次relabel時u仍active，故新height同受此界；其餘時height不變。每內點最多2n−1次relabel，全圖O(n^2)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

終止時為最大流，還需另證一定終止

若全部內點excess0，preflow恢復守恆，是可行flow。若residual有s-t簡單路、長k≤n−1，沿有效標號得到n=h(s)≤h(t)+k=k≤n−1，矛盾。因此無增廣路，由max-flow min-cut得最大流。

Sink得到的excess是流值；一開始推出的多餘流可以回source，不必全進sink。

「終止時正確」不等於「必會終止」。接著分開限制relabel、saturating push與nonsaturating push數量，取得容量值無關的有限界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

飽和 Push 總數 O(nm)

固定方向uv。首次飽和時h(u)=h(v)+1且殘餘歸0；要再正向推送，必先有vu反向push恢復殘餘。該步h(v)=h(u)+1，height又不降，所以v至少比前次增2。

到uv再次飽和，u又需比v高1，因此u也至少比前次正向飽和時高2。Height≤2n−1，每個方向最多O(n)次；候選方向≤2m，總O(nm)。

此界依高度差與單調性，而非每次δ大小，對小數容量也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非飽和 Push 的勢能計費

取Φ=Σ_(active u)h(u)，初始0、始終非負。Relabel增加Φ恰為該height增加量，所有點全程height總增加≤(n−2)(2n−1)=O(n^2)。

飽和push只可能使目的點新active，Φ增加至多2n−1；共O(nm)次，正增加共O(n^2m)。非飽和push清空u；若v新active，最多加h(v)=h(u)−1，若v已active或為終端則不加。因此Φ至少減1。

由非負勢能，這類下降次數不超全部正增加，為O(n^2+n^2m)，m≥1時O(n^2m)。連同relabel與飽和push，generic基本操作共O(n^2m)；m=0時初始化後已無active，直接終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.5 The relabel-to-front algorithm

標準linked list L始終含V\{s,t}全部點，包括e=0者；不是只存active。固定N(u)={v：c(u,v)>0或c(v,u)>0}，含反向候選。current跨discharge保存：

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u,v)>0 and h[u]==h[v]+1:
            push(u,v) # 此步不前進current
        else: current[u]=next-neighbor(v)
```

飽和後下一迭代才略過該邊；若推送已清空u，立刻返回，current仍指原鄰居。不能把「每push自動前進」的另一版程式與此trace混搭。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

略過的 Current Arc 為何不用立即重看

固定u高度不變的一段。若uv因殘餘0略過，未來只能經vu反向push恢復；當時h(v)=h(u)+1，新uv往高處走，仍非admissible。若有殘餘但不admissible，有效標號給h(u)≤h(v)，v之後只升高，也不會成下降1層。

亦可從一般性質看：push不新增admissible邊；鄰居relabel後無admissible入邊。只有u自身relabel才可能新增自己的admissible出邊，屆時current重設。

所以掃到NIL表示所有已略過邊現在仍不可推，relabel確實合法；不是因為「曾經看過」就忽略邊的狀態改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Move-front 後讀取新位置的 Next

```
initialize_preflow()
L=all vertices except s,t, in any fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # 移動後的新位置
```

Move在整次discharge結束後做，不是每次內部relabel就做。中間u移到head後，next是舊head，會重查先前可能收到excess的點。不能保存舊next直接跳過它們。

List 不變量的兩種分支

每次外圈測試：(1)L為非s/t點的admissible network拓撲序；(2)u之前的點皆無excess。初態非source高度皆0，彼此無admissible邊，任意L符合，u在head。

若discharge無relabel，push不新增admissible邊，L仍拓撲有序；只把excess推到後方點或終端，不影響前綴。u最後清空，下一個u以前仍全無excess。

若曾relabel，最後一次relabel後u無admissible入邊，其後push也不新增；移到head保持拓撲序。新next之前只有剛清空的u，前綴條件恢復，其他點相對順序不變。

到u=NIL時，全部L皆為無excess前綴，故守恆，結合generic正確性即最大流。這個證明不能直接套到另一種只存active點的排程。

五點網路的初始化與 a 的完整 Discharge

容量s→a=4、s→b=3、s→c=2、a→b=2、a→t=1、b→c=2、b→t=4、c→t=5。固定N(a)=[b,t,s]、N(b)=[c,t,a,s]、N(c)=[t,b,s]，L=[a,b,c]。

初始化h(s)=5、其餘0，飽和source出邊。以下在每個push/relabel剛完成時記current，未合併下一次掃描：

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)
初始化	4, 3, 2, 0	0, 0, 0	b,c,t
a掃b,t,s到NIL；relabel	4, 3, 2, 0	1, 0, 0	b,c,t
a→b推2	2, 5, 2, 0	1, 0, 0	b,c,t
略b到t；a→t推1	1, 5, 2, 1	1, 0, 0	t,c,t
掃t,s到NIL；再relabel	1, 5, 2, 1	6, 0, 0	b,c,t
略b,t到s；a→s退1	0, 5, 2, 1	6, 0, 0	s,c,t

五點網路的初始化與 a 的完整 Discharge （續）

a的height從0變6，結束後move-front；因原在head，L不變，接著掃b。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

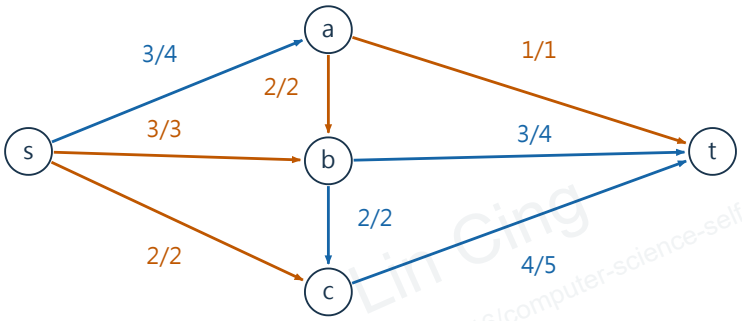
b、c 的 Discharge 與真正移動時點

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)	L
b掃c,t,a,s到NIL；relabel	0, 5, 2, 1	6, 1, 0	s,c,t	a,b,c
b→c推2	0, 3, 4, 1	6, 1, 0	s,c,t	a,b,c
略c到t；b→t推3	0, 0, 4, 4	6, 1, 0	s,t,t	a,b,c
b結束後移front	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
再掃a，e=0無動作	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
c掃t,b,s到NIL；relabel	0, 0, 4, 4	6, 1, 1	s,t,t	b,a,c
c→t推4；結束後移front	0, 0, 0, 8	6, 1, 1	s,t,t	c,b,a

下一點是新位置的b，接著a，皆無excess，最後NIL。b→c後t仍1，只有b→t後才變4。最終source淨流3+3+2=8、sink淨入1+3+4=8。

以 Cut 獨立核對最後的最大流

最後的可行流：每邊標 f/c



h(a,b,c)=(6,1,1)

e(a,b,c)=(0,0,0)

current=(s,t,t)

L=[c,b,a]

e(s)=−8 · e(t)=8

residual 可達 S={s,a}；橘色跨割容量 3+2+2+1=8

a收3送2+1；b收3+2送2+3；c收2+2送4。殘餘可達S={s,a}，跨割原容量3+2+2+1=8，等於可行流值8。初始推出9，其中1由a退回source，所以最後淨流值為8；這不是漏流。割證書獨立於height和排程。

$O(n^3)$ 的完整計費

總relabel $O(n^2)$。兩次相鄰relabel間稱一個phase；沒有relabel時discharge使u沿L向後走，最多n次，含relabel的一次使進入新phase。因此discharge呼叫$O(n^3)$；move-front持有效雙向串列handle可$O(1)$。

每點current在兩次自身relabel之間掃鄰居一次，自身relabel $O(n)$次，全部advances與重標找min合計$O(n\Sigma deg(u))=O(nm)$。飽和push $O(nm)$。每次discharge中非飽和push一旦發生就清空u並返回，所以至多一次，總$O(n^3)$。

合計$O(n^3+nm)=O(n^3)$，因簡單容量網路$m \le n(n-1)$。稀疏鄰接／反向記錄、height/excess/current與L共$O(n+m)$空間。這先是算術次數界，容量加法與比較仍依實際bit長付費。

Generic 操作次數不是免費的選步成本

若每步都掃全部頂點找active，再重掃全部鄰居，不能僅由基本操作$O(n^2m)$就宣稱同一時間界。可另維護active集合：push只改兩端excess，以handle與標記$O(1)$加入／移除，任取active點處理。

鄰居用同一current規則，全部advances與relabel掃描$O(nm)$，push本身$O(1)$，才能實現初始化後$O(n^2m)$算術時間。Relabel-to-front以固定全體串列及discharge另得$O(n^3)$，不是把active集合改個名字。

整數Ford–Fulkerson依F^*、Edmonds–Karp依nm^2、generic依n^2m、relabel-to-front依n^3，是不同機制的最壞操作界，不直接代表某台機器的實測速度。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 26。

Cormen et al., *Introduction to Algorithms*, 3rd ed., § 26.5.

《演算法_wjungle》手寫筆記，網路流、逃生問題與節點容量，PDF實體第37–42頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33 計算幾何

Part VII Selected Topics

本章定位

幾何演算法必須把數學判定、退化情形及資料結構順序一致處理；本章以方向測試、交線、凸包及最近點對證明局部判斷如何形成正確全域結果。

前置知識

向量、叉積、排序與掃描線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cross product 判斷線段方向與轉向。
- 以 sweep line 找任兩線段相交。
- 分析 convex hull 與 closest pair。
- 33.1 Line-segment properties
- 33.2 Determining whether any pair of segments intersects
- 33.3 Finding the convex hull
- 33.4 Finding the closest pair of points

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.1 Line-segment properties

平面點p=(x,y)，閉線段ab={a+t(b−a)：0≤t≤1}。閉線段的交集包含端點接觸、共線重疊及退化為單點的線段，不只真正穿越。

定義方向判定

$$\operatorname{orient}(a,b,c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

本章採一般向右的x軸、向上的y軸：正為a→b→c左轉（counterclockwise），負為右轉，零為共線。這是有向平行四邊形面積；三角形面積為其絕對值的一半。若改用(b−a)與(c−a)的相反叉積順序，正負判準也必反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方向判定要精確，不能任意套 Epsilon

若整數座標絕對值≤B，座標差≤2B，兩乘積之差的絕對值≤8B²。儲存原座標的整數型別未必能儲存乘積；必須先擴寬再相減／相乘，或使用任意精度整數。b位元座標的判定涉及O(b)位元整數的乘法，並非任意輸入都能O(1)。

浮點運算可能因相消把很小的非零方向算成0。固定ε不能對任意尺度保證正確：縮放座標λ倍會把orient乘λ²，而拓樸的左／右關係不變。將「差不多共線」當作精確等價，還可能破壞排序比較的傳遞性。

本章正確性採精確實數判定模型；可重算例用整數／有理數精確計算。需要浮點實作時，須使用有誤差界的篩選、失敗後提高精度的精確判定，不能用未證的ε取代符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線段相交的完整判定

令 o_1=orient(a,b,c)、o_2=orient(a,b,d)、o_3=orient(c,d,a)、o_4=orient(c,d,b)。on(a,b,p)必須同時檢查orient=0與兩軸的閉區間：

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

比較符號即可，不必直接乘兩個可能很大的orientation值。垂直線段不需斜率，單點線段也由on處理。若已知共線，on可只檢查bounding box；未知共線時不能省掉第一個條件。

為什麼兩邊都跨越才足夠

在沒有零方向的情形，o_1,o_2異號表示c,d在ab所在直線的不同側；o_3,o_4異號表示a,b在cd直線不同側。前者使cd與ab所在直線的唯一交點落在cd內部；後者使同一個交點也落在ab內部，故兩閉線段相交。

反過來，兩線段若在各自內部穿越，每對端點必分居對方直線兩側。若方向為0，交點不是一般穿越：可能是一端落在另線段上，或兩段共線。兩個共線閉區間有交集時，至少一個端點落在另一區間內；四個on測試完整涵蓋。若其中一段是單點，也正是測該點是否在另段上。

只測一對異號不夠：c,d可以跨過ab的延長線，卻在ab端點之外才相交。

同一判定涵蓋穿越、接觸與分離

取a=(0,0)、b=(4,0)：

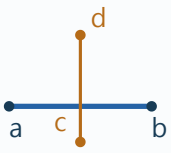
c,d	(o_1,o_2)	(o_3,o_4)	結果
(2,−1),(2,2)	(−4,8)	(6,−6)	內部穿越(2,0)
(4,0),(6,2)	(0,8)	(8,0)	共用端點b
(2,0),(6,0)	(0,0)	(0,0)	重疊[2,4]
(5,0),(6,0)	(0,0)	(0,0)	bounding box分離

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一判定涵蓋穿越、接觸與分離 （續）

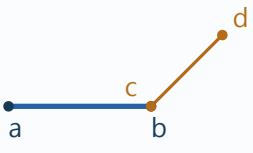
閉線段交集：端點與共線也必判定

內部穿越



$o_1=-4$，$o_2=8$

共用端點



$o_1=0$，$o_2=8$

共線重疊

共同區間 [2,4]

共線分離

bounding boxes 不交

藍段 a=(0,0)→b=(4,0)；四圖 x、y 使用相同單位長度，y 方向向上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.2 Determining whether any pair of segments intersects

輸入n條閉線段，目標只回傳是否存在一對相交，不要求列出所有交點。所有配對直接判定需$\Theta(n^2)$；若有$\Theta(n^2)$個交點，完整列舉本來就不能有O(n log n)的總輸出界。

先看非垂直線段。每段整理為左端l、右端r，l_x<r_x。在目前掃描位置x，active集合包含l_x≤x≤r_x的線段；status以交點高度y_s(x)由低至高排序。Event（事件）為左右端點。插入新段只測與上下鄰居；刪除前測將成為鄰居的上下兩段。

比較的key不是線段ID，而是當下的幾何高度。對s=(l,r)，

`y_s(x)=[l_y(r_x−l_x)+(r_y−l_y)(x−l_x)]/(r_x−l_x)`。

分母為正，兩段比較可交叉相乘，避免除法。每次精確算術O(1)的模型下，平衡樹每操作O(log n)。

只在端點更新 Status 的演算法

同x事件先處理所有左端，再右端；同類依y、ID確定次序。高度相同用ID作樹內tie-break，但仍對相鄰段做完整閉線段相交測試。

```
T=empty balanced ordered tree
for each endpoint in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

不存在的鄰居略過。交集測試針對整條線段，所以可能在實際到達交點以前就回傳true。此版本一旦找到就停止；不能把return改成print後照舊繼續，因越過交點後status順序會改變。

鄰居測試為何不漏：最早交點證明

若回傳true，必由精確兩段判定找到真交集，沒有誤報。反證存在交集卻全程回false。選最小x的交點p。p之前沒有任何交叉，active段的高度順序不會自行交換，所以status仍正確。

若兩條在p相交的段在p左側之間還有別段，該段的高度被兩者夾住，連續性使它也通過p；在通過p的有限段群中，至少有一對在p之前相鄰。若相交始於端點，則在該x插入時與相同高度群的鄰居相接；先插後刪保留了恰於同點結束的段。

這對段第一次成為鄰居，只可能由插入其中之一，或刪掉它們之間最後一段造成。兩種事件都做完整交集測試，應已回true，矛盾。共線重疊同樣在重疊開始的端點形成相同高度群，其相鄰段也相交。

這也解釋為何不用對每個x重新排序：尚未找到交集前，順序無改變；若即將改變，先前的鄰居檢查已使演算法停止。

刪除中間段時才出現的新鄰居

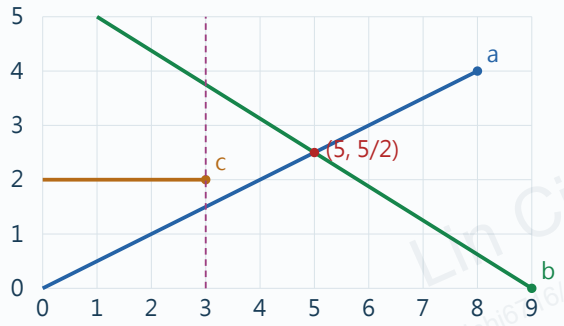
a:(0,0)→(8,4)，b:(1,5)→(9,0)，c:(0,2)→(3,2)。a,b交於(5,5/2)，c與另兩段都不相交。

x與事件	status由低至高	新測試
0插a	a	無
0插c	a,c	a,c不交
1插b	a,c,b	c,b不交
3刪c之前	a,c,b	a,b相交，回true

在x=3三段高度分別3/2、2、15/4，刪c後a,b才相鄰。只測新插入線段會漏掉這類情況。

刪除中間段時才出現的新鄰居 (續)

刪除 c 後，才需要檢查新鄰居 a、b



x=3 的高度：

a：3/2

c：2（將刪除）

b：15/4

刪除前先測 a,b

交點在未來也立即接受

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

垂直線段、端點與成本的完整契約

對一般閉線段，按相同x分批處理：先插入該x開始的非垂直段；再處理該x所有垂直段；最後刪除該x結束的非垂直段。如此端點接觸仍包含於active集合。

垂直段v的y範圍為[L,H]。在status找高度≥L的第一段，若其高度≤H就相交；否則沒有非垂直段在範圍內。這是有序樹lower_bound，O(log n)，不是把整段v硬放成只有一個高度的key。同x的垂直段依L排序；維護已見最大H，若下一個L≤最大H即有重疊，否則更新最大H。單點線段也放入此閉區間群。

排序O(n log n)，2n次非垂直插／刪及n次以內range query各O(log n)，垂直群排序的總成本不超過O(n log n)，故一般版本O(n log n)、空間O(n)。這個界採精確常數成本幾何判定；b位元整數座標的交叉乘積成本須另乘相應位元成本，不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.3 Finding the convex hull

凸集包含其任意兩點間整條線段。有限點集Q的凸包CH(Q)是所有凸組合Σλ_ip_i（λ_i≥0、Σλ_i=1）的集合，也是包含Q的最小凸集。

輸出約定：重複座標先去重，只輸出極端頂點，逆時針排列，末尾不重複首點；位於同一條邊中間的點不另列。空集輸出空；單點輸出該點；全部共線且至少兩點時只輸出兩個端點。

Graham scan按極角排序後維護候選stack；Jarvis march每次選下一條支持邊，成本依凸包頂點數h。兩者不需要以浮點atan2計角度：精確orientation與距離平方比較足以決定順序及共線取捨。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graham Scan：先處理角度相同的點

取p_0為y最小、同y時x最小的點，其餘點都在上半平面或正x方向。對u,v以orient(p_0,u,v)>0判u極角較小；零時比較離p_0距離平方。相同射線只保留最遠點，其餘在p_0和最遠點的線段內，刪掉不改凸包。

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

長度檢查使不足三點時也不讀空stack。全部共線先回端點；非共線時掃描結果閉合為凸多邊形。用≤0刪去平角中間點，與本章「只輸出極端頂點」的契約一致；若要求列出每個邊界點，必須另改同射線及最後共線鏈的處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pop 的幾何理由：舊頂點已被新三角形包含

固定一次新點p_i。Stack由p_0向上按極角遞增，考慮倒數兩點r,t。若orient(r,t,p_i)≤0，則t不是新凸包頂點。

若$r=p_0$，因t與p_i的極角嚴格遞增且跨度小於π，orient(p_0,t,p_i)>0，這一輪不會pop。因此真正需要pop時$r\neq p_0$。把p_0平移到原點，r,t,p_i的射線按角度依序排列且跨度小於π，所以t位於r與p_i張成的錐內，可寫$t=\alpha r+\beta p_i$，$\alpha,\beta\geq 0$。令D=orient(0,r,p_i)>0，則

`orient(r,t,p_i)=(α+β−1)D`。

非左轉推出$\alpha+\beta\leq 1$，所以$t=\alpha r+\beta p_i+(1-\alpha-\beta)p_0$，是三角形$p_0$,r,p_i的凸組合。刪t不改變加入p_i後的凸包；等號恰為在r,p_i邊上，仍非極端頂點。這是可以反覆pop的證據，不是只以「凸包不能右轉」當充分理由。

Stack 不變量與完整正確性

處理p_i前，stack恰為已處理點凸包的逆時針頂點。初始一／兩點符合退化凸包；首次非共線點形成三角形。

新p_i的極角比先前點大，$p_0\to$p_i是新集合的一條支持邊：所有已處理點都在此有向邊的右側或線上。從舊凸包末端向前看，對p_i不可見的舊邊不變；可見的末端連續鏈被新邊取代。每個pop按上一頁證明是新三角形內的非極端點，不改CH；直到末端(r,t,p_i)為左轉，t→p_i成為另一條切線，留下的舊凸邊與p_i→p_0圍成凸多邊形。

此多邊形包含舊凸包及新點，且頂點全來自輸入，故同時包含CH且包含於CH，兩者相等。歸納到最後即得到CH(Q)。每點push一次、pop至多一次，掃描O(n)；初始排序O(n log n)主導，空間O(n)。

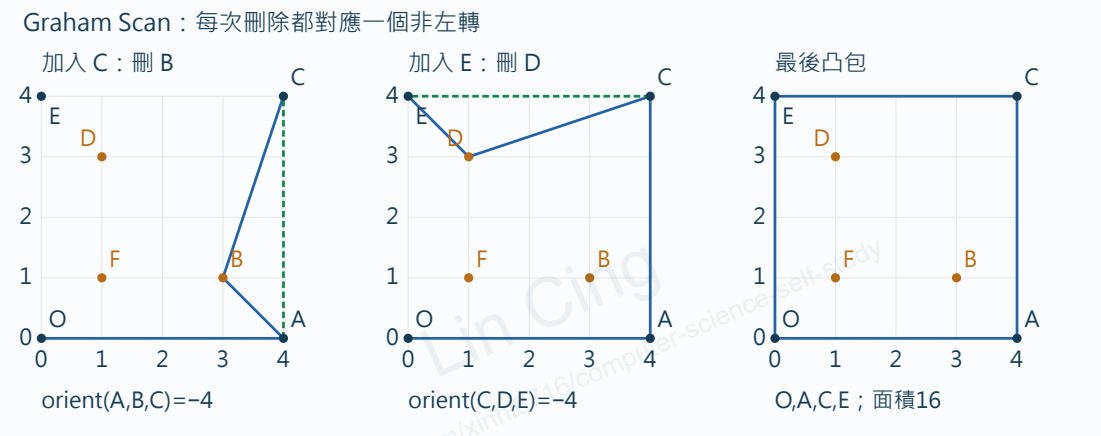
凸包的兩次 Pop 都能用數值核對

p_0=O(0,0)、A(4,0)、B(3,1)、C(4,4)、D(1,3)、E(0,4)，極角次序O,A,B,C,D,E。另有F(1,1)，與C同射線，先刪F。

新點	判定	Pop後／加入後stack
O,A,B	orient(O,A,B)=4	O,A,B
C	orient(A,B,C)=−4，刪B；orient(O,A,C)=16	O,A,C
D	orient(A,C,D)=12	O,A,C,D
E	orient(C,D,E)=−4，刪D；orient(A,C,E)=16	O,A,C,E

B=O/4+A/2+C/4，D=O/4+C/4+E/2，正是被其他點的凸組合包含。最後正方形頂點O,A,C,E，面積16。

凸包的兩次 Pop 都能用數值核對 (續)



Jarvis March：每步找到一條支持邊

從字典序最小點p開始。選任一q≠p，掃描所有r≠p：若orient(p,q,r)<0就令q=r；共線且r更遠也令q=r。掃完後所有點都在p→q左側或線上，q就是逆時針下一個極端頂點。輸出p，再令p=q，回到起點時停止。

為何一次掃描足夠？p是凸包頂點，所有其餘點方向落在一個跨度至多π的支持半平面；在這個半平面內，orientation決定方向的線性次序。每次更新保留已掃點中最靠順時針的射線；同射線保留最遠端。掃完便是一條支持邊，其他點位於同一側。從極端點走到下一極端點，直到繞回，不會走入內點。

每步檢查n−1點，共h條邊，時間O(nh)，空間O(h)含輸出；h≤n，所以最壞O(n^2)。同一正方形例每步選A,C,E,O，h=4。它在h很小時有利，但不能無條件稱比Graham快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.4 Finding the closest pair of points

給n≥2個帶唯一ID的點，求歐氏距離最小的一對。重複座標允許，答案可為0；若n<2回報「無點對」。比較距離平方即可避免為每個候選求平方根。

先依(x,y,ID)排X、依(y,x,ID)排Y。每次按X的前⌈n/2⌉個ID分成L，其餘R；分割線ℓ選在兩組x範圍之間（同x可取該x）。沿Y一次穩定分流成Y_L,Y_R，必用ID membership，不能只用x<ℓ，否則所有點同x時會分不平衡。

遞迴求兩側最佳距離δ_L、δ_R，令$\delta=\min$。若$\delta=0$可直接回傳。任何更近的跨組點對，兩點距分割線都<δ，且縱向距離也<δ，所以只需檢查寬2δ的strip內候選。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最近點對的分治偽碼

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; select dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

Strip範圍使用兩側答案的固定δ；比較時best可變小，但每點最多看後7點。只求一對時，與已知最小值相等的候選不必再更新，因此可以使用嚴格strip與≥break。若要求輸出所有最小點對，需要另設相等距離與輸出規則。

為何只需後七點：分組的裝箱證明

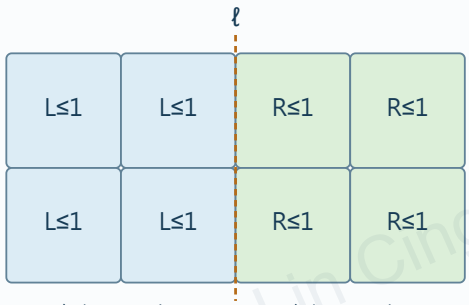
固定strip內一點p，只可能改善答案的後續點q滿足0≤q_y−p_y<δ，故落在寬2δ、高δ的矩形。左組點位於左半δ×δ，右組點位於右半δ×δ；同一組任兩點距離至少δ，這是遞迴最小值的保證。

將每個半正方形分為四個δ/2×δ/2小格，以固定半開邊界分配每點。單格直徑δ/√2<δ，所以同組單格至多一點，每組至多4點，合計至多8點。扣除p本身，其餘至多7點，q在依y排序的後7個位置內。

落在分割線上的點按ID屬於其中一組，各用自己的半正方形計帳；不能把兩組都在分割線的點誤當成同組而排除。若跨組有重複座標，距離0也會由此候選比較找到；若同組已重複，遞迴早已回0。

為何只需後七點：分組的裝箱證明 （續）

寬 2δ、高 δ：依左右組別，各自最多四點



同組任兩點距離 ≥δ

小格邊長 δ/2

小格直徑 δ/√2<δ

含 p 共至多8點

所以只看後7點

分割線上的點仍按 ID 歸組；同一幾何位置可各有一個 L、R 點。

最近點對的完整合併數例

X依序為A(0,0),B(1,4),C(2,1),D(3,7),E(4,1),F(5,5),G(6,0),H(7,8)。左組A–D，右組E–H，ℓ=7/2。

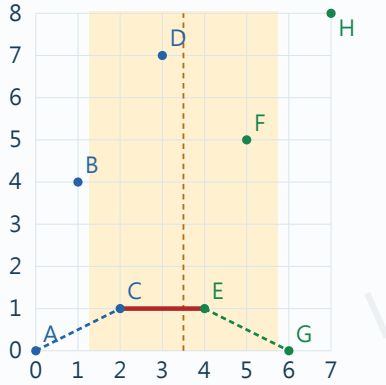
左組最佳A,C，距離平方5；右組最佳E,G，也為5。δ=√5，strip包含C,E,F,D，按y順序正是C(2,1)、E(4,1)、F(5,5)、D(3,7)。

目前點	後續檢查	結果
C	E：平方距離4；下一F的y差4	best改C,E；4≥2，停止此列
E	F的y差4	不可能改善2，停止
F	D的y差2	相等下界也不會改善，停止
D	無後點	結束

跨組C,E的距離2改善兩個子問題的√5。圖中strip按固定√5畫，不隨best縮窄。

最近點對的完整合併數例 （續）

兩側答案 √5，跨組 C、E 的距離 2 更小



$\ell = 7/2$，固定 $\delta = \sqrt{5}$

左組 A–D，右組 E–H

Strip 依 y 排序：C,E,F,D

$|C-E|^2 = (4-2)^2 + 0^2 = 4$

兩側最短距離平方都是5

檢查跨組後 $best^2 = 4$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複雜度來自穩定分流，不是每層重排

每次分割X用rank，Y以ID穩定分流，建立strip與每點至多7次比較，非遞迴工作O(n)。故

$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + O(n) = O(n \log n)$。

最初兩次排序也O(n log n)。若每個子問題重排Y，合併工作變O(n log n)，總界便$O(n \log^2 n)$。ID membership可預先記每點在本層的左右標記；穩定分流不能對每個Y點線性搜尋整個L。

按深度先遞迴、子問題返回後釋放暫存，單條遞迴鏈的陣列總長$n+n/2+\cdots=O(n)$，加當前兄弟與回傳資訊仍O(n)工作空間，stack O(log n)。若刻意保存整棵遞迴樹的所有陣列，會用O(n log n)，那不是這個線性空間實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 33。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法

Part VII Selected Topics

本章定位

動態多執行緒以依賴DAG分開工作總量、關鍵路徑與排程；本章由spawn/sync與競爭修復，推導矩陣乘法和平行merge/sort的完整算法、一般證明及空間條件，避免把平均平行度當收益截止點。

前置知識

DAG 排程、並行成本模型與遞迴分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 spawn、sync 描述動態多執行緒計算。
- 區分 work、span、parallelism。
- 分析矩陣乘法與合併排序的並行界限。
- 27.1 The basics of dynamic multithreading
- 27.2 Multithreaded matrix multiplication
- 27.3 Multithreaded merge sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27.1 The basics of dynamic multithreading

動態多執行緒模型讓程式描述哪些計算可以並行，由排程器決定何時、在哪個處理器執行。邏輯task不是固定綁定的OS thread；spawn不承諾立刻取得另一個核心。

本章採相同速度的P個理想處理器與循序一致的共享記憶體：每條指令可看成某個符合各依賴的交錯順序，load/store各自是單一步驟；讀、加、寫的合成操作不自動具有原子性。單位成本算術、索引、spawn/sync為常數工作，排程器本身的時間、頻寬、cache及通訊成本不計入此理想界。

非空有限計算用DAG表示。節點可是一個strand（不含平行控制的一段順序指令），帶自己的正執行時間；空strand可刪除，要用單位節點證明時，可再拆成單位指令鏈。控制與資料依賴決定有向邊，而不是把畫面上同層的節點一律當成可並行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Spawn、call、sync 的確切語義

`x = spawn F(...)`允許F與呼叫者後續程式並行；普通`y = G(...)`則呼叫者須等待G返回。`sync`等待本程序此前spawn的child全部完成，才執行後面的指令；它不是全系統的barrier。

spawn參數先在parent求值。child的區域變數屬該次呼叫，指向共享陣列的參數仍可能讓多task存取同一格；不能只看變數名稱不同就判獨立。讀spawn回傳值之前必須有相應依賴，通常明寫sync。

本章程序返回前隱含sync，但若先求`return x+y`的值才返回，隱含等待不足以允許提早讀x；因此先sync再算回傳式。刪掉spawn/sync並將parallel for改普通for得到serialization；它能解出正確答案，不代表所有平行交錯也正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci 範例的程式與正確性

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

兩個child只讀自己的n並回傳各自結果，沒有共享寫入。n下降使呼叫終止；基例F0=0、F1=1，加上歸納假設與sync後的x+y，得到正確Fn。若把兩個child都spawn，仍必等兩者後才能相加，漸近work/span不變，只改常數。

這個版本故意沒有memoization，同一參數會算多次。高平行度不等於work有效率：順序迭代法只需O(n)次算術，不能因遞迴版本可以占用更多核心，就忽略其指數工作量。

同一 strand 模型的 work 與 span

為了逐點驗算，將每個非基例呼叫分為三個單位strand：pre（到spawn）、cont（到普通第二呼叫）、join（sync後相加返回）；每個基例為一個單位strand。pre先於左child及cont，cont先於右child，兩個child結束都先於join。

令W為全部strand成本，D為最長依賴路徑成本，則

$$W_n = W_{n-1} + W_{n-2} + 3, \quad W_0 = W_1 = 1;$$

$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \quad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

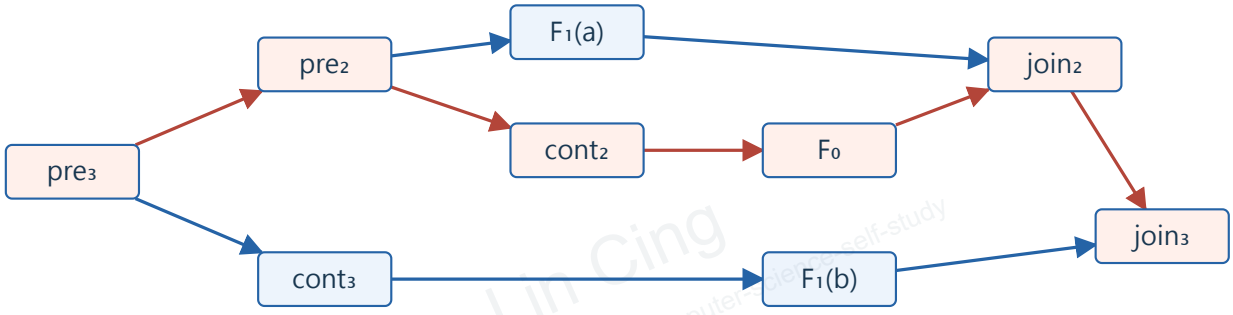
同一 strand 模型的 work 與 span （續）

Wn=4F(n+1)−3可直接代入歸納，Dn=2n對n≥2成立。令 $\varphi = (1+\sqrt{5})/2$；由ϕ²=ϕ+1，歸納得ϕ^(n−2)≤Fn≤ϕ^(n−1)（n≥2）：n=2、3可核對，兩界的相鄰和仍是下一個ϕ次方。因此W=Θ(ϕ^n)、D=Θ(n)。這是明列的單位strand模型，不把一個完整遞迴呼叫誤算成一個O(1)節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

九個 strand 的依賴可逐條追蹤

P-FIB(3) 的完整九個 strand：箭頭是先後依賴



每格恰為一個單位 strand；W=9。紅色最長路徑含6格，因此 D=6。

圖的pre3先分到pre2及cont3；cont3才呼叫右側F1。pre2也先分到其F1與cont2，cont2再呼叫F0。兩個基例都完成才可做join2，join2與右側F1都完成才可做join3。

紅路徑pre3→pre2→cont2→F0→join2→join3共6個節點，W=9、D=6。另一條pre3→cont3→F1→join3只有4個節點。圖上箭頭是依賴方向，不是把同一呼叫的所有節點排成一條長鏈。

Work law、span law 與可達的加速比

記 $W = T_1$、$D = T_\infty$、實際P處理器時間 T_P。每個時間單位最多完成P單位工作，且關鍵路徑不可並行，故

$$T_P \geq \max\{W/P, D\};$$

單位節點離散模型還有 $T_P \geq \lceil W/P \rceil$。加速比 $S_P = W/T_P \leq \min(P, W/D)$，效率 $S_P/P \leq 1$。平均平行度A=W/D限制加速比；P>A便無法有完美的P倍加速，不代表第P+1顆處理器必然沒有收益。

順序合成兩段時，W相加、D也相加；若兩段獨立地fork後join，W仍相加，D取兩段最大值，另加明列的fork/join成本。空計算W=D=0直接時間0，不把0/0稱為平行度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy 排程的完整上界證明

每個時間步，只要有至少P個ready節點便執行P個；不足P時全部執行，這是greedy。令完整步C、不完整步I，總時間T=C+I。計算尚未結束時，每個不完整步至少執行一個節點，故W≥PC+I。

不完整步會移除剩餘DAG的所有source。步後的最長路徑，可在該步開始前的DAG中向前延伸到至少一個被移除的source；若步後已空，步前span也至少1。因此每個不完整步使剩餘span至少減1，最多有D個不完整步，I≤D。代入得

$$T \leq \frac{W - I}{P} + I \leq \frac{W - D}{P} + D \leq W/P + D.$$

最後一步使用P≥1使I的係數1−1/P非負。單位節點證明可對整數長度strand拆鏈後套用；它不是無成本處理任意不可切割、長度未知task的另一個模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩倍近似與 slackness 的量化

最優排程也受 $T_P^* \geq \max(W/P, D)$ 限制，故greedy的T≤W/P+D≤2T*。更精確地可得 $(2 - 1/P)T_P^*$ 上界，但無需預知未來DAG就有常數比保證。

定義slackness $\sigma = W/(PD) = A/P$。對D>0，上下界給

$$1 \leq \frac{T_P}{W/P} \leq 1 + \frac{1 - 1/P}{\sigma} \leq 1 + 1/\sigma.$$

所以σ很大時，理想greedy時間接近W/P；這是數學界，不是對真機頻寬或排程器開銷的實測。最優排程可以停用多餘處理器，所以其時間隨P不增；任意實作的測得時間另受開銷與排程選擇影響。

平行度30，第31顆仍有收益

697個互相獨立的單位工作全部完成後，才能開始23個單位工作的依賴鏈。W=697+23=720，D=1+23=24，A=30。

P	獨立階段的完整配置	階段時間	鏈時間	精確最短時間
30	23輪各30項，再7項	24	23	47
31	22輪各31項，再15項	23	23	46

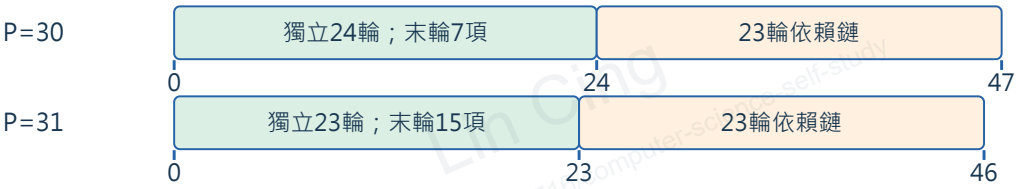
所有前驅都完成前鏈不能開始，所以ceil(697/P)+23同時是可行排程與下界。S30=720/47≈15.32、S31=720/46≈15.65，均≤30，但增加處理器確實仍有收益。P≥697才可把第一階段縮成1輪、達到D=24。

平行度30，第31顆仍有收益 （續）

相同 DAG：31個處理器仍比30個快

W=697+23=720
D=1+23=24

平均平行度 W/D=30；以下兩列使用相同時間比例尺。



$T_{30}=47$、$T_{31}=46$：加速比至多30，不代表第31個處理器沒有收益。

減 work 但增加 span，可能改變優劣

兩個正確版本分別有(W,D)=(1024,4)、(768,16)，都用理想greedy排程。

P=8時，第一版時間至少1024/8=128，第二版時間至多768/8+16=112，因此第二版一定較快。P=128時，第一版至多1024/128+4=12，第二版至少16，因此第一版反而一定較快。

這個判定用到不重疊的上下界，不把W/P+D的上界誤當精確時間。若兩版本的可行時間區間重疊，單靠W,D未必能判定哪個實際較快。少量核心上的單次時間，也不足以識別work與span各自的改變。

Parallel for 如何以 fork/join 實作

以下與合併算法均採0-based半開區間[l,r)，空區間l=r。

```
PAR-FOR(l,r,body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l,m,body)
    PAR-FOR(m,r,body)
    sync
```

n個非空迭代恰對應n葉、n−1內節點；每個內節點常數spawn/sync工作，所以額外work $O(n)$，不會憑空多出$n \lg n$。最長呼叫與join路徑深度$\Theta(\lg n)$，因此總span是$\Theta(\lg n)$+各迭代span最大值，前提是各迭代真的獨立。

若用普通for依序spawn n個task，parent本身就有$\Theta(n)$長的發出鏈，不能宣稱同樣對數span。平行迴圈結尾的sync也確保下一段讀取所有結果時已完成。

Matrix-vector：只平行不同 row 是安全的

```
MAT-VEC(A,x):
    n = A.rows
    allocate y[n]
    parallel for i = 0 to n-1:
        y[i] = 0
        for j = 0 to n-1:
            y[i] += A[i,j]*x[j]
    return y
```

每個i獨占y[i]，A,x只讀；內圈第j步後是前j+1個乘積的和，所以正確。work $\Theta(n^2)$，span $\Theta(\lg n)+\Theta(n)=\Theta(n)$，平均平行度$\Theta(n)$。n=0回空向量。

不能只把內圈也改parallel for：各j對同一y[i]做讀改寫，會產生determinacy race（未由依賴排序的衝突存取）。兩個邏輯並行存取同格、至少一個寫入便有此衝突；即使碰巧寫相同值，也不能用「跑過一次正確」證明獨立。

循序一致仍會遺失更新

遺失更新：逐指令讀值、暫存值與最後寫回

初始 x=0；每個 task 內部仍依 load → add → store。

時間步	Task 1	Task 2	共享 x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1+1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2+1 = 1$	1
6		store $x \leftarrow r_2$	1

兩者結尾 sync 後讀到1：等待完成不能把三條指令變成不可分割操作。

圖的交錯尊重各task的load→add→store，最後x卻為1；如果第一個task全部完成才換第二個，會得到2。循序一致只限制單條指令的交錯，不把三條指令合成原子操作。

本章以互不衝突的分工或reduction消除此類競爭。原子加或鎖也可建立依賴，但其同格序列化、工作量與span必須另算；不能保留「全部加法同時完成」的分析。

用 reduction 正確地平行內積

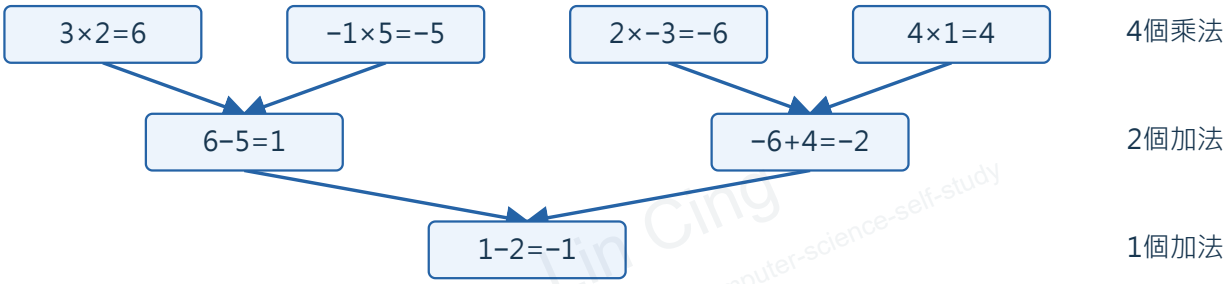
```
DOT(Arow,x,l,r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m = floor((l+r)/2)
    left = spawn DOT(Arow,x,l,m)
    right = DOT(Arow,x,m,r)
    sync
    return left+right
```

每個child只回傳自己的值，沒有共同累加格；結合律與歸納保證結果是區間內積。work為$\Theta(n)$，span $\Theta(\lg(n+1))$。把MAT-VEC每個row改成這個DOT，得到$\Theta(n^2)$work、$\Theta(\lg n)$span（$n \geq 2$）。

例row=[3,−1,2,4]、x=[2,5,−3,1]，乘積[6,−5,−6,4]，第一層和[1,−2]，最後−1。

用 reduction 正確地平行內積 （續）

內積不是共寫一個總和：四個私有乘積沿樹合併



算術 DAG：work=7、span=3；fork/join 另有 Θ(log n) 控制深度。

對浮點加法，固定的樹可能和順序累加有不同捨入結果；正確性證明先在精確、可結合算術下成立，不能順手承諾逐bit相同。

粒度與記憶體配置的模型

將n個常數成本迭代分成每批最多g個、以平衡PAR-FOR發出各批，1≤g≤n，work Θ(n)，span Θ(lg⌈n/g⌉+g)。較大g減少task常數開銷，也增加每葉序列成本；是否值得依P及slackness而定。

本章allocate取得未初始化、互不重疊儲存空間的描述符，抽象為常數span；若實作需要逐格初始化，必須計入相應工作並以parallel loop初始化才保留對數span。不是把一個串行清零n^2格的操作藏進O(1)。

矩陣區塊用(row起點、column起點、stride、階數)的view表示，不隱藏整塊複製。任務控制區、分配器、cache與記憶體頻寬在實作時另有成本；以下數學界皆在已明列的理想模型內。

27.2 Multithreaded matrix multiplication

```
P-MULTIPLY(A,B):
    n = A.rows                 // both n by n
    allocate C[n,n]
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1:
            C[i,j] = 0
            for k = 0 to n-1:
                C[i,j] += A[i,k]*B[k,j]
    return C
```

每個(i,j)獨占一格輸出，內圈仍依序累加。work $\Theta(n^3)$，span兩層發出樹$\Theta(\lg n)+\Theta(\lg n)$，再加一條內圈$\Theta(n)$，總$\Theta(n)$，平均平行度$\Theta(n^2)$。兩個parallel關鍵字沒有消除每格的n項依賴。

若每個Cij改由前述DOT類型的二分內積回傳，work仍$\Theta(n^3)$，span則$\Theta(\lg n)$，平均平行度$\Theta(n^3/\lg n)$。直接並行寫入Cij不是這個reduction算法。

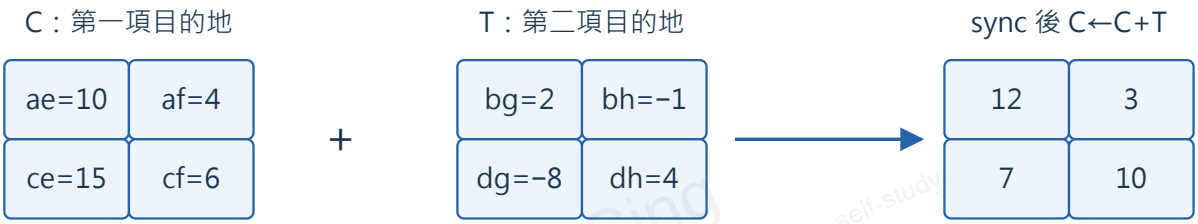
八次子乘法必須寫八個不同目的地

令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，此處字母是同階區塊。先計C=[[ae,af],[ce,cf]]與暫存T=[[bg,bh],[dg,dh]]，八個乘積各有獨占目的地。sync後才做C=C+T。

數例A=[[2,−1],[3,4]]、B=[[5,2],[−2,1]]：C暫時[[10,4],[15,6]]，T=[[2,−1],[−8,4]]，相加得[[12,3],[7,10]]。

八次子乘法必須寫八個不同目的地 （續）

八個獨立乘法：目的地分開，回收時才相加



八個乘法各寫 C 或 T 的一格；先等待全部乘法，再逐格相加。

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]；同格的兩項沒有同時做 +=。

若把ae與bg兩個task都寫成C11+=product，兩者便衝突。分出T是在用額外空間解除寫入依賴，不是只加spawn就能平行。

遞迴矩陣乘法的完整程序

```
P-REC(C,A,B):
    n = A.rows
    if n == 1: C[0,0] = A[0,0]*B[0,0]; return
    allocate T[n,n]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    view C and T into four n/2 by n/2 blocks
    spawn P-REC(C11,a,e); spawn P-REC(C12,a,f)
    spawn P-REC(C21,c,e); spawn P-REC(C22,c,f)
    spawn P-REC(T11,b,g); spawn P-REC(T12,b,h)
    spawn P-REC(T21,d,g); P-REC(T22,d,h)
    sync
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1: C[i,j] += T[i,j]
```

前提為同階方陣、階數是2的冪，A與B只讀，C不與它們重疊。任意正階數先補零到下一個2冪再取回結果，n=0直接回空矩陣。T及每個child工作區也獨立；八個spawn/call數量是固定常數，不是n長的發出鏈。

八路矩陣乘法的歸納與遞迴成本

基例寫出正確單乘積。歸納假設各n/2子乘積正確，八個child因目的地不同而無競爭；sync確保全部乘積可讀，再逐格相加，依區塊乘法恆等式得到AB。只有共同讀A、B不構成衝突。

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

span沿八個child中一條最長路徑，不是八者相加；每層回收的nested parallel for又有Θ(lg n)發出／join深度，不能當O(1)。n=2^L時Σr=1..L r=Θ(L^2)，同時給上、下界。n=4有8^2=64個純量乘法葉；平行化沒有少做乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 平行化的三個同步階段

沿用第4章M編號，先以平行逐格加減準備(a+d)、(e+h)、(c+d)、(f−h)、(g−e)、(a+b)、(c−a)、(e+f)、(b−d)、(g+h)。輸入準備完成後，七乘積同時可執行：

$$M_1 = (a+d)(e+h), \quad M_2 = (c+d)e,$$
$$M_3 = a(f-h), \quad M_4 = d(g-e),$$
$$M_5 = (a+b)h, \quad M_6 = (c-a)(e+f),$$
$$M_7 = (b-d)(g+h).$$

七者完成後sync，再平行回收C11=M1+M4−M5+M7，C12=M3+M5，C21=M2+M4，C22=M1−M2+M3+M6。不能在輸入和差未完成前spawn消費它們，也不能在乘積未完成前回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的 work、span 與平行工作區

每層七個同階child，常數個矩陣加減：

$$W_S(n) = 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}),$$
$$D_S(n) = D_S(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

正確性由第4章已展開的四個區塊恆等式及遞迴歸納，平行化另需上述三階段依賴。其平均平行度 Θ(n^(lg7)/lg²n)小於八乘法版本，卻具有較低work；平行度的比值不是單獨選算法的目標。

順序深度優先的O(n²)暫存不能直接移給所有child同時展開的版本。若每個呼叫分配自己的Θ(n²)工作區並保持到join，八路的完全展開空間滿足Q(n)=8Q(n/2)+Θ(n²)=Θ(n³)；七路則為Θ(n^(lg7))。有限P與工作區重用可降低峰值，但必同時核對排程與span，不能沿用一個未實作的空間界。

平行 Strassen 的可執行規格

定義PADD(X,Y,s)配置獨立R，以兩層parallel for寫R[i,j]=X[i,j]+s・Y[i,j]，等待迴圈完成再回R。取s=±1；其work Θ(n²)、span Θ(lg n)。以下A、B同階且先補到2冪，n=0由外層直接回空矩陣。

```
P-STRASSEN(A,B):
    n = A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    S1 = PADD(a,d,+1); S2 = PADD(e,h,+1)
    S3 = PADD(c,d,+1); S4 = PADD(f,h,-1)
    S5 = PADD(g,e,-1); S6 = PADD(a,b,+1)
    S7 = PADD(c,a,-1); S8 = PADD(e,f,+1)
    S9 = PADD(b,d,-1); S10 = PADD(g,h,+1)
    M1 = spawn P-STRASSEN(S1,S2)
    M2 = spawn P-STRASSEN(S3,e)
    M3 = spawn P-STRASSEN(a,S4)
    M4 = spawn P-STRASSEN(d,S5)
```

平行 Strassen 的可執行規格 （續）

```
    M5 = spawn P-STRASSEN(S6,h)
    M6 = spawn P-STRASSEN(S7,S8)
    M7 = P-STRASSEN(S9,S10)
    sync
    allocate C[n,n], viewed as four blocks
    parallel for i = 0 to n/2-1:
        parallel for j = 0 to n/2-1:
            C11[i,j] = M1[i,j]+M4[i,j]-M5[i,j]+M7[i,j]
            C12[i,j] = M3[i,j]+M5[i,j]
            C21[i,j] = M2[i,j]+M4[i,j]
            C22[i,j] = M1[i,j]-M2[i,j]+M3[i,j]+M6[i,j]
    return C
```

十個PADD在此依序呼叫，數量固定，故總span仍Θ(lg n)；先全部完成才spawn七乘法，與三階段依賴一致。沒有把任意數量的串行發出忽略掉。

27.3 Multithreaded merge sort

只把兩個遞迴sort並行、仍用順序MERGE，並不充分：

```
PAR-SORT-SERIAL-MERGE(A,l,r):
    if r-l <= 1: return
    m = floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A,l,m)
    PAR-SORT-SERIAL-MERGE(A,m,r)
    sync
    serial MERGE the two sorted halves using an auxiliary buffer
```

work為2W(n/2)+Θ(n)=Θ(n lg n)，span卻是D(n/2)+Θ(n)=Θ(n)，平均平行度僅Θ(lg n)。合併前的sync必要，否則會讀到未排序完成的run；要進一步縮短span，必須平行化merge本身。

二分切點先定義半開區間與平手規則

兩個輸入run位於只讀陣列T的[l1,r1)、[l2,r2)，輸出O[o,o+N)不得覆寫T，N為兩段長度和。為保證穩定性，每個record以(key,原始位置)作總序比較；原始位置唯一，相同key仍按原先順序。

```
LOWER-BOUND(T,l,r,x):
    while l < r:
        m = floor((l+r)/2)
        if T[m] < x: l = m+1
        else: r = m
    return l
```

不變量是原區間中小於x者的切點位於當前[l,r]；在原輸入run內，當前l左方都<x、r右方（含r）都≥x。每次依中點比較保留可能切點，長度嚴格縮小；空run直接回l，永遠不讀不存在的T[l]。工作及span O(1+lg(n2+1))，明列空run的常數處理。

可處理空 run 的完整 Parallel Merge

```
P-MERGE(T,l1,r1,l2,r2,O,o):
    n1 = r1-l1; n2 = r2-l2
    if n1 < n2: swap (l1,r1) with (l2,r2); swap n1,n2
    if n1 == 0: return
    m = floor((l1+r1)/2)
    j = LOWER-BOUND(T,l2,r2,T[m])
    q = o+(m-l1)+(j-l2)
    O[q] = T[m]
    spawn P-MERGE(T,l1,m,l2,j,O,o)
    P-MERGE(T,m+1,r1,j,r2,O,q+1)
    sync
```

先把較長run放第一描述符；交換的是view，不移動或重新標記record。若第一段空，第二段也空；若僅第二段空，仍從第一段取median逐步定位，不讀空段。每次拿走一個pivot，兩個child總長N−1，因此有限輸入必終止。

輸出位置、正確性與穩定性的一般證明

左側應有m−l1個第一run元素、j−l2個第二run元素，總長q−o。它們都在總序上小於pivot；其餘元素都大於pivot。故pivot的全域輸出位置是q，兩個child輸出區間恰為[o,q)與[q+1,o+N)，彼此及pivot均不重疊。

對N歸納：N=0空合併正確；兩child均較小，由歸納假設正確合併相應元素。左答案、pivot、右答案串接已排序且每個輸入record恰出現一次，得到完整merge。T只讀、O的寫入區間互斥，所以沒有determinacy race，sync後全部結果可用。

相同key按全域原始位置比較，輸出保持原先相對次序，因此穩定。若只比key且一律使用同一lower_bound規則，長度交換可能把相等的右run元素放到左run之前；普通排序正確不自動等於穩定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八個 record 的根切點與二分步驟

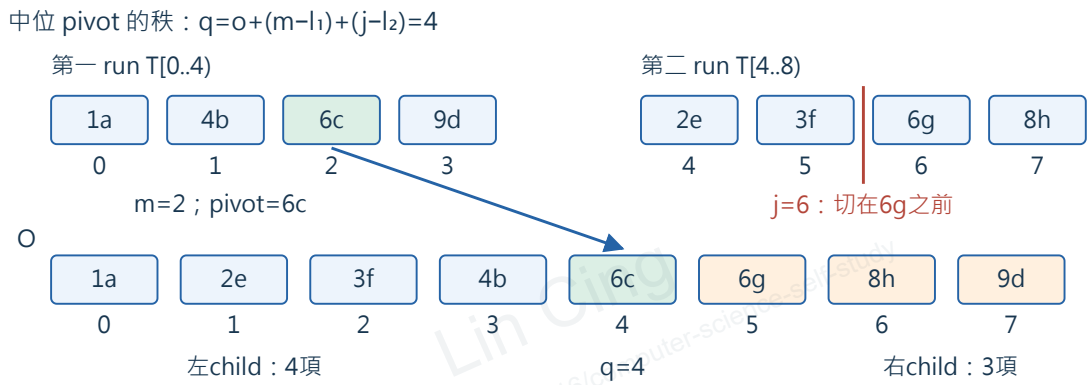
T=[1a,4b,6c,9d | 2e,3f,6g,8h]，字母依a..h表示原始位置0..7；比較先key再原始位置。
l1=0,r1=4,l2=4,r2=8,o=0。第一run長4，m=2、pivot=6c。

當前搜尋[l,r)	mid	T[mid]與6c比較	更新
[4,8)	6	6g>6c（key相等、位置較後）	r=6
[4,6)	5	3f<6c	l=6
[6,6)	—	空搜尋區間	回j=6

q=0+(2−0)+(6−4)=4；左child共4元素，右child共3元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八個 record 的根切點與二分步驟 (續)



比較 (key, 原始位置)：6c < 6g；左區、pivot、右區的輸出互不重疊。

同一數例的全部輸出寫入

以下為一個合法深度優先serialization的pivot寫入順序；平行執行可以改交錯，但目的格相同、各寫一次。

寫入序	pivot	q（0-based）	來源／輸出意義
1	6c	4	根的左右分界
2	4b	3	根左段的最後元素
3	3f	2	較長run交換後的median
4	2e	1	小run的切點
5	1a	0	另一run已空
6	8h	6	根右段交換後的median
7	6g	5	右段左child
8	9d	7	右段右child

同一數例的全部輸出寫入 （續）

最終O=[1a,2e,3f,4b,6c,6g,8h,9d]，兩個6仍c在g之前。即使改成先跑右child，只讀T及唯一輸出格也保證相同答案；不是靠整個陣列寫入順序維持排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge 最壞 span 的上界與下界

較長run長n1≥n2，取median後任一child至多⌊n1/2⌋+n2≤(N+n2)/2≤3N/4。即使一側切點落在端點，這個上界仍成立。二分搜尋至多O(lg(N+1))，所以

$$D_M(N) \le D_M(3N/4) + O(\lg(N+1)) = O(\lg^2(N+1)).$$

沿路每次固定比例縮小，有O(lg N)層，第r層剩餘log大小O(lg N−cr)，等差和給平方對數。

下界用近等長交錯run，例如[0,2,⋯,2m−2]與[1,3,⋯,2m−1]，m是2冪。左child持續成為同型的兩個等長交錯run，長度每層減半，且搜尋另一run的median切點仍需Θ(lg m)比較。一路累加lg m+(lg m−1)+⋯給Ω(lg²m)，所以最壞span為Θ(lg²N)。若另一run始終空，搜尋只需常數、span則Θ(lg N)；不能把最壞界稱為每個輸入的精確成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性 work：一次寫出不等於已付清搜尋

每個非空呼叫挑走一個不同pivot，所以有N個非空呼叫；各生兩child，含空返回共2N+1個呼叫。N次輸出給Ω(N)，但O(N)上界還必須支付每個非空節點的二分搜尋，不能只說「每元素寫一次」。

大小m的節點非遞迴工作≤c lg(m+1)。把此成本平均分攤給它仍含有的m個輸入record，每個收 c・lg(m+1)/m。固定record x只在一條ancestor鏈上被收費，直到自己被選成pivot為止。

從最後一次含x的呼叫向根反數r層，因child≤3parent/4，該層大小至少(4/3)^r。下一頁將此一般不等式求和，連空child也可處理，不需要取lg0或聲稱小規模的兩個child都至少N/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個 record 的搜尋費總和有常數界

$g(u)=\lg(u+1)/u$ 在u>0遞減，因導數分子的符號等於u/(u+1)−ln(u+1)<0；後者由積分 ∫ 0^u 1/(1+t)dt>u/(1+u)得到。因此反數r層的分攤費至多

$$c\frac{\lg((4/3)^r+1)}{(4/3)^r}\le c\,[1+r\lg(4/3)](3/4)^r.$$

幾何和Σq^r=1/(1−q)，以及由有限和求導／取極限所得Σr q^r=q/(1−q)²，對q=3/4都收斂。因此每個record總費O(1)，N個record共O(N)；再加N+1個空呼叫的常數費，總work O(N)。與輸出下界合併，W_M(N)=Θ(N)。

這是對任意合法切點的一般計費證明；空run與不平衡的小child已包含，不以有限例子取代上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個 buffer 的完整 Parallel Merge Sort

先將輸入record附上原始位置，令X與Y兩個獨立長n陣列都含相同輸入（一次parallel copy），呼叫SORT(X,Y,0,n)，最後取Y。

```
SORT(src,dst,l,r):
    if l == r: return
    if r-l == 1: dst[l] = src[l]; return
    m = floor((l+r)/2)
    spawn SORT(dst,src,l,m)
    SORT(dst,src,m,r)
    sync
    P-MERGE(src,l,m,m,r,dst,l)
```

每個子呼叫交換buffer角色；parent的兩個已排序run會留在src，sync後merge到dst。P-MERGE輸入與輸出必須是不同實體buffer。這個雙buffer寫法避免每個sort呼叫再配置長度n的暫存，不把原地寫入與讀取尚未消費的輸入混用。

Buffer 角色交換的正確性與競爭檢查

每個SORT進入時，兩個buffer的當前區間都仍含該區間的原始輸入。根由copy建立；parent在啟動兩child前未改自身區間，兩個child又只碰各自互斥的半區，故每個child進入時也成立。

基例把唯一record複製到dst。由歸納假設，兩child結束後，parent的src左右兩run分別排序且元素不變；sync後的P-MERGE把兩者穩定合併到dst。歸納完成排序規格；parent無需承諾src最後仍與dst相同，只需在呼叫入口滿足條件。

兩child即使交換buffer角色，也仍在相同的一對全域陣列中寫互斥索引區間；parent只在sync後合併。總資料workspace $O(n)$，峰值task描述符$O(n)$、每條呼叫鏈$O(\lg n)$；初始化copy工作$\Theta(n)$、span $\Theta(\lg n)$，不改後述排序階數。

排序的緊界不能只把最壞成本相加

每層merge work Θ(n)，有Θ(lg n)層，故W_S(n)=Θ(n lg n)。span上界是

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\lg^2 n) = O(\lg^3 n).$$

要證最壞下界，需同一輸入在沿途各層都產生昂貴merge。對n=2^L，按L位元反轉索引的順序放入鍵；n=8例[0,4,2,6,1,5,3,7]。根的兩半排序後是偶數與奇數交錯run；每個子問題的相對秩亦是相同構造。故每層都有前述Ω(lg²(size))的merge span，且parent merge在兩child都完成後才開始。

沿一條child鏈相加Σr=1..L r²=Θ(L³)，得到二冪規模的最壞D_S(n)=Θ(lg³n)。平均平行度為Θ(n/lg²n)，不是把merge的平方對數錯當sort的span。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非二冪規模也能構造同時昂貴的各層

任意n可遞迴安排各區間的相對秩：n偶數時，前半放偶秩、後半放奇秩；n奇數時，較短的前半放奇秩、較長的後半放偶秩。再對兩半各自的有序鍵集合做相同安排。每個sort節點排序後的兩run都互相交錯，長度差至多1；二冪時就是前頁的位元反轉族。

對這種merge，較長run的median左方有m個本run元素，另一run左方有m或m+1個。左child仍是長度差至多1的交錯run，總長介於N/2−1與N/2+1；N≥8時，兩run的長度也都為Θ(N)。所以沿該路徑有Θ(lg N)層，每層二分搜尋須Θ(lg N)比較（規模隨層數固定比例下降），總span為Θ(lg²N)。

同一遞迴秩配置讓每個sort節點都滿足此條件。沿較大的child走，規模每層約減半，且須等它完成後才開始parent merge，故可串接各層關鍵路徑，得到Ω(lg³n)。這補足任意n的緊下界，不只在不同層各自選互不相容的最壞輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模型下比較 work 與最壞 span

算法	Work	最壞 span	平均平行度
遞迴P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
row平行matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
reduction matrix-vector	$\Theta(n^2)$	$\Theta(\lg n)$	$\Theta(n^2/\lg n)$
雙外圈平行matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
八路遞迴matrix product	$\Theta(n^3)$	$\Theta(\lg^2 n)$	$\Theta(n^3/\lg^2 n)$
平行Strassen	$\Theta(n^{\lg 7})$	$\Theta(\lg^2 n)$	$\Theta(n^{\lg 7}/\lg^2 n)$
DOT reduction matrix product	$\Theta(n^3)$	$\Theta(\lg n)$	$\Theta(n^3/\lg n)$
Parallel merge	$\Theta(n)$	$\Theta(\lg^2 n)$	$\Theta(n/\lg^2 n)$
Parallel merge sort	$\Theta(n\lg n)$	$\Theta(\lg^3 n)$	$\Theta(n/\lg^2 n)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模型下比較 work 與最壞 span （續）

表中漸近式採n充分大；空與單元素依程序基例。Work有效率、span小、空間少是不同目標；對固定P，還要把W/P、D及記憶體需求一起比較，不以一個平行度比值代替整個分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 27。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§27.1-27.3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28 矩陣運算

Part VII Selected Topics

本章定位

消去法必須同步保存置換與先前乘數；本章由LUP完整算法推導求逆、Schur補及乘法歸約，再以SPD與正交殘差連結Cholesky和最小平方，區分精確算術、rank與數值敏感度。

前置知識

線性代數、高斯消去與最小平方。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 LUP 解線性系統並解釋置換。
- 用分塊與 LU 分析矩陣反轉。
- 處理 SPD 矩陣與最小平方正規方程。
- 28.1 Solving systems of linear equations
- 28.2 Inverting matrices
- 28.3 Symmetric positive-definite matrices and least-squares approximation

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.1 Solving systems of linear equations

本章先在實數、精確算術模型討論。n×n矩陣A若非奇異，每個b都有唯一解Ax=b；若奇異，可能無解或無限多解，不能以「消去遇零」直接斷言所有b都無解。非奇異也不代表不換row就能消去，例如A=[0 1;1 0]可逆但第一個對角pivot是0。

LUP分解的契約是 **PA=LU**：P交換row，L為對角全1的下三角矩陣，U為對角非零的上三角矩陣。P用陣列p表示：目前第i個row來自原第p[i]個row，所以(Pb)[i]=b[p[i]]。這是row置換，不是對未知向量x的column置換。

由PAx=Pb與PA=LU，先解Ly=Pb，再解Ux=y。回代可得$Ax=P^{-1}Ly=P^{-1}Pb=b$，並由非零對角與P可逆得到唯一性。若P=I才可簡寫A=LU；不能一邊假設A=LU，一邊無來由把右端改成Pb。

前向與後向代入的依賴

L為unit lower triangular，依i遞增：$y[i]=b[p[i]]-\Sigma_{j<i} L[i,j]y[j]$。算第i項時，所有右側的y[j]已知；代入保證前i列Ly=Pb成立。

依i遞減：$x[i]=(y[i]-\Sigma_{j>i} U[i,j]x[j])/U[i,i]$。算第i項時，後面的未知已解；回代後最後n−i+1列Ux=y成立。對角非零才可除。兩個不變量分別完成全部列，這不是把未知量「同時猜出」。

```
Solve(L,U,p,b):
  for i=1..n:
    y[i]=b[p[i]]-sum(j=1..i-1,L[i,j]*y[j])
  for i=n..1:
    x[i]=(y[i]-sum(j=i+1..n,U[i,j]*x[j]))/U[i,i]
  return x
```

各做n(n−1)/2個求和乘積，加減與n次除法，時間$\Theta(n^2)$，額外向量空間O(n)。已分解的A可重用於r個右端，總時間$\Theta(n^3+rn^2)$；每個b都重新分解會浪費已得的相同因子。

消去、Schur Complement 與存在性

若目前第一個pivot為$a\neq 0$，把待處理矩陣寫成$[A]=[a\ w^T;v\ B]$。消去的恆等式是

$[a\ w^T;v\ B]=[1\ 0;v/a\ I]\cdot[a\ w^T;0\ B-vw^T/a]$。

右下$S=B-vw^T/a$稱Schur complement。左因子可逆，因此若原矩陣非奇異，右因子亦非奇異；$a\neq 0$時，若S有非零z滿足$Sz=0$，向量$(-w^Tz/a,z)$會落在右因子的nullspace，矛盾。因此S非奇異，可繼續遞迴。

若pivot column全部是0，當前S有零column、必奇異，連同已消去的非零pivots推出原A奇異。反之非奇異S的第一column總有非零項，換到pivot位置後，上述推理可持續直到n。這證明**每個非奇異A都存在LUP**，但沒有證明不置換的LU總存在。

原地 LUP：換 Row 必須包含先前因子

work的上三角含U、對角下方儲存L的乘數，L的對角1不另存。以下交換整個row，包含columns $1..k-1$已存的L乘數。

```
work=copy(A); p=[1,..,n]
for k=1..n:
  r=argmax(i=k..n,abs(work[i,k]))
  if work[r,k]==0: 回報singular
  swap(work[k,1..n],work[r,1..n]); swap(p[k],p[r])
  for i=k+1..n:
    work[i,k]=work[i,k]/work[k,k]
    for j=k+1..n:
      work[i,j]=work[i,j]-work[i,k]*work[k,j]
```

為何舊L也要換？第k輪之前已有$PA=[L_{11}\ 0;L_{21}\ I]\cdot[U_{11}\ U_{12};0\ S]$。只交換active rows的置換R會把S變RS，也把L_{21}變RL_{21}，否則兩因子乘積的下半row仍指向錯的原方程。接著本輪消去再把新乘數併入L，維持$PA=LU$的分塊不變量。

$\Sigma_k(n-k)^2=\Theta(n^3)$個尾塊更新；找pivot共$\Theta(n^2)$，不改立方級序。這些是算術操作界，並非任意長有理數的位元成本界。

兩次 Row Swap 的完整數例

取A=[0 2 1;1 1 0;2 3 4]、b=(0,0,7)ᵀ。第一column選abs最大2，交換row1、3，p=(3,2,1)。乘數 $m_{21}=1/2$、$m_{31}=0$，尾塊成[−1/2 −2;2 1]。

第二column選abs最大2，再交換目前row2、3，p=(3,1,2)。**L第一column的1/2也從row2移到row3**。新乘數$m_{32}=(-1/2)/2=-1/4$，最後pivot為$-2-(-1/4)\cdot 1=-7/4$。

L=[1 0 0;0 1 0;1/2 −1/4 1]，U=[2 3 4;0 2 1;0 0 −7/4]。

逐row乘LU得到[2 3 4;0 2 1;1 1 0]，恰為PA依原row3、1、2重排。若只換尾塊而沒換舊L，row2的L乘數會錯留1/2，LU≠PA。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩次 Row Swap 的完整數例 （續）

兩次 Pivot：尾塊與已存 L 必須同步

綠格 = 已存L乘數；藍格 = U或尚待消去的尾塊。L對角1不另存。

第2次交換時，第一column的1/2必須從目前row2移到row3。

最後 m31=1/2、m32=−1/4、U33=−7/4，對應PA=LU。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例的前後代入與穩定性界線

$Pb=(7,0,0)^T$。前向代入：$y_1=7$、$y_2=0$、$y_3=0-(1/2)\cdot 7-(-1/4)\cdot 0=-7/2$。

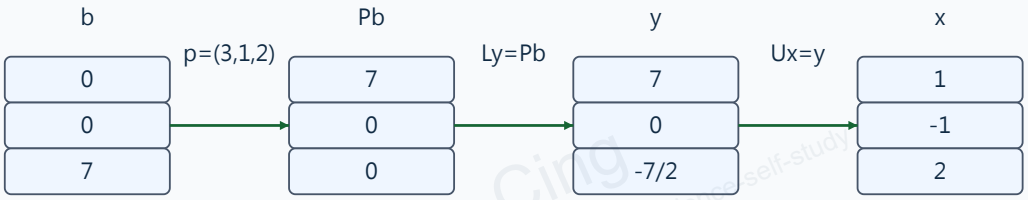
後向代入：$x_3=(-7/2)/(-7/4)=2$；$x_2=(0-1\cdot 2)/2=-1$；$x_1=(7-3\cdot(-1)-4\cdot 2)/2=1$。代回原A，三列分別 $0-2+2=0$、$1-1=0$、$2-3+8=7$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例的前後代入與穩定性界線 （續）

從 b 到 Pb、y、x：兩個三角系統

同一A與b；row置換只重排方程右端，沒有重排未知量x。



前向：$y3=0-(1/2)\cdot 7=-7/2$。

後向：$x3=2$，$x2=-1$，$x1=1$；代回原A得$(0,0,7)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial pivoting（部分選主元）只在當前column選abs最大值，精確算術保證本輪$|m_{ik}|\le 1$；它防止此column產生過大乘數，**不是對所有矩陣都保證浮點誤差小**。消去仍可能有growth，病態A也可能把小資料擾動放大。數學上的「非零pivot」與程式採用的數值tolerance不是相同定義；近奇異判定需連同尺度和誤差模型說明。

28.2 Inverting matrices

A^{-1}的第j個column是$Ax=e_j$的解；先LUP一次，再做n次代入，$\Theta(n^3)+n\cdot\Theta(n^2)=\Theta(n^3)$。輸出必須核對$AA^{-1}=I$；在精確算術，方陣的右逆就是唯一逆。解單一Ax=b通常直接代入，不必先求逆再乘b。

例A=[1 1;2 5]，U=[1 1;0 3]、L=[1 0;2 1]。對e_1：y=(1,−2)，回代$x_2=-2/3$、$x_1=5/3$。對e_2：y=(0,1)，回代$x_2=1/3$、$x_1=-1/3$。因此A^{-1}=[5/3 −1/3;−2/3 1/3]，不是各entry倒數。

乘回左上5/3−2/3=1、右上−1/3+1/3=0、左下10/3−10/3=0、右下−2/3+5/3=1。若A奇異，某些特定b仍可解，也不會使這n個單位向量都具有唯一解，因此不能交付同一定義的A^{-1}。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block Inverse 的次序與前提

令A=[B C;D E]，先要求B可逆，設$S=E-DB^{-1}C$。分解A=[I 0;DB^{-1} I]・[B C;0 S]；若A也可逆，S必可逆。把兩因子的逆依**反序**相乘，得到

```
A⁻¹ = [ B⁻¹+B⁻¹CS⁻¹DB⁻¹    −B⁻¹CS⁻¹ ]
      [ −S⁻¹DB⁻¹              S⁻¹     ]
```

矩陣乘法不交換，不能把C、D或逆的位置任意互換。上例B=1,C=1,D=2,E=5，S=3，左上1+1・1・(1/3)・2・1=5/3，得到與逐column求逆完全相同的矩陣。

這個特定分塊程序不能對所有可逆A直接遞迴。例如[0 1;1 0]可逆，但選B=0就不能形成B^{-1}。下一節的SPD性質保證leading block與Schur都可逆；一般矩陣要加置換或先使用下述理論歸約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block Inverse 的次序與前提 （續）

Schur Complement：消去後的確切 Block

左乘消去矩陣 [I 0;−DB⁻¹ I]；B需可逆，乘法次序不可交換。

A

B	C
D	E

→ 消去後

B	C
0	E−DB⁻¹C

S=E−DB⁻¹C；A=[I 0;DB⁻¹ I]·[B C;0 S]。

例B=1,C=1,D=2,E=5：S=3，A^{-1}=[5/3 −1/3;−2/3 1/3]。

若B=0，不能先寫B^{-1}；整個A可逆也不能省掉此前提。

乘法與反矩陣的兩向歸約

給兩個n×n矩陣B,C，構造3n×3n的T=[I B 0;0 I C;0 0 I]，則T^{-1}=[I −B BC;0 I −C;0 0 I]，乘回的右上block為BC−BC=0。只要inversion成本I(n)=Ω(n^2)且I(3n)=O(I(n))，建構Θ(n^2)加一次求逆即可在O(I(n))取出BC。必須寫出維度與regularity，不能只說兩問題「看起來相似」。

反方向，先對SPD矩陣分塊。B與S也SPD（下一節證明），可遞迴逆兩個n/2方塊，再做常數個矩陣product：T(n)≤2T(n/2)+O(M(n))。若M(n)=Ω(n^2)、padding到不到2n只改常數，且M(n/2)≤cM(n)、c<1/2，展開後第ℓ層至多$(2c)^{\ell}$O(M(n))，等比級數收斂，故T(n)=O(M(n))。M(n)=Θ(n^ω)、ω≥2滿足條件。

一般實非奇異A的G=A^{T}A為SPD，因z^{T}Gz=$||Az||^2$>0。A^{-1}=$G^{-1}A^{\mathsf{T}}$，乘右側A可驗為I，總成本仍O(M(n))。非2冪維度可padding成diag(A,I)，逆的原block不變。這是**精確算術複雜度歸約**；浮點實作不宜為了這個證明特意形成A^{T}A，因其condition number會平方。複數版本改用A^*，不能用普通轉置宣稱正定。

28.3 Symmetric positive-definite matrices and least-squares approximation

實對稱正定SPD意指$A=A^{\mathsf T}$且對每個非零z皆有$z^{\mathsf T}Az>0$。若A奇異，存在非零z使$Az=0$，會給$z^{\mathsf T}Az=0$，矛盾，因此SPD必非奇異。對任意leading block B，將其非零向量z補零成$(z,0)$，其二次型仍正，故B亦SPD。

對SPD分塊$A=[B\ C;C^{\mathsf T}\ E]$，$S=E-C^{\mathsf T}B^{-1}C$對稱。取任意非零$y$，令$z=-B^{-1}Cy$；則

$(z,y)^{\mathsf T}A(z,y)=z^{\mathsf T}Bz+2z^{\mathsf T}Cy+y^{\mathsf T}Ey=y^{\mathsf T}(E-C^{\mathsf T}B^{-1}C)y=y^{\mathsf T}Sy>0$。

所以S也SPD。第一個pivot為$e_1^{\mathsf T}Ae_1=A_{11}>0$；消去後繼續處理SPD Schur，歸納全部pivot皆正。這才給出SPD不用pivoting也不會除0的完整理由；單憑「對角線原本都正」不夠，例如$[1\ 2;2\ 1]$對角正，第二pivot卻為-3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

$LDL^{\mathsf T}$ 與 Cholesky 的具體計算

對稱消去每步得到正pivot d_j與unit-lower乘數，可歸納$A=L\,D\,L^{\mathsf T}$，其中D是全正對角。令$C=L\,D^{1/2}$，就得Cholesky $A=CC^{\mathsf T}$，C為正對角下三角。也可直接依column計算：

$C[j,j]=\mathrm{sqrt}(A[j,j]-\Sigma_{k<j}\,C[j,k]^2)$，$C[i,j]=(A[i,j]-\Sigma_{k<j}\,C[i,k]C[j,k])/C[j,j]$，$i>j$。

已證的SPD Schur讓每個sqrt內為正，因此算法可以繼續。三重求和為$\Theta(n^3)$，只需保存一個triangle；常見flop計數下約$n^3/3$，與一般LU約$2n^3/3$的領先項不同，這不表示所有機器上精確快兩倍。

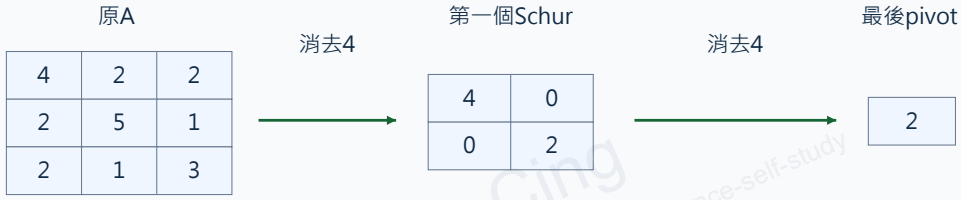
例$A=[4\ 2\ 2;2\ 5\ 1;2\ 1\ 3]$：$C_{11}=2$，$C_{21}=C_{31}=1$；$C_{22}=\mathrm{sqrt}(5-1)=2$；$C_{32}=(1-1\cdot 1)/2=0$；$C_{33}=\mathrm{sqrt}(3-1-0)=\sqrt{2}$。因此$C=[2\ 0\ 0;1\ 2\ 0;1\ 0\ \sqrt{2}]$，乘回三個對角為4、5、3，兩個非對角2、2與另一個1均相符。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LDL^T 與 Cholesky 的具體計算 （續）

正定 Schur 讓 Cholesky 每步合法

SPD消去的每個pivot都正；本例依序4、4、2。



C rows=[2,0,0]、[1,2,0]、[1,0,√2]。

CC^{T}對角為4,5,3，非對角為2,2,1，逐格回到原A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Least Squares：必要條件與全域最小的充分性

給$A\in\mathbb{R}^{m\times n}$、$b\in\mathbb{R}^m$，目標是$\min_x f(x)=||Ax-b||_2^2$。這個「對係數線性」模型也可用basis functions：$A[i,j]=f_j(t_i)$，不要求f_j對t是直線，例如1,t,t^2仍對係數線性。

沿任意方向z展開：$f(x+tz)=f(x)+2t\,z^TA^T(Ax-b)+t^2||Az||^2$。若x是最小值，所有方向的一次項都必為0，否則選足夠小且相反號的t可下降，所以$A^TAx=A^Tb$。反過來若此正規方程成立，t=1時$f(x+z)=f(x)+||Az||^2\geq f(x)$，證明它確為全域最小，不只是stationary point。

A^TA總為對稱半正定，$z^TA^TAz=||Az||^2$。它是SPD **當且僅當columns線性獨立**。full column rank時唯一解；rank不足時係數有多解，但預測Ax仍唯一：兩個最小解差z滿足Az=0。至少一個解總存在，可取A中獨立columns作basis，先解該full-rank子矩陣的SPD正規方程，其殘差垂直basis，也垂直所有columns；若A全零，則直接取x=0、p=0，殘差為b。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full-column-rank 的 Pseudoinverse 與投影

若A有full column rank，定義$A^{+}=(A^{T}A)^{-1}A^{T}$，維度n×m。正規方程給$\hat{x}=A^{+}b$。$A^{+}A=I_{n}$，但一般AA^{+}不是I_{m}；它是$Q=A(A^{T}A)^{-1}A^{T}$，滿足$Q^{T}=Q$與$Q^{2}=Q$，且對每個$Az\in Col(A)$，$QAz=Az$。因此Q把b投到Col(A)，殘差$(I-Q)b$滿足$A^{T}(I-Q)b=0$。

對下頁3×2的design matrix，A^{+} rows為(5/6,1/3,−1/6)、(−1/2,0,1/2)。乘b=(0,2,1)得到(1/2,1/2)。Q rows為(5/6,1/3,−1/6)、(1/3,1/3,1/3)、(−1/6,1/3,5/6)，顯然不等於3×3單位矩陣；乘b得到(1/2,1,3/2)，不是原b。

rank不足時$(A^{T}A)^{-1}$未定義，這個公式不能直接使用。更一般的Moore–Penrose pseudoinverse可由SVD構造；本章的公式與唯一係數結論限full column rank，不把奇異Gram硬求逆。

三個不共線資料點的完整擬合

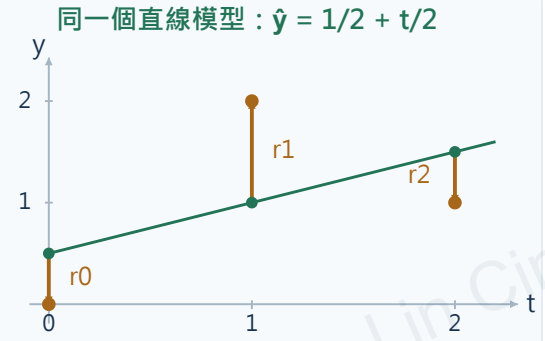
保留三點(t,y)=(0,0),(1,2),(2,1)，設$x=(c,m)^{T}$、模型$\hat{y}=c+mt$。

A=[1 0;1 1;1 2]，b=$(0,2,1)^{T}$，$A^{T}A$=[3 3;3 5]，$A^{T}b=(3,4)^{T}$。方程3c+3m=3、3c+5m=4相減得m=1/2，c=1/2。

t	觀測y	預測$\hat{y}$	$r=y-\hat{y}$
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$A^{T}r$的兩分量為−1/2+1−1/2=0與0・(−1/2)+1・1+2・(−1/2)=0；$||r||^{2}=3/2$。columns(1,1,1)與(0,1,2)獨立，才是唯一係數的原因；**資料點是否共線不決定這個design matrix的column rank**。若仍用相同t、改y=(1,2,3)，三資料共線，columns仍獨立，只是殘差變0。

三個不共線資料點的完整擬合 （續）

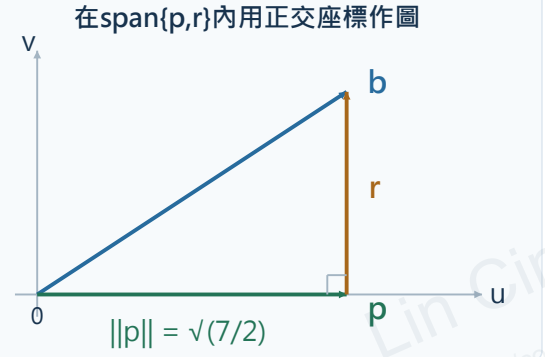


t	y	ŷ	r
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$r^Tr = 1/4 + 1 + 1/4 = 3/2$

橘色：固定t的垂直誤差；綠色：預測點與擬合線

把三個觀測值視為R^3中的一個向量



R^3座標

b = (0, 2, 1)
p = (1/2, 1, 3/2)
r = (−1/2, 1, −1/2)

u = p / ||p||
v = r / ||r||

$A^Tr = (0, 0)$
$||r||^2 = 3/2$

截面E=span{p,r}；E與Col(A)的交集是水平方向span{p}

本例p、r非零且正交，取u=p/||p||、v=r/||r||作正交單位座標軸；右側列出的b、p、r則是原本的R^3座標。$A^Tr=0$使r垂直整個Col(A)。截面E=span{p,r}只沿span{p}與Col(A)相交，圖中的水平線只表示這個交集；Col(A)本身仍是R^3中的二維平面。

正交殘差、Condition Number 與 Rank 缺失

設$p=A\hat{x}$、$r=b-p$。$A^Tr=0$表示r垂直整個Col(A)，且$b=p+r$。對任一係數$\hat{x}+z$，$b-A(\hat{x}+z)=r-Az$，兩項正交，所以平方誤差$=||r||^2+||Az||^2$；這把代數條件連到最小距離幾何。

在(t,y)的散點圖，r_i是固定t_i時的**垂直y誤差**，不是該點到擬合線的最短距離。在觀測向量圖中，整組b是R^3中的一個向量，r才是點b到子空間Col(A)的最短距離向量；兩個圖使用不同的空間。

若columns重複，例如A=[1 1;1 1;1 1]、b=(0,2,1)，只決定$c_1+c_2=1$；全部$(c_1,c_2)=(s,1-s)$都是解，預測仍為(1,1,1)，殘差(−1,1,0)唯一。

full column rank時，2-norm條件數$\kappa_2(A^TA)=\kappa_2(A)^2$，因奇異值變成平方。例diag$(1,10^{-3})$的κ為$10^3$，Gram的κ為$10^6$；這是數值敏感度，不是浮點算術次數。實作常用QR或SVD求least squares，尤其接近rank缺失時；正規方程的精確證明不能當成所有浮點資料都穩定的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 28。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29 線性規劃

Part VII Selected Topics

本章定位

由等價建模、精確pivot與防循環推導two-phase simplex，並以對偶和ray交付三種結果的證書。

前置知識

凸多面體、標準形與對偶。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 將問題轉成 standard form 與 slack form。
- 從基本可行解執行 simplex。
- 使用 weak/strong duality 與可行性啟動方法。
- 29.1 Standard and slack forms
- 29.2 Formulating problems as linear programs
- 29.3 The simplex algorithm
- 29.4 Duality
- 29.5 The initial basic feasible solution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.1 Standard and slack forms

本章的standard form固定為 `max cᵀx · Ax≤b · x≥0`，A∈ℝ^(m×n)。所有限制非嚴格，決策變數為連續實數；有理係數時才能以有限binary輸入談bit complexity。其他書也可能把等式形式叫standard，須先確認慣例。

Feasible point滿足全部限制；optimal point在可行點中達最大目標。Infeasible是沒有可行點；unbounded是可行且目標可任意大，不是「可行區域無界」的同義詞。例如x≥0、max(−x)的區域無界，最優值仍是0。

有限個線性半空間的交集是凸多面體：若Ax≤b、Ay≤b，則A(λx+(1−λ)y)≤b，0≤λ≤1。凸性不保證整數解，也不容許把非線性乘積當成線性限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四種轉換都要能還原原變數

最小化f改最大化−f，輸出目標記得取負；$a^\top x \ge b$乘−1改≤；等式拆成≤與≥。無符號限制的u改$u=u^+-u^-$、$u^+,u^-\ge 0$：任意u可取(max(u,0),max(−u,0))，反向取差也恢復原點，雖不是一對一，最優值仍對應。

例 `min 2u−v · u+v=3 · u−2v≥−3 · v≥0 · u unrestricted`，改最大化$-2u^+ +2u^- +v$，限制

$u^+-u^-+v\le 3$；$-u^+ +u^- -v\le -3$；$-u^+ +u^- +2v\le 3$。

代u=3−v得v≤2，原目標6−3v，故v=2,u=1、最小0；新目標最大0。若原有n變數、m限制，拆符號至多2n變數、拆等式至多2m限制，係數位元長僅增加常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Slack Form、Basis 與可行基本解

對Ax≤b引入s=b−Ax≥0，等式矩陣M=[A I_m]、全變數q=(x,s)≥0，Mq=b。Slack有唯一值，不是另一組可任意分配的決策。

選m個線性獨立columns作basis B，令其餘N非基本。由可逆M_B得

$q_B=M_B^{-1}b-M_B^{-1}M_Nq_N$，$z=v+\rho_N^Tq_N$。

非基本全設0為basic solution；可行當且僅當$M_B^{-1}b$逐項≥0。初始slack basis是I，故初始點可行當且僅當b≥0。Basic不等於正：值0的基本變數是退化的來源。給定B後字典唯一，因解出線性方程所用的inverse唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

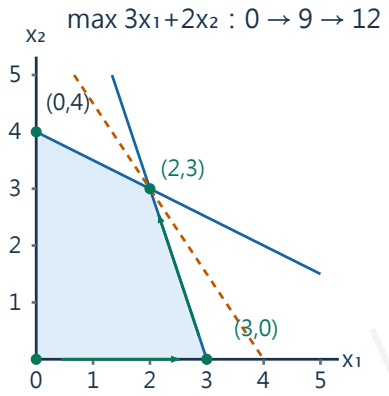
幾何、等值線與 Basis 是同一個例子

以下全部使用 $\max\ 3x_1+2x_2$，$x_1+2x_2\le8$，$3x_1+x_2\le9$，x≥0。可行四邊形頂點(0,0)、(3,0)、(2,3)、(0,4)。目標沿頂點路徑0→9→12改善，最後(2,3)。

初始字典：$x_3=8-x_1-2x_2$；$x_4=9-3x_1-x_2$；$z=3x_1+2x_2$。Basis={x_3,x_4}，初始slack=(8,9)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

幾何、等值線與 Basis 是同一個例子 （續）



藍線：兩個資源限制

$x_1+2x_2=8$

$3x_1+x_2=9$

綠線：simplex 的兩次移動

橘虛線：$3x_1+2x_2=12$

交點 (2,3)，目標 12

最優點也可能形成整條edge，例如改目標x_1+2x_2，邊(0,4)到(2,3)都同值8。頂點法會回其中一個最優基本解，不宣稱最優解唯一。

29.2 Formulating problems as linear programs

建立模型先列「變數代表什麼、單位、允許範圍」，再逐項把可行解映到原問題，也把每個原可行方案映回變數。只有目標相似而漏了容量或守恆，並不是等價模型。

最短路使用potential d_v，固定d_s=0，對每邊uv限制d_v−d_u≤w_uv，最大化d_t。d沒有非負限制，負權路徑可能使d_t<0。

先取全部頂點從s可達且無負cycle、t可達的情形。沿任意s-t路把不等式相加，得d_t≤該路長，所以任何可行d是最短距離的下界；真距離δ(s,v)由三角不等式滿足所有限制，達到上界δ(s,t)。故最大化而非最小化恰得到最短距離。

Potential 的邊界不是程式中的 Infinity

若只研究s可達子圖，無負cycle時所有δ有限，可用前頁模型；t不可達要另外回報，不能把∞當實數LP變數。若保留不可達子圖，當中負cycle會使模型不可行，即使s到t距離完全正常，所以不能漏這個前提。

對無任何負cycle的全圖，可加0權supersource求一組全圖可行potential；若t不從s可達，可在不可達集合整體加任意正K，仍保持所有差限制，因無可達→不可達邊，故d_t無上界。這時LP unbounded對應t不可達，不是「存在負長路」。

數例s→a權2、a→t權−1、s→t權4，限制d_a≤2、d_t≤d_a−1、d_t≤4。取(0,2,1)可行，且兩邊路給d_t≤1，答案1；若錯加d≥0在別的負總長例會排除正確解。

Flow 模型：每個 Arc 的非負量

此節用非負arc flow g_uv，而非上一章的反對稱淨流。最大流：對每個原邊設0≤g_uv≤c_uv，內點滿足Σ_out g−Σ_in g=0，最大化source淨流出。允許antiparallel，只要每個原arc各有自己的變數與容量；不能再加g_uv=−g_vu。

給原flow就滿足模型；給模型解，其容量與守恆正是flow定義。原邊變數m，限制O(n+m)，不必配置n^2個非邊變數。

最小成本流另固定source淨流出D、sink淨流入D，最小化Σ a_uv g_uv。D是「恰好需求」，不可只最大化總流再把成本當附帶值。原邊有有限容量時，負成本cycle可送循環流但不會因而無界；禁止循環需要另外建模。

最小成本流的完整數值模型

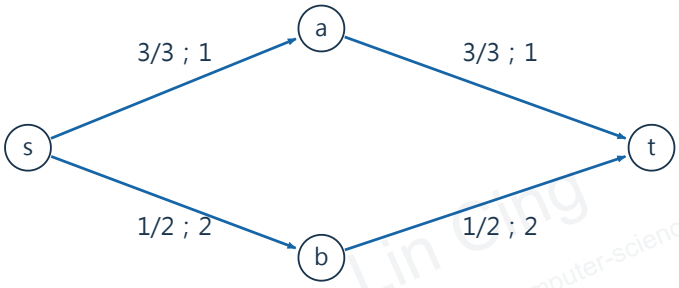
原邊s→a容量3、單位成本1；a→t容量3、成本1；s→b容量2、成本2；b→t容量2、成本2。需求D=4。守恆令上下兩路流分別u,v，得到u+v=4，0≤u≤3、0≤v≤2，最小化2u+4v。

代v=4−u，成本16−2u；可行2≤u≤3，故u=3,v=1，成本10。這同時交付每邊flow、需求、可行區間與最優性，不只列出一條看似便宜的路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小成本流的完整數值模型 （續）

需求 D=4；標記為「流 / 容量；單位成本」



u+v=4

2≤u≤3

成本=16−2u

最優 u=3,v=1

上路成本 3×(1+1)=6；下路成本 1×(2+2)=4；總 10

若把需求錯寫u+v≤4，零流成本0會被誤認最佳；若省容量u≤3，會錯選u=4，成本8。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multicommodity 不能拆成互不相干的最大流

對commodity k給(s_k,t_k,D_k)，每原邊有$g^k_{uv}\geq 0$。每種貨物各自滿足內點守恆、端點淨供需$\pm D_k$；共享容量是$\Sigma_k g^k_{uv}\leq c_{uv}$。若只問可行，目標可設0；可另加總成本。

同一瓶頸u→v容量1，兩種貨都唯一經過此邊，需求各3/4。各別都可送，但合併需求3/2>1，不可行。兩條獨立最大流會漏掉共享資源。

k種貨、m邊、n點有km變數、O(kn+m)限制，對給定有理D與容量為多項式大小LP。這是可分割貨流；要求每種貨只能選一條路或要求整數量，會改成不同的離散問題，不能沿用一般LP的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.3 The simplex algorithm

字典固定寫 $x_i=b_i-\Sigma_{(j\in N)}a_{ij}\ x_j$、$z=v+\Sigma_{(j\in N)}c_j\ x_j$。此處a是「被減去」的係數，與畫在式子中的符號相反；後面所有ratio與pivot都用這個契約。

給可行字典$b_i\geq 0$，選$c_e>0$入基，令$x_e=\theta\geq 0$、其他非基本0。基本值$x_i=b_i-a_{ie}\theta$。$a_{ie}\leq 0$不限制增加；$a_{ie}>0$要求$\theta\leq b_i/a_{ie}$。因此取這些正分母的最小非負比值，包括0。

沒有正分母時，方向$q_e=1$、$q_i=-a_{ie}\geq 0$、其他$q=0$給一條可行ray：$x(\theta)=x(0)+\theta q$，目標$v+c_e\theta\to\infty$。不能只寫$x_e=\infty$，∞不是可行實數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pivot 必須保留離基變數的係數

選離基l，a_le≠0，從l列精確解出

x_e=b_l/a_le−x_l/a_le−Σ_(j∈N\{e})(a_lj/a_le)x_j。

這一列代回所有其他列與目標，再交換B、N。原l變成非基本，但不是從方程中消失；漏掉−x_l/a_le會改變可行集合。

新a_el=1/a_le、b_e=b_l/a_le、a_ej=a_lj/a_le。對i≠l：b'_i=b_i−a_ie b_e，a'_il=−a_ie/a_le，a'_ij=a_ij−a_ie a_lj/a_le。目標v'=v+c_e b_e、c'_l=−c_e/a_le、c'_j=c_j−c_e a_lj/a_le。

以上是可逆代入，不只對目前基本解成立，而是保留全部滿足等式的點及其目標；配上所有變數非負，得到同一LP。

Ratio Test 保持可行的一般證明

新入基值b'_e=b_l/a_le=θ≥0。其餘b'_i=b_i−a_ieθ：若a_ie>0，θ≤b_i/a_ie使其非負；若a_ie≤0，則b'_i≥b_i≥0。新非基本全0，所以新基本解可行。

目標增加c_eθ≥0。θ>0時嚴格改善；θ=0時點與目標可能完全不變，只是basis不同。此證明同時說明為何不限制a_ie<0的列，以及為何不能排除零比值。

```
while some reduced cost c[e]>0:
    choose entering e
    candidates={i in B : a[i,e]>0}
    if candidates empty: return feasible point + improving ray
    l=minimum-ratio row, with a specified anti-cycling rule
    pivot(l,e)
return current basic solution
```

同一二維例的兩次 Pivot

初始B={3,4}。先讓x_1入基，比值8/1=8、9/3=3，故x_4離基、θ=3：

$x_1=3-x_2/3-x_4/3$；$x_3=5-5x_2/3+x_4/3$；$z=9+x_2-x_4$。

再選x_2，x_1列比值3/(1/3)=9、x_3列5/(5/3)=3，故x_3離基：

$x_2=3-3x_3/5+x_4/5$；$x_1=2+x_3/5-2x_4/5$；$z=12-3x_3/5-4x_4/5$。

令非基本$x_3=x_4=0$，得到$(x_1,x_2)=(2,3)$，兩限制恰等號。所有reduced costs≤0，任意可行點的目標都≤12，所以此點最優。不必再數值搜尋整個可行多邊形。

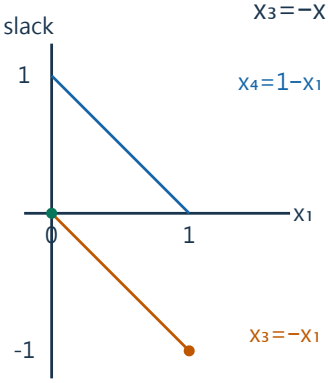
零步長是合法 Pivot

$\max\ z=x_1$；$x_3=-x_1$；$x_4=1-x_1$，所有變數非負，初始B={3,4}、N={1}、基本值(0,1)。

基本列	正分母	比值	θ=0	錯取θ=1
x_3	1	0	0	−1，不可行
x_4	1	1	1	0

令x_3離基，得$x_1=-x_3$、$x_4=1+x_3$、$z=-x_3$。新基本點仍(0,0,1)，目標仍0，卻已能由非正reduced cost證最優。若只取positive ratio就立即越出可行域。

零步長是合法 Pivot （續）



$x_3=-x_1$，$x_4=1-x_1$：零比值不能略過

$x_1=0$：$x_3=0$、$x_4=1$，可行

ratio：0/1=0 與 1/1=1

合法步長 $\theta=0$，x_3 離基

錯取 $\theta=1$：$x_3=-1$，不可行

可行的 x_1 只有 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有限 Basis 本身不能排除 Cycling

M=[A I]有m行，任何basis選m個獨立columns；至多C(n+m,m)種。給定basis後字典唯一，所以若永不終止，必重複basis。但退化時目標可以不變，僅說「目標不下降」不能排除重複。

本章用精確lexicographic perturbation給出一個可自足證明的防循環版本。給當前可行起始字典 $q_B=\beta-Dq_N$，把RHS改$\beta_i+\varepsilon^i$，i=1,…,m；ε是正無窮小，**不是**某個任意浮點小數。常數項非負且每行有正的第一個ε係數，所以初始字典在此有序域嚴格可行。

計算與比較改用係數向量：先比常數，再比ε、ε^2,…；首個非零係數決定正負。後續pivot仍用正分母的最小比值，若實數比值相同，用後面的係數分出順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lexicographic 規則為何有限終止

把初始字典等式矩陣記$M_0=[I\ D]$，perturbed RHS $\beta+\Sigma e_i \varepsilon^i$。任何可逆basis K的基本值是$M_0,K^{-1}\beta + \Sigma(M_0,K^{-1}e_i)\varepsilon^i$。

Inverse每行不全零，因此每個基本值作為ε多項式都不恒等於0；可行性又保證它非負，故其實嚴格正。兩個正分母ratio不可能完全相同：否則inverse兩行成比例，違反可逆。每次合法pivot的θ因此嚴格正，c_e>0使目標在lex順序嚴格增加。

同一basis的字典／目標唯一，不能重訪。有限basis保證終止。取所有係數的常數項，就回到原LP的可行基本解；reduced costs與RHS無關，若全≤0則常數解也最優。若出現改善ray，其係數同樣不依ε，從常數可行點出發便證原LP無界。故不是靠挑一個「夠小」的浮點ε猜測正確性。

Lexicographic 平手的可重算例子

$x_3=2-x_1$；$x_4=2-x_1-x_2$；max x_1+x_2。初始把兩個RHS改$2+\varepsilon$、$2+\varepsilon^2$。x_1入基的兩比值是$2+\varepsilon$與$2+\varepsilon^2$，後者較小，所以x_4離基。

新列 $x_1=2+\varepsilon^2-x_2-x_4$、$x_3=\varepsilon-\varepsilon^2+x_2+x_4$、$z=2+\varepsilon^2-x_4$。後者已無正reduced cost，常數解$x_1=2$、$x_2=x_3=x_4=0$，目標2，符合原限制。$x_3$雖常數0，perturbed值$\varepsilon-\varepsilon^2$嚴格正，正是退化被符號分開的地方。

另一常用防循環法Bland規則是選最小索引的正reduced-cost入基，最小比值平手時選最小索引離基；兩處規則都要給。以上完整終止證明採lex版本，不把任意選取也說成必終止。

每次 Pivot 的成本與全程成本要分開

普通dense字典一次更新O(mn)個係數，不能由此稱整個simplex多項式；basis數C(n+m,m)對維度可能指數大。Lex版本另存每個RHS的m+1個係數，更新與比較增加多項式因子，仍只有有限／指數的全程最壞界。

有理輸入用精確分數做pivot。Basis解由矩陣子行列式的比值表示；清除原分母後，行列式有至多m!項，bit長受維度與輸入bit長的多項式界控制。這說明單一basis的精確表示不必無限制膨脹，不代表pivot次數變多項式。

浮點實作要另訂feasibility／reduced-cost容許值、ratio近零與重算殘差；把負的b無條件截成0可能改題。Ellipsoid／interior-point屬不同演算法，其多項式理論不由上述pivot計數推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.4 Duality

Primal $\max\ c^{\mathsf{T}}x \cdot Ax \le b \cdot x \ge 0$ 對偶為 $\min\ b^{\mathsf{T}}y \cdot A^{\mathsf{T}}y \ge c \cdot y \ge 0$。m個資源限制對應m個價格y；n個原變數對應n個價格下界。A需轉置，不能只交換max/min。

給兩側可行解，依$x \ge 0$、$A^{\mathsf{T}}y \ge c$與$y \ge 0$、$Ax \le b$依序得

$c^{\mathsf{T}}x \le x^{\mathsf{T}}A^{\mathsf{T}}y = y^{\mathsf{T}}Ax \le y^{\mathsf{T}}b$。

這是weak duality。任一可行dual給primal上界；若上下界相等，兩者同最優。只有相同目標值而不檢查兩側可行，不能當最佳性證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從終止 Basis 構造 Dual，一般證明

令M=[A I]、d=(c,0)，終止basis B的矩陣可逆。定義row vector $y^T=d_B^T M_B^{-1}$。代入 $q_B=M_B^{-1}b-M_B^{-1}M_N q_N$得

$z=y^Tb+\Sigma_{(j\in N)}(d_j-y^TM_j)q_j$。

所有columns的reduced cost $\rho_j=d_j-y^TM_j$，基本columns恰0。最優字典的$\rho\le 0$：對slack column e_i有$\rho_{n+i}=-y_i\le 0$，所以$y\ge 0$；對decision column A_j有$\rho_j=c_j-y^TA_j\le 0$，所以$A^Ty\ge c$。

目前可行基本解的目標是y^Tb，故達weak-duality等號。這證明有限最優primal必有同值dual；也得到讀取規則「$y_i=-$原slack變數的reduced cost」，若該slack基本則成本0。證明依完整字典恆等式，並非只在單一最優點比較係數。

同一數例的 Dual 與 Complementary Slackness

二維例dual是 $\min\ 8y_1+9y_2$，$y_1+3y_2\ge 3$、$2y_1+y_2\ge 2$、$y\ge 0$。終止目標$z=12-3x_3/5-4x_4/5$給$y=(3/5,4/5)$。兩個dual限制均等號，值24/5+36/5=12，與x=(2,3)吻合。

一般gap精確分解為

$b^Ty-c^Tx = y^T(b-Ax) + x^T(A^Ty-c)$。

對兩側可行，右側每項非負，所以gap=0當且僅當每項都0：$y_i(b_i-A_i\ x)=0$、$x_j((A^Ty)_j-c_j)=0$。這就是complementary slackness，必要與充分都來自此分解；有正slack則對應價格0，有正變數則對應dual限制緊束。

另例$\max\ 2x_1+x_2$，$x_1+x_2\le 4$，$x_1\le 2$，$x\ge 0$，dual可取y=(1,1)，上界6。x=(2,2)值6、全部互補乘積0；x'=(1,3)值5，唯一非零乘積為$y_2(2-x'_1)=1$。

同一數例的 Dual 與 Complementary Slackness （續）

$$\text{gap} = y^T(b-Ax) + x^T(A^Ty-c) \geq 0$$

$x=(2,2)$

$y=(1,1)$，dual 值 6

primal 6

$y_2(2-x_1) = 0$

其餘三個乘積皆 0

$x'=(1,3)$

$y=(1,1)$，dual 值 6

primal 5

$y_2(2-x_1) = 1$

其餘三個乘積皆 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無界與不可行不能隨便互換

Primal無界時dual不可能可行，否則固定dual值由weak duality限制所有primal值；同理dual向下無界則primal不可行。逆向單靠這句不成立：primal不可行不代表dual無界，兩者可能都不可行。

例如 `max −x · 0·x≤−1 · x≥0` 的primal不可行，dual `min −y · 0·y≥−1 · y≥0` 可行但向下無界。改為 `max x · 0·x≤−1 · x≥0`，dual限制0≥1也不可行，兩側皆不可行。

對一般符號可先轉standard再建dual；原primal等式限制的乘子可正可負，free primal變數對應dual等式。這些符號規則由拆成兩個非負變數或兩個相反不等式合併而來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.5 The initial basic feasible solution

初始b有負項時，不能直接跑只維持可行性的ratio test。對$Ax\le b, x\ge 0$加入$a\ge 0$，解auxiliary：$\max -a$ · $Ax-a\cdot 1\le b$。它總可行：取x=0、a=max(0,−min_i b_i)。目標≤0，所以不會向上無界。

原LP可行⇒同一x配a=0可達目標0，已是auxiliary最優。反向auxiliary最優0⇒a=0⇒同一x滿足原限制。若最優<0，原LP不可行。不能以「目前人工變數仍正」提早判不可行，必先確定auxiliary已最優。

有m=0時直接取原點為可行基本解；若某c_j>0便有無界ray，否則原點最優，不必求空集合的min。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次特殊 Pivot 建立 Auxiliary 可行基底

令$s_i=b_i-A_i x+a$，挑r使$b_r=\min_i b_i<0$。讓a入基、s_r離基；此步pivot係數−1，**不是**從可行基底選改善方向的ratio步。

解出 $a=-b_r+A_r\ x+s_r$，代其他列 $s_i=b_i-b_r+(A_r-A_i)x+s_r$。設新非基本x=$s_r$=0，得到$a=-b_r>0$、$s_i=b_i-b_r\ge 0$，故建立可行基底。現在才可使用前述simplex迴圈及lex防循環。

Lex防循環從這個可行字典開始：可只為迴圈加入符號RHS，再取常數項；reduced costs不依RHS，因此得到原auxiliary的真最優值及可行基本解，不會把ε目標誤判成原問題的不可行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

人工變數為零，仍要清理 Basis

Auxiliary最優0時，若a非基本，直接刪a欄。若a基本且值0，其列為 $a=-\Sigma\ a_{aj}\ x_j$；若存在非零係數，任選該x_j入基、a離基。這個θ=0的代數pivot不改目前點，係數可正可負，不是改善目標的ratio選步。

若整列係數皆0，則a=0是冗餘等式，刪此列與a即可。一般等式／多人工變數實作必處理此分支；對獨立的完整slack矩陣可另由row rank排除，但不應除以0。

最後把**原目標**中的基本變數逐一用目前列代入，得到只含非基本變數的目標；不可只改標題或把auxiliary目標−a當原答案。這時完成Phase I，才用新的可行basis跑Phase II。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Phase I 與原目標的完整操作例

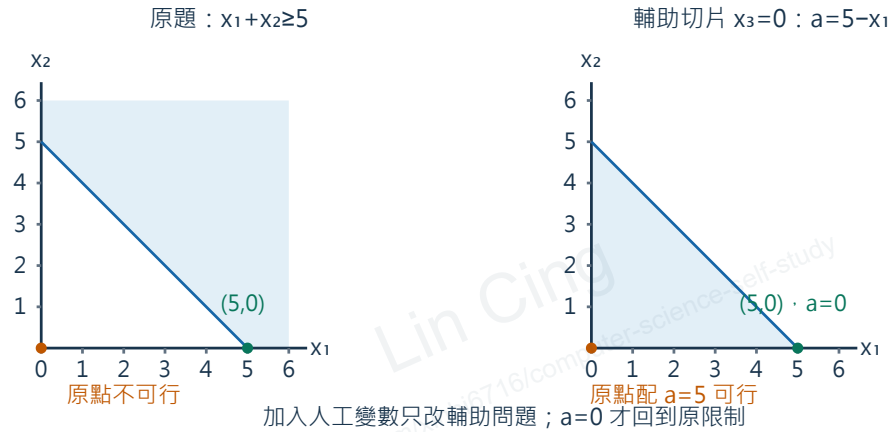
原題 $\min\ z=2x_1+x_2\ \cdot\ x_1+x_2\geq 5\ \cdot\ x\geq 0$。Surplus $x_3\geq 0$給$x_1+x_2-x_3=5$；引人工a：$a=5-x_1-x_2+x_3$，Phase I最小化w=a（等價最大化−a）。

狀態	基本列	非基本0時	目標
初始	$a=5-x_1-x_2+x_3$	$a=5$	$w=5$
x_1入、a離	$x_1=5-x_2+x_3-a$	$x_1=5$	$w=a$，最小0
刪a、代原目標	$x_1=5-x_2+x_3$	$x_1=5$	$z=10-x_2+2x_3$
x_2入、x_1離	$x_2=5-x_1+x_3$	$x_2=5$	$z=5+x_1+x_3$

最小化終止時係數≥0，故最優z=5。起始$(x_1,x_2)=(0,0)$對原題不可行；配a=5後只對輔助題可行。這個單等式例用一個人工欄；上一頁的共用a模型則直接適用所有Ax≤b列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Phase I 與原目標的完整操作例 (續)



加入人工變數只改輔助問題；a=0 才回到原限制

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不可行也要有算完的證書

原題 $x\le 1 \cdot x\ge 2 \cdot x\ge 0$，加共用a後為$x-a\le 1$；$-x-a\le -2$。相加得$-2a\le -1$，故$a\ge 1/2$；取$x=3/2$、$a=1/2$兩式恰等號，所以auxiliary最大$-a=-1/2<0$，原題不可行。

完整字典：$s_1=1-x+a$、$s_2=-2+x+a$。先讓a入、s_2離，得$a=2-x+s_2$、$s_1=3-2x+s_2$、$w=-2+x-s_2$。x入基兩ratio為2、3/2，s_1離基，得

$x=3/2-s_1/2+s_2/2$、$a=1/2+s_1/2+s_2/2$、$w=-1/2-s_1/2-s_2/2$。

兩個reduced costs非正，確已最優，不是因尚有人工量就猜不可行。亦可直接把原兩個相反限制相加得到$0\le -1$作獨立矛盾證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種結果與基本定理的依賴關係

Phase I先有可行auxiliary基底，lex迴圈有限終止且auxiliary不向上無界；最優<0回infeasible，最優0清理人工欄並恢復原目標。

Phase II仍以可行字典啟動，lex規則保證有限終止。若沒有正分母，交付可行點與改善ray，證unbounded；否則終止時reduced costs≤0，交付可行基本解與同值dual，證finite optimum。

所以每個本章標準形式LP恰屬不可行、可行而目標無界、可行且達有限最優之一。非嚴格限制與有限維LP結構很關鍵；若改成$\max\ x\ :\ 0\le x<1$，上確界1不被任何點達到，便不是這個三分結論的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 29。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31 數論演算法

Part VII Selected Topics

本章定位

由整除、群與CRT逐步證明模方程、模冪、RSA與隨機質性檢驗，區分確定因數證書和rho時間假設。

前置知識

整除、模算術、群結構與質數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用歐幾里得法與擴充 GCD。
- 解模線性方程與 CRT 系統。
- 分析 RSA、隨機質性測試與整數分解。

節次	主題	節次	主題
31.1	Elementary number-theoretic notions	31.2	Greatest common divisor
31.3	Modular arithmetic	31.4	Solving modular linear equations
31.5	The Chinese remainder theorem	31.6	Powers of an element
31.7	The RSA public-key cryptosystem	31.8	Primality testing
31.9	Integer factorization		

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.1 Elementary number-theoretic notions

整數d整除a，記d|a，是存在整數k使a=kd。質數p是p≥2且正因數只有1,p；1不是質數。Gcd(a,b)取非負最大公因數，a,b不全0時為正；另約定gcd(0,0)=0，程式先把負輸入取絕對值。

對n≥1，a≡b(mod n)代表n|(a−b)，a mod n為[0,n−1]唯一代表。負數也適用：−7 mod5=3。若a≡a'、b≡b'，則和、差、積都同餘，因差仍是n的倍數；除法不能直接照搬。

β位元整數的值可達約2^β；跑到√n的迴圈對β是指數。以下分開算術操作數與bit成本，乘除不是無條件O(1)。採普通乘法／長除法時，兩個O(β)位元整數的乘法與取餘上界$O(\beta^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bezout、消去律與唯一分解的依賴

對a,b不全0，考慮所有正線性組合ax+by的最小值g。以g除a，餘數a−qg也是a,b線性組合；若餘數正且小於g矛盾，所以g|a，同理g|b。所有公因數都整除g，故g=gcd(a,b)，且存在x,y使ax+by=g。

若gcd(a,n)=1，寫au+nv=1，乘b得b=abu+nbv；所以n|ab⇒n|b。這是合法消去a的前提。取n=6,a=2，2・1≡2・4卻1≠4，顯示省互質條件會錯。

質數p若不整除a，則gcd(a,p)=1，上式推出p|ab⇒p|b。每個n≥2可依較小因數遞迴分成質數；兩組分解比較時，第一組任一質數必整除另一組某質數，兩者相等，消去後歸納，得唯一質因數分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.2 Greatest common divisor

令a=qb+r、0≤r<b。共同整除a,b的整數也整除r=a−qb；共同整除b,r的整數也整除a=qb+r。故gcd(a,b)=gcd(b,r)。

```
euclid(a,b):
    a=abs(a); b=abs(b)
    while b!=0: (a,b)=(b,a mod b)
    return a
```

每輪第二參數嚴格下降至0，所以終止；不變量為當前gcd=原gcd，終態gcd(a,0)=a。若0<b≤a，第一餘數r<b；若r≤b/2已減半，若r>b/2則下一餘數b mod r=b−r<b/2。每兩次使第二參數至少減半，遞除次數O(log b)，β位元輸入為O(β)。

Fibonacci 最壞遞除鏈與 Bit 界

若a>b≥1需要k次遞除才到0，反推每步較早餘數≥後兩餘數之和，得到a≥F_(k+2)、b≥F_(k+1)，$F_1=F_2=1$。F每兩項至少倍增，所以k=O(log b)。相鄰Fibonacci輸入每步商1（末步除外），恰實現同階長鏈。

普通長除法單輪粗估$O(\beta^2)$，乘O(β)輪可安全得$O(\beta^3)$bit上界；這不是最佳界。若追蹤各輪操作數縮短可得到更精細$O(\beta^2)$傳統gcd界，但此處未用較精細界宣稱實作速度。

空間可只保留兩個餘數，O(β)位元；若為教學保存每列trace，便要另付儲存成本。沒有已知質因數分解也能算gcd，不能把「先分解」當成Euclid的前置工作。

Extended Euclid 的一般回推

若遞迴對(b,r)回傳g=b x'+r y'，代r=a−qb可得g=a y'+b(x'−q y')，所以回傳(x,y)=(y',x'−q y')。基例b=0回傳(g,x,y)=(a,1,0)。

```
extended(a,b): # 此處 a,b>=0
    if b==0: return (a,1,0)
    (g,x1,y1)=extended(b,a mod b)
    return (g,y1,x1-floor(a/b)*y1)
```

例252=1・198+54，198=3・54+36，54=1・36+18，36=2・18。回推18=54−36=4・54−198=4・252−5・198。故gcd=18，Bezout係數(4,−5)。負原輸入最後再對對應係數調符號。

Extended Euclid 的一般回推 (續)

Euclid：每個餘數都保留原輸入的整數線性組合

餘數 r	252 的係數 x	198 的係數 y	驗算 252x+198y
252	1	0	252+(0)=252
198	0	1	0+(198)=198
54	1	-1	252+(-198)=54
36	-3	4	-756+(792)=36
18	4	-5	1008+(-990)=18

商 1,3,1,2；最後非零餘數 18 = 4×252 − 5×198

31.3 Modular arithmetic

群G的運算須封閉、有identity、結合律、每元素有inverse。交換群另要求可交換；逆唯一，因若 ab=ac=e，則b=b(ac)=(ba)c=c。

(ℤ_n,+)是有限交換群，identity0、a的逆為−a；(ℤ_n,×)通常不是群，因0無乘法逆。其units U(n)= {a∈[0,n−1]:gcd(a,n)=1}才是乘法群，n≥2。

由Bezout，gcd(a,n)=1⇒au+nv=1⇒u是模逆；反向若au≡1，任何公因數整除au−nv=1，只能為1。Unit 乘積仍可逆，逆為$b^{-1}a^{-1}$，封閉性由此成立。模n的「除以a」只在a為unit時有唯一意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Lagrange 不是只背一個整除結論

子群H⊑G包含identity、封閉且有逆。左coset gH有|H|個元素，因乘g是雙射。若gH與kH有共同元素 $gh_1=kh_2$，則$k^{-1}g=h_2h_1^{-1}\in H$，兩cosets相同；否則不交。Cosets分割有限G，故|G|是|H|的整數倍，真子群大小≤|G|/2。

有限群的非空乘法封閉子集也自動是子群：任取a，有限性使a^i=a^j，i<j，乘群中的a^(−i)得a^(j−i)=e在子集；a^(j−i−1)給a的逆（若j−i=1，a=e，其逆即e）。

元素a的order是最小正t使a^t=e；$a^0,\cdots,a^{(t-1)}$兩兩不同，所有powers按模t重複，所以生成子群大小 =t。Lagrange給t||G|，進而a^|G|=e。有限性不可刪，否則整數加法群有無限order元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-st

Totient 的計數與真正的消去

$\phi(n)=|U(n)|$。對p質數，p^α中不互質的代表恰為p的倍數，共$p^{(\alpha-1)}$個，所以$\phi(p^\alpha)=p^{(\alpha-1)}(p-1)$。稍後CRT證兩兩互質乘積的unit集合為各unit集合的Cartesian product，故$\phi(n)=\prod p_i^{(\alpha_i-1)}(p_i-1)$。

例n=15，U(15)={1,2,4,7,8,11,13,14}，$\phi=8$。$7\cdot 13=91\equiv 1$，所以4/7代表$4\cdot 13\equiv 7$，不是整數商0。

加法群$\mathbb{Z}_8$中2生成{0,2,4,6}、order4；乘法群U(7)中2的powers為1,2,4，order3，而3的powers為1,3,2,6,4,5，order6。運算不同時，相同記號〈a〉不是同一組數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.4 Solving modular linear equations

解$ax\equiv b \pmod n$，$n\geq 1$。令$g=\gcd(a,n)$，必要條件$g|b$，因$ax+ny=b$的左側都被g整除。Extended Euclid給$au+nv=g$；若$b=gh$，則$x_0=uh$是一個解，證充分。

把式子除g，得$(a/g)x\equiv b/g \pmod{n/g}$，前係數與新模互質，故唯一解$x_0$模$n/g$。若兩解相減，$n/g$整除$(a/g)(x-x_0)$，互質消去推出$n/g|(x-x_0)$。所以模n的全部g個不同解為

$x=x_0+k(n/g) \bmod n \cdot k=0,\ldots,g-1$。

這給解的存在、個數、完整性與實際演算法。a=0時g=n：若n|b，全部n個代表都是解；否則無解。n=1只有代表0，不在此模數上談一般unit除法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方程例題與輸出成本

$18x \equiv 30 \pmod{42}$，$g=6$。Euclid給$6=-2\cdot 18+1\cdot 42$，$x_0=-2\cdot 5=-10\equiv 4 \pmod 7$。全部解為$4,11,18,25,32,39$；例如$18\cdot 4=72\equiv 30$，增加7使乘積增$126=3\cdot 42$，因此全數成立。

若右邊改31，$6\nmid 31$，無解。不能強行寫$18^{-1} \bmod 42$，因它根本不存在。若只回傳解集的壓縮形式$(x_0,n/g,g)$，Euclid後不必枚舉；若要列全g項，就至少有$\Omega(g)$輸出，不能只報$O(\log n)$時間。

先把$a \bmod n$歸一化，再求gcd，遞除$O(\log n)$，加g個輸出共$O(\log n+g)$算術操作；每項仍有$O(\log n)$位元，輸出bit數也須計入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.5 The Chinese remainder theorem

模數$m_1,\cdots,m_k \geq 2$兩兩互質，$M=\prod m_i$，$M_i=M/m_i$，$z_i=M_i^{-1} \bmod m_i$。定義$e_i=M_i z_i$、$x=\Sigma a_i e_i \bmod M$。

對固定j，$i\neq j$時$m_j \mid M_i$，故$e_i\equiv 0$；$i=j$時$e_j\equiv 1$。因此$x\equiv a_j \pmod{m_j}$，證存在。若x,x'皆滿足，各m_i都整除差；兩兩互質配合互質消去，逐一得到乘積M整除差，證模M唯一。

映射$x\mapsto(x \bmod m_1,\cdots,x \bmod m_k)$因此是雙射，且對加、乘保留運算：整數運算先後取餘等價。Unit也恰對應每個分量都是unit，否則至少與一個質因數不互質；所以totient對互質乘積可乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT 的三個單位座標實算

解$x\equiv 2 \pmod 3$、$x\equiv 4 \pmod 5$、$x\equiv 1 \pmod 7$。$M=105$，$M_i=(35,21,15)$，各inverse=(2,1,1)，所以e=(70,21,15)。

$x=2\cdot 70+4\cdot 21+1\cdot 15=239\equiv 29 \pmod{105}$。

檢查29 mod3=2、mod5=4、mod7=1。e_1的分量是(1,0,0)，e_2=(0,1,0)，e_3=(0,0,1)，這些是環的分量選擇器；它們也滿足$e_i^2=e_i$、$i\neq j$時$e_i\, e_j=0$、$\Sigma e_i=1$，均在mod M下成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT 的三個單位座標實算 (續)

CRT selectors：各 e_i 只選中一個同餘分量

e1 = 70	e2 = 21	e3 = 15
mod (3,5,7)	mod (3,5,7)	mod (3,5,7)
(1,0,0)	(0,1,0)	(0,0,1)
a1·e1 = 2×70 = 140	a2·e2 = 4×21 = 84	a3·e3 = 1×15 = 15

$140 + 84 + 15 = 239 \equiv 29 \pmod{105}$

29 → (2,4,1)；重建與取餘互為逆映射

運算的中間整數不能越界截斷；M的bit長是$\Sigma \log m_i$，因此可用大整數與逐步取餘，不能把M的數值大小當成位元記憶體成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非互質系統的相容條件

兩式x≡a(mod m)、x≡b(mod n)，令g=gcd(m,n)。寫x=a+mk，代第二式得到mk≡b−a(mod n)。由前節，恰在g|(b−a)時可解；解的周期為lcm(m,n)=mn/g，而非mn。

例x≡2(mod6)、x≡5(mod9)，g=3、差3可整除。6k≡3(mod9)化2k≡1(mod3)，得k≡2，x=14 mod18。若第二餘數改4，差2不被3整除，無解。

逐次合併可解一般系統；每一步都要檢查g條件並更新lcm。不能把pairwise-coprime的inverse M_i^{-1}公式直接套在m=6,n=9，因這些inverse不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.6 Powers of an element

對a∈U(n)，Lagrange給Euler定理a^φ(n)≡1(mod n)；n=p質數時φ=p−1，得Fermat定理。對所有a mod p，a^p≡a：a=0另直接成立。

若g的order為φ(n)，它生成全部units，稱primitive root。此時每個a=g^k，k模φ(n)唯一，稱discrete logarithm ind_(n,g)(a)。g^x=g^y當且僅當g^(x−y)=1，依order恰等價x≡y(modφ(n))。

例mod7以g=3，指數k=0,1,2,3,4,5對應a=1,3,2,6,4,5。所以ind(4)=4。若改g=2，只有1,2,4，對a=3沒有此base的log，不能對非generator保證所有unit都有表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何有限體的乘法群有 Generator

在有限交換群中，若元素order為r,s且互質，乘積order為rs：若(ab)^k=1，則a^k=b^(−k)同時位於兩個互質大小子群，交集只有identity，故r|k且s|k；反向rs次方為1。

對F_p的非零乘法群，把各元素order的lcm記L。每個質數ℓ在L中的最高冪ℓ^v由某個元素order提供；適當取該元素的power，得到order恰ℓ^v。把各質數分量相乘，用前段得order L的元素。

所有p−1個非零元素都是多項式X^L−1的根；體上非零degree L多項式至多L個根（用每個根因式X−a連除歸納）。所以p−1≤L。另一方面每個order整除p−1，由Lagrange得L≤p−1，故L=p−1，generator存在。這不是把任意有限群都說成cyclic。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

奇質數冪的 Generator 也能建構

取mod p的generator g，p為奇質數。比較g與g+p：模p^2有$(g+p)^{(p-1)}\equiv g^{(p-1)}+(p-1)g^{(p-2)}p$，後項除p後非0 mod p。因此至少可選h∈{g,g+p}使h^(p−1)=1+cp且p∤c。

二項式展開給：若v_p(z−1)=j≥1，則對奇p有v_p(z^p−1)=j+1。首項為p(z−1)，其餘項至少多一個p因子（末項也因p≥3而足夠），不會取消首項。因此v_p(h^((p−1)p^k)−1)=k+1。

mod p^α的order需被p−1整除，又整除φ=p^(α−1)(p−1)；上面的valuation排除少一個p因子，故恰φ，證U(p^α) cyclic。這也說明不能任選mod p的generator就不檢查地當mod p^2的generator。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些模數的 Unit Group 是 Cyclic

n>1時恰為2、4、p^α、2p^α，其中p為奇質數。前頁證odd prime power；U(2p^α)由CRT同構U(2)×U(p^α)，第一分量只有1，故仍cyclic；U(2)、U(4)直接可列。

排除其他n：若8|n，U(n)投影到U(8)為滿射，而U(8)每個元素平方都是1、大小4，非cyclic；cyclic群的homomorphic image必cyclic，矛盾。滿射可由odd prime-power的CRT分量及任意odd residue mod2^α得到。

若n含至少兩個odd prime factors，CRT讓每個奇質數冪分量獨立取±1，所以至少有4個平方為1的units；若4|n且另含odd prime，也由U(4)與odd分量給4根。Cyclic群中g^(2k)=1至多兩個k mod order，故這些n不cyclic。上述情況涵蓋所有未列型態。

非平凡平方根如何暴露合數

對odd prime power p^α，若$x^2\equiv 1$，則p^α|(x−1)(x+1)。兩因子的gcd只整除2，p為odd不可能同時分兩側，所以整個p^α只能整除其中一側，得$x\equiv \pm 1$。

若mod n發現$x^2\equiv 1$但$x\not\equiv \pm 1$，則n不可能質數；而gcd(x−1,n)、gcd(x+1,n)都是非平凡因數。它們都小於n，否則$x=\pm 1$；都大於1，否則對某側互質消去會迫使另一側被n整除。

例n=77，x=34：$34^2=1156\equiv 1$，$34\not\equiv 1,76$；gcd(33,77)=11、gcd(35,77)=7。CRT解讀是$x\equiv 1 \bmod 11$、$x\equiv -1 \bmod 7$，兩種符號分量沒有同步。

Square-and-multiply 的完整迴圈

輸入n≥1、b≥0，先a←a mod n，r←1 mod n。由最高位到最低位讀b的所有bits：

```
r=1 mod n
for bit in binary(b), highest first: # b=0可視為空串
    r=(r*r) mod n
    if bit==1: r=(r*a) mod n
return r
```

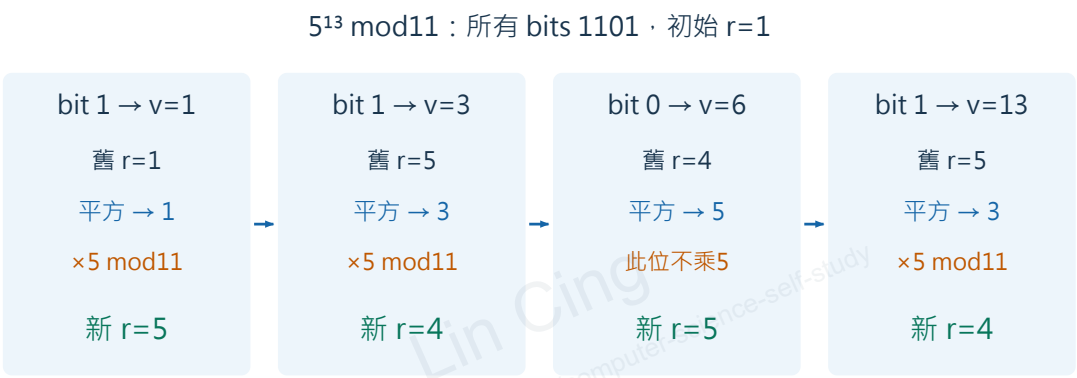
不變量：讀過的前綴值v時r=a^v mod n。下一bit β把v改2v+β，平方與可選乘a正好更新。空前綴v=0正確初始化；終態v=b得到目標。b=0返回1 mod n，包含n=1返回0；不要求gcd(a,n)=1。負指數則需要inverse，是另一輸入契約。

5^{13} mod11 的每一位

13的binary是1101，r從1掃全部四位：

bit	新前綴v	舊r	平方mod11	乘5後／保留
1	1	1	1	5
1	3	5	3	4
0	6	4	5	5
1	13	5	3	4

5^{13} mod11 的每一位 （續）



若r=a代表已讀最高1，只能再掃101；r=a卻掃完整1101會走指數3,7,14,29、模值4,3,9,9，得到錯的9。k位指數至多2k次模乘；若n有β位，普通模乘$O(\beta^2)$bit，總$O(k\beta^2)$，當$k=O(\beta)$為$O(\beta^3)$。另需計首次歸一化超長a的成本。

31.7 The RSA public-key cryptosystem

選相異odd質數p,q，n=pq，$\phi=(p-1)(q-1)$；選$1<e<\phi$且$\gcd(e,\phi)=1$，以Extended Euclid求$d=e^{-1} \bmod \phi$，取$1\le d<\phi$。公開(n,e)，保留d與p,q。對$0\le m<n$，數學轉換$c=m^e \bmod n$、$m'=c^d \bmod n$。

對任意m（不限unit）證$m^{ed}\equiv m$：$ed=1+k(p-1)(q-1)$。mod p若$p\mid m$兩側0；否則Fermat使$m^{p-1}=1$，得$m^{ed}=m$；mod q同理。CRT唯一性給mod n相等。若只用Euler定理會漏掉與n不互質的合法m。

同樣冪運算反向亦還原，所以私鑰冪／公鑰冪構成互逆的數學置換。這個正確性不等於密碼協定的安全性。

非 Unit 訊息也要能解密

取p=11,q=17，n=187、φ=160，e=7；1=23・7−1・160，故d=23。訊息m=11不是unit，仍合法。

加密平方鏈$11^2\equiv121$、$11^4\equiv55$，所以$11^7\equiv55\cdot121\cdot11\equiv88\pmod{187}$。解密$88^2\equiv77$、$88^4\equiv132$、$88^8\equiv33$、$88^{16}\equiv154$；23=16+4+2+1，乘四個對應值：154・132≡132，132・77≡66，66・88≡11。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非 Unit 訊息也要能解密 （續）

n=187 · e=7 · d=23；m=11 不是 unit

mod11 分量

11→0；88→0；$0^{23}\to0$
p | m：此分量直接還原0

mod17 分量

11→11；88→3；$3^{23}\to11$
此分量是 unit · 使用 Fermat

CRT：(0 mod11 · 11 mod17) 唯一重建 11 mod187

若e為固定bit長，公鑰操作$O(\beta^2)$普通bit成本；d有$O(\beta)$位時私鑰$O(\beta^3)$。由n分解出p,q可求φ,d；不能逆稱「RSA逆轉與整數分解已被證明等價」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Textbook RSA 的代數限制

無編碼的RSA是deterministic：相同m總得相同c，可列舉小訊息域。又$E(m_1)E(m_2)\equiv E(m_1m_2)$，因此若c=E(m)，選r∈U(n)，c・r^e對應mr；這是可操作性，不是安全證明。

直接用m^d作「簽章」也不具完整安全性：任取σ再設m=σ^e便有一組驗證會過的(message,signature)，卻非授權簽出的語意訊息。實用方案需另訂隨機化編碼、訊息雜湊與驗證格式；本章只推導基礎置換，不把toy參數或裸冪當部署規格。

這些弱點由明確代數式推出，與「大質數分解很難」可以同時成立。不可用數學解密正確性取代攻擊模型與安全定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.8 Primality testing

Trial division只需試到⌊√n⌋：若n=ab合數且兩者都>√n會矛盾；若沒有小因數便質數。但對β位n，√n≈2^(β/2)，不是多項式bit時間。

質數定理給π(X)~X/ln X，故在大致同量級區間均勻取數，質數密度約1/ln X，只取odd約2/ln X。這是漸近估計，不是每個候選必在ln X次內找到質數的界，也不是下述錯誤率證明。

Fermat測試若a^(n−1)≠1，可確定合數；若等於1只叫probable prime。341=11・31且$2^{10}=1024\equiv 1 \bmod 341$，所以$2^{340}=1$仍合數。Carmichael數甚至讓所有units通過：561=3・11・17，各p−1=2,10,16都整除560，CRT與Fermat給全部units的560次方為1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller-Rabin 的完整基例與平方鏈

先處理n<2為非質數；n=2,3為質數；其餘偶數為合數。只對odd n≥5寫n−1=2^s d、d odd、s≥1，均勻取整數a∈[2,n−2]。

```
x=modular_power(a,d,n)
if x==1 or x==n-1: pass this round
else:
    repeat s-1 times:
        x=x*x mod n
        if x==n-1: pass this round
    if no -1 occurred: return COMPOSITE
```

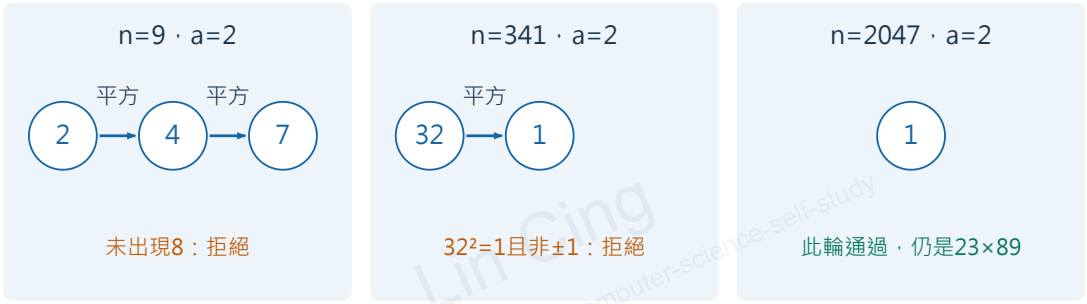
重做r個獨立基底，皆通過才回probably prime。質數n的非零a滿足a^(n−1)=1；鏈若非從1起，第一次平方到1前的值只能是−1，因體中平方為1只有±1。因此質數必通過，每個被拒者都確實是合數。n=3若未特判會抽空區間[2,1]。

同樣通過 Fermat，強測試能再分辨

n,a	分解n−1	$x_0=a^d$	後續平方	結論
5,2	$4=2^2\cdot 1$	2	4=−1	通過
9,2	$8=2^3\cdot 1$	2	4,7	拒絕，未見−1
341,2	$340=2^2\cdot 85$	32	1	拒絕，32非平凡平方根
2047,2	$2046=2\cdot 1023$	1	不需再平方	本輪通過，仍是23・89

同樣通過 Fermat，強測試能再分辨 （續）

Miller–Rabin：必須在同一條鏈正確判讀 −1



通過只是 probable prime；固定同一 base 反覆測不能放大可信度

2047例說明單個固定base不能證一般整數為質數。錯誤方向是合數被接受，質數不會被拒；不要用未定義positive/negative顛倒這個事件。以下針對每個固定合數證random-base上界。

計算 Strong Liars 先數 Prime-power Roots

固定odd合數n=∏_(i=1)^r p_i^α_i，n−1=2^s d。令p_i−1=2^t_i u_i（u_i odd），D_i=gcd(d,u_i)，τ=min(s,t₁,⋯,t_r)。通過的a必是unit，因若某p_i|a，其所有正冪在此分量都0，不可能成±1。

F_(p_i)^×有generator，寫a=g^k；`a^d=1`有D_i個解；`a^(2^j d)=−1`轉成指數同餘，恰在j<t_i時有2^j D_i個解。理由是係數gcd=2^min(j,t_i)D_i，能整除(p_i−1)/2當且僅當j<t_i。

這些根各唯一提升到p_i^α_i：若z已解X^h−η≡0(mod p_i^a)，試z+c p_i^a，下一級只需解`(z^h−η)/p_i^a+h z^(h−1)c≡0(mod p_i)`。h=2^j d不被p_i整除（p_i∤n−1），z是unit，導數非零，因此c唯一。故同樣根數在每個prime-power分量成立。

CRT 合併根數，而不是猜一個子群

通過情形互不相交：a^d=1，或某個唯一的j∈[0,s−1]使a^(2^j d)=−1；不同j不可能都為−1，因先平方一次就成1。CRT使各分量獨立，但「同一j、全部−1」必同步。

所以全部a∈[1,n−1]的strong liars個數恰為

$$L = \left(1 + \sum_{j=0}^{\tau-1} 2^{jr}\right) \prod_i D_i.$$

第一項是全部分量的a^d=1，j項是全部分量同時在第j層為−1。不能把每個分量「各自某層為−1」也算進來，那會多算不同步而應拒絕的a。此公式只用質因數分解做**分析**，測試程式不需事先分解n。

1/4 界：一個或至少三個質因數

r=1時n=p^α、α≥2。p−1|p^α−1，故τ=t、D=u，公式得L=p−1。因此`L/(n−1)=(p−1)/(p^α−1)≤1/(p+1)≤1/4`。

r≥3時φ(n)=∏p_i^(α_i−1)2^t_i u_i，且D_i≤u_i、t_i≥τ，所以

`L/φ(n) ≤ [1+Σ_(j=0)^(τ−1)2^(jr)]/2^(rτ)`。

等比和可寫`1/(2^r−1)+(2^r−2)/[(2^r−1)2^(rτ)]`，隨τ≥1遞減，最大在τ=1為2^(1−r)≤1/4。φ(n)≤n−1，得到L≤(n−1)/4。n=9達L=2=(9−1)/4，所以此全代表域的常數不能普遍再縮。

1/4 界：兩個質因數的必要分支

r=2時若$t_1 \neq t_2$，或$s<\min(t_1,t_2)$，則$t_1+t_2 \geq 2\tau+1$。由公式

$L/\phi(n) \leq [1+\Sigma 4^j]/2^{(2\tau+1)}=(4^\tau+2)/(6\cdot 4^\tau) \leq 1/4$。

剩下$t_1=t_2=\tau \leq s$。如果某$\alpha_i \geq 2$，ϕ的prime-power額外因子≥ 3；如果某$D_i<u_i$，因u_i/D_i為odd整數也至少3。至少出現其中一種：否則n=pq squarefree且$D_i=u_i$，會有$p-1|n-1$、$q-1|n-1$；由$pq-1$模$p-1$得到$p-1|q-1$，反向同理，迫使p=q，矛盾。

因此額外除以至少3，$L/\phi(n) \leq (4^\tau+2)/(9\cdot 4^\tau) \leq 1/6<1/4$。所有odd合數情形已覆蓋，一般1/4界不再依賴未指定的最大位階或Carmichael分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

抽樣分母、重複次數與條件機率

1與n−1永遠通過。由全域$L \leq (n-1)/4$，排除它們後，[2,n−2]的比例為$(L-2)/(n-3) \leq 1/4$。因此對**固定合數n**，獨立均勻抽r次、全通過機率$\leq 4^{(-r)}$；不是「看到通過後，n為合數的機率」直接等於此值。

若候選的先驗質數率$\pi>0$，質數通過率1，合數接受率$\leq \varepsilon=4^{(-r)}$，Bayes給$Pr(\text{composite} \mid \text{pass}) \leq \varepsilon(1-\pi)/[\pi+\varepsilon(1-\pi)]$。須有候選分布才能使用π；不能替攻擊者挑的固定輸入假設均勻質數密度。

每輪$O(\log n)$次模乘，普通bit界$O(\beta^3)$，r輪$O(r\beta^3)$，另含抽樣成本。固定base清單只有在其適用輸入上限已證明時才可叫deterministic。先隨機抽一個base再反覆沿用，不屬獨立抽樣；每輪IID有放回抽樣則可以偶然抽中相同值，仍符合契約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.9 Integer factorization

判合數不等於得到分解。Trial division可確定找因數，但最壞$\sqrt{n}$次；Pollard rho利用「mod某因數先碰撞」，不需知道該因數。

取整數多項式$f(x)=x^2+c \bmod n$，若$p|n$，則$f(x) \bmod p$等於對$x \bmod p$計同一多項式。只要$x_i \equiv x_j \bmod p$，便有$p|(x_i-x_j)$，所以$d=\gcd(|x_i-x_j|,n)>1$；若$x_i \not\equiv x_j \bmod n$，$d<n$，得到真因數。

這一步的正確性完全確定，且回傳後可直接檢查$1<d<n$與$d|n$。難點是何時碰撞、是否同時在所有分量碰撞；後者會得到$d=n$，不能把它當成功分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd 與 Checkpoint 兩種找碰撞方法

```
x=y=x0
repeat until work budget exhausted:
    x=f(x); y=f(f(y))
    d=gcd(abs(x-y),n)
    if 1<d<n: return d
    if d==n: restart with another x0/c
return no factor found within budget
```

Floyd每輪慢指標走1、快指標走2。對有限函數圖，一段tail後進cycle；兩者都進cycle後，相對位置每輪差1，所以至多cycle長λ輪相遇，總$O(\mu+\lambda)$次函數計算，$O(1)$個整數儲存。

另一checkpoint版固定$y=x_{(2^k)}$，讓x繼續走到下個$2^{(k+1)}$；當checkpoint已進cycle且區間長$\geq\lambda$，必在區間內遇到同分量。這是另一排程，不與Floyd的i、j trace混用。兩者都需處理$d=n$與工作上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8051 的三輪因數追蹤

取n=8051、x_0=2、$f(x)=x^2+1$ mod8051，使用Floyd。

輪	x=f(x)	y=f(f(y))	差的絕對值	gcd(diff,8051)
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

第3輪677 mod97=95、871 mod97=95，先在97分量碰撞；mod83則13與41尚不同。因此gcd不是8051而是97，除得8051/97=83，兩者經trial division至其平方根皆質數，完整分解8051=83・97。

8051 的三輪因數追蹤 （續）

Floyd 第3輪：x=677，y=871，差194

mod97：先碰撞

mod83：尚未碰撞

677 mod97	871 mod97	677 mod83	871 mod83
95	95	13	41

97 | 194，但83 ∤ 194，所以 gcd(194,8051)=97

驗算：8051 = 83×97；回傳 proper divisor 才算找到因數

Birthday 估計的假設與實際限制

若mod未知質因數p的迭代可近似均勻random mapping，首次碰撞前逐步探索新點，k次仍未碰撞的機率約$\prod_{j=0}^{k-1}(1-j/p)\le\exp(-k(k-1)/(2p))$，所以典型碰撞尺度$\sqrt{p}$；tail/cycle方法在同階步數內檢出。

固定x^2+c不是獨立亂數，這個假設不提供它的最壞時間或每次restart成功率。對平衡semiprime $p\approx\sqrt{n}$，heuristic尺度$n^{1/4}=2^{\beta/4}$，仍非β的多項式。每輪另有β位模平方與gcd成本；依本章安全粗gcd界可報$O(\beta^3)$，不能把一次gcd默認O(1)。

回傳真因數後還要遞迴分解兩側，先排偶數、確認質數或probable-prime的可信條件；若預算用完，只能說尚未找到因數，不能改判質數。Rho的因數證書是確定的，完成時間保證則與該heuristic分析分開。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 31。

M. O. Rabin, "Probabilistic Algorithm for Testing Primality," *Journal of Number Theory* 12(1), 1980, doi:10.1016/0022-314X(80)90084-0.

L. Monier, "Evaluation and Comparison of Two Efficient Probabilistic Primality Testing Algorithms," *Theoretical Computer Science* 12(1), 1980, doi:10.1016/0304-3975(80)90007-9.

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32 字串匹配

Part VII Selected Topics

本章定位

字串匹配把已知相等的前後綴轉成可重用狀態；本章從逐字比較與模數碰撞，推導有限自動機與KMP的失敗鏈、正確性及線性總成本。

前置知識

模式前綴、雜湊與有限自動機。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 valid shifts 與 prefix 問題。
- 比較 naive、Rabin-Karp、automaton 與 KMP。
- 證明 KMP 前綴函數與線性總時間。
- 32.1 The naive string-matching algorithm
- 32.2 The Rabin-Karp algorithm
- 32.3 String matching with finite automata
- 32.4 The Knuth-Morris-Pratt algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.1 The naive string-matching algorithm

文字（text）T 的長度為 n，模式（pattern）P 的長度為 m；字元來自有限 alphabet Σ，相等比較採 O(1)。本章字元位置一律從1開始，shift s 從0開始：合法位移滿足 0≤s≤n−m 且對每個 j=1,⋯,m 都有 P[j]=T[s+j]。輸出所有位移，允許重疊。

P_k 表示 P[1..k]，P_0=ε 為空字串。Prefix 是前綴，suffix 是後綴；proper prefix/suffix 不得等於整個字串。Border 指同時為前綴與後綴的字串。

若 m=0，規格定為輸出0,⋯,n這 n+1 個邊界；若 m>n，輸出空集合。後續主演算法先處理這兩種情形，才假設1≤m≤n，避免讀取不存在的 P[1]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐字比較：不漏位移也不接受錯字

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

內圈每次檢查前，P[1..j−1]=T[s+1..s+j−1]。相等才延伸不變量；不等就知道此位移不合法；j=m+1 表示全部相等。外圈恰枚舉所有合法範圍，因此聲稱的每個匹配正確，真正匹配也不會漏掉。

共有 n−m+1 個窗口，每個至多 m 次比較，最壞 Θ((n−m+1)m)。取 T=a^n、P=a^m，每個窗口都用滿 m 次，證明此界可達；即使沒有碰撞，也可能有很多真正匹配。額外工作空間 O(1)，不含輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一文字的所有窗口與比較次數

T=abababaca、P=aba。遇到第一個不相等就停止：

s	窗口	依序比較	次數	結果
0	aba	a=a，b=b，a=a	3	輸出0
1	bab	a≠b	1	拒絕
2	aba	a=a，b=b，a=a	3	輸出2
3	bac	a≠b	1	拒絕
4	aca	a=a，b≠c	2	拒絕

共10次比較。s=0與s=2共用文字位置3，仍是兩個合法匹配。Naive 重新檢查同一字元對齊；KMP 將利用已知相等的區段排除不可能的位移。

32.2 The Rabin-Karp algorithm

將字元單射編為0,…,d−1，以 radix d 表示長 m 字串。其整數值 H(X)=Σ_(j=1)^m X[j]d^(m−j)，但 m 位數的大整數運算不能當 O(1)。因此保留 h_q(X)=H(X) mod q，q≥2，並在 hash 相等時逐字驗證。

令 p=h_q(P)、t_s=h_q(T[s+1..s+m])、h=d^(m−1) mod q。由移除最高位、左移一位、加入新低位，得到

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

此恆等式對任意 q≥2 成立，不必是質數；選質數有利於某些隨機化分析，卻不自動保證固定字串的碰撞率為1/q。實作須將負餘數正規化為0,…,q−1，並讓乘法中間值不溢位。

Hash 只排除，字元驗證才接受

```
h=1; p=0; t=0
for j=1..m-1: h=(d*h) mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner 法與 rolling 恆等式歸納保證 t 始終是當前窗口 hash。真正相等的字串必有相同 hash，所以不會漏報；hash 不同可拒絕，hash 相同再比較原字串，所以不會誤報。碰撞影響時間，不影響這個驗證版本的正確性。

若省略原字串驗證，則變成可能誤報的演算法，不能沿用「總是正確」的結論。

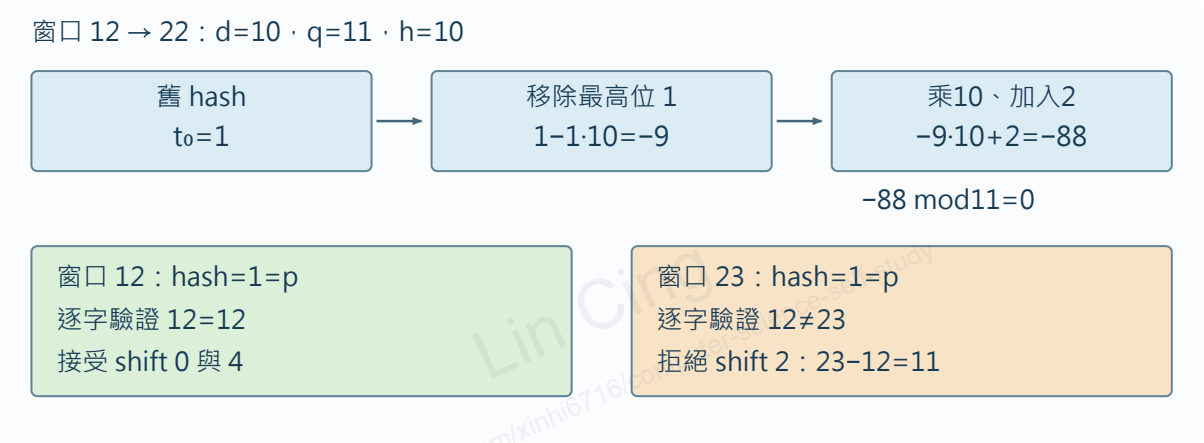
一次真正匹配、一次碰撞、再一次匹配

取 T=122312、P=12、d=10、q=11。h=10，p=12 mod11=1：

s	窗口	t_s	計算下一個 hash	字元驗證
0	12	1	(10(1−1・10)+2) mod11=0	12=12，輸出0
1	22	0	(10(0−2・10)+3) mod11=1	hash不同，略過
2	23	1	(10(1−2・10)+1) mod11=9	12≠23，拒絕碰撞
3	31	9	(10(9−3・10)+2) mod11=1	hash不同，略過
4	12	1	最後窗口，不再讀新字元	12=12，輸出4

Spurious hit（假命中）是 hash 相等而字串不同；此例23−12=11，正好被模數消掉。即使 q 是質數，碰撞仍可發生。

一次真正匹配、一次碰撞、再一次匹配 （續）



成本取決於真匹配與假命中

採 word-RAM：字元、q、所有模乘中間值能在 O(1) word 操作完成，或提供 O(1) 模乘。若直接算 d(t−out・h)，也必保證其中間值可容納；可先把差正規化後再乘 d，使中間值少於 dq。任意精度版本須另乘位元運算成本。

預處理 Θ(m)，rolling 掃描 Θ(n−m+1)。設真匹配數 z、假命中數 f，每次驗證至多 m 字元，總時間 $O(n+m+m(z+f))$。

最壞 z 或 f 可達 Θ(n−m+1)，所以最壞仍 Θ((n−m+1)m)。例如全 a 文字與全 a 模式只有真命中，照樣反覆驗證；不能把退化全歸因於選錯 hash。輸出匹配本身要 Ω(z)，本版本另外花 m 次字元驗證每個真命中。

有證明的隨機 Hash：明寫亂數與條件

另一種可分析的版本：固定質數 q>d−1，字元編碼不同即在域 F_q 不同；在輸入固定後，獨立均勻選 base b∈F_q，以 ΣX[j]b^(m−j) mod q 作 hash。Rolling 公式以 b 取代 d，b=0也依恆等式處理，m=1時 h=b^0=1。

對固定相異字串 X,Y，差多項式 D(b) 非零、次數至多 m−1。非零域上多項式至多有其次數個根：若 r 是根，因式定理提出(b−r)，再對次數歸納。因此

$Pr[h_b(X)=h_b(Y)]\le \min(1,(m-1)/q)$。

對每個非匹配窗口的碰撞指示變數取期望再相加，不需要窗口事件獨立，得到 E[f]≤(n−m+1−z)(m−1)/q。故期望時間為 O(n+m+mz+mnm/q) 的寬鬆界；選 q 足夠大才有低假命中成本，真匹配的 mz 仍在。q 的位元長度與模乘成本也必計入，不能任意增大後仍無條件稱 O(1)。

32.3 String matching with finite automata

有限自動機由(Q,q_0,A,Σ,δ)組成：有限狀態集合、初態、接受狀態集、字母表及每個狀態／字元對的轉移函數。匹配器令 Q={0,⋯,m}、q_0=0、A={m}。

定義 σ(x)=max{k≤m：P_k 是 x 的後綴}。空字串總是後綴，所以最大值存在。狀態 q=σ(目前已讀文字)，意義是「現在可延伸的最長匹配前綴」，不是已經找到幾次完整匹配。

令 δ(q,a)=σ(P_q a)。讀入一個字元後做一次轉移；當狀態等於 m 就輸出目前結尾所決定的位移。接受狀態並非停止狀態，仍有對每個 a 的轉移，才能找到重疊匹配。

為何只記最長前綴長度就夠

要證明 δ(σ(x),a)=σ(xa)。令 q=σ(x)、r=σ(xa)。若 r>0，把匹配的最後一字移去，P_(r−1) 是 x 的後綴，因此 r−1≤q。P_q 也是 x 的後綴；兩個後綴按長度包含，故 P_(r−1) 也是 P_q 的後綴，再接 a，P_r 就是 P_q a 的後綴。因此 r≤σ(P_q a)。r=0時此不等式也成立。

反過來，P_q a 本身是 xa 的後綴，任何同時為 P 前綴的 P_q a 後綴也必為 xa 後綴，故 σ(P_q a)≤σ(xa)=r。兩向合併得等式。

由初態 σ(ε)=0歸納，每次轉移後 q=σ(T[1..i])。q=m 當且僅當 P 是 T[1..i] 的後綴，恰好等價於 shift i−m 匹配；這證明接受判斷的必要性與充分性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配

P=abab，Σ={a,b,c}，每格是 δ(q,字元)：

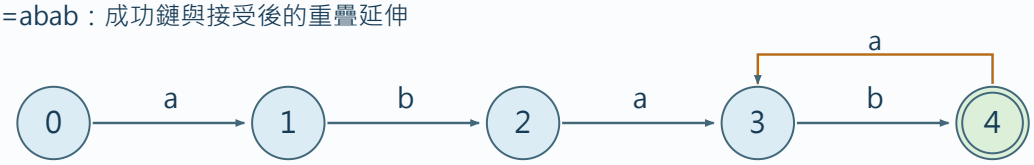
q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

T=ababab 的狀態依序1,2,3,4,3,4，在 i=4與6輸出 shift0與2。δ(4,a)=3，因 ababa 的最長可用後綴是 aba。若匹配後無條件清零，就會錯過第二次匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配 （續）

P=abab：成功鏈與接受後的重疊延伸



完整轉移表見正文；此圖只畫成功鏈與 δ(4,a)=3。

T[i]	a	b	a	b	a	b
狀態	1	2	3	4	3	4

讀到第4與第6字接受：shift 0、2；完整匹配之後不清零。

建表成本不能混入掃描成本

直接按 δ 的定義，對每個 q,a 從 min(m,q+1) 往下試 k，逐字驗證 P_k 是否為 P_q a 的後綴，可花 $O(m^2)$ 每格，總 $O(m^3|\Sigma|)$。這是正確但慢的建法；掃描仍是 Θ(n)。

有了下一節的 prefix function π，可按 q 遞增建表：

```
for a in Σ: δ(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Σ:
        if q<m and a==P[q+1]: δ(q,a)=q+1
        else: δ(q,a)=δ(π[q],a)
```

Mismatch 時，下一個候選只能是 P_q 的 proper border；最長的長度為 π[q]。所有較短候選都在它的後綴內，所以同一字元轉移可重用其表格。π[q]<q使依賴格已算完。建 π 要 Θ(m)，建表 $\Theta(m|\Sigma|)$ 時間與空間；只有在可 O(1) 索引 alphabet 的模型下，每字元查表才是 O(1)。

32.4 The Knuth-Morris-Pratt algorithm

Prefix function（前綴函數）定為 π[q]=max{k<q：P_k 是 P_q 的後綴}，q=1,⋯,m；因此 π[1]=0且π[q]<q。它只依模式，與文字無關。

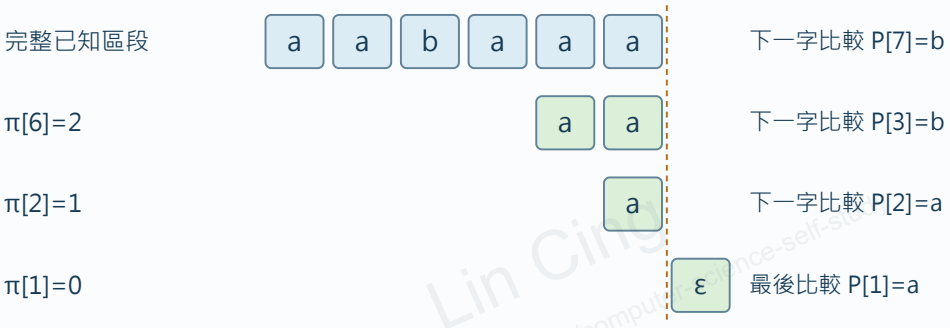
對 P=aabaaab，π=[0,1,0,1,2,2,3]。P_6=aabaaa 的 proper borders 是 aa、a、ε，長度鏈為 6→2→1→0。6本身只是搜尋的起點，不算 proper border。

KMP不儲存整張 δ 表。下一字元無法延伸時，就沿 π 鏈找下一個可能延伸的前綴。每次減少候選長度，同一個新文字字元保持不動，已讀過的文字不重讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.4 The Knuth-Morris-Pratt algorithm （續）

P=aabaaab，已知文字後綴 aabaaa (q=6)



新字元是 c：6→2→1→0，所有比較皆失敗；同一個 c 尚未向前移動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

失敗鏈列出了全部候選，不會跳過可行長度

令 B(q) 為 P_q 的所有 proper border 長度。若 b=π[q]>0，任何 k<b 都有

$k \in B(q) \Leftrightarrow k \in B(b)$。

證明：P_b 是 P_q 的後綴，因此它的長 k 後綴就是 P_q 的長 k 後綴；而 P_k 是兩者共用的前綴。任一方向的前後綴相等條件因此相同。b 是最長 proper border，不可能有 b<k<q 的候選。若 b=0，沒有正長度的 proper border。

所以 B(q)={b}∪B(b)，依嚴格遞減長度歸納，反覆 π[q]、π[π[q]]、⋯、0恰好列出所有 proper border。這是 fallback 安全的原因；不是因「短一點大概會匹配」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix Function 的逐項建構

```
π[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=π[k]
    if P[k+1]==P[i]: k++
    π[i]=k
```

在處理 i 前，k=π[i−1]。P_i 若有長 r>0 的 proper border，去掉最後字元後，長 r−1 必是 P_(i−1) 的 proper border；反之，只要該 border 之後的 P[r] 等於 P[i]，就能延長成長 r 的 border。

故由最大候選 π[i−1] 開始，沿完整失敗鏈遞減，第一個可延伸者產生最大合法 r；到0仍不相等則答案0。先前已算正確的 π 值只在較小索引被使用，歸納成立。因 π[i−1]≤i−2，這一步不會把整個 P_i 自身當 proper border。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模式的建表全軌跡

P=aabaaab：

i	P[i]	初始k	比較與fallback	π[i]
1	a	—	單字元無非空proper border	0
2	a	0	P[1]=a，延長	1
3	b	1	P[2]=a≠b；1→0；P[1]仍不符	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a；2→1；P[2]=a	2
7	b	2	P[3]=b	3

π[6]=2不是3：aabaaa的末三字aaa不等於前三字aab。建表不是在不相等時一律 k−−，而是只保留已知可行的 border。

KMP 掃描與匹配後的狀態

```
π=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=π[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=π[m]
```

在接受檢查時，q=σ(T[1..i])，與自動機相同。若未達 m，便保存此 q 供下輪使用；若達 m，先輸出，再保存 π[m]，即已讀文字中最長的 proper 模式前綴後綴。下一輪延伸新匹配只需要 proper border，因目前完整長 m 前綴不可能再增至 m+1。

Mismatch 的所有可行較短候選恰為失敗鏈，第一個能與 T[i] 延伸者就是 δ 的結果；若無候選則為0。這同時證明不漏任何匹配，以及讀取 P[q+1] 前永遠 q<m，不會越界。

完整匹配之後仍能接重疊匹配

P=aabaaab、π[7]=3，取 T=aabaaabaaab：

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7，輸出0	3
8	a	3	4	4
9	a	4	5	5

完整匹配之後仍能接重疊匹配 （續）

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
10	a	5	6	6
11	b	6	7，輸出4	3

兩次匹配共用位置5–7的 aab。清零會丟掉這段已知相等的證據。

一個字元可多次 fallback，總成本仍線性

狀態 q=6、下一字元 c 時，依序比較 P[7]=b、P[3]=b、P[2]=a、P[1]=a，都不等 c；鏈6→2→1→0，最後保存0。同一個 c 被比較多次，但文字索引 i 沒有倒退。

掃描中 q 初始0，每處理一字最多增加1，總增加量≤n。每次 while body 至少減1，輸出後的 q←π[m] 也只減不增；q 從不負。因此 while body 的總次數≤n，另加每字常數次外圈檢查及輸出，匹配 Θ(n)。相同記帳適用建 π：總增加≤m−1，while body總次數≤m−1，建表 Θ(m)。

總時間 Θ(n+m)，額外工作空間 Θ(m)，不含輸出。這是整段序列的界，不是說每個字元最壞 O(1)；若需每字固定最壞轉移時間，可改付完整自動機表的空間。

四種方法使用了哪些假設

方法	前處理	掃描最壞時間	模式額外空間
Naive	無	Θ((n−m+1)m)	O(1)
RK，hash相等後驗證	Θ(m)	Θ((n−m+1)m)	O(1)
完整自動機，π建表	Θ(m	Σ	)
KMP	Θ(m)	Θ(n)	Θ(m)

本表採1≤m≤n、固定成本字元及word模算術，預處理與掃描分開列。RK期望快需要額外亂數／碰撞條件；KMP線性不需要隨機輸入或隨機模式。Unicode實作要先選定匹配單位是byte、code point或grapheme cluster；本章不把不同編碼的字元序列自動視為相同字串。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 32。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性

Part VII Selected Topics

本章定位

NP完備性以位元編碼、量詞及可驗證證書界定難度；本章逐步建立核心歸約的構造、雙向語義與多項式成本，讓困難性證明可重建。

前置知識

決策問題、圖靈機、約化與布林電路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 P、NP、NP-hard、NP-complete。
- 建立 polynomial-time many-one reduction。
- 用 SAT、clique、vertex cover、Hamiltonian cycle 做證明。
- 34.1 Polynomial time
- 34.2 Polynomial-time verification
- 34.3 NP-completeness and reducibility
- 34.4 NP-completeness proofs
- 34.5 NP-complete problems

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.1 Polynomial time

決策問題、編碼與輸入長度

定義與前提

abstract problem 可把 instance 集合映到 answers。決策問題的答案只有 yes 與 no，因此可用語言 `L={x: answer(x)=yes}` 表示。

同一個數學物件可能有不同 encoding。例如整數 t 用 binary 表示約需 `lg(t+1)` 位，用 unary 表示需 t 個符號。多項式時間必須先指定 encoding，因為演算法的界限以實際輸入位元數衡量。

$$L \in \mathrm{P} \iff \exists \text{ deterministic TM } M,\ \exists \text{ polynomial } p : \forall x,\ M(x) \text{ halts in } p(|x|) \text{ and } (M(x) = 1 \iff x \in L).$$

`forall x` 不能寫成「存在某個 x」。P 的演算法必須對每個合格式處理，且錯格式也須在 polynomial time 拒絕或依規格處理。

P 的封閉性與 poly-time 組合

操作與推導

若程序 A 在 `p_A(n)` 時間產生可行 encoding，程序 B 在 `p_B(n)` 時間解第二個問題，先後執行成本為 `p_A(n)+p_B(p_A(n))`。多項式代入多項式仍為多項式。

這提供兩個常用結論。

1. 若 A 可在 polynomial time 直接求解，而 B 可把 instance 直接解算轉成 A，則 B 也可 polynomial-time 求解。
2. 多個 polynomial-time 轉換可串接，因為錯誤方向與長度爆炸都被 poly 界限控制。

但「演算法在某部真實電腦跑得快」不是成員證明。`O(n^100)` 仍屬 P，而常數很低的 exponential algorithm 可能在小 n 更實用。計算類別與工程效能是兩個問題。

bit input、數值參數與 pseudopolynomial

可追蹤數例

令 $\mathrm{bitlen}(t)$ 為 t 的 binary 表示位數。若演算法成本 $O(nt)$，而 $t < 2^{\mathrm{bitlen}(t)}$，成本相對於輸入位元數可能是指數。

考慮 subset-sum：輸入 `n` 個整數與目標 B，DP 表有 `n(B+1)` 格，因此 `O(nB)`。B binary 編碼時，`nB` 不是輸入長度的多項式；這種以數值上界為參數、而非以位元長度為參數的演算法稱為 pseudopolynomial algorithm。

可追蹤族。 令 $n = m$、$t = B = 2^m$，並令 $a_i = 2^{m-1} + i$、$i = 1, \dots, m$、$m \geq 2$，使每個 a_i 都有 $\Theta(m)$ bits。目標有 $m + 1$ bits，總輸入長度 $N = \Theta(m^2)$，但 DP 成本為

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

若它是 input-bit polynomial，對某個 c 與常數 C 最終應有 $m2^m \leq C(m^2)^c$；然而 $m2^m / m^{2c} \to \infty$，矛盾。

pseudopolynomial 不表示演算法無用。它表示「在 B 小或 unary encoding 下 polynomial」，不能因此宣稱 subset-sum 在 binary 標準編碼下屬於 P。

34.2 Polynomial-time verification

NP 的完整量詞

NP 類常被說成「答案可被快速檢查」。完整定義需要 certificate、長度上界與 deterministic verifier。

$$L \in \mathrm{NP} \iff \exists V,\ \exists\ \text{polynomials}\ p, q : \forall x,\quad x \in L \iff \exists y\ \big(|y| \leq q(|x|) \wedge V(x, y) = 1\big),$$

且 V 是 deterministic algorithm，對所有 x、y 在 `p(|x|+|y|)` 時間內終止。y 稱為 certificate 或 witness。

量詞順序不能交換。對 yes instance，存在 witness；對 no instance，任意 witness 都不能讓 verifier 正確接受。所謂「猜 witness」的 NTM 說法，是把這個存在量詞轉成 nondeterministic computation path。

verifier 的 soundness 與 completeness

何時失效

針對某個關係 `R(x,y)`，可定義 verifier 的兩種方向：

- completeness：若 `x in L`，存在 polynomial-length y 使 `R(x,y)=1`。
- soundness：若存在 y 使 `R(x,y)=1`，則 `x in L`。

合併兩者才是 `x in L iff exists y R(x,y)`。只有 completeness 可能接受錯誤 instance；只有 soundness 可能漏掉某些 yes instance。

以 Hamiltonian cycle 為例，certificate 是 n 個頂點的一條序列。Verifier 必須檢查：序列是所有頂點的一次排列、每個相鄰 pair 都是原圖邊、末點連回首點。四項全部成立才接受。一個只檢查「共有 n 個頂點」的 verifier 不 sound。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P 包含於 NP 的證明

假設 `L in P`，存在 deterministic polynomial-time M 判定 L。定義 `V(x,y)` 忽略 y，直接執行 M(x) 並輸出 M 的答案。取空字串 `epsilon` 為 certificate，長度 0。

若 `x in L`，M(x)=1，因此存在 short witness epsilon 使 V=1。若 x notin L，因 M 對所有 y 都回 0，不存在 witness 使 V 接受。這同時得到 soundness 與 completeness。

因此 `P subseteq NP`。是否有逆包含 `NP subseteq P` 仍是 open problem；教材不能把「尚未找到 polynomial algorithm」誤寫成「已證明不存在」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.3 NP-completeness and reducibility

many-one reduction 的完整定義

`A <=_p B` 表示存在函數 f，滿足：

1. f 對 A 的每個合格式 input 都可在 `poly(|x|)` 時間計算。
2. f 的輸出是 B 的合格式。
3. 對所有 x，`x in A iff f(x) in B`。

方向是「把 source A 的 instance 轉成 target B 的 instance」。若想證明新問題 B 困難，可以把已知困難 A 轉成 B；不能把 B 轉成已知容易問題就下結論。

此 reduction 是 many-one，因為每個 x 只產生一個 B instance。它不允許根據求解器答案再回頭修改 x，也不允許呼叫 B solver 多次，除非那些呼叫本身可被包進合法 polynomial-time 轉換。

雙向語義：completeness 與 soundness

`iff` 要拆成兩個檢查：

- completeness direction：`x in A implies f(x) in B`。A 的每個 yes instance 都要構造出 B 的 yes instance。
- soundness direction：`f(x) in B implies x in A`。任何由 f 產生的 B yes instance 必須能解回 A 的 yes instance。

名稱在文獻中可能因視角互換；解題時應直接寫方向，避免只寫「sound/complete」造成歧義。

以 CLIQUE 為例，轉成的圖若存在 k-clique，具體 clique 必須提供足以還原一組 satisfying assignment 的資訊。若圖的 size、k 或邊的 gadget 允許不存在的 assignment，soundness 就壞掉。

transitivity 定理與完整證明

定理。 若 `A <=_p B` 且 `B <=_p C`，則 `A <=_p C`。

證明。 令 f 是 A 到 B 的 reduction，g 是 B 到 C 的 reduction。定義 `h(x)=g(f(x))`。

若 `x in A`，由 f 的 iff 得 `f(x) in B`；再由 g 的 iff 得 `g(f(x)) in C`。反向若 `g(f(x)) in C`，由 g 得 `f(x) in B`，再由 f 得 `x in A`。因此 `x in A iff h(x) in C`。

令 f 在 `p_f(n)` 時間終止，g 在 `p_g(m)` 時間終止。因 `|f(x)| <= p_f(|x|)`，算 `h(x)` 的時間至多 `p_f(n)+p_g(p_f(n))`。多項式合成仍為多項式。故 A <=_p C。

NP-hard 與 NP-complete

語言 L 是 NP-hard，若對每個 `A in NP` 都有 `A <=_p L`。語言 L 是 NP-complete，若：

1. `L in NP`；
2. L 是 NP-hard。

若已知 C 是 NP-complete，且 `L in NP`，要證明 L 是 NP-complete，只需證 `C <=_p L`。任取 `A in NP`，因 C NP-hard 得 `A <=_p C`；再用已知 `C <=_p L` 與 transitivity 得 `A <=_p L`。

缺少 `L in NP` 只能得到 NP-hard，不能得到 NP-complete。缺少 `C <=_p L` 則沒有把已知困難性傳到 L。

第一個 NP-complete 問題的構造模型

CIRCUIT-SAT 輸入是有限、有向無環的 Boolean circuit，gate 為 fan-in 至多二的 AND、OR 與一輸入 NOT；問是否有一組輸入使指定輸出為 1。certificate 只需原始輸入 bits，依拓撲序計算每個 gate、檢查輸出，時間與 gate/wire 數成線性，故在 NP。

要證 NP-hard，任取 L in NP，固定其 deterministic verifier V、certificate 長度界 q(n) 與時間界。給 x，n=|x|，電路 C_x 的自由輸入必須是 witness y，而不是把一個固定 computation 的接受狀態當作未知。

可將witness規格化為固定長度q(n)+b(n)，其中長度欄b(n)=1+⌈$\log_2(q(n)+1)$⌉，至少1 bit。前b(n)位編碼實際長度ℓ，後q(n)位放witness並補零；新verifier先檢查$0 \le \ell \le q(n)$，再讀前ℓ個witness bits並模擬V。q(n)=0時仍有一位長度欄，只允許$\ell=0$。這不改變「存在可接受witness」，也不會把不同長度默認成相同字串。

由局部 transition 逐 gate 展開計算

固定一部模擬此verifier的單帶deterministic TM，讓它於接受或拒絕後停留在同一終止狀態。多帶轉單帶至多帶來多項式時間損失。令T(n)為包含witness長度後的多項式步數界，初始帶長N=O(n+q(n)+1)，包含前頁至少一位的長度欄，q(n)=0也有定義。

一步最多移動一格，所以只需表示 O(N+T) 個 tape cells，兩端額外保留 T+1 空白格，頭不會碰到外界。每格標籤包含 tape symbol，以及「此格是否有頭、若有則是哪個 state」。機器字母表與狀態集固定，故每格用固定 b bits。

時間 t+1 的第 j 格標籤，只取決於時間 t 的 j-1、j、j+1 三格標籤。這是固定 3b 輸入的 Boolean 函數。逐個輸出 bit，用真值表寫成若干 minterms 的 OR，每個 minterm 為輸入 bit 或 NOT bit 的 AND，再把多輸入 AND/OR 拆成二輸入 gates。因 b 與機器固定，每個局部更新只需 O(1) 個 gates。

逐層接線、規模與 iff 證明

初始層將 x、空白格、初始頭位置與 state 接成常數，只有規格化 witness bits 自由。對 t=0,...,T-1，每個 tape cell 複製上述固定局部電路，將其輸出接到下一層相同 cell。最後用 OR tree 檢查第 T 層是否有接受 state 標記。

總 gate 數為 O(T(N+T))，wire 數同階，gate 索引使用 O(log(T(N+T))) bits；列舉時間層、cell 與固定真值表即可在多項式時間輸出電路，沒有隱藏指數大小的硬體。

對每組 y，依 t 歸納：第零層是合法初始 configuration；局部 transition 產生恰好下一 configuration。因此所有層都是 V(x,y) 的真實執行，無效局部標籤永遠不會由合法初始層出現。接受狀態持續不變，故 C_x(y)=1 iff V(x,y) 接受。

若 x in L，存在接受 y，於是 C_x 可滿足；若 C_x 可滿足，其自由輸入 y 使 V 接受，於是 x in L。每個 NP 語言都如此多項式轉入 CIRCUIT-SAT，加上 NP membership，完成第一個 NP-complete 證明。

34.4 NP-completeness proofs

CIRCUIT-SAT 到 SAT 的局部 gate 約束

為每條 wire（可共享同一 gate output）設一變數，並把輸出 wire z_out 本身作為 unit clause，要求它為真。每個 gate 加入輸出等於 gate 函數的條件：

Gate 關係	等價 CNF clauses
z = u AND v	(¬z∨u) AND (¬z∨v) AND (z∨¬u∨¬v)
z = u OR v	(z∨¬u) AND (z∨¬v) AND (¬z∨u∨v)
z = NOT u	(z∨u) AND (¬z∨¬u)

例如 AND 的前兩 clauses 強迫 z=true 時 u,v 皆真；最後一 clause 強迫 u=v=true 時 z=true。OR 與 NOT 可同樣分真假兩向直接檢查。常數 wires 用 unit clause 固定。

電路接受輸入可延伸為所有 wire 的真值，滿足每個局部 clause 與 output clause。反向滿足公式時，局部等價式沿拓撲序迫使每個 wire 等於實際計算值，output=true，所以原電路可滿足。每 gate 至多三 clauses，總長 O(gates+wires)，不把共享子電路反覆展開成指數長樹狀公式。

SAT 到 3-CNF 的 parse-tree gadget

任意公式先解析成 NOT、一輸入或 AND/OR 二輸入樹；例如 implication 可用 ¬a∨b 改寫，其他固定 arity connective 亦用固定大小 truth-table gadget。為每個內節點設新變數，使用前頁 gate clauses，根變數設真。若同一節點輸入重複，先在所得 clauses 去除重複 literal；含 l 與 ¬l 的 clause 是 tautology，可整個刪掉。

此時 clauses 長度至多三。若要求每 clause 恰含三個相異變數，逐 clause 使用新的 padding variables：

- 兩 literal (a∨b) 改為 (a∨b∨p) AND (a∨b∨¬p)。a∨b 真則任意 p 都可；若假，兩式分別強迫 p 與 ¬p，矛盾。
- 一 literal a 改為四式 (a∨p∨q)、(a∨p∨¬q)、(a∨¬p∨q)、(a∨¬p∨¬q) 的 AND。a 真則皆真；a 假時，任何 p,q 指派都令其中一 clause 為假。
- 空 clause 表示不滿足，直接輸出上述 padding 過的 z AND ¬z；空 AND 表示恆真，輸出固定可滿足的三變數 clause。

原 assignment 可用每個 parse node 的真值延伸；反向沿樹恢復所有節點語義。padding 保留每 clause 的真假要求，因此整體只保留 equisatisfiability，而非把加入新變數後的公式當作原變數的逐字同一函數。每節點常數 clauses，每 clause 至多擴成四個，所以大小與建構時間皆多項式。

長 clause 拆分與可追蹤 gate 例

若已得到長 clause $l_1 \vee \cdots \vee l_r$，$r > 3$，也可用 chain：

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \\ \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

若所有 l_i 都假，首式逼 $y_1 = \text{true}$，中間逐次逼下一個 $y_j = \text{true}$，末式卻要求最後 $y_{r-3} = \text{false}$，矛盾。若至少一個 l_i 真，令 y_j 表示前 $j+1$ 個 literals 是否全假，即可逐式滿足。此拆分用 $r-3$ 新變數、$r-2$ clauses；不能直接用 distributivity 把任意公式轉 CNF，那可能產生指數長度。

例 F=(a∨b) AND ¬a，設 u=a∨b、v=¬a、z=u AND v，給三組 gate clauses 並加 z。a=false,b=true 時 u=v=z=true，滿足所有約束。若 a=true，NOT gadget 強制 v=false，而 z 與 AND gadget要求 v=true，故這類指派不能假造接受。這同時示範 witness 延伸與反解。

原書 3-CNF 定義為三個相異 literals；本教材的主要 reduction 進一步可規格化為三個相異變數。出現重複 literal 的輸入仍可先依上述去重、padding 規則處理，不影響後續 CLIQUE occurrence 構造。

3-CNF-SAT <=_p CLIQUE 的構造

令 F 為 3-CNF formula，記 `|F|` 為其實際 binary encoding 長度。若 F 有 k 個 clauses，為每個 clause 中的每個 literal occurrence 各建一個 vertex，並令 CLIQUE 的參數也為 k。非退化情形取 `k>=1`；空 AND formula 沒有 clauses，恆為真，另行映射到 `(空圖,0)`。

邊的規則正好兩條：

1. 同一 clause 內的 vertices 彼此不相連。
2. 不同 clauses 的兩個 vertices 相連，若且唯若它們不是互補 literals。

每個 clause 恰有三個 occurrence，因此 vertices 數是 `3k`。先把實際出現的變數名稱編為n個ID（n≤3k），逐一檢查vertex pairs需$O(k^2)$次O(1) word比較。若逐bit計費，ID比較及edge-list端點各需O(log(n+k)) bits，建圖可用$O(|F| \log|F|+k^2 \log(n+k))$的多項式bit上界；若輸出adjacency matrix則有$\Theta(k^2)$個bits。不能把任意長變數名稱的字串比較無條件當作O(1)。

可追蹤 3-CNF-SAT 例子

令

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

取 assignment `x1=false, x2=false, x3=true`。三個 clauses 都為 true。

選 literal occurrences：第一 clause 的 `x3`、第二 clause 的 `¬x1`、第三 clause 的 `x3`。它們來自不同 clause，因此建立跨群 edges；彼此不是 complement，所以三點兩兩相連，形成 size 3 clique。

這個 assignment 並不是唯一 witness。若改挑第一 clause 的 x1 與第二 clause 的 ¬x1，這兩個 occurrence 會因互補而缺邊，不能一起組 clique；在目前 assignment 下 x1 也為假，不應被選作真 literal。這正好說明 graph 不會把矛盾 literals 同時當成一致 witness。

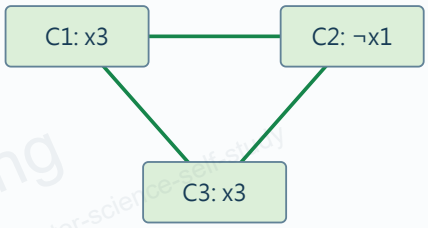
可追蹤 3-CNF-SAT 例子 （續）

不同 clause 且非互補，才連邊

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ 相連；× 互補而缺邊

選到的三個 occurrence 彼此一致



x1=0，x2=0，x3=1：每群一個真 literal

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3-CNF-SAT 到 CLIQUE 的 iff 證明

Completeness。 若 F satisfiable，固定一組 satisfying assignment。每個 clause 至少有一個 true literal occurrence，選其中一個。因 assignment 一致，兩個被選 occurrences 不會互為 complement，所以對應 vertices 之間都有邊。k 個 clauses 得到 k-clique。

Soundness。 若 G_F 有 k-clique，同一 clause 的 vertices 沒有邊，所以 clique 必須在每個 clause 中恰好選一個 occurrence。任意兩個被選 literals 都有邊，代表它們不互為 complement。因此同一個 variable 不會同時被要求為 true 與 false。令每個被選 variable 取得使其 literal 為真的值；未選 variables 任意指定；這些要求彼此一致，且每 clause 有一個 true literal，故 F satisfiable。

兩方向合併即 `F satisfiable iff G_F has k-clique`。CLIQUE 也有 polynomial certificate（k 個 vertices）與 checker，所以 CLIQUE 是 NP-complete。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER

independent set S 是圖中任兩點都沒有 edge 的集合。vertex cover C 是每條 edge 至少一端在 C 的集合。

對 n 點、無 self-loop 的無向圖 G=(V,E)，令 `Gbar=(V, {uv: u!=v and uv notin E})`。同一 pair 在 G 有 edge 若且唯若在 Gbar 沒有 edge。因此：

$$C \text{ is a clique in } G \iff C \text{ is an independent set in } \overline{G}.$$

若 S 是 independent set，則 V\S 是 vertex cover。假設有 Gbar 的 edge 未被 V\S 覆蓋，它的兩端都在 S，違反 S independent。反向若 V\S 是 vertex cover，任何 Gbar edge 不可能兩端都在 S；所以 S independent，也是在 G 的 clique。

令 `k in {0,...,n}`。把上面兩步串起，得到

`CLIQUE(G,k)` iff `VERTEX-COVER(Gbar,n-k)`。

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER （續）

`(G,k) -> (Gbar,n-k)` 與反方向的 `(H,l) -> (Hbar,n-l)` 都以 `O(n^2)` 次配對檢查建立補圖。這裡CLIQUE可說「至少k點」或「恰k點」，因較大clique任取k點仍是clique；VC採「至多n−k點」，較小cover可補點至恰n−k點，所以同一iff成立。這需0≤k≤n；範圍外門檻先直接判定並映射至固定yes/no實例，不能產生負大小certificate。

CLIQUE 與 VERTEX-COVER 的可追蹤例

令 `G` 有頂點 {1,2,3}，edges {12,23}，取 k=2。{1,2} 在 G 是 clique。

補圖 `Gbar` 只有 edge {13}。`{1,2}` 在 Gbar 是 independent set，因為 1 與 2 沒有 edge。其補集 `{3}` 是 Gbar 的 vertex cover，大小為 `3-2=1`，且它覆蓋唯一 edge 13。

轉換是 `VC(Gbar, n-k)`。所以 `CLIQUE(G,k) iff VERTEX-COVER(Gbar,n-k)`。這個等價直接給出雙向約化：complement 本身就是 polynomial-time、可逆的 encoding transformation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems

VERTEX-COVER 到 HAMILTONIAN-CYCLE 的端口 gadget

HC certificate 是全部頂點的排列，逐邊與閉合檢查給 NP membership。下面完整建立 NP-hardness，不需要平面 crossing gadget；圖中交叉畫線不代表交叉處是頂點。

對原圖每邊 {u,v} 建十二個新點 a1,...,a6、b1,...,b6，其中 ai=[u,v,i]、bi=[v,u,i]。內邊共十四條：

- 兩條鏈 a1-a2-a3-a4-a5-a6 與 b1-b2-b3-b4-b5-b6，共十條。
- 四條交叉連結 a1-b3、a3-b1、a4-b6、a6-b4。

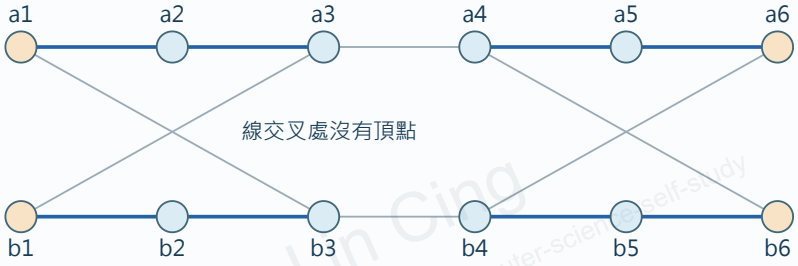
只有四個端口 a1,a6,b1,b6 可接 gadget 外部。其他八點在最後 HC 中必有兩條內邊。a2,b2,a5,b5 在 gadget 本身 degree=2，故它們的八條相鄰鏈邊被迫選入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems （續）

HC gadget：只有 a1、a6、b1、b6 能接外部

a_i=[u,v,i]；b_i=[v,u,i]



藍色8邊由 a2、a5、b2、b5 的 degree=2 強迫選入；灰色6邊仍待決定。

兩條中邊 a3–a4、b3–b4 決定其餘4條cross edges；橘色為4個端口。

gadget 只有三種合法的覆蓋方式

餘下選擇由中間兩邊 a3-a4、b3-b4 決定。每個內部點仍需 degree=2，故這兩邊一旦選或不選，四條交叉連結的使用亦被迫確定。

中邊選擇	對全部十二點的路徑覆蓋
兩條都選	a1-a2-a3-a4-a5-a6，以及 b1-b2-b3-b4-b5-b6，兩條 disjoint paths
只選 b3-b4	a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6，一條 a1 到 a6 的完整 path
只選 a3-a4	對稱的一條 b1 到 b6 的完整 path
兩條都不選	上下各形成一個封閉六點 cycle，與外部無法連成單一 HC，因此禁止

這四種窮盡所有可行內度配置，故合法 HC 只能使用前三種。尤其不能從 a 端口進來卻沿 b 的外部鏈離開；也不能讓十二點完全無人覆蓋。此局部事實正好表示原邊至少有一個端點被選。

中邊如何強迫端口的外部使用

令x表示是否選a3–a4，y表示是否選b3–b4，值為0或1。內點a3已有a2–a3，所以a3–b1必取1–x；a4同理使a4–b6取1–x。b3,b4則使a1–b3與a6–b4取1–y。

因此a1,a6的內部度都是2–y，還需要的外部度為y；b1,b6的外部度為x。端口不可能一邊連外、一邊封閉：同一原頂點側的兩端總是同時接外或同時不接外。

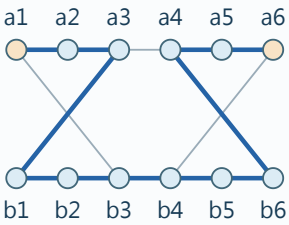
x=y=0時，下列兩個六點cycle已各自用滿度2：a1–a2–a3–b1–b2–b3–a1，以及a4–a5–a6–b4–b5–b6–a4。它們不能接到selector，故不可能是整張圖的單一HC。其餘三種配置的全部路徑如下圖，逐點可核對沒有漏點或換側離開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用 （續）

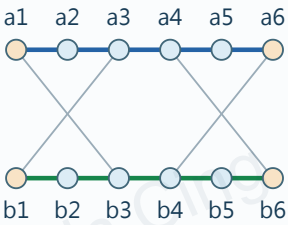
三種合法模式都不會跨原頂點身分離開

只走u側：12點



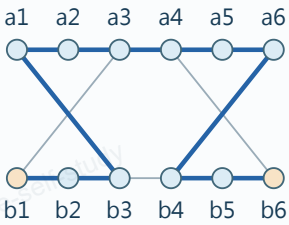
x=0 · y=1

u、v各走6點



x=1 · y=1

只走v側：12點



x=1 · y=0

灰邊未選；藍 / 綠是各條路徑；橘色端口各需一條外邊補到 degree=2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串接原頂點的 incident gadgets 與 selectors

VERTEX-COVER 輸入採「大小至多 k」。先移除孤點，不影響最小 cover。k<0 或 k=0 且有邊，輸出固定 no HC instance；無邊或 k 大於等於剩餘頂點數，輸出固定 yes triangle。以下可假設無孤點且 1<=k<|V|；一個較小 cover 可以補入其他非孤點成恰 k 點。

對每個原頂點 u，任排其鄰居 u(1),...,u(d)。把各 gadget 的 u 側依序串接：加入 [u,u(i),6]-[u,u(i+1),1]，i=1,...,d-1。這些連結使一條外部路徑一旦選 u，便沿它全部 incident gadgets 走到末端。

另建 k 個 selector s1,...,sk。每個 selector 都接到每個 u 鏈的首端 [u,u(1),1] 與末端 [u,u(d),6]，沒有其他外邊。圖共有 12|E|+k 點；十四內邊乘 |E|，串鏈至多 2|E|，selector 連邊 2k|V|，全為多項式規模。原圖有向無向不可混淆，此構造與目標皆為無向圖。

VC 到 HC 的完整 iff

若 C={u1,...,uk} 是 cover，將每個被選 u 的 incident-gadget 鏈作一條 cover path。原邊若只有 u 被選，使用從 u 側進出的十二點完整 path；若兩端都選，使用兩條各六點 path。因每條原邊至少一端被選，每個 gadget 的十二點恰好各被訪問一次。依序用 s1、u1 鏈、s2、u2 鏈、...、sk、uk 鏈、s1 串成一個 cycle，亦各訪 selector 一次，故為 HC。

反向若新圖有 HC，移去 k selectors 後，cycle 被切成 k 條 disjoint cover paths。局部 gadget 的三模式禁止切換端點身分；每條 path 必沿同一原頂點 u 的 incident 鏈從頭到尾。不同 paths 不能選同一 u，否則會共用其鏈端或 gadget 點。取這 k 個 u 為 C。每個 gadget 的點都被 HC 訪到，而三種模式至少有 u 或 v 的 cover path 經過，所以每條原邊 {u,v} 至少一端在 C。得到大小 k 的 vertex cover。

因此 VC(G,k) iff HC(G')；構造、反解與規模皆多項式。固定 yes/no 例處理前處理的退化情況，不需用不存在的空鏈湊 selectors。

VC gadget 的小例

原圖為 path u-v-w、k=1，可選 cover {v}。建兩個十二點 gadgets W_uv、W_vw；把 v 側第一個 gadget 的端口 6 接第二個 gadget 的端口 1，再用唯一 selector 接整條 v 鏈的首尾。

兩條原邊只有端點 v 被選，所以每個 gadget 都由 v 側完整走過十二點。selector 與兩個 gadgets 共二十五點形成 HC。反之若選 u 的鏈，W_vw 沒有任何被選端點，無法覆蓋，因此不會假造 cover {u}。

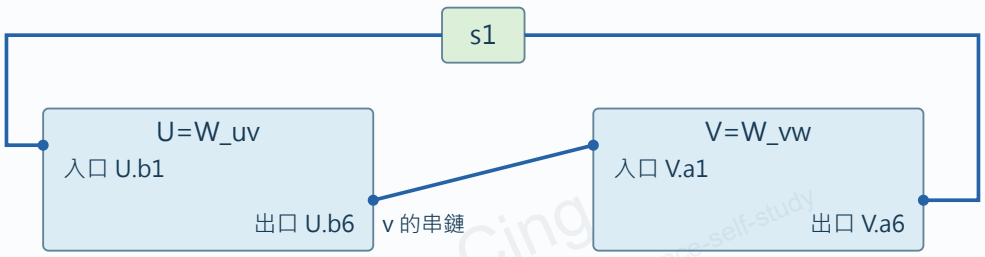
完整二十五點cycle可由以下兩條十二點路徑接成。記U=W_uv、V=W_vw；U的b側代表v，V的a側代表v：

- U：b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6。
- V：a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6。

由s1→U.b1開始，走完U後經串接邊U.b6→V.a1，再走完V，最後V.a6→s1閉合。U與V的同名點是不同頂點，故24個gadget點加s1恰25點，每點只出現一次。每個相鄰pair都能在十四內邊、串鏈或selector邊中逐項查得。

VC gadget 的小例 （續）

原圖 u–v–w、cover={v}：唯一 selector 串成25點 cycle



各框內是已列出的完整12點路徑．U與V的同名點互不相同。

此圖顯示選定的HC；完整外邊依selector與incident-chain構造定義。

HAMILTONIAN cycle <=_p TSP decision

給 HC instance `G=(V,E)`，`n=|V|`。建立完全圖 `K_n`，邊權重：

`w(u,v)=1` 若 `{u,v} in E`，否則 `w(u,v)=2`。令門檻 `B=n`。

HC 到 tour。 G 的 Hamiltonian cycle 有 n 條原圖 edges，每條 weight 1，因此在 K_n 是 cost n 的 tour。

Tour 到 HC。 K_n 任一個 tour 恰有 n 條邊。若其 cost <= n，且每條 weight 至少 1，則每條邊都必須 weight 1。這些邊都在 G，tour 因而也是 G 的 Hamiltonian cycle。

構造是 `O(n^2)`。所以 `HC(G) iff TSP(K_n,w,n)`，且 TSP decision的 certificate 是一條 tour，檢查排列與總成本即可，故 TSP decision in NP。

TSP 實例追蹤

令G是三角形，edges={12,23,31}，n=3。完全圖的三條邊全部權重1，因此tour 1→2→3→1的成本3，門檻B=3。

門檻 B=3。此 tour cost 恰好等於 B，對應 G 的 Hamiltonian cycle。

若原圖刪去31，構造出的完全圖仍有31，但權重改為2。任何三點tour都含三條邊，成本1+1+2=4>B，所以不存在合格tour，與原圖沒有HC一致。對n<3的簡單無向圖，本章HC無解，直接映射至固定no TSP instance；上述每tour恰n邊的主要論證採n≥3。

optimization 版 TSP 要求真正最小總成本；decision 版只問是否存在 cost <= B。若 optimization algorithm 在 polynomial time 回傳最佳值，決策版可直接比較 B；決策版 NP-hard 因而支持 optimization 版也難以精確 polynomial-time 求解。

優化問題與決策問題

很多 NP-complete 問題有 optimization counterpart。例：TSP 最佳 tour、SET-COVER 最少集合數、MAX-CLIQUE 最大 clique size。

若優化版在多項式時間可精確解，直接把最佳值與門檻比較便能解決策版。另一方向，若已有正確的決策 oracle，可對整數門檻binary search求最佳值。決策版NP-hard且精確優化可解時，便能解全部NP語言；故除非P=NP，這些精確優化問題不會有多項式時間演算法。

binary search 有兩個假設：

- 成本或答案介於可由 input 多項式位元表示的上下界；
- decision procedure 對所有門檻都正確。

得到最佳值不等於直接得到 witness。若也要輸出構造，需額外做 reconstruction，例如逐步固定決策並重跑 decision oracle；不能把「知道最佳值」冒充成「已回傳最佳解」。

classification 的精確邊界

名稱	精確定義	已能推出什麼	不能直接推出什麼
NP-hard language L	每個 `A in NP` 都有 `A <=_p L`	若 `L in P`，則 `P=NP`	L 本身在 NP
NP-complete language L	`L in NP` 且 L NP-hard	它有一個 polynomial verifier，且可承接任一 NP source 的 reduction	一個 optimization 程式自動成為 language
CLIQUE decision	對 input `(G,k)` 回答是否存在 size-k clique	本節已證它在 NP 且 NP-hard	它輸出最大 clique
MAX-CLIQUE optimization	對 G 輸出最大 clique size（可再要求 witness）	若另有 polynomial algorithm，可由答案解 CLIQUE decision	optimization 的問題型別可直接套用 `NP-complete` 定義

`NP-hard` 是 reduction 的普遍性，`NP-complete` 額外要求 membership。decision 回答 yes/no；optimization 輸出最佳值，search 再輸出 witness。證明可互相解出時，要說明程式如何使用一個問題的完整 output，以及是否真的得到另一問題要求的答案。

SUBSET-SUM、bit encoding 與 DP

decision 版 SUBSET-SUM 問：給正整數集合 S 與 t，是否存在子集合元素和恰為 t。

Certificate 是一組 selected indices；verifier 檢查每個索引在範圍內、不重複，且所選整數和恰為 t。長度與檢查時間 polynomial，所以 SUBSET-SUM in NP。

DP 狀態 `dp[i,s]` 表示前 i 個 elements 可否達和 s。轉移為

`dp[i,s]=dp[i-1,s] or (a_i<=s and dp[i-1,s-a_i])`。

計算量 `O(n t)`。這是 pseudopolynomial，不是以 t 的 binary bit length 衡量的 polynomial。NP-hardness 的完整無進位數位表與兩向證明接在下頁；其巨大數值仍有多項式 bit 長，與此 DP 的 pseudopolynomial 界限一致。

3-CNF-SAT 到 SUBSET-SUM 的數位表

先把公式規格化為 n 個變數、k 個 clauses，每 clause 三個相異變數、無互補 tautology，每個變數至少出現一次；前面的去重/padding 處理可達成。恆真與恆假公式另輸出固定 yes/no 整數實例。

建立十進位 n+k 欄，前 n 欄對應變數，後 k 欄對應 clauses。這是緊湊建立整數的方式，最終仍以 binary 編碼交付。

- 每變數 xi 建兩數 T_i、F_i。兩者的第 i 個變數欄為 1，其他變數欄為 0。T_i 在包含正 literal xi 的 clause 欄為 1，F_i 在包含 ¬xi 的欄為 1，其餘為 0。
- 每 clause j 加 slack 數 S_j、S'_j，只在該 clause 欄分別放 1、2，其他欄為 0。
- target 的每個變數欄為 1，每個 clause 欄為 4。

總共 2n+2k 個正整數，每數 n+k 位十進位，故每數 binary 長度 O(n+k)，輸出總 bit 長 O((n+k)^2)，建構為多項式。若輸入要求集合而非 multiset，上述非退化假設保證兩個同變數數字不相同；不同變數的高位欄不同，slack 欄也不混同。

為何無進位，以及兩向解碼

任意子集合在每個變數欄至多加到 2；每個 clause 含三個 distinct literals，對應 T_i/F_i 數最多提供三個 1，slack 最多加 1+2=3，因此 clause 欄至多 6<10。任何子集合都不會進位，不只是正確 witness 才不進位。

若原式可滿足，每變數依真假選 T_i 或 F_i，使變數欄和全為 1。每 clause 的真 literals 數 r 為 1、2 或 3，可用 slack 子集合湊 4-r：r=1 選 1+2，r=2 選 2，r=3 選 1。所有欄符合 target，得到 subset-sum yes。

反向若數字子集合和為 target，無進位迫使每變數欄恰為 1，所以 T_i、F_i 恰選一個，定義一致 assignment。clause target 為 4，但兩 slack 合計至多 3，所以至少一個被選 T_i/F_i 在該欄有 1，對應原 clause 中一個真 literal。每 clause 都滿足。兩向與多項式規模一起完成 NP-hardness，再加索引 certificate 的 NP membership，SUBSET-SUM 為 NP-complete。

可手算的無進位例子

取 F=(x1∨x2∨x3) AND (¬x1∨x2∨¬x3)，欄序 x1,x2,x3,C1,C2：

數	十進位欄表示
T1,F1	10010，10001
T2,F2	01011，01000
T3,F3	00110，00101
S1,S1'	00010，00020
S2,S2'	00001，00002
target	11144

assignment x1=true,x2=true,x3=false 選 T1+T2+F3=11122。每個 clause 真 literal 數都是二，再選 S1'=20、S2'=2，得 11144。沒有進位；高位每欄的 1 也直接證明每個變數只選一個真值數。

可手算的無進位例子 (續)

數值可能與 10^{n+k} 同階，但 binary 表示僅 $O(n+k)$ bits。這正是 reduction 可多項式輸出，而 $O(nt)$ 精確 DP 仍非輸入 bit 長度多項式的原因。

T1+T2+F3=11122；再加 slack 00022，得到 target 11144

	x1	x2	x3	C1	C2	
T1	1	0	0	1	0	變數欄各恰為1
T2	0	1	0	1	1	兩個 clause 各有
F3	0	0	1	0	1	2個真 literal
slack	0	0	0	2	2	各以 slack 2 補到4
target	1	1	1	4	4	

所有子集合都不能進位，才可反向解碼

若只對「依滿足指派選數」證明無進位，尚不足以證明反向：某個錯誤子集合也許靠進位偽造target。本構造對任意子集合都有變數欄≤2、clause欄≤6，故十進位下絕不進位。進位排除是所有候選的全稱命題。

每個數的位元長度O(n+k)，把十進位欄轉成binary可用逐欄乘10加digit的$O((n+k)^2)$簡單bit演算法；2n+2k個數也仍多項式。驗證子集合和時，至多加O(n+k)個O(n+k)位元整數，總和多出O(log(n+k))位元，亦多項式。

例如只取x1=false,x2=false,x3=false，兩clause的真literal數為0、2。第一欄即使兩個slack都取，也只有3，不能達4；不會向其他欄借出或借入一個位值來掩飾不滿足。

編碼不變性、P 的封閉性與 coNP

兩種合理編碼若可相互在多項式時間轉換，P membership不變：先轉碼再求解，輸出長度受轉碼時間限制，組合仍多項式。Binary與unary整數編碼不具此雙向保證，因把b位元數2^(b−1)寫成unary需指數長輸出。

P對補集、聯集、交集封閉：把判定結果取反或依序執行兩個判定器即可。對串接L1L2，枚舉n+1個切點，每個執行兩個多項式判定器。對Kleene star L^*，用reachable[0]=true，對i<j以L判定x[i+1..j]是否可作下一段；$O(n^2)$次多項式判定與DP可決定reachable[n]，空字串直接接受。因此兩者也在P。

coNP={L：補集L在NP}，表示no instance有短證書，不等同「不是NP」。尚未知P=NP或NP=coNP。若NP-complete語言C也在coNP，任取A∈NP經A≤pC，則補集A≤p補集C；把補集C的NP證書接上多項式轉換，可知補集A∈NP，得到NP⊆coNP，再取補集得相等。此為條件推論，不能當成已證分離。

NP-complete 證明鏈與各問題條件

教材使用的證明鏈可整理成：

`CIRCUIT-SAT <=_p SAT <=_p 3-CNF-SAT <=_p CLIQUE`。

再由 complement 得到 `CLIQUE <=_p INDEPENDENT-SET <=_p VERTEX-COVER` 的等價轉換。另由 `VERTEX-COVER <=_p HAMILTONIAN-CYCLE <=_p TSP-decision`。再以 3-CNF-SAT 到 SUBSET-SUM 的無進位十進位欄位構造，完成另一條 NP-complete 證明。

每個箭頭都要保留自己的 hypotheses：HC 用無向圖；TSP reduction 的 complete graph 可由原圖 polynomial-size 建出；subset-sum 的 t binary encoding 不能省；optimization TSP 與 decision TSP 不可混寫成同一語言。

設計新 NP-complete 證明的檢查表

1. 把決策問題精確寫成 yes 集合，列出 instance 參數與 output。
2. 證明它 in NP：指定 certificate、長度上界、verifier、接受／拒絕條件。
3. 選一個已知 NP-complete source A，構造 `A <=_p f -> B`。
4. 證明 completeness direction：source yes 可構造 target yes。
5. 證明 soundness direction：target yes 可解回 source yes。
6. 證明 f 與 target verifier 都是 polynomial time，且 output size 是 polynomial。
7. 檢查退化 edge cases，例如空圖、k=0、重複 literals、圖不連通與 unreachable value。

若只有「看起來像」gadget，沒有第 4、5 步，就沒有約化證明。若只把新問題轉回已知 NP-complete，方向也反了。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 34。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

35 近似演算法

Part VII Selected Topics

本章定位

近似演算法以可證界限交換精確最優解的成本；本章比較cover、metric TSP與subset-sum的下界、誤差和位元成本，區分常數比、對數比與FPTAS。

前置知識

最佳化、近似比與 NP-hard 問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 rho(n) approximation ratio。
- 分析 vertex cover、TSP、set cover、subset sum。
- 說明 randomization、LP rounding 與 PTAS/FPTAS 的邊界。
- 35.1 The vertex-cover problem
- 35.2 The traveling-salesman problem
- 35.3 The set-covering problem
- 35.4 Randomization and linear programming
- 35.5 The subset-sum problem

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

近似保證先說清楚目標、可行性與分母

本章處理非負目標值的最佳化問題。最小化的ρ近似要求每個輸入都回傳可行解，且ALG≤ρ・OPT；最大化則要求ALG≥OPT/ρ，ρ≥1。OPT=0時用不等式定義，不做0/0；最小化必須回0，最大化的非負可行值則已符合。

「對這個例子很好」不是近似保證；演算法須在所有合法輸入上滿足界限，而且計算時間以輸入編碼長度衡量。最小化證明常用可計算下界LB≤OPT，再證ALG≤ρLB；最大化則用上界UB≥OPT，證ALG≥UB/ρ。

PTAS（多項式時間近似方案）對每個固定ε>0，其時間對輸入長度N為多項式；例如N^(1/ε)可符合。FPTAS（完全多項式時間近似方案）還要求同時對N與數值1/ε為多項式。這不是只對log(1/ε)多項式，也不保證ε極小時仍便宜。

35.1 The vertex-cover problem

無向簡單圖G=(V,E)的vertex cover C滿足每條邊至少一端在C。求最小|C|是NP-hard最佳化問題；本節的保證針對未加權版本。空邊集合時回∅、OPT=0。

```
C=empty; matched[v]=false for all v
for each edge (u,v), in any fixed order:
    if not matched[u] and not matched[v]:
        add u,v to C
        matched[u]=matched[v]=true
return C
```

這等價於每次選任一未覆蓋邊，把兩端加入C、刪除所有已覆蓋邊。用端點標記而非反覆重建整張圖，可在O(|V|+|E|)時間、O(|V|)工作空間完成；輸入圖原儲存另計。

Maximal Matching 給了可證的下界

設被選的邊集合為M。因選一邊後兩端立即標記，M中任兩邊不共享端點，是matching（匹配）。每條未被選的邊在掃描時至少一端已標記，因此最後被C覆蓋；演算法回傳可行cover。

M是maximal matching（再也不能加邊），不必是maximum matching（邊數最多）。任一cover，包括C*，都必為M的每條邊選至少一端；這些邊互不共享端點，故至少要|M|個不同頂點。於是

$|C| = 2|M| \leq 2|C^*|$。

證明不需要知道OPT，也不需要求最大匹配。不能把「每條任意邊都需一點」直接相加成|E|下界，因相鄰邊可能共用同一cover頂點；matching才排除了重複計費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可達比值二，也可能正好最優

四葉star的中心c、葉a,b,d,e，先選(c,a)：C={c,a}，其餘三條邊因端點c已選而略過。最優只需{c}，|C|/OPT=2，保證可達。

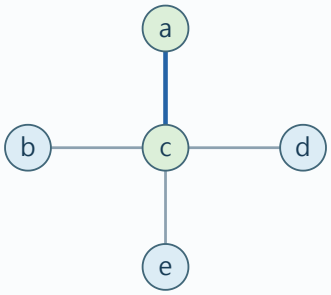
三角形a,b,c先選ab：C={a,b}，其餘邊已覆蓋；任取一點仍漏另一條對邊，所以OPT=2，這次ALG/OPT=1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

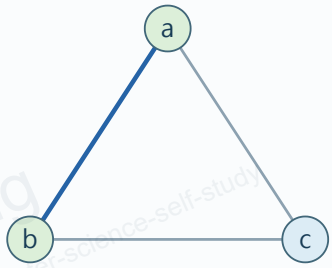
可達比值二，也可能正好最優 （續）

同樣選matching兩端：star達2倍，三角形恰好最優

藍粗邊是選取的 matching；綠點是輸出的 cover。



|M|=1 · ALG=2 · OPT=1



|M|=1 · ALG=2 · OPT=2

若改為頂點有權重，這個選兩端法沒有2倍權重保證。只有一條邊、兩端權重1與100，就會取兩點成本101，最優為1；第35.4節需用權重敏感的LP捨入。

35.2 The traveling-salesman problem

本節主要輸入為n≥3的完全無向圖，邊權c(u,v)≥0、對稱且滿足triangle inequality（三角不等式）c(u,w)≤c(u,v)+c(v,w)。這稱metric TSP；允許不同頂點間成本0時也稱pseudometric，以下證明仍成立。

TSP tour每個頂點恰訪一次後回起點。Double-tree演算法：求最小生成樹T，任選根做完整DFS walk，按頂點第一次出現的順序形成排列，最後連回起點。最後連邊必存在，依賴完全圖前提；一般稀疏圖不能直接這樣shortcut。

在完整權重矩陣上用簡單Prim需$O(n^2)$，DFS／輸出$O(n)$，所以總$O(n^2)$次word算術與比較；若權重為大有理數，須另計每次算術／比較的bit成本。

兩倍保證的三條不等式各用什麼前提

從最佳tour H^*移去任一邊，得到生成樹；移去非負邊不增加成本，MST最小性給c(T)≤c(H^*)=OPT。

完整DFS walk W每條樹邊去、回各一次，故c(W)=2c(T)。W可能重複頂點，所以還不是TSP tour。

若W的某一段為u→v_1→⋯→v_k→w，而中間點之前都已拜訪，可以直接改走u→w。反覆三角不等式給c(u,w)≤這段原成本；每次省去重複拜訪都不增加成本。保留各頂點首次出現及最後回起點，就得到可行tour H，且

c(H)≤c(W)=2c(T)≤2OPT。

下界需要非負，shortcut需要metric，輸出需要完全圖；這些不是可以省略的同一個「TSP」假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Double-tree 的每條路與實際成本

取四點Manhattan距離c(p,q)=|p_x−q_x|+|p_y−q_y|：A(0, 0)、B(2, 0)、C(2, 1)、D(0, 1)。三角不等式由各座標絕對值的不等式相加得到。

可取MST邊AD=1、DC=2、CB=1，總4。六條邊成本為1, 1, 2, 2, 3, 3，任何生成樹的三條邊成本至少1+1+2=4，所以此鏈確實最小。根A、child鏈A→D→C→B，完整walk為A,D,C,B,C,D,A，成本1+2+1+1+2+1=8。首次順序A,D,C,B，再回A，tour成本1+2+1+2=6。

每個tour成本至少6：要訪最左與最右x座標，封閉路徑的x總變動至少4；同理y總變動至少2。矩形周邊tour正好6，所以OPT=6。此例不是把walk8誤當成真正tour成本6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

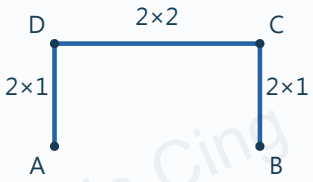
Double-tree 的每條路與實際成本 (續)

同一矩形、同一Manhattan成本：MST 4 → walk 8 → tour 6

最小生成樹

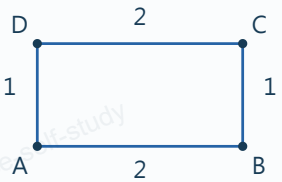
AD+DC+CB=1+2+1=4

每樹邊各走2次



A,D,C,B,C,D,A；成本8

取首次出現、連回A



A,D,C,B,A；成本6

A(0,0)、B(2,0)、C(2,1)、D(0,1)；座標兩軸同尺度，邊標示為Manhattan成本。

Christofides：最小匹配帶來 3/2 保證

沿用metric前提。令O為MST中奇度頂點。度數和為2|E|，所以|O|為偶數；在O上求最小權完美匹配M，將其邊與T組成multigraph。每個奇度點被加一次後變偶，其餘度不變；圖仍連通，故有Euler closed walk，shortcut得到tour。

取最佳tour，按它造訪O的順序shortcut，只保留O，得到總成本≤OPT的封閉循環。把循環邊交替分成兩個完美匹配（|O|=2時視為兩份往返邊），兩者成本和≤OPT，至少一個≤OPT/2。因此最小匹配 c(M)≤OPT/2，最後tour成本≤c(T)+c(M)≤3OPT/2。

此演算法使用「一般圖最小權完美匹配可在多項式時間求得」作子程序定理；不能用任意greedy matching替代最小權匹配後仍沿用此證明。求Euler walk與shortcut本身為多項式；證明與double-tree共用metric前提。

非 Metric TSP：常數比會解出 HC

假設存在確定性、多項式時間且對每個非負完全圖都保證ρ近似的演算法A，ρ≥1是固定常數。取固定整數R≥ρ，對HC輸入G建立完全圖：原邊權1、非邊權M=Rn+1。

有HC時OPT=n，故A輸出成本≤ρn≤Rn。無HC時任何tour至少使用一條非邊，成本≥M+(n−1)=Rn+n>Rn。因此只要檢查A輸出是否≤Rn，就能多項式判定HC；將推出P=NP。

M的binary長度O(log n)，完整圖輸出O(n^2 log n) bits，沒有以巨大數值暗藏超多項式輸出。這些權重通常違反三角不等式，所以不抵觸metric近似演算法。結論是「除非P=NP，一般TSP無固定常數比的此類演算法」，不是已證P≠NP，也不把只有期望保證的隨機演算法直接套成確定性判定。

35.3 The set-covering problem

Universe X有N個元素、候選family F={S_1,⋯,S_M}，每個S_j⊆X，聯集須為X。Set cover選若干候選集合覆蓋X；未加權最小化集合數，加權版每組有正有理成本w_j、最小化成本和。空X回∅；聯集不足則回報無可行解。

```
U=X; C=empty
while U is not empty:
    choose S with |S∩U|>0 minimizing w(S)/|S∩U|
    C=C union {S}; U=U minus S
return C
```

未加權w=1時等價於每次選新增覆蓋最多者。每次至少覆蓋一個新元素，且選過的集合之後新增數0不再選，故至多min(N,M)輪。簡單N×M incidence矩陣每輪O(NM)即可算全部新增數，總O(NM min(N,M))次操作，已是多項式；有理成本比用交叉相乘精確比較。

35.3 The set-covering problem （續）

Set-cover的精確最佳化為NP-hard：由vertex-cover輸入G建立X=E，每個原頂點v對應候選集合S_v={與v相接的邊}、成本1。選至多k組覆蓋X，當且僅當原圖有至多k個頂點覆蓋所有邊。Incidence總數2|E|，構造長度與時間均多項式；加權版包含成本全1的特例，因此也NP-hard。此歸約沒有聲稱每個特殊family都困難。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用實際元素收費，得到 Harmonic 界

某輪選S、成本w，新增k個元素，就給每個新元素charge=w/k。每元素只第一次被覆蓋時付費，故Σ_(e∈X)charge(e)=ALG。

固定任一候選集合A，令|A|=d_A。將其元素按首次被覆蓋次序排$e_1,\dots,e_d$。同輪多個元素任意排。在e_j被覆蓋的那一輪開始，A至少還有d_A−j+1個未覆蓋元素；A本身也是合法候選，所以greedy選擇的每元素成本

`charge(e_j)≤w(A)/(d_A−j+1)`。

即使e_j同輪不是該集合第一個被覆蓋者，當輪開始的未覆蓋數只會更多，此界仍成立。加總得Σ_(e∈A)charge(e)≤w(A)H_(d_A)，其中H_d=1+1/2+⋯+1/d，H_0=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對最佳 Cover 相加時可以重複，不能漏元素

設C^*為最佳cover，d=max|S_j|。每個元素至少屬於C^*中的一組，charge非負，故

$$ALG = \sum_{e \in X} charge(e) \leq \sum_{A \in C^*} \sum_{e \in A} charge(e) \leq \sum_{A \in C^*} w(A) H_{|A|} \leq H_d \, OPT.$$

重複覆蓋的元素在右邊可能付多次，只會使上界更大，方向正確。又H_d≤1+ln d：對j≥2，1/j≤∫_(j−1)^j dx/x，逐項相加即可。

未加權與加權版共享此證明，但greedy規則不同；有成本時仍只挑最多新增元素，不符合前頁的每單位成本比較，不能沿用H_d保證。

Greedy 三組，最優兩組：每個元素都對上

X={1, 2, 3, 4, 5, 6}，S_1={1, 2, 3}、S_2={4, 5, 6}、S_3={1, 2, 4, 5}，成本皆1。先選S_3新增4，再以同額時S_1優先：

輪	進入時U	新增數(S_1,S_2,S_3)	選擇	新元素charge
1	1, 2, 3, 4, 5, 6	3, 3, 4	S_3	1, 2, 4, 5各1/4
2	3, 6	1, 1, 0	S_1	3付1
3	6	0, 1, 0	S_2	6付1

ALG=3；$S_1 \cup S_2$=X，任一單組都不夠，所以OPT=2，比值3/2。charge總和4・(1/4)+1+1=3；在S_1上合計3/2≤H_3=11/6，S_2同樣，符合一般計費證明。

Greedy 三組，最優兩組：每個元素都對上 (續)

S3先覆蓋4個，但最後需要3組；S1+S2只需2組

	1	2	3	4	5	6
S1	1	1	1	0	0	0
S2	0	0	0	1	1	1
S3	1	1	0	1	1	0

初始：1,2,3,4,5,6

取S3後：3,6

取S1後：6

取S2後：∅

charge(1,2,4,5)=1/4；charge(3)=charge(6)=1

總收費3=greedy成本；最優 S1∪S2={1,...,6}，且不存在單組cover。

35.4 Randomization and linear programming

MAX-3-CNF-SAT最大化滿足clause數m。先規定每clause三個相異變數，且不含同一變數與其否定；各變數獨立、公平地取0或1。對某一clause，三literal全假的機率1/8，故滿足機率7/8。

若能多項式精確求出此最佳化的OPT，檢查OPT=m便能判定第34章的3-CNF-SAT，故精確最佳化NP-hard；這不妨礙以下對所有輸入的近似保證。

令I_j是第j個clause被滿足的指示，Y=ΣI_j。各clause可共享變數，I_j不必獨立；期望線性仍給

$E[Y]=\Sigma E[I_j]=7m/8\geq 7OPT/8$。

這裡「隨機8/7近似」是指期望目標值保證，不是每次輸出都達7OPT/8，也不是E[OPT/Y]≤8/7。只有一個clause時，機率1/8會得到Y=0，後者比值甚至無法給有限界。重複literal如(a∨a∨a)只有1/2滿足率，不能直接套7/8；含a與¬a的clause恆真，可另行計數。

從期望到可控制的成功率

若m=0，直接回傳任意assignment，Y=OPT=0，達到門檻的成功率為1。以下假設m>0；在前述三個相異變數模型，0≤Y≤m且E[Y]=7m/8。設p=Pr[Y≥3m/4]；對未達門檻者用3m/4作上界，得

$7m/8=E[Y]\le(1-p)3m/4+p\cdot m$，所以$p\ge1/2$。

因此獨立重跑r次，保留滿足數最多的assignment，未達3m/4的機率≤2^(−r)。這是明列成功門檻與失敗率的4/3保證；不能把它說成同時以高機率達到7/8。

若需要確定性7/8目標值，可用下一頁的條件期望逐一固定變數，不需要以大量隨機試跑猜出某一個好assignment。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

條件期望：每次固定都不降低保證

在部分指派下，某clause已有真literal則條件滿足率1；已全部指定且全假則0；其餘有r個未指定變數時為1−2^(−r)。逐clause相加即可精確算E[Y｜目前指派]。

對下一變數x，原條件期望是「x=0後期望」與「x=1後期望」的平均，所以選較大的一邊不會下降。固定完n個變數後沒有亂數，條件期望就等於實際滿足數，至少7m/8≥7OPT/8。

例四個clauses：(a∨b∨c)、(¬a∨b∨c)、(a∨¬b∨c)、(a∨b∨¬c)。初始期望7/2；a=0後為13/4，a=1後為15/4，取a=1。之後b=0後為7/2、b=1後為4，取b=1。c任選，例如0，四clause全真。

直接每步重算m個三literal clauses需O(nm)次固定精度分數操作；此例每個條件機率分母至多8，沒有隱藏指數精度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted Vertex Cover 的 LP 下界與捨入

每個頂點有正有理權w_v。整數模型最小化Σw_vx_v，約束每邊uv有x_u+x_v≥1、x_v∈{0, 1}。放寬為0≤x_v≤1得到LP；所有整數cover仍可行，因此LP^*≤OPT。

求最優可行LP解x^*，取C={v：x_v^*≥1/2}。對任一邊，若兩端都未取，兩個值都<1/2，違反$x_u^*+x_v^*$≥1，所以C可行。又逐頂點有1_[v∈C]≤2x_v^*，乘非負w_v相加：

$w(C) \leq 2\Sigma w_v x_v^* = 2LP^* \leq 2OPT$。

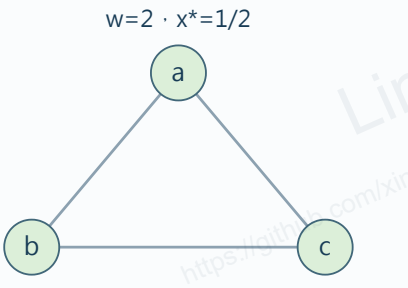
有理LP可用多項式bit複雜度的LP演算法求解；不能僅因simplex常常快就宣稱其最壞多項式。此證明採確實可行的LP解；若數值求解器有容許誤差，須另證可行性與成本界，不能任意將0.4999當成精確1/2。

LP 最佳值與整數最佳值是兩個不同下界

三角形a,b,c權重2, 3, 4。LP的x_a=x_b=x_c=1/2可行，成本9/2。它也是LP最優：將邊限制ab、ac、bc分別乘1/2、3/2、5/2後相加，左邊恰為2x_a+3x_b+4x_c，右邊為9/2，所以任何LP可行解成本≥9/2。

整數cover至少兩點，三種兩點成本5, 6, 7，故OPT=5。Threshold捨入把三點都取，ALG=9=2LP^*≤2OPT=10；實際比值9/5，而不是把LP^*=4.5誤當原問題最優。

加權三角形：LP*=9/2，整數OPT=5，捨入成本9



邊限制乘數：ab 1/2、ac 3/2、bc 5/2

加總：2x_a+3x_b+4x_c ≥9/2

半值達到9/2，所以為LP最優

整數取 a,b：成本2+3=5

≥1/2 捨入取三點：成本9

綠點全被threshold取入；LP下界與原整數最佳值不能混用。

隨機 Set-cover 捨入的成本與可行性分開驗證

作為LP方法的延伸，設N≥2，LP有每元素Σ_(S∋e)x_S≥1、x_S≥0及非負成本。令整數k=3⌈$\log_2 N$⌉≥3 ln N，獨立以p_S=min(1,kx_S)選每個集合。若LP解為有理數，p亦為有理數；以分母為範圍均勻抽整數即可實現，不需精確抽取含ln N的無理數機率。

若包含e的某組p=1，e必覆蓋；否則漏覆機率Π_(S∋e)(1−p_S)≤exp(−Σp_S)≤exp(−k)≤N^(−3)。Union bound得至少一個元素漏覆的機率≤N^(−2)≤1/4。期望成本≤k・LP^*；Markov得成本>2k・LP^*的機率≤1/2。兩種失敗事件相加≤3/4，所以每次同時可行且成本合格的機率至少1/4，成本比2k=O(log N)。

實際檢查覆蓋與成本，不合格就獨立重試，期望至多4次；這是可驗證成功條件的期望多項式演算法，不是固定一次必成功。N=0直接空解；N=1選最便宜包含該元素的集合，另行處理ln1=0。

若LP^*=0，成本非負且期望成本0，故成本必為0，成本檢查自動成立；只需按相同覆蓋規則重試，不以0作Markov的分母。

35.5 The subset-sum problem

本節最佳化版：正整數$a_1,\cdots,a_n$與整數t≥0，找一個子集合使總和≤t而盡可能大。以索引分辨各項，即使數值相同也不能重用同一項。a_i>t可先刪；n=0或t=0直接回空集、值0。

精確清單演算法以$L_0=\{0\}$開始，L_i=merge_unique(L_(i−1),L_(i−1)+a_i)，再刪去>t。兩個輸入皆排序，以兩指標線性合併去重，不必重排。

每個前i項的子集合恰分「不取i」或「取i」，分別對應L_(i−1)與L_(i−1)+a_i。正數使已超過t的和未來不可能再下降，所以刪除安全；歸納得清單保留所有可行精確和。最壞有2^i個不同值，例如二的冪且t足夠大，故通常是指數時間，不因每輪合併線性就成多項式。

TRIM 只刪較大的近值，保留真實可行和

給排序清單L、δ>0：

```
R=[]
for y in L, in increasing order:
    if R is empty or y>(1+delta)*R[-1]: append y to R
return R
```

對每個候選y，若被刪，當時保留的last=z滿足z≤y≤(1+δ)z；若被保留，取z=y。所以每個y都有代表z∈R，y/(1+δ)≤z≤y。0會保留，任何正值都>0，故第一個正值不會被0刪掉。

保留的是候選清單中的真實整數和，不是將y數值除掉或四捨五入。用較小代表才能維持≤t的可行性；若用較大代表，原本可行y加未來項可能被錯誤丟棄。

每輪合併、Trim、刪超額的完整算法

選0<ε<1，n≥1，令δ=ε/(2n)：

```
L=[0]
for i=1..n:
    B=merge_unique(L, [z+a[i] for z in L])
    L=trim(B,delta)
    remove all values >t from L
return max(L)
```

每個保留值仍是前i項某子集合的真實和，故回傳值確實≤t且可行。若需輸出子集合，可為每個保留候選保留不可變的parent pointer與本輪是否取a_i；合併同值時任選一個合法來源。回溯最多n步，各索引嚴格往前，不會重用物品。

只算值時可滾動存清單；若保留各輪的回溯節點，空間需另計到O(nK)個節點，K是單輪清單上界，不能仍稱只需O(K)。

代表誤差的歸納：包括「取本項」分支

對任一前i項的精確可行和y≤t，證明存在z∈L_i使y/(1+δ)^i≤z≤y。i=0只有y=z=0。

若不取a_i，y是上一輪可行和，由歸納取u∈L_(i−1)，y/(1+δ)^(i−1)≤u≤y。候選B有u；trim後有z≥u/(1+δ)，故達所需界且z≤y≤t，不會被刪超額。

若取a_i，寫y=y'+a_i。正數保證y'≤t，由歸納有u≥y'/(1+δ)^(i−1)、u≤y'。候選B有u+a_i，且

`u+a_i≥y'/(1+δ)^(i−1)+a_i≥(y'+a_i)/(1+δ)^(i−1)`。

Trim再損失至多1+δ因子，代表z≤u+a_i≤y≤t，所以也不被超額刪除。這一步明用a_i≥0；若容許負數，早刪超額與此歸納都不能原樣沿用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

累積誤差真的落在 ε 內

把上一頁用於y=OPT，最終回傳z_max至少是其代表，所以OPT/z_max≤(1+δ)^n（OPT=0直接以不等式成立）。由ln(1+x)≤x，

`n ln(1+ε/(2n))≤ε/2`。

又0<ε<1時，ln(1+ε)=∫₀^ε(1+x)^(−1)dx≥ε/(1+ε)≥ε/2。因此(1+ε/(2n))^n≤1+ε，得到

`z_max≥OPT/(1+ε)≥(1−ε)OPT`。

這是每輪誤差累乘後的證明，不是把全局ε直接當每輪δ。若每輪都用δ=ε，誤差可累至(1+ε)^n，不能保證同一個全局1+ε。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四輪整數清單：被刪值與最終答案都可追蹤

a=[112, 108, 205, 103]、t=325、ε=2/5，n=4，δ=1/20：

i	合併去重B	TRIM結果	刪去>325之後
1	0, 112	0, 112	0, 112
2	0, 108, 112, 220	0, 108, 220	0, 108, 220
3	0, 108, 205, 220, 313, 425	全保留	0, 108, 205, 220, 313
4	0, 103, 108, 205, 211, 220, 308, 313, 323, 416	0, 103, 205, 220, 308, 416	0, 103, 205, 220, 308

第2輪112≤108・1.05=113.4，刪112；第4輪108≤103・1.05=108.15，211≤205・1.05=215.25，313及323≤308・1.05=323.4，均被較小真實和代表。

輸出308=205+103；精確最優323=112+108+103。實際比值323/308≈1.049≤1.4，且308≤325。被丟掉的112不是從輸入刪除，而是在該輪候選和中被108代表。

四輪整數清單：被刪值與最終答案都可追蹤 （續）

清單長度、位元成本與 FPTAS

Trim後相鄰正值比率>1+δ，最小正值≥1，最大≤t，所以正值數≤1+ln t/ln(1+δ)，另加0。因 ln(1+δ)≥δ/(1+δ)≥δ/2，單輪長度

$K=O(1+n\ \log(t+1)/\varepsilon)$。

每輪兩條長K清單線性合併、trim及刪除，總$O(nK)=O(n+n^2\ \log(t+1)/\varepsilon)$次算術／比較；只存值時O(K)工作空間。log(t+1)是輸入bit長量級，不是數值t；與精確DP的O(nt)不同。

若ε=α/β為精確有理數，trim判斷可改為$2n\beta\cdot y>(2n\beta+\alpha)\cdot last$，避免浮點相對誤差。另外讀取與刪除$a_i>t$需O(N)位元工作，N為原輸入長度；之後候選≤2t。中間整數位數由log(t+1)、log n與α/β的編碼長度多項式控制。每次乘法計入相應大整數成本，總時間仍對輸入長度與1/ε多項式，才完成FPTAS的時間證明。

各種保證不可互換

方法	可行性	對目標的保證	額外條件
未加權VC選matching兩端	每次可行	ALG≤2OPT	不適用任意權重
Double-tree TSP	每次可行	ALG≤2OPT	完全、非負、對稱、metric
Weighted set cover greedy	每次可行	ALG≤H_d OPT	按成本／新增數選
隨機MAX-3-CNF	每次有assignment	E[ALG]≥7OPT/8	每clause三相異變數
LP weighted VC threshold	每次可行	ALG≤2OPT	可行最優LP與非負權
Subset-sum TRIM	每次可行	ALG≥OPT/(1+ε)	非負項、δ=ε/(2n)

期望界、每次確定界與帶失敗率的保證回答不同問題。近似方案仍須把精度、清單長度、輸入位元及解的回溯成本一起交代，不能只寫「刪相近值，所以很快」。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 35。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A 求和

Part VIII Appendix

本章定位

從有限恆等式推導等差、冪和、幾何、消去與乘積，再用固定常數歸納、尾比率、dyadic切分與積分雙界分析迴圈；明列無限級數及換序的收斂前提。

前置知識

等差、等比、對數與積分估計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用求和公式與線性性。
- 以積分、分割與幾何級數建立界限。
- 避免把有限和與無窮級數混用。
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.1 Summation formulas and properties

$\sum_{k=1}^{n} a_k$是有限加總，n=0時定義為0；$\prod_{k=1}^{n} a_k$是有限乘積，n=0時為1。空sum與空product是加法／乘法單位元素，使迴圈零次執行的公式也成立。有限sum可任意重排，並有$\sum(ca_k + b_k) = c\sum a_k + \sum b_k$。

無限級數$\sum_{k=1}^{\infty} a_k$定義為partial sums S_n的極限，不是「把∞代進有限公式」。若極限不存在或無有限值即發散。$a_k \to 0$是收斂必要條件（$a_k = S_k - S_{k-1}$），卻非充分；1/k趨0但harmonic sum發散。

若$\sum_{k=1}^{\infty} |a_k|$收斂，稱絕對收斂，可以任意重排而不改和。條件收斂如$\sum_{k=1}^{\infty} (-1)^{k+1}/k$不保此性：重排會改partial sums的抵消模式。以下無限sum的逐項微積分須有收斂區間，不能把有限線性性無條件擴成任何無限操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

等差和的配對與歸納

$S_n = 1 + 2 + \ldots + n$，倒序同和n+(n−1)+⋯+1；逐項相加有$2S_n = n(n+1)$，故$S_n = n(n+1)/2$，含n=0。

歸納也能核：S_0=0；若$S_n = n(n+1)/2$，則$S_{n+1} = S_n + (n+1) = (n+1)(n+2)/2$。這裡歸納假設的係數固定，不隨n悄悄變動。

一般a+(a+d)+⋯+(a+(n−1)d)=na+d n(n−1)/2。例迴圈j=3..7每次2j−1次工作，和為$(2 \cdot \sum_{j=1}^{7} j - 7) - (2 \cdot \sum_{j=1}^{2} j - 2) = 49 - 4 = 45$；直接5+7+9+11+13=45。改起點要扣掉正確prefix，不是把n=7誤當項數5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平方與立方和從消去式推導

由(k+1)³−k³=3k²+3k+1，k=0..n加總：

$(n+1)^3 = 3\sum_{k=0}^{n} k^2 + 3n(n+1)/2 + (n+1)$。整理得Σk²=n(n+1)(2n+1)/6。

由(k+1)⁴−k⁴=4k³+6k²+4k+1再加總，代入平方和與等差和，得到Σk³=n²(n+1)²/4=[n(n+1)/2]²。兩式n=0也成立。

例n=4：平方0+1+4+9+16=30；公式4・5・9/6=30。立方0+1+8+27+64=100；(4・5/2)²=100。公式是由整段telescoping恆等式推得，核小n只用來排除抄算失誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

等比和：有限、無限、邊界分開

$G_n(x) = \sum_{k=0}^{n} x^k$。把$xG_n - G_n$展開，中間項消去，得$(x-1)G_n = x^{n+1} - 1$；x≠1時$G_n = (x^{n+1} - 1)/(x-1)$。x=1須獨立給$G_n = n+1$，不可除0。

|x|<1時$x^{n+1} \to 0$，$\sum_{k=0}^{\infty} x^k = 1/(1-x)$。x=1時partial sums無界；x=−1時partial sums在1與0間振盪；|x|>1時terms不趨0，所以無有限和。

例x=−1/2、n=3，G=1−1/2+1/4−1/8=5/8；有限公式同值。無限和則2/3，不能把四項和直接寫成無限和。截尾誤差$\sum_{k=n+1}^{\infty} x^k = x^{n+1}/(1-x)$，絕對值$\le |x|^{n+1}/(1-|x|)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

權重等比和與微積分的合法區間

有限多項式G_n可逐項微分。更直接，$W_n=\sum_{k=0}^{n}kx^k$，計W_n-xW_n，得

$(1-x)W_n=x+x^2+\ldots+x^n-nx^{n+1}$，所以$W_n=[x-(n+1)x^{n+1}+nx^{n+2}]/(1-x)^2$（x≠1）。x=1時$W_n=n(n+1)/2$。

|x|<1時$n|x|^n\to 0$（固定底數指數超過多項式），取極限得$\sum_{k=0}^{\infty}kx^k=x/(1-x)^2$。不必未證明就交換無限sum與微分。power series在收斂半徑內可逐項微分／積分；本例半徑1，端點須另查。

再次對收斂半徑內的式子作用x・d/dx，得$\sum_{k=0}^{\infty}k^2x^k=x(1+x)/(1-x)^3$。x=1/2時$\sum_{k=0}^{\infty}k/2^k=2$、$\sum_{k=0}^{\infty}k^2/2^k=6$；不是由「每項趨0」直接推出收斂。

Telescoping：只留下兩端

$\sum_{k=1}^{n}(a_k-a_{k-1})=a_n-a_0$，每個內部$a_k$恰加、減各一次。計算前先對齊索引及端點。

例如1/[k(k+2)]=(1/2)(1/k−1/(k+2))，故

$\sum_{k=1}^{n}1/[k(k+2)]=(1/2)(1+1/2-1/(n+1)-1/(n+2))$。

n=3時1/3+1/8+1/15=21/40；右式(1/2)(3/2−1/4−1/5)=21/40。取n→∞和為3/4，這一步還用了末端分母發散至∞。

乘積也可消去：$\prod_{k=2}^{n}(1-1/k^2)=\prod[(k-1)/k]\cdot\prod[(k+1)/k]=(1/n)\cdot((n+1)/2)=(n+1)/(2n)$，n≥2；空乘積n=1時公式也為1。

乘積轉sum與幾個不能省的前提

若每個$a_k > 0$，$\ln \prod a_k = \sum \ln a_k$；若有0，ln不定義；若有負值，也不能直接用實數ln。例$\prod_{k=1}^{n} 2^{3k+1} = 2^{\sum(3k+1)} = 2^{3n(n+1)/2+n}$。

對$0 \le u_k < 1$，$\ln(1-u_k)$介於$-u_k/(1-u_k)$與$-u_k$（由ln界），可把乘積變成和來控制，但u_k接近1時下界分母會變大，不能當固定常數。

若對所有k≥1有0<c₁f(k)≤h(k)≤c₂f(k)，加總立即得c₁Σf≤Σh≤c₂Σf。若只有尾段成立，須把有限prefix單獨保留；當Σf→∞且prefix有限時不改Θ階。常數必對整段k一致，不可每個k各挑一個再宣稱統一Θ。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.2 Bounding summations

先定要精確值、上界或雙邊階，再選方法。可用歸納、逐項界、固定比率界、切分區間、積分；一個粗上界不等於已找到緊界。

例$S_n = \sum_{k=0}^{n} 4^k$。欲證$S_n \le c4^n$，n0要求c≥1；若$S_n \le c4^n$，則$S_{n+1} \le c4^n + 4^{n+1} = (c/4+1)4^{n+1}$，要$\le c4^{n+1}$需c≥4/3。取固定c=4/3對全部n成立。末項$4^n$給下界，所以$S_n = \Theta(4^n)$。

錯誤歸納「$\sum_{k=1}^{n} k = O(n)$，加n+1仍O(n+1)」隱藏了會長大的常數。若假設Σk≤cn，下一步需cn+n+1≤c(n+1)，即n+1≤c，不可能對任意n成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大項上界與尾段下界

對非負a_k，$\sum_{k=1}^{n} a_k \le n \cdot \max_{1\le k\le n} a_k$；對任何子集合J，$\sum a_k \ge \sum_{k\in J} a_k$。上界可用全體項，下界常只保留夠多的大項。

Σk≤n²。取k=ceil(n/2)..n，至少n/2筆，每項≥n/2，得Σk≥n²/4。對奇偶n同時成立，故Θ(n²)，不必先求精確公式。

固定實數r≥0，$\sum k^r \le n^{r+1}$；同一尾段給$\ge (n/2)(n/2)^r = 2^{-r-1}n^{r+1}$，故Θ$(n^{r+1})$。r固定很重要；若r=n，上下界常數$2^{-r-1}$不再與n無關，不能用此證明宣稱統一階。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定比例才有geometric尾界

假設$a_k \ge 0$且存在固定0<ρ<1，使$a_{k+1} \le \rho a_k$對所有k≥K成立。歸納得$a_{K+j} \le a_K \rho^j$，所以$\sum_{k=K}^{\infty} a_k \le a_K/(1-\rho)$。用乘法不等式寫可涵蓋$a_k = 0$，不需除0。

例$a_k = k^2/3^k$，k≥2時$a_{k+1}/a_k = (1/3)(1+1/k)^2 \le 3/4$。保留k0,1的prefix和1/3，從a₂=4/9起的尾和≤(4/9)/(1/4)=16/9，總和≤19/9；精確加權等比式給3/2，較小但同為有限界。

harmonic的比率k/(k+1)<1，卻趨1；不存在同一ρ<1控制全尾，不能套此法推收斂。若prefix本身依n，例如前兩項各n²，不能把「只有兩項」誤當O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

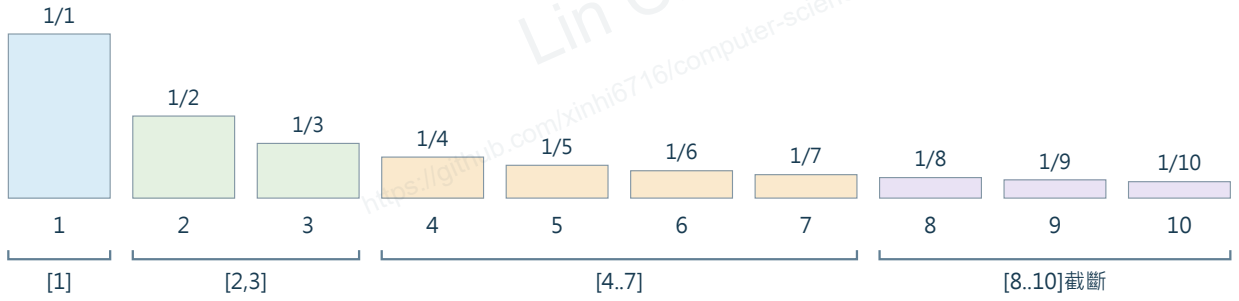
Dyadic分塊看出harmonic的log層數

$H_n = \sum_{k=1}^{n} 1/k$。第j塊$k = 2^j..2^{j+1}-1$有2^j項，每項介於$1/2^{j+1}$與$1/2^j$，所以完整塊貢獻介於1/2與1。

上界補齊最後不完整塊：共有floor(lg n)+1塊，每塊≤1，$H_n \le \text{floor}(\lg n)+1$。下界取前floor(lg n)個完整塊，$H_n \ge \text{floor}(\lg n)/2$，n≥2時為Ω(lg n)。兩界一起給Θ(lg n)，也證無限harmonic發散。

例n=10：區塊[1]、[2,3]、[4,5,6,7]、[8,9,10]，實際貢獻1、5/6、319/420、121/360；總H_{10}=7381/2520≈2.928968。最後一塊只有3項，不應誤列為8項。

n=10：每個完整dyadic塊的和介於1/2與1



積分界由每個單位區間證明

若f在[m−1,n+1]可積且單調不減，對每整數k有$\int_{k-1}^{k} f(x)dx \le f(k) \le \int_{k}^{k+1} f(x)dx$；加總得$\int_{m-1}^{n} f \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f$。

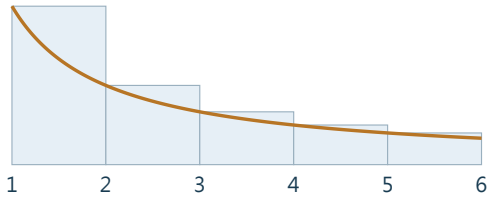
若f單調不增，各不等式反向：$\int_{m}^{n+1} f \le \sum f(k) \le \int_{m-1}^{n} f$。這裡不是把sum任意換成integral；左右端點的位移是控制誤差所需。

例如f=1/x在x>0遞減。下界$H_n \ge \int_{1}^{n+1} dx/x = \ln(n+1)$。上界不能從0積起，因1/x在0奇異；先取首項1，再對k2..n積分，得$H_n \le 1 + \int_{1}^{n} dx/x = 1 + \ln n$。

積分界由每個單位區間證明 （續）

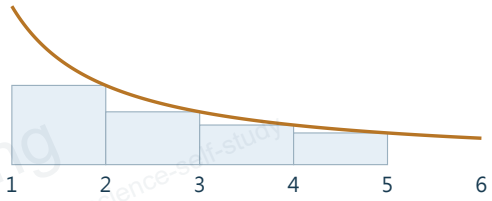
遞減f(x)=1/x：矩形高是1/k，兩邊的區間不同

下界：$\int_1^6 1/x\,dx \le H_5$



每個[k,k+1]：曲線≤1/k

上界：$H_5 - 1 \le \int_1^5 1/x\,dx$



每個[k−1,k]：曲線≥1/k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可重算的積分上下界

n=5時H_5=137/60≈2.283333，ln6≈1.791759≤H_5≤1+ln5≈2.609438。夾界有可量化寬度，不是宣稱sum等於ln n。一般$\ln(n+1) \le H_n \le \ln n + 1$，所以$H_n = \ln n + O(1)$。

對p>1，$\sum_{k=1}^{n} 1/k^p \le 1 + \int_1^n x^{-p}dx \le 1 + 1/(p-1)$，單調有界partial sums收斂。p=1發散如前；0≤p<1，積分下界隨n^{1-p}增長，和為$\Theta(n^{1-p})$，不是所有terms趨0都收斂。

對$f = x^r$、固定r≥0，積分給$n^{r+1}/(r+1) \le \sum_{k=1}^{n} k^r \le [(n+1)^{r+1} - 1]/(r+1)$，再次得$\Theta(n^{r+1})$，也說明模型中的常數可依固定r。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙重迴圈與改變加總次序

程式for i=1..n、for j=i..n，每次常數工作，次數$\sum_{i=1}^{n}(n-i+1)=n(n+1)/2$。也可依j計：每j有i=1..j，共$\sum_{j=1}^{n} j$。兩者數的是同一個索引集合{(i,j):1≤i≤j≤n}。

若每次工作j，則$\sum_{i=1}^{n}\sum_{j=i}^{n} j=\sum_{j=1}^{n}\sum_{i=1}^{j} j=\sum j^2=n(n+1)(2n+1)/6$。n=3各row和6、5、3，共14；按column1・1+2・2+3・3亦14。

這裡有限集合任意重排安全；無限雙重sum若含正負項，必先證絕對收斂，不能只因有限情況可換序就直接照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B 集合、關係、函數、圖與樹

Part VIII Appendix

本章定位

以逐元素邏輯與一般證明串聯sets、equivalence partitions、functions、graphs與trees；明列圖型/路徑/方向/空集合及binary slots慣例，完整證tree六等價與二分圖判準。

前置知識

基本離散數學符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 正確使用集合、關係與函數的定義。
- 辨識 graph 與 tree 的結構性質。
- 連結笛卡兒積、等價關係與遞迴結構。
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.1 Sets

set是不重複、無順序的元素集合，{u,v,u}={v,u}；multiset另記multiplicity，sequence另記位置，三者不能互換。x∈A表示元素，A⊆B表示對每個x，x∈A⇒x∈B；A⊊B另要求不相等。∅⊆A對任何A成立，∅∈A則未必；{∅}有一個元素，並非空集合。

集合相等須A⊆B且B⊆A。自然數本章採N={0,1,⋯}，Z為整數，R為實數。寫{x∈Z:x可被3整除}是用性質限定既有集合，不是排序後的list。

集合運算的定義是逐元素邏輯：x∈A∪B iff (x∈A or x∈B)，交集用and，差A\B用(x∈A and x∉B)。complement必指定universe U，A^c=U\A；更換U會更換complement。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

集合恆等式是對任意元素的命題

性質	兩個對偶版本
單位／吸收	A∪∅=A；A∩∅=∅
冪等	A∪A=A；A∩A=A
交換／結合	A∪B=B∪A；A∪(B∪C)=(A∪B)∪C，交集同理
分配	A∩(B∪C)=(A∩B)∪(A∩C)；∪/∩互換亦成立
吸收	A∩(A∪B)=A；A∪(A∩B)=A
De Morgan	$(B\cap C)^c=B^c\cup C^c$；$(B\cup C)^c=B^c\cap C^c$

例如$x \in A \setminus (B \cap C)$ iff x∈A且非(x∈B且x∈C)，iff (x∈A且x∉B)或(x∈A且x∉C)，iff $x \in (A \setminus B) \cup (A \setminus C)$。這是任意x的雙向等價；只畫一個有限Venn例子不代替全稱證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partition、cardinality與重疊的計數

partition是非空blocks的集合：兩兩不交且聯集為S，使每個x∈S恰屬一塊。S=∅時允許空blocks集合，條件皆成立。例S={0,⋯,7}按mod3分為{0,3,6}、{1,4,7}、{2,5}。

對有限A,B，將A∪B拆成互斥的A\B、A∩B、B\A，可得|A∪B|=|A|+|B|−|A∩B|。例A={0,2,4,6}、B={2,3,6}，4+3−2=5；不能只把cardinalities相加而重複計交集。

三集合公式加三個單集合、減三個pair intersections、加triple intersection。一般inclusion-exclusion可逐元素證：一個落在r≥1個集合的元素係數為$\sum_{j=1}^{r}(-1)^{j+1}\binom{r}{j}=1$，因$(1-1)^r=0$。未落任何集合者係數0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Power set與Cartesian product的不同分母

power set P(S)={T:T⊆S}。若S有n個可區分元素，一個subset對每個元素只選入／不入，與n-bit字串一一對應，故$|P(S)|=2^n$；P(∅)={∅}仍有1筆。

ordered pair(a,b)保留位置，一般不同於(b,a)。A×B={(a,b):a∈A,b∈B}；固定a有|B|個pair，|A|個互斥rows給|A×B|=|A||B|。n-fold product計ordered n-tuples，有限集合時大小為各大小乘積；A^0含唯一空tuple，大小1。

例如A={p,q}、B={0,1,2}，A×B有6筆；P(A)只有∅,{p},{q},{p,q}四筆。sequence是由位置到值的function；相同值在不同位置可重複，不會像set自動合併。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無限集合的大小要靠bijection

兩集合等cardinality指存在bijection，不是「兩個都無限就一樣多」。N到Z可設f(0)=0，f(2k−1)=k、f(2k)=−k（k≥1）；每個正負整數各有唯一原像，所以Z可數。N到偶數的2n也是bijection，無限集合可與自己的真子集等大。

R不可數可用diagonal argument：若(0,1)能列為$x_1, x_2, \cdots$，每個取不以無窮9結尾的十進位表示。建y的第i位：若x_i第i位不是1取1，否則取2。y∈(0,1)，只用1/2而無表示歧義，卻與每個x_i至少在第i位不同，矛盾。

有限集合的「注入且同大小即滿射」不能原封套到無限集合；f(n)=n+1在N上注入卻漏0。定理要清楚寫finite前提。

B.2 Relations

binary relation R⊆A×B；在單一A上則R⊆A×A。aRb表示(a,b)∈R，沒有先要求每個a都有對象，也沒有要求唯一對象。

性質	量詞：適用所有a,b,c∈A
reflexive	aRa
symmetric	aRb⇒bRa
antisymmetric	aRb且bRa⇒a=b
transitive	aRb且bRc⇒aRc

symmetric與antisymmetric不是互為否定：相等關係同時滿足兩者；含a↔b與b→c卻沒有c→b的relation可兩者都不滿足。空relation在非空A上對稱、反對稱、遞移皆真，卻不reflexive；不能憑「對稱且遞移」擅自假設aRb存在。

Equivalence relation如何產生partition

equivalence relation同時reflexive、symmetric、transitive。定[a]={b∈A:aRb}。由aRa，[a]非空且各a被覆蓋。若c∈[a]∩[b]，則aRc、bRc，對稱及遞移給aRb；任意x∈[a]有bRaRx，故x∈[b]；反向同理，所以兩類只可能相同或不交。去掉重複類後便是partition。

反向給partition，定aRb iff a,b在同一block。每a恰在一塊，故reflexive；同塊是symmetric；a,b同塊且b,c同塊，由block唯一性三者同塊，故transitive。a所在block正是[a]，兩個建構互相對應。

例如mod3等價：a−b為3的倍數。0=3・0證反身，負號證對稱，相加證遞移；classes是餘數0,1,2三類，限制到有限S時得到前述三blocks。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial order、total order與極大／最大

partial order是reflexive、antisymmetric、transitive；total order再要求每對a,b可比較。total preorder只要求reflexive、transitive與任兩者可比較，允許不同元素互相等級相同，不必antisymmetric。

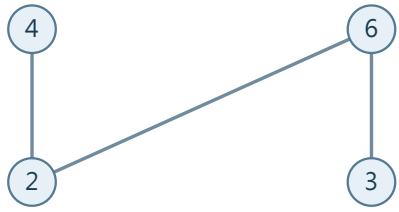
在poset內，maximum M滿足每個x≤M；maximal m只要求沒有嚴格更大的元素。minimum／minimal對偶。maximum若存在必唯一且maximal：若M,N都是maximum，M≤N且N≤M，由反對稱得相等；反向maximal不保證maximum。

S={2,3,4,6}以整除為≤。minimal={2,3}、maximal={4,6}，沒有minimum或maximum。Hasse圖只畫cover：x<y且沒有z使x<z<y；邊向上表示較大，省掉反身與傳遞邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial order、total order與極大／最大 （續）

S={2,3,4,6}：整除偏序的Hasse圖



向上：被整除的較大元素

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

沒有minimum / maximum

線段只表示cover，不代表數值距離；反身與傳遞關係不另畫邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.3 Functions

function f:A→B是在relation中要求每a∈A**恰有一個**b∈B的特例。A是domain，B是codomain；range f(A)可能只是B的子集。partial function若有a未定義，就不符合這裡的total function定義。

injection要求f(a)=f(a′)⇒a=a′；surjection要求對每b∈B存在a使f(a)=b；bijection同時滿足。例 f:N→N、f(n)=2n，注入非滿射；把codomain改成非負偶數，便是bijection。f(n)=floor(n/2)在N→N滿射非注入（0,1同映0）。

有限A,B，注入的不同輸入占不同codomain元素，故|A|≤|B|；滿射須每b至少一個原像，故|A|≥|B|。若有限且同大小，一個條件便推出另一個；無限時前頁已有反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverse、image與composition的條件

對bijection f:A→B，f^{-1}(b)是唯一a使f(a)=b，並有$f^{-1} \circ f = id_A$、$f \circ f^{-1} = id_B$。若只injection，inverse只能定義在range；若非injection，relational inverse一般不是function。

set preimage f^{-1}(T)={a:f(a)∈T}對任意function及T⊆B都可定義，不要求可逆。它與「inverse function」同符號但不是同一個物件。

f:A→B、g:B→C的composition為(g∘f)(a)=g(f(a))。若f,g均注入，g(f(a))=g(f(a′))先取消g再取消f，得a=a′；若均滿射，給c先找b再找a，得composition滿射。兩者均bijection時，$(g \circ f)^{-1} = f^{-1} \circ g^{-1}$，順序反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.4 Graphs

本章有限directed graph G=(V,E)，E⊆V×V，可有self-loop但不記parallel edges；undirected simple graph的E是不同頂點的unordered pairs，沒有self-loop。若用multigraph須另保每條edge identity及loop的度數計法。

directed edge(u,v)從u出、到v入；outdegree數出邊，indegree數入邊，self-loop各計一次。undirected degree數incident edges。每無向邊恰貢獻兩個端點，故Σdeg=2|E|；有向Σoutdeg=Σindeg=|E|。因此無向奇度頂點個數必偶數。

同一張圖換頂點位置或畫成曲線不改E。兩條線交叉處除非明畫頂點，並非可轉向的節點；edge方向、頂點identity與數值須由圖例或edge list確定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Walk、simple path與cycle的約定

walk是一串$v_0, \ldots, v_k$，每相鄰一對有edge，長度k數邊不數點；CLRS的path允許重複，對應這裡walk。simple path要求所有頂點不同；u到自己的長0路徑存在。

本章談acyclic時，undirected cycle指長≥3、除起終點外頂點不重複的封閉路徑；只沿同一邊u→v→u往返不是cycle。directed simple cycle可長1（self-loop）或2（雙向邊），方向須逐邊成立。一般closed walk可有重複，不能與simple cycle混為同一物件。

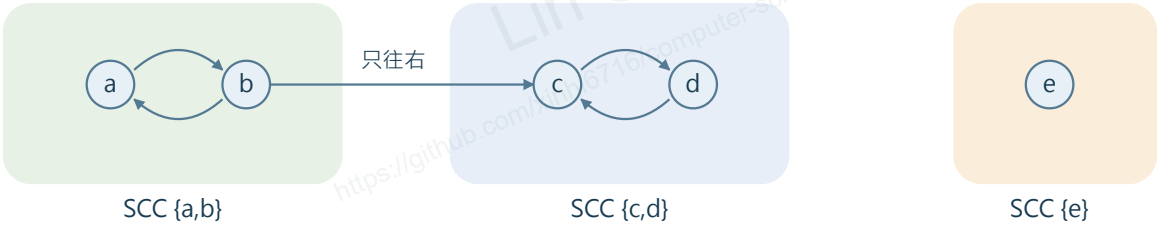
若u到v有walk，遇重複頂點$v_i = v_j$，刪去兩次出現之間的閉段仍連成walk且更短；反覆刪除，有限長度降到simple path。這個一般消去論證使「可達」不必靠無限列出walk判定。

連通分量、SCC與方向

無向可達關係reflexive（長0）、symmetric（倒走）、transitive（串接），所以components是equivalence classes。有向可達仍反身、遞移，但可能不對稱；mutual reachability則三性皆具，classes稱SCC。

原創digraph：a→b、b→a、b→c、c→d、d→c，另e孤立。SCC為{a,b}、{c,d}、{e}；存在a→d卻沒有d→a，故不能把兩個SCC合併。忽略方向的版本只有{a,b,c,d}與{e}兩個components。

可達不等於互相可達；每條箭頭都有指定方向



a可達d，d不可達a。忽略方向後a/b/c/d同一component，e仍孤立。

Subgraph、isomorphism與contraction

subgraph可取部分頂點與部分原edges，edges兩端仍須在所取頂點。induced subgraph G[W]則必保留所有兩端都在W的原edges，不能挑掉不喜歡的一條。例前圖W={a,b,c}的induced edges恰a→b,b→a,b→c。

isomorphism是bijection f:V→V′且(u,v)∈E iff (f(u),f(v))∈E′；只保一方向不足。頂點數、邊數、度數multiset相同是必要條件，不是充分：六頂點cycle C_6與兩個互斥三角形皆6邊、各degree2，但一個連通、一個不連通。

contract無向edge{u,v}時將端點合為新x，其他incident edges改接x，刪被縮edge；若保持simple graph表示，同一鄰居產生的重邊合一、自loop刪除。若演算法需保parallel edges（如某些隨機收縮），必明用multigraph，不能默默去重。

常見圖類與二分圖的一般判準

complete graph K_n每對不同頂點相鄰，|E|=n(n−1)/2。bipartite有V=L⊔R且每條edge跨兩側，等價可用兩色proper coloring。DAG是directed acyclic graph；tree／forest是無向圖概念，不能只看「箭頭沒有環」就叫tree。

二分圖不可能有奇cycle：沿cycle每跨一邊換側，奇次不能回到同一側。反向若無奇cycle，在每個component取根，依最短無向距離的奇偶分兩側。若edge(u,v)同奇偶，根到u、該edge、v回根構成奇長closed walk；任意奇closed walk若有重複頂點可拆為兩個closed walks，至少一個奇，反覆縮短會得到奇simple cycle，矛盾。

hypergraph的一條hyperedge可含多個頂點；可建incidence bipartite graph，一側原頂點、一側hyperedges，membership成為跨側edge。這個圖的節點種類與原圖不同，不能把一條hyperedge直接當普通pair。

B.5 Trees

本節free tree要求n=|V|≥1、simple undirected、connected且沒有simple cycle。forest只要求acyclic，可有多個components與isolates；空forest允許，空圖不作本節tree六等價條件的輸入。

一個n≥2的tree必有leaf：取最長simple path的一個端點v。若v另有不在路徑的鄰居，可延長；若另接路徑內非相鄰頂點，便成cycle，皆矛盾。故v只有路徑上的一個鄰居，degree1。

移除leaf及其唯一邊後仍connected（其他點間simple path不用leaf作內部點）且acyclic。歸納n1有0邊，tree n筆有n−1邊。forest若有c個components，每個component是tree，逐塊相加得$|E| = \sum(n_i - 1) = n - c$（包含空forest n=c=0）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六個條件為什麼等價：先看路徑與邊

對非空有限simple無向圖，下列六項等價：①tree；②每對頂點恰有一條simple path；③connected且刪任何edge都斷；④connected且m=n−1；⑤acyclic且m=n−1；⑥acyclic且加任何缺少的edge都成cycle。

①⇒②：連通給至少一條；若兩條不同simple paths，取第一次分歧與之後第一次會合的兩段，內部不交，合起來成cycle，矛盾。②⇒③：edge{u,v}本身就是唯一u-v路徑，刪後無路。

③⇒①：若有cycle，刪cycle一邊仍可沿其餘部分替代任何使用該邊的路徑，所以不斷，矛盾。①⇒④與⑤由前頁n−1邊證明。這些推論不把「有一條路」誤當「唯一一條路」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六個條件的其餘方向與可重建構造

任何connected graph可建spanning tree：從一頂點S開始，只要S≠V，由連通性必有跨S與V\S的edge；加該edge與一個新頂點。每步只接一個新點，不成cycle；n−1步後得到n−1邊的tree，故一般connected graph m≥n−1。

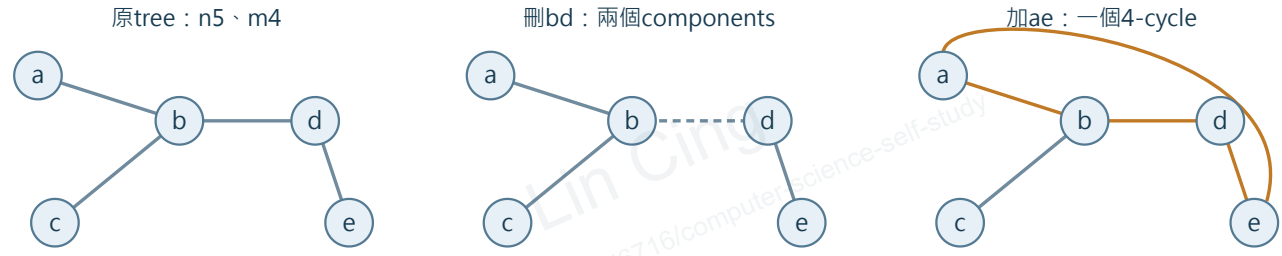
④⇒①：上述spanning tree已用盡G的n−1邊，G無額外邊，故就是tree。⑤⇒①：由forest的m=n−c及m=n−1得c=1。①⇒⑥：任一新edge接u,v，與tree中唯一u-v路徑形成cycle。⑥⇒①：若有兩個不同components，加入跨component edge不可能成cycle，與條件矛盾；故connected。

六個方向已封閉。注意「m=n−1」單獨不夠：三角形加一個isolated vertex有n4,m3卻不是tree。相反「acyclic」也不保connected。

同一張圖的tree、非tree與新增cycle

原創tree V={a,b,c,d,e}，E={ab,bc,bd,de}，n5,m4。唯一a到e的simple path為a,b,d,e，長3。刪bd切成{a,b,c}與{d,e}；加入ae則形成a,b,d,e,a的4-cycle。

同一組頂點：刪tree邊必斷，加缺邊必成cycle



中圖虛線是被刪掉的邊；右圖橙色cycle沿a–b–d–e再走新邊e–a返回。

這裡有cycle時仍可能有spanning tree，只是圖本身不再是tree；minimum spanning tree的加權最佳化還需額外目標，並不是tree定義的一部分。

Rooted與ordered：高度、深度與degree要說慣例

從free tree指定root r後，每個非根x的parent是唯一r-x simple path中前一點；其餘向下鄰居是children。ancestor／descendant本章包含自己，proper才排除自己；subtree rooted at x取x全部descendants。

depth(x)=根到x的邊數，height(x)=x到最深leaf的邊數；leaf height0，root depth0，tree height=max depth。rooted degree數children；free-tree graph degree還包含parent，故非根兩者差1。

若前頁tree以b為root，children a,c,d，d另有child e；depth(a,c,d)=1、depth(e)=2，height(b)=2、height(d)=1，其他0。b graph degree與children均3，d graph degree2但children1。

ordered tree另外定每節點children先後；把b的(a,c,d)改成(c,a,d)，rooted關係同但ordered tree不同。

Binary／positional tree與空child

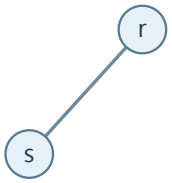
binary tree遞迴定義為空樹NIL，或一個root加互斥的left/right binary subtrees；只一個child時left與right仍不同。ordered tree只有一個child時沒有左slot／右slot之別，所以「degree≤2的ordered tree」不足以完全描述binary tree。

positional k-ary tree有固定slots1..k，空slot也帶位置資訊。將binary tree每個缺少child替成external NIL leaf，可得到每個internal node恰有兩個children的full binary tree，保留原left/right位置。

Binary／positional tree與空child （續）

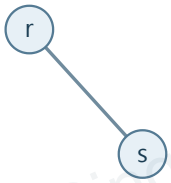
空slot也有位置：單一child不能省掉left/right資訊

只有left child



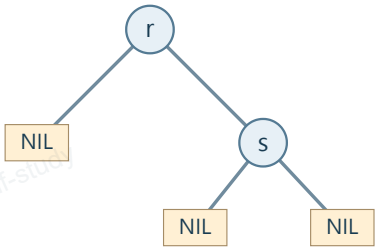
binary位置不同

只有right child



binary位置不同

右圖擴充：I2、L3



若原binary tree非空且有n≥1個實體nodes，總child slots2n，實體parent-child edges n−1，缺少slots=2n−(n−1)=n+1；擴充表示就有n個internal與n+1個external leaves。NIL sentinel若共用單一物件，邏輯空位置數仍是n+1，不等於實際配置n+1物件。空樹沒有實體父節點，故上述(parent,side) child slots為0；若另外表示空根的NIL位置，那是另一種計數。

Full、complete與height的精確計數

full k-ary tree每個internal恰k個children，leaves可不同深度。以I internal、L leaves計edge：kI=I+L−1，故L=(k−1)I+1。k2時L=I+1。

CLRS此附錄的complete k-ary另要求全部leaves同深h（常有文獻稱perfect）；對k≥2，第d層k^d個nodes，leaves數 $L = k^h$，internal數 $I = \sum_{d=0}^{h-1} k^d = (k^h - 1)/(k - 1)$。例k3,h2：levels1,3,9，I4,L9、總13，edges12=3I。k1是鏈，須另算I=h、L1，不可除k−1=0。

heap章節常稱complete binary為「末層可不滿、由左填滿」，與這裡all-leaves-same-depth慣例不同，使用時須看定義。一般非空binary tree高h至多$\sum_{d=0}^{h} 2^d = 2^{h+1} - 1$個nodes，所以h≥ceil(lg(n+1))−1；最斜鏈h=n−1，沒有平衡前提不能聲稱所有binary tree高Θ(lg n)。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C 計數與機率

Part VIII Appendix

本章定位

從sample object與統一分母建立計數、條件機率、Bayes及指標分析，再推完整geometric/binomial矩與尾界；明定獨立層級、零機率/支撐邊界及中心化r和總門檻t的區別。

前置知識

排列組合、機率公理與隨機變數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用乘法、加法與補集計數。
- 計算期望與變異數，處理幾何與二項分布。
- 以 Chernoff bound 約束二項尾機率。
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.1 Counting

先定「一個結果」包含哪些資訊，再計數。sum rule要求互斥分類；product rule要求每個prefix後的選擇數已知。選擇集合可依先前結果改變，只要每層分支數固定，仍可相乘；若分支數不固定就逐branch加總，不能硬乘平均。

原創例：從6名學生選兩個不同職位，第一職6選、第二職5選，共30；若只是兩人committee、不分職位，每組被計兩個先後，除2得15。若允許同一人兼任，則變6^2=36。三個答案對應三種不同sample object。

長m、alphabet大小q的strings有q^m，允許重複且保位置。substring須連續，subsequence可跳位置；同一內容在不同起點是否算不同，必先約定。長n字串的長k≥1位置型substrings有max(0,n−k+1)個，不等於不同內容種類數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permutations、combinations與不重複計數

相異n筆全部排列有n!；ordered選k筆無重複有P(n,k)=n!/(n−k)!，0≤k≤n。unordered k-subset每組恰對應k!個ordered結果，故$\binom{n}{k}$=n!/[k!(n−k)!]。k0唯一空選擇，$\binom{n}{0}$=1；k<0或k>n可擴充定為0。

若records可區分但keys重複，排列records仍n!；若只看key字串、各multiplicity為$m_1, \ldots, m_s$，則每key字串對應$\prod m_i!$種identity排列，故有$n!/\prod m_i!$個不同key字串。除法前須證每個結果被重複相同次數。

$\binom{n}{k}=\binom{n}{n-k}$由取補集bijection得。Pascal identity $\binom{n}{k}=\binom{n-1}{k}+\binom{n-1}{k-1}$，依某個指定元素不在／在subset分為兩個互斥且完備的類別。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二項式展開與entropy界的來源

展開$(x+y)^n$時，每個factor選x或y；恰k個位置選x共有$\binom{n}{k}$種，故$(x+y)^n=\sum_{k=0}^{n}\binom{n}{k}x^k y^{n-k}$。x=y=1得$\sum\binom{n}{k}=2^n$，這也是按subset大小分類的計數。

對1≤k≤n，$\binom{n}{k}=\prod_{j=0}^{k-1}(n-j)/(k-j)\geq(n/k)^k$，因n≥k使每個factor≥n/k。上界$\binom{n}{k}\leq n^k/k!\leq(en/k)^k$，最後用$k!\geq(k/e)^k$。

另令α=k/n∈(0,1)，展開$(\alpha+1-\alpha)^n=1$，其中一項非負且≤1，故$\binom{n}{k}\leq\alpha^{-k}(1-\alpha)^{-(n-k)}=2^{nH(\alpha)}$，H(α)=−α lgα−(1−α)lg(1−α)。端點以0lg0=0連續延伸，$\binom{n}{0}=\binom{n}{n}=1$。H′=lg((1−α)/α)，最大在1/2，值1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.2 Probability

sample space S的elementary outcome是一個完整可區分結果，event是合法可測子集。有限／可數離散情況本章取所有subsets為events；不可數情況須指定對補集與可數聯集封閉的event族，不能無條件把每個subset都當event。

機率滿足P(A)≥0、P(S)=1，以及對兩兩互斥的有限／可數events，有$P(\cup A_i)=\sum P(A_i)$。因此P(∅)=0、P(A^c)=1−P(A)、A⊆B⇒P(A)≤P(B)。離散時$P(A)=\sum_{s\in A}P(s)$。

有限S只有在每個outcome等機率時才可P(A)=|A|/|S|。例如丟兩個公平且獨立的0/1 bits，四個ordered結果各1/4；把它們壓成「0、1、2個1」後三種計數不是等機率，而是1/4、1/2、1/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

聯集、union bound與不需要的獨立性

由互斥分解A∪B=(A\B)⊔(A∩B)⊔(B\A)，P(A∪B)=P(A)+P(B)−P(A∩B)≤P(A)+P(B)。多事件定B₁=A₁、$B_i = A_i \setminus (\bigcup_{j<i} A_j)$，$B_i$互斥、聯集相同且$B_i \subseteq A_i$，故$P(\cup A_i) = \sum P(B_i) \leq \sum P(A_i)$，可數版本也成立。

此union bound不需獨立；右側可大於1，實用時再取min(1,右側)。相反若要把$P(\cap A_i)$寫成$\prod P(A_i)$，必須有mutual independence，不能因為union bound不需獨立就混用。

例三個failure events各≤0.01，不論相關性，至少一個失敗機率≤0.03。若三個事件其實完全相同，聯集機率0.01而不是1−0.99³；後式需要獨立前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditional probability與chain rule

P(A|B)=P(A∩B)/P(B)，要求P(B)>0；這是在保留B內outcomes後重新normalize。P(A∩B)=P(B)P(A|B)。反覆應用得$P(\cap_{i=1}^n A_i) = P(A_1) \prod_{i=2}^n P(A_i | A_1 \cap \ldots \cap A_{i-1})$，各conditional使用時須分母正；若某prefix機率0，整交集已為0，不能再寫未定義的比值乘積。

原創抽樣：袋中3藍2紅，無放回連抽兩球，P(兩藍)=(3/5)(2/4)=3/10；第二次藍的邊際仍3/5，但在第一次藍的條件下只有1/2，故兩次不獨立。若放回並重新均勻抽取，才是(3/5)²。

知道「至少一事件發生」與知道「指定那一次發生」通常不同。描述觀測或篩選規則是機率模型的一部分，不可只看到剩兩種文字敘述就自動各給1/2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence、互斥與pairwise的差異

A,B獨立指P(A∩B)=P(A)P(B)，這個定義也涵蓋機率0 events；若P(B)>0才可等價寫P(A|B)=P(A)。互斥指交集空，若兩者機率皆正則不獨立。空event則與任意event獨立。

mutual independence要求每個有限子集合的交集機率都等於邊際乘積；pairwise只核每一對，不足以乘三個以上。

取U,V為公平獨立bits，W=U XOR V，outcomes(0,0,0),(0,1,1),(1,0,1),(1,1,0)各1/4。U,V,W每對的四種組合都各1/4，故pairwise independent；但P(U=V=W=1)=0，而三邊際乘積=1/8，非mutual。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence、互斥與pairwise的差異 (續)

同一張完整表：pairwise independent，非mutually independent

U	V	W=U XOR V	P(outcome)
0	0	0	1/4
0	1	1	1/4
1	0	1	1/4
1	1	0	1/4

任一對：00/01/10/11各1/4

三個邊際P(=1)都為1/2

但111根本不在sample space

$0 \neq (1/2)^3 = 1/8$

每對核對成立，不代表三者交集可直接乘邊際；這裡variance可加，三重乘積不可分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Total probability與Bayes的分母

若B_i形成S的可數partition且$P(B_i)>0$，由$A=\sqcup(A\cap B_i)$，$P(A)=\sum P(B_i)P(A|B_i)$。Bayes再給$P(B_j|A)=P(B_j)P(A|B_j)/P(A)$，須P(A)>0。

原創監測器模型：故障prior P(F)=0.02；有故障時發alarm機率0.9，無故障時誤報0.1。則P(alarm)=0.02・0.9+0.98・0.1=0.116；posterior P(F|alarm)=0.018/0.116=9/58≈0.155172。

alarm的高sensitivity0.9不等於posterior0.9；大量正常樣本提供0.098誤報分母。兩次測試若只在給定F／F^c後獨立，雙alarm機率=0.02・0.9^2+0.98・0.1^2=0.026，posterior=0.0162/0.026=81/130≈0.623077；沒有先給條件獨立，不能平方likelihood。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Continuous uniform不給每點正機率

X在[a,b]連續uniform、a<b時，P(c≤X≤d)=(d−c)/(b−a)，a≤c≤d≤b。單點c=d機率0，但整區間機率1；可數可加性不要求對不可數個單點直接相加。

例[a,b]=[2,8]，P(3≤X≤5)=2/6=1/3，去掉兩端點結果相同。若Z=floor X，Z的值2..7各機率1/6，而Z=8只在單點X8出現，機率0；不得把七種整數標籤平均分配。

接下來主講離散random variables及pmf。離散機率質量P(X=x)與連續density不同：density可大於1，單點機率仍0。對連續分布應積分區間，不能把density當成每點機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.3 Discrete random variables

random variable X:S→R是outcome的數值函數，event{X=x}是其preimage。pmf $p_X(x) = P(X = x)$，非負且$\sum_x p_X(x) = 1$；本章用「probability mass function」，避免把離散質量與連續density混淆。

joint pmf p(x,y)=P(X=x,Y=y)，marginals為$p_X(x) = \sum_y p(x, y)$、$p_Y(y) = \sum_x p(x, y)$。給$p_Y(y) > 0$，conditional pmf為$p(x, y)/p_Y(y)$。X,Y獨立當且僅當所有x,y都有$p(x, y) = p_X(x)p_Y(y)$。

原創：兩個獨立公平四面骰A,B∈{1,2,3,4}，令M=max(A,B)。P(M≤m)=$m^2/16$，P(M=m)=[m^2−$(m-1)^2$]/16=(2m−1)/16，m1..4質量1,3,5,7除16；不同M值不是等機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expectation是加權和，不必是可能結果

$E[X] = \sum_x x p_X(x)$。有限支撐必可算；可數支撐要E|X|<∞才有有限且可重排的有號期望。非負X允許E[X]=∞；不能把∞−∞當合法期望運算。

對四面骰最大值M，E[M]=(1・1+2・3+3・5+4・7)/16=50/16=25/8；E[M^2]=(1+12+45+112)/16=170/16=85/8。期望3.125不在整數支撐，仍是正確平均。

任意g，只要E|g(X)|<∞，按每個X值的preimage分類得到$E[g(X)] = \sum_x g(x)p_X(x)$，不用先重新列g(X)的所有不同值。非線性時一般E[g(X)]≠g(E[X])；平方的差就是variance。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性期望的完整推導與indicator

若X,Y可積，從joint pmf有$E[X+Y]=\sum_x\sum_y(x+y)p(x,y)=\sum_x x\sum_y p(x,y)+\sum_y y\sum_x p(x,y)=E[X]+E[Y]$；絕對可積允許拆與換序，沒有使用獨立。E[aX+b]=aE[X]+b。有限多變數逐次套用。

indicator I_A在A發生為1，否則0，$E[I_A]=P(A)$。若X計某些events發生次數，$X=\sum I_i$，故$E[X]=\sum P(A_i)$，即使相依也成立。例uniform排列n個相異keys，inversion指標每對期望1/2，總n(n−1)/4。

若X,Y獨立且可積，joint分解後$E[XY]=(\sum xp_X)(\sum yp_Y)=E[X]E[Y]$。這次用到獨立；若X=Y是公平bit，E[XY]=E[X]=1/2≠1/4。多項乘積通常需mutual independence，而非只pairwise。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail-sum與Jensen的理由

非負整數X逐outcome有$X=\sum_{i\geq 1}IX\geq i$。有限截尾先求期望再單調取極限（每項非負），得$E[X]=\sum_{i\geq 1}P(X\geq i)$，可含∞。若X可以負值，不能直接用這個式子。

convex f滿足f(λx+(1−λ)y)≤λf(x)+(1−λ)f(y)，0≤λ≤1。有限支撐以此二點定義歸納：把前m−1個權重normalize成一點，與最後一點再套凸性，得到f(E[X])≤E[f(X)]。

一般可積情況，若f在包含支撐及μ=E[X]的區間凸，μ有supporting line f(x)≥f(μ)+a(x−μ)，取期望即E[f(X)]≥f(μ)；端點退化情況X必集中該端點。需兩側期望有定義；這不是任意函數都可把f移入／移出E的規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Variance、covariance與相依成本

若E[X²]<∞，μ=E[X]，Var(X)=E[(X−μ)²]=E[X²]−μ²≥0；σ=sqrt(Var)為standard deviation。Var(aX+b)=a²Var(X)，不是乘a。前例M的Var=85/8−(25/8)²=55/64，σ=sqrt55/8。

Cov(X,Y)=E[XY]−E[X]E[Y]，展開平方得Var(X+Y)=VarX+VarY+2Cov。n項時$Var(\sum X_i) = \sum VarX_i + 2\sum_{i<j} Cov(X_i, X_j)$。pairwise independent且二次矩有限足以使covariances全0；不需mutual independent。

但只同分布不足。若$X_1 = \ldots = X_n$ =同一公平bit，sum只有0或n，各1/2，variance=n²/4；逐項variance相加只有n/4。缺少covariance會把不確定性嚴重低估。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov與Chebyshev：各自的適用條件

若X≥0且E[X]<∞，對t>0逐點有X≥t I{X≥t}，取期望得P(X≥t)≤E[X]/t（Markov）。不需獨立，但不能套負值變數；有正負抵消時平均小不表示正尾小。

對二次矩有限Y，取非負X=(Y−μ)²與t=a²，a>0，得到P(|Y−μ|≥a)≤Var(Y)/a²（Chebyshev）。它控制雙尾；若只要一側也可用同上界，但可能寬鬆。

例如四面骰最大值M的μ25/8、variance55/64，a=2，雙尾≤55/256≈0.214844；真事件只有M1，機率1/16。上界不是宣稱真機率等於bound；超過1的界可取1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.4 The geometric and binomial distributions

Bernoulli(p)取1（success）機率p、0機率q=1−p。$X^2=X$，故E[X]=p，Var=p−p^2=pq。說一串Bernoulli trials時，以下另要求mutual independence與相同p；只有各次邊際p不夠。

geometric本章與CLRS採**包含首次成功那次的trial數**G∈{1,2,⋯}，0<p≤1。G=k要求先k−1次失敗、再成功，$P(G=k)=q^{k-1}p$；p0沒有成功，G=∞，不是這個有限值geometric分布。

若別處採「成功前失敗次數」F=G−1，支撐從0開始、pmf為$q^f p$、E[F]=q/p；不能把兩種mean都寫1/p。p1時G≡1、F≡0，公式與variance0仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric的和、期望與variance

$\sum_{k\ge1} q^{k-1}p = p/(1-q) = 1$。$P(G>t)=q^t$（整數t≥0），所以$P(G\ge i)=q^{i-1}$；tail-sum得E[G]=1/(1−q)=1/p。

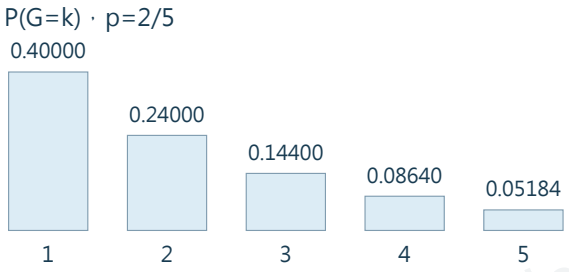
由加權等比公式可得$E[G^2]=(1+q)/p^2$：例如$\sum_{k\ge1} k^2q^{k-1} = (1+q)/(1-q)^3$，乘p即得。Var=$(1+q)/p^2-1/p^2=q/p^2$。q0情況直接由G1處理，不做除q。

原創p=2/5，k1..5的pmf依次2/5、6/25、18/125、54/625、162/3125；總2882/3125，剩尾$P(G>5)=(3/5)^5=243/3125$。E[G]=5/2、Var=15/4。期望不是「第2.5次真的成功」，而是多次重做此完整實驗的平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric的和、期望與variance （續）

先固定計數慣例，再計pmf、尾端與期望



一個實現：F → F → F → S

G=4：包含最後成功trial

F=G−1=3：只數失敗

五次仍未成功：q^5=243/3125

此尾端沒有被前五根柱吃掉

橫軸是包含首次成功的trial index；E[G]=2.5，E[F]=1.5，兩種計數不可交換。

Memoryless與重試的模型邊界

對0<p<1、整數s,t≥0，$P(G > s+t | G > s) = q^{s+t}/q^s = q^t = P(G > t)$。已失敗s次不改未來的success率，因各trial獨立且同p；不是「已經失敗很久，下一次更容易成功」。

若retry成功率隨次數改、系統有同一持續故障、或抽樣無放回，都不符合相同獨立trials。此時尾端可能是$\prod_{i=1}^{t}(1-p_i)$（獨立但不同p），或需要conditional probabilities，不能硬寫q^t。

若每trial固定成本c，總成本cG期望c/p；若成本依成功／失敗不同，總成本是$c_{\text{fail}}(G-1)+c_{\text{success}}$，期望$c_{\text{fail}}q/p + c_{\text{success}}$。先定成本與trial的關係，再套線性期望。

Binomial的pmf、mean與variance

互相獨立同p的n次trial，$X = \sum_{i=1}^{n} I_i \in 0, \ldots, n$。固定一組k個成功位置，其機率$p^k q^{n-k}$；$\binom{n}{k}$個互斥位置集合給$b(k; n, p) = \binom{n}{k} p^k q^{n-k}$。總和$(p+q)^n = 1$。

線性期望給μ=np，不需獨立；variance要各對covariance0才可Σpq=npq。完整binomial pmf則用了mutual independence，pairwise只夠mean／variance，不保整個分布。n0退化X0；p0退化0、p1退化n，0⁰在pmf端點依空乘積視為1。

不同p_i但mutually independent時是Poisson-binomial，mean為$\sum_i p_i$、variance為$\sum_i p_i(1-p_i)$，不應把平均p塞進binomial pmf就說相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一張能逐項核對的binomial表

n=8,p=1/4,q=3/4，各pmf的共同分母4⁸=65536；分子$\binom{8}{k}3^{8-k}$如下。

k	0	1	2	3	4	5	6	7	8
分子	6561	17496	20412	13608	5670	1512	252	24	1

分子總65536，Σk・分子=131072給E[X]=2；Σk²・分子=360448給E[X²]=11/2，所以Var=11/2−4=3/2。

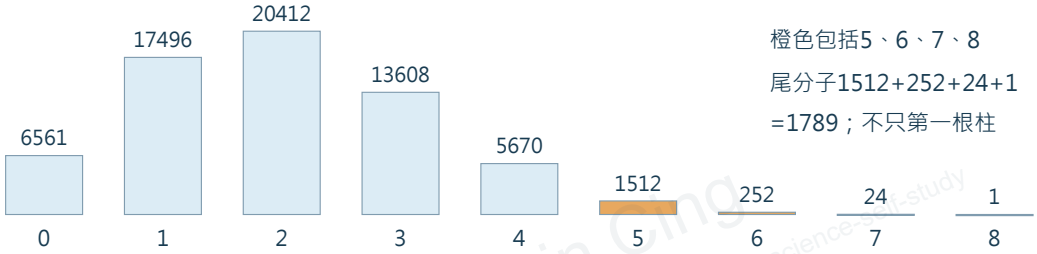
精確尾P(X≥5)=(1512+252+24+1)/65536=1789/65536≈0.027298。不是只算b(5)=1512/65536，也不是把「五次以上」漏掉k8。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一張能逐項核對的binomial表 （續）

單一機率柱與尾端總和不同：門檻5含5

每柱高度 = 分子 / 65536；n8、p1/4



橙色包括5、6、7、8

尾分子1512+252+24+1

=1789；不只第一根柱

Σ分子=65536；mean2、variance1.5；P(X≥5)=1789/65536≈0.027298。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mode由相鄰比率定，不能直接當mean

對0<p<1、1≤k≤n，有b(k)/b(k−1)=[(n−k+1)p]/(kq)。比率>1 iff k<(n+1)p；等於1 iff k=(n+1)p。

若(n+1)p不是整數，唯一mode=floor((n+1)p)；若是整數m，m−1與m都是modes。n8,p1/4時(n+1)p=2.25，mode2，mean2恰巧相同；n4,p1/2時m=2.5，mode2；n3,p1/2時m2，modes1,2，mean1.5沒有整數機率柱。

對0≤k≤n，前述entropy/binomial coefficient界給$b(k) \le (np/k)^k(nq/(n-k))^{n-k}$，端點以連續或0⁰慣例處理。這是單一pmf上界，不是整個tail上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.5 The tails of the binomial distribution

先對齊嚴格／非嚴格尾：整數X的P(X≥a)從ceil(a)加到n；P(X>a)從floor(a)+1起。P(X<k)（整數k）只加0..k−1，別和P(X≤k)混用。

對X~Bin(n,p)，事件X≥k意味存在一組k個指定trials全部success。每組機率p^k，共$\binom{n}{k}$組；用union bound（組與組不必互斥）得$P(X \geq k) \leq \binom{n}{k} p^k$。k0界1，k>n真事件空。左尾用failures Y=n−X~Bin(n,q)：$P(X \leq k) \leq \binom{n}{k} q^{n-k}$。

數例n8,p1/4,k5，右尾≤56/1024=7/128≈0.054688，真尾1789/65536≈0.027298。重複計入同一次出現6個以上success的outcomes，正是上界通常不相等的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左尾的geometric比率界：推導而非背公式

假設0<p<1、整數0<k<np，q=1−p。對1≤i≤k，b(i−1)/b(i)=iq/[(n−i+1)p]<kq/[(n−k)p]=ρ；因k<np，ρ<1。

由後往前，$b(k-j) < \rho^j b(k)$，j1..k。因此$P(X < k) < b(k) \sum_{j \geq 1} \rho^j = [\rho/(1-\rho)] b(k) = [kq/(np-k)] b(k)$。分母為正是前提，不可在k=np直接代入。

若0<k≤np/2，係數kq/(np−k)≤q≤1，故$L_k = P(X < k) < b(k)$，於是$L_k / L_{k+1} = L_k / (L_k + b(k)) < 1/2$。這說遠左尾再往左縮一個整數門檻時，機率至少減半。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

右尾由failures換元，門檻也要換

令Y=n−X~Bin(n,q)。當np<k<n，n−k<nq，將左尾界套到整數門檻n−k：

P(X>k)=P(Y<n−k)<[(n−k)p/(nq−(n−k))]b(n−k;n,q)=[(n−k)p/(k−np)]b(k;n,p)。

若k>(np+n)/2，則n−k<nq/2，左尾減半性給P(X>k)<(1/2)P(X>k−1)。success與failure的角色互換，不能只把原公式中的<改成>。

n8,p1/4,k4：係數(8−4)(1/4)/(4−2)=1/2，故P(X>4)<2835/65536≈0.043259；真尾1789/65536。它比本例union界7/128小，但不同參數時不必總是較緊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exponential moment把尾事件轉成非負量

令I_i為mutually independent Bernoulli trials（參數p_i），$X=\sum I_i$，$\mu=\sum p_i$。對任意λ>0及t，exp單調與Markov給$P(X\ge t)\le e^{-\lambda t}E[e^{\lambda X}]$。獨立性在下一步不可省：

$E[e^{\lambda X}]=\prod E[e^{\lambda I_i}]=\prod[1+p_i(e^{\lambda}-1)]\le \exp[\mu(e^{\lambda}-1)]$，因$1+z\le e^{z}$。故$P(X\ge t)\le \exp[\mu(e^{\lambda}-1)-\lambda t]$。

若μ>0、t>μ，對λ微分$\mu e^{\lambda}-t=0$，最小在λ=ln(t/μ)>0，二階$\mu e^{\lambda}>0$。代入得$P(X\ge t)\le e^{-\mu}(e\mu/t)^{t}$。此界允許各$p_i$不同；不需同分布，但需mutual independence。μ0時全部$I_i=0$幾乎必然，正尾直接0，不代ln(t/0)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

原書中心化r界與較精細的t界不能混用

CLRS所列較鬆形式：對r>μ>0，$P(X-\mu \geq r) \leq (e\mu/r)^r$。可直接證：$E[e^{\lambda(I_i-p_i)}] = p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda})$。相乘再用Markov，得$P(X-\mu \geq r) \leq \exp(\mu e^{\lambda} - \lambda r)$；取λ=ln(r/μ)即得。

本式r是**超過mean的幅度**，事件門檻是t=μ+r；前頁t則是**總success門檻**。若μ10,r40，中心化事件X≥50，較鬆bound為$(e/4)^{40} \approx 1.947 \times 10^{-7}$；精細t界則$e^{-10}(e/5)^{50} \approx 2.650 \times 10^{-18}$。兩者都是上界，不能把40與50交換後仍稱同一推導。

r只略大μ時$(e\mu/r)^r$可超1，仍合法卻無資訊，需取min(1,bound)。若是整數binomial，真sum起點為ceil(μ+r)；不把非整數門檻向下捨。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一binomial尾端比較不同保證

原創X~Bin(1000,0.01)，μ10，關心X≥50：Markov≤10/50=0.2；Chebyshev以Var9.9及偏差40給≤9.9/1600=0.0061875；中心化r40界約1.947e−7；精細t50界約2.650e−18。

精確值仍是$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}$，可用相鄰比率遞推或log-pmf避免直接算大factorial後浮點溢出。這些界用到的條件不同：Markov只需非負與mean；Chebyshev需variance；最後兩個用mutual independent indicators。

若所有I_i共用一個Bernoulli(.01)，X只可能0或1000，P(X≥50)=.01，仍符合Markov，但原variance與exponential-moment乘積均不適用。相同marginals不是使用binomial／Chernoff的充分理由。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix C。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D 矩陣

Part VIII Appendix

本章定位

以entry、尺寸及field前提重建矩陣運算，完整追蹤消去、rank因式分解與inverse；證明row/column rank、det運算律與Gram正定條件，明確區分實數、複數與有限field。

前置知識

向量空間、線性映射與矩陣乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用矩陣運算與轉置。
- 辨識 rank、determinant、inverse 的關係。
- 理解 SPD 與特徵值的必要條件。
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D.1 Matrices and matrix operations

本附錄除另註外取輸入維度為正整數、entries在R，vector預設column；rank0的factorization允許0個中間columns／rows，空加總定為0。$A \in \mathbb{R}^{m \times n}$有m個rows、n個columns，$a_{ij}$先row再column。矩陣相等須尺寸相同且每個entry相同；矩陣加法與scalar multiplication逐entry定義。

例$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}$是2×3，$A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$是3×2，$(A^T)_{ji} = a_{ij}$。轉置不是把同一串記憶體不加定義就改形狀；row-major／column-major是儲存慣例，數學transpose由索引對應定義。

向量$x = (1, -1, 2)^T$是3×1，x^T是1×3。e_i是第i筆1、其他0的standard basis vector，維度由使用處指定；0矩陣亦須尺寸相符。scalar0與2×3零矩陣不是相同型別。

常見square結構的精確零位置

類型	n×n矩陣的entry條件
diagonal	$i \neq j \Rightarrow a_{ij} = 0$
identity I_n	對角1、其他0
tridiagonal	$\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$
upper triangular	$i > j \Rightarrow a_{ij} = 0$
lower triangular	$i < j \Rightarrow a_{ij} = 0$
unit triangular	對應triangle且全部對角1
permutation	每個row及每個column恰一個1、其餘0
real symmetric	$A = A^T$

常見square結構的精確零位置 (續)

這些類別可重疊：identity同時diagonal、上下triangular、symmetric、permutation。tridiagonal不保symmetric，因上下對角線數值可不同。unit triangular不表示整張矩陣都是1；unit vector也不是全部entries1的向量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法先核維度，再核每個收縮索引

若$A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times p}$，才有$C = AB \in \mathbb{R}^{m\times p}$，$c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$。中間索引k被加總，外側i,j留下；不是逐entry相乘。

沿用A，取$B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$，則$AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$。四筆計算依序：1・2+2・(−1)+0・4=0；1・1+2・3+0・0=7；0・2+1・(−1)+1・4=3；0・1+1・3+1・0=3。

反向BA也有定義但為3×3：$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}$，與AB連尺寸都不同。即使兩者同為square也通常AB≠BA；交換律不能由scalar算術照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法先核維度，再核每個收縮索引 (續)

先看尺寸，再沿同一個k對齊的三項相加

A：2 rows × 3 columns

1	2	0
0	1	1

×

B：3 rows × 2 columns

2	1
-1	3
4	0

=

C=AB：2×2

0	7
3	3

C[1,2] = A的row1 · B的column2 = 1×1 + 2×3 + 0×0 = 7

黃色兩段各長3，收縮同一個k；輸出保留row1、column2，不能逐entry相乘。

矩陣乘法的可實作版本與成本

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

$n \geq 1$時共mnp次scalar multiplies、$mp(n-1)$次scalar additions，另有mp次輸出寫入及迴圈控制。單次scalar算術$O(1)$的RAM模型下為$\Theta(mnp)$；$n \times n$為$\Theta(n^3)$。若先s=0並做n次s+=，則多做mp次加法，是另一個精確計數但同階。

輸出C不得alias尚待讀的A/B；否則寫C可能破壞後續輸入。任意精度整數或field元素的bit成本須另計；「矩陣乘法做n^3次乘法」不是「任意bit長皆n^3時間」的保證。

結合、分配、轉置：每個等式都要尺寸相容

令A為m×n、B為n×p、C為p×q。第(i,j)筆的$(AB)C$為$\sum_{k=1}^{p}\sum_{\ell=1}^{n}a_{i\ell}b_{\ell k}c_{kj}$；有限sum換序得$\sum_{\ell=1}^{n}a_{i\ell}\sum_{k=1}^{p}b_{\ell k}c_{kj}=(A(BC))_{ij}$，故結合成立。分配律由scalar分配逐entry得；單位元須寫$I_mA=AI_n=A$。

$(AB)^T=B^TA^T$：左邊第(j,i)筆是$\sum_k a_{ik}b_{kj}$；右邊同一位置$\sum_k(B^T)_{jk}(A^T)_{ki}=\sum_k b_{kj}a_{ik}$，由scalar交換相等。矩陣乘法本身不交換，轉置後順序反轉不可漏。

若A,B都real symmetric，A+B與A−B仍對稱，但AB對稱 iff AB=BA，因$(AB)^T=BA$。不能把「兩個對稱矩陣」直接當成「其積也對稱」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ax、inner product與outer product的三個輸出型別

將A的columns記$a_1,\dots,a_n$，$Ax=\sum_j x_ja_j$是m-vector。沿用A與$x=(1,-1,2)^T$，$Ax=(1,0)^T-(2,1)^T+2(0,1)^T=(-1,1)^T$，與逐row點積相同。

同長real vectors u,v的u^Tv是scalar；uv^T是matrix。例$u=(1,2)^T$、$v=(3,-1)^T$，$u^Tv=1$，$uv^T=\begin{bmatrix}3&-1\\6&-2\end{bmatrix}$。outer product每個column為$v_ju$，故rank≤1；u,v都非零時至少一個column為非零倍u，rank=1。任一為0則rank0。

$\|x\|_2=\sqrt{x^Tx}$只對real x直接如此寫；complex需$x^*x=\sum_i|x_i|^2$，其中*是conjugate transpose，不是普通transpose。例x=[i]，$x^Tx=-1$，不是長度平方。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permutation左乘換rows，右乘換columns

取$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，則$Px = (x_3, x_1, x_2)^T$。若A有3個rows，PA把rows改為舊row3、row1、row2；因$(PA)_{ij} = \sum_k p_{ik} a_{kj}$，每row恰選一筆。

若M有3個columns，MP的columns則是舊column2、column3、column1，因P的三columns依序e_2, e_3, e_1。左乘與右乘的重新排列方向要從entry式或basis columns讀取，不能只記「都照3,1,2」。

不同columns of P是不同basis vectors，$P^T P = I = P P^T$，故$P^{-1} = P^T$。兩個permutation matrices相乘仍是permutation matrix，因它們的basis重排composition仍恰是一個bijection。

D.2 Basic matrix properties

vectors $v_1, \ldots, v_n$線性獨立指$\sum_j c_j v_j = 0$只可能所有$c_j = 0$；若存在非全0係數就是dependent。A的column rank是最大獨立column數，row rank類似，zero matrix rank0。

因$Ax = \sum_j x_j a_j$，columns獨立 iff Ax=0只有x=0；若有非零x使Ax=0，本文稱nonzero null vector。整個null space含0，不能把「沒有非零null vector」誤寫成「null space沒有任何向量」。

rank不是非零entries個數，也不是只數原始主對角非零數。例$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$有8個非零entries，但column3=column1+column2，且前兩column獨立，所以rank2；null vector為$(-1, -1, 1)^T$。

消去法的操作與完整中間狀態

三種elementary row operations都可逆：交換兩rows；一row乘非零scalar；一row加另一row的倍數。每種可表示左乘invertible elementary matrix，所以不改column之間的linear dependence；rows本身的span也不改。

對M：$R_2 \leftarrow R_2 - 2R_1$，得 $\begin{bmatrix}1&2&3\\0&0&0\\0&1&1\end{bmatrix}$；交換$R_2, R_3$，得 $\begin{bmatrix}1&2&3\\0&1&1\\0&0&0\end{bmatrix}$；$R_1 \leftarrow R_1 - 2R_2$，得$R = \begin{bmatrix}1&0&1\\0&1&1\\0&0&0\end{bmatrix}$。

R有pivots columns1,2，第三變數free，解$Rx=0$為$x_1=-x_3$、$x_2=-x_3$，所以null space為$\{t(-1,-1,1)^T : t \in \mathbb{R}\}$。非零row2個，彼此leading positions不同而獨立；非pivot column是pivot columns之和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

消去法的操作與完整中間狀態 （續）

M=BC：中間維度2，從消去結果追回原columns

原M：3×3、rank2

1	2	3
2	4	6
0	1	1

=

B：原pivot columns

1	2
2	4
0	1

×

C：各column的座標

1	0	1
0	1	1

M的column3 = column1 + column2，所以C的column3=$(1,1)^T$。

B保留原向量；消去後的unit columns只用於讀取座標，不能直接取代B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不預設主對角可用的Gauss–Jordan

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

每步pivot前，前r個pivot columns已是distinct unit columns；未用rows在它們上皆0。新pivot取剩餘row、normalization後消去其他rows，不破壞舊pivot columns，因新row在舊pivot位置0。若剩餘rows該column全0才跳過。迴圈有限，得到r個pivots與zero rows，field operations O(mn・min(m,n))。

精確算術中!=0依field判定；浮點則需pivoting與尺度／容許值，不能把固定|pivot|<10^{-12}當成對所有矩陣普遍正確的rank定義。

為什麼row rank等於column rank

設R=EA為row-reduced形式，E是invertible。E乘在每個column上是同一可逆線性映射，所以columns的相依係數在A與R相同；R恰r個pivot columns獨立，其他column是它們線性組合，故column rank(A)=r。

row operations每步以可逆組合改rows，因此row span(A)=row span(R)。R非零rows r個，依pivot位置可逐係數消去證獨立，故row rank(A)=r。於是可統稱rank(A)，介於0與min(m,n)。

nullity=n−r：pivot variables由n−r個free variables唯一線性決定；每次只令一個free variable1、其餘0得到一個null basis。它們獨立（看free coordinates），也生成所有null solutions，完成dimension計數，不只在M一例成立。

Rank factorization與乘積rank界

取A原本的r個pivot columns組成B（m×r），每個原column在B下的coordinates排成C（r×n），則A=BC。B columns獨立；C含r×r identity位置。反向若A=UV且U只有s個columns，A的columns全在span(U)，rank(A)≤s，所以r也是最小可因式分解的中間維度。

M的原pivot columns給$B=\begin{bmatrix}1&2\\2&4\\0&1\end{bmatrix}$，$C=\begin{bmatrix}1&0&1\\0&1&1\end{bmatrix}$，逐column乘回恰M。不可拿R的pivot columns當成原M的columns，因row operation改變了向量本身。

對相容A,D，若$A=B_rC_r$，則$AD=B_r(C_rD)$，rank(AD)≤rank(A)；若$D=U_sV_s$，$AD=(AU_s)V_s$，亦≤rank(D)。故rank(AD)≤min(rank A,rank D)。若左因子P可逆，配合A=P⁻¹(PA)反向界得到rank(PA)=rank(A)；右側可逆同理。

Inverse存在與唯一性的全部條件

n×n的inverse B要求AB=I=BA。若B,C皆inverse，B=B(AC)=(BA)C=C，故唯一。可逆A,B有(AB)⁻¹=B⁻¹A⁻¹；左右各乘檢查都回I。$(A^{-1})^T=(A^T)^{-1}$亦由轉置兩個inverse等式得。

square A full rank iff無非零null vector，等價linear map x↦Ax injective。n個獨立columns在n維空間成basis，故每e_j有唯一解$Ab_j=e_j$；把b_j排成B得AB=I。再由A(BA−I)=0及A injective，逐column得BA=I，故full rank iff invertible。反向若Ax=0，乘inverse即x=0。

rectangular時只一側inverse不保另一側：A=[1,0]、$B=(1,0)^T$有AB=[1]，但BA=diag(1,0)≠I₂。不能把square定理不改尺寸套用。

一個可逐步核對的inverse

取$H=\begin{bmatrix}2&1\\3&2\end{bmatrix}$，從augmented matrix[H|I]做同一組row operations：

$$\left[\begin{array}{cc|cc}2&1&1&0\\3&2&0&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1/2&-3/2&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1&-3&2\end{array}\right].$$

依序為$R_2-(3/2)R_1$、R_2乘2；再R_1-R_2，最後R_1除2，得到$[I\mid H^{-1}]$，$H^{-1}=\begin{bmatrix}2&-1\\-3&2\end{bmatrix}$。檢查$HH^{-1}=\begin{bmatrix}4-3&-2+2\\6-6&-3+4\end{bmatrix}=I$，反向乘也為I。

消去法對左半做E，右半同時變E；當EH=I，E就是inverse。若左半無法產生n個pivots，不能假裝最後右半就是inverse。

Determinant、minor與cofactor

對square A，minor matrix A[ij]刪row i及column j；cofactor為$C_{ij}=(-1)^{i+j}\det A[ij]$。n1時det[a]=a；若採0×0 convention，det(empty)=1以統一cofactor式。沿row i展開$\det A=\sum_{j=1}^{n}a_{ij}C_{ij}$。

原創$N=\begin{bmatrix}2&1&0\\1&3&2\\0&-1&1\end{bmatrix}$，第一row signs為+−+，所以detN=2・(3・1−2・(−1))

−1・(1・1−2・0)+0=10−1=9。minor是矩陣；其det是scalar；cofactor再加正負號，三者不混同。

一般permutation式為$\det A=\sum_{\sigma\in S_n}\operatorname{sgn}(\sigma)\prod_{i=1}^{n}a_{i,\sigma(i)}$。每項在每row／column各選一筆，sign由permutation parity決定；按固定row選到的column分組便得到cofactor展開。

Determinant運算律的原因

permutation式對每個column各自linear；兩columns相同時，把交換這兩column的permutations配對，項互消，故det0；交換兩columns使sign反號。對rows同理，且σ↦σ⁻¹使$\det(A^T) = \det(A)$。因此一row乘α，det乘α；一row加另一row的倍數，multilinearity加出的重複row項為0，det不變。

任一對columns交替multilinear的scalar函數F，把每column依basis e_i展開；重複basis index的項為0，留下各permutation，且$F(e_{\sigma(1)}, \ldots, e_{\sigma(n)}) = sgn(\sigma)F(I)$，所以F(B)=F(I)det(B)。

固定A，令F(B)=det(AB)；AB每column為A乘該column，故F仍交替multilinear，F(I)=det(A)。由唯一展開得det(AB)=det(A)det(B)。這證明不要求A預先可逆；若det A=0，F亦處處為0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Determinant與singularity，不只背2×2公式

upper triangular的非零permutation項須σ(i)≥i對每i成立；但Σσ(i)=Σi，所以只能全部σ(i)=i，故det是對角乘積。lower triangular同理，或轉置化成upper。row echelon若有zero row，det0；若全部n個pivots非零，triangular對角乘積非零。

消去過程的row swap、非零scaling只乘非零因子，row-add不改det，故原det為0 iff echelon缺pivot iff rank<n iff singular。於是det≠0是可逆判準，不代表數值上well-conditioned。

例diag(1,ε)，ε≠0時det ε且可逆；ε很小仍是精確full rank，inverse含1/ε，誤差可能放大。實際解Ax=b通常以穩定factorization求解，不必先顯式形成inverse；數學存在性與數值可靠性是不同問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Real symmetric正定與Gram的一般證明

本附錄正定採**real symmetric** S且對每個非零real x有$x^T Sx > 0$；semidefinite把>改≥。若不要求對稱，quadratic form只看$(S + S^T)/2$，不能由$x^T Sx > 0$自動推出$S = S^T$。

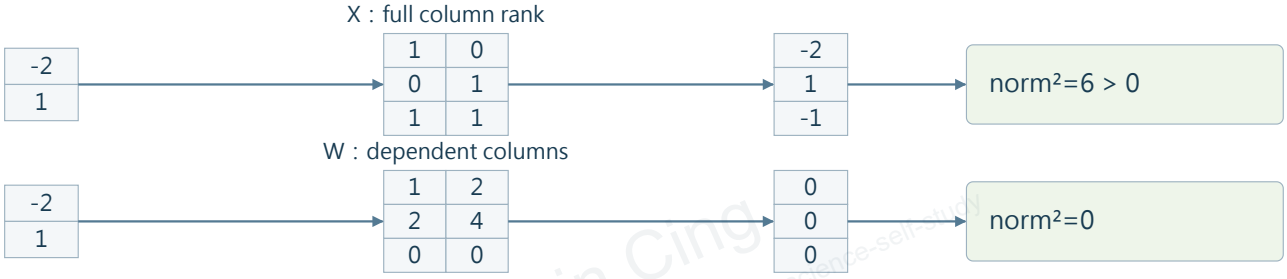
對任意real m×n A，$G = A^T A$為symmetric，且$x^T Gx = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0$。等於0 iff Ax=0。因此G正定 iff A無非零null vector iff A full column rank；若n>m就不可能。只說「A非零」不足。

原創$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$，$X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$；對$x = (s,t)^T$，$x^T Gx = s^2 + t^2 + (s+t)^2 > 0$除非s=t=0。對$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}$，$W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix}$，取$x = (-2,1)^T$使Wx=0，只有semidefinite。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Real symmetric正定與Gram的一般證明 (續)

比較$x^T A^T Ax = \text{norm}(Ax)^2$：同一輸入，兩種rank



同一非零$x=(-2,1)^T$。一般正定結論要證全部非零x；W只需一個null vector即可反證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複數與有限field：哪些定理仍可用

前述加乘、inverse、rank及det的field論證可改用C或prime field F_p；但R的正負及euclidean內積不能照搬。composite模數的ring不一定每非零元素可除，例如mod6的2非零卻無inverse，所以pivot normalization與field rank定理須重查。

complex版本以A^*為conjugate transpose，Hermitian要求$S^* = S$。Gram A^*A semidefinite，full column rank時positive definite，因$x^*A^*Ax = \sum_i |(Ax)_i|^2$。普通$A^TA$不保此結論：A=[i]時$A^TA = [-1]$，$A^*A = [1]$。

對real或complex square A，分別有$\det(A^TA) = (\det A)^2$，與$\det(A^*A) = |\det A|^2 \geq 0$；第一式只在real情況可推≥0。代數中的transpose恆等式和內積中的conjugation須依field清楚區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix D。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study